Sixth Edition

John L. Hennessy | David A. Patterson

COMPUTER ARCHITECTURE

A Quantitative Approach



MORGAN KAUFMANN

Computer Architecture Formulas

1. *CPU time* = Instruction count × Clock cycles per instruction × Clock cycle time
2. X is *n* times faster than Y: $n = \text{Execution time}_Y / \text{Execution time}_X = \text{Performance}_X / \text{Performance}_Y$
3. *Amdahl's Law:* $\text{Speedup}_{\text{overall}} = \dfrac{\text{Execution time}_{\text{old}}}{\text{Execution time}_{\text{new}}} = \dfrac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \dfrac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$
4. $\textit{Energy}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2$
5. $\textit{Power}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$
6. $\textit{Power}_{\text{static}} \propto \text{Current}_{\text{static}} \times \text{Voltage}$
7. *Availability* = Mean time to fail / (Mean time to fail + Mean time to repair)
8. *Die yield* = Wafer yield × $1 / (1 + \text{Defects per unit area} \times \text{Die area})^N$

   where Wafer yield accounts for wafers that are so bad they need not be tested and *N* is a parameter called the process-complexity factor, a measure of manufacturing difficulty. *N* ranges from 11.5 to 15.5 in 2011.
9. *Means—arithmetic (AM), weighted arithmetic (WAM), and geometric (GM):*

$$\text{AM} = \frac{1}{n} \sum_{i=1}^{n} \text{Time}_i \quad \text{WAM} = \sum_{i=1}^{n} \text{Weight}_i \times \text{Time}_i \quad \text{GM} = \sqrt[n]{\prod_{i=1}^{n} \text{Time}_i}$$

   where Time_i is the execution time for the *i*th program of a total of *n* in the workload, Weight_i is the weighting of the *i*th program in the workload.
10. *Average memory-access time* = Hit time + Miss rate × Miss penalty
11. *Misses per instruction* = Miss rate × Memory access per instruction
12. *Cache index size:* 2^{index} = Cache size / (Block size × Set associativity)
13. *Power Utilization Effectiveness* (*PUE*) of a Warehouse Scale Computer = $\dfrac{\text{Total Facility Power}}{\text{IT Equipment Power}}$

Rules of Thumb

1. *Amdahl/Case Rule:* A balanced computer system needs about 1 MB of main memory capacity and 1 megabit per second of I/O bandwidth per MIPS of CPU performance.
2. *90/10 Locality Rule:* A program executes about 90% of its instructions in 10% of its code.
3. *Bandwidth Rule:* Bandwidth grows by at least the square of the improvement in latency.
4. *2:1 Cache Rule:* The miss rate of a direct-mapped cache of size *N* is about the same as a two-way set-associative cache of size *N*/2.
5. *Dependability Rule:* Design with no single point of failure.
6. *Watt-Year Rule*: The fully burdened cost of a Watt per year in a Warehouse Scale Computer in North America in 2011, including the cost of amortizing the power and cooling infrastructure, is about $2.

In Praise of *Computer Architecture: A Quantitative Approach* Sixth Edition

"Although important concepts of architecture are timeless, this edition has been thoroughly updated with the latest technology developments, costs, examples, and references. Keeping pace with recent developments in open-sourced architecture, the instruction set architecture used in the book has been updated to use the RISC-V ISA."

—from the foreword by Norman P. Jouppi, Google

"*Computer Architecture: A Quantitative Approach* is a classic that, like fine wine, just keeps getting better. I bought my first copy as I finished up my undergraduate degree and it remains one of my most frequently referenced texts today."

—James Hamilton, Amazon Web Service

"Hennessy and Patterson wrote the first edition of this book when graduate students built computers with 50,000 transistors. Today, warehouse-size computers contain that many servers, each consisting of dozens of independent processors and billions of transistors. The evolution of computer architecture has been rapid and relentless, but *Computer Architecture: A Quantitative Approach* has kept pace, with each edition accurately explaining and analyzing the important emerging ideas that make this field so exciting."

—James Larus, Microsoft Research

"Another timely and relevant update to a classic, once again also serving as a window into the relentless and exciting evolution of computer architecture! The new discussions in this edition on the slowing of Moore's law and implications for future systems are must-reads for both computer architects and practitioners working on broader systems."

—Parthasarathy (Partha) Ranganathan, Google

"I love the 'Quantitative Approach' books because they are written by engineers, for engineers. John Hennessy and Dave Patterson show the limits imposed by mathematics and the possibilities enabled by materials science. Then they teach through real-world examples how architects analyze, measure, and compromise to build working systems. This sixth edition comes at a critical time: Moore's Law is fading just as deep learning demands unprecedented compute cycles. The new chapter on domain-specific architectures documents a number of promising approaches and prophesies a rebirth in computer architecture. Like the scholars of the European Renaissance, computer architects must understand our own history, and then combine the lessons of that history with new techniques to remake the world."

—Cliff Young, Google

This page intentionally left blank

Computer Architecture
A Quantitative Approach

Sixth Edition

John L. Hennessy is a Professor of Electrical Engineering and Computer Science at Stanford University, where he has been a member of the faculty since 1977 and was, from 2000 to 2016, its 10th President. He currently serves as the Director of the Knight-Hennessy Fellowship, which provides graduate fellowships to potential future leaders. Hennessy is a Fellow of the IEEE and ACM, a member of the National Academy of Engineering, the National Academy of Science, and the American Philosophical Society, and a Fellow of the American Academy of Arts and Sciences. Among his many awards are the 2001 Eckert-Mauchly Award for his contributions to RISC technology, the 2001 Seymour Cray Computer Engineering Award, and the 2000 John von Neumann Award, which he shared with David Patterson. He has also received 10 honorary doctorates.

In 1981, he started the MIPS project at Stanford with a handful of graduate students. After completing the project in 1984, he took a leave from the university to cofound MIPS Computer Systems, which developed one of the first commercial RISC microprocessors. As of 2017, over 5 billion MIPS microprocessors have been shipped in devices ranging from video games and palmtop computers to laser printers and network switches. Hennessy subsequently led the DASH (Director Architecture for Shared Memory) project, which prototyped the first scalable cache coherent multiprocessor; many of the key ideas have been adopted in modern multiprocessors. In addition to his technical activities and university responsibilities, he has continued to work with numerous start-ups, both as an early-stage advisor and an investor.

David A. Patterson became a Distinguished Engineer at Google in 2016 after 40 years as a UC Berkeley professor. He joined UC Berkeley immediately after graduating from UCLA. He still spends a day a week in Berkeley as an Emeritus Professor of Computer Science. His teaching has been honored by the Distinguished Teaching Award from the University of California, the Karlstrom Award from ACM, and the Mulligan Education Medal and Undergraduate Teaching Award from IEEE. Patterson received the IEEE Technical Achievement Award and the ACM Eckert-Mauchly Award for contributions to RISC, and he shared the IEEE Johnson Information Storage Award for contributions to RAID. He also shared the IEEE John von Neumann Medal and the C & C Prize with John Hennessy. Like his co-author, Patterson is a Fellow of the American Academy of Arts and Sciences, the Computer History Museum, ACM, and IEEE, and he was elected to the National Academy of Engineering, the National Academy of Sciences, and the Silicon Valley Engineering Hall of Fame. He served on the Information Technology Advisory Committee to the President of the United States, as chair of the CS division in the Berkeley EECS department, as chair of the Computing Research Association, and as President of ACM. This record led to Distinguished Service Awards from ACM, CRA, and SIGARCH. He is currently Vice-Chair of the Board of Directors of the RISC-V Foundation.

At Berkeley, Patterson led the design and implementation of RISC I, likely the first VLSI reduced instruction set computer, and the foundation of the commercial SPARC architecture. He was a leader of the Redundant Arrays of Inexpensive Disks (RAID) project, which led to dependable storage systems from many companies. He was also involved in the Network of Workstations (NOW) project, which led to cluster technology used by Internet companies and later to cloud computing. His current interests are in designing domain-specific architectures for machine learning, spreading the word on the open RISC-V instruction set architecture, and in helping the UC Berkeley RISELab (Real-time Intelligent Secure Execution).

Computer Architecture
A Quantitative Approach

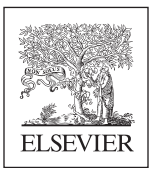
Sixth Edition

John L. Hennessy
Stanford University

David A. Patterson
University of California, Berkeley

With Contributions by

Krste Asanović
University of California, Berkeley
Jason D. Bakos
University of South Carolina
Robert P. Colwell
R&E Colwell & Assoc. Inc.
Abhishek Bhattacharjee
Rutgers University
Thomas M. Conte
Georgia Tech
José Duato
Proemisa
Diana Franklin
University of Chicago
David Goldberg
eBay
Norman P. Jouppi
Google
Sheng Li
Intel Labs
Naveen Muralimanohar
HP Labs
Gregory D. Peterson
University of Tennessee
Timothy M. Pinkston
University of Southern California
Parthasarathy Ranganathan
Google
David A. Wood
University of Wisconsin–Madison
Cliff Young
Google
Amr Zaky
University of Santa Clara



MORGAN KAUFMANN PUBLISHERS

AN IMPRINT OF ELSEVIER

Morgan Kaufmann is an imprint of Elsevier
50 Hampshire Street, 5th Floor, Cambridge, MA 02139, United States

© 2019 Elsevier Inc. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or any information storage and retrieval system, without permission in writing from the publisher. Details on how to seek permission, further information about the Publisher's permissions policies and our arrangements with organizations such as the Copyright Clearance Center and the Copyright Licensing Agency, can be found at our website: www.elsevier.com/permissions.

This book and the individual contributions contained in it are protected under copyright by the Publisher (other than as may be noted herein).

Notices
Knowledge and best practice in this field are constantly changing. As new research and experience broaden our understanding, changes in research methods, professional practices, or medical treatment may become necessary.

Practitioners and researchers must always rely on their own experience and knowledge in evaluating and using any information, methods, compounds, or experiments described herein. In using such information or methods they should be mindful of their own safety and the safety of others, including parties for whom they have a professional responsibility.

To the fullest extent of the law, neither the Publisher nor the authors, contributors, or editors, assume any liability for any injury and/or damage to persons or property as a matter of products liability, negligence or otherwise, or from any use or operation of any methods, products, instructions, or ideas contained in the material herein.

Library of Congress Cataloging-in-Publication Data
A catalog record for this book is available from the Library of Congress

British Library Cataloguing-in-Publication Data
A catalogue record for this book is available from the British Library

ISBN: 978-0-12-811905-1

For information on all Morgan Kaufmann publications
visit our website at https://www.elsevier.com/books-and-journals



Publisher: Katey Birtcher
Acquisition Editor: Stephen Merken
Developmental Editor: Nate McFadden
Production Project Manager: Stalin Viswanathan
Cover Designer: Christian J. Bilbow

Typeset by SPi Global, India

To Andrea, Linda, and our four sons

This page intentionally left blank

Foreword

by Norman P. Jouppi, Google

Much of the improvement in computer performance over the last 40 years has been provided by computer architecture advancements that have leveraged Moore's Law and Dennard scaling to build larger and more parallel systems. Moore's Law is the observation that the maximum number of transistors in an integrated circuit doubles approximately every two years. Dennard scaling refers to the reduction of MOS supply voltage in concert with the scaling of feature sizes, so that as transistors get smaller, their power density stays roughly constant. With the end of Dennard scaling a decade ago, and the recent slowdown of Moore's Law due to a combination of physical limitations and economic factors, the sixth edition of the preeminent textbook for our field couldn't be more timely. Here are some reasons.

First, because domain-specific architectures can provide equivalent performance and power benefits of three or more historical generations of Moore's Law and Dennard scaling, they now can provide better implementations than may ever be possible with future scaling of general-purpose architectures. And with the diverse application space of computers today, there are many potential areas for architectural innovation with domain-specific architectures. Second, high-quality implementations of open-source architectures now have a much longer lifetime due to the slowdown in Moore's Law. This gives them more opportunities for continued optimization and refinement, and hence makes them more attractive. Third, with the slowing of Moore's Law, different technology components have been scaling heterogeneously. Furthermore, new technologies such as 2.5D stacking, new nonvolatile memories, and optical interconnects have been developed to provide more than Moore's Law can supply alone. To use these new technologies and nonhomogeneous scaling effectively, fundamental design decisions need to be reexamined from first principles. Hence it is important for students, professors, and practitioners in the industry to be skilled in a wide range of both old and new architectural techniques. All told, I believe this is the most exciting time in computer architecture since the industrial exploitation of instruction-level parallelism in microprocessors 25 years ago.

The largest change in this edition is the addition of a new chapter on domain-specific architectures. It's long been known that customized domain-specific architectures can have higher performance, lower power, and require less silicon area than general-purpose processor implementations. However when general-purpose

processors were increasing in single-threaded performance by 40% per year (see Fig. 1.11), the extra time to market required to develop a custom architecture vs. using a leading-edge standard microprocessor could cause the custom architecture to lose much of its advantage. In contrast, today single-core performance is improving very slowly, meaning that the benefits of custom architectures will not be made obsolete by general-purpose processors for a very long time, if ever. Chapter 7 covers several domain-specific architectures. Deep neural networks have very high computation requirements but lower data precision requirements – this combination can benefit significantly from custom architectures. Two example architectures and implementations for deep neural networks are presented: one optimized for inference and a second optimized for training. Image processing is another example domain; it also has high computation demands and benefits from lower-precision data types. Furthermore, since it is often found in mobile devices, the power savings from custom architectures are also very valuable. Finally, by nature of their reprogrammability, FPGA-based accelerators can be used to implement a variety of different domain-specific architectures on a single device. They also can benefit more irregular applications that are frequently updated, like accelerating internet search.

Although important concepts of architecture are timeless, this edition has been thoroughly updated with the latest technology developments, costs, examples, and references. Keeping pace with recent developments in open-sourced architecture, the instruction set architecture used in the book has been updated to use the RISC-V ISA.

On a personal note, after enjoying the privilege of working with John as a graduate student, I am now enjoying the privilege of working with Dave at Google. What an amazing duo!

Contents

Foreword ix

Preface xvii

Acknowledgments xxv

Chapter 1 **Fundamentals of Quantitative Design and Analysis**

1.1 Introduction 2
1.2 Classes of Computers 6
1.3 Defining Computer Architecture 11
1.4 Trends in Technology 18
1.5 Trends in Power and Energy in Integrated Circuits 23
1.6 Trends in Cost 29
1.7 Dependability 36
1.8 Measuring, Reporting, and Summarizing Performance 39
1.9 Quantitative Principles of Computer Design 48
1.10 Putting It All Together: Performance, Price, and Power 55
1.11 Fallacies and Pitfalls 58
1.12 Concluding Remarks 64
1.13 Historical Perspectives and References 67
Case Studies and Exercises by Diana Franklin 67

Chapter 2 **Memory Hierarchy Design**

2.1 Introduction 78
2.2 Memory Technology and Optimizations 84
2.3 Ten Advanced Optimizations of Cache Performance 94
2.4 Virtual Memory and Virtual Machines 118
2.5 Cross-Cutting Issues: The Design of Memory Hierarchies 126
2.6 Putting It All Together: Memory Hierarchies in the ARM Cortex-A53 and Intel Core i7 6700 129
2.7 Fallacies and Pitfalls 142
2.8 Concluding Remarks: Looking Ahead 146
2.9 Historical Perspectives and References 148

Case Studies and Exercises by Norman P. Jouppi, Rajeev Balasubramonian, Naveen Muralimanohar, and Sheng Li 148

Chapter 3 **Instruction-Level Parallelism and Its Exploitation**

3.1 Instruction-Level Parallelism: Concepts and Challenges 168
3.2 Basic Compiler Techniques for Exposing ILP 176
3.3 Reducing Branch Costs With Advanced Branch Prediction 182
3.4 Overcoming Data Hazards With Dynamic Scheduling 191
3.5 Dynamic Scheduling: Examples and the Algorithm 201
3.6 Hardware-Based Speculation 208
3.7 Exploiting ILP Using Multiple Issue and Static Scheduling 218
3.8 Exploiting ILP Using Dynamic Scheduling, Multiple Issue, and Speculation 222
3.9 Advanced Techniques for Instruction Delivery and Speculation 228
3.10 Cross-Cutting Issues 240
3.11 Multithreading: Exploiting Thread-Level Parallelism to Improve Uniprocessor Throughput 242
3.12 Putting It All Together: The Intel Core i7 6700 and ARM Cortex-A53 247
3.13 Fallacies and Pitfalls 258
3.14 Concluding Remarks: What's Ahead? 264
3.15 Historical Perspective and References 266
Case Studies and Exercises by Jason D. Bakos and Robert P. Colwell 266

Chapter 4 **Data-Level Parallelism in Vector, SIMD, and GPU Architectures**

4.1 Introduction 282
4.2 Vector Architecture 283
4.3 SIMD Instruction Set Extensions for Multimedia 304
4.4 Graphics Processing Units 310
4.5 Detecting and Enhancing Loop-Level Parallelism 336
4.6 Cross-Cutting Issues 345
4.7 Putting It All Together: Embedded Versus Server GPUs and Tesla Versus Core i7 346
4.8 Fallacies and Pitfalls 353
4.9 Concluding Remarks 357
4.10 Historical Perspective and References 357
Case Study and Exercises by Jason D. Bakos 357

Chapter 5 **Thread-Level Parallelism**

5.1 Introduction 368
5.2 Centralized Shared-Memory Architectures 377
5.3 Performance of Symmetric Shared-Memory Multiprocessors 393

5.4 Distributed Shared-Memory and Directory-Based Coherence 404
5.5 Synchronization: The Basics 412
5.6 Models of Memory Consistency: An Introduction 417
5.7 Cross-Cutting Issues 422
5.8 Putting It All Together: Multicore Processors and Their Performance 426
5.9 Fallacies and Pitfalls 438
5.10 The Future of Multicore Scaling 442
5.11 Concluding Remarks 444
5.12 Historical Perspectives and References 445
Case Studies and Exercises by Amr Zaky and David A. Wood 446

Chapter 6 **Warehouse-Scale Computers to Exploit Request-Level and Data-Level Parallelism**

6.1 Introduction 466
6.2 Programming Models and Workloads for Warehouse-Scale Computers 471
6.3 Computer Architecture of Warehouse-Scale Computers 477
6.4 The Efficiency and Cost of Warehouse-Scale Computers 482
6.5 Cloud Computing: The Return of Utility Computing 490
6.6 Cross-Cutting Issues 501
6.7 Putting It All Together: A Google Warehouse-Scale Computer 503
6.8 Fallacies and Pitfalls 514
6.9 Concluding Remarks 518
6.10 Historical Perspectives and References 519
Case Studies and Exercises by Parthasarathy Ranganathan 519

Chapter 7 **Domain-Specific Architectures**

7.1 Introduction 540
7.2 Guidelines for DSAs 543
7.3 Example Domain: Deep Neural Networks 544
7.4 Google's Tensor Processing Unit, an Inference Data Center Accelerator 557
7.5 Microsoft Catapult, a Flexible Data Center Accelerator 567
7.6 Intel Crest, a Data Center Accelerator for Training 579
7.7 Pixel Visual Core, a Personal Mobile Device Image Processing Unit 579
7.8 Cross-Cutting Issues 592
7.9 Putting It All Together: CPUs Versus GPUs Versus DNN Accelerators 595
7.10 Fallacies and Pitfalls 602
7.11 Concluding Remarks 604
7.12 Historical Perspectives and References 606
Case Studies and Exercises by Cliff Young 606

Appendix A **Instruction Set Principles**

A.1	Introduction	A-2
A.2	Classifying Instruction Set Architectures	A-3
A.3	Memory Addressing	A-7
A.4	Type and Size of Operands	A-13
A.5	Operations in the Instruction Set	A-15
A.6	Instructions for Control Flow	A-16
A.7	Encoding an Instruction Set	A-21
A.8	Cross-Cutting Issues: The Role of Compilers	A-24
A.9	Putting It All Together: The RISC-V Architecture	A-33
A.10	Fallacies and Pitfalls	A-42
A.11	Concluding Remarks	A-46
A.12	Historical Perspective and References	A-47
	Exercises by Gregory D. Peterson	A-47

Appendix B **Review of Memory Hierarchy**

B.1	Introduction	B-2
B.2	Cache Performance	B-15
B.3	Six Basic Cache Optimizations	B-22
B.4	Virtual Memory	B-40
B.5	Protection and Examples of Virtual Memory	B-49
B.6	Fallacies and Pitfalls	B-57
B.7	Concluding Remarks	B-59
B.8	Historical Perspective and References	B-59
	Exercises by Amr Zaky	B-60

Appendix C **Pipelining: Basic and Intermediate Concepts**

C.1	Introduction	C-2
C.2	The Major Hurdle of Pipelining—Pipeline Hazards	C-10
C.3	How Is Pipelining Implemented?	C-26
C.4	What Makes Pipelining Hard to Implement?	C-37
C.5	Extending the RISC V Integer Pipeline to Handle Multicycle Operations	C-45
C.6	Putting It All Together: The MIPS R4000 Pipeline	C-55
C.7	Cross-Cutting Issues	C-65
C.8	Fallacies and Pitfalls	C-70
C.9	Concluding Remarks	C-71
C.10	Historical Perspective and References	C-71
	Updated Exercises by Diana Franklin	C-71

Online Appendices

Appendix D **Storage Systems**

Appendix E **Embedded Systems**
by Thomas M. Conte

Appendix F **Interconnection Networks**
by Timothy M. Pinkston and José Duato

Appendix G **Vector Processors in More Depth**
by Krste Asanovic

Appendix H **Hardware and Software for VLIW and EPIC**

Appendix I **Large-Scale Multiprocessors and Scientific Applications**

Appendix J **Computer Arithmetic**
by David Goldberg

Appendix K **Survey of Instruction Set Architectures**

Appendix L **Advanced Concepts on Address Translation**
by Abhishek Bhattacharjee

Appendix M **Historical Perspectives and References**

References R-1

Index I-1

This page intentionally left blank

Preface

Why We Wrote This Book

Through six editions of this book, our goal has been to describe the basic principles underlying what will be tomorrow's technological developments. Our excitement about the opportunities in computer architecture has not abated, and we echo what we said about the field in the first edition: "It is not a dreary science of paper machines that will never work. No! It's a discipline of keen intellectual interest, requiring the balance of marketplace forces to cost-performance-power, leading to glorious failures and some notable successes."

Our primary objective in writing our first book was to change the way people learn and think about computer architecture. We feel this goal is still valid and important. The field is changing daily and must be studied with real examples and measurements on real computers, rather than simply as a collection of definitions and designs that will never need to be realized. We offer an enthusiastic welcome to anyone who came along with us in the past, as well as to those who are joining us now. Either way, we can promise the same quantitative approach to, and analysis of, real systems.

As with earlier versions, we have strived to produce a new edition that will continue to be as relevant for professional engineers and architects as it is for those involved in advanced computer architecture and design courses. Like the first edition, this edition has a sharp focus on new platforms—personal mobile devices and warehouse-scale computers—and new architectures—specifically, domain-specific architectures. As much as its predecessors, this edition aims to demystify computer architecture through an emphasis on cost-performance-energy trade-offs and good engineering design. We believe that the field has continued to mature and move toward the rigorous quantitative foundation of long-established scientific and engineering disciplines.

This Edition

The ending of Moore's Law and Dennard scaling is having as profound effect on computer architecture as did the switch to multicore. We retain the focus on the extremes in size of computing, with personal mobile devices (PMDs) such as cell phones and tablets as the clients and warehouse-scale computers offering cloud computing as the server. We also maintain the other theme of parallelism in all its forms: *data-level parallelism (DLP)* in Chapters 1 and 4, *instruction-level parallelism (ILP)* in Chapter 3, *thread-level parallelism* in Chapter 5, and *request-level parallelism* (RLP) in Chapter 6.

The most pervasive change in this edition is switching from MIPS to the RISC-V instruction set. We suspect this modern, modular, open instruction set may become a significant force in the information technology industry. It may become as important in computer architecture as Linux is for operating systems.

The newcomer in this edition is Chapter 7, which introduces domain-specific architectures with several concrete examples from industry.

As before, the first three appendices in the book give basics on the RISC-V instruction set, memory hierarchy, and pipelining for readers who have not read a book like *Computer Organization and Design*. To keep costs down but still supply supplemental material that is of interest to some readers, available online at https://www.elsevier.com/books-and-journals/book-companion/9780128119051 are nine more appendices. There are more pages in these appendices than there are in this book!

This edition continues the tradition of using real-world examples to demonstrate the ideas, and the "Putting It All Together" sections are brand new. The "Putting It All Together" sections of this edition include the pipeline organizations and memory hierarchies of the ARM Cortex A8 processor, the Intel core i7 processor, the NVIDIA GTX-280 and GTX-480 GPUs, and one of the Google warehouse-scale computers.

Topic Selection and Organization

As before, we have taken a conservative approach to topic selection, for there are many more interesting ideas in the field than can reasonably be covered in a treatment of basic principles. We have steered away from a comprehensive survey of every architecture a reader might encounter. Instead, our presentation focuses on core concepts likely to be found in any new machine. The key criterion remains that of selecting ideas that have been examined and utilized successfully enough to permit their discussion in quantitative terms.

Our intent has always been to focus on material that is not available in equivalent form from other sources, so we continue to emphasize advanced content wherever possible. Indeed, there are several systems here whose descriptions cannot be found in the literature. (Readers interested strictly in a more basic introduction to computer architecture should read *Computer Organization and Design: The Hardware/Software Interface*.)

An Overview of the Content

Chapter 1 includes formulas for energy, static power, dynamic power, integrated circuit costs, reliability, and availability. (These formulas are also found on the front inside cover.) Our hope is that these topics can be used through the rest of the book. In addition to the classic quantitative principles of computer design and performance measurement, it shows the slowing of performance improvement of general-purpose microprocessors, which is one inspiration for domain-specific architectures.

Our view is that the instruction set architecture is playing less of a role today than in 1990, so we moved this material to Appendix A. It now uses the RISC-V architecture. (For quick review, a summary of the RISC-V ISA can be found on the back inside cover.) For fans of ISAs, Appendix K was revised for this edition and covers 8 RISC architectures (5 for desktop and server use and 3 for embedded use), the 80×86, the DEC VAX, and the IBM 360/370.

We then move onto memory hierarchy in Chapter 2, since it is easy to apply the cost-performance-energy principles to this material, and memory is a critical resource for the rest of the chapters. As in the past edition, Appendix B contains an introductory review of cache principles, which is available in case you need it. Chapter 2 discusses 10 advanced optimizations of caches. The chapter includes virtual machines, which offer advantages in protection, software management, and hardware management, and play an important role in cloud computing. In addition to covering SRAM and DRAM technologies, the chapter includes new material both on Flash memory and on the use of stacked die packaging for extending the memory hierarchy. The PIAT examples are the ARM Cortex A8, which is used in PMDs, and the Intel Core i7, which is used in servers.

Chapter 3 covers the exploitation of instruction-level parallelism in high-performance processors, including superscalar execution, branch prediction (including the new tagged hybrid predictors), speculation, dynamic scheduling, and simultaneous multithreading. As mentioned earlier, Appendix C is a review of pipelining in case you need it. Chapter 3 also surveys the limits of ILP. Like Chapter 2, the PIAT examples are again the ARM Cortex A8 and the Intel Core i7. While the third edition contained a great deal on Itanium and VLIW, this material is now in Appendix H, indicating our view that this architecture did not live up to the earlier claims.

The increasing importance of multimedia applications such as games and video processing has also increased the importance of architectures that can exploit data level parallelism. In particular, there is a rising interest in computing using graphical processing units (GPUs), yet few architects understand how GPUs really work. We decided to write a new chapter in large part to unveil this new style of computer architecture. Chapter 4 starts with an introduction to vector architectures, which acts as a foundation on which to build explanations of multimedia SIMD instruction set extensions and GPUs. (Appendix G goes into even more depth on vector architectures.) This chapter introduces the Roofline performance model and then uses it to compare the Intel Core i7 and the NVIDIA GTX 280 and GTX 480 GPUs. The chapter also describes the Tegra 2 GPU for PMDs.

Chapter 5 describes multicore processors. It explores symmetric and distributed-memory architectures, examining both organizational principles and performance. The primary additions to this chapter include more comparison of multicore organizations, including the organization of multicore-multilevel caches, multicore coherence schemes, and on-chip multicore interconnect. Topics in synchronization and memory consistency models are next. The example is the Intel Core i7. Readers interested in more depth on interconnection networks should read Appendix F, and those interested in larger scale multiprocessors and scientific applications should read Appendix I.

Chapter 6 describes warehouse-scale computers (WSCs). It was extensively revised based on help from engineers at Google and Amazon Web Services. This chapter integrates details on design, cost, and performance of WSCs that few architects are aware of. It starts with the popular MapReduce programming model before describing the architecture and physical implementation of WSCs, including cost. The costs allow us to explain the emergence of cloud computing, whereby it can be cheaper to compute using WSCs in the cloud than in your local datacenter. The PIAT example is a description of a Google WSC that includes information published for the first time in this book.

The new Chapter 7 motivates the need for Domain-Specific Architectures (DSAs). It draws guiding principles for DSAs based on the four examples of DSAs. Each DSA corresponds to chips that have been deployed in commercial settings. We also explain why we expect a renaissance in computer architecture via DSAs given that single-thread performance of general-purpose microprocessors has stalled.

This brings us to Appendices A through M. Appendix A covers principles of ISAs, including RISC-V, and Appendix K describes 64-bit versions of RISC V, ARM, MIPS, Power, and SPARC and their multimedia extensions. It also includes some classic architectures (80x86, VAX, and IBM 360/370) and popular embedded instruction sets (Thumb-2, microMIPS, and RISC V C). Appendix H is related, in that it covers architectures and compilers for VLIW ISAs.

As mentioned earlier, Appendix B and Appendix C are tutorials on basic caching and pipelining concepts. Readers relatively new to caching should read Appendix B before Chapter 2, and those new to pipelining should read Appendix C before Chapter 3.

Appendix D, "Storage Systems," has an expanded discussion of reliability and availability, a tutorial on RAID with a description of RAID 6 schemes, and rarely found failure statistics of real systems. It continues to provide an introduction to queuing theory and I/O performance benchmarks. We evaluate the cost, performance, and reliability of a real cluster: the Internet Archive. The "Putting It All Together" example is the NetApp FAS6000 filer.

Appendix E, by Thomas M. Conte, consolidates the embedded material in one place.

Appendix F, on interconnection networks, is revised by Timothy M. Pinkston and José Duato. Appendix G, written originally by Krste Asanović, includes a description of vector processors. We think these two appendices are some of the best material we know of on each topic.

Appendix H describes VLIW and EPIC, the architecture of Itanium.

Appendix I describes parallel processing applications and coherence protocols for larger-scale, shared-memory multiprocessing. Appendix J, by David Goldberg, describes computer arithmetic.

Appendix L, by Abhishek Bhattacharjee, is new and discusses advanced techniques for memory management, focusing on support for virtual machines and design of address translation for very large address spaces. With the growth in clouds processors, these architectural enhancements are becoming more important.

Appendix M collects the "Historical Perspective and References" from each chapter into a single appendix. It attempts to give proper credit for the ideas in each chapter and a sense of the history surrounding the inventions. We like to think of this as presenting the human drama of computer design. It also supplies references that the student of architecture may want to pursue. If you have time, we recommend reading some of the classic papers in the field that are mentioned in these sections. It is both enjoyable and educational to hear the ideas directly from the creators. "Historical Perspective" was one of the most popular sections of prior editions.

Navigating the Text

There is no single best order in which to approach these chapters and appendices, except that all readers should start with Chapter 1. If you don't want to read everything, here are some suggested sequences:

- *Memory Hierarchy:* Appendix B, Chapter 2, and Appendices D and M.
- *Instruction-Level Parallelism:* Appendix C, Chapter 3, and Appendix H
- *Data-Level Parallelism:* Chapters 4, 6, and 7, Appendix G
- *Thread-Level Parallelism:* Chapter 5, Appendices F and I
- *Request-Level Parallelism:* Chapter 6
- *ISA:* Appendices A and K

Appendix E can be read at any time, but it might work best if read after the ISA and cache sequences. Appendix J can be read whenever arithmetic moves you. You should read the corresponding portion of Appendix M after you complete each chapter.

Chapter Structure

The material we have selected has been stretched upon a consistent framework that is followed in each chapter. We start by explaining the ideas of a chapter. These ideas are followed by a "Crosscutting Issues" section, a feature that shows how the ideas covered in one chapter interact with those given in other chapters. This is

followed by a "Putting It All Together" section that ties these ideas together by showing how they are used in a real machine.

Next in the sequence is "Fallacies and Pitfalls," which lets readers learn from the mistakes of others. We show examples of common misunderstandings and architectural traps that are difficult to avoid even when you know they are lying in wait for you. The "Fallacies and Pitfalls" sections is one of the most popular sections of the book. Each chapter ends with a "Concluding Remarks" section.

Case Studies With Exercises

Each chapter ends with case studies and accompanying exercises. Authored by experts in industry and academia, the case studies explore key chapter concepts and verify understanding through increasingly challenging exercises. Instructors should find the case studies sufficiently detailed and robust to allow them to create their own additional exercises.

Brackets for each exercise (<chapter.section>) indicate the text sections of primary relevance to completing the exercise. We hope this helps readers to avoid exercises for which they haven't read the corresponding section, in addition to providing the source for review. Exercises are rated, to give the reader a sense of the amount of time required to complete an exercise:

[10] Less than 5 min (to read and understand)

[15] 5–15 min for a full answer

[20] 15–20 min for a full answer

[25] 1 h for a full written answer

[30] Short programming project: less than 1 full day of programming

[40] Significant programming project: 2 weeks of elapsed time

[Discussion] Topic for discussion with others

Solutions to the case studies and exercises are available for instructors who register at *textbooks.elsevier.com*.

Supplemental Materials

A variety of resources are available online at https://www.elsevier.com/books/computer-architecture/hennessy/978-0-12-811905-1, including the following:

- Reference appendices, some guest authored by subject experts, covering a range of advanced topics
- Historical perspectives material that explores the development of the key ideas presented in each of the chapters in the text

- Instructor slides in PowerPoint
- Figures from the book in PDF, EPS, and PPT formats
- Links to related material on the Web
- List of errata

New materials and links to other resources available on the Web will be added on a regular basis.

Helping Improve This Book

Finally, it is possible to make money while reading this book. (Talk about cost performance!) If you read the Acknowledgments that follow, you will see that we went to great lengths to correct mistakes. Since a book goes through many printings, we have the opportunity to make even more corrections. If you uncover any remaining resilient bugs, please contact the publisher by electronic mail (ca6bugs@mkp.com).

We welcome general comments to the text and invite you to send them to a separate email address at ca6comments@mkp.com.

Concluding Remarks

Once again, this book is a true co-authorship, with each of us writing half the chapters and an equal share of the appendices. We can't imagine how long it would have taken without someone else doing half the work, offering inspiration when the task seemed hopeless, providing the key insight to explain a difficult concept, supplying over-the-weekend reviews of chapters, and commiserating when the weight of our other obligations made it hard to pick up the pen.

Thus, once again, we share equally the blame for what you are about to read.

John Hennessy ■ *David Patterson*

This page intentionally left blank

Acknowledgments

Although this is only the sixth edition of this book, we have actually created ten different versions of the text: three versions of the first edition (alpha, beta, and final) and two versions of the second, third, and fourth editions (beta and final). Along the way, we have received help from hundreds of reviewers and users. Each of these people has helped make this book better. Thus, we have chosen to list all of the people who have made contributions to some version of this book.

Contributors to the Sixth Edition

Like prior editions, this is a community effort that involves scores of volunteers. Without their help, this edition would not be nearly as polished.

Reviewers

Jason D. Bakos, University of South Carolina; Rajeev Balasubramonian, University of Utah; Jose Delgado-Frias, Washington State University; Diana Franklin, The University of Chicago; Norman P. Jouppi, Google; Hugh C. Lauer, Worcester Polytechnic Institute; Gregory Peterson, University of Tennessee; Bill Pierce, Hood College; Parthasarathy Ranganathan, Google; William H. Robinson, Vanderbilt University; Pat Stakem, Johns Hopkins University; Cliff Young, Google; Amr Zaky, University of Santa Clara; Gerald Zarnett, Ryerson University; Huiyang Zhou, North Carolina State University.

Members of the University of California-Berkeley Par Lab and RAD Lab who gave frequent reviews of Chapters 1, 4, and 6 and shaped the explanation of GPUs and WSCs: Krste Asanović, Michael Armbrust, Scott Beamer, Sarah Bird, Bryan Catanzaro, Jike Chong, Henry Cook, Derrick Coetzee, Randy Katz, Yunsup Lee, Leo Meyervich, Mark Murphy, Zhangxi Tan, Vasily Volkov, and Andrew Waterman.

Appendices

Krste Asanović, University of California, Berkeley (Appendix G); Abhishek Bhattacharjee, Rutgers University (Appendix L); Thomas M. Conte, North Carolina State University (Appendix E); José Duato, Universitat Politècnica de

València and Simula (Appendix F); David Goldberg, Xerox PARC (Appendix J); Timothy M. Pinkston, University of Southern California (Appendix F).

José Flich of the Universidad Politécnica de Valencia provided significant contributions to the updating of Appendix F.

Case Studies With Exercises

Jason D. Bakos, University of South Carolina (Chapters 3 and 4); Rajeev Balasubramonian, University of Utah (Chapter 2); Diana Franklin, The University of Chicago (Chapter 1 and Appendix C); Norman P. Jouppi, Google, (Chapter 2); Naveen Muralimanohar, HP Labs (Chapter 2); Gregory Peterson, University of Tennessee (Appendix A); Parthasarathy Ranganathan, Google (Chapter 6); Cliff Young, Google (Chapter 7); Amr Zaky, University of Santa Clara (Chapter 5 and Appendix B).

Jichuan Chang, Junwhan Ahn, Rama Govindaraju, and Milad Hashemi assisted in the development and testing of the case studies and exercises for Chapter 6.

Additional Material

John Nickolls, Steve Keckler, and Michael Toksvig of NVIDIA (Chapter 4 NVIDIA GPUs); Victor Lee, Intel (Chapter 4 comparison of Core i7 and GPU); John Shalf, LBNL (Chapter 4 recent vector architectures); Sam Williams, LBNL (Roofline model for computers in Chapter 4); Steve Blackburn of Australian National University and Kathryn McKinley of University of Texas at Austin (Intel performance and power measurements in Chapter 5); Luiz Barroso, Urs Hölzle, Jimmy Clidaris, Bob Felderman, and Chris Johnson of Google (the Google WSC in Chapter 6); James Hamilton of Amazon Web Services (power distribution and cost model in Chapter 6).

Jason D. Bakos of the University of South Carolina updated the lecture slides for this edition.

This book could not have been published without a publisher, of course. We wish to thank all the Morgan Kaufmann/Elsevier staff for their efforts and support. For this fifth edition, we particularly want to thank our editors Nate McFadden and Steve Merken, who coordinated surveys, development of the case studies and exercises, manuscript reviews, and the updating of the appendices.

We must also thank our university staff, Margaret Rowland and Roxana Infante, for countless express mailings, as well as for holding down the fort at Stanford and Berkeley while we worked on the book.

Our final thanks go to our wives for their suffering through increasingly early mornings of reading, thinking, and writing.

Contributors to Previous Editions

Reviewers

George Adams, Purdue University; Sarita Adve, University of Illinois at Urbana-Champaign; Jim Archibald, Brigham Young University; Krste Asanović, Massachusetts Institute of Technology; Jean-Loup Baer, University of Washington; Paul Barr, Northeastern University; Rajendra V. Boppana, University of Texas, San Antonio; Mark Brehob, University of Michigan; Doug Burger, University of Texas, Austin; John Burger, SGI; Michael Butler; Thomas Casavant; Rohit Chandra; Peter Chen, University of Michigan; the classes at SUNY Stony Brook, Carnegie Mellon, Stanford, Clemson, and Wisconsin; Tim Coe, Vitesse Semiconductor; Robert P. Colwell; David Cummings; Bill Dally; David Douglas; José Duato, Universitat Politècnica de València and Simula; Anthony Duben, Southeast Missouri State University; Susan Eggers, University of Washington; Joel Emer; Barry Fagin, Dartmouth; Joel Ferguson, University of California, Santa Cruz; Carl Feynman; David Filo; Josh Fisher, Hewlett-Packard Laboratories; Rob Fowler, DIKU; Mark Franklin, Washington University (St. Louis); Kourosh Gharachorloo; Nikolas Gloy, Harvard University; David Goldberg, Xerox Palo Alto Research Center; Antonio González, Intel and Universitat Politècnica de Catalunya; James Goodman, University of Wisconsin-Madison; Sudhanva Gurumurthi, University of Virginia; David Harris, Harvey Mudd College; John Heinlein; Mark Heinrich, Stanford; Daniel Helman, University of California, Santa Cruz; Mark D. Hill, University of Wisconsin-Madison; Martin Hopkins, IBM; Jerry Huck, Hewlett-Packard Laboratories; Wen-mei Hwu, University of Illinois at Urbana-Champaign; Mary Jane Irwin, Pennsylvania State University; Truman Joe; Norm Jouppi; David Kaeli, Northeastern University; Roger Kieckhafer, University of Nebraska; Lev G. Kirischian, Ryerson University; Earl Killian; Allan Knies, Purdue University; Don Knuth; Jeff Kuskin, Stanford; James R. Larus, Microsoft Research; Corinna Lee, University of Toronto; Hank Levy; Kai Li, Princeton University; Lori Liebrock, University of Alaska, Fairbanks; Mikko Lipasti, University of Wisconsin-Madison; Gyula A. Mago, University of North Carolina, Chapel Hill; Bryan Martin; Norman Matloff; David Meyer; William Michalson, Worcester Polytechnic Institute; James Mooney; Trevor Mudge, University of Michigan; Ramadass Nagarajan, University of Texas at Austin; David Nagle, Carnegie Mellon University; Todd Narter; Victor Nelson; Vojin Oklobdzija, University of California, Berkeley; Kunle Olukotun, Stanford University; Bob Owens, Pennsylvania State University; Greg Papadapoulous, Sun Microsystems; Joseph Pfeiffer; Keshav Pingali, Cornell University; Timothy M. Pinkston, University of Southern California; Bruno Preiss, University of Waterloo; Steven Przybylski; Jim Quinlan; Andras Radics; Kishore Ramachandran, Georgia Institute of Technology; Joseph Rameh, University of Texas, Austin; Anthony Reeves, Cornell University; Richard Reid, Michigan State University; Steve Reinhardt, University of Michigan; David Rennels, University of California, Los Angeles; Arnold L. Rosenberg, University of Massachusetts, Amherst; Kaushik Roy, Purdue

University; Emilio Salgueiro, Unysis; Karthikeyan Sankaralingam, University of Texas at Austin; Peter Schnorf; Margo Seltzer; Behrooz Shirazi, Southern Methodist University; Daniel Siewiorek, Carnegie Mellon University; J. P. Singh, Princeton; Ashok Singhal; Jim Smith, University of Wisconsin-Madison; Mike Smith, Harvard University; Mark Smotherman, Clemson University; Gurindar Sohi, University of Wisconsin-Madison; Arun Somani, University of Washington; Gene Tagliarin, Clemson University; Shyamkumar Thoziyoor, University of Notre Dame; Evan Tick, University of Oregon; Akhilesh Tyagi, University of North Carolina, Chapel Hill; Dan Upton, University of Virginia; Mateo Valero, Universidad Politécnica de Cataluña, Barcelona; Anujan Varma, University of California, Santa Cruz; Thorsten von Eicken, Cornell University; Hank Walker, Texas A&M; Roy Want, Xerox Palo Alto Research Center; David Weaver, Sun Microsystems; Shlomo Weiss, Tel Aviv University; David Wells; Mike Westall, Clemson University; Maurice Wilkes; Eric Williams; Thomas Willis, Purdue University; Malcolm Wing; Larry Wittie, SUNY Stony Brook; Ellen Witte Zegura, Georgia Institute of Technology; Sotirios G. Ziavras, New Jersey Institute of Technology.

Appendices

The vector appendix was revised by Krste Asanović of the Massachusetts Institute of Technology. The floating-point appendix was written originally by David Goldberg of Xerox PARC.

Exercises

George Adams, Purdue University; Todd M. Bezenek, University of Wisconsin-Madison (in remembrance of his grandmother Ethel Eshom); Susan Eggers; Anoop Gupta; David Hayes; Mark Hill; Allan Knies; Ethan L. Miller, University of California, Santa Cruz; Parthasarathy Ranganathan, Compaq Western Research Laboratory; Brandon Schwartz, University of Wisconsin-Madison; Michael Scott; Dan Siewiorek; Mike Smith; Mark Smotherman; Evan Tick; Thomas Willis.

Case Studies With Exercises

Andrea C. Arpaci-Dusseau, University of Wisconsin-Madison; Remzi H. Arpaci-Dusseau, University of Wisconsin-Madison; Robert P. Colwell, R&E Colwell & Assoc., Inc.; Diana Franklin, California Polytechnic State University, San Luis Obispo; Wen-mei W. Hwu, University of Illinois at Urbana-Champaign; Norman P. Jouppi, HP Labs; John W. Sias, University of Illinois at Urbana-Champaign; David A. Wood, University of Wisconsin-Madison.

Special Thanks

Duane Adams, Defense Advanced Research Projects Agency; Tom Adams; Sarita Adve, University of Illinois at Urbana-Champaign; Anant Agarwal; Dave

Albonesi, University of Rochester; Mitch Alsup; Howard Alt; Dave Anderson; Peter Ashenden; David Bailey; Bill Bandy, Defense Advanced Research Projects Agency; Luiz Barroso, Compaq's Western Research Lab; Andy Bechtolsheim; *C. Gordon* Bell; Fred Berkowitz; John Best, IBM; Dileep Bhandarkar; Jeff Bier, BDTI; Mark Birman; David Black; David Boggs; Jim Brady; Forrest Brewer; Aaron Brown, University of California, Berkeley; E. Bugnion, Compaq's Western Research Lab; Alper Buyuktosunoglu, University of Rochester; Mark Callaghan; Jason F. Cantin; Paul Carrick; Chen-Chung Chang; Lei Chen, University of Rochester; Pete Chen; Nhan Chu; Doug Clark, Princeton University; Bob Cmelik; John Crawford; Zarka Cvetanovic; Mike Dahlin, University of Texas, Austin; Merrick Darley; the staff of the DEC Western Research Laboratory; John DeRosa; Lloyd Dickman; J. Ding; Susan Eggers, University of Washington; Wael El-Essawy, University of Rochester; Patty Enriquez, Mills; Milos Ercegovac; Robert Garner; K. Gharachorloo, Compaq's Western Research Lab; Garth Gibson; Ronald Greenberg; Ben Hao; John Henning, Compaq; Mark Hill, University of Wisconsin-Madison; Danny Hillis; David Hodges; Urs Hölzle, Google; David Hough; Ed Hudson; Chris Hughes, University of Illinois at Urbana-Champaign; Mark Johnson; Lewis Jordan; Norm Jouppi; William Kahan; Randy Katz; Ed Kelly; Richard Kessler; Les Kohn; John Kowaleski, Compaq Computer Corp; Dan Lambright; Gary Lauterbach, Sun Microsystems; Corinna Lee; Ruby Lee; Don Lewine; Chao-Huang Lin; Paul Losleben, Defense Advanced Research Projects Agency; Yung-Hsiang Lu; Bob Lucas, Defense Advanced Research Projects Agency; Ken Lutz; Alan Mainwaring, Intel Berkeley Research Labs; Al Marston; Rich Martin, Rutgers; John Mashey; Luke McDowell; Sebastian Mirolo, Trimedia Corporation; Ravi Murthy; Biswadeep Nag; Lisa Noordergraaf, Sun Microsystems; Bob Parker, Defense Advanced Research Projects Agency; Vern Paxson, Center for Internet Research; Lawrence Prince; Steven Przybylski; Mark Pullen, Defense Advanced Research Projects Agency; Chris Rowen; Margaret Rowland; Greg Semeraro, University of Rochester; Bill Shannon; Behrooz Shirazi; Robert Shomler; Jim Slager; Mark Smotherman, Clemson University; the SMT research group at the University of Washington; Steve Squires, Defense Advanced Research Projects Agency; Ajay Sreekanth; Darren Staples; Charles Stapper; Jorge Stolfi; Peter Stoll; the students at Stanford and Berkeley who endured our first attempts at creating this book; Bob Supnik; Steve Swanson; Paul Taysom; Shreekant Thakkar; Alexander Thomasian, New Jersey Institute of Technology; John Toole, Defense Advanced Research Projects Agency; Kees A. Vissers, Trimedia Corporation; Willa Walker; David Weaver; Ric Wheeler, EMC; Maurice Wilkes; Richard Zimmerman.

John Hennessy ■ *David Patterson*

1.1	Introduction	2
1.2	Classes of Computers	6
1.3	Defining Computer Architecture	11
1.4	Trends in Technology	18
1.5	Trends in Power and Energy in Integrated Circuits	23
1.6	Trends in Cost	29
1.7	Dependability	36
1.8	Measuring, Reporting, and Summarizing Performance	39
1.9	Quantitative Principles of Computer Design	48
1.10	Putting It All Together: Performance, Price, and Power	55
1.11	Fallacies and Pitfalls	58
1.12	Concluding Remarks	64
1.13	Historical Perspectives and References	67
	Case Studies and Exercises by Diana Franklin	67

1

Fundamentals of Quantitative Design and Analysis

An iPod, a phone, an Internet mobile communicator… these are NOT three separate devices! And we are calling it iPhone! Today Apple is going to reinvent the phone. And here it is.

Steve Jobs, January 9, 2007

New information and communications technologies, in particular high-speed Internet, are changing the way companies do business, transforming public service delivery and democratizing innovation. With 10 percent increase in high speed Internet connections, economic growth increases by 1.3 percent.

The World Bank, July 28, 2009

Computer Architecture. https://doi.org/10.1016/B978-0-12-811905-1.00001-8
© 2019 Elsevier Inc. All rights reserved.

1.1 Introduction

Computer technology has made incredible progress in the roughly 70 years since the first general-purpose electronic computer was created. Today, less than $500 will purchase a cell phone that has as much performance as the world's fastest computer bought in 1993 for $50 million. This rapid improvement has come both from advances in the technology used to build computers and from innovations in computer design.

Although technological improvements historically have been fairly steady, progress arising from better computer architectures has been much less consistent. During the first 25 years of electronic computers, both forces made a major contribution, delivering performance improvement of about 25% per year. The late 1970s saw the emergence of the microprocessor. The ability of the microprocessor to ride the improvements in integrated circuit technology led to a higher rate of performance improvement—roughly 35% growth per year.

This growth rate, combined with the cost advantages of a mass-produced microprocessor, led to an increasing fraction of the computer business being based on microprocessors. In addition, two significant changes in the computer marketplace made it easier than ever before to succeed commercially with a new architecture. First, the virtual elimination of assembly language programming reduced the need for object-code compatibility. Second, the creation of standardized, vendor-independent operating systems, such as UNIX and its clone, Linux, lowered the cost and risk of bringing out a new architecture.

These changes made it possible to develop successfully a new set of architectures with simpler instructions, called RISC (Reduced Instruction Set Computer) architectures, in the early 1980s. The RISC-based machines focused the attention of designers on two critical performance techniques, the exploitation of *instruction-level parallelism* (initially through pipelining and later through multiple instruction issue) and the use of caches (initially in simple forms and later using more sophisticated organizations and optimizations).

The RISC-based computers raised the performance bar, forcing prior architectures to keep up or disappear. The Digital Equipment Vax could not, and so it was replaced by a RISC architecture. Intel rose to the challenge, primarily by translating 80x86 instructions into RISC-like instructions internally, allowing it to adopt many of the innovations first pioneered in the RISC designs. As transistor counts soared in the late 1990s, the hardware overhead of translating the more complex x86 architecture became negligible. In low-end applications, such as cell phones, the cost in power and silicon area of the x86-translation overhead helped lead to a RISC architecture, ARM, becoming dominant.

Figure 1.1 shows that the combination of architectural and organizational enhancements led to 17 years of sustained growth in performance at an annual rate of over 50%—a rate that is unprecedented in the computer industry.

The effect of this dramatic growth rate during the 20th century was fourfold. First, it has significantly enhanced the capability available to computer users. For many applications, the highest-performance microprocessors outperformed the supercomputer of less than 20 years earlier.

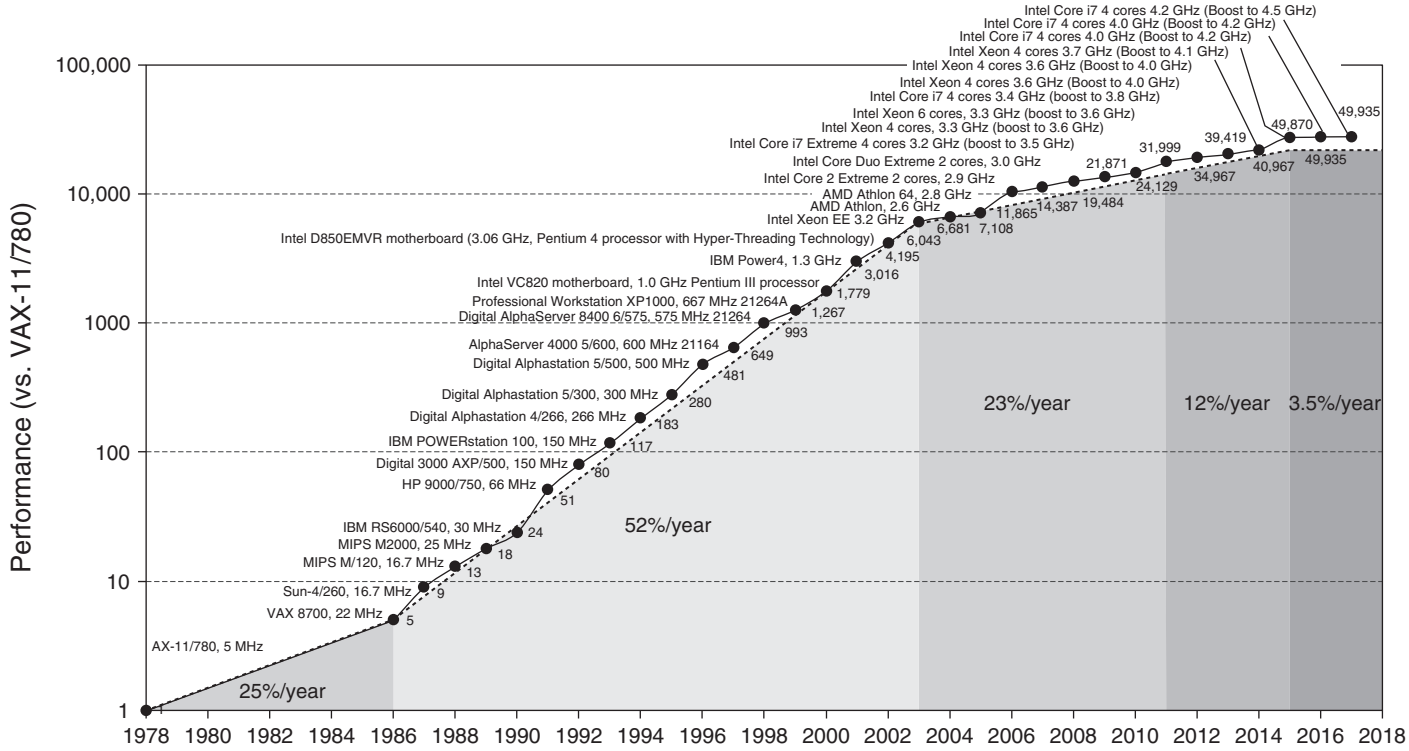


Figure 1.1 Growth in processor performance over 40 years. This chart plots program performance relative to the VAX 11/780 as measured by the SPEC integer benchmarks (see Section 1.8). Prior to the mid-1980s, growth in processor performance was largely technology-driven and averaged about 22% per year, or doubling performance every 3.5 years. The increase in growth to about 52% starting in 1986, or doubling every 2 years, is attributable to more advanced architectural and organizational ideas typified in RISC architectures. By 2003 this growth led to a difference in performance of an approximate factor of 25 versus the performance that would have occurred if it had continued at the 22% rate. In 2003 the limits of power due to the end of Dennard scaling and the available instruction-level parallelism slowed uniprocessor performance to 23% per year until 2011, or doubling every 3.5 years. (The fastest SPECintbase performance since 2007 has had automatic parallelization turned on, so uniprocessor speed is harder to gauge. These results are limited to single-chip systems with usually four cores per chip.) From 2011 to 2015, the annual improvement was less than 12%, or doubling every 8 years in part due to the limits of parallelism of Amdahl's Law. Since 2015, with the end of Moore's Law, improvement has been just 3.5% per year, or doubling every 20 years! Performance for floating-point-oriented calculations follows the same trends, but typically has 1% to 2% higher annual growth in each shaded region. Figure 1.11 on page 27 shows the improvement in clock rates for these same eras. Because SPEC has changed over the years, performance of newer machines is estimated by a scaling factor that relates the performance for different versions of SPEC: SPEC89, SPEC92, SPEC95, SPEC2000, and SPEC2006. There are too few results for SPEC2017 to plot yet.

Second, this dramatic improvement in cost-performance led to new classes of computers. Personal computers and workstations emerged in the 1980s with the availability of the microprocessor. The past decade saw the rise of smart cell phones and tablet computers, which many people are using as their primary computing platforms instead of PCs. These mobile client devices are increasingly using the Internet to access warehouses containing 100,000 servers, which are being designed as if they were a single gigantic computer.

Third, improvement of semiconductor manufacturing as predicted by Moore's law has led to the dominance of microprocessor-based computers across the entire range of computer design. Minicomputers, which were traditionally made from off-the-shelf logic or from gate arrays, were replaced by servers made by using microprocessors. Even mainframe computers and high-performance supercomputers are all collections of microprocessors.

The preceding hardware innovations led to a renaissance in computer design, which emphasized both architectural innovation and efficient use of technology improvements. This rate of growth compounded so that by 2003, high-performance microprocessors were 7.5 times as fast as what would have been obtained by relying solely on technology, including improved circuit design, that is, 52% per year versus 35% per year.

This hardware renaissance led to the fourth impact, which was on software development. This 50,000-fold performance improvement since 1978 (see Figure 1.1) allowed modern programmers to trade performance for productivity. In place of performance-oriented languages like C and C++, much more programming today is done in managed programming languages like Java and Scala. Moreover, scripting languages like JavaScript and Python, which are even more productive, are gaining in popularity along with programming frameworks like AngularJS and Django. To maintain productivity and try to close the performance gap, interpreters with just-in-time compilers and trace-based compiling are replacing the traditional compiler and linker of the past. Software deployment is changing as well, with Software as a Service (SaaS) used over the Internet replacing shrink-wrapped software that must be installed and run on a local computer.

The nature of applications is also changing. Speech, sound, images, and video are becoming increasingly important, along with predictable response time that is so critical to the user experience. An inspiring example is Google Translate. This application lets you hold up your cell phone to point its camera at an object, and the image is sent wirelessly over the Internet to a warehouse-scale computer (WSC) that recognizes the text in the photo and translates it into your native language. You can also speak into it, and it will translate what you said into audio output in another language. It translates text in 90 languages and voice in 15 languages.

Alas, Figure 1.1 also shows that this 17-year hardware renaissance is over. The fundamental reason is that two characteristics of semiconductor processes that were true for decades no longer hold.

In 1974 Robert Dennard observed that power density was constant for a given area of silicon even as you increased the number of transistors because of smaller dimensions of each transistor. Remarkably, transistors could go faster but use less

power. *Dennard scaling* ended around 2004 because current and voltage couldn't keep dropping and still maintain the dependability of integrated circuits.

This change forced the microprocessor industry to use multiple efficient processors or cores instead of a single inefficient processor. Indeed, in 2004 Intel canceled its high-performance uniprocessor projects and joined others in declaring that the road to higher performance would be via multiple processors per chip rather than via faster uniprocessors. This milestone signaled a historic switch from relying solely on instruction-level parallelism (ILP), the primary focus of the first three editions of this book, to *data-level parallelism* (DLP) and *thread-level parallelism* (TLP), which were featured in the fourth edition and expanded in the fifth edition. The fifth edition also added WSCs and *request-level parallelism* (RLP), which is expanded in this edition. Whereas the compiler and hardware conspire to exploit ILP implicitly without the programmer's attention, DLP, TLP, and RLP are explicitly parallel, requiring the restructuring of the application so that it can exploit explicit parallelism. In some instances, this is easy; in many, it is a major new burden for programmers.

Amdahl's Law (Section 1.9) prescribes practical limits to the number of useful cores per chip. If 10% of the task is serial, then the maximum performance benefit from parallelism is 10 no matter how many cores you put on the chip.

The second observation that ended recently is *Moore's Law*. In 1965 Gordon Moore famously predicted that the number of transistors per chip would double every year, which was amended in 1975 to every two years. That prediction lasted for about 50 years, but no longer holds. For example, in the 2010 edition of this book, the most recent Intel microprocessor had 1,170,000,000 transistors. If Moore's Law had continued, we could have expected microprocessors in 2016 to have 18,720,000,000 transistors. Instead, the equivalent Intel microprocessor has just 1,750,000,000 transistors, or off by a factor of 10 from what Moore's Law would have predicted.

The combination of

- transistors no longer getting much better because of the slowing of Moore's Law and the end of Dinnard scaling,
- the unchanging power budgets for microprocessors,
- the replacement of the single power-hungry processor with several energy-efficient processors, and
- the limits to multiprocessing to achieve Amdahl's Law

caused improvements in processor performance to slow down, that is, to *double every 20 years,* rather than every 1.5 years as it did between 1986 and 2003 (see Figure 1.1).

The only path left to improve energy-performance-cost is specialization. Future microprocessors will include several domain-specific cores that perform only one class of computations well, but they do so remarkably better than general-purpose cores. The new Chapter 7 in this edition introduces *domain-specific architectures*.

This text is about the architectural ideas and accompanying compiler improvements that made the incredible growth rate possible over the past century, the reasons for the dramatic change, and the challenges and initial promising approaches to architectural ideas, compilers, and interpreters for the 21st century. At the core is a quantitative approach to computer design and analysis that uses empirical observations of programs, experimentation, and simulation as its tools. It is this style and approach to computer design that is reflected in this text. The purpose of this chapter is to lay the quantitative foundation on which the following chapters and appendices are based.

This book was written not only to explain this design style but also to stimulate you to contribute to this progress. We believe this approach will serve the computers of the future just as it worked for the implicitly parallel computers of the past.

1.2 Classes of Computers

These changes have set the stage for a dramatic change in how we view computing, computing applications, and the computer markets in this new century. Not since the creation of the personal computer have we seen such striking changes in the way computers appear and in how they are used. These changes in computer use have led to five diverse computing markets, each characterized by different applications, requirements, and computing technologies. Figure 1.2 summarizes these mainstream classes of computing environments and their important characteristics.

Internet of Things/Embedded Computers

Embedded computers are found in everyday machines: microwaves, washing machines, most printers, networking switches, and all automobiles. The phrase

Feature	Personal mobile device (PMD)	Desktop	Server	Clusters/warehouse-scale computer	Internet of things/ embedded
Price of system	$100–$1000	$300–$2500	$5000–$10,000,000	$100,000–$200,000,000	$10–$100,000
Price of microprocessor	$10–$100	$50–$500	$200–$2000	$50–$250	$0.01–$100
Critical system design issues	Cost, energy, media performance, responsiveness	Price-performance, energy, graphics performance	Throughput, availability, scalability, energy	Price-performance, throughput, energy proportionality	Price, energy, application-specific performance

Figure 1.2 A summary of the five mainstream computing classes and their system characteristics. Sales in 2015 included about 1.6 billion PMDs (90% cell phones), 275 million desktop PCs, and 15 million servers. The total number of embedded processors sold was nearly 19 billion. In total, 14.8 billion ARM-technology-based chips were shipped in 2015. Note the wide range in system price for servers and embedded systems, which go from USB keys to network routers. For servers, this range arises from the need for very large-scale multiprocessor systems for high-end transaction processing.

Internet of Things (IoT) refers to embedded computers that are connected to the Internet, typically wirelessly. When augmented with sensors and actuators, IoT devices collect useful data and interact with the physical world, leading to a wide variety of "smart" applications, such as smart watches, smart thermostats, smart speakers, smart cars, smart homes, smart grids, and smart cities.

Embedded computers have the widest spread of processing power and cost. They include 8-bit to 32-bit processors that may cost one penny, and high-end 64-bit processors for cars and network switches that cost $100. Although the range of computing power in the embedded computing market is very large, price is a key factor in the design of computers for this space. Performance requirements do exist, of course, but the primary goal often meets the performance need at a minimum price, rather than achieving more performance at a higher price. The projections for the number of IoT devices in 2020 range from 20 to 50 billion.

Most of this book applies to the design, use, and performance of embedded processors, whether they are off-the-shelf microprocessors or microprocessor cores that will be assembled with other special-purpose hardware.

Unfortunately, the data that drive the quantitative design and evaluation of other classes of computers have not yet been extended successfully to embedded computing (see the challenges with EEMBC, for example, in Section 1.8). Hence we are left for now with qualitative descriptions, which do not fit well with the rest of the book. As a result, the embedded material is concentrated in Appendix E. We believe a separate appendix improves the flow of ideas in the text while allowing readers to see how the differing requirements affect embedded computing.

Personal Mobile Device

Personal mobile device (PMD) is the term we apply to a collection of wireless devices with multimedia user interfaces such as cell phones, tablet computers, and so on. Cost is a prime concern given the consumer price for the whole product is a few hundred dollars. Although the emphasis on energy efficiency is frequently driven by the use of batteries, the need to use less expensive packaging—plastic versus ceramic—and the absence of a fan for cooling also limit total power consumption. We examine the issue of energy and power in more detail in Section 1.5. Applications on PMDs are often web-based and media-oriented, like the previously mentioned Google Translate example. Energy and size requirements lead to use of Flash memory for storage (Chapter 2) instead of magnetic disks.

The processors in a PMD are often considered embedded computers, but we are keeping them as a separate category because PMDs are platforms that can run externally developed software, and they share many of the characteristics of desktop computers. Other embedded devices are more limited in hardware and software sophistication. We use the ability to run third-party software as the dividing line between nonembedded and embedded computers.

Responsiveness and predictability are key characteristics for media applications. A *real-time performance* requirement means a segment of the application has an absolute maximum execution time. For example, in playing a video on a

PMD, the time to process each video frame is limited, since the processor must accept and process the next frame shortly. In some applications, a more nuanced requirement exists: the average time for a particular task is constrained as well as the number of instances when some maximum time is exceeded. Such approaches—sometimes called *soft real-time*—arise when it is possible to miss the time constraint on an event occasionally, as long as not too many are missed. Real-time performance tends to be highly application-dependent.

Other key characteristics in many PMD applications are the need to minimize memory and the need to use energy efficiently. Energy efficiency is driven by both battery power and heat dissipation. The memory can be a substantial portion of the system cost, and it is important to optimize memory size in such cases. The importance of memory size translates to an emphasis on code size, since data size is dictated by the application.

Desktop Computing

The first, and possibly still the largest market in dollar terms, is desktop computing. Desktop computing spans from low-end netbooks that sell for under $300 to high-end, heavily configured workstations that may sell for $2500. Since 2008, more than half of the desktop computers made each year have been battery operated laptop computers. Desktop computing sales are declining.

Throughout this range in price and capability, the desktop market tends to be driven to optimize *price-performance*. This combination of performance (measured primarily in terms of compute performance and graphics performance) and price of a system is what matters most to customers in this market, and hence to computer designers. As a result, the newest, highest-performance microprocessors and cost-reduced microprocessors often appear first in desktop systems (see Section 1.6 for a discussion of the issues affecting the cost of computers).

Desktop computing also tends to be reasonably well characterized in terms of applications and benchmarking, though the increasing use of web-centric, interactive applications poses new challenges in performance evaluation.

Servers

As the shift to desktop computing occurred in the 1980s, the role of servers grew to provide larger-scale and more reliable file and computing services. Such servers have become the backbone of large-scale enterprise computing, replacing the traditional mainframe.

For servers, different characteristics are important. First, availability is critical. (We discuss availability in Section 1.7.) Consider the servers running ATM machines for banks or airline reservation systems. Failure of such server systems is far more catastrophic than failure of a single desktop, since these servers must operate seven days a week, 24 hours a day. Figure 1.3 estimates revenue costs of downtime for server applications.

Application	Cost of downtime per hour	Annual losses with downtime of		
		1% (87.6 h/year)	0.5% (43.8 h/year)	0.1% (8.8 h/year)
Brokerage service	$4,000,000	$350,400,000	$175,200,000	$35,000,000
Energy	$1,750,000	$153,300,000	$76,700,000	$15,300,000
Telecom	$1,250,000	$109,500,000	$54,800,000	$11,000,000
Manufacturing	$1,000,000	$87,600,000	$43,800,000	$8,800,000
Retail	$650,000	$56,900,000	$28,500,000	$5,700,000
Health care	$400,000	$35,000,000	$17,500,000	$3,500,000
Media	$50,000	$4,400,000	$2,200,000	$400,000

Figure 1.3 Costs rounded to nearest $100,000 of an unavailable system are shown by analyzing the cost of downtime (in terms of immediately lost revenue), assuming three different levels of availability, and that downtime is distributed uniformly. These data are from Landstrom (2014) and were collected and analyzed by Contingency Planning Research.

A second key feature of server systems is scalability. Server systems often grow in response to an increasing demand for the services they support or an expansion in functional requirements. Thus the ability to scale up the computing capacity, the memory, the storage, and the I/O bandwidth of a server is crucial.

Finally, servers are designed for efficient throughput. That is, the overall performance of the server—in terms of transactions per minute or web pages served per second—is what is crucial. Responsiveness to an individual request remains important, but overall efficiency and cost-effectiveness, as determined by how many requests can be handled in a unit time, are the key metrics for most servers. We return to the issue of assessing performance for different types of computing environments in Section 1.8.

Clusters/Warehouse-Scale Computers

The growth of Software as a Service (SaaS) for applications like search, social networking, video viewing and sharing, multiplayer games, online shopping, and so on has led to the growth of a class of computers called *clusters*. Clusters are collections of desktop computers or servers connected by local area networks to act as a single larger computer. Each node runs its own operating system, and nodes communicate using a networking protocol. WSCs are the largest of the clusters, in that they are designed so that tens of thousands of servers can act as one. Chapter 6 describes this class of extremely large computers.

Price-performance and power are critical to WSCs since they are so large. As Chapter 6 explains, the majority of the cost of a warehouse is associated with power and cooling of the computers inside the warehouse. The annual amortized computers themselves and the networking gear cost for a WSC is $40 million, because they are usually replaced every few years. When you are buying that

much computing, you need to buy wisely, because a 10% improvement in price-performance means an annual savings of $4 million (10% of $40 million) per WSC; a company like Amazon might have 100 WSCs!

WSCs are related to servers in that availability is critical. For example, Amazon.com had $136 billion in sales in 2016. As there are about 8800 hours in a year, the average revenue per hour was about $15 million. During a peak hour for Christmas shopping, the potential loss would be many times higher. As Chapter 6 explains, the difference between WSCs and servers is that WSCs use redundant, inexpensive components as the building blocks, relying on a software layer to catch and isolate the many failures that will happen with computing at this scale to deliver the availability needed for such applications. Note that scalability for a WSC is handled by the local area network connecting the computers and not by integrated computer hardware, as in the case of servers.

Supercomputers are related to WSCs in that they are equally expensive, costing hundreds of millions of dollars, but supercomputers differ by emphasizing floating-point performance and by running large, communication-intensive batch programs that can run for weeks at a time. In contrast, WSCs emphasize interactive applications, large-scale storage, dependability, and high Internet bandwidth.

Classes of Parallelism and Parallel Architectures

Parallelism at multiple levels is now the driving force of computer design across all four classes of computers, with energy and cost being the primary constraints. There are basically two kinds of parallelism in applications:

1. *Data-level parallelism (DLP)* arises because there are many data items that can be operated on at the same time.
2. *Task-level parallelism (TLP)* arises because tasks of work are created that can operate independently and largely in parallel.

Computer hardware in turn can exploit these two kinds of application parallelism in four major ways:

1. *Instruction-level parallelism* exploits data-level parallelism at modest levels with compiler help using ideas like pipelining and at medium levels using ideas like speculative execution.
2. *Vector architectures, graphic processor units (GPUs), and multimedia instruction sets* exploit data-level parallelism by applying a single instruction to a collection of data in parallel.
3. *Thread-level parallelism* exploits either data-level parallelism or task-level parallelism in a tightly coupled hardware model that allows for interaction between parallel threads.
4. *Request-level parallelism* exploits parallelism among largely decoupled tasks specified by the programmer or the operating system.

When Flynn (1966) studied the parallel computing efforts in the 1960s, he found a simple classification whose abbreviations we still use today. They target data-level parallelism and task-level parallelism. He looked at the parallelism in the instruction and data streams called for by the instructions at the most constrained component of the multiprocessor and placed all computers in one of four categories:

1. *Single instruction stream, single data stream* (SISD)—This category is the uniprocessor. The programmer thinks of it as the standard sequential computer, but it can exploit ILP. Chapter 3 covers SISD architectures that use ILP techniques such as superscalar and speculative execution.
2. *Single instruction stream, multiple data streams* (SIMD)—The same instruction is executed by multiple processors using different data streams. SIMD computers exploit *data-level parallelism* by applying the same operations to multiple items of data in parallel. Each processor has its own data memory (hence, the MD of SIMD), but there is a single instruction memory and control processor, which fetches and dispatches instructions. Chapter 4 covers DLP and three different architectures that exploit it: vector architectures, multimedia extensions to standard instruction sets, and GPUs.
3. *Multiple instruction streams, single data stream* (MISD)—No commercial multiprocessor of this type has been built to date, but it rounds out this simple classification.
4. *Multiple instruction streams, multiple data streams* (MIMD)—Each processor fetches its own instructions and operates on its own data, and it targets task-level parallelism. In general, MIMD is more flexible than SIMD and thus more generally applicable, but it is inherently more expensive than SIMD. For example, MIMD computers can also exploit data-level parallelism, although the overhead is likely to be higher than would be seen in an SIMD computer. This overhead means that grain size must be sufficiently large to exploit the parallelism efficiently. Chapter 5 covers tightly coupled MIMD architectures, which exploit *thread-level parallelism* because multiple cooperating threads operate in parallel. Chapter 6 covers loosely coupled MIMD architectures—specifically, *clusters* and *warehouse-scale computers*—that exploit *request-level parallelism*, where many independent tasks can proceed in parallel naturally with little need for communication or synchronization.

This taxonomy is a coarse model, as many parallel processors are hybrids of the SISD, SIMD, and MIMD classes. Nonetheless, it is useful to put a framework on the design space for the computers we will see in this book.

1.3 Defining Computer Architecture

The task the computer designer faces is a complex one: determine what attributes are important for a new computer, then design a computer to maximize

performance and energy efficiency while staying within cost, power, and availability constraints. This task has many aspects, including instruction set design, functional organization, logic design, and implementation. The implementation may encompass integrated circuit design, packaging, power, and cooling. Optimizing the design requires familiarity with a very wide range of technologies, from compilers and operating systems to logic design and packaging.

A few decades ago, the term *computer architecture* generally referred to only instruction set design. Other aspects of computer design were called *implementation,* often insinuating that implementation is uninteresting or less challenging.

We believe this view is incorrect. The architect's or designer's job is much more than instruction set design, and the technical hurdles in the other aspects of the project are likely more challenging than those encountered in instruction set design. We'll quickly review instruction set architecture before describing the larger challenges for the computer architect.

Instruction Set Architecture: The Myopic View of Computer Architecture

We use the term *instruction set architecture* (ISA) to refer to the actual programmer-visible instruction set in this book. The ISA serves as the boundary between the software and hardware. This quick review of ISA will use examples from 80x86, ARMv8, and RISC-V to illustrate the seven dimensions of an ISA. The most popular RISC processors come from ARM (Advanced RISC Machine), which were in 14.8 billion chips shipped in 2015, or roughly 50 times as many chips that shipped with 80x86 processors. Appendices A and K give more details on the three ISAs.

RISC-V ("RISC Five") is a modern RISC instruction set developed at the University of California, Berkeley, which was made free and openly adoptable in response to requests from industry. In addition to a full software stack (compilers, operating systems, and simulators), there are several RISC-V implementations freely available for use in custom chips or in field-programmable gate arrays. Developed 30 years after the first RISC instruction sets, RISC-V inherits its ancestors' good ideas—a large set of registers, easy-to-pipeline instructions, and a lean set of operations—while avoiding their omissions or mistakes. It is a free and open, elegant example of the RISC architectures mentioned earlier, which is why more than 60 companies have joined the RISC-V foundation, including AMD, Google, HP Enterprise, IBM, Microsoft, Nvidia, Qualcomm, Samsung, and Western Digital. We use the integer core ISA of RISC-V as the example ISA in this book.

1. *Class of ISA*—Nearly all ISAs today are classified as general-purpose register architectures, where the operands are either registers or memory locations. The 80x86 has 16 general-purpose registers and 16 that can hold floating-point data, while RISC-V has 32 general-purpose and 32 floating-point registers (see Figure 1.4). The two popular versions of this class are *register-memory* ISAs,

Register	Name	Use	Saver
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	–
x4	tp	Thread pointer	–
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP temporaries	Caller
f8-f9	fs0-fs1	FP saved registers	Callee
f10-f11	fa0-fa1	FP function arguments/return values	Caller
f12-f17	fa2-fa7	FP function arguments	Caller
f18-f27	fs2-fs11	FP saved registers	Callee
f28-f31	ft8-ft11	FP temporaries	Caller

Figure 1.4 RISC-V registers, names, usage, and calling conventions. In addition to the 32 general-purpose registers (x0–x31), RISC-V has 32 floating-point registers (f0–f31) that can hold either a 32-bit single-precision number or a 64-bit double-precision number. The registers that are preserved across a procedure call are labeled "Callee" saved.

such as the 80x86, which can access memory as part of many instructions, and *load-store* ISAs, such as ARMv8 and RISC-V, which can access memory only with load or store instructions. All ISAs announced since 1985 are load-store.

2. *Memory addressing*—Virtually all desktop and server computers, including the 80x86, ARMv8, and RISC-V, use byte addressing to access memory operands. Some architectures, like ARMv8, require that objects must be *aligned*. An access to an object of size s bytes at byte address A is aligned if A mod $s = 0$. (See Figure A.5 on page A-8.) The 80x86 and RISC-V do not require alignment, but accesses are generally faster if operands are aligned.
3. *Addressing modes*—In addition to specifying registers and constant operands, addressing modes specify the address of a memory object. RISC-V addressing modes are Register, Immediate (for constants), and Displacement, where a constant offset is added to a register to form the memory address. The 80x86 supports those three modes, plus three variations of displacement: no register (absolute), two registers (based indexed with displacement), and two registers

where one register is multiplied by the size of the operand in bytes (based with scaled index and displacement). It has more like the last three modes, minus the displacement field, plus register indirect, indexed, and based with scaled index. ARMv8 has the three RISC-V addressing modes plus PC-relative addressing, the sum of two registers, and the sum of two registers where one register is multiplied by the size of the operand in bytes. It also has autoincrement and autodecrement addressing, where the calculated address replaces the contents of one of the registers used in forming the address.

4. *Types and sizes of operands*—Like most ISAs, 80x86, ARMv8, and RISC-V support operand sizes of 8-bit (ASCII character), 16-bit (Unicode character or half word), 32-bit (integer or word), 64-bit (double word or long integer), and IEEE 754 floating point in 32-bit (single precision) and 64-bit (double precision). The 80x86 also supports 80-bit floating point (extended double precision).
5. *Operations*—The general categories of operations are data transfer, arithmetic logical, control (discussed next), and floating point. RISC-V is a simple and easy-to-pipeline instruction set architecture, and it is representative of the RISC architectures being used in 2017. Figure 1.5 summarizes the integer RISC-V ISA, and Figure 1.6 lists the floating-point ISA. The 80x86 has a much richer and larger set of operations (see Appendix K).
6. *Control flow instructions*—Virtually all ISAs, including these three, support conditional branches, unconditional jumps, procedure calls, and returns. All three use PC-relative addressing, where the branch address is specified by an address field that is added to the PC. There are some small differences. RISC-V conditional branches (`BE`, `BNE`, etc.) test the contents of registers, and the 80x86 and ARMv8 branches test condition code bits set as side effects of arithmetic/logic operations. The ARMv8 and RISC-V procedure call places the return address in a register, whereas the 80x86 call (`CALLF`) places the return address on a stack in memory.
7. *Encoding an ISA*—There are two basic choices on encoding: *fixed length* and *variable length*. All ARMv8 and RISC-V instructions are 32 bits long, which simplifies instruction decoding. Figure 1.7 shows the RISC-V instruction formats. The 80x86 encoding is variable length, ranging from 1 to 18 bytes. Variable-length instructions can take less space than fixed-length instructions, so a program compiled for the 80x86 is usually smaller than the same program compiled for RISC-V. Note that choices mentioned previously will affect how the instructions are encoded into a binary representation. For example, the number of registers and the number of addressing modes both have a significant impact on the size of instructions, because the register field and addressing mode field can appear many times in a single instruction. (Note that ARMv8 and RISC-V later offered extensions, called Thumb-2 and RV64IC, that provide a mix of 16-bit and 32-bit length instructions, respectively, to reduce program size. Code size for these compact versions of RISC architectures are smaller than that of the 80x86. See Appendix K.)

Instruction type/opcode	Instruction meaning
Data transfers	*Move data between registers and memory, or between the integer and FP or special registers; only memory address mode is 12-bit displacement + contents of a GPR*
`lb`, `lbu`, `sb`	Load byte, load byte unsigned, store byte (to/from integer registers)
`lh`, `lhu`, `sh`	Load half word, load half word unsigned, store half word (to/from integer registers)
`lw`, `lwu`, `sw`	Load word, load word unsigned, store word (to/from integer registers)
`ld`, `sd`	Load double word, store double word (to/from integer registers)
`flw`, `fld`, `fsw`, `fsd`	Load SP float, load DP float, store SP float, store DP float
`fmv._.x`, `fmv.x._`	Copy from/to integer register to/from floating-point register; "__" = S for single-precision, D for double-precision
`csrrw`, `csrrwi`, `csrrs`, `csrrsi`, `csrrc`, `csrrci`	Read counters and write status registers, which include counters: clock cycles, time, instructions retired
Arithmetic/logical	*Operations on integer or logical data in GPRs*
`add`, `addi`, `addw`, `addiw`	Add, add immediate (all immediates are 12 bits), add 32-bits only & sign-extend to 64 bits, add immediate 32-bits only
`sub`, `subw`	Subtract, subtract 32-bits only
`mul`, `mulw`, `mulh`, `mulhsu`, `mulhu`	Multiply, multiply 32-bits only, multiply upper half, multiply upper half signed-unsigned, multiply upper half unsigned
`div`, `divu`, `rem`, `remu`	Divide, divide unsigned, remainder, remainder unsigned
`divw`, `divuw`, `remw`, `remuw`	Divide and remainder: as previously, but divide only lower 32-bits, producing 32-bit sign-extended result
`and`, `andi`	And, and immediate
`or`, `ori`, `xor`, `xori`	Or, or immediate, exclusive or, exclusive or immediate
`lui`	Load upper immediate; loads bits 31-12 of register with immediate, then sign-extends
`auipc`	Adds immediate in bits 31–12 with zeros in lower bits to PC; used with `JALR` to transfer control to any 32-bit address
`sll`, `slli`, `srl`, `srli`, `sra`, `srai`	Shifts: shift left logical, right logical, right arithmetic; both variable and immediate forms
`sllw`, `slliw`, `srlw`, `srliw`, `sraw`, `sraiw`	Shifts: as previously, but shift lower 32-bits, producing 32-bit sign-extended result
`slt`, `slti`, `sltu`, `sltiu`	Set less than, set less than immediate, signed and unsigned
Control	*Conditional branches and jumps; PC-relative or through register*
`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`	Branch GPR equal/not equal; less than; greater than or equal, signed and unsigned
`jal`, `jalr`	Jump and link: save PC + 4, target is PC-relative (`JAL`) or a register (`JALR`); if specify `x0` as destination register, then acts as a simple jump
`ecall`	Make a request to the supporting execution environment, which is usually an OS
`ebreak`	Debuggers used to cause control to be transferred back to a debugging environment
`fence`, `fence.i`	Synchronize threads to guarantee ordering of memory accesses; synchronize instructions and data for stores to instruction memory

Figure 1.5 Subset of the instructions in RISC-V. RISC-V has a base set of instructions (R64I) and offers optional extensions: multiply-divide (RVM), single-precision floating point (RVF), double-precision floating point (RVD). This figure includes RVM and the next one shows RVF and RVD. Appendix A gives much more detail on RISC-V.

Instruction type/opcode	Instruction meaning
Floating point	*FP operations on DP and SP formats*
`fadd.d`, `fadd.s`	Add DP, SP numbers
`fsub.d`, `fsub.s`	Subtract DP, SP numbers
`fmul.d`, `fmul.s`	Multiply DP, SP floating point
`fmadd.d`, `fmadd.s`, `fnmadd.d`, `fnmadd.s`	Multiply-add DP, SP numbers; negative multiply-add DP, SP numbers
`fmsub.d`, `fmsub.s`, `fnmsub.d`, `fnmsub.s`	Multiply-sub DP, SP numbers; negative multiply-sub DP, SP numbers
`fdiv.d`, `fdiv.s`	Divide DP, SP floating point
`fsqrt.d`, `fsqrt.s`	Square root DP, SP floating point
`fmax.d`, `fmax.s`, `fmin.d`, `fmin.s`	Maximum and minimum DP, SP floating point
`fcvt._._`, `fcvt._._u`, `fcvt._u._`	Convert instructions: `FCVT.x.y` converts from type `x` to type `y`, where `x` and `y` are `L` (64-bit integer), `W` (32-bit integer), `D` (DP), or `S` (SP). Integers can be unsigned (`U`)
`feq._`, `flt._`,`fle._`	Floating-point compare between floating-point registers and record the Boolean result in integer register; "`_`" = `S` for single-precision, `D` for double-precision
`fclass.d`, `fclass.s`	Writes to integer register a 10-bit mask that indicates the class of the floating-point number ($-\infty$, $+\infty$, -0, +0, NaN, …)
`fsgnj._`, `fsgnjn._`, `fsgnjx._`	Sign-injection instructions that changes only the sign bit: copy sign bit from other source, the oppositive of sign bit of other source, XOR of the 2 sign bits

Figure 1.6 Floating point instructions for RISC-V. RISC-V has a base set of instructions (R64I) and offers optional extensions for single-precision floating point (RVF) and double-precision floating point (RVD). SP = single precision; DP = double precision.

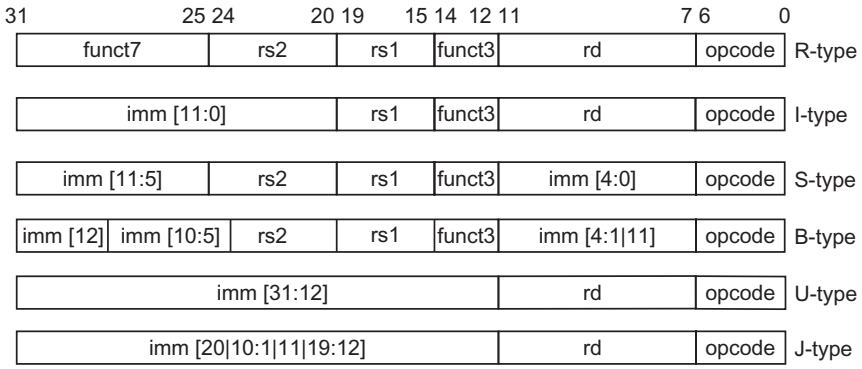


Figure 1.7 The base RISC-V instruction set architecture formats. All instructions are 32 bits long. The R format is for integer register-to-register operations, such as ADD, SUB, and so on. The I format is for loads and immediate operations, such as LD and ADDI. The B format is for branches and the J format is for jumps and link. The S format is for stores. Having a separate format for stores allows the three register specifiers (rd, rs1, rs2) to always be in the same location in all formats. The U format is for the wide immediate instructions (LUI, AUIPC).

The other challenges facing the computer architect beyond ISA design are particularly acute at the present, when the differences among instruction sets are small and when there are distinct application areas. Therefore, starting with the fourth edition of this book, beyond this quick review, the bulk of the instruction set material is found in the appendices (see Appendices A and K).

Genuine Computer Architecture: Designing the Organization and Hardware to Meet Goals and Functional Requirements

The implementation of a computer has two components: organization and hardware. The term *organization* includes the high-level aspects of a computer's design, such as the memory system, the memory interconnect, and the design of the internal processor or CPU (central processing unit—where arithmetic, logic, branching, and data transfer are implemented). The term *microarchitecture* is also used instead of organization. For example, two processors with the same instruction set architectures but different organizations are the AMD Opteron and the Intel Core i7. Both processors implement the 80x86 instruction set, but they have very different pipeline and cache organizations.

The switch to multiple processors per microprocessor led to the term *core* also being used for processors. Instead of saying multiprocessor microprocessor, the term *multicore* caught on. Given that virtually all chips have multiple processors, the term central processing unit, or CPU, is fading in popularity.

Hardware refers to the specifics of a computer, including the detailed logic design and the packaging technology of the computer. Often a line of computers contains computers with identical instruction set architectures and very similar organizations, but they differ in the detailed hardware implementation. For example, the Intel Core i7 (see Chapter 3) and the Intel Xeon E7 (see Chapter 5) are nearly identical but offer different clock rates and different memory systems, making the Xeon E7 more effective for server computers.

In this book, the word *architecture* covers all three aspects of computer design—instruction set architecture, organization or microarchitecture, and hardware.

Computer architects must design a computer to meet functional requirements as well as price, power, performance, and availability goals. Figure 1.8 summarizes requirements to consider in designing a new computer. Often, architects also must determine what the functional requirements are, which can be a major task. The requirements may be specific features inspired by the market. Application software typically drives the choice of certain functional requirements by determining how the computer will be used. If a large body of software exists for a particular instruction set architecture, the architect may decide that a new computer should implement an existing instruction set. The presence of a large market for a particular class of applications might encourage the designers to incorporate requirements that would make the computer competitive in that market. Later chapters examine many of these requirements and features in depth.

Functional requirements	Typical features required or supported
Application area	*Target of computer*
Personal mobile device	Real-time performance for a range of tasks, including interactive performance for graphics, video, and audio; energy efficiency (Chapters 2–5 and 7; Appendix A)
General-purpose desktop	Balanced performance for a range of tasks, including interactive performance for graphics, video, and audio (Chapters 2–5; Appendix A)
Servers	Support for databases and transaction processing; enhancements for reliability and availability; support for scalability (Chapters 2, 5, and 7; Appendices A, D, and F)
Clusters/warehouse-scale computers	Throughput performance for many independent tasks; error correction for memory; energy proportionality (Chapters 2, 6, and 7; Appendix F)
Internet of things/embedded computing	Often requires special support for graphics or video (or other application-specific extension); power limitations and power control may be required; real-time constraints (Chapters 2, 3, 5, and 7; Appendices A and E)
Level of software compatibility	*Determines amount of existing software for computer*
At programming language	Most flexible for designer; need new compiler (Chapters 3, 5, and 7; Appendix A)
Object code or binary compatible	Instruction set architecture is completely defined—little flexibility—but no investment needed in software or porting programs (Appendix A)
Operating system requirements	*Necessary features to support chosen OS* (Chapter 2; Appendix B)
Size of address space	Very important feature (Chapter 2); may limit applications
Memory management	Required for modern OS; may be paged or segmented (Chapter 2)
Protection	Different OS and application needs: page versus segment; virtual machines (Chapter 2)
Standards	*Certain standards may be required by marketplace*
Floating point	Format and arithmetic: IEEE 754 standard (Appendix J), special arithmetic for graphics or signal processing
I/O interfaces	For I/O devices: Serial ATA, Serial Attached SCSI, PCI Express (Appendices D and F)
Operating systems	UNIX, Windows, Linux, CISCO IOS
Networks	Support required for different networks: Ethernet, Infiniband (Appendix F)
Programming languages	Languages (ANSI C, C++, Java, Fortran) affect instruction set (Appendix A)

Figure 1.8 Summary of some of the most important functional requirements an architect faces. The left-hand column describes the class of requirement, while the right-hand column gives specific examples. The right-hand column also contains references to chapters and appendices that deal with the specific issues.

Architects must also be aware of important trends in both the technology and the use of computers because such trends affect not only the future cost but also the longevity of an architecture.

1.4 Trends in Technology

If an instruction set architecture is to prevail, it must be designed to survive rapid changes in computer technology. After all, a successful new instruction set

architecture may last decades—for example, the core of the IBM mainframe has been in use for more than 50 years. An architect must plan for technology changes that can increase the lifetime of a successful computer.

To plan for the evolution of a computer, the designer must be aware of rapid changes in implementation technology. Five implementation technologies, which change at a dramatic pace, are critical to modern implementations:

- *Integrated circuit logic technology*—Historically, transistor density increased by about 35% per year, quadrupling somewhat over four years. Increases in die size are less predictable and slower, ranging from 10% to 20% per year. The combined effect was a traditional growth rate in transistor count on a chip of about 40%–55% per year, or doubling every 18–24 months. This trend is popularly known as Moore's Law. Device speed scales more slowly, as we discuss below. Shockingly, Moore's Law is no more. The number of devices per chip is still increasing, but at a decelerating rate. Unlike in the Moore's Law era, we expect the doubling time to be stretched with each new technology generation.
- *Semiconductor DRAM* (dynamic random-access memory)—This technology is the foundation of main memory, and we discuss it in Chapter 2. The growth of DRAM has slowed dramatically, from quadrupling every three years as in the past. The 8-gigabit DRAM was shipping in 2014, but the 16-gigabit DRAM won't reach that state until 2019, and it looks like there will be no 32-gigabit DRAM (Kim, 2005). Chapter 2 mentions several other technologies that may replace DRAM when it hits its capacity wall.
- *Semiconductor Flash* (electrically erasable programmable read-only memory)—This nonvolatile semiconductor memory is the standard storage device in PMDs, and its rapidly increasing popularity has fueled its rapid growth rate in capacity. In recent years, the capacity per Flash chip increased by about 50%–60% per year, doubling roughly every 2 years. Currently, Flash memory is 8–10 times cheaper per bit than DRAM. Chapter 2 describes Flash memory.
- *Magnetic disk technology*—Prior to 1990, density increased by about 30% per year, doubling in three years. It rose to 60% per year thereafter, and increased to 100% per year in 1996. Between 2004 and 2011, it dropped back to about 40% per year, or doubled every two years. Recently, disk improvement has slowed to less than 5% per year. One way to increase disk capacity is to add more platters at the same areal density, but there are already seven platters within the one-inch depth of the 3.5-inch form factor disks. There is room for at most one or two more platters. The last hope for real density increase is to use a small laser on each disk read-write head to heat a 30 nm spot to 400°C so that it can be written magnetically before it cools. It is unclear whether Heat Assisted Magnetic Recording can be manufactured economically and reliably, although Seagate announced plans to ship HAMR in limited production in 2018. HAMR is the last chance for continued improvement in areal density of hard disk

drives, which are now 8–10 times cheaper per bit than Flash and 200–300 times cheaper per bit than DRAM. This technology is central to server- and warehouse-scale storage, and we discuss the trends in detail in Appendix D.

- *Network technology*—Network performance depends both on the performance of switches and on the performance of the transmission system. We discuss the trends in networking in Appendix F.

These rapidly changing technologies shape the design of a computer that, with speed and technology enhancements, may have a lifetime of 3–5 years. Key technologies such as Flash change sufficiently that the designer must plan for these changes. Indeed, designers often design for the next technology, knowing that, when a product begins shipping in volume, the following technology may be the most cost-effective or may have performance advantages. Traditionally, cost has decreased at about the rate at which density increases.

Although technology improves continuously, the impact of these increases can be in discrete leaps, as a threshold that allows a new capability is reached. For example, when MOS technology reached a point in the early 1980s where between 25,000 and 50,000 transistors could fit on a single chip, it became possible to build a single-chip, 32-bit microprocessor. By the late 1980s, first-level caches could go on a chip. By eliminating chip crossings within the processor and between the processor and the cache, a dramatic improvement in cost-performance and energy-performance was possible. This design was simply unfeasible until the technology reached a certain point. With multicore microprocessors and increasing numbers of cores each generation, even server computers are increasingly headed toward a single chip for all processors. Such technology thresholds are not rare and have a significant impact on a wide variety of design decisions.

Performance Trends: Bandwidth Over Latency

As we shall see in Section 1.8, *bandwidth* or *throughput* is the total amount of work done in a given time, such as megabytes per second for a disk transfer. In contrast, *latency* or *response time* is the time between the start and the completion of an event, such as milliseconds for a disk access. Figure 1.9 plots the relative improvement in bandwidth and latency for technology milestones for microprocessors, memory, networks, and disks. Figure 1.10 describes the examples and milestones in more detail.

Performance is the primary differentiator for microprocessors and networks, so they have seen the greatest gains: 32,000–40,000 × in bandwidth and 50–90 × in latency. Capacity is generally more important than performance for memory and disks, so capacity has improved more, yet bandwidth advances of 400–2400 × are still much greater than gains in latency of 8–9 ×.

Clearly, bandwidth has outpaced latency across these technologies and will likely continue to do so. A simple rule of thumb is that bandwidth grows by at least the square of the improvement in latency. Computer designers should plan accordingly.

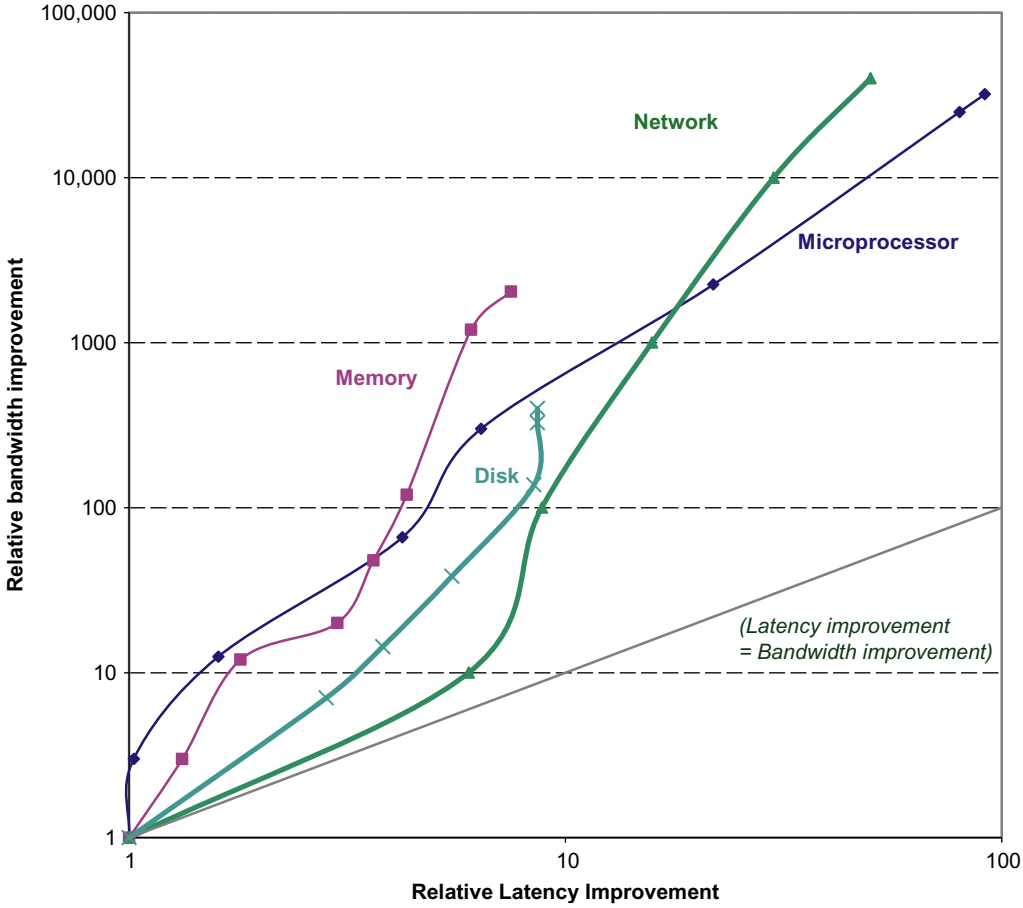


Figure 1.9 Log-log plot of bandwidth and latency milestones in Figure 1.10 relative to the first milestone. Note that latency improved 8–91 ×, while bandwidth improved about 400–32,000 ×. Except for networking, we note that there were modest improvements in latency and bandwidth in the other three technologies in the six years since the last edition: 0%–23% in latency and 23%–70% in bandwidth. Updated from Patterson, D., 2004. Latency lags bandwidth. Commun. ACM 47 (10), 71–75.

Scaling of Transistor Performance and Wires

Integrated circuit processes are characterized by the *feature size*, which is the minimum size of a transistor or a wire in either the x or y dimension. Feature sizes decreased from 10 μm in 1971 to 0.016 μm in 2017; in fact, we have switched units, so production in 2017 is referred to as "16 nm," and 7 nm chips are underway. Since the transistor count per square millimeter of silicon is determined by the surface area of a transistor, the density of transistors increases quadratically with a linear decrease in feature size.

Microprocessor	16-Bit address/ bus, microcoded	32-Bit address/ bus, microcoded	5-Stage pipeline, on-chip I & D caches, FPU	2-Way superscalar, 64-bit bus	Out-of-order 3-way superscalar	Out-of-order superpipelined, on-chip L2 cache	Multicore OOO 4-way on chip L3 cache, Turbo
Product	Intel 80286	Intel 80386	Intel 80486	Intel Pentium	Intel Pentium Pro	Intel Pentium 4	Intel Core i7
Year	1982	1985	1989	1993	1997	2001	2015
Die size (mm^2)	47	43	81	90	308	217	122
Transistors	134,000	275,000	1,200,000	3,100,000	5,500,000	42,000,000	1,750,000,000
Processors/chip	1	1	1	1	1	1	4
Pins	68	132	168	273	387	423	1400
Latency (clocks)	6	5	5	5	10	22	14
Bus width (bits)	16	32	32	64	64	64	196
Clock rate (MHz)	12.5	16	25	66	200	1500	4000
Bandwidth (MIPS)	2	6	25	132	600	4500	64,000
Latency (ns)	320	313	200	76	50	15	4
Memory module	DRAM	Page mode DRAM	Fast page mode DRAM	Fast page mode DRAM	Synchronous DRAM	Double data rate SDRAM	DDR4 SDRAM
Module width (bits)	16	16	32	64	64	64	64
Year	1980	1983	1986	1993	1997	2000	2016
Mbits/DRAM chip	0.06	0.25	1	16	64	256	4096
Die size (mm^2)	35	45	70	130	170	204	50
Pins/DRAM chip	16	16	18	20	54	66	134
Bandwidth (MBytes/s)	13	40	160	267	640	1600	27,000
Latency (ns)	225	170	125	75	62	52	30
Local area network	Ethernet	Fast Ethernet	Gigabit Ethernet	10 Gigabit Ethernet	100 Gigabit Ethernet	400 Gigabit Ethernet	
IEEE standard	802.3	803.3u	802.3ab	802.3ac	802.3ba	802.3bs	
Year	1978	1995	1999	2003	2010	2017	
Bandwidth (Mbits/seconds)	10	100	1000	10,000	100,000	400,000	
Latency (μs)	3000	500	340	190	100	60	
Hard disk	3600 RPM	5400 RPM	7200 RPM	10,000 RPM	15,000 RPM	15,000 RPM	
Product	CDC WrenI 94145-36	Seagate ST41600	Seagate ST15150	Seagate ST39102	Seagate ST373453	Seagate ST600MX0062	
Year	1983	1990	1994	1998	2003	2016	
Capacity (GB)	0.03	1.4	4.3	9.1	73.4	600	
Disk form factor	5.25 in.	5.25 in.	3.5 in.	3.5 in.	3.5 in.	3.5 in.	
Media diameter	5.25 in.	5.25 in.	3.5 in.	3.0 in.	2.5 in.	2.5 in.	
Interface	ST-412	SCSI	SCSI	SCSI	SCSI	SAS	
Bandwidth (MBytes/s)	0.6	4	9	24	86	250	
Latency (ms)	48.3	17.1	12.7	8.8	5.7	3.6	

Figure 1.10 Performance milestones over 25–40 years for microprocessors, memory, networks, and disks. The microprocessor milestones are several generations of IA-32 processors, going from a 16-bit bus, microcoded 80286 to a 64-bit bus, multicore, out-of-order execution, superpipelined Core i7. Memory module milestones go from 16-bit-wide, plain DRAM to 64-bit-wide double data rate version 3 synchronous DRAM. Ethernet advanced from 10 Mbits/s to 400 Gbits/s. Disk milestones are based on rotation speed, improving from 3600 to 15,000 RPM. Each case is best-case bandwidth, and latency is the time for a simple operation assuming no contention. Updated from Patterson, D., 2004. Latency lags bandwidth. Commun. ACM 47 (10), 71–75.

The increase in transistor performance, however, is more complex. As feature sizes shrink, devices shrink quadratically in the horizontal dimension and also shrink in the vertical dimension. The shrink in the vertical dimension requires a reduction in operating voltage to maintain correct operation and reliability of the transistors. This combination of scaling factors leads to a complex interrelationship between transistor performance and process feature size. To a first approximation, in the past the transistor performance improved linearly with decreasing feature size.

The fact that transistor count improves quadratically with a linear increase in transistor performance is both the challenge and the opportunity for which computer architects were created! In the early days of microprocessors, the higher rate of improvement in density was used to move quickly from 4-bit, to 8-bit, to 16-bit, to 32-bit, to 64-bit microprocessors. More recently, density improvements have supported the introduction of multiple processors per chip, wider SIMD units, and many of the innovations in speculative execution and caches found in Chapters 2–5.

Although transistors generally improve in performance with decreased feature size, wires in an integrated circuit do not. In particular, the signal delay for a wire increases in proportion to the product of its resistance and capacitance. Of course, as feature size shrinks, wires get shorter, but the resistance and capacitance per unit length get worse. This relationship is complex, since both resistance and capacitance depend on detailed aspects of the process, the geometry of a wire, the loading on a wire, and even the adjacency to other structures. There are occasional process enhancements, such as the introduction of copper, which provide one-time improvements in wire delay.

In general, however, wire delay scales poorly compared to transistor performance, creating additional challenges for the designer. In addition to the power dissipation limit, wire delay has become a major design obstacle for large integrated circuits and is often more critical than transistor switching delay. Larger and larger fractions of the clock cycle have been consumed by the propagation delay of signals on wires, but power now plays an even greater role than wire delay.

1.5 Trends in Power and Energy in Integrated Circuits

Today, energy is the biggest challenge facing the computer designer for nearly every class of computer. First, power must be brought in and distributed around the chip, and modern microprocessors use hundreds of pins and multiple interconnect layers just for power and ground. Second, power is dissipated as heat and must be removed.

Power and Energy: A Systems Perspective

How should a system architect or a user think about performance, power, and energy? From the viewpoint of a system designer, there are three primary concerns.

First, what is the maximum power a processor ever requires? Meeting this demand can be important to ensuring correct operation. For example, if a processor

attempts to draw more power than a power-supply system can provide (by drawing more current than the system can supply), the result is typically a voltage drop, which can cause devices to malfunction. Modern processors can vary widely in power consumption with high peak currents; hence they provide voltage indexing methods that allow the processor to slow down and regulate voltage within a wider margin. Obviously, doing so decreases performance.

Second, what is the sustained power consumption? This metric is widely called the *thermal design power* (TDP) because it determines the cooling requirement. TDP is neither peak power, which is often 1.5 times higher, nor is it the actual average power that will be consumed during a given computation, which is likely to be lower still. A typical power supply for a system is typically sized to exceed the TDP, and a cooling system is usually designed to match or exceed TDP. Failure to provide adequate cooling will allow the junction temperature in the processor to exceed its maximum value, resulting in device failure and possibly permanent damage. Modern processors provide two features to assist in managing heat, since the highest power (and hence heat and temperature rise) can exceed the long-term average specified by the TDP. First, as the thermal temperature approaches the junction temperature limit, circuitry lowers the clock rate, thereby reducing power. Should this technique not be successful, a second thermal overload trap is activated to power down the chip.

The third factor that designers and users need to consider is energy and energy efficiency. Recall that power is simply energy per unit time: 1 watt = 1 joule per second. Which metric is the right one for comparing processors: energy or power? In general, energy is always a better metric because it is tied to a specific task and the time required for that task. In particular, the energy to complete a workload is equal to the average power times the execution time for the workload.

Thus, if we want to know which of two processors is more efficient for a given task, we need to compare energy consumption (not power) for executing the task. For example, processor A may have a 20% higher average power consumption than processor B, but if A executes the task in only 70% of the time needed by B, its energy consumption will be $1.2 \times 0.7 = 0.84$, which is clearly better.

One might argue that in a large server or cloud, it is sufficient to consider the average power, since the workload is often assumed to be infinite, but this is misleading. If our cloud were populated with processor Bs rather than As, then the cloud would do less work for the same amount of energy expended. Using energy to compare the alternatives avoids this pitfall. Whenever we have a fixed workload, whether for a warehouse-size cloud or a smartphone, comparing energy will be the right way to compare computer alternatives, because the electricity bill for the cloud and the battery lifetime for the smartphone are both determined by the energy consumed.

When is power consumption a useful measure? The primary legitimate use is as a constraint: for example, an air-cooled chip might be limited to 100 W. It can be used as a metric if the workload is fixed, but then it's just a variation of the true metric of energy per task.

Energy and Power Within a Microprocessor

For CMOS chips, the traditional primary energy consumption has been in switching transistors, also called *dynamic energy*. The energy required per transistor is proportional to the product of the capacitive load driven by the transistor and the square of the voltage:

$$\text{Energy}_{\text{dynamic}} \propto \text{Capacitive load} \times \text{Voltage}^2$$

This equation is the energy of pulse of the logic transition of $0 \rightarrow 1 \rightarrow 0$ or $1 \rightarrow 0 \rightarrow 1$. The energy of a single transition ($0 \rightarrow 1$ or $1 \rightarrow 0$) is then:

$$\text{Energy}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2$$

The power required per transistor is just the product of the energy of a transition multiplied by the frequency of transitions:

$$\text{Power}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$$

For a fixed task, slowing clock rate reduces power, but not energy.

Clearly, dynamic power and energy are greatly reduced by lowering the voltage, so voltages have dropped from 5 V to just under 1 V in 20 years. The capacitive load is a function of the number of transistors connected to an output and the technology, which determines the capacitance of the wires and the transistors.

Example Some microprocessors today are designed to have adjustable voltage, so a 15% reduction in voltage may result in a 15% reduction in frequency. What would be the impact on dynamic energy and on dynamic power?

Answer Because the capacitance is unchanged, the answer for energy is the ratio of the voltages

$$\frac{\text{Energy}_{\text{new}}}{\text{Energy}_{\text{old}}} = \frac{(\text{Voltage} \times 0.85)^2}{\text{Voltage}^2} = 0.85^2 = 0.72$$

which reduces energy to about 72% of the original. For power, we add the ratio of the frequencies

$$\frac{\text{Power}_{\text{new}}}{\text{Power}_{\text{old}}} = 0.72 \times \frac{(\text{Frequency switched} \times 0.85)}{\text{Frequency switched}} = 0.61$$

shrinking power to about 61% of the original.

As we move from one process to the next, the increase in the number of transistors switching and the frequency with which they change dominate the decrease in load capacitance and voltage, leading to an overall growth in power consumption and energy. The first microprocessors consumed less than a watt, and the first

32-bit microprocessors (such as the Intel 80386) used about 2 W, whereas a 4.0 GHz Intel Core i7-6700K consumes 95 W. Given that this heat must be dissipated from a chip that is about 1.5 cm on a side, we are near the limit of what can be cooled by air, and this is where we have been stuck for nearly a decade.

Given the preceding equation, you would expect clock frequency growth to slow down if we can't reduce voltage or increase power per chip. Figure 1.11 shows that this has indeed been the case since 2003, even for the microprocessors in Figure 1.1 that were the highest performers each year. Note that this period of flatter clock rates corresponds to the period of slow performance improvement range in Figure 1.1.

Distributing the power, removing the heat, and preventing hot spots have become increasingly difficult challenges. Energy is now the major constraint to using transistors; in the past, it was the raw silicon area. Therefore modern

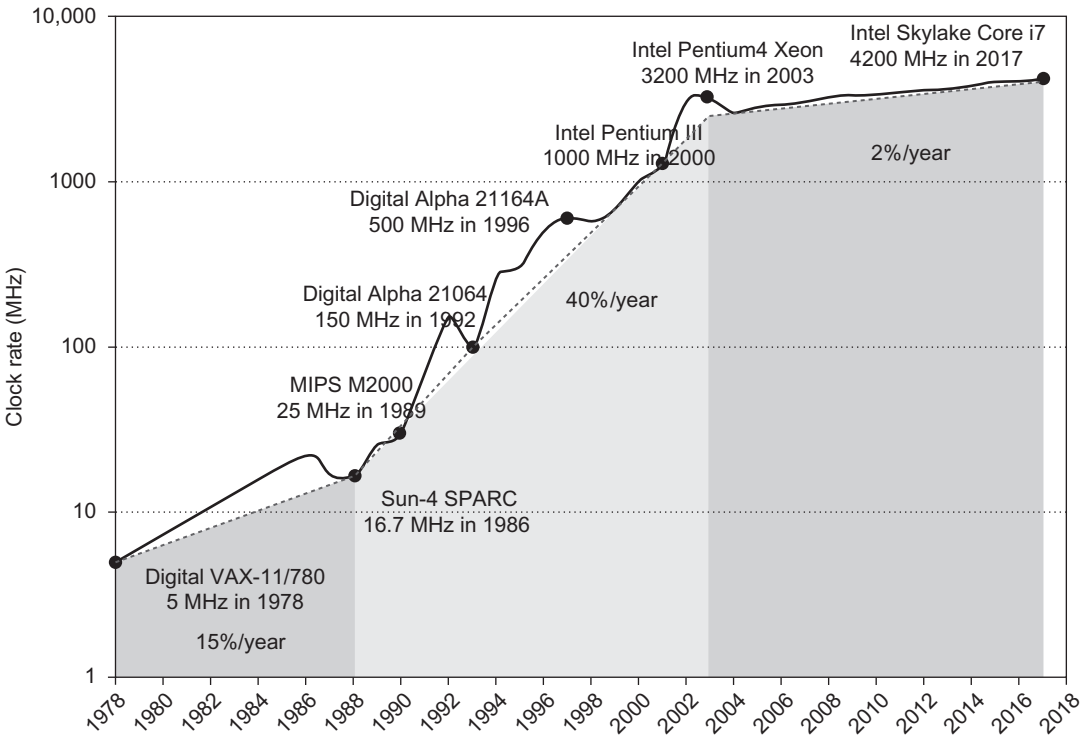


Figure 1.11 Growth in clock rate of microprocessors in Figure 1.1. Between 1978 and 1986, the clock rate improved less than 15% per year while performance improved by 22% per year. During the "renaissance period" of 52% performance improvement per year between 1986 and 2003, clock rates shot up almost 40% per year. Since then, the clock rate has been nearly flat, growing at less than 2% per year, while single processor performance improved recently at just 3.5% per year.

microprocessors offer many techniques to try to improve energy efficiency despite flat clock rates and constant supply voltages:

1. *Do nothing well.* Most microprocessors today turn off the clock of inactive modules to save energy and dynamic power. For example, if no floating-point instructions are executing, the clock of the floating-point unit is disabled. If some cores are idle, their clocks are stopped.
2. *Dynamic voltage-frequency scaling (DVFS).* The second technique comes directly from the preceding formulas. PMDs, laptops, and even servers have periods of low activity where there is no need to operate at the highest clock frequency and voltages. Modern microprocessors typically offer a few clock frequencies and voltages in which to operate that use lower power and energy. Figure 1.12 plots the potential power savings via DVFS for a server as the workload shrinks for three different clock rates: 2.4, 1.8, and 1 GHz. The overall server power savings is about 10%–15% for each of the two steps.
3. *Design for the typical case.* Given that PMDs and laptops are often idle, memory and storage offer low power modes to save energy. For example, DRAMs have a series of increasingly lower power modes to extend battery life in PMDs and laptops, and there have been proposals for disks that have a mode that spins more slowly when unused to save power. However, you cannot access DRAMs or disks in these modes, so you must return to fully active mode to read or write, no matter how low the access rate. As mentioned, microprocessors for PCs have been designed instead for heavy use at high operating temperatures, relying on on-chip temperature sensors to detect when activity should be reduced automatically to avoid overheating. This "emergency slowdown" allows manufacturers to design for a more typical case and then rely on this safety mechanism if someone really does run programs that consume much more power than is typical.

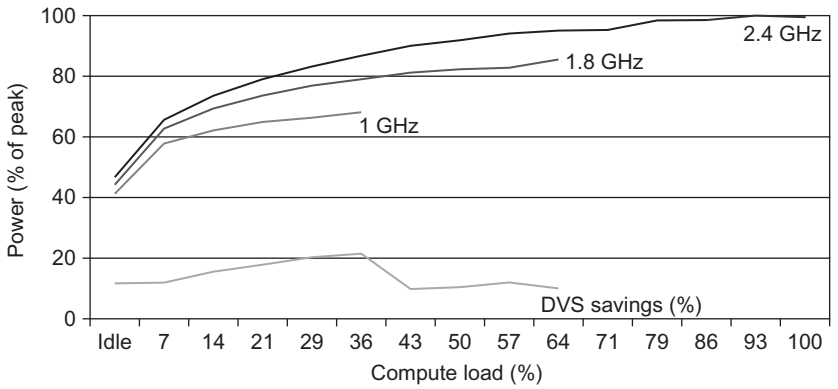


Figure 1.12 Energy savings for a server using an AMD Opteron microprocessor, 8 GB of DRAM, and one ATA disk. At 1.8 GHz, the server can handle at most up to two-thirds of the workload without causing service-level violations, and at 1 GHz, it can safely handle only one-third of the workload (Figure 5.11 in Barroso and Hölzle, 2009).

4. *Overclocking*. Intel started offering *Turbo mode* in 2008, where the chip decides that it is safe to run at a higher clock rate for a short time, possibly on just a few cores, until temperature starts to rise. For example, the 3.3 GHz Core i7 can run in short bursts for 3.6 GHz. Indeed, the highest-performing microprocessors each year since 2008 shown in Figure 1.1 have all offered temporary overclocking of about 10% over the nominal clock rate. For single-threaded code, these microprocessors can turn off all cores but one and run it faster. Note that, although the operating system can turn off Turbo mode, there is no notification once it is enabled, so the programmers may be surprised to see their programs vary in performance because of room temperature!

Although dynamic power is traditionally thought of as the primary source of power dissipation in CMOS, static power is becoming an important issue because leakage current flows even when a transistor is off:

$$\text{Power}_{\text{static}} \propto \text{Current}_{\text{static}} \times \text{Voltage}$$

That is, static power is proportional to the number of devices.

Thus increasing the number of transistors increases power even if they are idle, and current leakage increases in processors with smaller transistor sizes. As a result, very low-power systems are even turning off the power supply (*power gating*) to inactive modules in order to control loss because of leakage. In 2011 the goal for leakage was 25% of the total power consumption, with leakage in high-performance designs sometimes far exceeding that goal. Leakage can be as high as 50% for such chips, in part because of the large SRAM caches that need power to maintain the storage values. (The S in SRAM is for static.) The only hope to stop leakage is to turn off power to the chips' subsets.

Finally, because the processor is just a portion of the whole energy cost of a system, it can make sense to use a faster, less energy-efficient processor to allow the rest of the system to go into a sleep mode. This strategy is known as *race-to-halt*.

The importance of power and energy has increased the scrutiny on the efficiency of an innovation, so the primary evaluation now is tasks per joule or performance per watt, contrary to performance per mm^2 of silicon as in the past. This new metric affects approaches to parallelism, as we will see in Chapters 4 and 5.

The Shift in Computer Architecture Because of Limits of Energy

As transistor improvement decelerates, computer architects must look elsewhere for improved energy efficiency. Indeed, given the energy budget, it is easy today to design a microprocessor with so many transistors that they cannot all be turned on at the same time. This phenomenon has been called *dark silicon*, in that much of a chip cannot be unused ("dark") at any moment in time because of thermal constraints. This observation has led architects to reexamine the fundamentals of processors' design in the search for a greater energy-cost performance.

Figure 1.13, which lists the energy cost and area cost of the building blocks of a modern computer, reveals surprisingly large ratios. For example, a 32-bit

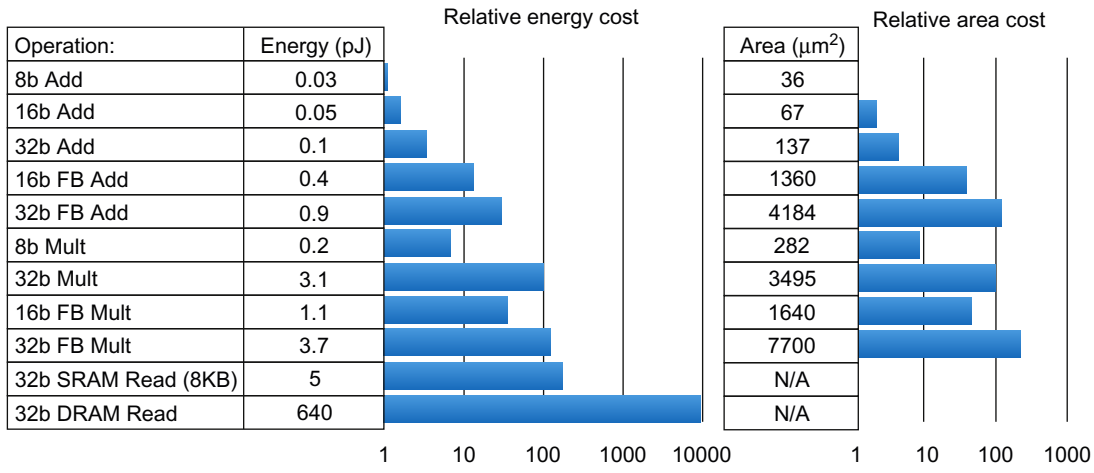


Figure 1.13 Comparison of the energy and die area of arithmetic operations and energy cost of accesses to SRAM and DRAM. [Azizi][Dally]. Area is for TSMC 45 nm technology node.

floating-point addition uses 30 times as much energy as an 8-bit integer add. The area difference is even larger, by 60 times. However, the biggest difference is in memory; a 32-bit DRAM access takes 20,000 times as much energy as an 8-bit addition. A small SRAM is 125 times more energy-efficient than DRAM, which demonstrates the importance of careful uses of caches and memory buffers.

The new design principle of minimizing energy per task combined with the relative energy and area costs in Figure 1.13 have inspired a new direction for computer architecture, which we describe in Chapter 7. Domain-specific processors save energy by reducing wide floating-point operations and deploying special-purpose memories to reduce accesses to DRAM. They use those saving to provide 10–100 more (narrower) integer arithmetic units than a traditional processor. Although such processors perform only a limited set of tasks, they perform them remarkably faster and more energy efficiently than a general-purpose processor.

Like a hospital with general practitioners and medical specialists, computers in this energy-aware world will likely be combinations of general-purpose cores that can perform any task and special-purpose cores that do a few things extremely well and even more cheaply.

1.6 Trends in Cost

Although costs tend to be less important in some computer designs—specifically supercomputers—cost-sensitive designs are of growing significance. Indeed, in the past 35 years, the use of technology improvements to lower cost, as well as increase performance, has been a major theme in the computer industry.

Textbooks often ignore the cost half of cost-performance because costs change, thereby dating books, and because the issues are subtle and differ across industry segments. Nevertheless, it's essential for computer architects to have an understanding of cost and its factors in order to make intelligent decisions about whether a new feature should be included in designs where cost is an issue. (Imagine architects designing skyscrapers without any information on costs of steel beams and concrete!)

This section discusses the major factors that influence the cost of a computer and how these factors are changing over time.

The Impact of Time, Volume, and Commoditization

The cost of a manufactured computer component decreases over time even without significant improvements in the basic implementation technology. The underlying principle that drives costs down is the *learning curve*—manufacturing costs decrease over time. The learning curve itself is best measured by change in *yield*—the percentage of manufactured devices that survives the testing procedure. Whether it is a chip, a board, or a system, designs that have twice the yield will have half the cost.

Understanding how the learning curve improves yield is critical to projecting costs over a product's life. One example is that the price per megabyte of DRAM has dropped over the long term. Since DRAMs tend to be priced in close relationship to cost—except for periods when there is a shortage or an oversupply—price and cost of DRAM track closely.

Microprocessor prices also drop over time, but because they are less standardized than DRAMs, the relationship between price and cost is more complex. In a period of significant competition, price tends to track cost closely, although microprocessor vendors probably rarely sell at a loss.

Volume is a second key factor in determining cost. Increasing volumes affect cost in several ways. First, they decrease the time needed to get through the learning curve, which is partly proportional to the number of systems (or chips) manufactured. Second, volume decreases cost because it increases purchasing and manufacturing efficiency. As a rule of thumb, some designers have estimated that costs decrease about 10% for each doubling of volume. Moreover, volume decreases the amount of development costs that must be amortized by each computer, thus allowing cost and selling price to be closer and still make a profit.

Commodities are products that are sold by multiple vendors in large volumes and are essentially identical. Virtually all the products sold on the shelves of grocery stores are commodities, as are standard DRAMs, Flash memory, monitors, and keyboards. In the past 30 years, much of the personal computer industry has become a commodity business focused on building desktop and laptop computers running Microsoft Windows.

Because many vendors ship virtually identical products, the market is highly competitive. Of course, this competition decreases the gap between cost and selling

price, but it also decreases cost. Reductions occur because a commodity market has both volume and a clear product definition, which allows multiple suppliers to compete in building components for the commodity product. As a result, the overall product cost is lower because of the competition among the suppliers of the components and the volume efficiencies the suppliers can achieve. This rivalry has led to the low end of the computer business being able to achieve better price-performance than other sectors and has yielded greater growth at the low end, although with very limited profits (as is typical in any commodity business).

Cost of an Integrated Circuit

Why would a computer architecture book have a section on integrated circuit costs? In an increasingly competitive computer marketplace where standard parts—disks, Flash memory, DRAMs, and so on—are becoming a significant portion of any system's cost, integrated circuit costs are becoming a greater portion of the cost that varies between computers, especially in the high-volume, cost-sensitive portion of the market. Indeed, with PMDs' increasing reliance of whole *systems on a chip* (SOC), the cost of the integrated circuits is much of the cost of the PMD. Thus computer designers must understand the costs of chips in order to understand the costs of current computers.

Although the costs of integrated circuits have dropped exponentially, the basic process of silicon manufacture is unchanged: A *wafer* is still tested and chopped into *dies* that are packaged (see Figures 1.14–1.16). Therefore the cost of a packaged integrated circuit is

$$\text{Cost of integrated circuit} = \frac{\text{Cost of die} + \text{Cost of testing die} + \text{Cost of packaging and final test}}{\text{Final test yield}}$$

In this section, we focus on the cost of dies, summarizing the key issues in testing and packaging at the end.

Learning how to predict the number of good chips per wafer requires first learning how many dies fit on a wafer and then learning how to predict the percentage of those that will work. From there it is simple to predict cost:

$$\text{Cost of die} = \frac{\text{Cost of wafer}}{\text{Dies per wafer} \times \text{Die yield}}$$

The most interesting feature of this initial term of the chip cost equation is its sensitivity to die size, shown below.

The number of dies per wafer is approximately the area of the wafer divided by the area of the die. It can be more accurately estimated by

$$\text{Dies per wafer} = \frac{\pi \times (\text{Wafer diameter}/2)^2}{\text{Die area}} - \frac{\pi \times \text{Wafer diameter}}{\sqrt{2 \times \text{Die area}}}$$

The first term is the ratio of wafer area (πr^2) to die area. The second compensates for the "square peg in a round hole" problem—rectangular dies near the periphery

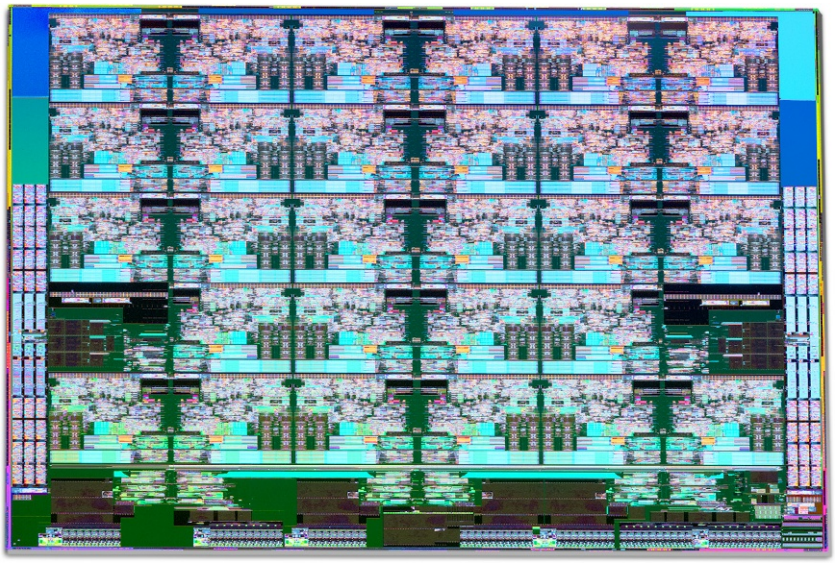


Figure 1.14 Photograph of an Intel Skylake microprocessor die, which is evaluated in Chapter 4.

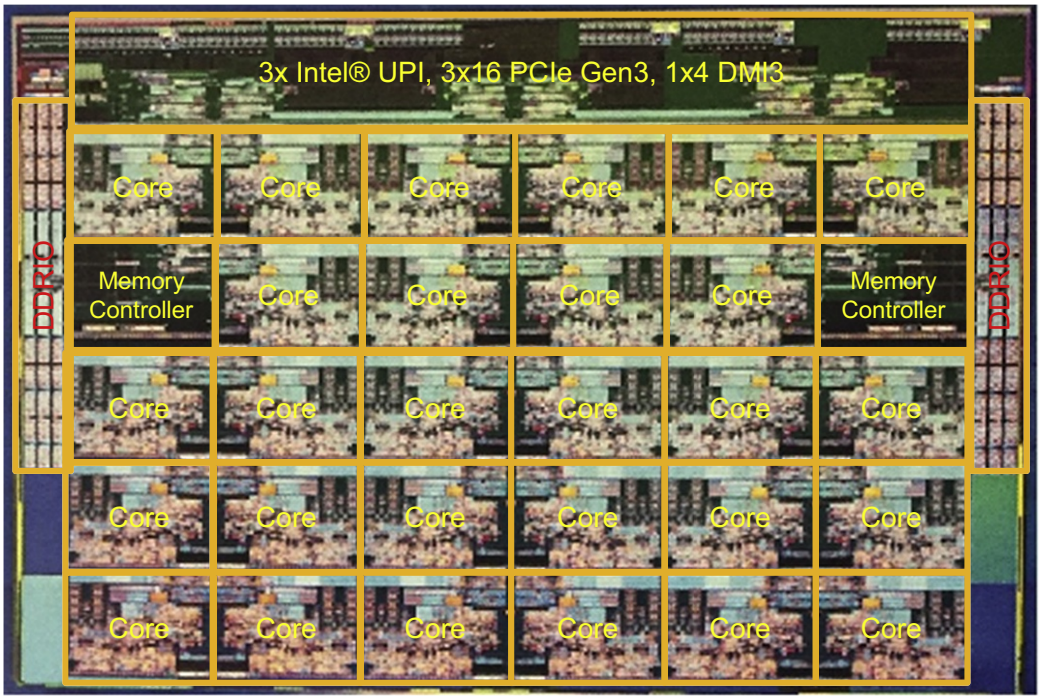


Figure 1.15 The components of the microprocessor die in Figure 1.14 are labeled with their functions.

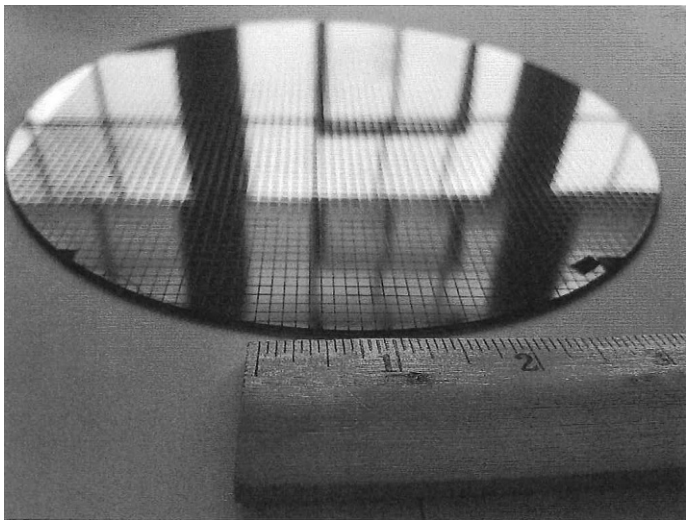


Figure 1.16 This 200 mm diameter wafer of RISC-V dies was designed by SiFive. It has two types of RISC-V dies using an older, larger processing line. An FE310 die is 2.65 mm × 2.72 mm and an SiFive test die that is 2.89 mm × 2.72 mm. The wafer contains 1846 of the former and 1866 of the latter, totaling 3712 chips.

of round wafers. Dividing the circumference (πd) by the diagonal of a square die is approximately the number of dies along the edge.

Example Find the number of dies per 300 mm (30 cm) wafer for a die that is 1.5 cm on a side and for a die that is 1.0 cm on a side.

Answer When die area is 2.25 cm^2:

$$\text{Dies per wafer} = \frac{\pi \times (30/2)^2}{2.25} - \frac{\pi \times 30}{\sqrt{2 \times 2.25}} = \frac{706.9}{2.25} - \frac{94.2}{2.12} = 270$$

Because the area of the larger die is 2.25 times bigger, there are roughly 2.25 as many smaller dies per wafer:

$$\text{Dies per wafer} = \frac{\pi \times (30/2)^2}{1.00} - \frac{\pi \times 30}{\sqrt{2 \times 1.00}} = \frac{706.9}{1.00} - \frac{94.2}{1.41} = 640$$

However, this formula gives only the maximum number of dies per wafer. The critical question is: What is the fraction of *good* dies on a wafer, or the *die yield*? A simple model of integrated circuit yield, which assumes that defects are randomly

distributed over the wafer and that yield is inversely proportional to the complexity of the fabrication process, leads to the following:

$$\text{Die yield} = \text{Wafer yield} \times 1/(1 + \text{Defects per unit area} \times \text{Die area})^N$$

This Bose-Einstein formula is an empirical model developed by looking at the yield of many manufacturing lines (Sydow, 2006), and it still applies today. *Wafer yield* accounts for wafers that are completely bad and so need not be tested. For simplicity, we'll just assume the wafer yield is 100%. Defects per unit area is a measure of the random manufacturing defects that occur. In 2017 the value was typically 0.08–0.10 defects per square inch for a 28-nm node and 0.10–0.30 for the newer 16 nm node because it depends on the maturity of the process (recall the learning curve mentioned earlier). The metric versions are 0.012–0.016 defects per square centimeter for 28 nm and 0.016–0.047 for 16 nm. Finally, N is a parameter called the process-complexity factor, a measure of manufacturing difficulty. For 28 nm processes in 2017, N is 7.5–9.5. For a 16 nm process, N ranges from 10 to 14.

Example Find the die yield for dies that are 1.5 cm on a side and 1.0 cm on a side, assuming a defect density of 0.047 per cm^2 and N is 12.

Answer The total die areas are 2.25 and 1.00 cm^2. For the larger die, the yield is

$$\text{Die yield} = 1/(1 + 0.047 \times 2.25)^{12} \times 270 = 120$$

For the smaller die, the yield is

$$\text{Die yield} = 1/(1 + 0.047 \times 1.00)^{12} \times 640 = 444$$

The bottom line is the number of good dies per wafer. Less than half of all the large dies are good, but nearly 70% of the small dies are good.

Although many microprocessors fall between 1.00 and 2.25 cm^2, low-end embedded 32-bit processors are sometimes as small as 0.05 cm^2, processors used for embedded control (for inexpensive IoT devices) are often less than 0.01 cm^2, and high-end server and GPU chips can be as large as 8 cm^2.

Given the tremendous price pressures on commodity products such as DRAM and SRAM, designers have included redundancy as a way to raise yield. For a number of years, DRAMs have regularly included some redundant memory cells so that a certain number of flaws can be accommodated. Designers have used similar techniques in both standard SRAMs and in large SRAM arrays used for caches within microprocessors. GPUs have 4 redundant processors out of 84 for the same reason. Obviously, the presence of redundant entries can be used to boost the yield significantly.

In 2017 processing of a 300 mm (12-inch) diameter wafer in a 28-nm technology costs between $4000 and $5000, and a 16-nm wafer costs about $7000. Assuming a processed wafer cost of $7000, the cost of the 1.00 cm^2 die would be around $16, but the cost per die of the 2.25 cm^2 die would be about $58, or almost four times the cost of a die that is a little over twice as large.

What should a computer designer remember about chip costs? The manufacturing process dictates the wafer cost, wafer yield, and defects per unit area, so the sole control of the designer is die area. In practice, because the number of defects per unit area is small, the number of good dies per wafer, and therefore the cost per die, grows roughly as the square of the die area. The computer designer affects die size, and thus cost, both by what functions are included on or excluded from the die and by the number of I/O pins.

Before we have a part that is ready for use in a computer, the die must be tested (to separate the good dies from the bad), packaged, and tested again after packaging. These steps all add significant costs, increasing the total by half.

The preceding analysis focused on the variable costs of producing a functional die, which is appropriate for high-volume integrated circuits. There is, however, one very important part of the fixed costs that can significantly affect the cost of an integrated circuit for low volumes (less than 1 million parts), namely, the cost of a mask set. Each step in the integrated circuit process requires a separate mask. Therefore, for modern high-density fabrication processes with up to 10 metal layers, mask costs are about $4 million for 16 nm and $1.5 million for 28 nm.

The good news is that semiconductor companies offer "shuttle runs" to dramatically lower the costs of tiny test chips. They lower costs by putting many small designs onto a single die to amortize the mask costs, and then later split the dies into smaller pieces for each project. Thus TSMC delivers 80–100 untested dies that are 1.57 × 1.57 mm in a 28 nm process for $30,000 in 2017. Although these die are tiny, they offer the architect millions of transistors to play with. For example, several RISC-V processors would fit on such a die.

Although shuttle runs help with prototyping and debugging runs, they don't address small-volume production of tens to hundreds of thousands of parts. Because mask costs are likely to continue to increase, some designers are incorporating reconfigurable logic to enhance the flexibility of a part and thus reduce the cost implications of masks.

Cost Versus Price

With the commoditization of computers, the margin between the cost to manufacture a product and the price the product sells for has been shrinking. Those margins pay for a company's research and development (R&D), marketing, sales, manufacturing equipment maintenance, building rental, cost of financing, pretax profits, and taxes. Many engineers are surprised to find that most companies spend only 4% (in the commodity PC business) to 12% (in the high-end server business) of their income on R&D, which includes all engineering.

Cost of Manufacturing Versus Cost of Operation

For the first four editions of this book, cost meant the cost to build a computer and price meant price to purchase a computer. With the advent of WSCs, which contain tens of thousands of servers, the cost to operate the computers is significant in addition to the cost of purchase. Economists refer to these two costs as capital expenses (CAPEX) and operational expenses (OPEX).

As Chapter 6 shows, the amortized purchase price of servers and networks is about half of the monthly cost to operate a WSC, assuming a short lifetime of the IT equipment of 3–4 years. About 40% of the monthly operational costs are for power use and the amortized infrastructure to distribute power and to cool the IT equipment, despite this infrastructure being amortized over 10–15 years. Thus, to lower operational costs in a WSC, computer architects need to use energy efficiently.

1.7 Dependability

Historically, integrated circuits were one of the most reliable components of a computer. Although their pins may be vulnerable, and faults may occur over communication channels, the failure rate inside the chip was very low. That conventional wisdom is changing as we head to feature sizes of 16 nm and smaller, because both transient faults and permanent faults are becoming more commonplace, so architects must design systems to cope with these challenges. This section gives a quick overview of the issues in dependability, leaving the official definition of the terms and approaches to Section D.3 in Appendix D.

Computers are designed and constructed at different layers of abstraction. We can descend recursively down through a computer seeing components enlarge themselves to full subsystems until we run into individual transistors. Although some faults are widespread, like the loss of power, many can be limited to a single component in a module. Thus utter failure of a module at one level may be considered merely a component error in a higher-level module. This distinction is helpful in trying to find ways to build dependable computers.

One difficult question is deciding when a system is operating properly. This theoretical point became concrete with the popularity of Internet services. Infrastructure providers started offering *service level agreements* (SLAs) or *service level objectives* (SLOs) to guarantee that their networking or power service would be dependable. For example, they would pay the customer a penalty if they did not meet an agreement of some hours per month. Thus an SLA could be used to decide whether the system was up or down.

Systems alternate between two states of service with respect to an SLA:

1. *Service accomplishment*, where the service is delivered as specified.
2. *Service interruption*, where the delivered service is different from the SLA.

Transitions between these two states are caused by *failures* (from state 1 to state 2) or *restorations* (2 to 1). Quantifying these transitions leads to the two main measures of dependability:

- *Module reliability* is a measure of the continuous service accomplishment (or, equivalently, of the time to failure) from a reference initial instant. Therefore the *mean time to failure* (MTTF) is a reliability measure. The reciprocal of MTTF is a rate of failures, generally reported as failures per billion hours of operation, or *FIT* (for *failures in time*). Thus an MTTF of 1,000,000 hours equals $10^9/10^6$ or 1000 FIT. Service interruption is measured as *mean time to repair* (MTTR). *Mean time between failures* (MTBF) is simply the sum of MTTF + MTTR. Although MTBF is widely used, MTTF is often the more appropriate term. If a collection of modules has exponentially distributed lifetimes—meaning that the age of a module is not important in probability of failure—the overall failure rate of the collection is the sum of the failure rates of the modules.
- *Module availability* is a measure of the service accomplishment with respect to the alternation between the two states of accomplishment and interruption. For nonredundant systems with repair, module availability is

$$\text{Module availability} = \frac{\text{MTTF}}{(\text{MTTF} + \text{MTTR})}$$

Note that reliability and availability are now quantifiable metrics, rather than synonyms for dependability. From these definitions, we can estimate reliability of a system quantitatively if we make some assumptions about the reliability of components and that failures are independent.

Example Assume a disk subsystem with the following components and MTTF:

- 10 disks, each rated at 1,000,000-hour MTTF
- 1 ATA controller, 500,000-hour MTTF
- 1 power supply, 200,000-hour MTTF
- 1 fan, 200,000-hour MTTF
- 1 ATA cable, 1,000,000-hour MTTF

Using the simplifying assumptions that the lifetimes are exponentially distributed and that failures are independent, compute the MTTF of the system as a whole.

Answer The sum of the failure rates is

$$\text{Failure rate}_{\text{system}} = 10 \times \frac{1}{1{,}000{,}000} + \frac{1}{500{,}000} + \frac{1}{200{,}000} + \frac{1}{200{,}000} + \frac{1}{1{,}000{,}000}$$

$$= \frac{10 + 2 + 5 + 5 + 1}{1{,}000{,}000 \text{ hours}} = \frac{23}{1{,}000{,}000} = \frac{23{,}000}{1{,}000{,}000{,}000 \text{ hours}}$$

or 23,000 FIT. The MTTF for the system is just the inverse of the failure rate

$$\text{MTTF}_{\text{system}} = \frac{1}{\text{Failure rate}_{\text{system}}} = \frac{1{,}000{,}000{,}000 \text{ hours}}{23{,}000} = 43{,}500 \text{ hours}$$

or just under 5 years.

The primary way to cope with failure is redundancy, either in time (repeat the operation to see if it still is erroneous) or in resources (have other components to take over from the one that failed). Once the component is replaced and the system is fully repaired, the dependability of the system is assumed to be as good as new. Let's quantify the benefits of redundancy with an example.

Example Disk subsystems often have redundant power supplies to improve dependability. Using the preceding components and MTTFs, calculate the reliability of redundant power supplies. Assume that one power supply is sufficient to run the disk subsystem and that we are adding one redundant power supply.

Answer We need a formula to show what to expect when we can tolerate a failure and still provide service. To simplify the calculations, we assume that the lifetimes of the components are exponentially distributed and that there is no dependency between the component failures. MTTF for our redundant power supplies is the mean time until one power supply fails divided by the chance that the other will fail before the first one is replaced. Thus, if the chance of a second failure before repair is small, then the MTTF of the pair is large.

Since we have two power supplies and independent failures, the mean time until one supply fails is $\text{MTTF}_{\text{power supply}}/2$. A good approximation of the probability of a second failure is MTTR over the mean time until the other power supply fails. Therefore a reasonable approximation for a redundant pair of power supplies is

$$\text{MTTF}_{\text{power supply pair}} = \frac{\text{MTTF}_{\text{power supply}}/2}{\frac{\text{MTTR}_{\text{power supply}}}{\text{MTTF}_{\text{power supply}}}} = \frac{\text{MTTF}^2_{\text{power supply}}/2}{\text{MTTR}_{\text{power supply}}} = \frac{\text{MTTF}^2_{\text{power supply}}}{2 \times \text{MTTR}_{\text{power supply}}}$$

Using the preceding MTTF numbers, if we assume it takes on average 24 hours for a human operator to notice that a power supply has failed and to replace it, the reliability of the fault tolerant pair of power supplies is

$$\text{MTTF}_{\text{power supply pair}} = \frac{\text{MTTF}^2_{\text{power supply}}}{2 \times \text{MTTR}_{\text{power supply}}} = \frac{200{,}000^2}{2 \times 24} \cong 830{,}000{,}000$$

making the pair about 4150 times more reliable than a single power supply.

Having quantified the cost, power, and dependability of computer technology, we are ready to quantify performance.

1.8 Measuring, Reporting, and Summarizing Performance

When we say one computer is faster than another one is, what do we mean? The user of a cell phone may say a computer is faster when a program runs in less time, while an Amazon.com administrator may say a computer is faster when it completes more transactions per hour. The cell phone user wants to reduce *response time*—the time between the start and the completion of an event—also referred to as *execution time*. The operator of a WSC wants to increase *throughput*—the total amount of work done in a given time.

In comparing design alternatives, we often want to relate the performance of two different computers, say, X and Y. The phrase "X is faster than Y" is used here to mean that the response time or execution time is lower on X than on Y for the given task. In particular, "X is n times as fast as Y" will mean

$$\frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

Since execution time is the reciprocal of performance, the following relationship holds:

$$n = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = \frac{\dfrac{1}{\text{Performance}_Y}}{\dfrac{1}{\text{Performance}_X}} = \frac{\text{Performance}_X}{\text{Performance}_Y}$$

The phrase "the throughput of X is 1.3 times as fast as Y" signifies here that the number of tasks completed per unit time on computer X is 1.3 times the number completed on Y.

Unfortunately, time is not always the metric quoted in comparing the performance of computers. Our position is that the only consistent and reliable measure of performance is the execution time of real programs, and that all proposed alternatives to time as the metric or to real programs as the items measured have eventually led to misleading claims or even mistakes in computer design.

Even execution time can be defined in different ways depending on what we count. The most straightforward definition of time is called *wall-clock time*, *response time*, or *elapsed time*, which is the latency to complete a task, including storage accesses, memory accesses, input/output activities, operating system overhead—everything. With multiprogramming, the processor works on another program while waiting for I/O and may not necessarily minimize the elapsed time of one program. Thus we need a term to consider this activity. *CPU time* recognizes this distinction and means the time the processor is computing, *not* including the time waiting for I/O or running other programs. (Clearly, the response time seen by the user is the elapsed time of the program, not the CPU time.)

Computer users who routinely run the same programs would be the perfect candidates to evaluate a new computer. To evaluate a new system, these users would simply compare the execution time of their *workloads*—the mixture of programs

and operating system commands that users run on a computer. Few are in this happy situation, however. Most must rely on other methods to evaluate computers, and often other evaluators, hoping that these methods will predict performance for their usage of the new computer. One approach is benchmark programs, which are programs that many companies use to establish the relative performance of their computers.

Benchmarks

The best choice of benchmarks to measure performance is real applications, such as Google Translate mentioned in Section 1.1. Attempts at running programs that are much simpler than a real application have led to performance pitfalls. Examples include

- *Kernels*, which are small, key pieces of real applications.
- *Toy programs*, which are 100-line programs from beginning programming assignments, such as Quicksort.
- *Synthetic benchmarks*, which are fake programs invented to try to match the profile and behavior of real applications, such as Dhrystone.

All three are discredited today, usually because the compiler writer and architect can conspire to make the computer appear faster on these stand-in programs than on real applications. Regrettably for your authors—who dropped the fallacy about using synthetic benchmarks to characterize performance in the fourth edition of this book since we thought all computer architects agreed it was disreputable—the synthetic program Dhrystone is still the most widely quoted benchmark for embedded processors in 2017!

Another issue is the conditions under which the benchmarks are run. One way to improve the performance of a benchmark has been with benchmark-specific compiler flags; these flags often caused transformations that would be illegal on many programs or would slow down performance on others. To restrict this process and increase the significance of the results, benchmark developers typically require the vendor to use one compiler and one set of flags for all the programs in the same language (such as C++ or C). In addition to the question of compiler flags, another question is whether source code modifications are allowed. There are three different approaches to addressing this question:

1. No source code modifications are allowed.
2. Source code modifications are allowed but are essentially impossible. For example, database benchmarks rely on standard database programs that are tens of millions of lines of code. The database companies are highly unlikely to make changes to enhance the performance for one particular computer.
3. Source modifications are allowed, as long as the altered version produces the same output.

The key issue that benchmark designers face in deciding to allow modification of the source is whether such modifications will reflect real practice and provide useful insight to users, or whether these changes simply reduce the accuracy of the benchmarks as predictors of real performance. As we will see in Chapter 7, domain-specific architects often follow the third option when creating processors for well-defined tasks.

To overcome the danger of placing too many eggs in one basket, collections of benchmark applications, called *benchmark suites*, are a popular measure of performance of processors with a variety of applications. Of course, such collections are only as good as the constituent individual benchmarks. Nonetheless, a key advantage of such suites is that the weakness of any one benchmark is lessened by the presence of the other benchmarks. The goal of a benchmark suite is that it will characterize the real relative performance of two computers, particularly for programs not in the suite that customers are likely to run.

A cautionary example is the Electronic Design News Embedded Microprocessor Benchmark Consortium (or EEMBC, pronounced "embassy") benchmarks.

It is a set of 41 kernels used to predict performance of different embedded applications: automotive/industrial, consumer, networking, office automation, and telecommunications. EEMBC reports unmodified performance and "full fury" performance, where almost anything goes. Because these benchmarks use small kernels, and because of the reporting options, EEMBC does not have the reputation of being a good predictor of relative performance of different embedded computers in the field. This lack of success is why Dhrystone, which EEMBC was trying to replace, is sadly still used.

One of the most successful attempts to create standardized benchmark application suites has been the SPEC (Standard Performance Evaluation Corporation), which had its roots in efforts in the late 1980s to deliver better benchmarks for workstations. Just as the computer industry has evolved over time, so has the need for different benchmark suites, and there are now SPEC benchmarks to cover many application classes. All the SPEC benchmark suites and their reported results are found at http://www.spec.org.

Although we focus our discussion on the SPEC benchmarks in many of the following sections, many benchmarks have also been developed for PCs running the Windows operating system.

Desktop Benchmarks

Desktop benchmarks divide into two broad classes: processor-intensive benchmarks and graphics-intensive benchmarks, although many graphics benchmarks include intensive processor activity. SPEC originally created a benchmark set focusing on processor performance (initially called SPEC89), which has evolved into its sixth generation: SPEC CPU2017, which follows SPEC2006, SPEC2000, SPEC95 SPEC92, and SPEC89. SPEC CPU2017 consists of a set of 10 integer benchmarks (CINT2017) and 17 floating-point benchmarks (CFP2017). Figure 1.17 describes the current SPEC CPU benchmarks and their ancestry.

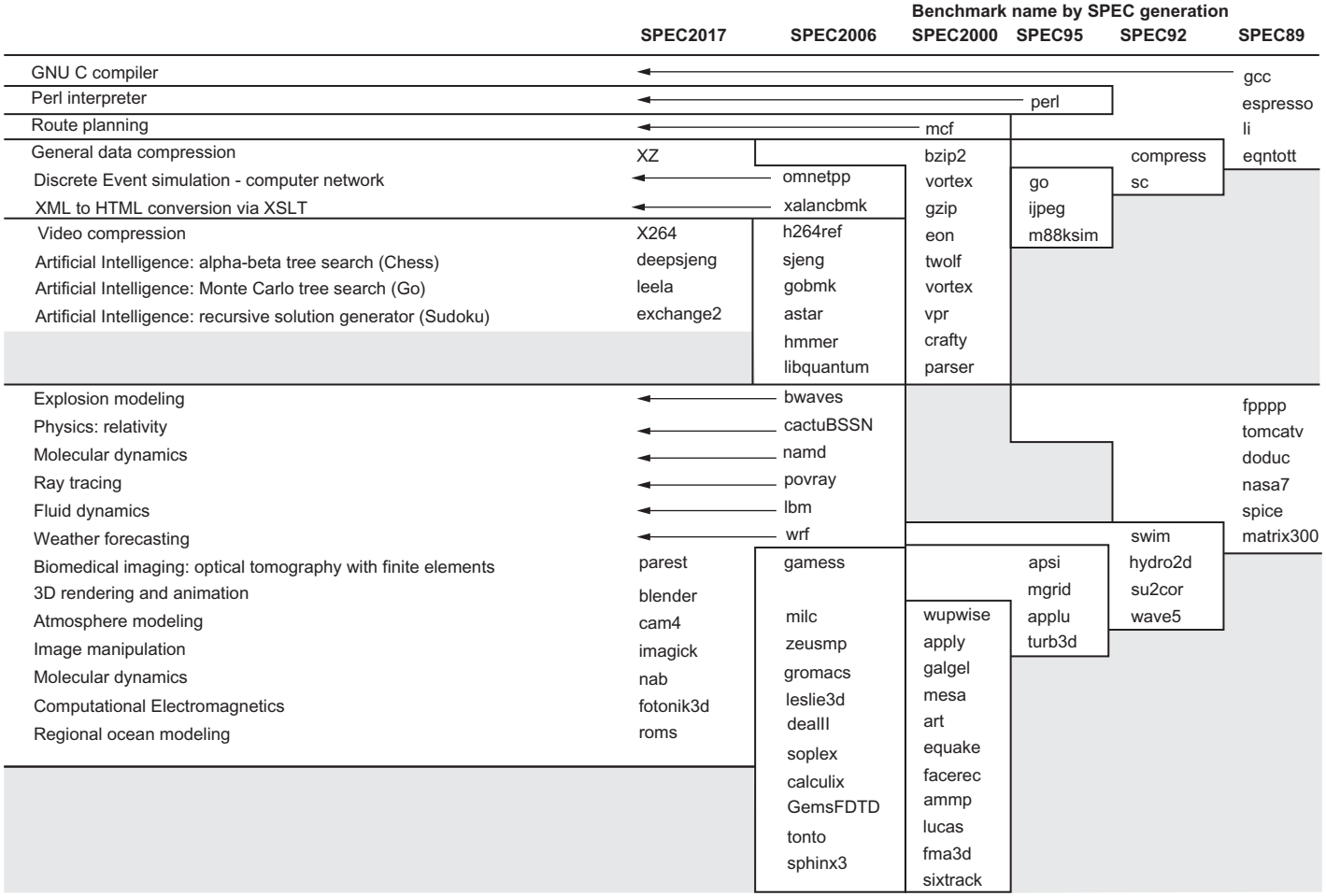


Figure 1.17 SPEC2017 programs and the evolution of the SPEC benchmarks over time, with integer programs above the line and floating-point programs below the line. Of the 10 SPEC2017 integer programs, 5 are written in C, 4 in C++., and 1 in Fortran. For the floating-point programs, the split is 3 in Fortran, 2 in C++, 2 in C, and 6 in mixed C, C++, and Fortran. The figure shows all 82 of the programs in the 1989, 1992, 1995, 2000, 2006, and 2017 releases. Gcc is the senior citizen of the group. Only 3 integer programs and 3 floating-point programs survived three or more generations. Although a few are carried over from generation to generation, the version of the program changes and either the input or the size of the benchmark is often expanded to increase its running time and to avoid perturbation in measurement or domination of the execution time by some factor other than CPU time. The benchmark descriptions on the left are for SPEC2017 only and do not apply to earlier versions. Programs in the same row from different generations of SPEC are generally not related; for example, fpppp is not a CFD code like bwaves.

SPEC benchmarks are real programs modified to be portable and to minimize the effect of I/O on performance. The integer benchmarks vary from part of a C compiler to a go program to a video compression. The floating-point benchmarks include molecular dynamics, ray tracing, and weather forecasting. The SPEC CPU suite is useful for processor benchmarking for both desktop systems and single-processor servers. We will see data on many of these programs throughout this book. However, these programs share little with modern programming languages and environments and the Google Translate application that Section 1.1 describes. Nearly half of them are written at least partially in Fortran! They are even statically linked instead of being dynamically linked like most real programs. Alas, the SPEC2017 applications themselves may be real, but they are not inspiring. It's not clear that SPECINT2017 and SPECFP2017 capture what is exciting about computing in the 21st century.

In Section 1.11, we describe pitfalls that have occurred in developing the SPEC CPUbenchmark suite, as well as the challenges in maintaining a useful and predictive benchmark suite.

SPEC CPU2017 is aimed at processor performance, but SPEC offers many other benchmarks. Figure 1.18 lists the 17 SPEC benchmarks that are active in 2017.

Server Benchmarks

Just as servers have multiple functions, so are there multiple types of benchmarks. The simplest benchmark is perhaps a processor throughput-oriented benchmark. SPEC CPU2017 uses the SPEC CPU benchmarks to construct a simple throughput benchmark where the processing rate of a multiprocessor can be measured by running multiple copies (usually as many as there are processors) of each SPEC CPU benchmark and converting the CPU time into a rate. This leads to a measurement called the SPECrate, and it is a measure of request-level parallelism from Section 1.2. To measure thread-level parallelism, SPEC offers what they call high-performance computing benchmarks around OpenMP and MPI as well as for accelerators such as GPUs (see Figure 1.18).

Other than SPECrate, most server applications and benchmarks have significant I/O activity arising from either storage or network traffic, including benchmarks for file server systems, for web servers, and for database and transaction-processing systems. SPEC offers both a file server benchmark (SPECSFS) and a Java server benchmark. (Appendix D discusses some file and I/O system benchmarks in detail.) SPECvirt_Sc2013 evaluates end-to-end performance of virtualized data center servers. Another SPEC benchmark measures power, which we examine in Section 1.10.

Transaction-processing (TP) benchmarks measure the ability of a system to handle transactions that consist of database accesses and updates. Airline reservation systems and bank ATM systems are typical simple examples of TP; more sophisticated TP systems involve complex databases and decision-making.

Category	Name	Measures performance of
Cloud	Cloud_IaaS 2016	Cloud using NoSQL database transaction and K-Means clustering using map/reduce
CPU	CPU2017	Compute-intensive integer and floating-point workloads
Graphics and workstation performance	SPECviewperf® 12	3D graphics in systems running OpenGL and Direct X
	SPECwpc V2.0	Workstations running professional apps under the Windows OS
	SPECapcSM for 3ds Max 2015™	3D graphics running the proprietary Autodesk 3ds Max 2015 app
	SPECapcSM for Maya® 2012	3D graphics running the proprietary Autodesk 3ds Max 2012 app
	SPECapcSM for PTC Creo 3.0	3D graphics running the proprietary PTC Creo 3.0 app
	SPECapcSM for Siemens NX 9.0 and 10.0	3D graphics running the proprietary Siemens NX 9.0 or 10.0 app
	SPECapcSM for SolidWorks 2015	3D graphics of systems running the proprietary SolidWorks 2015 CAD/CAM app
High performance computing	ACCEL	Accelerator and host CPU running parallel applications using OpenCL and OpenACC
	MPI2007	MPI-parallel, floating-point, compute-intensive programs running on clusters and SMPs
	OMP2012	Parallel apps running OpenMP
Java client/server	SPECjbb2015	Java servers
Power	SPECpower_ssj2008	Power of volume server class computers running SPECjbb2015
Solution File Server (SFS)	SFS2014	File server throughput and response time
	SPECsfs2008	File servers utilizing the NFSv3 and CIFS protocols
Virtualization	SPECvirt_sc2013	Datacenter servers used in virtualized server consolidation

Figure 1.18 Active benchmarks from SPEC as of 2017.

In the mid-1980s, a group of concerned engineers formed the vendor-independent Transaction Processing Council (TPC) to try to create realistic and fair benchmarks for TP. The TPC benchmarks are described at http://www.tpc.org.

The first TPC benchmark, TPC-A, was published in 1985 and has since been replaced and enhanced by several different benchmarks. TPC-C, initially created in 1992, simulates a complex query environment. TPC-H models ad hoc decision support—the queries are unrelated and knowledge of past queries cannot be used to optimize future queries. The TPC-DI benchmark, a new data integration (DI) task also known as ETL, is an important part of data warehousing. TPC-E is an online transaction processing (OLTP) workload that simulates a brokerage firm's customer accounts.

Recognizing the controversy between traditional relational databases and "No SQL" storage solutions, TPCx-HS measures systems using the Hadoop file system running MapReduce programs, and TPC-DS measures a decision support system that uses either a relational database or a Hadoop-based system. TPC-VMS and TPCx-V measure database performance for virtualized systems, and TPC-Energy adds energy metrics to all the existing TPC benchmarks.

All the TPC benchmarks measure performance in transactions per second. In addition, they include a response time requirement so that throughput performance is measured only when the response time limit is met. To model real-world systems, higher transaction rates are also associated with larger systems, in terms of both users and the database to which the transactions are applied. Finally, the system cost for a benchmark system must be included as well to allow accurate comparisons of cost-performance. TPC modified its pricing policy so that there is a single specification for all the TPC benchmarks and to allow verification of the prices that TPC publishes.

Reporting Performance Results

The guiding principle of reporting performance measurements should be *reproducibility*—list everything another experimenter would need to duplicate the results. A SPEC benchmark report requires an extensive description of the computer and the compiler flags, as well as the publication of both the baseline and the optimized results. In addition to hardware, software, and baseline tuning parameter descriptions, a SPEC report contains the actual performance times, shown both in tabular form and as a graph. A TPC benchmark report is even more complete, because it must include results of a benchmarking audit and cost information. These reports are excellent sources for finding the real costs of computing systems, since manufacturers compete on high performance and cost-performance.

Summarizing Performance Results

In practical computer design, one must evaluate myriad design choices for their relative quantitative benefits across a suite of benchmarks believed to be relevant. Likewise, consumers trying to choose a computer will rely on performance measurements from benchmarks, which ideally are similar to the users' applications. In both cases, it is useful to have measurements for a suite of benchmarks so that the performance of important applications is similar to that of one or more benchmarks in the suite and so that variability in performance can be understood. In the best case, the suite resembles a statistically valid sample of the application space, but such a sample requires more benchmarks than are typically found in most suites and requires a randomized sampling, which essentially no benchmark suite uses.

Once we have chosen to measure performance with a benchmark suite, we want to be able to summarize the performance results of the suite in a unique number. A simple approach to computing a summary result would be to compare the arithmetic means of the execution times of the programs in the suite. An alternative would be to add a weighting factor to each benchmark and use the weighted arithmetic mean as the single number to summarize performance. One approach is to use weights that make all programs execute an equal time on some reference computer, but this biases the results toward the performance characteristics of the reference computer.

Rather than pick weights, we could normalize execution times to a reference computer by dividing the time on the reference computer by the time on the computer being rated, yielding a ratio proportional to performance. SPEC uses this approach, calling the ratio the SPECRatio. It has a particularly useful property that matches the way we benchmark computer performance throughout this text—namely, comparing performance ratios. For example, suppose that the SPECRatio of computer A on a benchmark is 1.25 times as fast as computer B; then we know

$$1.25 = \frac{\text{SPECRatio}_A}{\text{SPECRatio}_B} = \frac{\dfrac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_A}}{\dfrac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_B}} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{\text{Performance}_A}{\text{Performance}_B}$$

Notice that the execution times on the reference computer drop out and the choice of the reference computer is irrelevant when the comparisons are made as a ratio, which is the approach we consistently use. Figure 1.19 gives an example.

Because a SPECRatio is a ratio rather than an absolute execution time, the mean must be computed using the *geometric* mean. (Because SPECRatios have no units, comparing SPECRatios arithmetically is meaningless.) The formula is

$$\text{Geometric mean} = \sqrt[n]{\prod_{i=1}^{n} sample_i}$$

In the case of SPEC, $sample_i$ is the SPECRatio for program i. Using the geometric mean ensures two important properties:

1. The geometric mean of the ratios is the same as the ratio of the geometric means.
2. The ratio of the geometric means is equal to the geometric mean of the performance ratios, which implies that the choice of the reference computer is irrelevant.

Therefore the motivations to use the geometric mean are substantial, especially when we use performance ratios to make comparisons.

Example Show that the ratio of the geometric means is equal to the geometric mean of the performance ratios and that the reference computer of SPECRatio does not matter.

Answer Assume two computers A and B and a set of SPECRatios for each.

$$\frac{\text{Geometric mean}_A}{\text{Geometric mean}_B} = \frac{\sqrt[n]{\prod_{i=1}^{n}\text{SPECRatio A}_i}}{\sqrt[n]{\prod_{i=1}^{n}\text{SPECRatio B}_i}} = \sqrt[n]{\prod_{i=1}^{n}\frac{\text{SPECRatio A}_i}{\text{SPECRatio B}_i}}$$

$$= \sqrt[n]{\prod_{i=1}^{n}\frac{\dfrac{\text{Execution time}_{\text{reference}_i}}{\text{Execution time}_{A_i}}}{\dfrac{\text{Execution time}_{\text{reference}_i}}{\text{Execution time}_{B_i}}}} = \sqrt[n]{\prod_{i=1}^{n}\frac{\text{Execution time}_{B_i}}{\text{Execution time}_{A_i}}} = \sqrt[n]{\prod_{i=1}^{n}\frac{\text{Performance}_{A_i}}{\text{Performance}_{B_i}}}$$

That is, the ratio of the geometric means of the SPECRatios of A and B is the geometric mean of the performance ratios of A to B of all the benchmarks in the suite. Figure 1.19 demonstrates this validity using examples from SPEC.

Benchmarks	Sun Ultra Enterprise 2 time (seconds)	AMD A10-6800K time (seconds)	SPEC 2006Cint ratio	Intel Xeon E5-2690 time (seconds)	SPEC 2006Cint ratio	AMD/Intel times (seconds)	Intel/AMD SPEC ratios
perlbench	9770	401	24.36	261	37.43	1.54	1.54
bzip2	9650	505	19.11	422	22.87	1.20	1.20
gcc	8050	490	16.43	227	35.46	2.16	2.16
mcf	9120	249	36.63	153	59.61	1.63	1.63
gobmk	10,490	418	25.10	382	27.46	1.09	1.09
hmmer	9330	182	51.26	120	77.75	1.52	1.52
sjeng	12,100	517	23.40	383	31.59	1.35	1.35
libquantum	20,720	84	246.08	3	7295.77	29.65	29.65
h264ref	22,130	611	36.22	425	52.07	1.44	1.44
omnetpp	6250	313	19.97	153	40.85	2.05	2.05
astar	7020	303	23.17	209	33.59	1.45	1.45
xalancbmk	6900	215	32.09	98	70.41	2.19	2.19
Geometric mean			31.91		63.72	2.00	2.00

Figure 1.19 SPEC2006Cint execution times (in seconds) for the Sun Ultra 5—the reference computer of SPEC2006—and execution times and SPECRatios for the AMD A10 and Intel Xeon E5-2690. The final two columns show the ratios of execution times and SPEC ratios. This figure demonstrates the irrelevance of the reference computer in relative performance. The ratio of the execution times is identical to the ratio of the SPEC ratios, and the ratio of the geometric means (63.7231.91/20.86 = 2.00) is identical to the geometric mean of the ratios (2.00). Section 1.11 discusses libquantum, whose performance is orders of magnitude higher than the other SPEC benchmarks.

1.9 Quantitative Principles of Computer Design

Now that we have seen how to define, measure, and summarize performance, cost, dependability, energy, and power, we can explore guidelines and principles that are useful in the design and analysis of computers. This section introduces important observations about design, as well as two equations to evaluate alternatives.

Take Advantage of Parallelism

Using parallelism is one of the most important methods for improving performance. Every chapter in this book has an example of how performance is enhanced through the exploitation of parallelism. We give three brief examples here, which are expounded on in later chapters.

Our first example is the use of parallelism at the system level. To improve the throughput performance on a typical server benchmark, such as SPECSFS or TPC-C, multiple processors and multiple storage devices can be used. The workload of handling requests can then be spread among the processors and storage devices, resulting in improved throughput. Being able to expand memory and the number of processors and storage devices is called *scalability*, and it is a valuable asset for servers. Spreading of data across many storage devices for parallel reads and writes enables data-level parallelism. SPECSFS also relies on request-level parallelism to use many processors, whereas TPC-C uses thread-level parallelism for faster processing of database queries.

At the level of an individual processor, taking advantage of parallelism among instructions is critical to achieving high performance. One of the simplest ways to do this is through pipelining. (Pipelining is explained in more detail in Appendix C and is a major focus of Chapter 3.) The basic idea behind pipelining is to overlap instruction execution to reduce the total time to complete an instruction sequence. A key insight into pipelining is that not every instruction depends on its immediate predecessor, so executing the instructions completely or partially in parallel may be possible. Pipelining is the best-known example of ILP.

Parallelism can also be exploited at the level of detailed digital design. For example, set-associative caches use multiple banks of memory that are typically searched in parallel to find a desired item. Arithmetic-logical units use carry-lookahead, which uses parallelism to speed the process of computing sums from linear to logarithmic in the number of bits per operand. These are more examples of *data-level parallelism*.

Principle of Locality

Important fundamental observations have come from properties of programs. The most important program property that we regularly exploit is the *principle of locality*: programs tend to reuse data and instructions they have used recently. A widely held rule of thumb is that a program spends 90% of its execution time in only 10% of the code. An implication of locality is that we can predict with reasonable

accuracy what instructions and data a program will use in the near future based on its accesses in the recent past. The principle of locality also applies to data accesses, though not as strongly as to code accesses.

Two different types of locality have been observed. *Temporal locality* states that recently accessed items are likely to be accessed soon. *Spatial locality* says that items whose addresses are near one another tend to be referenced close together in time. We will see these principles applied in Chapter 2.

Focus on the Common Case

Perhaps the most important and pervasive principle of computer design is to focus on the common case: in making a design trade-off, favor the frequent case over the infrequent case. This principle applies when determining how to spend resources, because the impact of the improvement is higher if the occurrence is commonplace.

Focusing on the common case works for energy as well as for resource allocation and performance. The instruction fetch and decode unit of a processor may be used much more frequently than a multiplier, so optimize it first. It works on dependability as well. If a database server has 50 storage devices for every processor, storage dependability will dominate system dependability.

In addition, the common case is often simpler and can be done faster than the infrequent case. For example, when adding two numbers in the processor, we can expect overflow to be a rare circumstance and can therefore improve performance by optimizing the more common case of no overflow. This emphasis may slow down the case when overflow occurs, but if that is rare, then overall performance will be improved by optimizing for the normal case.

We will see many cases of this principle throughout this text. In applying this simple principle, we have to decide what the frequent case is and how much performance can be improved by making that case faster. A fundamental law, called *Amdahl's Law*, can be used to quantify this principle.

Amdahl's Law

The performance gain that can be obtained by improving some portion of a computer can be calculated using Amdahl's Law. Amdahl's Law states that the performance improvement to be gained from using some faster mode of execution is limited by the fraction of the time the faster mode can be used.

Amdahl's Law defines the *speedup* that can be gained by using a particular feature. What is speedup? Suppose that we can make an enhancement to a computer that will improve performance when it is used. Speedup is the ratio

$$\text{Speedup} = \frac{\text{Performance for entire task using the enhancement when possible}}{\text{Performance for entire task without using the enhancement}}$$

Alternatively,

$$\text{Speedup} = \frac{\text{Execution time for entire task without using the enhancement}}{\text{Execution time for entire task using the enhancement when possible}}$$

Speedup tells us how much faster a task will run using the computer with the enhancement contrary to the original computer.

Amdahl's Law gives us a quick way to find the speedup from some enhancement, which depends on two factors:

1. *The fraction of the computation time in the original computer that can be converted to take advantage of the enhancement*—For example, if 40 seconds of the execution time of a program that takes 100 seconds in total can use an enhancement, the fraction is 40/100. This value, which we call $\text{Fraction}_{\text{enhanced}}$, is always less than or equal to 1.
2. *The improvement gained by the enhanced execution mode, that is, how much faster the task would run if the enhanced mode were used for the entire program*—This value is the time of the original mode over the time of the enhanced mode. If the enhanced mode takes, say, 4 seconds for a portion of the program, while it is 40 seconds in the original mode, the improvement is 40/4 or 10. We call this value, which is always greater than 1, $\text{Speedup}_{\text{enhanced}}$.

The execution time using the original computer with the enhanced mode will be the time spent using the unenhanced portion of the computer plus the time spent using the enhancement:

$$\text{Execution time}_{\text{new}} = \text{Execution time}_{\text{old}} \times \left((1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right)$$

The overall speedup is the ratio of the execution times:

$$\text{Speedup}_{\text{overall}} = \frac{\text{Execution time}_{\text{old}}}{\text{Execution time}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

Example Suppose that we want to enhance the processor used for web serving. The new processor is 10 times faster on computation in the web serving application than the old processor. Assuming that the original processor is busy with computation 40% of the time and is waiting for I/O 60% of the time, what is the overall speedup gained by incorporating the enhancement?

Answer $\text{Fraction}_{\text{enhanced}} = 0.4$; $\text{Speedup}_{\text{enhanced}} = 10$; $\text{Speedup}_{\text{overall}} = \dfrac{1}{0.6 + \frac{0.4}{10}} = \dfrac{1}{0.64} \approx 1.56$

Amdahl's Law expresses the law of diminishing returns: The incremental improvement in speedup gained by an improvement of just a portion of the computation diminishes as improvements are added. An important corollary of Amdahl's Law is that if an enhancement is usable only for a fraction of a task, then we can't speed up the task by more than the reciprocal of 1 minus that fraction.

A common mistake in applying Amdahl's Law is to confuse "fraction of time converted *to use an enhancement*" and "fraction of time *after enhancement is in use*." If, instead of measuring the time that we *could use* the enhancement in a computation, we measure the time *after* the enhancement is in use, the results will be incorrect!

Amdahl's Law can serve as a guide to how much an enhancement will improve performance and how to distribute resources to improve cost-performance. The goal, clearly, is to spend resources proportional to where time is spent. Amdahl's Law is particularly useful for comparing the overall system performance of two alternatives, but it can also be applied to compare two processor design alternatives, as the following example shows.

Example A common transformation required in graphics processors is square root. Implementations of floating-point (FP) square root vary significantly in performance, especially among processors designed for graphics. Suppose FP square root (FSQRT) is responsible for 20% of the execution time of a critical graphics benchmark. One proposal is to enhance the FSQRT hardware and speed up this operation by a factor of 10. The other alternative is just to try to make all FP instructions in the graphics processor run faster by a factor of 1.6; FP instructions are responsible for half of the execution time for the application. The design team believes that they can make all FP instructions run 1.6 times faster with the same effort as required for the fast square root. Compare these two design alternatives.

Answer We can compare these two alternatives by comparing the speedups:

$$\text{Speedup}_{\text{FSQRT}} = \frac{1}{(1-0.2)+\frac{0.2}{10}} = \frac{1}{0.82} = 1.22$$

$$\text{Speedup}_{\text{FP}} = \frac{1}{(1-0.5)+\frac{0.5}{1.6}} = \frac{1}{0.8125} = 1.23$$

Improving the performance of the FP operations overall is slightly better because of the higher frequency.

Amdahl's Law is applicable beyond performance. Let's redo the reliability example from page 39 after improving the reliability of the power supply via redundancy from 200,000-hour to 830,000,000-hour MTTF, or 4150× better.

Example The calculation of the failure rates of the disk subsystem was

$$\text{Failure rate}_{\text{system}} = 10 \times \frac{1}{1{,}000{,}000} + \frac{1}{500{,}000} + \frac{1}{200{,}000} + \frac{1}{200{,}000} + \frac{1}{1{,}000{,}000}$$

$$= \frac{10+2+5+5+1}{1{,}000{,}000 \text{ hours}} = \frac{23}{1{,}000{,}000 \text{ hours}}$$

Therefore the fraction of the failure rate that could be improved is 5 per million hours out of 23 for the whole system, or 0.22.

Answer The reliability improvement would be

$$\text{Improvement}_{\text{power supply pair}} = \frac{1}{(1-0.22)+\dfrac{0.22}{4150}} = \frac{1}{0.78} = 1.28$$

Despite an impressive 4150 × improvement in reliability of one module, from the system's perspective, the change has a measurable but small benefit.

In the preceding examples, we needed the fraction consumed by the new and improved version; often it is difficult to measure these times directly. In the next section, we will see another way of doing such comparisons based on the use of an equation that decomposes the CPU execution time into three separate components. If we know how an alternative affects these three components, we can determine its overall performance. Furthermore, it is often possible to build simulators that measure these components before the hardware is actually designed.

The Processor Performance Equation

Essentially all computers are constructed using a clock running at a constant rate. These discrete time events are called *clock periods*, *clocks*, *cycles*, or *clock cycles*. Computer designers refer to the time of a clock period by its duration (e.g., 1 ns) or by its rate (e.g., 1 GHz). CPU time for a program can then be expressed two ways:

$$\text{CPU time} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

or

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

In addition to the number of clock cycles needed to execute a program, we can also count the number of instructions executed—the *instruction path length* or *instruction count* (IC). If we know the number of clock cycles and the instruction count, we can calculate the average number of *clock cycles per instruction* (CPI). Because it is easier to work with, and because we will deal with simple processors in this chapter, we use CPI. Designers sometimes also use *instructions per clock* (IPC), which is the inverse of CPI.

CPI is computed as

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

This processor figure of merit provides insight into different styles of instruction sets and implementations, and we will use it extensively in the next four chapters.

By transposing the instruction count in the preceding formula, clock cycles can be defined as IC × CPI. This allows us to use CPI in the execution time formula:

$$\text{CPU time} = \text{Instruction count} \times \text{Cycles per instruction} \times \text{Clock cycle time}$$

Expanding the first formula into the units of measurement shows how the pieces fit together:

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} = \frac{\text{Seconds}}{\text{Program}} = \text{CPU time}$$

As this formula demonstrates, processor performance is dependent upon three characteristics: clock cycle (or rate), clock cycles per instruction, and instruction count. Furthermore, CPU time is *equally* dependent on these three characteristics; for example, a 10% improvement in any one of them leads to a 10% improvement in CPU time.

Unfortunately, it is difficult to change one parameter in complete isolation from others because the basic technologies involved in changing each characteristic are interdependent:

- *Clock cycle time*—Hardware technology and organization
- *CPI*—Organization and instruction set architecture
- *Instruction count*—Instruction set architecture and compiler technology

Luckily, many potential performance improvement techniques primarily enhance one component of processor performance with small or predictable impacts on the other two.

In designing the processor, sometimes it is useful to calculate the number of total processor clock cycles as

$$\text{CPU clock cycles} = \sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i$$

where IC_i represents the number of times instruction i is executed in a program and CPI_i represents the average number of clocks per instruction for instruction i. This form can be used to express CPU time as

$$\text{CPU time} = \left(\sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i \right) \times \text{Clock cycle time}$$

and overall CPI as

$$\text{CPI} = \frac{\sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i}{\text{Instruction count}} = \sum_{i=1}^{n} \frac{\text{IC}_i}{\text{Instruction count}} \times \text{CPI}_i$$

The latter form of the CPI calculation uses each individual CPI_i and the fraction of occurrences of that instruction in a program (i.e., $\text{IC}_i \div$ Instruction count). Because it must include pipeline effects, cache misses, and any other memory system

inefficiencies, CPI_i should be measured and not just calculated from a table in the back of a reference manual.

Consider our performance example on page 52, here modified to use measurements of the frequency of the instructions and of the instruction CPI values, which, in practice, are obtained by simulation or by hardware instrumentation.

Example Suppose we made the following measurements:

Frequency of FP operations = 25%

Average CPI of FP operations = 4.0

Average CPI of other instructions = 1.33

Frequency of FSQRT = 2%

CPI of FSQRT = 20

Assume that the two design alternatives are to decrease the CPI of FSQRT to 2 or to decrease the average CPI of all FP operations to 2.5. Compare these two design alternatives using the processor performance equation.

Answer First, observe that only the CPI changes; the clock rate and instruction count remain identical. We start by finding the original CPI with neither enhancement:

$$\begin{aligned}\text{CPI}_{\text{original}} &= \sum_{i=1}^{n} \text{CPI}_i \times \left(\frac{\text{IC}_i}{\text{Instruction count}}\right) \\ &= (4 \times 25\%) + (1.33 \times 75\%) = 2.0\end{aligned}$$

We can compute the CPI for the enhanced FSQRT by subtracting the cycles saved from the original CPI:

$$\begin{aligned}\text{CPI}_{\text{with new FPSQR}} &= \text{CPI}_{\text{original}} - 2\% \times \left(\text{CPI}_{\text{old FPSQR}} - \text{CPI}_{\text{of new FPSQR only}}\right) \\ &= 2.0 - 2\% \times (20 - 2) = 1.64\end{aligned}$$

We can compute the CPI for the enhancement of all FP instructions the same way or by summing the FP and non-FP CPIs. Using the latter gives us

$$\text{CPI}_{\text{new FP}} = (75\% \times 1.33) + (25\% \times 2.5) = 1.625$$

Since the CPI of the overall FP enhancement is slightly lower, its performance will be marginally better. Specifically, the speedup for the overall FP enhancement is

$$\begin{aligned}\text{Speedup}_{\text{new FP}} &= \frac{\text{CPU time}_{\text{original}}}{\text{CPU time}_{\text{new FP}}} = \frac{\text{IC} \times \text{Clock cycle} \times \text{CPI}_{\text{original}}}{\text{IC} \times \text{Clock cycle} \times \text{CPI}_{\text{new FP}}} \\ &= \frac{\text{CPI}_{\text{original}}}{\text{CPI}_{\text{new FP}}} = \frac{2.00}{1.625} = 1.23\end{aligned}$$

Happily, we obtained this same speedup using Amdahl's Law on page 51.

It is often possible to measure the constituent parts of the processor performance equation. Such isolated measurements are a key advantage of using the processor performance equation versus Amdahl's Law in the previous example. In particular, it may be difficult to measure things such as the fraction of execution time for which a set of instructions is responsible. In practice, this would probably be computed by summing the product of the instruction count and the CPI for each of the instructions in the set. Since the starting point is often individual instruction count and CPI measurements, the processor performance equation is incredibly useful.

To use the processor performance equation as a design tool, we need to be able to measure the various factors. For an existing processor, it is easy to obtain the execution time by measurement, and we know the default clock speed. The challenge lies in discovering the instruction count or the CPI. Most processors include counters for both instructions executed and clock cycles. By periodically monitoring these counters, it is also possible to attach execution time and instruction count to segments of the code, which can be helpful to programmers trying to understand and tune the performance of an application. Often designers or programmers will want to understand performance at a more fine-grained level than what is available from the hardware counters. For example, they may want to know why the CPI is what it is. In such cases, the simulation techniques used are like those for processors that are being designed.

Techniques that help with energy efficiency, such as dynamic voltage frequency scaling and overclocking (see Section 1.5), make this equation harder to use, because the clock speed may vary while we measure the program. A simple approach is to turn off those features to make the results reproducible. Fortunately, as performance and energy efficiency are often highly correlated—taking less time to run a program generally saves energy—it's probably safe to consider performance without worrying about the impact of DVFS or overclocking on the results.

1.10 Putting It All Together: Performance, Price, and Power

In the "Putting It All Together" sections that appear near the end of every chapter, we provide real examples that use the principles in that chapter. In this section, we look at measures of performance and power-performance in small servers using the SPECpower benchmark.

Figure 1.20 shows the three multiprocessor servers we are evaluating along with their price. To keep the price comparison fair, all are Dell PowerEdge servers. The first is the PowerEdge R710, which is based on the Intel Xeon ×85670 microprocessor with a clock rate of 2.93 GHz. Unlike the Intel Core i7-6700 in Chapters 2–5, which has 20 cores and a 40 MB L3 cache, this Intel chip has 22 cores and a 55 MB L3 cache, although the cores themselves are identical. We selected a two-socket system—so 44 cores total—with 128 GB of ECC-protected 2400 MHz DDR4 DRAM. The next server is the PowerEdge C630, with the same processor, number of sockets, and DRAM. The main difference is a smaller rack-mountable package: "2U" high (3.5 inches) for the 730 versus "1U" (1.75 inches) for the 630.

	System 1		System 2		System 3	
Component		**Cost (% Cost)**		**Cost (% Cost)**		**Cost (% Cost)**
Base server	PowerEdge R710	$653 (7%)	PowerEdge R815	$1437 (15%)	PowerEdge R815	$1437 (11%)
Power supply	570 W		1100 W		1100 W	
Processor	Xeon X5670	$3738 (40%)	Opteron 6174	$2679 (29%)	Opteron 6174	$5358 (42%)
Clock rate	2.93 GHz		2.20 GHz		2.20 GHz	
Total cores	12		24		48	
Sockets	2		2		4	
Cores/socket	6		12		12	
DRAM	12 GB	$484 (5%)	16 GB	$693 (7%)	32 GB	$1386 (11%)
Ethernet Inter.	Dual 1-Gbit	$199 (2%)	Dual 1-Gbit	$199 (2%)	Dual 1-Gbit	$199 (2%)
Disk	50 GB SSD	$1279 (14%)	50 GB SSD	$1279 (14%)	50 GB SSD	$1279 (10%)
Windows OS		$2999 (32%)		$2999 (33%)		$2999 (24%)
Total		$9352 (100%)		$9286 (100%)		$12,658 (100%)
Max ssj_ops	910,978		926,676		1,840,450	
Max ssj_ops/$	97		100		145	

Figure 1.20 Three Dell PowerEdge servers being measured and their prices as of July 2016. We calculated the cost of the processors by subtracting the cost of a second processor. Similarly, we calculated the overall cost of memory by seeing what the cost of extra memory was. Hence the base cost of the server is adjusted by removing the estimated cost of the default processor and memory. Chapter 5 describes how these multisocket systems are connected together, and Chapter 6 describes how clusters are connected together.

The third server is a cluster of 16 of the PowerEdge 630 s that is connected together with a 1 Gbit/s Ethernet switch. All are running the Oracle Java HotSpot version 1.7 Java Virtual Machine (JVM) and the Microsoft Windows Server 2012 R2 Datacenter version 6.3 operating system.

Note that because of the forces of benchmarking (see Section 1.11), these are unusually configured servers. The systems in Figure 1.20 have little memory relative to the amount of computation, and just a tiny 120 GB solid-state disk. It is inexpensive to add cores if you don't need to add commensurate increases in memory and storage!

Rather than run statically linked C programs of SPEC CPU, SPECpower uses a more modern software stack written in Java. It is based on SPECjbb, and it represents the server side of business applications, with performance measured as the number of transactions per second, called *ssj_ops* for *server side Java operations per second*. It exercises not only the processor of the server, as does SPEC CPU, but also the caches, memory system, and even the multiprocessor interconnection system. In addition, it exercises the JVM, including the JIT runtime compiler and garbage collector, as well as portions of the underlying operating system.

As the last two rows of Figure 1.20 show, the performance winner is the cluster of 16 R630s, which is hardly a surprise since it is by far the most expensive. The price-performance winner is the PowerEdge R630, but it barely beats the cluster at 213 versus 211 ssj-ops/$. Amazingly, the 16 node cluster is within 1% of the same price-performances of a single node despite being 16 times as large.

While most benchmarks (and most computer architects) care only about performance of systems at peak load, computers rarely run at peak load. Indeed, Figure 6.2 in Chapter 6 shows the results of measuring the utilization of tens of thousands of servers over 6 months at Google, and less than 1% operate at an average utilization of 100%. The majority have an average utilization of between 10% and 50%. Thus the SPECpower benchmark captures power as the target workload varies from its peak in 10% intervals all the way to 0%, which is called Active Idle.

Figure 1.21 plots the ssj_ops (SSJ operations/second) per watt and the average power as the target load varies from 100% to 0%. The Intel R730 always has the lowest power and the single node R630 has the best ssj_ops per watt across each target workload level. Since watts = joules/second, this metric is proportional to SSJ operations per joule:

$$\frac{\text{ssj_operations/second}}{\text{Watt}} = \frac{\text{ssj_operations/second}}{\text{Joule/second}} = \frac{\text{ssj_operations}}{\text{Joule}}$$

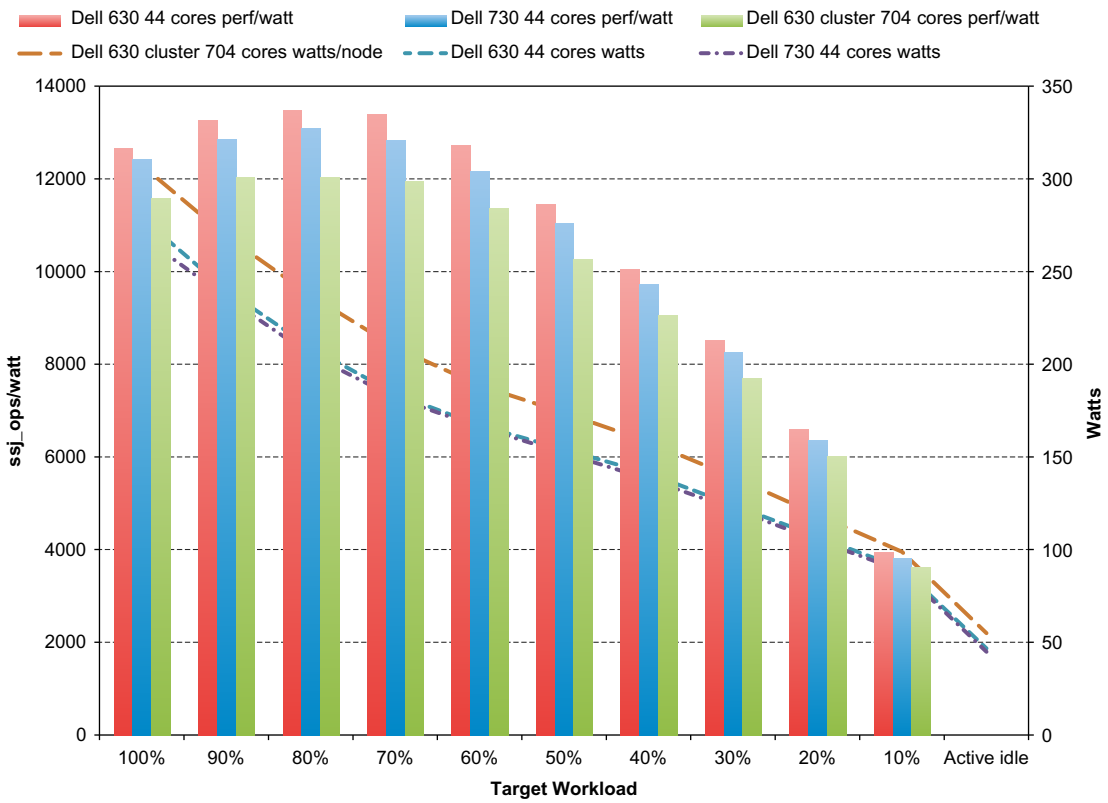


Figure 1.21 Power-performance of the three servers in Figure 1.20. Ssj_ops/watt values are on the left axis, with the three columns associated with it, and watts are on the right axis, with the three lines associated with it. The horizontal axis shows the target workload, as it varies from 100% to Active Idle. The single node R630 has the best ssj_ops/watt at each workload level, but R730 consumes the lowest power at each level.

To calculate a single number to use to compare the power efficiency of systems, SPECpower uses

$$\text{Overall ssj_ops/watt} = \frac{\sum \text{ssj_ops}}{\sum \text{power}}$$

The overall ssj_ops/watt of the three servers is 10,802 for the R730, 11,157 for the R630, and 10,062 for the cluster of 16 R630s. Therefore the single node R630 has the best power-performance. Dividing by the price of the servers, the ssj_ops/watt/$1,000 is 879 for the R730, 899 for the R630, and 789 (per node) for the 16-node cluster of R630s. Thus, after adding power, the single-node R630 is still in first place in performance/price, but now the single-node R730 is significantly more efficient than the 16-node cluster.

1.11 Fallacies and Pitfalls

The purpose of this section, which will be found in every chapter, is to explain some commonly held misbeliefs or misconceptions that you should avoid. We call such misbeliefs *fallacies.* When discussing a fallacy, we try to give a counterexample. We also discuss *pitfalls*—easily made mistakes. Often pitfalls are generalizations of principles that are true in a limited context. The purpose of these sections is to help you avoid making these errors in computers that you design.

Pitfall *All exponential laws must come to an end.*

The first to go was Dennard scaling. Dennard's 1974 observation was that power density was constant as transistors got smaller. If a transistor's linear region shrank by a factor 2, then both the current and voltage were also reduced by a factor of 2, and so the power it used fell by 4. Thus chips could be designed to operate faster and still use less power. Dennard scaling ended 30 years after it was observed, not because transistors didn't continue to get smaller but because integrated circuit dependability limited how far current and voltage could drop. The threshold voltage was driven so low that static power became a significant fraction of overall power.

The next deceleration was hard disk drives. Although there was no law for disks, in the past 30 years the maximum areal density of hard drives—which determines disk capacity—improved by 30%–100% per year. In more recent years, it has been less than 5% per year. Increasing density per drive has come primarily from adding more platters to a hard disk drive.

Next up was the venerable Moore's Law. It's been a while since the number of transistors per chip doubled every one to two years. For example, the DRAM chip introduced in 2014 contained 8B transistors, and we won't have a 16B transistor DRAM chip in mass production until 2019, but Moore's Law predicts a 64B transistor DRAM chip.

Moreover, the actual end of scaling of the planar logic transistor was even predicted to end by 2021. Figure 1.22 shows the predictions of the physical gate length

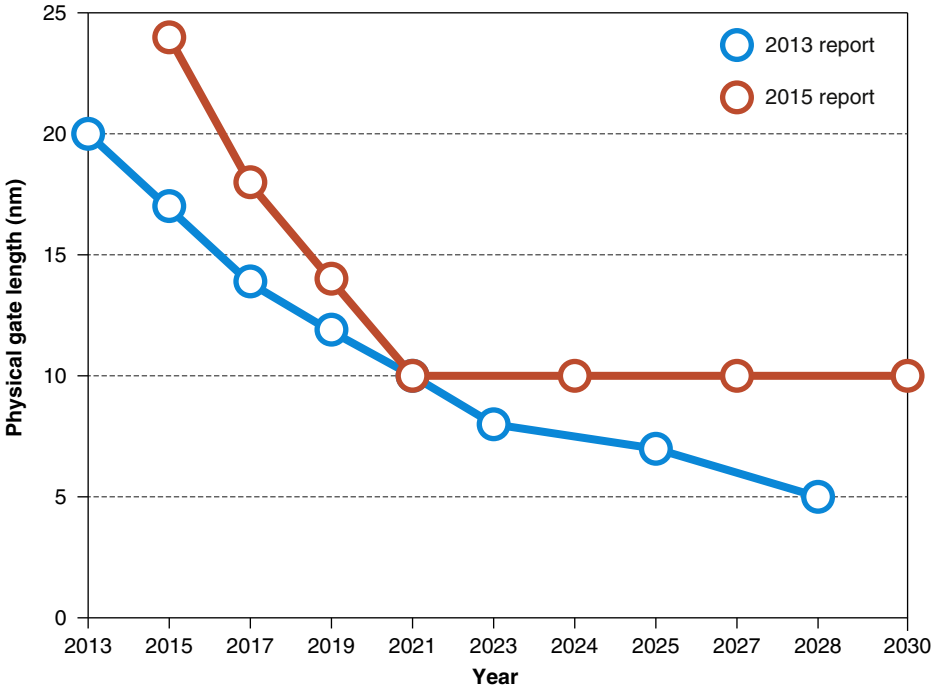


Figure 1.22 Predictions of logic transistor dimensions from two editions of the ITRS report. These reports started in 2001, but 2015 will be the last edition, as the group has disbanded because of waning interest. The only companies that can produce state-of-the-art logic chips today are GlobalFoundaries, Intel, Samsung, and TSMC, whereas there were 19 when the first ITRS report was released. With only four companies left, sharing of plans was too hard to sustain. From IEEE Spectrum, July 2016, "Transistors will stop shrinking in 2021, Moore's Law Roadmap Predicts," by Rachel Courtland.

of the logic transistor from two editions of the International Technology Roadmap for Semiconductors (ITRS). Unlike the 2013 report that projected gate lengths to reach 5 nm by 2028, the 2015 report projects the length stopping at 10 nm by 2021. Density improvements thereafter would have to come from ways other than shrinking the dimensions of transistors. It's not as dire as the ITRS suggests, as companies like Intel and TSMC have plans to shrink to 3 nm gate lengths, but the rate of change is decreasing.

Figure 1.23 shows the changes in increases in bandwidth over time for microprocessors and DRAM—which are affected by the end of Dennard scaling and Moore's Law—as well as for disks. The slowing of technology improvements is apparent in the dropping curves. The continued networking improvement is due to advances in fiber optics and a planned change in pulse amplitude modulation (PAM-4) allowing two-bit encoding so as to transmit information at 400 Gbit/s.

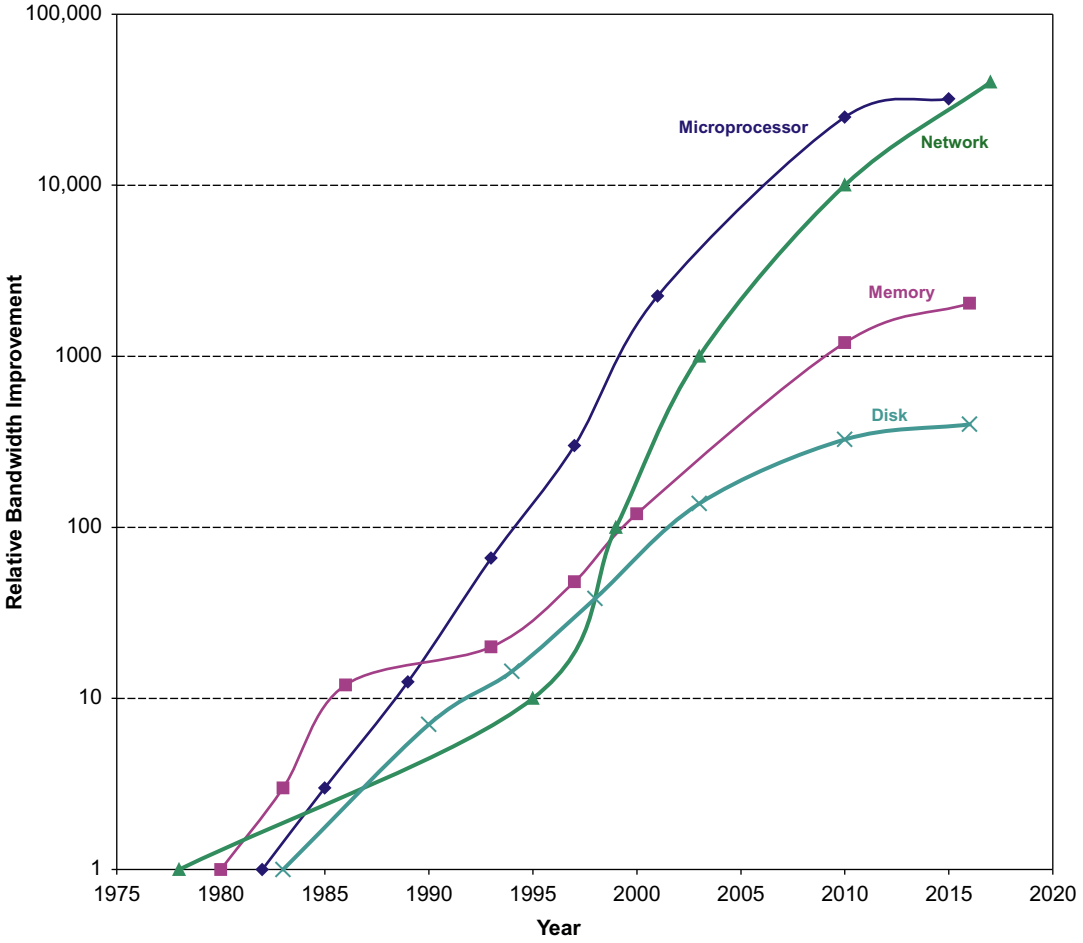


Figure 1.23 Relative bandwidth for microprocessors, networks, memory, and disks over time, based on data in Figure 1.10.

Fallacy *Multiprocessors are a silver bullet.*

The switch to multiple processors per chip around 2005 did not come from some breakthrough that dramatically simplified parallel programming or made it easy to build multicore computers. The change occurred because there was no other option due to the ILP walls and power walls. Multiple processors per chip do not guarantee lower power; it's certainly feasible to design a multicore chip that uses more power. The potential is just that it's possible to continue to improve performance by replacing a high-clock-rate, inefficient core with several lower-clock-rate, efficient cores. As technology to shrink transistors improves, it can shrink both capacitance and the supply voltage a bit so that we can get a modest increase in the

number of cores per generation. For example, for the past few years, Intel has been adding two cores per generation in their higher-end chips.

As we will see in Chapters 4 and 5, performance is now a programmer's burden. The programmers' La-Z-Boy era of relying on a hardware designer to make their programs go faster without lifting a finger is officially over. If programmers want their programs to go faster with each generation, they must make their programs more parallel.

The popular version of Moore's law—increasing performance with each generation of technology—is now up to programmers.

Pitfall *Falling prey to Amdahl's heartbreaking law.*

Virtually every practicing computer architect knows Amdahl's Law. Despite this, we almost all occasionally expend tremendous effort optimizing some feature before we measure its usage. Only when the overall speedup is disappointing do we recall that we should have measured first before we spent so much effort enhancing it!

Pitfall *A single point of failure.*

The calculations of reliability improvement using Amdahl's Law on page 53 show that dependability is no stronger than the weakest link in a chain. No matter how much more dependable we make the power supplies, as we did in our example, the single fan will limit the reliability of the disk subsystem. This Amdahl's Law observation led to a rule of thumb for fault-tolerant systems to make sure that every component was redundant so that no single component failure could bring down the whole system. Chapter 6 shows how a software layer avoids single points of failure inside WSCs.

Fallacy *Hardware enhancements that increase performance also improve energy efficiency, or are at worst energy neutral.*

Esmaeilzadeh et al. (2011) measured SPEC2006 on just one core of a 2.67 GHz Intel Core i7 using Turbo mode (Section 1.5). Performance increased by a factor of 1.07 when the clock rate increased to 2.94 GHz (or a factor of 1.10), but the i7 used a factor of 1.37 more joules and a factor of 1.47 more watt hours!

Fallacy *Benchmarks remain valid indefinitely.*

Several factors influence the usefulness of a benchmark as a predictor of real performance, and some change over time. A big factor influencing the usefulness of a benchmark is its ability to resist "benchmark engineering" or "benchmarketing." Once a benchmark becomes standardized and popular, there is tremendous pressure to improve performance by targeted optimizations or by aggressive interpretation of the rules for running the benchmark. Short kernels or programs that spend their time in a small amount of code are particularly vulnerable.

For example, despite the best intentions, the initial SPEC89 benchmark suite included a small kernel, called matrix300, which consisted of eight different 300×300 matrix multiplications. In this kernel, 99% of the execution time was in a single line (see SPEC, 1989). When an IBM compiler optimized this inner loop

(using a good idea called *blocking*, discussed in Chapters 2 and 4), performance improved by a factor of 9 over a prior version of the compiler! This benchmark tested compiler tuning and was not, of course, a good indication of overall performance, nor of the typical value of this particular optimization.

Figure 1.19 shows that if we ignore history, we may be forced to repeat it. SPEC Cint2006 had not been updated for a decade, giving compiler writers substantial time to hone their optimizers to this suite. Note that the SPEC ratios of all benchmarks but libquantum fall within the range of 16–52 for the AMD computer and from 22 to 78 for Intel. Libquantum runs about 250 times faster on AMD and 7300 times faster on Intel! This "miracle" is a result of optimizations by the Intel compiler that automatically parallelizes the code across 22 cores and optimizes memory by using bit packing, which packs together multiple narrow-range integers to save memory space and thus memory bandwidth. If we drop this benchmark and recalculate the geometric means, AMD SPEC Cint2006 falls from 31.9 to 26.5 and Intel from 63.7 to 41.4. The Intel computer is now about 1.5 times as fast as the AMD computer instead of 2.0 if we include libquantum, which is surely closer to their real relative performances. SPECCPU2017 dropped libquantum.

To illustrate the short lives of benchmarks, Figure 1.17 on page 43 lists the status of all 82 benchmarks from the various SPEC releases; Gcc is the lone survivor from SPEC89. Amazingly, about 70% of all programs from SPEC2000 or earlier were dropped from the next release.

Fallacy *The rated mean time to failure of disks is 1,200,000 hours or almost 140 years, so disks practically never fail.*

The current marketing practices of disk manufacturers can mislead users. How is such an MTTF calculated? Early in the process, manufacturers will put thousands of disks in a room, run them for a few months, and count the number that fail. They compute MTTF as the total number of hours that the disks worked cumulatively divided by the number that failed.

One problem is that this number far exceeds the lifetime of a disk, which is commonly assumed to be five years or 43,800 hours. For this large MTTF to make some sense, disk manufacturers argue that the model corresponds to a user who buys a disk and then keeps replacing the disk every 5 years—the planned lifetime of the disk. The claim is that if many customers (and their great-grandchildren) did this for the next century, on average they would replace a disk 27 times before a failure, or about 140 years.

A more useful measure is the percentage of disks that fail, which is called the *annual failure rate*. Assume 1000 disks with a 1,000,000-hour MTTF and that the disks are used 24 hours a day. If you replaced failed disks with a new one having the same reliability characteristics, the number that would fail in a year (8760 hours) is

$$\text{Failed disks} = \frac{\text{Number of disks} \times \text{Time period}}{\text{MTTF}} = \frac{1000 \text{ disks} \times 8760 \text{ hours/drive}}{1{,}000{,}000 \text{ hours/failure}} = 9$$

Stated alternatively, 0.9% would fail per year, or 4.4% over a 5-year lifetime.

Moreover, those high numbers are quoted assuming limited ranges of temperature and vibration; if they are exceeded, then all bets are off. A survey of disk drives in real environments (Gray and van Ingen, 2005) found that 3%–7% of drives failed per year, for an MTTF of about 125,000–300,000 hours. An even larger study found annual disk failure rates of 2%–10% (Pinheiro et al., 2007). Therefore the real-world MTTF is about 2–10 times worse than the manufacturer's MTTF.

Fallacy *Peak performance tracks observed performance.*

The only universally true definition of peak performance is "the performance level a computer is guaranteed not to exceed." Figure 1.24 shows the percentage of peak performance for four programs on four multiprocessors. It varies from 5% to 58%. Since the gap is so large and can vary significantly by benchmark, peak performance is not generally useful in predicting observed performance.

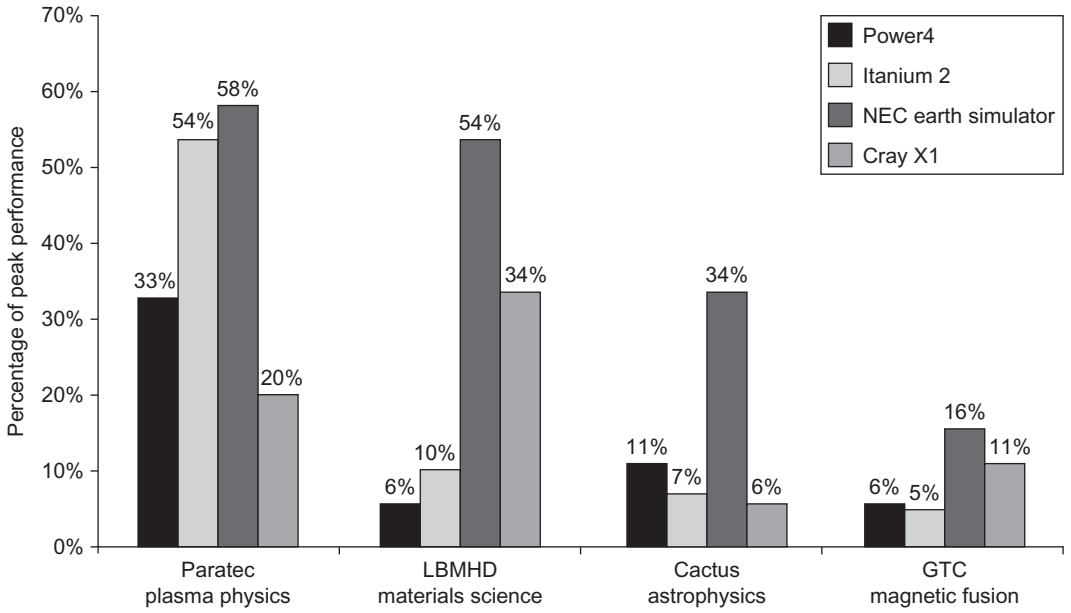


Figure 1.24 Percentage of peak performance for four programs on four multiprocessors scaled to 64 processors. The Earth Simulator and X1 are vector processors (see Chapter 4 and Appendix G). Not only did they deliver a higher fraction of peak performance, but they also had the highest peak performance and the lowest clock rates. Except for the Paratec program, the Power 4 and Itanium 2 systems delivered between 5% and 10% of their peak. From Oliker, L., Canning, A., Carter, J., Shalf, J., Ethier, S., 2004. Scientific computations on modern parallel vector systems. In: Proc. ACM/IEEE Conf. on Supercomputing, November 6–12, 2004, Pittsburgh, Penn., p. 10.

Pitfall *Fault detection can lower availability.*

This apparently ironic pitfall is because computer hardware has a fair amount of state that may not always be critical to proper operation. For example, it is not fatal if an error occurs in a branch predictor, because only performance may suffer.

In processors that try to exploit ILP aggressively, not all the operations are needed for correct execution of the program. Mukherjee et al. (2003) found that less than 30% of the operations were potentially on the critical path for the SPEC2000 benchmarks.

The same observation is true about programs. If a register is "dead" in a program—that is, the program will write the register before it is read again—then errors do not matter. If you were to crash the program upon detection of a transient fault in a dead register, it would lower availability unnecessarily.

The Sun Microsystems Division of Oracle lived this pitfall in 2000 with an L2 cache that included parity, but not error correction, in its Sun E3000 to Sun E10000 systems. The SRAMs they used to build the caches had intermittent faults, which parity detected. If the data in the cache were not modified, the processor would simply reread the data from the cache. Because the designers did not protect the cache with ECC (error-correcting code), the operating system had no choice but to report an error to dirty data and crash the program. Field engineers found no problems on inspection in more than 90% of the cases.

To reduce the frequency of such errors, Sun modified the Solaris operating system to "scrub" the cache by having a process that proactively wrote dirty data to memory. Because the processor chips did not have enough pins to add ECC, the only hardware option for dirty data was to duplicate the external cache, using the copy without the parity error to correct the error.

The pitfall is in detecting faults without providing a mechanism to correct them. These engineers are unlikely to design another computer without ECC on external caches.

1.12 Concluding Remarks

This chapter has introduced a number of concepts and provided a quantitative framework that we will expand on throughout the book. Starting with the last edition, energy efficiency is the constant companion to performance.

In Chapter 2, we start with the all-important area of memory system design. We will examine a wide range of techniques that conspire to make memory look infinitely large while still being as fast as possible. (Appendix B provides introductory material on caches for readers without much experience and background with them.) As in later chapters, we will see that hardware-software cooperation has become a key to high-performance memory systems, just as it has to high-performance pipelines. This chapter also covers virtual machines, an increasingly important technique for protection.

In Chapter 3, we look at ILP, of which pipelining is the simplest and most common form. Exploiting ILP is one of the most important techniques for building

high-speed uniprocessors. Chapter 3 begins with an extensive discussion of basic concepts that will prepare you for the wide range of ideas examined in both chapters. Chapter 3 uses examples that span about 40 years, drawing from one of the first supercomputers (IBM 360/91) to the fastest processors on the market in 2017. It emphasizes what is called the *dynamic* or *runtime approach* to exploiting ILP. It also talks about the limits to ILP ideas and introduces multithreading, which is further developed in both Chapters 4 and 5. Appendix C provides introductory material on pipelining for readers without much experience and background in pipelining. (We expect it to be a review for many readers, including those of our introductory text, *Computer Organization and Design: The Hardware/Software Interface*.)

Chapter 4 explains three ways to exploit data-level parallelism. The classic and oldest approach is vector architecture, and we start there to lay down the principles of SIMD design. (Appendix G goes into greater depth on vector architectures.) We next explain the SIMD instruction set extensions found in most desktop microprocessors today. The third piece is an in-depth explanation of how modern graphics processing units (GPUs) work. Most GPU descriptions are written from the programmer's perspective, which usually hides how the computer really works. This section explains GPUs from an insider's perspective, including a mapping between GPU jargon and more traditional architecture terms.

Chapter 5 focuses on the issue of achieving higher performance using multiple processors, or multiprocessors. Instead of using parallelism to overlap individual instructions, multiprocessing uses parallelism to allow multiple instruction streams to be executed simultaneously on different processors. Our focus is on the dominant form of multiprocessors, shared-memory multiprocessors, though we introduce other types as well and discuss the broad issues that arise in any multiprocessor. Here again we explore a variety of techniques, focusing on the important ideas first introduced in the 1980s and 1990s.

Chapter 6 introduces clusters and then goes into depth on WSCs, which computer architects help design. The designers of WSCs are the professional descendants of the pioneers of supercomputers, such as Seymour Cray, in that they are designing extreme computers. WSCs contain tens of thousands of servers, and the equipment and the building that holds them cost nearly $200 million. The concerns of price-performance and energy efficiency of the earlier chapters apply to WSCs, as does the quantitative approach to making decisions.

Chapter 7 is new to this edition. It introduces domain-specific architectures as the only path forward for improved performance and energy efficiency given the end of Moore's Law and Dennard scaling. It offers guidelines on how to build effective domain-specific architectures, introduces the exciting domain of deep neural networks, describes four recent examples that take very different approaches to accelerating neural networks, and then compares their cost-performance.

This book comes with an abundance of material online (see Preface for more details), both to reduce cost and to introduce readers to a variety of advanced topics. Figure 1.25 shows them all. Appendices A–C, which appear in the book, will be a review for many readers.

Appendix	Title
A	Instruction Set Principles
B	Review of Memory Hierarchies
C	Pipelining: Basic and Intermediate Concepts
D	Storage Systems
E	Embedded Systems
F	Interconnection Networks
G	Vector Processors in More Depth
H	Hardware and Software for VLIW and EPIC
I	Large-Scale Multiprocessors and Scientific Applications
J	Computer Arithmetic
K	Survey of Instruction Set Architectures
L	Advanced Concepts on Address Translation
M	Historical Perspectives and References

Figure 1.25 List of appendices.

In Appendix D, we move away from a processor-centric view and discuss issues in storage systems. We apply a similar quantitative approach, but one based on observations of system behavior and using an end-to-end approach to performance analysis. This appendix addresses the important issue of how to store and retrieve data efficiently using primarily lower-cost magnetic storage technologies. Our focus is on examining the performance of disk storage systems for typical I/O-intensive workloads, such as the OLTP benchmarks mentioned in this chapter. We extensively explore advanced topics in RAID-based systems, which use redundant disks to achieve both high performance and high availability. Finally, Appendix D introduces queuing theory, which gives a basis for trading off utilization and latency.

Appendix E applies an embedded computing perspective to the ideas of each of the chapters and early appendices.

Appendix F explores the topic of system interconnect broadly, including wide area and system area networks that allow computers to communicate.

Appendix H reviews VLIW hardware and software, which, in contrast, are less popular than when EPIC appeared on the scene just before the last edition.

Appendix I describes large-scale multiprocessors for use in high-performance computing.

Appendix J is the only appendix that remains from the first edition, and it covers computer arithmetic.

Appendix K provides a survey of instruction architectures, including the 80x86, the IBM 360, the VAX, and many RISC architectures, including ARM, MIPS, Power, RISC-V, and SPARC.

Appendix L is new and discusses advanced techniques for memory management, focusing on support for virtual machines and design of address translation

for very large address spaces. With the growth in cloud processors, these architectural enhancements are becoming more important.

We describe Appendix M next.

1.13 Historical Perspectives and References

Appendix M (available online) includes historical perspectives on the key ideas presented in each of the chapters in this text. These historical perspective sections allow us to trace the development of an idea through a series of machines or to describe significant projects. If you're interested in examining the initial development of an idea or processor or want further reading, references are provided at the end of each history. For this chapter, see Section M.2, "The Early Development of Computers," for a discussion on the early development of digital computers and performance measurement methodologies.

As you read the historical material, you'll soon come to realize that one of the important benefits of the youth of computing, compared to many other engineering fields, is that some of the pioneers are still alive—we can learn the history by simply asking them!

Case Studies and Exercises by Diana Franklin

Case Study 1: Chip Fabrication Cost

Concepts illustrated by this case study

- Fabrication Cost
- Fabrication Yield
- Defect Tolerance Through Redundancy

Many factors are involved in the price of a computer chip. Intel is spending $7 billion to complete its Fab 42 fabrication facility for 7 nm technology. In this case study, we explore a hypothetical company in the same situation and how different design decisions involving fabrication technology, area, and redundancy affect the cost of chips.

1.1 [10/10] <1.6> Figure 1.26 gives hypothetical relevant chip statistics that influence the cost of several current chips. In the next few exercises, you will be exploring the effect of different possible design decisions for the Intel chips.

Chip	Die Size (mm^2)	Estimated defect rate (per cm^2)	*N*	Manufacturing size (nm)	Transistors (billion)	Cores
BlueDragon	180	0.03	12	10	7.5	4
RedDragon	120	0.04	14	7	7.5	4
Phoenix[8]	200	0.04	14	7	12	8

Figure 1.26 Manufacturing cost factors for several hypothetical current and future processors.

a. [10] <1.6> What is the yield for the Phoenix chip?

b. [10] <1.6> Why does Phoenix have a higher defect rate than BlueDragon?

1.2 [20/20/20/20] <1.6> They will sell a range of chips from that factory, and they need to decide how much capacity to dedicate to each chip. Imagine that they will sell two chips. Phoenix is a completely new architecture designed with 7 nm technology in mind, whereas RedDragon is the same architecture as their 10 nm BlueDragon. Imagine that RedDragon will make a profit of $15 per defect-free chip. Phoenix will make a profit of $30 per defect-free chip. Each wafer has a 450 mm diameter.

a. [20] <1.6> How much profit do you make on each wafer of Phoenix chips?

b. [20] <1.6> How much profit do you make on each wafer of RedDragon chips?

c. [20] <1.6> If your demand is 50,000 RedDragon chips per month and 25,000 Phoenix chips per month, and your facility can fabricate 70 wafers a month, how many wafers should you make of each chip?

1.3 [20/20] <1.6> Your colleague at AMD suggests that, since the yield is so poor, you might make chips more cheaply if you released multiple versions of the same chip, just with different numbers of cores. For example, you could sell Phoenix^8, Phoenix^4, Phoenix^2, and Phoenix^1, which contain 8, 4, 2, and 1 cores on each chip, respectively. If all eight cores are defect-free, then it is sold as Phoenix^8. Chips with four to seven defect-free cores are sold as Phoenix^4, and those with two or three defect-free cores are sold as Phoenix^2. For simplification, calculate the yield for a single core as the yield for a chip that is 1/8 the area of the original Phoenix chip. Then view that yield as an independent probability of a single core being defect free. Calculate the yield for each configuration as the probability of at the corresponding number of cores being defect free.

a. [20] <1.6> What is the yield for a single core being defect free as well as the yield for Phoenix^4, Phoenix^2 and Phoenix^1?

b. [5] <1.6> Using your results from part a, determine which chips you think it would be worthwhile to package and sell, and why.

c. [10] <1.6> If it previously cost $20 dollars per chip to produce Phoenix^8, what will be the cost of the new Phoenix chips, assuming that there are no additional costs associated with rescuing them from the trash?

d. [20] <1.6> You currently make a profit of $30 for each defect-free Phoenix^8, and you will sell each Phoenix^4 chip for $25. How much is your profit per Phoenix^8 chip if you consider (i) the purchase price of Phoenix^4 chips to be entirely profit and (ii) apply the profit of Phoenix^4 chips to each Phoenix^8 chip in proportion to how many are produced? Use the yields calculated from part Problem 1.3a, not from problem 1.1a.

Case Study 2: Power Consumption in Computer Systems

Concepts illustrated by this case study

- Amdahl's Law
- Redundancy
- MTTF
- Power Consumption

Power consumption in modern systems is dependent on a variety of factors, including the chip clock frequency, efficiency, and voltage. The following exercises explore the impact on power and energy that different design decisions and use scenarios have.

1.4 [10/10/10/10] <1.5> A cell phone performs very different tasks, including streaming music, streaming video, and reading email. These tasks perform very different computing tasks. Battery life and overheating are two common problems for cell phones, so reducing power and energy consumption are critical. In this problem, we consider what to do when the user is not using the phone to its full computing capacity. For these problems, we will evaluate an unrealistic scenario in which the cell phone has no specialized processing units. Instead, it has a quad-core, general-purpose processing unit. Each core uses 0.5 W at full use. For email-related tasks, the quad-core is 8× as fast as necessary.

a. [10] <1.5> How much dynamic energy and power are required compared to running at full power? First, suppose that the quad-core operates for 1/8 of the time and is idle for the rest of the time. That is, the clock is disabled for 7/8 of the time, with no leakage occurring during that time. Compare total dynamic energy as well as dynamic power while the core is running.

b. [10] <1.5> How much dynamic energy and power are required using frequency and voltage scaling? Assume frequency and voltage are both reduced to 1/8 the entire time.

c. [10] <1.6, 1.9> Now assume the voltage may not decrease below 50% of the original voltage. This voltage is referred to as the *voltage floor*, and any voltage lower than that will lose the state. Therefore, while the frequency can keep decreasing, the voltage cannot. What are the dynamic energy and power savings in this case?

d. [10] <1.5> How much energy is used with a dark silicon approach? This involves creating specialized ASIC hardware for each major task and power

gating those elements when not in use. Only one general-purpose core would be provided, and the rest of the chip would be filled with specialized units. For email, the one core would operate for 25% the time and be turned completely off with power gating for the other 75% of the time. During the other 75% of the time, a specialized ASIC unit that requires 20% of the energy of a core would be running.

1.5 [10/10/10] <1.5> As mentioned in Exercise 1.4, cell phones run a wide variety of applications. We'll make the same assumptions for this exercise as the previous one, that it is 0.5 W per core and that a quad core runs email 3× as fast.

a. [10] <1.5> Imagine that 80% of the code is parallelizable. By how much would the frequency and voltage on a single core need to be increased in order to execute at the same speed as the four-way parallelized code?

b. [10] <1.5> What is the reduction in dynamic energy from using frequency and voltage scaling in part a?

c. [10] <1.5> How much energy is used with a dark silicon approach? In this approach, all hardware units are power gated, allowing them to turn off entirely (causing no leakage). Specialized ASICs are provided that perform the same computation for 20% of the power as the general-purpose processor. Imagine that each core is power gated. The video game requires two ASICS and two cores. How much dynamic energy does it require compared to the baseline of parallelized on four cores?

1.6 [10/10/10/10/10/20] <1.5,1.9> General-purpose processes are optimized for general-purpose computing. That is, they are optimized for behavior that is generally found across a large number of applications. However, once the domain is restricted somewhat, the behavior that is found across a large number of the target applications may be different from general-purpose applications. One such application is deep learning or neural networks. Deep learning can be applied to many different applications, but the fundamental building block of inference—using the learned information to make decisions—is the same across them all. Inference operations are largely parallel, so they are currently performed on graphics processing units, which are specialized more toward this type of computation, and not to inference in particular. In a quest for more performance per watt, Google has created a custom chip using tensor processing units to accelerate inference operations in deep learning.[1] This approach can be used for speech recognition and image recognition, for example. This problem explores the trade-offs between this process, a general-purpose processor (Haswell E5-2699 v3) and a GPU (NVIDIA K80), in terms of performance and cooling. If heat is not removed from the computer efficiently, the fans will blow hot air back onto the computer, not cold air. Note: The differences are more than processor—on-chip memory and DRAM also come into play. Therefore statistics are at a system level, not a chip level.

[1]Cite paper at this website: https://drive.google.com/file/d/0Bx4hafXDDq2EMzRNcy1vSUxtcEk/view.

a. [10] <1.9> If Google's data center spends 70% of its time on workload A and 30% of its time on workload B when running GPUs, what is the speedup of the TPU system over the GPU system?

b. [10] <1.9> If Google's data center spends 70% of its time on workload A and 30% of its time on workload B when running GPUs, what percentage of Max IPS does it achieve for each of the three systems?

c. [15] <1.5, 1.9> Building on (b), assuming that the power scales linearly from idle to busy power as IPS grows from 0% to 100%, what is the performance per watt of the TPU system over the GPU system?

d. [10] <1.9> If another data center spends 40% of its time on workload A, 10% of its time on workload B, and 50% of its time on workload C, what are the speedups of the GPU and TPU systems over the general-purpose system?

e. [10] <1.5> A cooling door for a rack costs $4000 and dissipates 14 kW (into the room; additional cost is required to get it out of the room). How many Haswell-, NVIDIA-, or Tensor-based servers can you cool with one cooling door, assuming TDP in Figures 1.27 and 1.28?

f. [20] <1.5> Typical server farms can dissipate a maximum of 200 W per square foot. Given that a server rack requires 11 square feet (including front and back clearance), how many servers from part (e) can be placed on a single rack, and how many cooling doors are required?

System	Chip	TDP	Idle power	Busy power
General-purpose	Haswell E5-2699 v3	504 W	159 W	455 W
Graphics processor	NVIDIA K80	1838 W	357 W	991 W
Custom ASIC	TPU	861 W	290 W	384 W

Figure 1.27 Hardware characteristics for general-purpose processor, graphical processing unit-based or custom ASIC-based system, including measured power (cite ISCA paper).

System	Chip	Throughput			% Max IPS		
		A	B	C	A	B	C
General-purpose	Haswell E5-2699 v3	5482	13,194	12,000	42%	100%	90%
Graphics processor	NVIDIA K80	13,461	36,465	15,000	37%	100%	40%
Custom ASIC	TPU	225,000	280,000	2000	80%	100%	1%

Figure 1.28 Performance characteristics for general-purpose processor, graphical processing unit-based or custom ASIC-based system on two neural-net workloads (cite ISCA paper). Workloads A and B are from published results. Workload C is a fictional, more general-purpose application.

Exercises

1.7 [10/15/15/10/10] <1.4, 1.5> One challenge for architects is that the design created today will require several years of implementation, verification, and testing before appearing on the market. This means that the architect must project what the technology will be like several years in advance. Sometimes, this is difficult to do.

a. [10] <1.4> According to the trend in device scaling historically observed by Moore's Law, the number of transistors on a chip in 2025 should be how many times the number in 2015?

b. [15] <1.5> The increase in performance once mirrored this trend. Had performance continued to climb at the same rate as in the 1990s, approximately what performance would chips have over the VAX-11/780 in 2025?

c. [15] <1.5> At the current rate of increase of the mid-2000s, what is a more updated projection of performance in 2025?

d. [10] <1.4> What has limited the rate of growth of the clock rate, and what are architects doing with the extra transistors now to increase performance?

e. [10] <1.4> The rate of growth for DRAM capacity has also slowed down. For 20 years, DRAM capacity improved by 60% each year. If 8 Gbit DRAM was first available in 2015, and 16 Gbit is not available until 2019, what is the current DRAM growth rate?

1.8 [10/10] <1.5> You are designing a system for a real-time application in which specific deadlines must be met. Finishing the computation faster gains nothing. You find that your system can execute the necessary code, in the worst case, twice as fast as necessary.

a. [10] <1.5> How much energy do you save if you execute at the current speed and turn off the system when the computation is complete?

b. [10] <1.5> How much energy do you save if you set the voltage and frequency to be half as much?

1.9 [10/10/20/20] <1.5> Server farms such as Google and Yahoo! provide enough compute capacity for the highest request rate of the day. Imagine that most of the time these servers operate at only 60% capacity. Assume further that the power does not scale linearly with the load; that is, when the servers are operating at 60% capacity, they consume 90% of maximum power. The servers could be turned off, but they would take too long to restart in response to more load. A new system has been proposed that allows for a quick restart but requires 20% of the maximum power while in this "barely alive" state.

a. [10] <1.5> How much power savings would be achieved by turning off 60% of the servers?

b. [10] <1.5> How much power savings would be achieved by placing 60% of the servers in the "barely alive" state?

c. [20] <1.5> How much power savings would be achieved by reducing the voltage by 20% and frequency by 40%?

d. [20] <1.5> How much power savings would be achieved by placing 30% of the servers in the "barely alive" state and 30% off?

1.10 [10/10/20] <1.7> Availability is the most important consideration for designing servers, followed closely by scalability and throughput.

a. [10] <1.7> We have a single processor with a failure in time (FIT) of 100. What is the mean time to failure (MTTF) for this system?

b. [10] <1.7> If it takes one day to get the system running again, what is the availability of the system?

c. [20] <1.7> Imagine that the government, to cut costs, is going to build a supercomputer out of inexpensive computers rather than expensive, reliable computers. What is the MTTF for a system with 1000 processors? Assume that if one fails, they all fail.

1.11 [20/20/20] <1.1, 1.2, 1.7> In a server farm such as that used by Amazon or eBay, a single failure does not cause the entire system to crash. Instead, it will reduce the number of requests that can be satisfied at any one time.

a. [20] <1.7> If a company has 10,000 computers, each with an MTTF of 35 days, and it experiences catastrophic failure only if 1/3 of the computers fail, what is the MTTF for the system?

b. [20] <1.1, 1.7> If it costs an extra $1000, per computer, to double the MTTF, would this be a good business decision? Show your work.

c. [20] <1.2> Figure 1.3 shows, on average, the cost of downtimes, assuming that the cost is equal at all times of the year. For retailers, however, the Christmas season is the most profitable (and therefore the most costly time to lose sales). If a catalog sales center has twice as much traffic in the fourth quarter as every other quarter, what is the average cost of downtime per hour during the fourth quarter and the rest of the year?

1.12 [20/10/10/10/15] <1.9> In this exercise, assume that we are considering enhancing a quad-core machine by adding encryption hardware to it. When computing encryption operations, it is 20 times faster than the normal mode of execution. We will define percentage of encryption as the percentage of time in the original execution that is spent performing encryption operations. The specialized hardware increases power consumption by 2%.

a. [20] <1.9> Draw a graph that plots the speedup as a percentage of the computation spent performing encryption. Label the y-axis "Net speedup" and label the x-axis "Percent encryption."

b. [10] <1.9> With what percentage of encryption will adding encryption hardware result in a speedup of 2?

c. [10] <1.9> What percentage of time in the new execution will be spent on encryption operations if a speedup of 2 is achieved?

d. [15] <1.9> Suppose you have measured the percentage of encryption to be 50%. The hardware design group estimates it can speed up the encryption hardware even more with significant additional investment. You wonder whether adding a second unit in order to support parallel encryption operations would be more useful. Imagine that in the original program, 90% of the encryption operations could be performed in parallel. What is the speedup of providing two or four encryption units, assuming that the parallelization allowed is limited to the number of encryption units?

1.13 [15/10] <1.9> Assume that we make an enhancement to a computer that improves some mode of execution by a factor of 10. Enhanced mode is used 50% of the time, measured as a percentage of the execution time *when the enhanced mode is in use*. Recall that Amdahl's Law depends on the fraction of the original, unenhanced execution time that could make use of enhanced mode. Thus we cannot directly use this 50% measurement to compute speedup with Amdahl's Law.

a. [15] <1.9> What is the speedup we have obtained from fast mode?

b. [10] <1.9> What percentage of the original execution time has been converted to fast mode?

1.14 [20/20/15] <1.9> When making changes to optimize part of a processor, it is often the case that speeding up one type of instruction comes at the cost of slowing down something else. For example, if we put in a complicated fast floating-point unit, that takes space, and something might have to be moved farther away from the middle to accommodate it, adding an extra cycle in delay to reach that unit. The basic Amdahl's Law equation does not take into account this trade-off.

a. [20] <1.9> If the new fast floating-point unit speeds up floating-point operations by, on average, 2x, and floating-point operations take 20% of the original program's execution time, what is the overall speedup (ignoring the penalty to any other instructions)?

b. [20] <1.9> Now assume that speeding up the floating-point unit slowed down data cache accesses, resulting in a 1.5x slowdown (or 2/3 speedup). Data cache accesses consume 10% of the execution time. What is the overall speedup now?

c. [15] <1.9> After implementing the new floating-point operations, what percentage of execution time is spent on floating-point operations? What percentage is spent on data cache accesses?

1.15 [10/10/20/20] <1.10> Your company has just bought a new 22-core processor, and you have been tasked with optimizing your software for this processor. You will run four applications on this system, but the resource requirements are not equal. Assume the system and application characteristics listed in Table 1.1.

Table 1.1 Four applications

Application	A	B	C	D
% resources needed	41	27	18	14
% parallelizable	50	80	60	90

The percentage of resources of assuming they are all run in serial. Assume that when you parallelize a portion of the program by X, the speedup for that portion is X.

a. [10] <1.10> How much speedup would result from running application A on the entire 22-core processor, as compared to running it serially?

b. [10] <1.10> How much speedup would result from running application D on the entire 22-core processor, as compared to running it serially?

c. [20] <1.10> Given that application A requires 41% of the resources, if we statically assign it 41% of the cores, what is the overall speedup if A is run parallelized but everything else is run serially?

d. [20] <1.10> What is the overall speedup if all four applications are statically assigned some of the cores, relative to their percentage of resource needs, and all run parallelized?

e. [10] <1.10> Given acceleration through parallelization, what new percentage of the resources are the applications receiving, considering only active time on their statically-assigned cores?

1.16 [10/20/20/20/25] <1.10> When parallelizing an application, the ideal speedup is speeding up by the number of processors. This is limited by two things: percentage of the application that can be parallelized and the cost of communication. Amdahl's Law takes into account the former but not the latter.

a. [10] <1.10> What is the speedup with *N* processors if 80% of the application is parallelizable, ignoring the cost of communication?

b. [20] <1.10> What is the speedup with eight processors if, for every processor added, the communication overhead is 0.5% of the original execution time.

c. [20] <1.10> What is the speedup with eight processors if, for every time the number of processors is doubled, the communication overhead is increased by 0.5% of the original execution time?

d. [20] <1.10> What is the speedup with *N* processors if, for every time the number of processors is doubled, the communication overhead is increased by 0.5% of the original execution time?

e. [25] <1.10> Write the general equation that solves this question: What is the number of processors with the highest speedup in an application in which P% of the original execution time is parallelizable, and, for every time the number of processors is doubled, the communication is increased by 0.5% of the original execution time?

2.1	Introduction	78
2.2	Memory Technology and Optimizations	84
2.3	Ten Advanced Optimizations of Cache Performance	94
2.4	Virtual Memory and Virtual Machines	118
2.5	Cross-Cutting Issues: The Design of Memory Hierarchies	126
2.6	Putting It All Together: Memory Hierarchies in the ARM Cortex-A53 and Intel Core i7 6700	129
2.7	Fallacies and Pitfalls	142
2.8	Concluding Remarks: Looking Ahead	146
2.9	Historical Perspectives and References	148
	Case Studies and Exercises by Norman P. Jouppi, Rajeev Balasubramonian, Naveen Muralimanohar, and Sheng Li	148

2

Memory Hierarchy Design

Ideally one would desire an indefinitely large memory capacity such that any particular… word would be immediately available… We are… forced to recognize the possibility of constructing a hierarchy of memories each of which has greater capacity than the preceding but which is less quickly accessible.

A. W. Burks, H. H. Goldstine, and J. von Neumann, *Preliminary Discussion of the Logical Design of an Electronic Computing Instrument* (1946).

Computer Architecture. https://doi.org/10.1016/B978-0-12-811905-1.00002-X
© 2019 Elsevier Inc. All rights reserved.

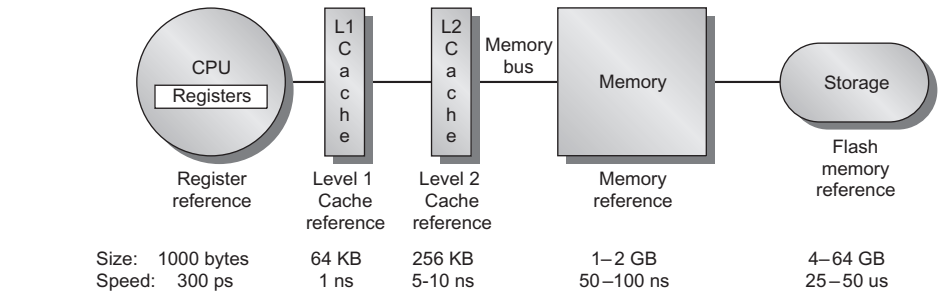
2.1 Introduction

Computer pioneers correctly predicted that programmers would want unlimited amounts of fast memory. An economical solution to that desire is a memory hierarchy, which takes advantage of locality and trade-offs in the cost-performance of memory technologies. The principle of locality, presented in the first chapter, says that most programs do not access all code or data uniformly. Locality occurs in time (temporal locality) and in space (spatial locality). This principle plus the guideline that for a given implementation technology and power budget, smaller hardware can be made faster led to hierarchies based on memories of different speeds and sizes. Figure 2.1 shows several different multilevel memory hierarchies, including typical sizes and speeds of access. As Flash and next generation memory technologies continue to close the gap with disks in cost per bit, such technologies are likely to increasingly replace magnetic disks for secondary storage. As Figure 2.1 shows, these technologies are already used in many personal computers and increasingly in servers, where the advantages in performance, power, and density are significant.

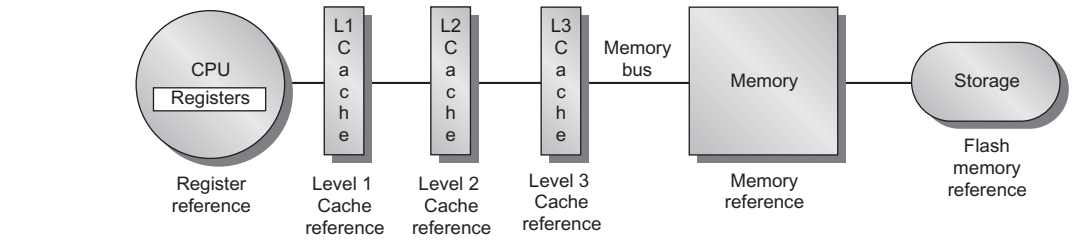
Because fast memory is more expensive, a memory hierarchy is organized into several levels—each smaller, faster, and more expensive per byte than the next lower level, which is farther from the processor. The goal is to provide a memory system with a cost per byte that is almost as low as the cheapest level of memory and a speed almost as fast as the fastest level. In most cases (but not all), the data contained in a lower level are a superset of the next higher level. This property, called the *inclusion property*, is always required for the lowest level of the hierarchy, which consists of main memory in the case of caches and secondary storage (disk or Flash) in the case of virtual memory.

The importance of the memory hierarchy has increased with advances in performance of processors. Figure 2.2 plots single processor performance projections against the historical performance improvement in time to access main memory. The processor line shows the increase in memory requests per second on average (i.e., the inverse of the latency between memory references), while the memory line shows the increase in DRAM accesses per second (i.e., the inverse of the DRAM access latency), assuming a single DRAM and a single memory bank. The reality is more complex because the processor request rate is not uniform, and the memory system typically has multiple banks of DRAMs and channels. Although the gap in access time increased significantly for many years, the lack of significant performance improvement in single processors has led to a slowdown in the growth of the gap between processors and DRAM.

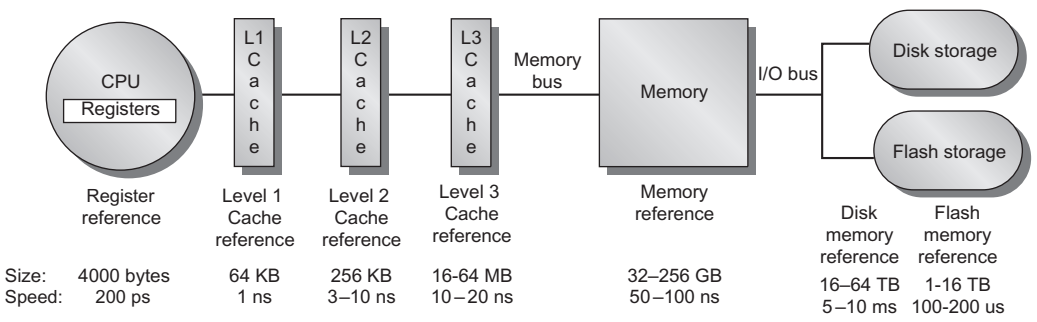
Because high-end processors have multiple cores, the bandwidth requirements are greater than for single cores. Although single-core bandwidth has grown more slowly in recent years, the gap between CPU memory demand and DRAM bandwidth continues to grow as the numbers of cores grow. A modern high-end desktop processor such as the Intel Core i7 6700 can generate two data memory references per core each clock cycle. With four cores and a 4.2 GHz clock rate, the i7 can generate a peak of 32.8 billion 64-bit data memory references per second, in addition to a peak instruction demand of about 12.8 billion 128-bit instruction



(A) Memory hierarchy for a personal mobile device



(B) Memory hierarchy for a laptop or a desktop



(C) Memory hierarchy for server

Figure 2.1 The levels in a typical memory hierarchy in a personal mobile device (PMD), such as a cell phone or tablet (A), in a laptop or desktop computer (B), and in a server (C). As we move farther away from the processor, the memory in the level below becomes slower and larger. Note that the time units change by a factor of 10^9 from picoseconds to milliseconds in the case of magnetic disks and that the size units change by a factor of 10^{10} from thousands of bytes to tens of terabytes. If we were to add warehouse-sized computers, as opposed to just servers, the capacity scale would increase by three to six orders of magnitude. Solid-state drives (SSDs) composed of Flash are used exclusively in PMDs, and heavily in both laptops and desktops. In many desktops, the primary storage system is SSD, and expansion disks are primarily hard disk drives (HDDs). Likewise, many servers mix SSDs and HDDs.

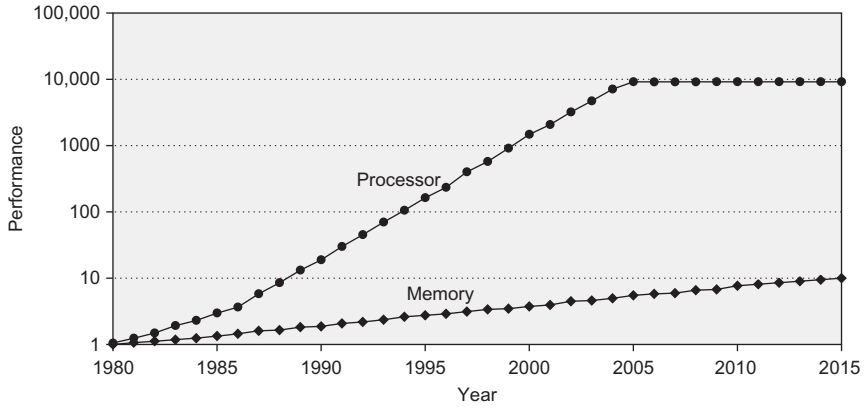


Figure 2.2 Starting with 1980 performance as a baseline, the gap in performance, measured as the difference in the time between processor memory requests (for a single processor or core) and the latency of a DRAM access, is plotted over time. In mid-2017, AMD, Intel and Nvidia all announced chip sets using versions of HBM technology. Note that the vertical axis must be on a logarithmic scale to record the size of the processor-DRAM performance gap. The memory baseline is 64 KiB DRAM in 1980, with a 1.07 per year performance improvement in latency (see Figure 2.4 on page 88). The processor line assumes a 1.25 improvement per year until 1986, a 1.52 improvement until 2000, a 1.20 improvement between 2000 and 2005, and only small improvements in processor performance (on a per-core basis) between 2005 and 2015. As you can see, until 2010 memory access times in DRAM improved slowly but consistently; since 2010 the improvement in access time has reduced, as compared with the earlier periods, although there have been continued improvements in bandwidth. See Figure 1.1 in Chapter 1 for more information.

references; this is a total peak demand bandwidth of 409.6 GiB/s! This incredible bandwidth is achieved by multiporting and pipelining the caches; by using three levels of caches, with two private levels per core and a shared L3; and by using a separate instruction and data cache at the first level. In contrast, the peak bandwidth for DRAM main memory, using two memory channels, is only 8% of the demand bandwidth (34.1 GiB/s). Upcoming versions are expected to have an L4 DRAM cache using embedded or stacked DRAM (see Sections 2.2 and 2.3).

Traditionally, designers of memory hierarchies focused on optimizing average memory access time, which is determined by the cache access time, miss rate, and miss penalty. More recently, however, power has become a major consideration. In high-end microprocessors, there may be 60 MiB or more of on-chip cache, and a large second- or third-level cache will consume significant power both as leakage when not operating (called *static power*) and as active power, as when performing a read or write (called *dynamic power*), as described in Section 2.3. The problem is even more acute in processors in PMDs where the CPU is less aggressive and the power budget may be 20 to 50 times smaller. In such cases, the caches can account for 25% to 50% of the total power consumption. Thus more designs must consider both performance and power trade-offs, and we will examine both in this chapter.

Basics of Memory Hierarchies: A Quick Review

The increasing size and thus importance of this gap led to the migration of the basics of memory hierarchy into undergraduate courses in computer architecture, and even to courses in operating systems and compilers. Thus we'll start with a quick review of caches and their operation. The bulk of the chapter, however, describes more advanced innovations that attack the processor—memory performance gap.

When a word is not found in the cache, the word must be fetched from a lower level in the hierarchy (which may be another cache or the main memory) and placed in the cache before continuing. Multiple words, called a *block* (or *line*), are moved for efficiency reasons, and because they are likely to be needed soon due to spatial locality. Each cache block includes a *tag* to indicate which memory address it corresponds to.

A key design decision is where blocks (or lines) can be placed in a cache. The most popular scheme is *set associative*, where a *set* is a group of blocks in the cache. A block is first mapped onto a set, and then the block can be placed anywhere within that set. Finding a block consists of first mapping the block address to the set and then searching the set—usually in parallel—to find the block. The set is chosen by the address of the data:

$$(\textit{Block address}) \text{ MOD } (\textit{Number of sets in cache})$$

If there are n blocks in a set, the cache placement is called *n-way set associative*. The end points of set associativity have their own names. A *direct-mapped* cache has just one block per set (so a block is always placed in the same location), and a *fully associative* cache has just one set (so a block can be placed anywhere).

Caching data that is only read is easy because the copy in the cache and memory will be identical. Caching writes is more difficult; for example, how can the copy in the cache and memory be kept consistent? There are two main strategies. A *write-through* cache updates the item in the cache *and* writes through to update main memory. A *write-back* cache only updates the copy in the cache. When the block is about to be replaced, it is copied back to memory. Both write strategies can use a *write buffer* to allow the cache to proceed as soon as the data are placed in the buffer rather than wait for full latency to write the data into memory.

One measure of the benefits of different cache organizations is miss rate. *Miss rate* is simply the fraction of cache accesses that result in a miss—that is, the number of accesses that miss divided by the number of accesses.

To gain insights into the causes of high miss rates, which can inspire better cache designs, the three Cs model sorts all misses into three simple categories:

- *Compulsory*—The very first access to a block *cannot* be in the cache, so the block must be brought into the cache. Compulsory misses are those that occur even if you were to have an infinite-sized cache.
- *Capacity*—If the cache cannot contain all the blocks needed during execution of a program, capacity misses (in addition to compulsory misses) will occur because of blocks being discarded and later retrieved.

- *Conflict*—If the block placement strategy is not fully associative, conflict misses (in addition to compulsory and capacity misses) will occur because a block may be discarded and later retrieved if multiple blocks map to its set and accesses to the different blocks are intermingled.

Figure B.8 on page 24 shows the relative frequency of cache misses broken down by the three Cs. As mentioned in Appendix B, the three C's model is conceptual, and although its insights usually hold, it is not a definitive model for explaining the cache behavior of individual references.

As we will see in Chapters 3 and 5, multithreading and multiple cores add complications for caches, both increasing the potential for capacity misses as well as adding a fourth C, for *coherency* misses due to cache flushes to keep multiple caches coherent in a multiprocessor; we will consider these issues in Chapter 5.

However, miss rate can be a misleading measure for several reasons. Therefore some designers prefer measuring *misses per instruction* rather than misses per memory reference (miss rate). These two are related:

$$\frac{\text{Misses}}{\text{Instruction}} = \frac{\text{Miss rate} \times \text{Memory accesses}}{\text{Instruction count}} = \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}}$$

(This equation is often expressed in integers rather than fractions, as misses per 1000 instructions.)

The problem with both measures is that they don't factor in the cost of a miss. A better measure is the *average memory access time*,

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

where *hit time* is the time to hit in the cache and *miss penalty* is the time to replace the block from memory (that is, the cost of a miss). Average memory access time is still an indirect measure of performance; although it is a better measure than miss rate, it is not a substitute for execution time. In Chapter 3 we will see that speculative processors may execute other instructions during a miss, thereby reducing the effective miss penalty. The use of multithreading (introduced in Chapter 3) also allows a processor to tolerate misses without being forced to idle. As we will examine shortly, to take advantage of such latency tolerating techniques, we need caches that can service requests while handling an outstanding miss.

If this material is new to you, or if this quick review moves too quickly, see Appendix B. It covers the same introductory material in more depth and includes examples of caches from real computers and quantitative evaluations of their effectiveness.

Section B.3 in Appendix B presents six basic cache optimizations, which we quickly review here. The appendix also gives quantitative examples of the benefits of these optimizations. We also comment briefly on the power implications of these trade-offs.

1. *Larger block size to reduce miss rate*—The simplest way to reduce the miss rate is to take advantage of spatial locality and increase the block size. Larger blocks

reduce compulsory misses, but they also increase the miss penalty. Because larger blocks lower the number of tags, they can slightly reduce static power. Larger block sizes can also increase capacity or conflict misses, especially in smaller caches. Choosing the right block size is a complex trade-off that depends on the size of cache and the miss penalty.

2. *Bigger caches to reduce miss rate*—The obvious way to reduce capacity misses is to increase cache capacity. Drawbacks include potentially longer hit time of the larger cache memory and higher cost and power. Larger caches increase both static and dynamic power.
3. *Higher associativity to reduce miss rate*—Obviously, increasing associativity reduces conflict misses. Greater associativity can come at the cost of increased hit time. As we will see shortly, associativity also increases power consumption.
4. *Multilevel caches to reduce miss penalty*—A difficult decision is whether to make the cache hit time fast, to keep pace with the high clock rate of processors, or to make the cache large to reduce the gap between the processor accesses and main memory accesses. Adding another level of cache between the original cache and memory simplifies the decision. The first-level cache can be small enough to match a fast clock cycle time, yet the second-level (or third-level) cache can be large enough to capture many accesses that would go to main memory. The focus on misses in second-level caches leads to larger blocks, bigger capacity, and higher associativity. Multilevel caches are more power-efficient than a single aggregate cache. If L1 and L2 refer, respectively, to first- and second-level caches, we can redefine the average memory access time:

$$\text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2})$$

5. *Giving priority to read misses over writes to reduce miss penalty*—A write buffer is a good place to implement this optimization. Write buffers create hazards because they hold the updated value of a location needed on a read miss—that is, a read-after-write hazard through memory. One solution is to check the contents of the write buffer on a read miss. If there are no conflicts, and if the memory system is available, sending the read before the writes reduces the miss penalty. Most processors give reads priority over writes. This choice has little effect on power consumption.
6. *Avoiding address translation during indexing of the cache to reduce hit time*—Caches must cope with the translation of a virtual address from the processor to a physical address to access memory. (Virtual memory is covered in Sections 2.4 and B.4.) A common optimization is to use the page offset—the part that is identical in both virtual and physical addresses—to index the cache, as described in Appendix B, page B.38. This virtual index/physical tag method introduces some system complications and/or limitations on the size and structure of the L1 cache, but the advantages of removing the translation lookaside buffer (TLB) access from the critical path outweigh the disadvantages.

Note that each of the preceding six optimizations has a potential disadvantage that can lead to increased, rather than decreased, average memory access time.

The rest of this chapter assumes familiarity with the preceding material and the details in Appendix B. In the "Putting It All Together" section, we examine the memory hierarchy for a microprocessor designed for a high-end desktop or smaller server, the Intel Core i7 6700, as well as one designed for use in a PMD, the Arm Cortex-53, which is the basis for the processor used in several tablets and smartphones. Within each of these classes, there is a significant diversity in approach because of the intended use of the computer.

Although the i7 6700 has more cores and bigger caches than the Intel processors designed for mobile uses, the processors have similar architectures. A processor designed for small servers, such as the i7 6700, or larger servers, such as the Intel Xeon processors, typically is running a large number of concurrent processes, often for different users. Thus memory bandwidth becomes more important, and these processors offer larger caches and more aggressive memory systems to boost that bandwidth.

In contrast, PMDs not only serve one user but generally also have smaller operating systems, usually less multitasking (running of several applications simultaneously), and simpler applications. PMDs must consider both performance and energy consumption, which determines battery life. Before we dive into more advanced cache organizations and optimizations, one needs to understand the various memory technologies and how they are evolving.

2.2 Memory Technology and Optimizations

…the one single development that put computers on their feet was the invention of a reliable form of memory, namely, the core memory. …Its cost was reasonable, it was reliable and, because it was reliable, it could in due course be made large. (p. 209)

Maurice Wilkes.
Memoirs of a Computer Pioneer (1985)

This section describes the technologies used in a memory hierarchy, specifically in building caches and main memory. These technologies are SRAM (static random-access memory), DRAM (dynamic random-access memory), and Flash. The last of these is used as an alternative to hard disks, but because its characteristics are based on semiconductor technology, it is appropriate to include in this section.

Using SRAM addresses the need to minimize access time to caches. When a cache miss occurs, however, we need to move the data from the main memory as quickly as possible, which requires a high bandwidth memory. This high memory bandwidth can be achieved by organizing the many DRAM chips that make up the main memory into multiple memory banks and by making the memory bus wider, or by doing both.

To allow memory systems to keep up with the bandwidth demands of modern processors, memory innovations started happening inside the DRAM chips

themselves. This section describes the technology inside the memory chips and those innovative, internal organizations. Before describing the technologies and options, we need to introduce some terminology.

With the introduction of burst transfer memories, now widely used in both Flash and DRAM, memory latency is quoted using two measures—access time and cycle time. *Access time* is the time between when a read is requested and when the desired word arrives, and *cycle time* is the minimum time between unrelated requests to memory.

Virtually all computers since 1975 have used DRAMs for main memory and SRAMs for cache, with one to three levels integrated onto the processor chip with the CPU. PMDs must balance power and performance, and because they have more modest storage needs, PMDs use Flash rather than disk drives, a decision increasingly being followed by desktop computers as well.

SRAM Technology

The first letter of SRAM stands for *static*. The dynamic nature of the circuits in DRAM requires data to be written back after being read—thus the difference between the access time and the cycle time as well as the need to refresh. SRAMs don't need to refresh, so the access time is very close to the cycle time. SRAMs typically use six transistors per bit to prevent the information from being disturbed when read. SRAM needs only minimal power to retain the charge in standby mode.

In earlier times, most desktop and server systems used SRAM chips for their primary, secondary, or tertiary caches. Today, all three levels of caches are integrated onto the processor chip. In high-end server chips, there may be as many as 24 cores and up to 60 MiB of cache; such systems are often configured with 128–256 GiB of DRAM per processor chip. The access times for large, third-level, on-chip caches are typically two to eight times that of a second-level cache. Even so, the L3 access time is usually at least five times faster than a DRAM access.

On-chip, cache SRAMs are normally organized with a width that matches the block size of the cache, with the tags stored in parallel to each block. This allows an entire block to be read out or written into a single cycle. This capability is particularly useful when writing data fetched after a miss into the cache or when writing back a block that must be evicted from the cache. The access time to the cache (ignoring the hit detection and selection in a set associative cache) is proportional to the number of blocks in the cache, whereas the energy consumption depends both on the number of bits in the cache (static power) and on the number of blocks (dynamic power). Set associative caches reduce the initial access time to the memory because the size of the memory is smaller, but increase the time for hit detection and block selection, a topic we will cover in Section 2.3.

DRAM Technology

As early DRAMs grew in capacity, the cost of a package with all the necessary address lines was an issue. The solution was to multiplex the address lines, thereby

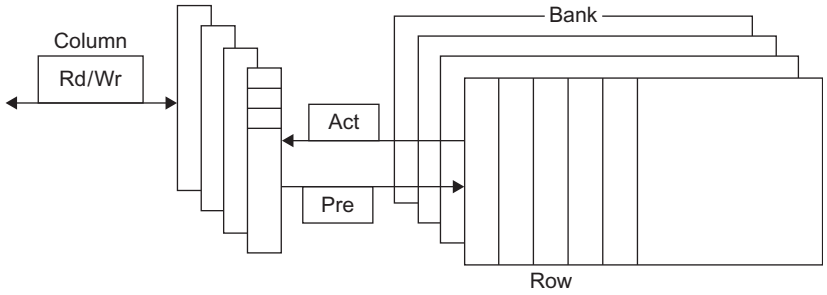


Figure 2.3 Internal organization of a DRAM. Modern DRAMs are organized in banks, up to 16 for DDR4. Each bank consists of a series of rows. Sending an ACT (Activate) command opens a bank and a row and loads the row into a row buffer. When the row is in the buffer, it can be transferred by successive column addresses at whatever the width of the DRAM is (typically 4, 8, or 16 bits in DDR4) or by specifying a block transfer and the starting address. The Precharge commend (PRE) closes the bank and row and readies it for a new access. Each command, as well as block transfers, are synchronized with a clock. See the next section discussing SDRAM. The row and column signals are sometimes called RAS and CAS, based on the original names of the signals.

cutting the number of address pins in half. Figure 2.3 shows the basic DRAM organization. One-half of the address is sent first during the *row access strobe* (RAS). The other half of the address, sent during the *column access strobe* (CAS), follows it. These names come from the internal chip organization, because the memory is organized as a rectangular matrix addressed by rows and columns.

An additional requirement of DRAM derives from the property signified by its first letter, *D*, for *dynamic*. To pack more bits per chip, DRAMs use only a single transistor, which effectively acts as a capacitor, to store a bit. This has two implications: first, the sensing wires that detect the charge must be precharged, which sets them "halfway" between a logical 0 and 1, allowing the small charge stored in the cell to cause a 0 or 1 to be detected by the sense amplifiers. On reading, a row is placed into a row buffer, where CAS signals can select a portion of the row to read out from the DRAM. Because reading a row destroys the information, it must be written back when the row is no longer needed. This write back happens in overlapped fashion, but in early DRAMs, it meant that the cycle time before a new row could be read was larger than the time to read a row and access a portion of that row.

In addition, to prevent loss of information as the charge in a cell leaks away (assuming it is not read or written), each bit must be "refreshed" periodically. Fortunately, all the bits in a row can be refreshed simultaneously just by reading that row and writing it back. Therefore every DRAM in the memory system must access every row within a certain time window, such as 64 ms. DRAM controllers include hardware to refresh the DRAMs periodically.

This requirement means that the memory system is occasionally unavailable because it is sending a signal telling every chip to refresh. The time for a refresh is a row activation and a precharge that also writes the row back (which takes

roughly 2/3 of the time to get a datum because no column select is needed), and this is required for each row of the DRAM. Because the memory matrix in a DRAM is conceptually square, the number of steps in a refresh is usually the square root of the DRAM capacity. DRAM designers try to keep time spent refreshing to less than 5% of the total time. So far we have presented main memory as if it operated like a Swiss train, consistently delivering the goods exactly according to schedule. In fact, with SDRAMs, a DRAM controller (usually on the processor chip) tries to optimize accesses by avoiding opening new rows and using block transfer when possible. Refresh adds another unpredictable factor.

Amdahl suggested as a rule of thumb that memory capacity should grow linearly with processor speed to keep a balanced system. Thus a 1000 MIPS processor should have 1000 MiB of memory. Processor designers rely on DRAMs to supply that demand. In the past, they expected a fourfold improvement in capacity every three years, or 55% per year. Unfortunately, the performance of DRAMs is growing at a much slower rate. The slower performance improvements arise primarily because of smaller decreases in the row access time, which is determined by issues such as power limitations and the charge capacity (and thus the size) of an individual memory cell. Before we discuss these performance trends in more detail, we need to describe the major changes that occurred in DRAMs starting in the mid-1990s.

Improving Memory Performance Inside a DRAM Chip: SDRAMs

Although very early DRAMs included a buffer allowing multiple column accesses to a single row, without requiring a new row access, they used an asynchronous interface, which meant that every column access and transfer involved overhead to synchronize with the controller. In the mid-1990s, designers added a clock signal to the DRAM interface so that the repeated transfers would not bear that overhead, thereby creating *synchronous DRAM* (SDRAM). In addition to reducing overhead, SDRAMs allowed the addition of a burst transfer mode where multiple transfers can occur without specifying a new column address. Typically, eight or more 16-bit transfers can occur without sending any new addresses by placing the DRAM in burst mode. The inclusion of such burst mode transfers has meant that there is a significant gap between the bandwidth for a stream of random accesses versus access to a block of data.

To overcome the problem of getting more bandwidth from the memory as DRAM density increased, DRAMS were made wider. Initially, they offered a four-bit transfer mode; in 2017, DDR2, DDR3, and DDR DRAMS had up to 4, 8, or 16 bit buses.

In the early 2000s, a further innovation was introduced: *double data rate* (DDR), which allows a DRAM to transfer data both on the rising and the falling edge of the memory clock, thereby doubling the peak data rate.

Finally, SDRAMs introduced *banks* to help with power management, improve access time, and allow interleaved and overlapped accesses to different banks.

Access to different banks can be overlapped with each other, and each bank has its own row buffer. Creating multiple banks inside a DRAM effectively adds another segment to the address, which now consists of bank number, row address, and column address. When an address is sent that designates a new bank, that bank must be opened, incurring an additional delay. The management of banks and row buffers is completely handled by modern memory control interfaces, so that when a subsequent access specifies the same row for an open bank, the access can happen quickly, sending only the column address.

To initiate a new access, the DRAM controller sends a bank and row number (called *Activate* in SDRAMs and formerly called RAS—row select). That command opens the row and reads the entire row into a buffer. A column address can then be sent, and the SDRAM can transfer one or more data items, depending on whether it is a single item request or a burst request. Before accessing a new row, the bank must be precharged. If the row is in the same bank, then the precharge delay is seen; however, if the row is in another bank, closing the row and precharging can overlap with accessing the new row. In synchronous DRAMs, each of these command cycles requires an integral number of clock cycles.

From 1980 to 1995, DRAMs scaled with Moore's Law, doubling capacity every 18 months (or a factor of 4 in 3 years). From the mid-1990s to 2010, capacity increased more slowly with roughly 26 months between a doubling. From 2010 to 2016, capacity only doubled! Figure 2.4 shows the capacity and access time for various generations of DDR SDRAMs. From DDR1 to DDR3, access times improved by a factor of about 3, or about 7% per year. DDR4 improves power and bandwidth over DDR3, but has similar access latency.

As Figure 2.4 shows, DDR is a sequence of standards. DDR2 lowers power from DDR1 by dropping the voltage from 2.5 to 1.8 V and offers higher clock rates: 266, 333, and 400 MHz. DDR3 drops voltage to 1.5 V and has a maximum clock speed of 800 MHz. (As we discuss in the next section, GDDR5 is a graphics

			Best case access time (no precharge)			Precharge needed
Production year	**Chip size**	**DRAM type**	**RAS time (ns)**	**CAS time (ns)**	**Total (ns)**	**Total (ns)**
2000	256M bit	DDR1	21	21	42	63
2002	512M bit	DDR1	15	15	30	45
2004	1G bit	DDR2	15	15	30	45
2006	2G bit	DDR2	10	10	20	30
2010	4G bit	DDR3	13	13	26	39
2016	8G bit	DDR4	13	13	26	39

Figure 2.4 Capacity and access times for DDR SDRAMs by year of production. Access time is for a random memory word and assumes a new row must be opened. If the row is in a different bank, we assume the bank is precharged; if the row is not open, then a precharge is required, and the access time is longer. As the number of banks has increased, the ability to hide the precharge time has also increased. DDR4 SDRAMs were initially expected in 2014, but did not begin production until early 2016.

Standard	I/O clock rate	M transfers/s	DRAM name	MiB/s/DIMM	DIMM name
DDR1	133	266	DDR266	2128	PC2100
DDR1	150	300	DDR300	2400	PC2400
DDR1	200	400	DDR400	3200	PC3200
DDR2	266	533	DDR2-533	4264	PC4300
DDR2	333	667	DDR2-667	5336	PC5300
DDR2	400	800	DDR2-800	6400	PC6400
DDR3	533	1066	DDR3-1066	8528	PC8500
DDR3	666	1333	DDR3-1333	10,664	PC10700
DDR3	800	1600	DDR3-1600	12,800	PC12800
DDR4	1333	2666	DDR4-2666	21,300	PC21300

Figure 2.5 Clock rates, bandwidth, and names of DDR DRAMS and DIMMs in 2016. Note the numerical relationship between the columns. The third column is twice the second, and the fourth uses the number from the third column in the name of the DRAM chip. The fifth column is eight times the third column, and a rounded version of this number is used in the name of the DIMM. DDR4 saw significant first use in 2016.

RAM and is based on DDR3 DRAMs.) DDR4, which shipped in volume in early 2016, but was expected in 2014, drops the voltage to 1–1.2 V and has a maximum expected clock rate of 1600 MHz. DDR5 is unlikely to reach production quantities until 2020 or later.

With the introduction of DDR, memory designers increasing focused on bandwidth, because improvements in access time were difficult. Wider DRAMs, burst transfers, and double data rate all contributed to rapid increases in memory bandwidth. DRAMs are commonly sold on small boards called *dual inline memory modules* (DIMMs) that contain 4–16 DRAM chips and that are normally organized to be 8 bytes wide (+ ECC) for desktop and server systems. When DDR SDRAMs are packaged as DIMMs, they are confusingly labeled by the peak *DIMM* bandwidth. Therefore the DIMM name PC3200 comes from 200 MHz $\times 2 \times 8$ bytes, or 3200 MiB/s; it is populated with DDR SDRAM chips. Sustaining the confusion, the chips themselves are labeled with *the number of bits per second* rather than their clock rate, so a 200 MHz DDR chip is called a DDR400. Figure 2.5 shows the relationships' I/O clock rate, transfers per second per chip, chip bandwidth, chip name, DIMM bandwidth, and DIMM name.

Reducing Power Consumption in SDRAMs

Power consumption in dynamic memory chips consists of both dynamic power used in a read or write and static or standby power; both depend on the operating voltage. In the most advanced DDR4 SDRAMs, the operating voltage has dropped to 1.2 V, significantly reducing power versus DDR2 and DDR3 SDRAMs. The addition of banks also reduced power because only the row in a single bank is read.

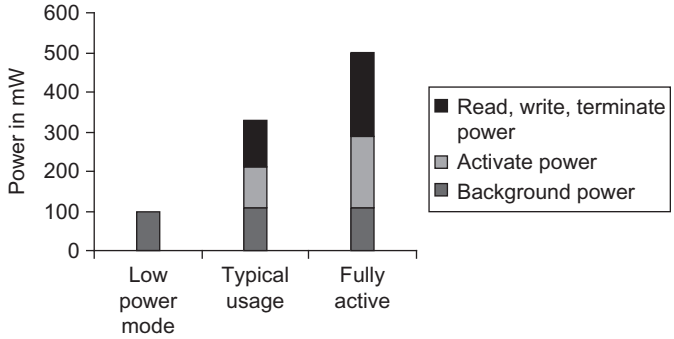


Figure 2.6 Power consumption for a DDR3 SDRAM operating under three conditions: low-power (shutdown) mode, typical system mode (DRAM is active 30% of the time for reads and 15% for writes), and fully active mode, where the DRAM is continuously reading or writing. Reads and writes assume bursts of eight transfers. These data are based on a Micron 1.5V 2GB DDR3-1066, although similar savings occur in DDR4 SDRAMs.

In addition to these changes, all recent SDRAMs support a power-down mode, which is entered by telling the DRAM to ignore the clock. Power-down mode disables the SDRAM, except for internal automatic refresh (without which entering power-down mode for longer than the refresh time will cause the contents of memory to be lost). Figure 2.6 shows the power consumption for three situations in a 2 GB DDR3 SDRAM. The exact delay required to return from low power mode depends on the SDRAM, but a typical delay is 200 SDRAM clock cycles.

Graphics Data RAMs

GDRAMs or GSDRAMs (Graphics or Graphics Synchronous DRAMs) are a special class of DRAMs based on SDRAM designs but tailored for handling the higher bandwidth demands of graphics processing units. GDDR5 is based on DDR3 with earlier GDDRs based on DDR2. Because graphics processor units (GPUs; see Chapter 4) require more bandwidth per DRAM chip than CPUs, GDDRs have several important differences:

1. GDDRs have wider interfaces: 32-bits versus 4, 8, or 16 in current designs.
2. GDDRs have a higher maximum clock rate on the data pins. To allow a higher transfer rate without incurring signaling problems, GDRAMS normally connect directly to the GPU and are attached by soldering them to the board, unlike DRAMs, which are normally arranged in an expandable array of DIMMs.

Altogether, these characteristics let GDDRs run at two to five times the bandwidth per DRAM versus DDR3 DRAMs.

Packaging Innovation: Stacked or Embedded DRAMs

The newest innovation in 2017 in DRAMs is a packaging innovation, rather than a circuit innovation. It places multiple DRAMs in a stacked or adjacent fashion embedded within the same package as the processor. (Embedded DRAM also is used to refer to designs that place DRAM on the processor chip.) Placing the DRAM and processor in the same package lowers access latency (by shortening the delay between the DRAMs and the processor) and potentially increases bandwidth by allowing more and faster connections between the processor and DRAM; thus several producers have called it *high bandwidth memory (HBM)*.

One version of this technology places the DRAM die directly on the CPU die using solder bump technology to connect them. Assuming adequate heat management, multiple DRAM dies can be stacked in this fashion. Another approach stacks only DRAMs and abuts them with the CPU in a single package using a substrate (interposer) containing the connections. Figure 2.7 shows these two different interconnection schemes. Prototypes of HBM that allow stacking of up to eight chips have been demonstrated. With special versions of SDRAMs, such a package could contain 8 GiB of memory and have data transfer rates of 1 TB/s. The 2.5D technique is currently available. Because the chips must be specifically manufactured to stack, it is quite likely that most early uses will be in high-end server chipsets.

In some applications, it may be possible to internally package enough DRAM to satisfy the needs of the application. For example, a version of an Nvidia GPU used as a node in a special-purpose cluster design is being developed using HBM, and it is likely that HBM will become a successor to GDDR5 for higher-end applications. In some cases, it may be possible to use HBM as main memory, although the cost limitations and heat removal issues currently rule out this technology for some embedded applications. In the next section, we consider the possibility of using HBM as an additional level of cache.

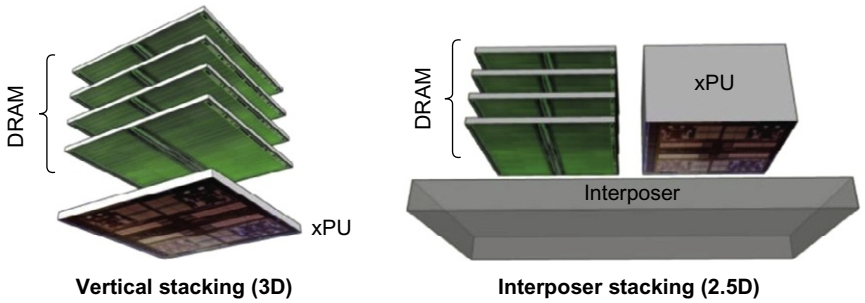


Figure 2.7 Two forms of die stacking. The 2.5D form is available now. 3D stacking is under development and faces heat management challenges due to the CPU.

Flash Memory

Flash memory is a type of EEPROM (electronically erasable programmable read-only memory), which is normally read-only but can be erased. The other key property of Flash memory is that it holds its contents without any power. We focus on NAND Flash, which has higher density than NOR Flash and is more suitable for large-scale nonvolatile memories; the drawback is that access is sequential and writing is slower, as we explain below.

Flash is used as the secondary storage in PMDs in the same manner that a disk functions in a laptop or server. In addition, because most PMDs have a limited amount of DRAM, Flash may also act as a level of the memory hierarchy, to a much greater extent than it might have to do in a desktop or server with a main memory that might be 10–100 times larger.

Flash uses a very different architecture and has different properties than standard DRAM. The most important differences are

1. Reads to Flash are sequential and read an entire page, which can be 512 bytes, 2 KiB, or 4 KiB. Thus NAND Flash has a long delay to access the first byte from a random address (about 25 μS), but can supply the remainder of a page block at about 40 MiB/s. By comparison, a DDR4 SDRAM takes about 40 ns to the first byte and can transfer the rest of the row at 4.8 GiB/s. Comparing the time to transfer 2 KiB, NAND Flash takes about 75 μS, while DDR SDRAM takes less than 500 ns, making Flash about 150 times slower. Compared to magnetic disk, however, a 2 KiB read from Flash is 300 to 500 times faster. From these numbers, we can see why Flash is not a candidate to replace DRAM for main memory, but is a candidate to replace magnetic disk.
2. Flash memory must be erased (thus the name flash for the "flash" erase process) before it is overwritten, and it is erased in blocks rather than individual bytes or words. This requirement means that when data must be written to Flash, an entire block must be assembled, either as new data or by merging the data to be written and the rest of the block's contents. For writing, Flash is about 1500 times slower then SDRAM, and about 8–15 times as fast as magnetic disk.
3. Flash memory is nonvolatile (i.e., it keeps its contents even when power is not applied) and draws significantly less power when not reading or writing (from less than half in standby mode to zero when completely inactive).
4. Flash memory limits the number of times that any given block can be written, typically at least 100,000. By ensuring uniform distribution of written blocks throughout the memory, a system can maximize the lifetime of a Flash memory system. This technique, called *write leveling*, is handled by Flash memory controllers.
5. High-density NAND Flash is cheaper than SDRAM but more expensive than disks: roughly $2/GiB for Flash, $20 to $40/GiB for SDRAM, and $0.09/GiB for magnetic disks. In the past five years, Flash has decreased in cost at a rate that is almost twice as fast as that of magnetic disks.

Like DRAM, Flash chips include redundant blocks to allow chips with small numbers of defects to be used; the remapping of blocks is handled in the Flash chip. Flash controllers handle page transfers, provide caching of pages, and handle write leveling.

The rapid improvements in high-density Flash have been critical to the development of low-power PMDs and laptops, but they have also significantly changed both desktops, which increasingly use solid state disks, and large servers, which often combine disk and Flash-based storage.

Phase-Change Memory Technology

Phase-change memory (PCM) has been an active research area for decades. The technology typically uses a small heating element to change the state of a bulk substrate between its crystalline form and an amorphous form, which have different resistive properties. Each bit corresponds to a crosspoint in a two-dimensional network that overlays the substrate. Reading is done by sensing the resistance between an x and y point (thus the alternative name *memristor*), and writing is accomplished by applying a current to change the phase of the material. The absence of an active device (such as a transistor) should lead to lower costs and greater density than that of NAND Flash.

In 2017 Micron and Intel began delivering Xpoint memory chips that are believed to be based on PCM. The technology is expected to have much better write durability than NAND Flash and, by eliminating the need to erase a page before writing, achieve an increase in write performance versus NAND of up to a factor of ten. Read latency is also better than Flash by perhaps a factor of 2–3. Initially, it is expected to be priced slightly higher than Flash, but the advantages in write performance and write durability may make it attractive, especially for SSDs. Should this technology scale well and be able to achieve additional cost reductions, it may be the solid state technology that will depose magnetic disks, which have reigned as the primary bulk nonvolatile store for more than 50 years.

Enhancing Dependability in Memory Systems

Large caches and main memories significantly increase the possibility of errors occurring both during the fabrication process and dynamically during operation. Errors that arise from a change in circuitry and are repeatable are called *hard errors* or *permanent faults.* Hard errors can occur during fabrication, as well as from a circuit change during operation (e.g., failure of a Flash memory cell after many writes). All DRAMs, Flash memory, and most SRAMs are manufactured with spare rows so that a small number of manufacturing defects can be accommodated by programming the replacement of a defective row by a spare row. Dynamic errors, which are changes to a cell's contents, not a change in the circuitry, are called *soft errors* or *transient faults*.

Dynamic errors can be detected by parity bits and detected and fixed by the use of error correcting codes (ECCs). Because instruction caches are read-only, parity

suffices. In larger data caches and in main memory, ECC is used to allow errors to be both detected and corrected. Parity requires only one bit of overhead to detect a single error in a sequence of bits. Because a multibit error would be undetected with parity, the number of bits protected by a parity bit must be limited. One parity bit per 8 data bits is a typical ratio. ECC can detect two errors and correct a single error with a cost of 8 bits of overhead per 64 data bits.

In very large systems, the possibility of multiple errors as well as complete failure of a single memory chip becomes significant. Chipkill was introduced by IBM to solve this problem, and many very large systems, such as IBM and SUN servers and the Google Clusters, use this technology. (Intel calls their version SDDC.) Similar in nature to the RAID approach used for disks, Chipkill distributes the data and ECC information so that the complete failure of a single memory chip can be handled by supporting the reconstruction of the missing data from the remaining memory chips. Using an analysis by IBM and assuming a 10,000 processor server with 4 GiB per processor yields the following rates of unrecoverable errors in three years of operation:

- Parity only: About 90,000, or one unrecoverable (or undetected) failure every 17 minutes.
- ECC only: About 3500, or about one undetected or unrecoverable failure every 7.5 hours.
- Chipkill: About one undetected or unrecoverable failure every 2 months.

Another way to look at this is to find the maximum number of servers (each with 4 GiB) that can be protected while achieving the same error rate as demonstrated for Chipkill. For parity, even a server with only one processor will have an unrecoverable error rate higher than a 10,000-server Chipkill protected system. For ECC, a 17-server system would have about the same failure rate as a 10,000-server Chipkill system. Therefore Chipkill is a requirement for the 50,000–100,00 servers in warehouse-scale computers (see Section 6.8 of Chapter 6).

2.3 Ten Advanced Optimizations of Cache Performance

The preceding average memory access time formula gives us three metrics for cache optimizations: hit time, miss rate, and miss penalty. Given the recent trends, we add cache bandwidth and power consumption to this list. We can classify the 10 advanced cache optimizations we examine into five categories based on these metrics:

1. *Reducing the hit time*—Small and simple first-level caches and way-prediction. Both techniques also generally decrease power consumption.
2. *Increasing cache bandwidth*—Pipelined caches, multibanked caches, and nonblocking caches. These techniques have varying impacts on power consumption.

3. *Reducing the miss penalty*—Critical word first and merging write buffers. These optimizations have little impact on power.
4. *Reducing the miss rate*—Compiler optimizations. Obviously any improvement at compile time improves power consumption.
5. *Reducing the miss penalty or miss rate via parallelism*—Hardware prefetching and compiler prefetching. These optimizations generally increase power consumption, primarily because of prefetched data that are unused.

In general, the hardware complexity increases as we go through these optimizations. In addition, several of the optimizations require sophisticated compiler technology, and the final one depends on HBM. We will conclude with a summary of the implementation complexity and the performance benefits of the 10 techniques presented in Figure 2.18 on page 113. Because some of these are straightforward, we cover them briefly; others require more description.

First Optimization: Small and Simple First-Level Caches to Reduce Hit Time and Power

The pressure of both a fast clock cycle and power limitations encourages limited size for first-level caches. Similarly, use of lower levels of associativity can reduce both hit time and power, although such trade-offs are more complex than those involving size.

The critical timing path in a cache hit is the three-step process of addressing the tag memory using the index portion of the address, comparing the read tag value to the address, and setting the multiplexor to choose the correct data item if the cache is set associative. Direct-mapped caches can overlap the tag check with the transmission of the data, effectively reducing hit time. Furthermore, lower levels of associativity will usually reduce power because fewer cache lines must be accessed.

Although the total amount of on-chip cache has increased dramatically with new generations of microprocessors, because of the clock rate impact arising from a larger L1 cache, the size of the L1 caches has recently increased either slightly or not at all. In many recent processors, designers have opted for more associativity rather than larger caches. An additional consideration in choosing the associativity is the possibility of eliminating address aliases; we discuss this topic shortly.

One approach to determining the impact on hit time and power consumption in advance of building a chip is to use CAD tools. CACTI is a program to estimate the access time and energy consumption of alternative cache structures on CMOS microprocessors within 10% of more detailed CAD tools. For a given minimum feature size, CACTI estimates the hit time of caches as a function of cache size, associativity, number of read/write ports, and more complex parameters. Figure 2.8 shows the estimated impact on hit time as cache size and associativity are varied. Depending on cache size, for these parameters, the model suggests that the hit time for direct mapped is slightly faster than two-way set associative and that two-way set associative is 1.2 times as fast as four-way and four-way is 1.4

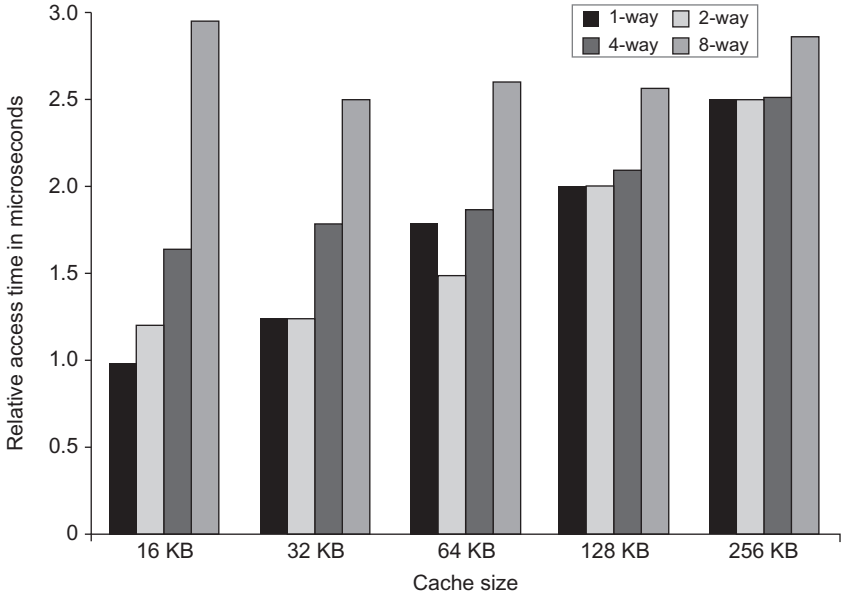


Figure 2.8 Relative access times generally increase as cache size and associativity are increased. These data come from the CACTI model 6.5 by Tarjan et al. (2005). The data assume typical embedded SRAM technology, a single bank, and 64-byte blocks. The assumptions about cache layout and the complex trade-offs between interconnect delays (that depend on the size of a cache block being accessed) and the cost of tag checks and multiplexing lead to results that are occasionally surprising, such as the lower access time for a 64 KiB with two-way set associativity versus direct mapping. Similarly, the results with eight-way set associativity generate unusual behavior as cache size is increased. Because such observations are highly dependent on technology and detailed design assumptions, tools such as CACTI serve to reduce the search space. These results are relative; nonetheless, they are likely to shift as we move to more recent and denser semiconductor technologies.

times as fast as eight-way. Of course, these estimates depend on technology as well as the size of the cache, and CACTI must be carefully aligned with the technology; Figure 2.8 shows the relative tradeoffs for one technology.

Example Using the data in Figure B.8 in Appendix B and Figure 2.8, determine whether a 32 KiB four-way set associative L1 cache has a faster memory access time than a 32 KiB two-way set associative L1 cache. Assume the miss penalty to L2 is 15 times the access time for the faster L1 cache. Ignore misses beyond L2. Which has the faster average memory access time?

Answer Let the access time for the two-way set associative cache be 1. Then, for the two-way cache,

$$\begin{aligned}\text{Average memory access time}_{\text{2-way}} &= \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}\\ &= 1 + 0.038 \times 15 = 1.38\end{aligned}$$

For the four-way cache, the access time is 1.4 times longer. The elapsed time of the miss penalty is 15/1.4 = 10.1. Assume 10 for simplicity:

$$\begin{aligned}\text{Average memory access time}_{\text{4-way}} &= \text{Hit time}_{\text{2-way}} \times 1.4 + \text{Miss rate} \times \text{Miss penalty} \\ &= 1.4 + 0.037 \times 10 = 1.77\end{aligned}$$

Clearly, the higher associativity looks like a bad trade-off; however, because cache access in modern processors is often pipelined, the exact impact on the clock cycle time is difficult to assess.

Energy consumption is also a consideration in choosing both the cache size and associativity, as Figure 2.9 shows. The energy cost of higher associativity ranges from more than a factor of 2 to negligible in caches of 128 or 256 KiB when going from direct mapped to two-way set associative.

As energy consumption has become critical, designers have focused on ways to reduce the energy needed for cache access. In addition to associativity, the other key factor in determining the energy used in a cache access is the number of blocks in the cache because it determines the number of "rows" that are accessed. A designer could reduce the number of rows by increasing the block size (holding total cache size constant), but this could increase the miss rate, especially in smaller L1 caches.

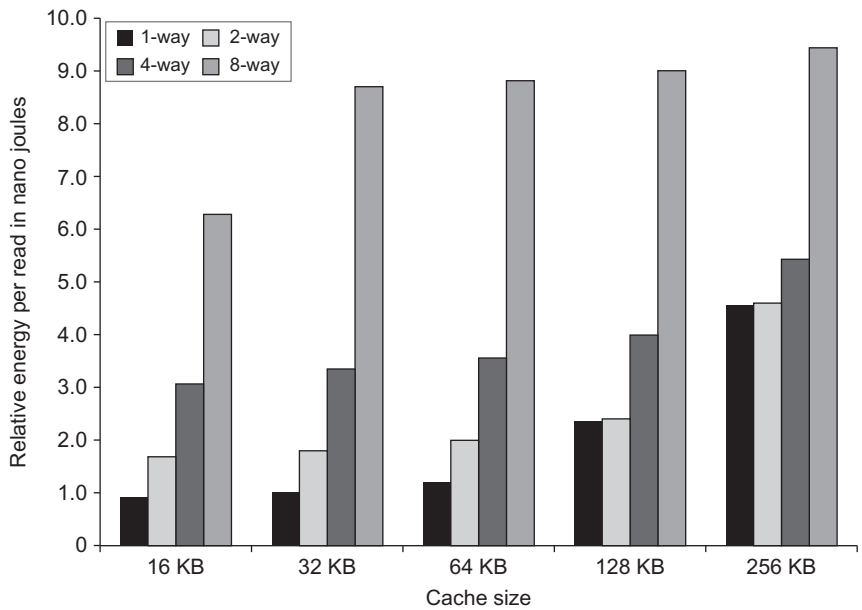


Figure 2.9 Energy consumption per read increases as cache size and associativity are increased. As in the previous figure, CACTI is used for the modeling with the same technology parameters. The large penalty for eight-way set associative caches is due to the cost of reading out eight tags and the corresponding data in parallel.

An alternative is to organize the cache in banks so that an access activates only a portion of the cache, namely the bank where the desired block resides. The primary use of multibanked caches is to increase the bandwidth of the cache, an optimization we consider shortly. Multibanking also reduces energy because less of the cache is accessed. The L3 caches in many multicores are logically unified, but physically distributed, and effectively act as a multibanked cache. Based on the address of a request, only one of the physical L3 caches (a bank) is actually accessed. We discuss this organization further in Chapter 5.

In recent designs, there are three other factors that have led to the use of higher associativity in first-level caches despite the energy and access time costs. First, many processors take at least 2 clock cycles to access the cache and thus the impact of a longer hit time may not be critical. Second, to keep the TLB out of the critical path (a delay that would be larger than that associated with increased associativity), almost all L1 caches should be virtually indexed. This limits the size of the cache to the page size times the associativity because then only the bits within the page are used for the index. There are other solutions to the problem of indexing the cache before address translation is completed, but increasing the associativity, which also has other benefits, is the most attractive. Third, with the introduction of multithreading (see Chapter 3), conflict misses can increase, making higher associativity more attractive.

Second Optimization: Way Prediction to Reduce Hit Time

Another approach reduces conflict misses and yet maintains the hit speed of direct-mapped cache. In *way prediction*, extra bits are kept in the cache to predict the way (or block within the set) of the *next* cache access. This prediction means the multiplexor is set early to select the desired block, and in that clock cycle, only a single tag comparison is performed in parallel with reading the cache data. A miss results in checking the other blocks for matches in the next clock cycle.

Added to each block of a cache are block predictor bits. The bits select which of the blocks to try on the next cache access. If the predictor is correct, the cache access latency is the fast hit time. If not, it tries the other block, changes the way predictor, and has a latency of one extra clock cycle. Simulations suggest that set prediction accuracy is in excess of 90% for a two-way set associative cache and 80% for a four-way set associative cache, with better accuracy on I-caches than D-caches. Way prediction yields lower average memory access time for a two-way set associative cache if it is at least 10% faster, which is quite likely. Way prediction was first used in the MIPS R10000 in the mid-1990s. It is popular in processors that use two-way set associativity and was used in several ARM processors, which have four-way set associative caches. For very fast processors, it may be challenging to implement the one-cycle stall that is critical to keeping the way prediction penalty small.

An extended form of way prediction can also be used to reduce power consumption by using the way prediction bits to decide which cache block to actually

access (the way prediction bits are essentially extra address bits); this approach, which might be called way selection, saves power when the way prediction is correct but adds significant time on a way misprediction, because the access, not just the tag match and selection, must be repeated. Such an optimization is likely to make sense only in low-power processors. Inoue et al. (1999) estimated that using the way selection approach with a four-way set associative cache increases the average access time for the I-cache by 1.04 and for the D-cache by 1.13 on the SPEC95 benchmarks, but it yields an average cache power consumption relative to a normal four-way set associative cache that is 0.28 for the I-cache and 0.35 for the D-cache. One significant drawback for way selection is that it makes it difficult to pipeline the cache access; however, as energy concerns have mounted, schemes that do not require powering up the entire cache make increasing sense.

Example Assume that there are half as many D-cache accesses as I-cache accesses and that the I-cache and D-cache are responsible for 25% and 15% of the processor's power consumption in a normal four-way set associative implementation. Determine if way selection improves performance per watt based on the estimates from the preceding study.

Answer For the I-cache, the savings in power is $25 \times 0.28 = 0.07$ of the total power, while for the D-cache it is $15 \times 0.35 = 0.05$ for a total savings of 0.12. The way prediction version requires 0.88 of the power requirement of the standard four-way cache. The increase in cache access time is the increase in I-cache average access time plus one-half the increase in D-cache access time, or $1.04 + 0.5 \times 0.13 = 1.11$ times longer. This result means that way selection has 0.90 of the performance of a standard four-way cache. Thus way selection improves performance per joule very slightly by a ratio of $0.90/0.88 = 1.02$. This optimization is best used where power rather than performance is the key objective.

Third Optimization: Pipelined Access and Multibanked Caches to Increase Bandwidth

These optimizations increase cache bandwidth either by pipelining the cache access or by widening the cache with multiple banks to allow multiple accesses per clock; these optimizations are the dual to the superpipelined and superscalar approaches to increasing instruction throughput. These optimizations are primarily targeted at L1, where access bandwidth constrains instruction throughput. Multiple banks are also used in L2 and L3 caches, but primarily as a power-management technique.

Pipelining L1 allows a higher clock cycle, at the cost of increased latency. For example, the pipeline for the instruction cache access for Intel Pentium processors in the mid-1990s took 1 clock cycle; for the Pentium Pro through Pentium III in the mid-1990s through 2000, it took 2 clock cycles; and for the Pentium 4, which became available in 2000, and the current Intel Core i7, it takes 4 clock cycles. Pipelining the instruction cache effectively increases the number of pipeline stages,

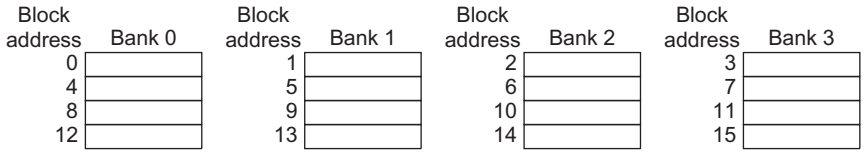


Figure 2.10 Four-way interleaved cache banks using block addressing. Assuming 64 bytes per block, each of these addresses would be multiplied by 64 to get byte addressing.

leading to a greater penalty on mispredicted branches. Correspondingly, pipelining the data cache leads to more clock cycles between issuing the load and using the data (see Chapter 3). Today, all processors use some pipelining of L1, if only for the simple case of separating the access and hit detection, and many high-speed processors have three or more levels of cache pipelining.

It is easier to pipeline the instruction cache than the data cache because the processor can rely on high performance branch prediction to limit the latency effects. Many superscalar processors can issue and execute more than one memory reference per clock (allowing a load or store is common, and some processors allow multiple loads). To handle multiple data cache accesses per clock, we can divide the cache into independent banks, each supporting an independent access. Banks were originally used to improve performance of main memory and are now used inside modern DRAM chips as well as with caches. The Intel Core i7 has four banks in L1 (to support up to 2 memory accesses per clock).

Clearly, banking works best when the accesses naturally spread themselves across the banks, so the mapping of addresses to banks affects the behavior of the memory system. A simple mapping that works well is to spread the addresses of the block sequentially across the banks, which is called *sequential interleaving*. For example, if there are four banks, bank 0 has all blocks whose address modulo 4 is 0, bank 1 has all blocks whose address modulo 4 is 1, and so on. Figure 2.10 shows this interleaving. Multiple banks also are a way to reduce power consumption in both caches and DRAM.

Multiple banks are also useful in L2 or L3 caches, but for a different reason. With multiple banks in L2, we can handle more than one outstanding L1 miss, if the banks do not conflict. This is a key capability to support nonblocking caches, our next optimization. The L2 in the Intel Core i7 has eight banks, while Arm Cortex processors have used L2 caches with 1–4 banks. As mentioned earlier, multibanking can also reduce energy consumption.

Fourth Optimization: Nonblocking Caches to Increase Cache Bandwidth

For pipelined computers that allow out-of-order execution (discussed in Chapter 3), the processor need not stall on a data cache miss. For example, the processor could

continue fetching instructions from the instruction cache while waiting for the data cache to return the missing data. A *nonblocking cache* or *lockup-free cache* escalates the potential benefits of such a scheme by allowing the data cache to continue to supply cache hits during a miss. This "hit under miss" optimization reduces the effective miss penalty by being helpful during a miss instead of ignoring the requests of the processor. A subtle and complex option is that the cache may further lower the effective miss penalty if it can overlap multiple misses: a "hit under multiple miss" or "miss under miss" optimization. The second option is beneficial only if the memory system can service multiple misses; most high-performance processors (such as the Intel Core processors) usually support both, whereas many lower-end processors provide only limited nonblocking support in L2.

To examine the effectiveness of nonblocking caches in reducing the cache miss penalty, Farkas and Jouppi (1994) did a study assuming 8 KiB caches with a 14-cycle miss penalty (appropriate for the early 1990s). They observed a reduction in the effective miss penalty of 20% for the SPECINT92 benchmarks and 30% for the SPECFP92 benchmarks when allowing one hit under miss.

Li et al. (2011) updated this study to use a multilevel cache, more modern assumptions about miss penalties, and the larger and more demanding SPECCPU2006 benchmarks. The study was done assuming a model based on a single core of an Intel i7 (see Section 2.6) running the SPECCPU2006 benchmarks. Figure 2.11 shows the reduction in data cache access latency when allowing 1, 2, and 64 hits under a miss; the caption describes further details of the memory system. The larger caches and the addition of an L3 cache since the earlier study have reduced the benefits with the SPECINT2006 benchmarks showing an average reduction in cache latency of about 9% and the SPECFP2006 benchmarks about 12.5%.

Example Which is more important for floating-point programs: two-way set associativity or hit under one miss for the primary data caches? What about integer programs? Assume the following average miss rates for 32 KiB data caches: 5.2% for floating-point programs with a direct-mapped cache, 4.9% for the programs with a two-way set associative cache, 3.5% for integer programs with a direct-mapped cache, and 3.2% for integer programs with a two-way set associative cache. Assume the miss penalty to L2 is 10 cycles, and the L2 misses and penalties are the same.

Answer For floating-point programs, the average memory stall times are

$$\text{Miss rate}_{\text{DM}} \times \text{Miss penalty} = 5.2\% \times 10 = 0.52$$
$$\text{Miss rate}_{\text{2-way}} \times \text{Miss penalty} = 4.9\% \times 10 = 0.49$$

The cache access latency (including stalls) for two-way associativity is 0.49/0.52 or 94% of direct-mapped cache. Figure 2.11 caption indicates that a hit under one miss reduces the average data cache access latency for floating-point programs to 87.5% of a blocking cache. Therefore, for floating-point programs, the

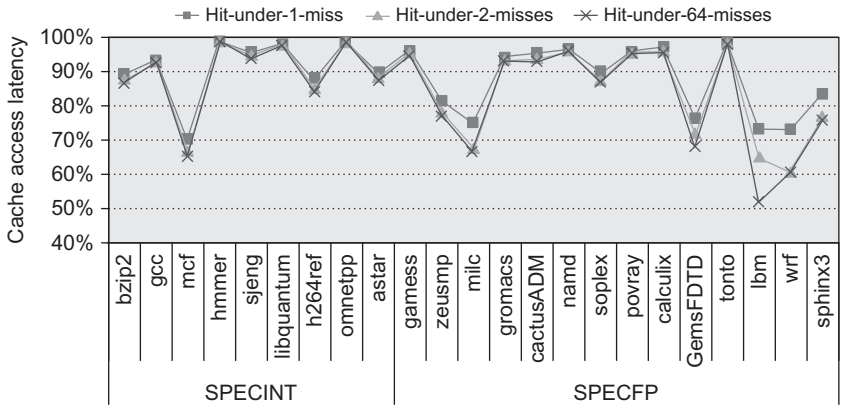


Figure 2.11 The effectiveness of a nonblocking cache is evaluated by allowing 1, 2, or 64 hits under a cache miss with 9 SPECINT (on the left) and 9 SPECFP (on the right) benchmarks. The data memory system modeled after the Intel i7 consists of a 32 KiB L1 cache with a four-cycle access latency. The L2 cache (shared with instructions) is 256 KiB with a 10-clock cycle access latency. The L3 is 2 MiB and a 36-cycle access latency. All the caches are eight-way set associative and have a 64-byte block size. Allowing one hit under miss reduces the miss penalty by 9% for the integer benchmarks and 12.5% for the floating point. Allowing a second hit improves these results to 10% and 16%, and allowing 64 results in little additional improvement.

direct-mapped data cache supporting one hit under one miss gives better performance than a two-way set-associative cache that blocks on a miss.

For integer programs, the calculation is

$$\text{Miss rate}_{\text{DM}} \times \text{Miss penalty} = 3.5\% \times 10 = 0.35$$
$$\text{Miss rate}_{\text{2-way}} \times \text{Miss penalty} = 3.2\% \times 10 = 0.32$$

The data cache access latency of a two-way set associative cache is thus 0.32/0.35 or 91% of direct-mapped cache, while the reduction in access latency when allowing a hit under one miss is 9%, making the two choices about equal.

The real difficulty with performance evaluation of nonblocking caches is that a cache miss does not necessarily stall the processor. In this case, it is difficult to judge the impact of any single miss and thus to calculate the average memory access time. The effective miss penalty is not the sum of the misses but the nonoverlapped time that the processor is stalled. The benefit of nonblocking caches is complex, as it depends upon the miss penalty when there are multiple misses, the memory reference pattern, and how many instructions the processor can execute with a miss outstanding.

In general, out-of-order processors are capable of hiding much of the miss penalty of an L1 data cache miss that hits in the L2 cache but are not capable

of hiding a significant fraction of a lower-level cache miss. Deciding how many outstanding misses to support depends on a variety of factors:

- The temporal and spatial locality in the miss stream, which determines whether a miss can initiate a new access to a lower-level cache or to memory.
- The bandwidth of the responding memory or cache.
- To allow more outstanding misses at the lowest level of the cache (where the miss time is the longest) requires supporting at least that many misses at a higher level, because the miss must initiate at the highest level cache.
- The latency of the memory system.

The following simplified example illustrates the key idea.

Example Assume a main memory access time of 36 ns and a memory system capable of a sustained transfer rate of 16 GiB/s. If the block size is 64 bytes, what is the maximum number of outstanding misses we need to support assuming that we can maintain the peak bandwidth given the request stream and that accesses never conflict. If the probability of a reference colliding with one of the previous four is 50%, and we assume that the access has to wait until the earlier access completes, estimate the number of maximum outstanding references. For simplicity, ignore the time between misses.

Answer In the first case, assuming that we can maintain the peak bandwidth, the memory system can support $(16 \times 10)^9/64 = 250$ million references per second. Because each reference takes 36 ns, we can support $250 \times 10^6 \times 36 \times 10^{-9} = 9$ references. If the probability of a collision is greater than 0, then we need more outstanding references, because we cannot start work on those colliding references; the memory system needs more independent references, not fewer! To approximate, we can simply assume that half the memory references do not have to be issued to the memory. This means that we must support twice as many outstanding references, or 18.

In Li, Chen, Brockman, and Jouppi's study, they found that the reduction in CPI for the integer programs was about 7% for one hit under miss and about 12.7% for 64. For the floating-point programs, the reductions were 12.7% for one hit under miss and 17.8% for 64. These reductions track fairly closely the reductions in the data cache access latency shown in Figure 2.11.

Implementing a Nonblocking Cache

Although nonblocking caches have the potential to improve performance, they are nontrivial to implement. Two initial types of challenges arise: arbitrating contention between hits and misses, and tracking outstanding misses so that we know when loads or stores can proceed. Consider the first problem. In a blocking cache, misses cause the processor to stall and no further accesses to the cache will occur

until the miss is handled. In a nonblocking cache, however, hits can collide with misses returning from the next level of the memory hierarchy. If we allow multiple outstanding misses, which almost all recent processors do, it is even possible for misses to collide. These collisions must be resolved, usually by first giving priority to hits over misses, and second by ordering colliding misses (if they can occur).

The second problem arises because we need to track multiple outstanding misses. In a blocking cache, we always know which miss is returning, because only one can be outstanding. In a nonblocking cache, this is rarely true. At first glance, you might think that misses always return in order, so that a simple queue could be kept to match a returning miss with the longest outstanding request. Consider, however, a miss that occurs in L1. It may generate either a hit or miss in L2; if L2 is also nonblocking, then the order in which misses are returned to L1 will not necessarily be the same as the order in which they originally occurred. Multicore and other multiprocessor systems that have nonuniform cache access times also introduce this complication.

When a miss returns, the processor must know which load or store caused the miss, so that instruction can now go forward; and it must know where in the cache the data should be placed (as well as the setting of tags for that block). In recent processors, this information is kept in a set of registers, typically called the *Miss Status Handling Registers (MSHRs)*. If we allow n outstanding misses, there will be n MSHRs, each holding the information about where a miss goes in the cache and the value of any tag bits for that miss, as well as the information indicating which load or store caused the miss (in the next chapter, you will see how this is tracked). Thus, when a miss occurs, we allocate an MSHR for handling that miss, enter the appropriate information about the miss, and tag the memory request with the index of the MSHR. The memory system uses that tag when it returns the data, allowing the cache system to transfer the data and tag information to the appropriate cache block and "notify" the load or store that generated the miss that the data is now available and that it can resume operation. Nonblocking caches clearly require extra logic and thus have some cost in energy. It is difficult, however, to assess their energy costs exactly because they may reduce stall time, thereby decreasing execution time and resulting energy consumption.

In addition to the preceding issues, multiprocessor memory systems, whether within a single chip or on multiple chips, must also deal with complex implementation issues related to memory coherency and consistency. Also, because cache misses are no longer atomic (because the request and response are split and may be interleaved among multiple requests), there are possibilities for deadlock. For the interested reader, Section I.7 in online Appendix I deals with these issues in detail.

Fifth Optimization: Critical Word First and Early Restart to Reduce Miss Penalty

This technique is based on the observation that the processor normally needs just one word of the block at a time. This strategy is impatience: don't wait for the full

block to be loaded before sending the requested word and restarting the processor. Here are two specific strategies:

- *Critical word first*—Request the missed word first from memory and send it to the processor as soon as it arrives; let the processor continue execution while filling the rest of the words in the block.
- *Early restart*—Fetch the words in normal order, but as soon as the requested word of the block arrives, send it to the processor and let the processor continue execution.

Generally, these techniques only benefit designs with large cache blocks because the benefit is low unless blocks are large. Note that caches normally continue to satisfy accesses to other blocks while the rest of the block is being filled.

However, given spatial locality, there is a good chance that the next reference is to the rest of the block. Just as with nonblocking caches, the miss penalty is not simple to calculate. When there is a second request in critical word first, the effective miss penalty is the nonoverlapped time from the reference until the second piece arrives. The benefits of critical word first and early restart depend on the size of the block and the likelihood of another access to the portion of the block that has not yet been fetched. For example, for SPECint2006 running on the i7 6700, which uses early restart and critical word first, there is more than one reference made to a block with an outstanding miss (1.23 references on average with a range from 0.5 to 3.0). We explore the performance of the i7 memory hierarchy in more detail in Section 2.6.

Sixth Optimization: Merging Write Buffer to Reduce Miss Penalty

Write-through caches rely on write buffers, as all stores must be sent to the next lower level of the hierarchy. Even write-back caches use a simple buffer when a block is replaced. If the write buffer is empty, the data and the full address are written in the buffer, and the write is finished from the processor's perspective; the processor continues working while the write buffer prepares to write the word to memory. If the buffer contains other modified blocks, the addresses can be checked to see if the address of the new data matches the address of a valid write buffer entry. If so, the new data are combined with that entry. *Write merging* is the name of this optimization. The Intel Core i7, among many others, uses write merging.

If the buffer is full and there is no address match, the cache (and processor) must wait until the buffer has an empty entry. This optimization uses the memory more efficiently because multiword writes are usually faster than writes performed one word at a time. Skadron and Clark (1997) found that even a merging four-entry write buffer generated stalls that led to a 5%–10% performance loss.

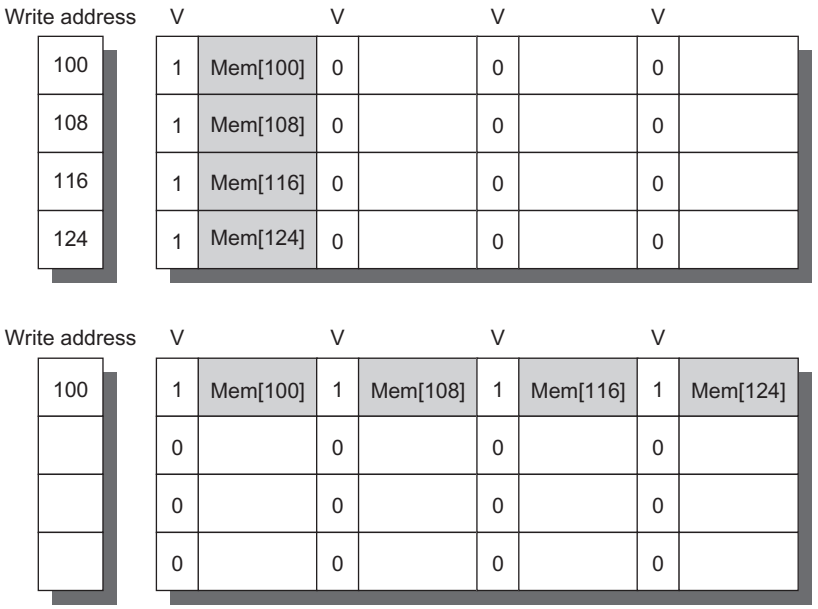


Figure 2.12 In this illustration of write merging, the write buffer on top does not use write merging while the write buffer on the bottom does. The four writes are merged into a single buffer entry with write merging; without it, the buffer is full even though three-fourths of each entry is wasted. The buffer has four entries, and each entry holds four 64-bit words. The address for each entry is on the left, with a valid bit (V) indicating whether the next sequential 8 bytes in this entry are occupied. (Without write merging, the words to the right in the upper part of the figure would be used only for instructions that wrote multiple words at the same time.)

The optimization also reduces stalls because of the write buffer being full. Figure 2.12 shows a write buffer with and without write merging. Assume we had four entries in the write buffer, and each entry could hold four 64-bit words. Without this optimization, four stores to sequential addresses would fill the buffer at one word per entry, even though these four words when merged fit exactly within a single entry of the write buffer.

Note that input/output device registers are often mapped into the physical address space. These I/O addresses *cannot* allow write merging because separate I/O registers may not act like an array of words in memory. For example, they may require one address and data word per I/O register rather than use multiword writes using a single address. These side effects are typically implemented by marking the pages as requiring nonmerging write through by the caches.

Seventh Optimization: Compiler Optimizations to Reduce Miss Rate

Thus far, our techniques have required changing the hardware. This next technique reduces miss rates without any hardware changes.

This magical reduction comes from optimized software—the hardware designer's favorite solution! The increasing performance gap between processors and main memory has inspired compiler writers to scrutinize the memory hierarchy to see if compile time optimizations can improve performance. Once again, research is split between improvements in instruction misses and improvements in data misses. The optimizations presented next are found in many modern compilers.

Loop Interchange

Some programs have nested loops that access data in memory in nonsequential order. Simply exchanging the nesting of the loops can make the code access the data in the order in which they are stored. Assuming the arrays do not fit in the cache, this technique reduces misses by improving spatial locality; reordering maximizes use of data in a cache block before they are discarded. For example, if `x` is a two-dimensional array of size [5000,100] allocated so that `x[i,j]` and `x[i,j+1]` are adjacent (an order called row major because the array is laid out by rows), then the two pieces of the following code show how the accesses can be optimized:

```
/* Before */
for (j = 0; j < 100; j = j + 1)
      for (i = 0; i < 5000; i = i + 1)
            x[i][j] = 2 * x[i][j];
/* After */
for (i = 0; i < 5000; i = i + 1)
      for (j = 0; j < 100; j = j + 1)
            x[i][j] = 2 * x[i][j];
```

The original code would skip through memory in strides of 100 words, while the revised version accesses all the words in one cache block before going to the next block. This optimization improves cache performance without affecting the number of instructions executed.

Blocking

This optimization improves temporal locality to reduce misses. We are again dealing with multiple arrays, with some arrays accessed by rows and some by columns. Storing the arrays row by row (*row major order*) or column by column (*column major order*) does not solve the problem because both rows and columns are used in every loop iteration. Such orthogonal accesses mean that transformations such as loop interchange still leave plenty of room for improvement.

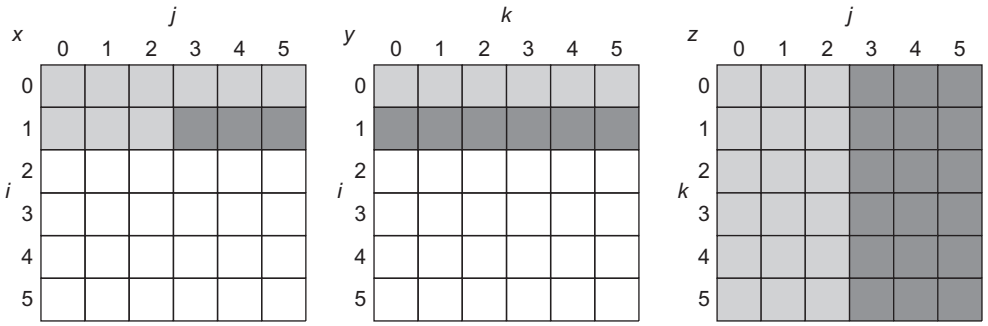


Figure 2.13 A snapshot of the three arrays `x`, `y`, and `z` when *N* = 6 and `i` = 1. The age of accesses to the array elements is indicated by shade: white means not yet touched, light means older accesses, and dark means newer accesses. The elements of `y` and `z` are read repeatedly to calculate new elements of `x`. The variables `i`, `j`, and `k` are shown along the rows or columns used to access the arrays.

Instead of operating on entire rows or columns of an array, blocked algorithms operate on submatrices or *blocks*. The goal is to maximize accesses to the data loaded into the cache before the data are replaced. The following code example, which performs matrix multiplication, helps motivate the optimization:

```
/* Before */
for (i = 0; i < N; i = i + 1)
      for (j = 0; j < N; j = j + 1)
           {r = 0;
            for (k = 0; k < N; k = k + 1)
                 r = r + y[i][k]*z[k][j];
            x[i][j] = r;
           };
```

The two inner loops read all `N`-by-`N` elements of `z`, read the same `N` elements in a row of `y` repeatedly, and write one row of `N` elements of `x`. Figure 2.13 gives a snapshot of the accesses to the three arrays. A dark shade indicates a recent access, a light shade indicates an older access, and white means not yet accessed.

The number of capacity misses clearly depends on `N` and the size of the cache. If it can hold all three `N`-by-`N` matrices, then all is well, provided there are no cache conflicts. If the cache can hold one `N-by-N` matrix and one row of `N`, then at least the `i`th row of `y` and the array `z` may stay in the cache. Less than that and misses may occur for both `x` and `z`. In the worst case, there would be $2N^3 + N^2$ memory words accessed for N^3 operations.

To ensure that the elements being accessed can fit in the cache, the original code is changed to compute on a submatrix of size `B` by `B`. Two inner loops now compute in steps of size `B` rather than the full length of `x` and `z`. `B` is called the *blocking factor*. (Assume `x` is initialized to zero.)

```
/* After */
for (jj = 0; jj < N; jj = jj + B)
for (kk = 0; kk < N; kk = kk + B)
for (i = 0; i < N; i = i + 1)
      for (j = jj; j < min(jj + B,N); j = j + 1)
          {r = 0;
           for (k = kk; k < min(kk + B,N); k = k + 1)
                r = r + y[i][k]*z[k][j];
           x[i][j] = x[i][j] + r;
          };
```

Figure 2.14 illustrates the accesses to the three arrays using blocking. Looking only at capacity misses, the total number of memory words accessed is $2N^3/B + N^2$. This total is an improvement by an approximate factor of B. Therefore blocking exploits a combination of spatial and temporal locality, because y benefits from spatial locality and z benefits from temporal locality. Although our example uses a square block (BxB), we could also use a rectangular block, which would be necessary if the matrix were not square.

Although we have aimed at reducing cache misses, blocking can also be used to help register allocation. By taking a small blocking size such that the block can be held in registers, we can minimize the number of loads and stores in the program.

As we shall see in Section 4.8 of Chapter 4, cache blocking is absolutely necessary to get good performance from cache-based processors running applications using matrices as the primary data structure.

Eighth Optimization: Hardware Prefetching of Instructions and Data to Reduce Miss Penalty or Miss Rate

Nonblocking caches effectively reduce the miss penalty by overlapping execution with memory access. Another approach is to prefetch items before the processor requests them. Both instructions and data can be prefetched, either directly into

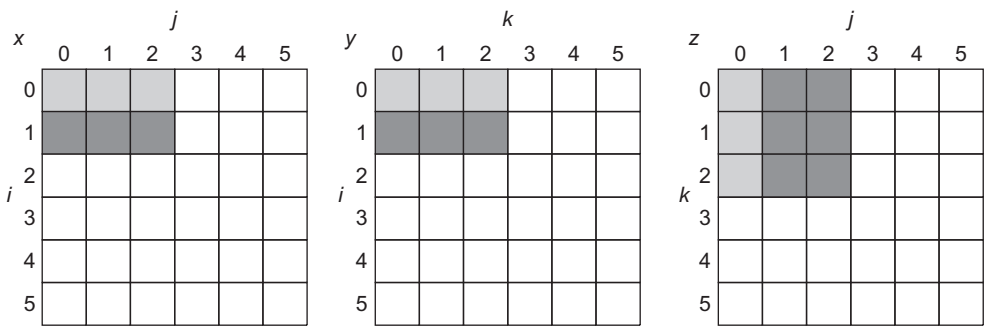


Figure 2.14 The age of accesses to the arrays x, y, and z when $B = 3$. Note that, in contrast to Figure 2.13, a smaller number of elements is accessed.

the caches or into an external buffer that can be more quickly accessed than main memory.

Instruction prefetch is frequently done in hardware outside of the cache. Typically, the processor fetches two blocks on a miss: the requested block and the next consecutive block. The requested block is placed in the instruction cache when it returns, and the prefetched block is placed in the instruction stream buffer. If the requested block is present in the instruction stream buffer, the original cache request is canceled, the block is read from the stream buffer, and the next prefetch request is issued.

A similar approach can be applied to data accesses (Jouppi, 1990). Palacharla and Kessler (1994) looked at a set of scientific programs and considered multiple stream buffers that could handle either instructions or data. They found that eight stream buffers could capture 50%–70% of all misses from a processor with two 64 KiB four-way set associative caches, one for instructions and the other for data.

The Intel Core i7 supports hardware prefetching into both L1 and L2 with the most common case of prefetching being accessing the next line. Some earlier Intel processors used more aggressive hardware prefetching, but that resulted in reduced performance for some applications, causing some sophisticated users to turn off the capability.

Figure 2.15 shows the overall performance improvement for a subset of SPEC2000 programs when hardware prefetching is turned on. Note that this figure

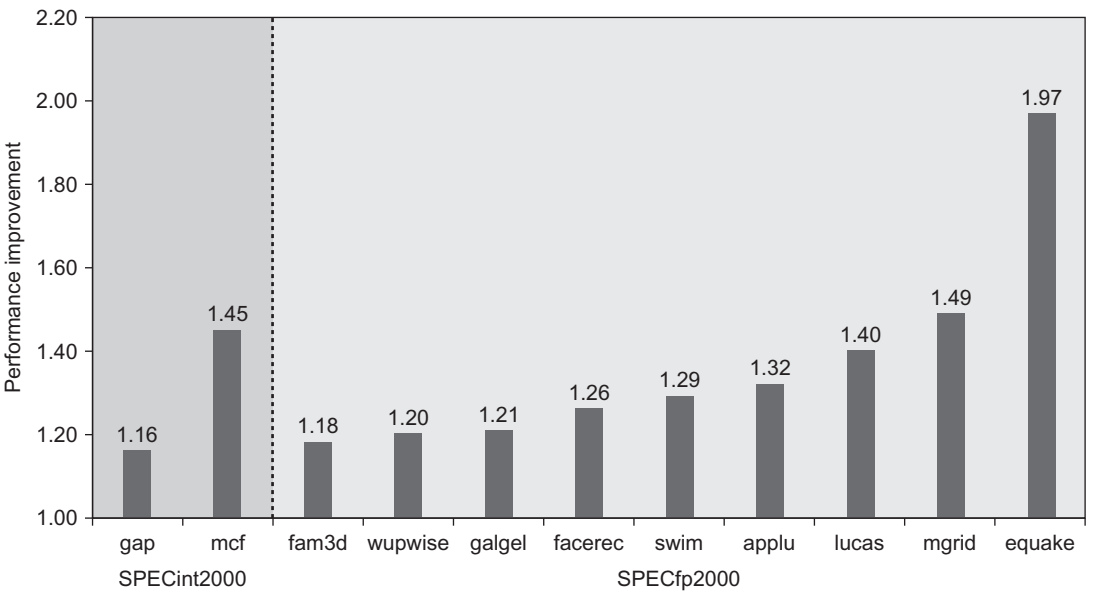


Figure 2.15 Speedup because of hardware prefetching on Intel Pentium 4 with hardware prefetching turned on for 2 of 12 SPECint2000 benchmarks and 9 of 14 SPECfp2000 benchmarks. Only the programs that benefit the most from prefetching are shown; prefetching speeds up the missing 15 SPECCPU benchmarks by less than 15% (Boggs et al., 2004).

includes only 2 of 12 integer programs, while it includes the majority of the SPECCPU floating-point programs. We will return to our evaluation of prefetching on the i7 in Section 2.6.

Prefetching relies on utilizing memory bandwidth that otherwise would be unused, but if it interferes with demand misses, it can actually lower performance. Help from compilers can reduce useless prefetching. When prefetching works well, its impact on power is negligible. When prefetched data are not used or useful data are displaced, prefetching will have a very negative impact on power.

Ninth Optimization: Compiler-Controlled Prefetching to Reduce Miss Penalty or Miss Rate

An alternative to hardware prefetching is for the compiler to insert prefetch instructions to request data before the processor needs it. There are two flavors of prefetch:

- *Register prefetch* loads the value into a register.
- *Cache prefetch* loads data only into the cache and not the register.

Either of these can be *faulting* or *nonfaulting*; that is, the address does or does not cause an exception for virtual address faults and protection violations. Using this terminology, a normal load instruction could be considered a "faulting register prefetch instruction." Nonfaulting prefetches simply turn into no-ops if they would normally result in an exception, which is what we want.

The most effective prefetch is "semantically invisible" to a program: it doesn't change the contents of registers and memory, *and* it cannot cause virtual memory faults. Most processors today offer nonfaulting cache prefetches. This section assumes nonfaulting cache prefetch, also called *nonbinding* prefetch.

Prefetching makes sense only if the processor can proceed while prefetching the data; that is, the caches do not stall but continue to supply instructions and data while waiting for the prefetched data to return. As you would expect, the data cache for such computers is normally nonblocking.

Like hardware-controlled prefetching, the goal is to overlap execution with the prefetching of data. Loops are the important targets because they lend themselves to prefetch optimizations. If the miss penalty is small, the compiler just unrolls the loop once or twice, and it schedules the prefetches with the execution. If the miss penalty is large, it uses software pipelining (see Appendix H) or unrolls many times to prefetch data for a future iteration.

Issuing prefetch instructions incurs an instruction overhead, however, so compilers must take care to ensure that such overheads do not exceed the benefits. By concentrating on references that are likely to be cache misses, programs can avoid unnecessary prefetches while improving average memory access time significantly.

Example For the following code, determine which accesses are likely to cause data cache misses. Next, insert prefetch instructions to reduce misses. Finally, calculate the number of prefetch instructions executed and the misses avoided by prefetching. Let's assume we have an 8 KiB direct-mapped data cache with 16-byte blocks, and it is a write-back cache that does write allocate. The elements of `a` and `b` are 8 bytes long because they are double-precision floating-point arrays. There are 3 rows and 100 columns for `a` and 101 rows and 3 columns for `b`. Let's also assume they are not in the cache at the start of the program.

```
for (i = 0; i < 3; i = i + 1)
      for (j = 0; j < 100; j = j + 1)
            a[i][j] = b[j][0] * b[j + 1][0];
```

Answer The compiler will first determine which accesses are likely to cause cache misses; otherwise, we will waste time on issuing prefetch instructions for data that would be hits. Elements of `a` are written in the order that they are stored in memory, so `a` will benefit from spatial locality: The even values of `j` will miss and the odd values will hit. Because `a` has 3 rows and 100 columns, its accesses will lead to $3 \times (100/2)$, or 150 misses.

The array `b` does not benefit from spatial locality because the accesses are not in the order it is stored. The array `b` does benefit twice from temporal locality: the same elements are accessed for each iteration of `i`, and each iteration of `j` uses the same value of `b` as the last iteration. Ignoring potential conflict misses, the misses because of `b` will be for `b[j+1][0]` accesses when `i=0`, and also the first access to `b[j][0]` when `j=0`. Because `j` goes from 0 to 99 when `i=0`, accesses to `b` lead to 100+1, or 101 misses.

Thus this loop will miss the data cache approximately 150 times for `a` plus 101 times for `b`, or 251 misses.

To simplify our optimization, we will not worry about prefetching the first accesses of the loop. These may already be in the cache, or we will pay the miss penalty of the first few elements of `a` or `b`. Nor will we worry about suppressing the prefetches at the end of the loop that try to prefetch beyond the end of `a` (`a[i][100]` … `a[i][106]`) and the end of `b` (`b[101][0]` … `b[107][0]`). If these were faulting prefetches, we could not take this luxury. Let's assume that the miss penalty is so large we need to start prefetching at least, say, seven iterations in advance. (Stated alternatively, we assume prefetching has no benefit until the eighth iteration.) We underline the changes to the preceding code needed to add prefetching.

```
for (j = 0; j < 100; j = j + 1) {
      prefetch(b[j + 7][0]);
      /* b(j,0) for 7 iterations later */
      prefetch(a[0][j + 7]);
      /* a(0,j) for 7 iterations later */
      a[0][j] = b[j][0] * b[j + 1][0];};
```

```
for (i = 1; i < 3; i = i + 1)
      for (j = 0; j < 100; j = j + 1) {
            prefetch(a[i][j + 7]);
            /* a(i,j) for + 7 iterations */
            a[i][j] = b[j][0] * b[j + 1][0];}
```

This revised code prefetches `a[i][7]` through `a[i][99]` and `b[7][0]` through `b[100][0]`, reducing the number of nonprefetched misses to

- 7 misses for elements `b[0][0]`, `b[1][0]`, … , `b[6][0]` in the first loop
- 4 misses ([7/2]) for elements `a[0][0]`, `a[0][1]`, … , `a[0][6]` in the first loop (spatial locality reduces misses to 1 per 16-byte cache block)
- 4 misses ([7/2]) for elements `a[1][0]`, `a[1][1]`, … , `a[1][6]` in the second loop
- 4 misses ([7/2]) for elements `a[2][0]`, `a[2][1]`, … , `a[2][6]` in the second loop

or a total of 19 nonprefetched misses. The cost of avoiding 232 cache misses is executing 400 prefetch instructions, likely a good trade-off.

Example Calculate the time saved in the preceding example. Ignore instruction cache misses and assume there are no conflict or capacity misses in the data cache. Assume that prefetches can overlap with each other and with cache misses, thereby transferring at the maximum memory bandwidth. Here are the key loop times ignoring cache misses: the original loop takes 7 clock cycles per iteration, the first prefetch loop takes 9 clock cycles per iteration, and the second prefetch loop takes 8 clock cycles per iteration (including the overhead of the outer for loop). A miss takes 100 clock cycles.

Answer The original doubly nested loop executes the multiply 3×100 or 300 times. Because the loop takes 7 clock cycles per iteration, the total is 300×7 or 2100 clock cycles plus cache misses. Cache misses add 251×100 or 25,100 clock cycles, giving a total of 27,200 clock cycles. The first prefetch loop iterates 100 times; at 9 clock cycles per iteration the total is 900 clock cycles plus cache misses. Now add 11×100 or 1100 clock cycles for cache misses, giving a total of 2000. The second loop executes 2×100 or 200 times, and at 8 clock cycles per iteration, it takes 1600 clock cycles plus 8×100 or 800 clock cycles for cache misses. This gives a total of 2400 clock cycles. From the prior example, we know that this code executes 400 prefetch instructions during the 2000 + 2400 or 4400 clock cycles to execute these two loops. If we assume that the prefetches are completely overlapped with the rest of the execution, then the prefetch code is 27,200/4400, or 6.2 times faster.

Although array optimizations are easy to understand, modern programs are more likely to use pointers. Luk and Mowry (1999) have demonstrated that compiler-based prefetching can sometimes be extended to pointers as well. Of 10 programs with recursive data structures, prefetching all pointers when a node is visited improved performance by 4%–31% in half of the programs. On the other hand, the remaining programs were still within 2% of their original performance. The issue is both whether prefetches are to data already in the cache and whether they occur early enough for the data to arrive by the time it is needed.

Many processors support instructions for cache prefetch, and high-end processors (such as the Intel Core i7) often also do some type of automated prefetch in hardware.

Tenth Optimization: Using HBM to Extend the Memory Hierarchy

Because most general-purpose processors in servers will likely want more memory than can be packaged with HBM packaging, it has been proposed that the in-package DRAMs be used to build massive L4 caches, with upcoming technologies ranging from 128 MiB to 1 GiB and more, considerably more than current on-chip L3 caches. Using such large DRAM-based caches raises an issue: where do the tags reside? That depends on the number of tags. Suppose we were to use a 64B block size; then a 1 GiB L4 cache requires 96 MiB of tags—far more static memory than exists in the caches on the CPU. Increasing the block size to 4 KiB, yields a dramatically reduced tag store of 256 K entries or less than 1 MiB total storage, which is probably acceptable, given L3 caches of 4–16 MiB or more in next-generation, multicore processors. Such large block sizes, however, have two major problems.

First, the cache may be used inefficiently when content of many blocks are not needed; this is called the *fragmentation problem*, and it also occurs in virtual memory systems. Furthermore, transferring such large blocks is inefficient if much of the data is unused. Second, because of the large block size, the number of distinct blocks held in the DRAM cache is much lower, which can result in more misses, especially for conflict and consistency misses.

One partial solution to the first problem is to add *sublocking*. *Subblocking* allow parts of the block to be invalid, requiring that they be fetched on a miss. Subblocking, however, does nothing to address the second problem.

The tag storage is the major drawback for using a smaller block size. One possible solution for that difficulty is to store the tags for L4 in the HBM. At first glance this seems unworkable, because it requires two accesses to DRAM for each L4 access: one for the tags and one for the data itself. Because of the long access time for random DRAM accesses, typically 100 or more processor clock cycles, such an approach had been discarded. Loh and Hill (2011) proposed a clever solution to this problem: place the tags and the data in the same row in the HBM SDRAM. Although opening the row (and eventually closing it) takes a large amount of time, the CAS latency to access a different part of the row is about one-third the new row access time. Thus we can access the tag portion of the block first, and if it is a hit,

then use a column access to choose the correct word. Loh and Hill (L-H) have proposed organizing the L4 HBM cache so that each SDRAM row consists of a set of tags (at the head of the block) and 29 data segments, making a 29-way set associative cache. When L4 is accessed, the appropriate row is opened and the tags are read; a hit requires one more column access to get the matching data.

Qureshi and Loh (2012) proposed an improvement called an *alloy cache* that reduces the hit time. An *alloy cache* molds the tag and data together and uses a direct mapped cache structure. This allows the L4 access time to be reduced to a single HBM cycle by directly indexing the HBM cache and doing a burst transfer of both the tag and data. Figure 2.16 shows the hit latency for the alloy cache, the L-H scheme, and SRAM based tags. The alloy cache reduces hit time by more than a factor of 2 versus the L-H scheme, in return for an increase in the miss rate by a factor of 1.1–1.2. The choice of benchmarks is explained in the caption.

Unfortunately, in both schemes, misses require two full DRAM accesses: one to get the initial tag and a follow-on access to the main memory (which is even

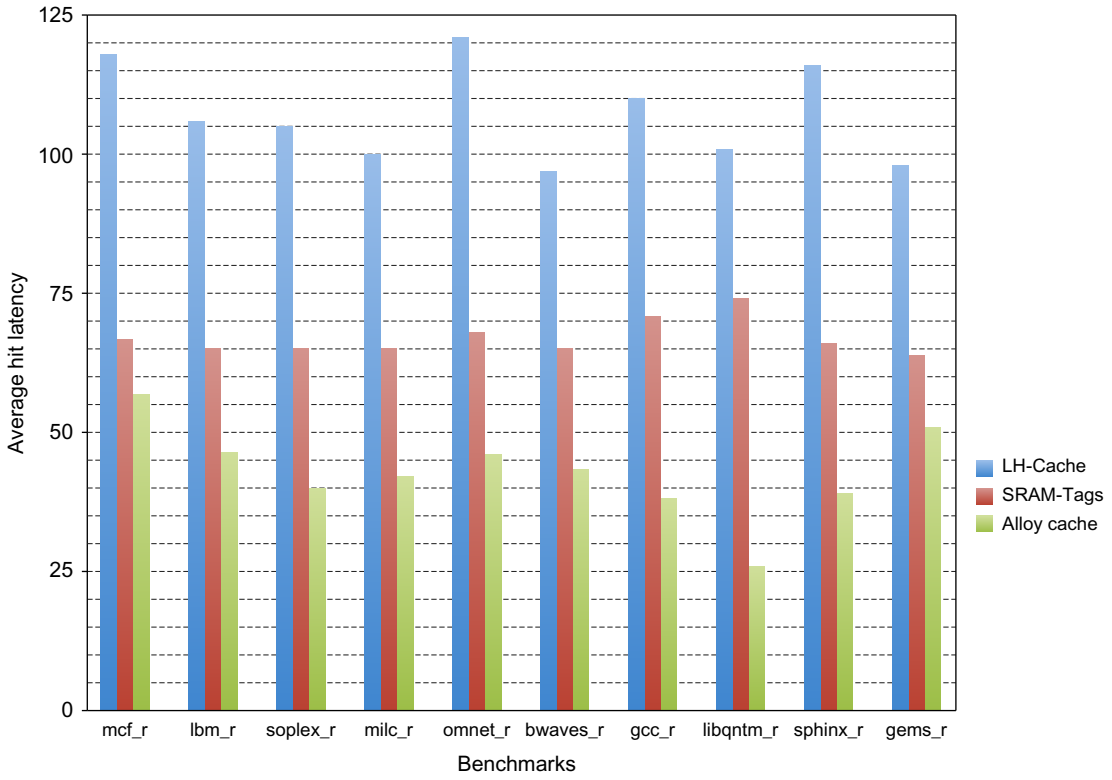


Figure 2.16 Average hit time latency in clock cycles for the L-H scheme, a currently-impractical scheme using SRAM for the tags, and the alloy cache organization. In the SRAM case, we assume the SRAM is accessible in the same time as L3 and that it is checked before L4 is accessed. The average hit latencies are 43 (alloy cache), 67 (SRAM tags), and 107 (L-H). The 10 SPECCPU2006 benchmarks used here are the most memory-intensive ones; each of them would run twice as fast if L3 were perfect.

slower). If we could speed up the miss detection, we could reduce the miss time. Two different solutions have been proposed to solve this problem: one uses a map that keeps track of the blocks in the cache (not the location of the block, just whether it is present); the other uses a memory access predictor that predicts likely misses using history prediction techniques, similar to those used for global branch prediction (see the next chapter). It appears that a small predictor can predict likely misses with high accuracy, leading to an overall lower miss penalty.

Figure 2.17 shows the speedup obtained on SPECrate for the memory-intensive benchmarks used in Figure 2.16. The alloy cache approach outperforms the LH scheme and even the impractical SRAM tags, because the combination of a fast access time for the miss predictor and good prediction results lead to a shorter time to predict a miss, and thus a lower miss penalty. The alloy cache performs close to the Ideal case, an L4 with perfect miss prediction and minimal hit time.

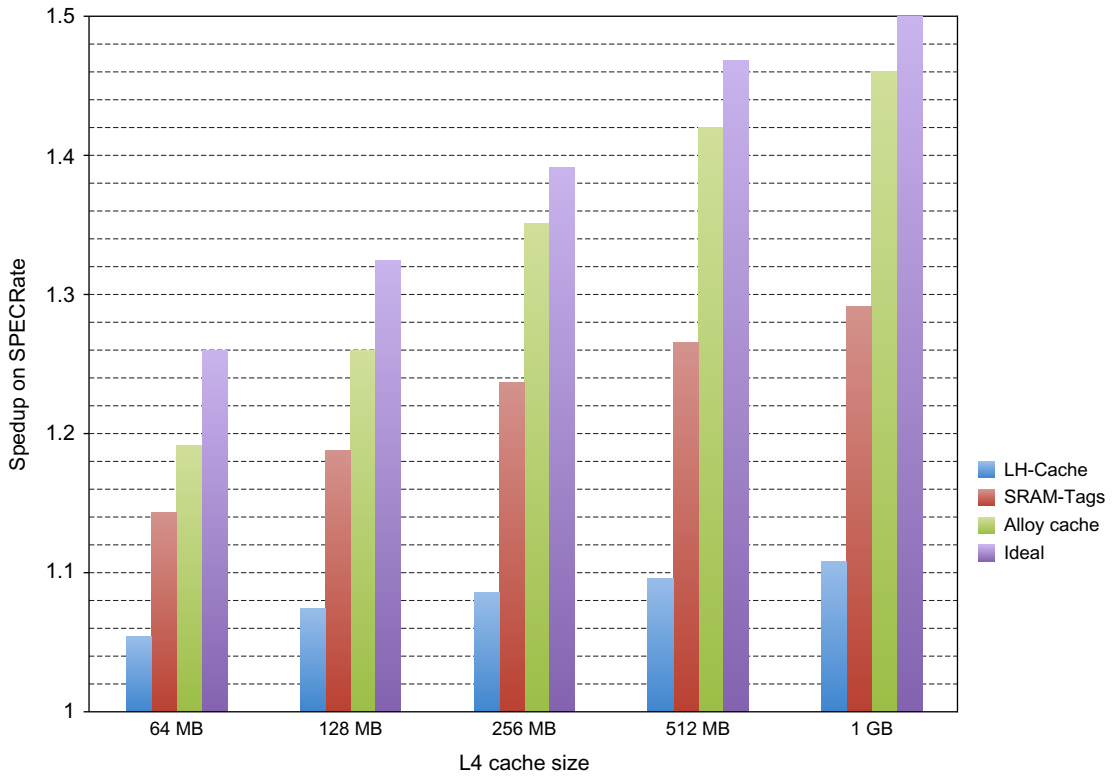


Figure 2.17 Performance speedup running the SPECrate benchmark for the LH scheme, an SRAM tag scheme, and an ideal L4 (Ideal); a speedup of 1 indicates no improvement with the L4 cache, and a speedup of 2 would be achievable if L4 were perfect and took no access time. The 10 memory-intensive benchmarks are used with each benchmark run eight times. The accompanying miss prediction scheme is used. The Ideal case assumes that only the 64-byte block requested in L4 needs to be accessed and transferred and that prediction accuracy for L4 is perfect (i.e., all misses are known at zero cost).

HBM is likely to have widespread use in a variety of different configurations, from containing the entire memory system for some high-performance, special-purpose systems to use as an L4 cache for larger server configurations.

Cache Optimization Summary

The techniques to improve hit time, bandwidth, miss penalty, and miss rate generally affect the other components of the average memory access equation as well as the complexity of the memory hierarchy. Figure 2.18 summarizes these techniques and estimates the impact on complexity, with + meaning that the technique

Technique	Hit time	Band-width	Miss penalty	Miss rate	Power consumption	Hardware cost/ complexity	Comment
Small and simple caches	+			–	+	0	Trivial; widely used
Way-predicting caches	+				+	1	Used in Pentium 4
Pipelined & banked caches	–	+				1	Widely used
Nonblocking caches		+	+			3	Widely used
Critical word first and early restart			+			2	Widely used
Merging write buffer			+			1	Widely used with write through
Compiler techniques to reduce cache misses				+		0	Software is a challenge, but many compilers handle common linear algebra calculations
Hardware prefetching of instructions and data			+	+	–	2 instr., 3 data	Most provide prefetch instructions; modern high-end processors also automatically prefetch in hardware
Compiler-controlled prefetching			+	+		3	Needs nonblocking cache; possible instruction overhead; in many CPUs
HBM as additional level of cache		+/–	–	+	+	3	Depends on new packaging technology. Effects depend heavily on hit rate improvements

Figure 2.18 Summary of 10 advanced cache optimizations showing impact on cache performance, power consumption, and complexity. Although generally a technique helps only one factor, prefetching can reduce misses if done sufficiently early; if not, it can reduce miss penalty. + means that the technique improves the factor, – means it hurts that factor, and blank means it has no impact. The complexity measure is subjective, with 0 being the easiest and 3 being a challenge.

improves the factor, – meaning it hurts that factor, and blank meaning it has no impact. Generally, no technique helps more than one category.

2.4 Virtual Memory and Virtual Machines

A virtual machine is taken to be an efficient, isolated duplicate of the real machine. We explain these notions through the idea of a virtual machine monitor (VMM)… a VMM has three essential characteristics. First, the VMM provides an environment for programs which is essentially identical with the original machine; second, programs run in this environment show at worst only minor decreases in speed; and last, the VMM is in complete control of system resources.

Gerald Popek and Robert Goldberg,
"Formal requirements for virtualizable third generation architectures,"
Communications of the ACM (July 1974).

Section B.4 in Appendix B describes the key concepts in virtual memory. Recall that virtual memory allows the physical memory to be treated as a cache of secondary storage (which may be either disk or solid state). Virtual memory moves pages between the two levels of the memory hierarchy, just as caches move blocks between levels. Likewise, TLBs act as caches on the page table, eliminating the need to do a memory access every time an address is translated. Virtual memory also provides separation between processes that share one physical memory but have separate virtual address spaces. Readers should ensure that they understand both functions of virtual memory before continuing.

In this section, we focus on additional issues in protection and privacy between processes sharing the same processor. Security and privacy are two of the most vexing challenges for information technology in 2017. Electronic burglaries, often involving lists of credit card numbers, are announced regularly, and it's widely believed that many more go unreported. Of course, such problems arise from programming errors that allow a cyberattack to access data it should be unable to access. Programming errors are a fact of life, and with modern complex software systems, they occur with significant regularity. Therefore both researchers and practitioners are looking for improved ways to make computing systems more secure. Although protecting information is not limited to hardware, in our view real security and privacy will likely involve innovation in computer architecture as well as in systems software.

This section starts with a review of the architecture support for protecting processes from each other via virtual memory. It then describes the added protection provided by virtual machines, the architecture requirements of virtual machines, and the performance of a virtual machine. As we will see in Chapter 6, virtual machines are a foundational technology for cloud computing.

Protection via Virtual Memory

Page-based virtual memory, including a TLB that caches page table entries, is the primary mechanism that protects processes from each other. Sections B.4 and B.5 in Appendix B review virtual memory, including a detailed description of protection via segmentation and paging in the 80x86. This section acts as a quick review; if it's too quick, please refer to the denoted Appendix B sections.

Multiprogramming, where several programs running concurrently share a computer, has led to demands for protection and sharing among programs and to the concept of a *process*. Metaphorically, a process is a program's breathing air and living space—that is, a running program plus any state needed to continue running it. At any instant, it must be possible to switch from one process to another. This exchange is called a *process switch* or *context switch*.

The operating system and architecture join forces to allow processes to share the hardware yet not interfere with each other. To do this, the architecture must limit what a process can access when running a user process yet allow an operating system process to access more. At a minimum, the architecture must do the following:

1. Provide at least two modes, indicating whether the running process is a user process or an operating system process. This latter process is sometimes called a *kernel* process or a *supervisor* process.
2. Provide a portion of the processor state that a user process can use but not write. This state includes a user/supervisor mode bit, an exception enable/disable bit, and memory protection information. Users are prevented from writing this state because the operating system cannot control user processes if users can give themselves supervisor privileges, disable exceptions, or change memory protection.
3. Provide mechanisms whereby the processor can go from user mode to supervisor mode and vice versa. The first direction is typically accomplished by a *system call*, implemented as a special instruction that transfers control to a dedicated location in supervisor code space. The PC is saved from the point of the system call, and the processor is placed in supervisor mode. The return to user mode is like a subroutine return that restores the previous user/supervisor mode.
4. Provide mechanisms to limit memory accesses to protect the memory state of a process without having to swap the process to disk on a context switch.

Appendix A describes several memory protection schemes, but by far the most popular is adding protection restrictions to each page of virtual memory. Fixed-sized pages, typically 4 KiB, 16 KiB, or larger, are mapped from the virtual address space into physical address space via a page table. The protection restrictions are included in each page table entry. The protection restrictions might determine whether a user process can read this page, whether a user process can write to this page, and whether code can be executed from this page. In addition, a process can

neither read nor write a page if it is not in the page table. Because only the OS can update the page table, the paging mechanism provides total access protection.

Paged virtual memory means that every memory access logically takes at least twice as long, with one memory access to obtain the physical address and a second access to get the data. This cost would be far too dear. The solution is to rely on the principle of locality; if the accesses have locality, then the *address translations* for the accesses must also have locality. By keeping these address translations in a special cache, a memory access rarely requires a second access to translate the address. This special address translation cache is referred to as a TLB.

A TLB entry is like a cache entry where the tag holds portions of the virtual address and the data portion holds a physical page address, protection field, valid bit, and usually a use bit and a dirty bit. The operating system changes these bits by changing the value in the page table and then invalidating the corresponding TLB entry. When the entry is reloaded from the page table, the TLB gets an accurate copy of the bits.

Assuming the computer faithfully obeys the restrictions on pages and maps virtual addresses to physical addresses, it would seem that we are done. Newspaper headlines suggest otherwise.

The reason we're not done is that we depend on the accuracy of the operating system as well as the hardware. Today's operating systems consist of tens of millions of lines of code. Because bugs are measured in number per thousand lines of code, there are thousands of bugs in production operating systems. Flaws in the OS have led to vulnerabilities that are routinely exploited.

This problem and the possibility that not enforcing protection could be much more costly than in the past have led some to look for a protection model with a much smaller code base than the full OS, such as virtual machines.

Protection via Virtual Machines

An idea related to virtual memory that is almost as old are virtual machines (VMs). They were first developed in the late 1960s, and they have remained an important part of mainframe computing over the years. Although largely ignored in the domain of single-user computers in the 1980s and 1990s, they have recently gained popularity because of

- the increasing importance of isolation and security in modern systems;
- the failures in security and reliability of standard operating systems;
- the sharing of a single computer among many unrelated users, such as in a data center or cloud; and
- the dramatic increases in the raw speed of processors, which make the overhead of VMs more acceptable.

The broadest definition of VMs includes basically all emulation methods that provide a standard software interface, such as the Java VM. We are interested in

VMs that provide a complete system-level environment at the binary instruction set architecture (ISA) level. Most often, the VM supports the same ISA as the underlying hardware; however, it is also possible to support a different ISA, and such approaches are often employed when migrating between ISAs in order to allow software from the departing ISA to be used until it can be ported to the new ISA. Our focus here will be on VMs where the ISA presented by the VM and the underlying hardware match. Such VMs are called (operating) *system virtual machines*. IBM VM/370, VMware ESX Server, and Xen are examples. They present the illusion that the users of a VM have an entire computer to themselves, including a copy of the operating system. A single computer runs multiple VMs and can support a number of different operating systems (OSes). On a conventional platform, a single OS "owns" all the hardware resources, but with a VM, multiple OSes all share the hardware resources.

The software that supports VMs is called a *virtual machine monitor* (VMM) or *hypervisor*; the VMM is the heart of virtual machine technology. The underlying hardware platform is called the *host*, and its resources are shared among the *guest* VMs. The VMM determines how to map virtual resources to physical resources: A physical resource may be time-shared, partitioned, or even emulated in software. The VMM is much smaller than a traditional OS; the isolation portion of a VMM is perhaps only 10,000 lines of code.

In general, the cost of processor virtualization depends on the workload. User-level processor-bound programs, such as SPECCPU2006, have zero virtualization overhead because the OS is rarely invoked, so everything runs at native speeds. Conversely, I/O-intensive workloads generally are also OS-intensive and execute many system calls (which doing I/O requires) and privileged instructions that can result in high virtualization overhead. The overhead is determined by the number of instructions that must be emulated by the VMM and how slowly they are emulated. Therefore, when the guest VMs run the same ISA as the host, as we assume here, the goal of the architecture and the VMM is to run almost all instructions directly on the native hardware. On the other hand, if the I/O-intensive workload is also *I/O-bound*, the cost of processor virtualization can be completely hidden by low processor utilization because it is often waiting for I/O.

Although our interest here is in VMs for improving protection, VMs provide two other benefits that are commercially significant:

1. *Managing software*—VMs provide an abstraction that can run the complete software stack, even including old operating systems such as DOS. A typical deployment might be some VMs running legacy OSes, many running the current stable OS release, and a few testing the next OS release.
2. *Managing hardware*—One reason for multiple servers is to have each application running with its own compatible version of the operating system on separate computers, as this separation can improve dependability. VMs allow these separate software stacks to run independently yet share hardware, thereby consolidating the number of servers. Another example is that most newer VMMs support migration of a running VM to a different computer, either to

balance load or to evacuate from failing hardware. The rise of cloud computing has made the ability to swap out an entire VM to another physical processor increasingly useful.

These two reasons are why cloud-based servers, such as Amazon's, rely on virtual machines.

Requirements of a Virtual Machine Monitor

What must a VM monitor do? It presents a software interface to guest software, it must isolate the state of guests from each other, and it must protect itself from guest software (including guest OSes). The qualitative requirements are

- Guest software should behave on a VM exactly as if it were running on the native hardware, except for performance-related behavior or limitations of fixed resources shared by multiple VMs.
- Guest software should not be able to directly change allocation of real system resources.

To "virtualize" the processor, the VMM must control just about everything—access to privileged state, address translation, I/O, exceptions and interrupts—even though the guest VM and OS currently running are temporarily using them.

For example, in the case of a timer interrupt, the VMM would suspend the currently running guest VM, save its state, handle the interrupt, determine which guest VM to run next, and then load its state. Guest VMs that rely on a timer interrupt are provided with a virtual timer and an emulated timer interrupt by the VMM.

To be in charge, the VMM must be at a higher privilege level than the guest VM, which generally runs in user mode; this also ensures that the execution of any privileged instruction will be handled by the VMM. The basic requirements of system virtual machines are almost identical to those for the previously mentioned paged virtual memory:

- At least two processor modes, system and user.
- A privileged subset of instructions that is available only in system mode, resulting in a trap if executed in user mode. All system resources must be controllable only via these instructions.

Instruction Set Architecture Support for Virtual Machines

If VMs are planned for during the design of the ISA, it's relatively easy to reduce both the number of instructions that must be executed by a VMM and how long it takes to emulate them. An architecture that allows the VM to execute directly on the hardware earns the title *virtualizable*, and the IBM 370 architecture proudly bears that label.

However, because VMs have been considered for desktop and PC-based server applications only fairly recently, most instruction sets were created without virtualization in mind. These culprits include 80x86 and most of the original RISC architectures, although the latter had fewer issues than the 80x86 architecture. Recent additions to the x86 architecture have attempted to remedy the earlier shortcomings, and RISC V explicitly includes support for virtualization.

Because the VMM must ensure that the guest system interacts only with virtual resources, a conventional guest OS runs as a user mode program on top of the VMM. Then, if a guest OS attempts to access or modify information related to hardware resources via a privileged instruction—for example, reading or writing the page table pointer—it will trap to the VMM. The VMM can then effect the appropriate changes to corresponding real resources.

Therefore, if any instruction that tries to read or write such sensitive information traps when executed in user mode, the VMM can intercept it and support a virtual version of the sensitive information as the guest OS expects.

In the absence of such support, other measures must be taken. A VMM must take special precautions to locate all problematic instructions and ensure that they behave correctly when executed by a guest OS, thereby increasing the complexity of the VMM and reducing the performance of running the VM. Sections 2.5 and 2.7 give concrete examples of problematic instructions in the 80x86 architecture. One attractive extension allows the VM and the OS to operate at different privilege levels, each of which is distinct from the user level. By introducing an additional privilege level, some OS operations—e.g., those that exceed the permissions granted to a user program but do not require intervention by the VMM (because they cannot affect any other VM)—can execute directly without the overhead of trapping and invoking the VMM. The Xen design, which we examine shortly, makes use of three privilege levels.

Impact of Virtual Machines on Virtual Memory and I/O

Another challenge is virtualization of virtual memory, as each guest OS in every VM manages its own set of page tables. To make this work, the VMM separates the notions of *real* and *physical memory* (which are often treated synonymously) and makes real memory a separate, intermediate level between virtual memory and physical memory. (Some use the terms *virtual memory, physical memory*, and *machine memory* to name the same three levels.) The guest OS maps virtual memory to real memory via its page tables, and the VMM page tables map the guests' real memory to physical memory. The virtual memory architecture is specified either via page tables, as in IBM VM/370 and the 80x86, or via the TLB structure, as in many RISC architectures.

Rather than pay an extra level of indirection on every memory access, the VMM maintains a *shadow page table* that maps directly from the guest virtual address space to the physical address space of the hardware. By detecting all modifications to the guest's page table, the VMM can ensure that the shadow page table

entries being used by the hardware for translations correspond to those of the guest OS environment, with the exception of the correct physical pages substituted for the real pages in the guest tables. Therefore the VMM must trap any attempt by the guest OS to change its page table or to access the page table pointer. This is commonly done by write protecting the guest page tables and trapping any access to the page table pointer by a guest OS. As previously noted, the latter happens naturally if accessing the page table pointer is a privileged operation.

The IBM 370 architecture solved the page table problem in the 1970s with an additional level of indirection that is managed by the VMM. The guest OS keeps its page tables as before, so the shadow pages are unnecessary. AMD has implemented a similar scheme for its 80x86.

To virtualize the TLB in many RISC computers, the VMM manages the real TLB and has a copy of the contents of the TLB of each guest VM. To pull this off, any instructions that access the TLB must trap. TLBs with Process ID tags can support a mix of entries from different VMs and the VMM, thereby avoiding flushing of the TLB on a VM switch. Meanwhile, in the background, the VMM supports a mapping between the VMs' virtual Process IDs and the real Process IDs. Section L.7 of online Appendix L describes additional details.

The final portion of the architecture to virtualize is I/O. This is by far the most difficult part of system virtualization because of the increasing number of I/O devices attached to the computer *and* the increasing diversity of I/O device types. Another difficulty is the sharing of a real device among multiple VMs, and yet another comes from supporting the myriad of device drivers that are required, especially if different guest OSes are supported on the same VM system. The VM illusion can be maintained by giving each VM generic versions of each type of I/O device driver, and then leaving it to the VMM to handle real I/O.

The method for mapping a virtual-to-physical I/O device depends on the type of device. For example, physical disks are normally partitioned by the VMM to create virtual disks for guest VMs, and the VMM maintains the mapping of virtual tracks and sectors to the physical ones. Network interfaces are often shared between VMs in very short time slices, and the job of the VMM is to keep track of messages for the virtual network addresses to ensure that guest VMs receive only messages intended for them.

Extending the Instruction Set for Efficient Virtualization and Better Security

In the past 5–10 years, processor designers, including those at AMD and Intel (and to a lesser extent ARM), have introduced instruction set extensions to more efficiently support virtualization. Two primary areas of performance improvement have been in handling page tables and TLBs (the cornerstone of virtual memory) and in I/O, specifically handling interrupts and DMA. Virtual memory performance is enhanced by avoiding unnecessary TLB flushes and by using the nested page table mechanism, employed by IBM decades earlier, rather than a complete

set of shadow page tables (see Section L.7 in Appendix L). To improve I/O performance, architectural extensions are added that allow a device to directly use DMA to move data (eliminating a potential copy by the VMM) and allow device interrupts and commands to be handled by the guest OS directly. These extensions show significant performance gains in applications that are intensive either in their memory-management aspects or in the use of I/O.

With the broad adoption of public cloud systems for running critical applications, concerns have risen about security of data in such applications. Any malicious code that is able to access a higher privilege level than data that must be kept secure compromises the system. For example, if you are running a credit card processing application, you must be absolutely certain that malicious users cannot get access to the credit card numbers, even when they are using the same hardware and intentionally attack the OS or even the VMM. Through the use of virtualization, we can prevent accesses by an outside user to the data in a different VM, and this provides significant protection compared to a multiprogrammed environment. That might not be enough, however, if the attacker compromises the VMM or can find out information by observations in another VMM. For example, suppose the attacker penetrates the VMM; the attacker can then remap memory so as to access any portion of the data.

Alternatively, an attack might rely on a Trojan horse (see Appendix B) introduced into the code that can access the credit cards. Because the Trojan horse is running in the same VM as the credit card processing application, the Trojan horse only needs to exploit an OS flaw to gain access to the critical data. Most cyberattacks have used some form of Trojan horse, typically exploiting an OS flaw, that either has the effect of returning access to the attacker while leaving the CPU still in privilege mode or allows the attacker to upload and execute code as if it were part of the OS. In either case, the attacker obtains control of the CPU and, using the higher privilege mode, can proceed to access anything within the VM. Note that encryption alone does not prevent this attacker. If the data in memory is unencrypted, which is typical, then the attacker has access to all such data. Furthermore, if the attacker knows where the encryption key is stored, the attacker can freely access the key and then access any encrypted data.

More recently, Intel introduced a set of instruction set extensions, called the software guard extensions (SGX), to allow user programs to create *enclaves*, portions of code and data that are always encrypted and decrypted only on use and only with the key provided by the user code. Because the enclave is always encrypted, standard OS operations for virtual memory or I/O can access the enclave (e.g., to move a page) but cannot extract any information. For an enclave to work, all the code and all the data required must be part of the enclave. Although the topic of finer-grained protection has been around for decades, it has gotten little traction before because of the high overhead and because other solutions that are more efficient and less intrusive have been acceptable. The rise of cyberattacks and the amount of confidential information online have led to a reexamination of techniques for improving such fine-grained security. Like Intel's SGX, IBM and AMD's recent processors support on-the-fly encryption of memory.

An Example VMM: The Xen Virtual Machine

Early in the development of VMs, a number of inefficiencies became apparent. For example, a guest OS manages its virtual-to-real page mapping, but this mapping is ignored by the VMM, which performs the actual mapping to physical pages. In other words, a significant amount of wasted effort is expended just to keep the guest OS happy. To reduce such inefficiencies, VMM developers decided that it may be worthwhile to allow the guest OS to be aware that it is running on a VM. For example, a guest OS could assume a real memory as large as its virtual memory so that no memory management is required by the guest OS.

Allowing small modifications to the guest OS to simplify virtualization is referred to as *paravirtualization*, and the open source Xen VMM is a good example. The Xen VMM, which is used in Amazon's web services data centers, provides a guest OS with a virtual machine abstraction that is similar to the physical hardware, but drops many of the troublesome pieces. For example, to avoid flushing the TLB, Xen maps itself into the upper 64 MiB of the address space of each VM. Xen allows the guest OS to allocate pages, checking only to be sure the guest OS does not violate protection restrictions. To protect the guest OS from the user programs in the VM, Xen takes advantage of the four protection levels available in the 80x86. The Xen VMM runs at the highest privilege level (0), the guest OS runs at the next level (1), and the applications run at the lowest privilege level (3). Most OSes for the 80x 86 keep everything at privilege levels 0 or 3.

For subsetting to work properly, Xen modifies the guest OS to not use problematic portions of the architecture. For example, the port of Linux to Xen changes about 3000 lines, or about 1% of the 80x86-specific code. These changes, however, do not affect the application binary interfaces of the guest OS.

To simplify the I/O challenge of VMs, Xen assigned privileged virtual machines to each hardware I/O device. These special VMs are called *driver domains*. (Xen calls VMs "domains.") Driver domains run the physical device drivers, although interrupts are still handled by the VMM before being sent to the appropriate driver domain. Regular VMs, called *guest domains*, run simple virtual device drivers that must communicate with the physical device drivers in the driver domains over a channel to access the physical I/O hardware. Data are sent between guest and driver domains by page remapping.

2.5 Cross-Cutting Issues: The Design of Memory Hierarchies

This section describes four topics discussed in other chapters that are fundamental to memory hierarchies.

Protection, Virtualization, and Instruction Set Architecture

Protection is a joint effort of architecture and operating systems, but architects had to modify some awkward details of existing instruction set architectures when virtual memory became popular. For example, to support virtual memory in the IBM

370, architects had to change the successful IBM 360 instruction set architecture that had been announced just 6 years before. Similar adjustments are being made today to accommodate virtual machines.

For example, the 80x86 instruction `POPF` loads the flag registers from the top of the stack in memory. One of the flags is the Interrupt Enable (IE) flag. Until recent changes to support virtualization, running the `POPF` instruction in user mode, rather than trapping it, simply changed all the flags except IE. In system mode, it does change the IE flag. Because a guest OS runs in user mode inside a VM, this was a problem, as the OS would expect to see a changed IE. Extensions of the 80x86 architecture to support virtualization eliminated this problem.

Historically, IBM mainframe hardware and VMM took three steps to improve performance of virtual machines:

1. Reduce the cost of processor virtualization.
2. Reduce interrupt overhead cost due to the virtualization.
3. Reduce interrupt cost by steering interrupts to the proper VM without invoking VMM.

IBM is still the gold standard of virtual machine technology. For example, an IBM mainframe ran thousands of Linux VMs in 2000, while Xen ran 25 VMs in 2004 (Clark et al., 2004). Recent versions of Intel and AMD chipsets have added special instructions to support devices in a VM to mask interrupts at lower levels from each VM and to steer interrupts to the appropriate VM.

Autonomous Instruction Fetch Units

Many processors with out-of-order execution and even some with simply deep pipelines decouple the instruction fetch (and sometimes initial decode), using a separate instruction fetch unit (see Chapter 3). Typically, the instruction fetch unit accesses the instruction cache to fetch an entire block before decoding it into individual instructions; such a technique is particularly useful when the instruction length varies. Because the instruction cache is accessed in blocks, it no longer makes sense to compare miss rates to processors that access the instruction cache once per instruction. In addition, the instruction fetch unit may prefetch blocks into the L1 cache; these prefetches may generate additional misses, but may actually reduce the total miss penalty incurred. Many processors also include data prefetching, which may increase the data cache miss rate, even while decreasing the total data cache miss penalty.

Speculation and Memory Access

One of the major techniques used in advanced pipelines is speculation, whereby an instruction is tentatively executed before the processor knows whether it is really needed. Such techniques rely on branch prediction, which if incorrect requires that

the speculated instructions are flushed from the pipeline. There are two separate issues in a memory system supporting speculation: protection and performance. With speculation, the processor may generate memory references, which will never be used because the instructions were the result of incorrect speculation. Those references, if executed, could generate protection exceptions. Obviously, such faults should occur only if the instruction is actually executed. In the next chapter, we will see how such "speculative exceptions" are resolved. Because a speculative processor may generate accesses to both the instruction and data caches, and subsequently not use the results of those accesses, speculation may increase the cache miss rates. As with prefetching, however, such speculation may actually lower the total cache miss penalty. The use of speculation, like the use of prefetching, makes it misleading to compare miss rates to those seen in processors without speculation, even when the ISA and cache structures are otherwise identical.

Special Instruction Caches

One of the biggest challenges in superscalar processors is to supply the instruction bandwidth. For designs that translate the instructions into micro-operations, such as most recent Arm and i7 processors, instruction bandwidth demands and branch misprediction penalties can be reduced by keeping a small cache of recently translated instructions. We explore this technique in greater depth in the next chapter.

Coherency of Cached Data

Data can be found in memory and in the cache. As long as the processor is the sole component changing or reading the data and the cache stands between the processor and memory, there is little danger in the processor seeing the old or *stale* copy. As we will see, multiple processors and I/O devices raise the opportunity for copies to be inconsistent and to read the wrong copy.

The frequency of the cache coherency problem is different for multiprocessors than for I/O. Multiple data copies are a rare event for I/O—one to be avoided whenever possible—but a program running on multiple processors will *want* to have copies of the same data in several caches. Performance of a multiprocessor program depends on the performance of the system when sharing data.

The *I/O cache coherency* question is this: where does the I/O occur in the computer—between the I/O device and the cache or between the I/O device and main memory? If input puts data into the cache and output reads data from the cache, both I/O and the processor see the same data. The difficulty in this approach is that it interferes with the processor and can cause the processor to stall for I/O. Input may also interfere with the cache by displacing some information with new data that are unlikely to be accessed soon.

The goal for the I/O system in a computer with a cache is to prevent the stale data problem while interfering as little as possible. Many systems therefore prefer that I/O occur directly to main memory, with main memory acting as an I/O buffer. If a write-through cache were used, then memory would have an up-to-date copy of the information, and there would be no stale data issue for output. (This benefit is a reason processors used write through.) However, today write through is usually found only in first-level data caches backed by an L2 cache that uses write back.

Input requires some extra work. The software solution is to guarantee that no blocks of the input buffer are in the cache. A page containing the buffer can be marked as noncachable, and the operating system can always input to such a page. Alternatively, the operating system can flush the buffer addresses from the cache before the input occurs. A hardware solution is to check the I/O addresses on input to see if they are in the cache. If there is a match of I/O addresses in the cache, the cache entries are invalidated to avoid stale data. All of these approaches can also be used for output with write-back caches.

Processor cache coherency is a critical subject in the age of multicore processors, and we will examine it in detail in Chapter 5.

2.6 Putting It All Together: Memory Hierarchies in the ARM Cortex-A53 and Intel Core i7 6700

This section reveals the ARM Cortex-A53 (hereafter called the A53) and Intel Core i76700 (hereafter called i7) memory hierarchies and shows the performance of their components on a set of single-threaded benchmarks. We examine the Cortex-A53 first because it has a simpler memory system; we go into more detail for the i7, tracing out a memory reference in detail. This section presumes that readers are familiar with the organization of a two-level cache hierarchy using virtually indexed caches. The basics of such a memory system are explained in detail in Appendix B, and readers who are uncertain of the organization of such a system are strongly advised to review the Opteron example in Appendix B. Once they understand the organization of the Opteron, the brief explanation of the A53 system, which is similar, will be easy to follow.

The ARM Cortex-A53

The Cortex-A53 is a configurable core that supports the ARMv8A instruction set architecture, which includes both 32-bit and 64-bit modes. The Cortex-A53 is delivered as an IP (intellectual property) core. IP cores are the dominant form of technology delivery in the embedded, PMD, and related markets; billions of ARM and MIPS processors have been created from these IP cores. Note that IP cores are different from the cores in the Intel i7 or AMD Athlon multicores. An IP core (which may itself be a multicore) is designed to be incorporated with other logic (thus it is the core of a chip), including application-specific processors

(such as an encoder or decoder for video), I/O interfaces, and memory interfaces, and then fabricated to yield a processor optimized for a particular application. For example, the Cortex-A53 IP core is used in a variety of tablets and smartphones; it is designed to be highly energy-efficient, a key criteria in battery-based PMDs. The A53 core is capable of being configured with multiple cores per chip for use in high-end PMDs; our discussion here focuses on a single core.

Generally, IP cores come in two flavors. *Hard cores* are optimized for a particular semiconductor vendor and are black boxes with external (but still on-chip) interfaces. Hard cores typically allow parametrization only of logic outside the core, such as L2 cache sizes, and the IP core cannot be modified. *Soft cores* are usually delivered in a form that uses a standard library of logic elements. A soft core can be compiled for different semiconductor vendors and can also be modified, although extensive modifications are very difficult because of the complexity of modern-day IP cores. In general, hard cores provide higher performance and smaller die area, while soft cores allow retargeting to other vendors and can be more easily modified.

The Cortex-A53 can issue two instructions per clock at clock rates up to 1.3 GHz. It supports both a two-level TLB and a two-level cache; Figure 2.19 summarizes the organization of the memory hierarchy. The critical term is returned first, and the processor can continue while the miss completes; a memory system with up to four banks can be supported. For a D-cache of 32 KiB and a page size of 4 KiB, each physical page could map to two different cache addresses; such aliases are avoided by hardware detection on a miss as in Section B.3 of Appendix B. Figure 2.20 shows how the 32-bit virtual address is used to index the TLB and the caches, assuming 32 KiB primary caches and a 1 MiB secondary cache with 16 KiB page size.

Structure	Size	Organization	Typical miss penalty (clock cycles)
Instruction MicroTLB	10 entries	Fully associative	2
Data MicroTLB	10 entries	Fully associative	2
L2 Unified TLB	512 entries	4-way set associative	20
L1 Instruction cache	8–64 KiB	2-way set associative; 64-byte block	13
L1 Data cache	8–64 KiB	2-way set associative; 64-byte block	13
L2 Unified cache	128 KiB to 2 MiB	16-way set associative; LRU	124

Figure 2.19 The memory hierarchy of the Cortex A53 includes multilevel TLBs and caches. A page map cache keeps track of the location of a physical page for a set of virtual pages; it reduces the L2 TLB miss penalty. The L1 caches are virtually indexed and physically tagged; both the L1 D cache and L2 use a write-back policy defaulting to allocate on write. Replacement policy is LRU approximation in all the caches. Miss penalties to L2 are higher if both a MicroTLB and L1 miss occur. The L2 to main memory bus is 64–128 bits wide, and the miss penalty is larger for the narrow bus.

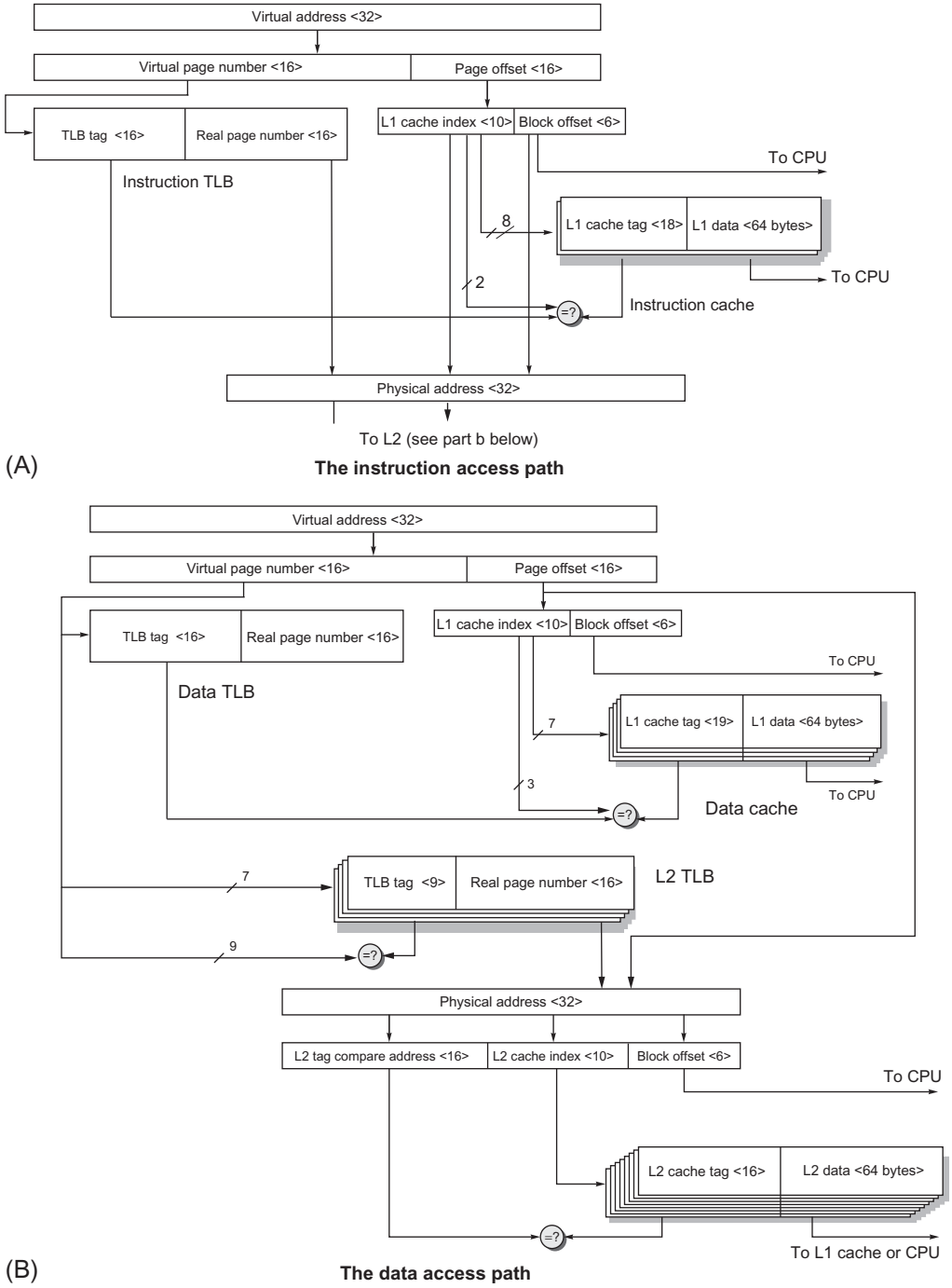


Figure 2.20 The virtual address, physical and data blocks for the ARM Cortex-A53 caches and TLBs, assuming 32-bit addresses. The top half (A) shows the instruction access; the bottom half (B) shows the data access, including L2. The TLB (instruction or data) is fully associative each with 10 entries, using a 64 KiB page in this example. The L1 I-cache is two-way set associative, with 64-byte blocks and 32 KiB capacity; the L1 D-cache is 32 KiB, four-way set associative, and 64-byte blocks. The L2 TLB is 512 entries and four-way set associative. The L2 cache is 16-way set associative with 64-byte blocks and 128 cKiB to 2 MiB capacity; a 1 MiB L2 is shown. This figure doesn't show the valid bits and protection bits for the caches and TLB.

Performance of the Cortex-A53 Memory Hierarchy

The memory hierarchy of the Cortex-A8 was measured with 32 KiB primary caches and a 1 MiB L2 cache running the SPECInt2006 benchmarks. The instruction cache miss rates for these SPECInt2006 are very small even for just the L1: close to zero for most and under 1% for all of them. This low rate probably results from the computationally intensive nature of the SPECCPU programs and the two-way set associative cache that eliminates most conflict misses.

Figure 2.21 shows the data cache results, which have significant L1 and L2 miss rates. The L1 rate varies by a factor of 75, from 0.5% to 37.3% with a median miss rate of 2.4%. The global L2 miss rate varies by a factor of 180, from 0.05% to 9.0% with a median of 0.3%. MCF, which is known as a cache buster, sets the upper bound and significantly affects the mean. Remember that the L2 global miss rate is significantly lower than the L2 local miss rate; for example, the median L2 stand-alone miss rate is 15.1% versus the global miss rate of 0.3%.

Using these miss penalties in Figure 2.19, Figure 2.22 shows the average penalty per data access. Although the L1 miss rates are about seven times higher than the L2 miss rate, the L2 penalty is 9.5 times as high, leading to L2 misses slightly dominating for the benchmarks that stress the memory system. In the next chapter, we will examine the impact of the cache misses on overall CPI.

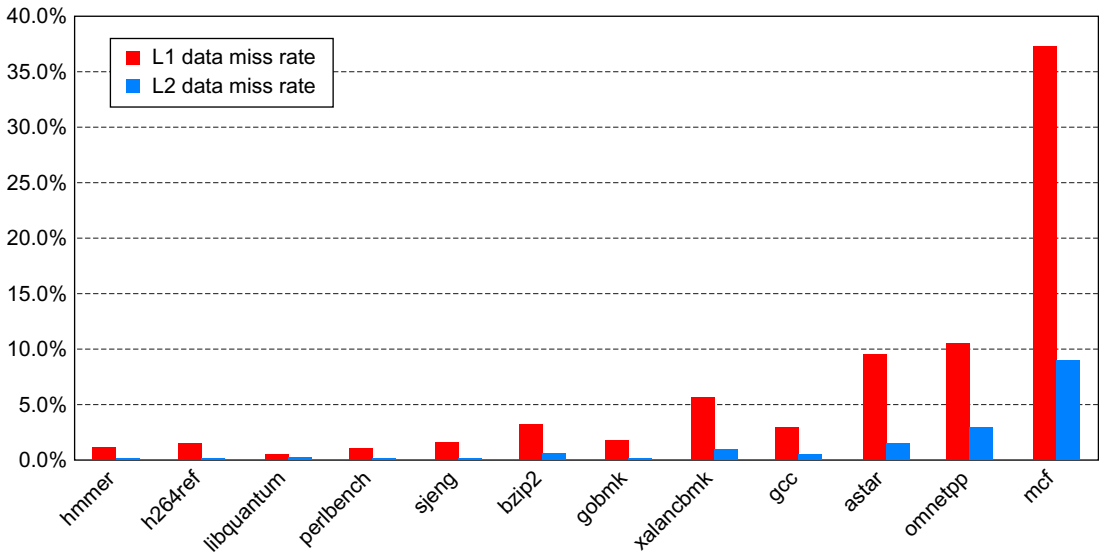


Figure 2.21 The data miss rate for ARM with a 32 KiB L1 and the global data miss rate for a 1 MiB L2 using the SPECInt2006 benchmarks are significantly affected by the applications. Applications with larger memory footprints tend to have higher miss rates in both L1 and L2. Note that the L2 rate is the global miss rate that is counting all references, including those that hit in L1. MCF is known as a cache buster.

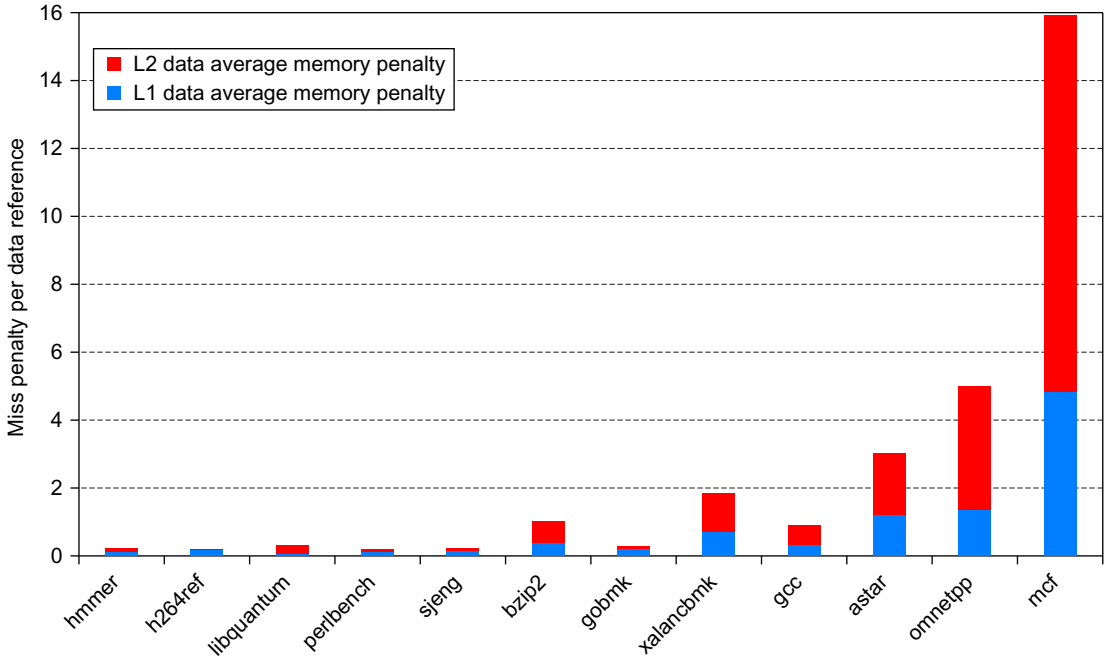


Figure 2.22 The average memory access penalty per data memory reference coming from L1 and L2 is shown for the A53 processor when running SPECInt2006. Although the miss rates for L1 are significantly higher, the L2 miss penalty, which is more than five times higher, means that the L2 misses can contribute significantly.

The Intel Core i7 6700

The i7 supports the x86-64 instruction set architecture, a 64-bit extension of the 80x86 architecture. The i7 is an out-of-order execution processor that includes four cores. In this chapter, we focus on the memory system design and performance from the viewpoint of a single core. The system performance of multiprocessor designs, including the i7 multicore, is examined in detail in Chapter 5.

Each core in an i7 can execute up to four 80x86 instructions per clock cycle, using a multiple issue, dynamically scheduled, 16-stage pipeline, which we describe in detail in Chapter 3. The i7 can also support up to two simultaneous threads per processor, using a technique called simultaneous multithreading, described in Chapter 4. In 2017 the fastest i7 had a clock rate of 4.0 GHz (in Turbo Boost mode), which yielded a peak instruction execution rate of 16 billion instructions per second, or 64 billion instructions per second for the four-core design. Of course, there is a big gap between peak and sustained performance, as we will see over the next few chapters.

The i7 can support up to three memory channels, each consisting of a separate set of DIMMs, and each of which can transfer in parallel. Using DDR3-1066 (DIMM PC8500), the i7 has a peak memory bandwidth of just over 25 GB/s.

i7 uses 48-bit virtual addresses and 36-bit physical addresses, yielding a maximum physical memory of 36 GiB. Memory management is handled with a two-level TLB (see Appendix B, Section B.4), summarized in Figure 2.23.

Figure 2.24 summarizes the i7's three-level cache hierarchy. The first-level caches are virtually indexed and physically tagged (see Appendix B, Section B.3), while the L2 and L3 caches are physically indexed. Some versions of the i7 6700 will support a fourth-level cache using HBM packaging.

Figure 2.25 is labeled with the steps of an access to the memory hierarchy. First, the PC is sent to the instruction cache. The instruction cache index is

$$2^{\text{Index}} = \frac{\text{Cache size}}{\text{Block size} \times \text{Set associativity}} = \frac{32\,\text{K}}{64 \times 8} = 64 = 2^6$$

Characteristic	Instruction TLB	Data DLB	Second-level TLB
Entries	128	64	1536
Associativity	8-way	4-way	12-way
Replacement	Pseudo-LRU	Pseudo-LRU	Pseudo-LRU
Access latency	1 cycle	1 cycle	8 cycles
Miss	9 cycles	9 cycles	Hundreds of cycles to access page table

Figure 2.23 Characteristics of the i7's TLB structure, which has separate first-level instruction and data TLBs, both backed by a joint second-level TLB. The first-level TLBs support the standard 4 KiB page size, as well as having a limited number of entries of large 2–4 MiB pages; only 4 KiB pages are supported in the second-level TLB. The i7 has the ability to handle two L2 TLB misses in parallel. See Section L.3 of online Appendix L for more discussion of multilevel TLBs and support for multiple page sizes.

Characteristic	L1	L2	L3
Size	32 KiB I/32 KiB D	256 KiB	2 MiB per core
Associativity	both 8-way	4-way	16-way
Access latency	4 cycles, pipelined	12 cycles	44 cycles
Replacement scheme	Pseudo-LRU	Pseudo-LRU	Pseudo-LRU but with an ordered selection algorithm

Figure 2.24 Characteristics of the three-level cache hierarchy in the i7. All three caches use write back and a block size of 64 bytes. The L1 and L2 caches are separate for each core, whereas the L3 cache is shared among the cores on a chip and is a total of 2 MiB per core. All three caches are nonblocking and allow multiple outstanding writes. A merging write buffer is used for the L1 cache, which holds data in the event that the line is not present in L1 when it is written. (That is, an L1 write miss does not cause the line to be allocated.) L3 is inclusive of L1 and L2; we explore this property in further detail when we explain multiprocessor caches. Replacement is by a variant on pseudo-LRU; in the case of L3, the block replaced is always the lowest numbered way whose access bit is off. This is not quite random but is easy to compute.

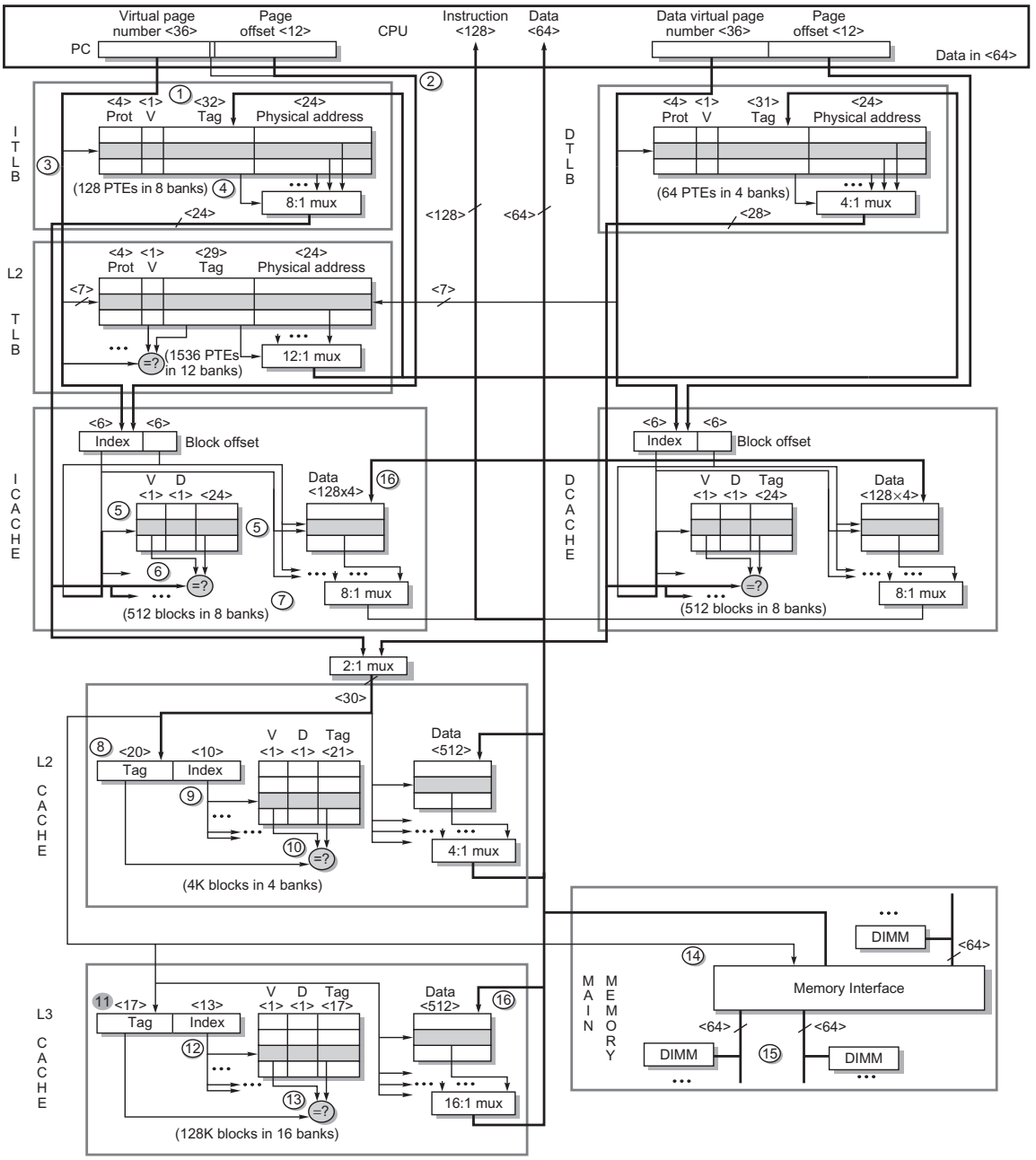


Figure 2.25 The Intel i7 memory hierarchy and the steps in both instruction and data access. We show only reads. Writes are similar, except that misses are handled by simply placing the data in a write buffer, because the L1 cache is not write-allocated.

or 6 bits. The page frame of the instruction's address (36 = 48 − 12 bits) is sent to the instruction TLB (step 1). At the same time, the 12-bit page offset from the virtual address is sent to the instruction cache (step 2). Notice that for the eight-way associative instruction cache, 12 bits are needed for the cache address: 6 bits to index the cache plus 6 bits of block offset for the 64-byte block, so no aliases are possible. The previous versions of the i7 used a four-way set associative I-cache, meaning that a block corresponding to a virtual address could actually be in two different places in the cache, because the corresponding physical address could have either a 0 or 1 in this location. For instructions this did not pose a problem because even if an instruction appeared in the cache in two different locations, the two versions must be the same. If such duplication, or aliasing, of data is allowed, the cache must be checked when the page map is changed, which is an infrequent event. Note that a very simple use of page coloring (see Appendix B, Section B.3) can eliminate the possibility of these aliases. If even-address virtual pages are mapped to even-address physical pages (and the same for odd pages), then these aliases can never occur because the low-order bit in the virtual and physical page number will be identical.

The instruction TLB is accessed to find a match between the address and a valid page table entry (PTE) (steps 3 and 4). In addition to translating the address, the TLB checks to see if the PTE demands that this access result in an exception because of an access violation.

An instruction TLB miss first goes to the L2 TLB, which contains 1536 PTEs of 4 KiB page sizes and is 12-way set associative. It takes 8 clock cycles to load the L1 TLB from the L2 TLB, which leads to the 9-cycle miss penalty including the initial clock cycle to access the L1 TLB. If the L2 TLB misses, a hardware algorithm is used to walk the page table and update the TLB entry. Sections L.5 and L.6 of online Appendix L describe page table walkers and page structure caches. In the worst case, the page is not in memory, and the operating system gets the page from secondary storage. Because millions of instructions could execute during a page fault, the operating system will swap in another process if one is waiting to run. Otherwise, if there is no TLB exception, the instruction cache access continues.

The index field of the address is sent to all eight banks of the instruction cache (step 5). The instruction cache tag is 36 bits − 6 bits (index) − 6 bits (block offset), or 24 bits. The four tags and valid bits are compared to the physical page frame from the instruction TLB (step 6). Because the i7 expects 16 bytes each instruction fetch, an additional 2 bits are used from the 6-bit block offset to select the appropriate 16 bytes. Therefore 6 + 2 or 8 bits are used to send 16 bytes of instructions to the processor. The L1 cache is pipelined, and the latency of a hit is 4 clock cycles (step 7). A miss goes to the second-level cache.

As mentioned earlier, the instruction cache is virtually addressed and physically tagged. Because the second-level caches are physically addressed, the physical page address from the TLB is composed with the page offset to make an address to access the L2 cache. The L2 index is

$$2^{\text{Index}} = \frac{\text{Cache size}}{\text{Block size} \times \text{Set associativity}} = \frac{256\text{K}}{64 \times 4} = 1024 = 2^{10}$$

so the 30-bit block address (36-bit physical address − 6-bit block offset) is divided into a 20-bit tag and a 10-bit index (step 8). Once again, the index and tag are sent to the four banks of the unified L2 cache (step 9), which are compared in parallel. If one matches and is valid (step 10), it returns the block in sequential order after the initial 12-cycle latency at a rate of 8 bytes per clock cycle.

If the L2 cache misses, the L3 cache is accessed. For a four-core i7, which has an 8 MiB L3, the index size is

$$2^{\text{Index}} = \frac{\text{Cache size}}{\text{Block size} \times \text{Set associativity}} = \frac{8\text{M}}{64 \times 16} = 8192 = 2^{13}$$

The 13-bit index (step 11) is sent to all 16 banks of the L3 (step 12). The L3 tag, which is $36 - (13 + 6) = 17$ bits, is compared against the physical address from the TLB (step 13). If a hit occurs, the block is returned after an initial latency of 42 clock cycles, at a rate of 16 bytes per clock and placed into both L1 and L3. If L3 misses, a memory access is initiated.

If the instruction is not found in the L3 cache, the on-chip memory controller must get the block from main memory. The i7 has three 64-bit memory channels that can act as one 192-bit channel, because there is only one memory controller and the same address is sent on both channels (step 14). Wide transfers happen when both channels have identical DIMMs. Each channel supports up to four DDR DIMMs (step 15). When the data return they are placed into L3 and L1 (step 16) because L3 is inclusive.

The total latency of the instruction miss that is serviced by main memory is approximately 42 processor cycles to determine that an L3 miss has occurred, plus the DRAM latency for the critical instructions. For a single-bank DDR4-2400 SDRAM and 4.0 GHz CPU, the DRAM latency is about 40 ns or 160 clock cycles to the first 16 bytes, leading to a total miss penalty of about 200 clock cycles. The memory controller fills the remainder of the 64-byte cache block at a rate of 16 bytes per I/O bus clock cycle, which takes another 5 ns or 20 clock cycles.

Because the second-level cache is a write-back cache, any miss can lead to an old block being written back to memory. The i7 has a 10-entry merging write buffer that writes back dirty cache lines when the next level in the cache is unused for a read. The write buffer is checked on a miss to see if the cache line exists in the buffer; if so, the miss is filled from the buffer. A similar buffer is used between the L1 and L2 caches. If this initial instruction is a load, the data address is sent to the data cache and data TLBs, acting very much like an instruction cache access.

Suppose the instruction is a store instead of a load. When the store issues, it does a data cache lookup just like a load. A miss causes the block to be placed in a write buffer because the L1 cache does not allocate the block on a write miss. On a hit, the store does not update the L1 (or L2) cache until later, after it is known to be nonspeculative. During this time, the store resides in a load-store queue, part of the out-of-order control mechanism of the processor.

The I7 also supports prefetching for L1 and L2 from the next level in the hierarchy. In most cases, the prefetched line is simply the next block in the cache. By prefetching only for L1 and L2, high-cost unnecessary fetches to memory are avoided.

Performance of the i7 memory system

We evaluate the performance of the i7 cache structure using the SPECint2006 benchmarks. The data in this section were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University. Their analysis is based on earlier work (see Prakash and Peng, 2008).

The complexity of the i7 pipeline, with its use of an autonomous instruction fetch unit, speculation, and both instruction and data prefetch, makes it hard to compare cache performance against simpler processors. As mentioned on page 110, processors that use prefetch can generate cache accesses independent of the memory accesses performed by the program. A cache access that is generated because of an actual instruction access or data access is sometimes called a *demand access* to distinguish it from a *prefetch access*. Demand accesses can come from both speculative instruction fetches and speculative data accesses, some of which are subsequently canceled (see Chapter 3 for a detailed description of speculation and instruction graduation). A speculative processor generates at least as many misses as an in-order nonspeculative processor, and typically more. In addition to demand misses, there are prefetch misses for both instructions and data.

The i7's instruction fetch unit attempts to fetch 16 bytes every cycle, which complicates comparing instruction cache miss rates because multiple instructions are fetched every cycle (roughly 4.5 on average). In fact, the entire 64-byte cache line is read and subsequent 16-byte fetches do not require additional accesses. Thus misses are tracked only on the basis of 64-byte blocks. The 32 KiB, eight-way set associative instruction cache leads to a very low instruction miss rate for the SPECint2006 programs. If, for simplicity, we measure the miss rate of SPECint2006 as the number of misses for a 64-byte block divided by the number of instructions that complete, the miss rates are all under 1% except for one benchmark (XALANCBMK), which has a 2.9% miss rate. Because a 64-byte block typically contains 16–20 instructions, the effective miss rate per instruction is much lower, depending on the degree of spatial locality in the instruction stream.

The frequency at which the instruction fetch unit is stalled waiting for the I-cache misses is similarly small (as a percentage of total cycles) increasing to 2% for two benchmarks and 12% for XALANCBMK, which has the highest I-cache miss rate. In the next chapter, we will see how stalls in the IFU contribute to overall reductions in pipeline throughput in the i7.

The L1 data cache is more interesting and even trickier to evaluate because in addition to the effects of prefetching and speculation, the L1 data cache is not write-allocated, and writes to cache blocks that are not present are not treated as misses. For this reason, we focus only on memory reads. The performance monitor measurements in the i7 separate out prefetch accesses from demand accesses, but only keep demand accesses for those instructions that graduate. The effect of speculative instructions that do not graduate is not negligible, although pipeline effects probably dominate secondary cache effects caused by speculation; we will return to the issue in the next chapter.

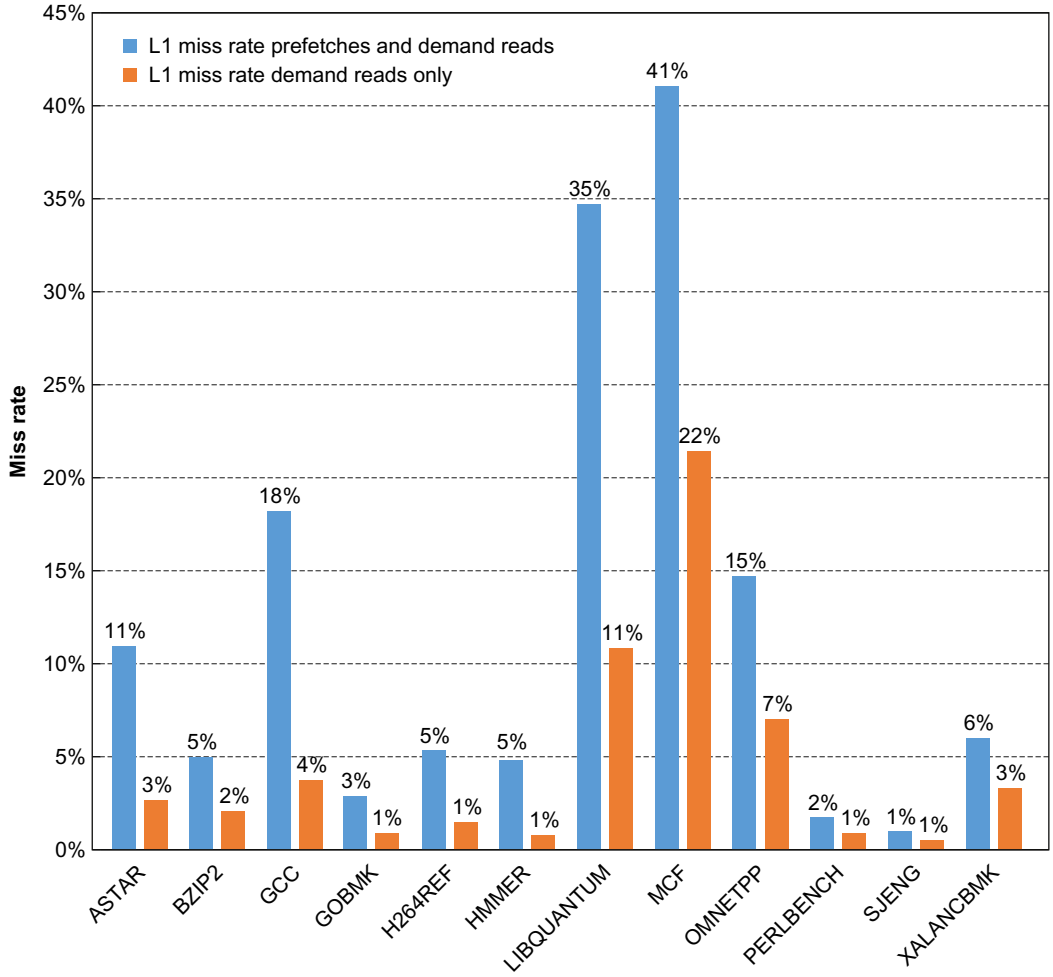


Figure 2.26 The L1 data cache miss rate for the SPECint2006 benchmarks is shown in two ways relative to the demand L1 reads: one including both demand and prefetch accesses and one including only demand accesses. The i7 separates out L1 misses for a block not present in the cache and L1 misses for a block already outstanding that is being prefetched from L2; we treat the latter group as hits because they would hit in a blocking cache. These data, like the rest in this section, were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University, based on earlier studies of the Intel Core Duo and other processors (see Peng et al., 2008).

To address these issues, while keeping the amount of data reasonable, Figure 2.26 shows the L1 data cache misses in two ways:

1. The L1 miss rate relative to demand references given by the L1 miss rate including prefetches and speculative loads/L1 demand read references for those instructions that graduate.

2. The demand miss rate given by L1 demand misses/L1 demand read references, both measurements only for instructions that graduate.

On average, the miss rate including prefetches is 2.8 times as high as the demand-only miss rate. Comparing this data to that from the earlier i7 920, which had the same size L1, we see that the miss rate including prefetches is higher on the newer i7, but the number of demand misses, which are more likely to cause a stall, are usually fewer.

To understand the effectiveness of the aggressive prefetch mechanisms in the i7, let's look at some measurements of prefetching. Figure 2.27 shows both the fraction of L2 requests that are prefetches versus demand requests and the prefetch miss rate. The data are probably astonishing at first glance: there are roughly 1.5 times as many prefetches as there are L2 demand requests, which come directly from L1 misses. Furthermore, the prefetch miss rate is amazingly high, with an average miss rate of 58%. Although the prefetch ratio varies considerably, the prefetch miss rate is always significant. At first glance, you might conclude that the designers made a mistake: they are prefetching too much, and the miss rate is too high. Notice, however, that the benchmarks with the higher prefetch ratios (ASTAR, BZIP2, HMMER, LIBQUANTUM, and OMNETPP) also show the greatest gap between the prefetch miss rate and the demand miss rate, more than a factor of 2 in each case. The aggressive prefetching is trading prefetch misses, which occur earlier, for demand misses, which occur later; and as a result, a pipeline stall is less likely to occur due to the prefetching.

Similarly, consider the high prefetch miss rate. Suppose that the majority of the prefetches are actually useful (this is hard to measure because it involves tracking individual cache blocks), then a prefetch miss indicates a likely L2 cache miss in the future. Uncovering and handling the miss earlier via the prefetch is likely to reduce the stall cycles. Performance analysis of speculative superscalars, like the i7, has shown that cache misses tend to be the primary cause of pipeline stalls, because it is hard to keep the processor going, especially for longer running L2 and L3 misses. The Intel designers could not easily increase the size of the caches without incurring both energy and cycle time impacts; thus the use of aggressive prefetching to try to lower effective cache miss penalties is an interesting alternative approach.

With the combination of the L1 demand misses and prefetches going to L2, roughly 17% of the loads generate an L2 request. Analyzing L2 performance requires including the effects of writes (because L2 is write-allocated), as well as the prefetch hit rate and the demand hit rate. Figure 2.28 shows the miss rates of the L2 caches for demand and prefetch accesses, both versus the number of L1 references (reads and writes). As with L1, prefetches are a significant contributor, generating 75% of the L2 misses. Comparing the L2 demand miss rate with that of earlier i7 implementations (again with the same L2 size) shows that the i7 6700 has a lower L2 demand miss rate by an approximate factor of 2, which may well justify the higher prefetch miss rate.

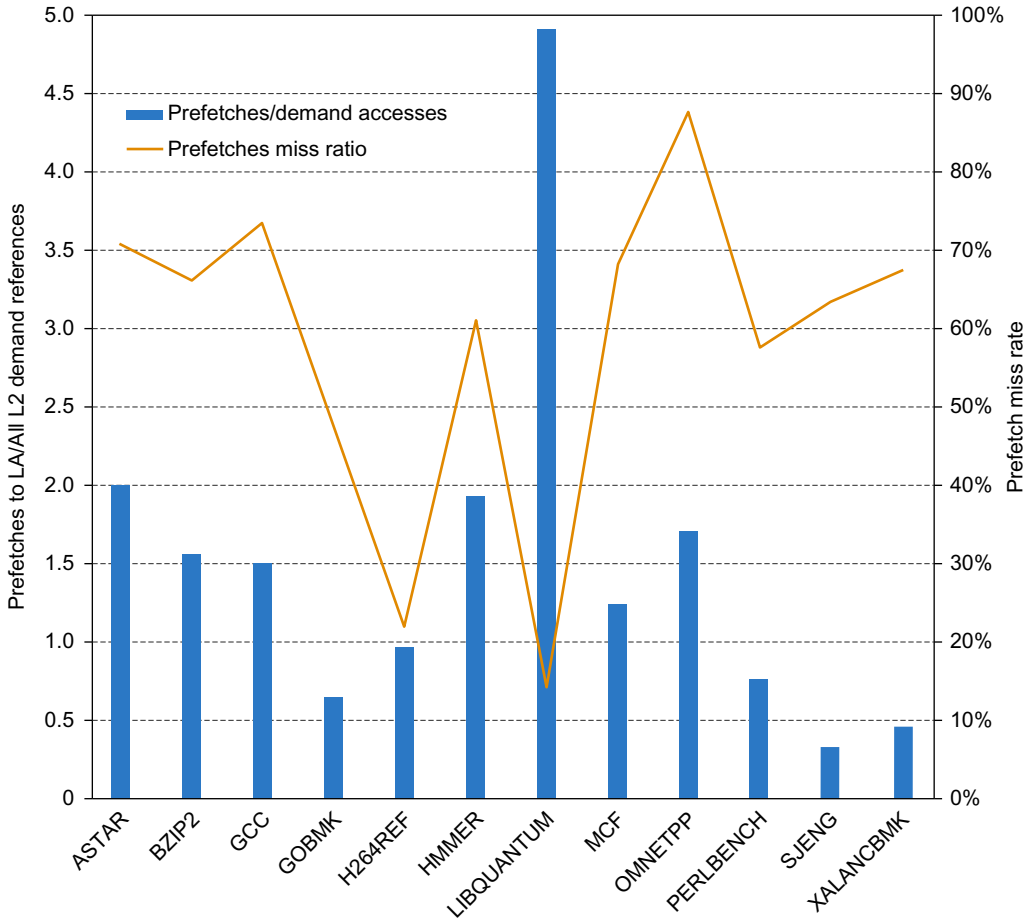


Figure 2.27 The fraction of L2 requests that are prefetches is shown via the columns and the left axis. The right axis and the line shows the prefetch hit rate. These data, like the rest in this section, were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University, based on earlier studies of the Intel Core Duo and other processors (see Peng et al., 2008).

Because the cost for a miss to memory is over 100 cycles and the average data miss rate in L2 combining both prefetch and demand misses is over 7%, L3 is obviously critical. Without L3 and assuming that about one-third of the instructions are loads or stores, L2 cache misses could add over two cycles per instruction to the CPI! Obviously, prefetching past L2 would make no sense without an L3.

In comparison, the average L3 data miss rate of 0.5% is still significant but less than one-third of the L2 demand miss rate and 10 times less than the L1 demand miss rate. Only in two benchmarks (OMNETPP and MCF) is the L3 miss rate

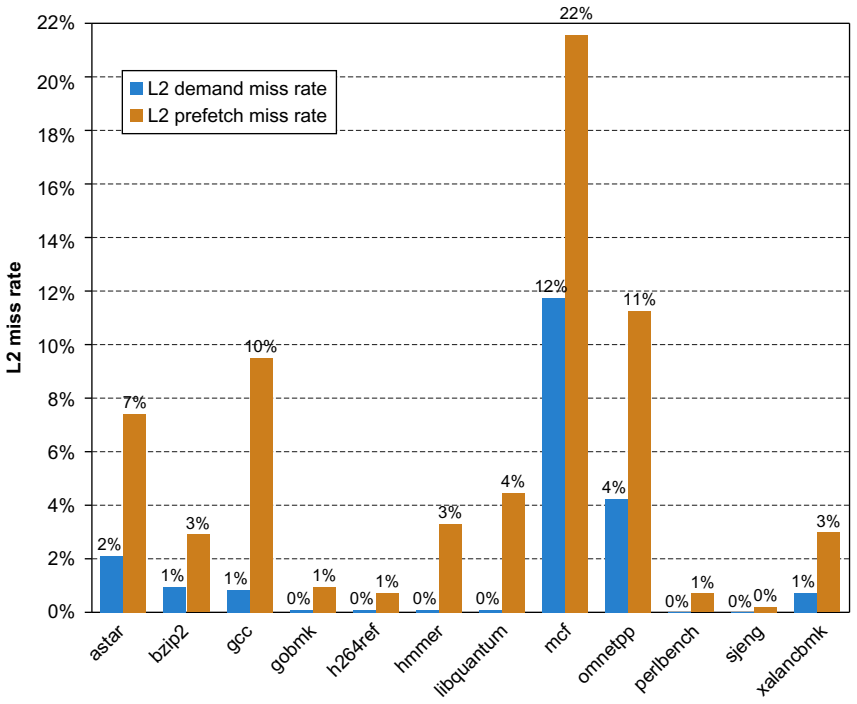


Figure 2.28 The L2 demand miss rate and prefetch miss rate, both shown relative to all the references to L1, which also includes prefetches, speculative loads that do not complete, and program-generated loads and stores (demand references). These data, like the rest in this section, were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University.

above 0.5%; in those two cases, the miss rate of about 2.3% likely dominates all other performance losses. In the next chapter, we will examine the relationship between the i7 CPI and cache misses, as well as other pipeline effects.

2.7 Fallacies and Pitfalls

As the most naturally quantitative of the computer architecture disciplines, memory hierarchy would seem to be less vulnerable to fallacies and pitfalls. Yet we were limited here not by lack of warnings, but by lack of space!

Fallacy *Predicting cache performance of one program from another.*

Figure 2.29 shows the instruction miss rates and data miss rates for three programs from the SPEC2000 benchmark suite as cache size varies. Depending on the

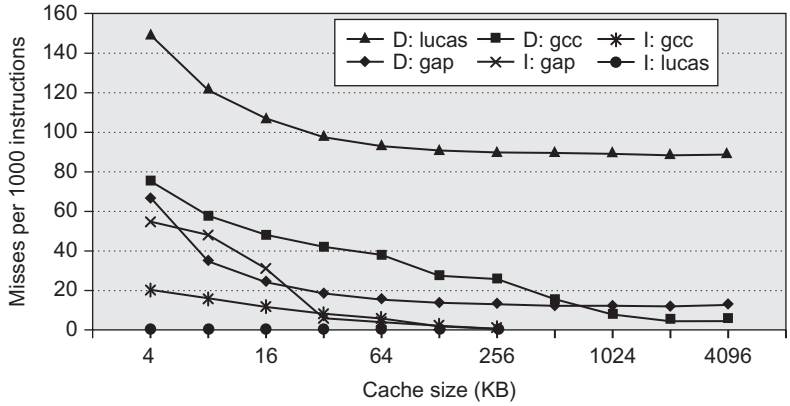


Figure 2.29 Instruction and data misses per 1000 instructions as cache size varies from 4 KiB to 4096 KiB. Instruction misses for gcc are 30,000–40,000 times larger than for lucas, and, conversely, data misses for lucas are 2–60 times larger than for gcc. The programs gap, gcc, and lucas are from the SPEC2000 benchmark suite.

program, the data misses per thousand instructions for a 4096 KiB cache are 9, 2, or 90, and the instruction misses per thousand instructions for a 4 KiB cache are 55, 19, or 0.0004. Commercial programs such as databases will have significant miss rates even in large second-level caches, which is generally not the case for the SPECCPU programs. Clearly, generalizing cache performance from one program to another is unwise. As Figure 2.24 reminds us, there is a great deal of variation, and even predictions about the relative miss rates of integer and floating-point-intensive programs can be wrong, as mcf and sphnix3 remind us!

Pitfall *Simulating enough instructions to get accurate performance measures of the memory hierarchy.*

There are really three pitfalls here. One is trying to predict performance of a large cache using a small trace. Another is that a program's locality behavior is not constant over the run of the entire program. The third is that a program's locality behavior may vary depending on the input.

Figure 2.30 shows the cumulative average instruction misses per thousand instructions for five inputs to a single SPEC2000 program. For these inputs, the average memory rate for the first 1.9 billion instructions is very different from the average miss rate for the rest of the execution.

Pitfall *Not delivering high memory bandwidth in a cache-based system.*

Caches help with average cache memory latency but may not deliver high memory bandwidth to an application that must go to main memory. The architect must design a high bandwidth memory behind the cache for such applications. We will revisit this pitfall in Chapters 4 and 5.

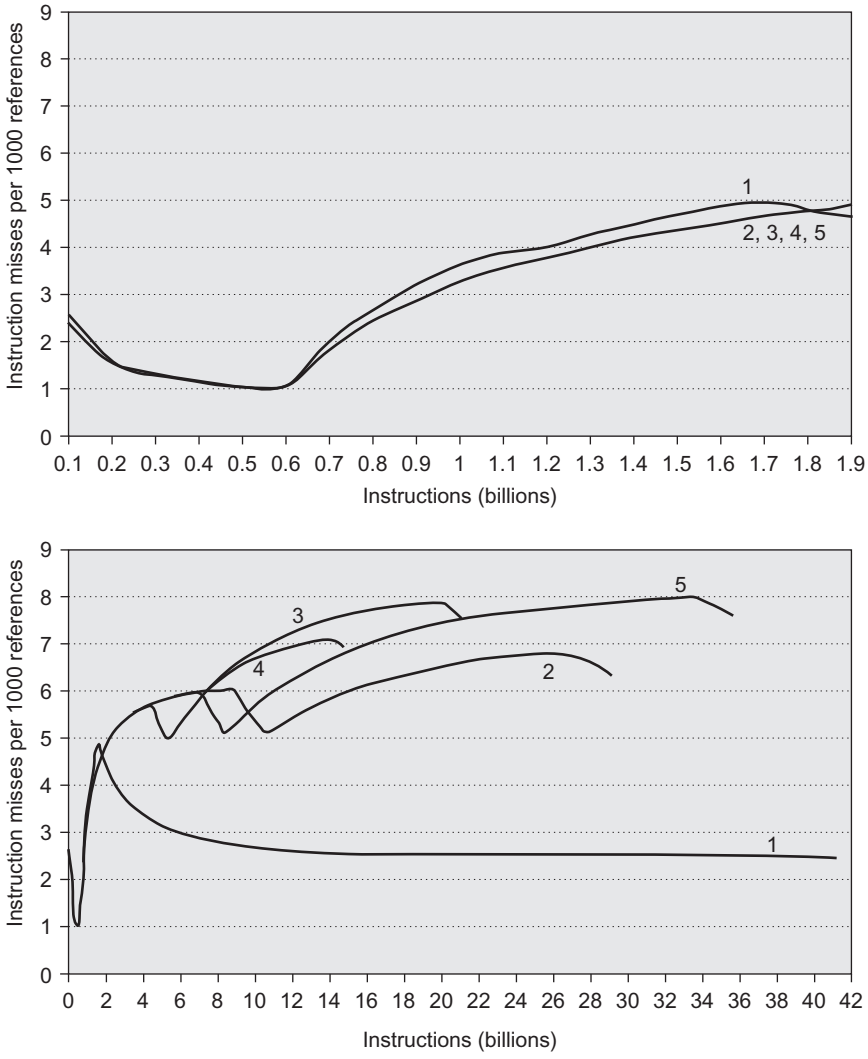


Figure 2.30 Instruction misses per 1000 references for five inputs to the perl benchmark in SPEC2000. There is little variation in misses and little difference between the five inputs for the first 1.9 billion instructions. Running to completion shows how misses vary over the life of the program and how they depend on the input. The top graph shows the running average misses for the first 1.9 billion instructions, which starts at about 2.5 and ends at about 4.7 misses per 1000 references for all five inputs. The bottom graph shows the running average misses to run to completion, which takes 16–41 billion instructions depending on the input. After the first 1.9 billion instructions, the misses per 1000 references vary from 2.4 to 7.9 depending on the input. The simulations were for the Alpha processor using separate L1 caches for instructions and data, each being two-way 64 KiB with LRU, and a unified 1 MiB direct-mapped L2 cache.

Pitfall *Implementing a virtual machine monitor on an instruction set architecture that wasn't designed to be virtualizable.*

Many architects in the 1970s and 1980s weren't careful to make sure that all instructions reading or writing information related to hardware resource information were privileged. This *laissez faire* attitude causes problems for VMMs for all of these architectures, including the 80x86, which we use here as an example.

Figure 2.31 describes the 18 instructions that cause problems for paravirtualization (Robin and Irvine, 2000). The two broad classes are instructions that

- read control registers in user mode that reveal that the guest operating system is running in a virtual machine (such as POPF mentioned earlier) and
- check protection as required by the segmented architecture but assume that the operating system is running at the highest privilege level.

Virtual memory is also challenging. Because the 80x86 TLBs do not support process ID tags, as do most RISC architectures, it is more expensive for the VMM and guest OSes to share the TLB; each address space change typically requires a TLB flush.

Problem category	Problem 80x86 instructions
Access sensitive registers without trapping when running in user mode	Store global descriptor table register (SGDT) Store local descriptor table register (SLDT) Store interrupt descriptor table register (SIDT) Store machine status word (SMSW) Push flags (PUSHF, PUSHFD) Pop flags (POPF, POPFD)
When accessing virtual memory mechanisms in user mode, instructions fail the 80x86 protection checks	Load access rights from segment descriptor (LAR) Load segment limit from segment descriptor (LSL) Verify if segment descriptor is readable (VERR) Verify if segment descriptor is writable (VERW) Pop to segment register (POP CS, POP SS, ...) Push segment register (PUSH CS, PUSH SS, ...) Far call to different privilege level (CALL) Far return to different privilege level (RET) Far jump to different privilege level (JMP) Software interrupt (INT) Store segment selector register (STR) Move to/from segment registers (MOVE)

Figure 2.31 Summary of 18 80x86 instructions that cause problems for virtualization (Robin and Irvine, 2000). The first five instructions of the top group allow a program in user mode to read a control register, such as a descriptor table register without causing a trap. The pop flags instruction modifies a control register with sensitive information but fails silently when in user mode. The protection checking of the segmented architecture of the 80x86 is the downfall of the bottom group because each of these instructions checks the privilege level implicitly as part of instruction execution when reading a control register. The checking assumes that the OS must be at the highest privilege level, which is not the case for guest VMs. Only the MOVE to segment register tries to modify control state, and protection checking foils it as well.

Virtualizing I/O is also a challenge for the 80x86, in part because it supports memory-mapped I/O and has separate I/O instructions, but more importantly because there are a very large number and variety of types of devices and device drivers of PCs for the VMM to handle. Third-party vendors supply their own drivers, and they may not properly virtualize. One solution for conventional VM implementations is to load real device drivers directly into the VMM.

To simplify implementations of VMMs on the 80x86, both AMD and Intel have proposed extensions to the architecture. Intel's VT-x provides a new execution mode for running VMs, a architected definition of the VM state, instructions to swap VMs rapidly, and a large set of parameters to select the circumstances where a VMM must be invoked. Altogether, VT-x adds 11 new instructions for the 80x86. AMD's Secure Virtual Machine (SVM) provides similar functionality.

After turning on the mode that enables VT-x support (via the new `VMXON` instruction), VT-x offers four privilege levels for the guest OS that are lower in priority than the original four (and fix issues like the problem with the `POPF` instruction mentioned earlier). VT-x captures all the states of a virtual machine in the Virtual Machine Control State (VMCS) and then provides atomic instructions to save and restore a VMCS. In addition to critical state, the VMCS includes configuration information to determine when to invoke the VMM and then specifically what caused the VMM to be invoked. To reduce the number of times the VMM must be invoked, this mode adds shadow versions of some sensitive registers and adds masks that check to see whether critical bits of a sensitive register will be changed before trapping. To reduce the cost of virtualizing virtual memory, AMD's SVM adds an additional level of indirection, called *nested page tables*, which makes shadow page tables unnecessary (see Section L.7 of Appendix L).

2.8 Concluding Remarks: Looking Ahead

Over the past thirty years there have been several predictions of the eminent [sic] cessation of the rate of improvement in computer performance. Every such prediction was wrong. They were wrong because they hinged on unstated assumptions that were overturned by subsequent events. So, for example, the failure to foresee the move from discrete components to integrated circuits led to a prediction that the speed of light would limit computer speeds to several orders of magnitude slower than they are now. Our prediction of the memory wall is probably wrong too but it suggests that we have to start thinking "out of the box."

Wm. A. Wulf and Sally A. McKee,
Hitting the Memory Wall: Implications of the Obvious,
Department of Computer Science, University of Virginia (December 1994).
This paper introduced the term *memory wall.*

The possibility of using a memory hierarchy dates back to the earliest days of general-purpose digital computers in the late 1940s and early 1950s. Virtual memory was introduced in research computers in the early 1960s and into IBM mainframes in the 1970s. Caches appeared around the same time. The basic concepts

have been expanded and enhanced over time to help close the access time gap between main memory and processors, but the basic concepts remain.

One trend that is causing a significant change in the design of memory hierarchies is a continued slowdown in both density and access time of DRAMs. In the past 15 years, both these trends have been observed and have been even more obvious over the past 5 years. While some increases in DRAM bandwidth have been achieved, decreases in access time have come much more slowly and almost vanished between DDR4 and DDR3. The end of Dennard scaling as well as a slowdown in Moore's Law both contributed to this situation. The trenched capacitor design used in DRAMs is also limiting its ability to scale. It may well be the case that packaging technologies such as stacked memory will be the dominant source of improvements in DRAM access bandwidth and latency.

Independently of improvements in DRAM, Flash memory has been playing a much larger role. In PMDs, Flash has dominated for 15 years and became the standard for laptops almost 10 years ago. In the past few years, many desktops have shipped with Flash as the primary secondary storage. Flash's potential advantage over DRAMs, specifically the absence of a per-bit transistor to control writing, is also its Achilles heel. Flash must use bulk erase-rewrite cycles that are considerably slower. As a result, although Flash has become the fastest growing form of secondary storage, SDRAMs still dominate for main memory.

Although phase-change materials as a basis for memory have been around for a while, they have never been serious competitors either for magnetic disks or for Flash. The recent announcement by Intel and Micron of the cross-point technology may change this. The technology appears to have several advantages over Flash, including the elimination of the slow erase-to-write cycle and greater longevity in terms. It could be that this technology will finally be the technology that replaces the electromechanical disks that have dominated bulk storage for more than 50 years!

For some years, a variety of predictions have been made about the coming memory wall (see previously cited quote and paper), which would lead to serious limits on processor performance. Fortunately, the extension of caches to multiple levels (from 2 to 4), more sophisticated refill and prefetch schemes, greater compiler and programmer awareness of the importance of locality, and tremendous improvements in DRAM bandwidth (a factor of over 150 times since the mid-1990s) have helped keep the memory wall at bay. In recent years, the combination of access time constraints on the size of L1 (which is limited by the clock cycle) and energy-related limitations on the size of L2 and L3 have raised new challenges. The evolution of the i7 processor class over 6–7 years illustrates this: the caches are the same size in the i7 6700 as they were in the first generation i7 processors! The more aggressive use of prefetching is an attempt to overcome the inability to increase L2 and L3. Off-chip L4 caches are likely to become more important because they are less energy-constrained than on-chip caches.

In addition to schemes relying on multilevel caches, the introduction of out-of-order pipelines with multiple outstanding misses has allowed available instruction-level parallelism to hide the memory latency remaining in a cache-based system. The introduction of multithreading and more thread-level parallelism takes this a step further by providing more parallelism and thus more latency-hiding

opportunities. It is likely that the use of instruction- and thread-level parallelism will be a more important tool in hiding whatever memory delays are encountered in modern multilevel cache systems.

One idea that periodically arises is the use of programmer-controlled scratchpad or other high-speed visible memories, which we will see are used in GPUs. Such ideas have never made the mainstream in general-purpose processors for several reasons: First, they break the memory model by introducing address spaces with different behavior. Second, unlike compiler-based or programmer-based cache optimizations (such as prefetching), memory transformations with scratchpads must completely handle the remapping from main memory address space to the scratchpad address space. This makes such transformations more difficult and limited in applicability. In GPUs (see Chapter 4), where local scratchpad memories are heavily used, the burden for managing them currently falls on the programmer. For domain-specific software systems that can use such memories, the performance gains are very significant. It is likely that HBM technologies will thus be used for caching in large, general-purpose computers and quite possibility as the main working memories in graphics and similar systems. As domain-specific architectures become more important in overcoming the limitations arising from the end of Dennard's Law and the slowdown in Moore's Law (see Chapter 7), scratchpad memories and vector-like register sets are likely to see more use.

The implications of the end of Dennard's Law affect both DRAM and processor technology. Thus, rather than a widening gulf between processors and main memory, we are likely to see a slowdown in both technologies, leading to slower overall growth rates in performance. New innovations in computer architecture and in related software that together increase performance and efficiency will be key to continuing the performance improvements seen over the past 50 years.

2.9 Historical Perspectives and References

In Section M.3 (available online) we examine the history of caches, virtual memory, and virtual machines. IBM plays a prominent role in the history of all three. References for further reading are included.

Case Studies and Exercises by Norman P. Jouppi, Rajeev Balasubramonian, Naveen Muralimanohar, and Sheng Li

Case Study 1: Optimizing Cache Performance via Advanced Techniques

Concepts illustrated by this case study

- Nonblocking Caches
- Compiler Optimizations for Caches
- Software and Hardware Prefetching
- Calculating Impact of Cache Performance on More Complex Processors

The transpose of a matrix interchanges its rows and columns; this concept is illustrated here:

$$\begin{bmatrix} A11 & A12 & A13 & A14 \\ A21 & A22 & A23 & A24 \\ A31 & A32 & A33 & A34 \\ A41 & A42 & A43 & A44 \end{bmatrix} \Rightarrow \begin{bmatrix} A11 & A21 & A31 & A41 \\ A12 & A22 & A32 & A42 \\ A13 & A23 & A33 & A43 \\ A14 & A24 & A34 & A44 \end{bmatrix}$$

Here is a simple C loop to show the transpose:

```
for (i = 0; i < 3; i++) {
 for (j = 0; j < 3; j++) {
 output[j][i] = input[i][j];
 }
}
```

Assume that both the input and output matrices are stored in the row major order (*row major order* means that the row index changes fastest). Assume that you are executing a 256·256 double-precision transpose on a processor with a 16 KB fully associative (don't worry about cache conflicts) least recently used (LRU) replacement L1 data cache with 64-byte blocks. Assume that the L1 cache misses or prefetches require 16 cycles and always hit in the L2 cache, and that the L2 cache can process a request every 2 processor cycles. Assume that each iteration of the preceding inner loop requires 4 cycles if the data are present in the L1 cache. Assume that the cache has a write-allocate fetch-on-write policy for write misses. Unrealistically, assume that writing back dirty cache blocks requires 0 cycles.

2.1 [10/15/15/12/20] <2.3> For the preceding simple implementation, this execution order would be nonideal for the input matrix; however, applying a loop interchange optimization would create a nonideal order for the output matrix. Because loop interchange is not sufficient to improve its performance, it must be blocked instead.

a. [10] <2.3> What should be the minimum size of the cache to take advantage of blocked execution?

b. [15] <2.3> How do the relative number of misses in the blocked and unblocked versions compare in the preceding minimum-sized cache?

c. [15] <2.3> Write code to perform a transpose with a block size parameter B that uses $B \cdot B$ blocks.

d. [12] <2.3> What is the minimum associativity required of the L1 cache for consistent performance independent of both arrays' position in memory?

e. [20] <2.3> Try out blocked and nonblocked 256·256 matrix transpositions on a computer. How closely do the results match your expectations based on what you know about the computer's memory system? Explain any discrepancies if possible.

2.2 [10] <2.3> Assume you are designing a hardware prefetcher for the preceding *unblocked* matrix transposition code. The simplest type of hardware prefetcher only prefetches sequential cache blocks after a miss. More complicated "nonunit stride" hardware prefetchers can analyze a miss reference stream and detect and prefetch nonunit strides. In contrast, software prefetching can determine nonunit strides as easily as it can determine unit strides. Assume prefetches write directly into the cache and that there is no "pollution" (overwriting data that must be used before the data that are prefetched). For best performance given a nonunit stride prefetcher, in the steady state of the inner loop, how many prefetches must be outstanding at a given time?

2.3 [15/20] <2.3> With software prefetching, it is important to be careful to have the prefetches occur in time for use but also to minimize the number of outstanding prefetches to live within the capabilities of the microarchitecture and minimize cache pollution. This is complicated by the fact that different processors have different capabilities and limitations.

a. [15] <2.3> Create a blocked version of the matrix transpose with software prefetching.

b. [20] <2.3> Estimate and compare the performance of the blocked and unblocked transpose codes both with and without software prefetching.

Case Study 2: Putting It All Together: Highly Parallel Memory Systems

Concept illustrated by this case study

■ Cross-Cutting Issues: The Design of Memory Hierarchies

The program in Figure 2.32 can be used to evaluate the behavior of a memory system. The key is having accurate timing and then having the program stride through memory to invoke different levels of the hierarchy. Figure 2.32 shows the code in C. The first part is a procedure that uses a standard utility to get an accurate measure of the user CPU time; this procedure may have to be changed to work on some systems. The second part is a nested loop to read and write memory at different strides and cache sizes. To get accurate cache timing, this code is repeated many times. The third part times the nested loop overhead only so that it can be subtracted from overall measured times to see how long the accesses were. The results are output in `.csv` file format to facilitate importing into spreadsheets. You may need to change `CACHE_MAX` depending on the question you are answering and the size of memory on the system you are measuring. Running the program in single-user mode or at least without other active applications will give more consistent results. The code in Figure 2.32 was derived from a program written by Andrea Dusseau at the University of California-Berkeley and was based on a detailed description found in Saavedra-Barrera (1992). It has been modified to fix a number of issues with more modern machines and to run under Microsoft

```
#include "stdafx.h"
#include <stdio.h>
#include <time.h>
#define ARRAY_MIN (1024) /* 1/4 smallest cache */
#define ARRAY_MAX (4096*4096) /* 1/4 largest cache */
int x[ARRAY_MAX]; /* array going to stride through */

double get_seconds() { /* routine to read time in seconds */
   __time64_t ltime;
   _time64( &ltime );
   return (double) ltime;
}
int label(int i) {/* generate text labels */
   if (i<1e3) printf("%1dB,",i);
   else if (i<1e6) printf("%1dK,",i/1024);
   else if (i<1e9) printf("%1dM,",i/1048576);
   else printf("%1dG,",i/1073741824);
   return 0;
}
int _tmain(int argc, _TCHAR* argv[]) {
int register nextstep, i, index, stride;
int csize;
double steps, tsteps;
double loadtime, lastsec, sec0, sec1, sec; /* timing variables */

/* Initialize output */
printf(" ,");
for (stride=1; stride <= ARRAY_MAX/2; stride=stride*2)
   label(stride*sizeof(int));
printf("\n");

/* Main loop for each configuration */
for (csize=ARRAY_MIN; csize <= ARRAY_MAX; csize=csize*2) {
   label(csize*sizeof(int)); /* print cache size this loop */
   for (stride=1; stride <= csize/2; stride=stride*2) {

      /* Lay out path of memory references in array */
      for (index=0; index < csize; index=index+stride)
         x[index] = index + stride; /* pointer to next */
      x[index-stride] = 0; /* loop back to beginning */

      /* Wait for timer to roll over */
      lastsec = get_seconds();
       sec0 = get_seconds(); while (sec0 == lastsec);

      /* Walk through path in array for twenty seconds */
      /* This gives 5% accuracy with second resolution */
      steps = 0.0; /* number of steps taken */
      nextstep = 0; /* start at beginning of path */
      sec0 = get_seconds(); /* start timer */
         { /* repeat until collect 20 seconds */
            (i=stride;i!=0;i=i-1) { /* keep samples same */
               nextstep = 0;
               do nextstep = x[nextstep]; /* dependency */
               while (nextstep != 0);
         }
         steps = steps + 1.0; /* count loop iterations */
         sec1 = get_seconds(); /* end timer */
      } while ((sec1 - sec0) < 20.0); /* collect 20 seconds */
      sec = sec1 - sec0;

      /* Repeat empty loop to loop subtract overhead */
      tsteps = 0.0; /* used to match no. while iterations */
      sec0 = get_seconds(); /* start timer */
         { /* repeat until same no. iterations as above */
            (i=stride;i!=0;i=i-1) { /* keep samples same */
               index = 0;
               do index = index + stride;
               while (index < csize);
         }
         tsteps = tsteps + 1.0;
         sec1 = get_seconds(); /* - overhead */
      } while (tsteps<steps); /* until = no. iterations */
      sec = sec - (sec1 - sec0);
      loadtime = (sec*1e9)/(steps*csize);
      /* write out results in .csv format for Excel */
      printf("%4.1f,", (loadtime<0.1) ? 0.1 : loadtime);
     }; /* end of inner for loop */
     printf("\n");
   }; /* end of outer for loop */
   return 0;
}
```

Figure 2.32 C program for evaluating memory system.

Visual C++. It can be downloaded from http://www.hpl.hp.com/research/cacti/aca_ch2_cs2.c.

The preceding program assumes that program addresses track physical addresses, which is true on the few machines that use virtually addressed caches, such as the Alpha 21264. In general, virtual addresses tend to follow physical addresses shortly after rebooting, so you may need to reboot the machine in order to get smooth lines in your results. To answer the following questions, assume that the sizes of all components of the memory hierarchy are powers of 2. Assume that the size of the page is much larger than the size of a block in a second-level cache (if there is one) and that the size of a second-level cache block is greater than or equal to the size of a block in a first-level cache. An example of the output of the program is plotted in Figure 2.33; the key lists the size of the array that is exercised.

2.4 [12/12/12/10/12] <2.6> Using the sample program results in Figure 2.33:

a. [12] <2.6> What are the overall size and block size of the second-level cache?

b. [12] <2.6> What is the miss penalty of the second-level cache?

c. [12] <2.6> What is the associativity of the second-level cache?

d. [10] <2.6> What is the size of the main memory?

e. [12] <2.6> What is the paging time if the page size is 4 KB?

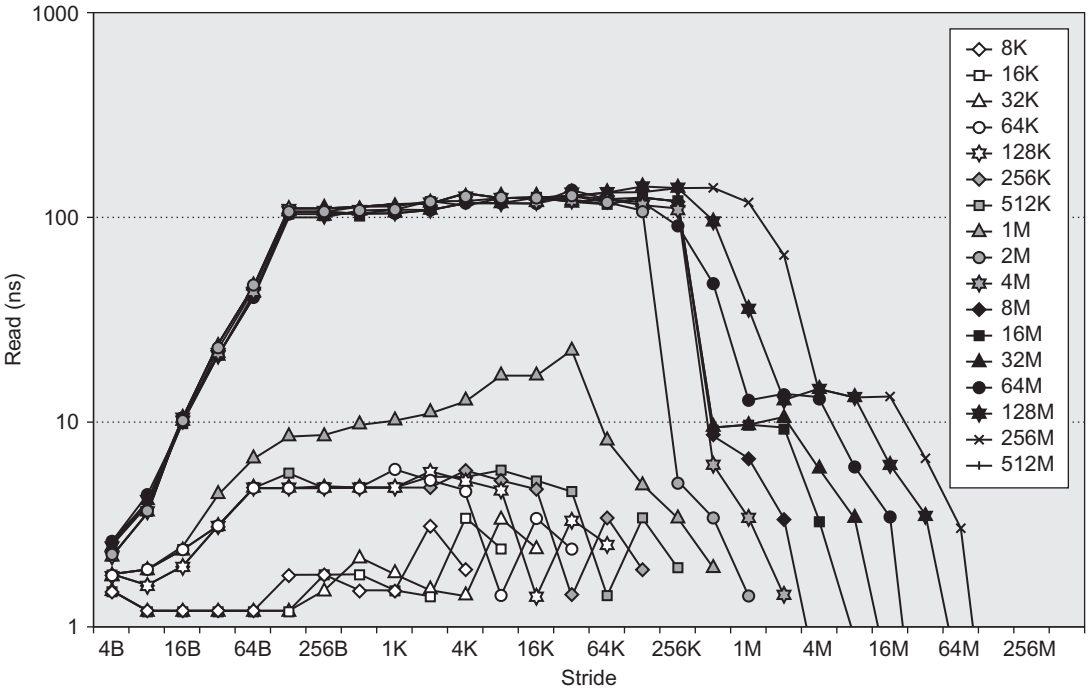


Figure 2.33 Sample results from program in Figure 2.32.

2.5 [12/15/15/20] <2.6> If necessary, modify the code in Figure 2.32 to measure the following system characteristics. Plot the experimental results with elapsed time on the *y*-axis and the memory stride on the *x*-axis. Use logarithmic scales for both axes, and draw a line for each cache size.

a. [12] <2.6> What is the system page size?

b. [15] <2.6> How many entries are there in the TLB?

c. [15] <2.6> What is the miss penalty for the TLB?

d. [20] <2.6> What is the associativity of the TLB?

2.6 [20/20] <2.6> In multiprocessor memory systems, lower levels of the memory hierarchy may not be able to be saturated by a single processor but should be able to be saturated by multiple processors working together. Modify the code in Figure 2.32, and run multiple copies at the same time. Can you determine:

a. [20] <2.6> How many actual processors are in your computer system and how many system processors are just additional multithreaded contexts?

b. [20] <2.6> How many memory controllers does your system have?

2.7 [20] <2.6> Can you think of a way to test some of the characteristics of an instruction cache using a program? *Hint*: The compiler may generate a large number of nonobvious instructions from a piece of code. Try to use simple arithmetic instructions of known length in your instruction set architecture (ISA).

Case Study 3: Studying the Impact of Various Memory System Organizations

Concepts illustrated by this case study

- DDR3 memory systems
- Impact of ranks, banks, row buffers on performance and power
- DRAM timing parameters

A processor chip typically supports a few DDR3 or DDR4 memory channels. We will focus on a single memory channel in this case study and explore how its performance and power are impacted by varying several parameters. Recall that the channel is populated with one or more DIMMs. Each DIMM supports one or more ranks—a rank is a collection of DRAM chips that work in unison to service a single command issued by the memory controller. For example, a rank may be composed of 16 DRAM chips, where each chip deals with a 4-bit input or output on every channel clock edge. Each such chip is referred to as a $\times 4$ (by four) chip. In other examples, a rank may be composed of 8×8 chips or 4×16 chips—note that in each case, a rank can handle data that are being placed on a 64-bit memory channel. A rank is itself partitioned into 8 (DDR3) or 16 (DDR4) banks. Each bank has a row buffer that essentially remembers the last row read out of a bank. Here's an example of a typical sequence of memory commands when performing a read from a bank:

(i) The memory controller issues a Precharge command to get the bank ready to access a new row. The precharge is completed after time tRP.

(ii) The memory controller then issues an Activate command to read the appropriate row out of the bank. The activation is completed after time tRCD and the row is deemed to be part of the row buffer.

(iii) The memory controller can then issue a column-read or CAS command that places a specific subset of the row buffer on the memory channel. After time CL, the first 64 bits of the data burst are placed on the memory channel. A burst typically includes eight 64-bit transfers on the memory channel, performed on the rising and falling edges of 4 memory clock cycles (referred to as transfer time).

(iv) If the memory controller wants to then access data in a different row of the bank, referred to as a row buffer miss, it repeats steps (i)–(iii). For now, we will assume that after CL has elapsed, the Precharge in step (i) can be issued; in some cases, an additional delay must be added, but we will ignore that delay here. If the memory controller wants to access another block of data in the same row, referred to as a row buffer hit, it simply issues another CAS command. Two back-to-back CAS commands have to be separated by at least 4 cycles so that the first data transfer is complete before the second data transfer can begin.

Note that a memory controller can issue commands to different banks in successive cycles so that it can perform many memory reads/writes in parallel and it is not sitting idle waiting for tRP, tRCD, and CL to elapse in a single bank. For the subsequent questions, assume that tRP = tRCD = CL = 13 ns, and that the memory channel frequency is 1 GHz, that is, a transfer time of 4 ns.

2.8 [10] <2.2> What is the read latency experienced by a memory controller on a row buffer miss?

2.9 [10] <2.2> What is the latency experienced by a memory controller on a row buffer hit?

2.10 [10] <2.2> If the memory channel supports only one bank and the memory access pattern is dominated by row buffer misses, what is the utilization of the memory channel?

2.11 [15] <2.2> Assuming a 100% row buffer miss rate, what is the minimum number of banks that the memory channel should support in order to achieve a 100% memory channel utilization?

2.12 [10] <2.2> Assuming a 50% row buffer miss rate, what is the minimum number of banks that the memory channel should support in order to achieve a 100% memory channel utilization?

2.13 [15] <2.2> Assume that we are executing an application with four threads and the threads exhibit zero spatial locality, that is, a 100% row buffer miss rate. Every 200 ns, each of the four threads simultaneously inserts a read operation into the

memory controller queue. What is the average memory latency experienced if the memory channel supports only one bank? What if the memory channel supported four banks?

2.14 [10] <2.2> From these questions, what have you learned about the benefits and downsides of growing the number of banks?

2.15 [20] <2.2> Now let's turn our attention to memory power. Download a copy of the Micron power calculator from this link: https://www.micron.com/~/media/documents/products/power-calculator/ddr3_power_calc.xlsm. This spreadsheet is preconfigured to estimate the power dissipation in a single 2 Gb ×8 DDR3 SDRAM memory chip manufactured by Micron. Click on the "Summary" tab to see the power breakdown in a single DRAM chip under default usage conditions (reads occupy the channel for 45% of all cycles, writes occupy the channel for 25% of all cycles, and the row buffer hit rate is 50%). This chip consumes 535 mW, and the breakdown shows that about half of that power is expended in Activate operations, about 38% in CAS operations, and 12% in background power. Next, click on the "System Config" tab. Modify the read/write traffic and the row buffer hit rate and observe how that changes the power profile. For example, what is the decrease in power when channel utilization is 35% (25% reads and 10% writes), or when row buffer hit rate is increased to 80%?

2.16 [20] <2.2> In the default configuration, a rank consists of eight ×8 2 Gb DRAM chips. A rank can also comprise16 × 4 chips or 4 × 16 chips. You can also vary the capacity of each DRAM chip—1 Gb, 2 Gb, and 4 Gb. These selections can be made in the "DDR3 Config" tab of the Micron power calculator. Tabulate the total power consumed for each rank organization. What is the most power-efficient approach to constructing a rank of a given capacity?

Exercises

2.17 [12/12/15] <2.3> The following questions investigate the impact of small and simple caches using CACTI and assume a 65 nm (0.065 m) technology. (CACTI is available in an online form at http://quid.hpl.hp.com:9081/cacti/.)

a. [12] <2.3> Compare the access times of 64 KB caches with 64-byte blocks and a single bank. What are the relative access times of two-way and four-way set associative caches compared to a direct mapped organization?

b. [12] <2.3> Compare the access times of four-way set associative caches with 64-byte blocks and a single bank. What are the relative access times of 32 and 64 KB caches compared to a 16 KB cache?

c. [15] <2.3> For a 64 KB cache, find the cache associativity between 1 and 8 with the lowest average memory access time given that misses per instruction for a certain workload suite is 0.00664 for direct-mapped, 0.00366 for two-way set associative, 0.000987 for four-way set associative, and 0.000266 for eight-way set associative cache. Overall, there are 0.3 data references per instruction. Assume cache misses take 10 ns in all models. To calculate the hit time in

cycles, assume the cycle time output using CACTI, which corresponds to the maximum frequency a cache can operate without any bubbles in the pipeline.

2.18 [12/15/15/10] <2.3> You are investigating the possible benefits of a way-predicting L1 cache. Assume that a 64 KB four-way set associative single-banked L1 data cache is the cycle time limiter in a system. For an alternative cache organization, you are considering a way-predicted cache modeled as a 64 KB direct-mapped cache with 80% prediction accuracy. Unless stated otherwise, assume that a mispredicted way access that hits in the cache takes one more cycle. Assume the miss rates and the miss penalties in question 2.8 part (c).

a. [12] <2.3> What is the average memory access time of the current cache (in cycles) versus the way-predicted cache?

b. [15] <2.3> If all other components could operate with the faster way-predicted cache cycle time (including the main memory), what would be the impact on performance from using the way-predicted cache?

c. [15] <2.3> Way-predicted caches have usually been used only for instruction caches that feed an instruction queue or buffer. Imagine that you want to try out way prediction on a data cache. Assume that you have 80% prediction accuracy and that subsequent operations (e.g., data cache access of other instructions, dependent operations) are issued assuming a correct way prediction. Thus a way misprediction necessitates a pipe flush and replay trap, which requires 15 cycles. Is the change in average memory access time per load instruction with data cache way prediction positive or negative, and how much is it?

d. [10] <2.3> As an alternative to way prediction, many large associative L2 caches serialize tag and data access so that only the required dataset array needs to be activated. This saves power but increases the access time. Use CACTI's detailed web interface for a 0.065 m process 1 MB four-way set associative cache with 64-byte blocks, 144 bits read out, 1 bank, only 1 read/write port, 30 bit tags, and ITRS-HP technology with global wires. What is the ratio of the access times for serializing tag and data access compared to parallel access?

2.19 [10/12] <2.3> You have been asked to investigate the relative performance of a banked versus pipelined L1 data cache for a new microprocessor. Assume a 64 KB two-way set associative cache with 64-byte blocks. The pipelined cache would consist of three pipe stages, similar in capacity to the Alpha 21264 data cache. A banked implementation would consist of two 32 KB two-way set associative banks. Use CACTI and assume a 65 nm (0.065 m) technology to answer the following questions. The cycle time output in the web version shows at what frequency a cache can operate without any bubbles in the pipeline.

a. [10] <2.3> What is the cycle time of the cache in comparison to its access time, and how many pipe stages will the cache take up (to two decimal places)?

b. [12] <2.3> Compare the area and total dynamic read energy per access of the pipelined design versus the banked design. State which takes up less area and which requires more power, and explain why that might be.

2.20 [12/15] <2.3> Consider the usage of critical word first and early restart on L2 cache misses. Assume a 1 MB L2 cache with 64-byte blocks and a refill path that is 16 bytes wide. Assume that the L2 can be written with 16 bytes every 4 processor cycles, the time to receive the first 16 byte block from the memory controller is 120 cycles, each additional 16 byte block from main memory requires 16 cycles, and data can be bypassed directly into the read port of the L2 cache. Ignore any cycles to transfer the miss request to the L2 cache and the requested data to the L1 cache.

a. [12] <2.3> How many cycles would it take to service an L2 cache miss with and without critical word first and early restart?

b. [15] <2.3> Do you think critical word first and early restart would be more important for L1 caches or L2 caches, and what factors would contribute to their relative importance?

2.21 [12/12] <2.3> You are designing a write buffer between a write-through L1 cache and a write-back L2 cache. The L2 cache write data bus is 16 B wide and can perform a write to an independent cache address every four processor cycles.

a. [12] <2.3> How many bytes wide should each write buffer entry be?

b. [15] <2.3> What speedup could be expected in the steady state by using a merging write buffer instead of a nonmerging buffer when zeroing memory by the execution of 64-bit stores if all other instructions could be issued in parallel with the stores and the blocks are present in the L2 cache?

c. [15] <2.3> What would the effect of possible L1 misses be on the number of required write buffer entries for systems with blocking and nonblocking caches?

2.22 [20] <2.1, 2.2, 2.3> A cache acts as a filter. For example, for every 1000 instructions of a program, an average of 20 memory accesses may exhibit low enough locality that they cannot be serviced by a 2 MB cache. The 2 MB cache is said to have an MPKI (misses per thousand instructions) of 20, and this will be largely true regardless of the smaller caches that precede the 2 MB cache. Assume the following cache/latency/MPKI values: 32 KB/1/100, 128 KB/2/80, 512 KB/4/50, 2 MB/8/40, 8 MB/16/10. Assume that accessing the off-chip memory system requires 200 cycles on average. For the following cache configurations, calculate the average time spent accessing the cache hierarchy. What do you observe about the downsides of a cache hierarchy that is too shallow or too deep?

a. 32 KB L1; 8 MB L2; off-chip memory

b. 32 KB L1; 512 KB L2; 8 MB L3; off-chip memory

c. 32 KB L1; 128 KB L2; 2 MB L3; 8 MB L4; off-chip memory

2.23 [15] <2.1, 2.2, 2.3> Consider a 16 MB 16-way L3 cache that is shared by two programs A and B. There is a mechanism in the cache that monitors cache miss rates for each program and allocates 1–15 ways to each program such that the overall number of cache misses is reduced. Assume that program A has an MPKI of 100 when it is assigned 1 MB of the cache. Each additional 1 MB assigned to program

A reduces the MPKI by 1. Program B has an MPKI of 50 when it is assigned 1 MB of cache; each additional 1 MB assigned to program B reduces its MPKI by 2. What is the best allocation of ways to programs A and B?

2.24 [20] <2.1, 2.6> You are designing a PMD and optimizing it for low energy. The core, including an 8 KB L1 data cache, consumes 1 W whenever it is not in hibernation. If the core has a perfect L1 cache hit rate, it achieves an average CPI of 1 for a given task, that is, 1000 cycles to execute 1000 instructions. Each additional cycle accessing the L2 and beyond adds a stall cycle for the core. Based on the following specifications, what is the size of L2 cache that achieves the lowest energy for the PMD (core, L1, L2, memory) for that given task?

a. The core frequency is 1 GHz, and the L1 has an MPKI of 100.

b. A 256 KB L2 has a latency of 10 cycles, an MPKI of 20, a background power of 0.2 W, and each L2 access consumes 0.5 nJ.

c. A 1 MB L2 has a latency of 20 cycles, an MPKI of 10, a background power of 0.8 W, and each L2 access consumes 0.7 nJ.

d. The memory system has an average latency of 100 cycles, a background power of 0.5 W, and each memory access consumes 35 nJ.

2.25 [15] <2.1, 2.6> You are designing a PMD that is optimized for low power. Qualitatively explain the impact on cache hierarchy (L2 and memory) power and overall application energy if you design an L2 cache with:

a. Small block size

b. Small cache size

c. High associativity

2.30 [10/10] <2.1, 2.2, 2.3> The ways of a set can be viewed as a priority list, ordered from high priority to low priority. Every time the set is touched, the list can be reorganized to change block priorities. With this view, cache management policies can be decomposed into three sub-policies: Insertion, Promotion, and Victim Selection. Insertion defines where newly fetched blocks are placed in the priority list. Promotion defines how a block's position in the list is changed every time it is touched (a cache hit). Victim Selection defines which entry of the list is evicted to make room for a new block when there is a cache miss.

a. Can you frame the LRU cache policy in terms of the Insertion, Promotion, and Victim Selection sub-policies?

b. Can you define other Insertion and Promotion policies that may be competitive and worth exploring further?

2.31 [15] <2.1, 2.3> In a processor that is running multiple programs, the last-level cache is typically shared by all the programs. This leads to interference, where one program's behavior and cache footprint can impact the cache available to other programs. First, this is a problem from a quality-of-service (QoS) perspective, where the interference leads to a program receiving fewer resources and lower

performance than promised, say by the operator of a cloud service. Second, this is a problem in terms of privacy. Based on the interference it sees, a program can infer the memory access patterns of other programs. This is referred to as a timing channel, a form of information leakage from one program to others that can be exploited to compromise data privacy or to reverse-engineer a competitor's algorithm. What policies can you add to your last-level cache so that the behavior of one program is immune to the behavior of other programs sharing the cache?

2.32 [15] <2.3> A large multimegabyte L3 cache can take tens of cycles to access because of the long wires that have to be traversed. For example, it may take 20 cycles to access a 16 MB L3 cache. Instead of organizing the 16 MB cache such that every access takes 20 cycles, we can organize the cache so that it is an array of smaller cache banks. Some of these banks may be closer to the processor core, while others may be further. This leads to nonuniform cache access (NUCA), where 2 MB of the cache may be accessible in 8 cycles, the next 2 MB in 10 cycles, and so on until the last 2 MB is accessed in 22 cycles. What new policies can you introduce to maximize performance in a NUCA cache?

2.33 [10/10/10] <2.2> Consider a desktop system with a processor connected to a 2 GB DRAM with *error-correcting code (ECC)*. Assume that there is only one memory channel of width 72 bits (64 bits for data and 8 bits for ECC).

a. [10] <2.2> How many DRAM chips are on the DIMM if 1 Gb DRAM chips are used, and how many data I/Os must each DRAM have if only one DRAM connects to each DIMM data pin?

b. [10] <2.2> What burst length is required to support 32 B L2 cache blocks?

c. [10] <2.2> Calculate the peak bandwidth for DDR2-667 and DDR2-533 DIMMs for reads from an active page excluding the ECC overhead.

2.34 [10/10] <2.2> A sample DDR2 SDRAM timing diagram is shown in Figure 2.34. tRCD is the time required to activate a row in a bank, and column address strobe (CAS) latency (CL) is the number of cycles required to read out a column in a row. Assume that the RAM is on a standard DDR2 DIMM with ECC, having 72 data lines. Also assume burst lengths of 8 that read out 8 bits, or a total of 64 B from the DIMM. Assume `tRCD = CAS (or CL) clock_frequency`, and `clock_frequency = transfers_per_second/2`. The on-chip latency

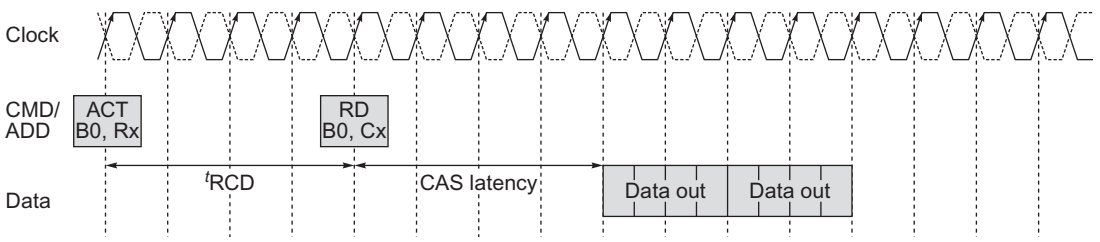


Figure 2.34 DDR2 SDRAM timing diagram.

on a cache miss through levels 1 and 2 and back, not including the DRAM access, is 20 ns.

a. [10] <2.2> How much time is required from presentation of the activate command until the last requested bit of data from the DRAM transitions from valid to invalid for the DDR2-667 1 Gb CL = 5 DIMM? Assume that for every request, we automatically prefetch another adjacent cache line in the same page.

b. [10] <2.2> What is the relative latency when using the DDR2-667 DIMM of a read requiring a bank activate versus one to an already open page, including the time required to process the miss inside the processor?

2.35 [15] <2.2> Assume that a DDR2-667 2 GB DIMM with CL = 5 is available for 130 and a DDR2-533 2 GB DIMM with CL = 4 is available for 100. Assume that two DIMMs are used in a system, and the rest of the system costs 800. Consider the performance of the system using the DDR2-667 and DDR2-533 DIMMs on a workload with 3.33 L2 misses per 1K instructions, and assume that 80% of all DRAM reads require an activate. What is the cost-performance of the entire system when using the different DIMMs, assuming only one L2 miss is outstanding at a time and an in-order core with a CPI of 1.5 not including L2 cache miss memory access time?

2.36 [12] <2.2> You are provisioning a server with eight-core 3 GHz CMP that can execute a workload with an overall CPI of 2.0 (assuming that L2 cache miss refills are not delayed). The L2 cache line size is 32 bytes. Assuming the system uses DDR2-667 DIMMs, how many independent memory channels should be provided so the system is not limited by memory bandwidth if the bandwidth required is sometimes twice the average? The workloads incur, on average, 6.67 L2 misses per 1 K instructions.

2.37 [15] <2.2> Consider a processor that has four memory channels. Should consecutive memory blocks be placed in the same bank, or should they be placed in different banks on different channels?

2.38 [12/12] <2.2> A large amount (more than a third) of DRAM power can be due to page activation (see http://download.micron.com/pdf/technotes/ddr2/TN4704.pdf and http://www.micron.com/systemcalc). Assume you are building a system with 2 GB of memory using either 8-bank 2 Gb ×8 DDR2 DRAMs or 8-bank 1 Gb ×8 DRAMs, both with the same speed grade. Both use a page size of 1 KB, and the last-level cache line size is 64 bytes. Assume that DRAMs that are not active are in precharged standby and dissipate negligible power. Assume that the time to transition from standby to active is not significant.

a. [12] <2.2> Which type of DRAM would be expected to provide the higher system performance? Explain why.

b. [12] <2.2> How does a 2 GB DIMM made of 1 Gb ×8 DDR2 DRAMs compare with a DIMM with similar capacity made of 1 Gb ×4 DDR2 DRAMs in terms of power?

2.39 [20/15/12] <2.2> To access data from a typical DRAM, we first have to activate the appropriate row. Assume that this brings an entire page of size 8 KB to the row buffer. Then we select a particular column from the row buffer. If subsequent accesses to DRAM are to the same page, then we can skip the activation step; otherwise, we have to close the current page and precharge the bitlines for the next activation. Another popular DRAM policy is to proactively close a page and precharge bitlines as soon as an access is over. Assume that every read or write to DRAM is of size 64 bytes and DDR bus latency (data from Figure 2.33) for sending 512 bits is Tddr.

a. [20] <2.2> Assuming DDR2-667, if it takes five cycles to precharge, five cycles to activate, and four cycles to read a column, for what value of the row buffer hit rate (r) will you choose one policy over another to get the best access time? Assume that every access to DRAM is separated by enough time to finish a random new access.

b. [15] <2.2> If 10% of the total accesses to DRAM happen back to back or contiguously without any time gap, how will your decision change?

c. [12] <2.2> Calculate the difference in average DRAM energy per access between the two policies using the previously calculated row buffer hit rate. Assume that precharging requires 2 nJ and activation requires 4 nJ and that 100 pJ/bit are required to read or write from the row buffer.

2.40 [15] <2.2> Whenever a computer is idle, we can either put it in standby (where DRAM is still active) or we can let it hibernate. Assume that, to hibernate, we have to copy just the contents of DRAM to a nonvolatile medium such as Flash. If reading or writing a cache line of size 64 bytes to Flash requires 2.56 J and DRAM requires 0.5 nJ, and if idle power consumption for DRAM is 1.6 W (for 8 GB), how long should a system be idle to benefit from hibernating? Assume a main memory of size 8 GB.

2.41 [10/10/10/10/10] <2.4> Virtual machines (VMs) have the potential for adding many beneficial capabilities to computer systems, such as improved total cost of ownership (TCO) or availability. Could VMs be used to provide the following capabilities? If so, how could they facilitate this?

a. [10] <2.4> Test applications in production environments using development machines?

b. [10] <2.4> Quick redeployment of applications in case of disaster or failure?

c. [10] <2.4> Higher performance in I/O-intensive applications?

d. [10] <2.4> Fault isolation between different applications, resulting in higher availability for services?

e. [10] <2.4> Performing software maintenance on systems while applications are running without significant interruption?

2.42 [10/10/12/12] <2.4> Virtual machines can lose performance from a number of events, such as the execution of privileged instructions, TLB misses, traps, and I/O.

Benchmark	Native	Pure	Para
Null call	0.04	0.96	0.50
Null I/O	0.27	6.32	2.91
Stat	1.10	10.69	4.14
Open/close	1.99	20.43	7.71
Install signal handler	0.33	7.34	2.89
Handle signal	1.69	19.26	2.36
Fork	56.00	513.00	164.00
Exec	316.00	2084.00	578.00
Fork + exec sh	1451.00	7790.00	2360.00

Figure 2.35 Early performance of various system calls under native execution, pure virtualization, and paravirtualization.

These events are usually handled in system code. Thus one way of estimating the slowdown when running under a VM is the percentage of application execution time in system versus user mode. For example, an application spending 10% of its execution in system mode might slow down by 60% when running on a VM. Figure 2.35 lists the early performance of various system calls under native execution, pure virtualization, and paravirtualization for LMbench using Xen on an Itanium system with times measured in microseconds (courtesy of Matthew Chapman of the University of New South Wales).

a. [10] <2.4> What types of programs would be expected to have smaller slowdowns when running under VMs?

b. [10] <2.4> If slowdowns were linear as a function of system time, given the preceding slowdown, how much slower would a program spending 20% of its execution in system time be expected to run?

c. [12] <2.4> What is the median slowdown of the system calls in the table above under pure virtualization and paravirtualization?

d. [12] <2.4> Which functions in the table above have the largest slowdowns? What do you think the cause of this could be?

2.43 [12] <2.4> Popek and Goldberg's definition of a virtual machine said that it would be indistinguishable from a real machine except for its performance. In this question, we will use that definition to find out if we have access to native execution on a processor or are running on a virtual machine. The Intel VT-x technology effectively provides a second set of privilege levels for the use of the virtual machine. What would a virtual machine running on top of another virtual machine have to do, assuming VT-x technology?

2.44 [20/25] <2.4> With the adoption of virtualization support on the x86 architecture, virtual machines are actively evolving and becoming mainstream. Compare and contrast the Intel VT-x and AMD's AMD-V virtualization technologies.

(Information on AMD-V can be found at http://sites.amd.com/us/business/it-solutions/virtualization/Pages/resources.aspx.)

a. [20] <2.4> Which one could provide higher performance for memory-intensive applications with large memory footprints?

b. [25] <2.4> Information on AMD's IOMMU support for virtualized I/O can be found at http://developer.amd.com/documentation/articles/pages/892006101.aspx. What do Virtualization Technology and an input/output memory management unit (IOMMU) do to improve virtualized I/O performance?

2.45 [30] <2.2, 2.3> Since instruction-level parallelism can also be effectively exploited on in-order superscalar processors and *very long instruction word (VLIW)* processors with speculation, one important reason for building an out-of-order (OOO) superscalar processor is the ability to tolerate unpredictable memory latency caused by cache misses. Thus you can think about hardware supporting OOO issue as being part of the memory system. Look at the floorplan of the Alpha 21264 in Figure 2.36 to find the relative area of the integer and floating-point issue queues and mappers versus the caches. The queues schedule instructions for issue,

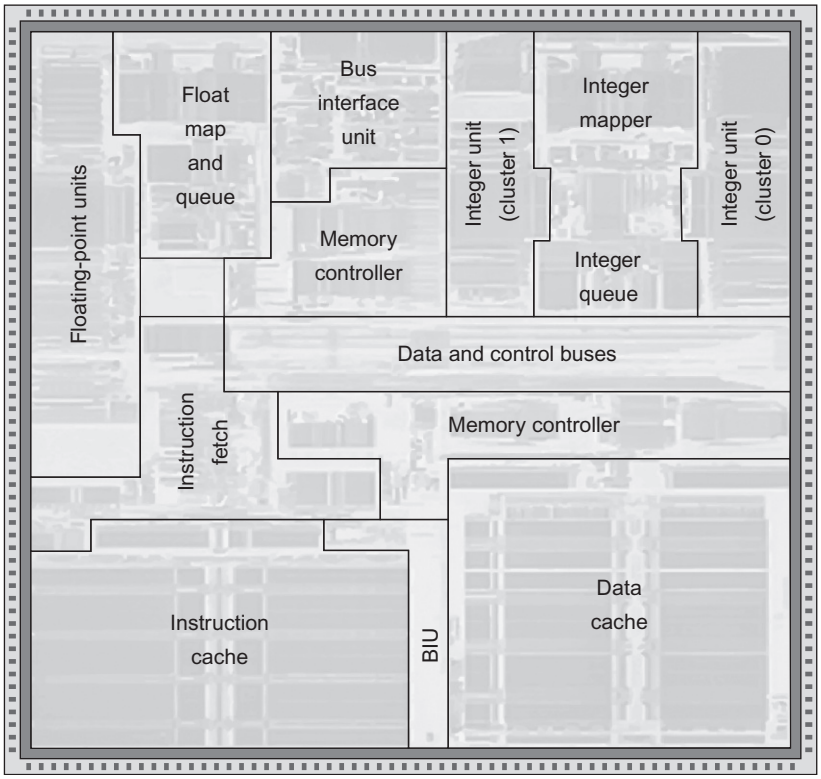


Figure 2.36 Floorplan of the Alpha 21264 [Kessler 1999].

and the mappers rename register specifiers. Therefore these are necessary additions to support OOO issue. The 21264 only has L1 data and instruction caches on chip, and they are both 64 KB two-way set associative. Use an OOO superscalar simulator such as SimpleScalar (http://www.cs.wisc.edu/~mscalar/simplescalar.html) on memory-intensive benchmarks to find out how much performance is lost if the area of the issue queues and mappers is used for additional L1 data cache area in an in-order superscalar processor, instead of OOO issue in a model of the 21264. Make sure the other aspects of the machine are as similar as possible to make the comparison fair. Ignore any increase in access or cycle time from larger caches and effects of the larger data cache on the floorplan of the chip. (Note that this comparison will not be totally fair, as the code will not have been scheduled for the in-order processor by the compiler.)

2.46 [15] <2.2, 2.7> As discussed in Section 2.7, the Intel i7 processor has an aggressive prefetcher. What are potential disadvantages in designing a prefetcher that is extremely aggressive?

2.47 [20/20/20] <2.6> The Intel performance analyzer VTune can be used to make many measurements of cache behavior. A free evaluation version of VTune on both Windows and Linux can be downloaded from http://software.intel.com/en-us/articles/intel-vtune-amplifier-xe/. The program (`aca_ch2_cs2.c`) used in Case Study 2 has been modified so that it can work with VTune out of the box on Microsoft Visual C++. The program can be downloaded from http://www.hpl.hp.com/research/cacti/aca_ch2_cs2_vtune.c. Special VTune functions have been inserted to exclude initialization and loop overhead during the performance analysis process. Detailed VTune setup directions are given in the README section in the program. The program keeps looping for 20 seconds for every configuration. In the following experiment, you can find the effects of data size on cache and overall processor performance. Run the program in VTune on an Intel processor with the input dataset sizes of 8 KB, 128 KB, 4 MB, and 32 MB, and keep a stride of 64 bytes (stride one cache line on Intel i7 processors). Collect statistics on overall performance and L1 data cache, L2, and L3 cache performance.

a. [20] <2.6> List the number of misses per 1K instruction of L1 data cache, L2, and L3 for each dataset size and your processor model and speed. Based on the results, what can you say about the L1 data cache, L2, and L3 cache sizes on your processor? Explain your observations.

b. [20] <2.6> List the *instructions per clock* (IPC) for each dataset size and your processor model and speed. Based on the results, what can you say about the L1, L2, and L3 miss penalties on your processor? Explain your observations.

c. [20] <2.6> Run the program in VTune with input dataset size of 8 KB and 128 KB on an Intel OOO processor. List the number of L1 data cache and L2 cache misses per 1K instructions and the CPI for both configurations. What can you say about the effectiveness of memory latency hiding techniques in high-performance OOO processors? *Hint*: You need to find the L1 data cache miss latency for your processor. For recent Intel i7 processors, it is approximately 11 cycles.

This page intentionally left blank

3.1	Instruction-Level Parallelism: Concepts and Challenges	168
3.2	Basic Compiler Techniques for Exposing ILP	176
3.3	Reducing Branch Costs With Advanced Branch Prediction	182
3.4	Overcoming Data Hazards With Dynamic Scheduling	191
3.5	Dynamic Scheduling: Examples and the Algorithm	201
3.6	Hardware-Based Speculation	208
3.7	Exploiting ILP Using Multiple Issue and Static Scheduling	218
3.8	Exploiting ILP Using Dynamic Scheduling, Multiple Issue, and Speculation	222
3.9	Advanced Techniques for Instruction Delivery and Speculation	228
3.10	Cross-Cutting Issues	240
3.11	Multithreading: Exploiting Thread-Level Parallelism to Improve Uniprocessor Throughput	242
3.12	Putting It All Together: The Intel Core i7 6700 and ARM Cortex-A53	247
3.13	Fallacies and Pitfalls	258
3.14	Concluding Remarks: What's Ahead?	264
3.15	Historical Perspective and References	266
	Case Studies and Exercises by Jason D. Bakos and Robert P. Colwell	266

3

Instruction-Level Parallelism and Its Exploitation

"Who's first?"
"America."
"Who's second?"
"Sir, there is no second."

Dialog between two observers of the sailing race in 1851, later named "The America's Cup," which was the inspiration for John Cocke's naming of an IBM research processor as "America," the first superscalar processor, and a precursor to the PowerPC.

Thus, the IA-64 gambles that, in the future, power will not be the critical limitation, and massive resources…will not penalize clock speed, path length, or CPI factors. My view is clearly skeptical…

Marty Hopkins (2000), IBM Fellow and Early RISC pioneer commenting in 2000 on the new Intel Itanium, a joint development of Intel and HP. The Itanium used a static ILP approach (see Appendix H) and was a massive investment for Intel. It never accounted for more than 0.5% of Intel's microprocessor sales.

Computer Architecture. https://doi.org/10.1016/B978-0-12-811905-1.00003-1
© 2019 Elsevier Inc. All rights reserved.

3.1 Instruction-Level Parallelism: Concepts and Challenges

All processors since about 1985 have used pipelining to overlap the execution of instructions and improve performance. This potential overlap among instructions is called *instruction-level parallelism* (ILP), because the instructions can be evaluated in parallel. In this chapter and Appendix H, we look at a wide range of techniques for extending the basic pipelining concepts by increasing the amount of parallelism exploited among instructions.

This chapter is at a considerably more advanced level than the material on basic pipelining in Appendix C. If you are not thoroughly familiar with the ideas in Appendix C, you should review that appendix before venturing into this chapter.

We start this chapter by looking at the limitation imposed by data and control hazards and then turn to the topic of increasing the ability of the compiler and the processor to exploit parallelism. These sections introduce a large number of concepts, which we build on throughout this chapter and the next. While some of the more basic material in this chapter could be understood without all of the ideas in the first two sections, this basic material is important to later sections of this chapter.

There are two largely separable approaches to exploiting ILP: (1) an approach that relies on hardware to help discover and exploit the parallelism dynamically, and (2) an approach that relies on software technology to find parallelism statically at compile time. Processors using the dynamic, hardware-based approach, including all recent Intel and many ARM processors, dominate in the desktop and server markets. In the personal mobile device market, the same approaches are used in processors found in tablets and high-end cell phones. In the IOT space, where power and cost constraints dominate performance goals, designers exploit lower levels of instruction-level parallelism. Aggressive compiler-based approaches have been attempted numerous times beginning in the 1980s and most recently in the Intel Itanium series, introduced in 1999. Despite enormous efforts, such approaches have been successful only in domain-specific environments or in well-structured scientific applications with significant data-level parallelism.

In the past few years, many of the techniques developed for one approach have been exploited within a design relying primarily on the other. This chapter introduces the basic concepts and both approaches. A discussion of the limitations on ILP approaches is included in this chapter, and it was such limitations that directly led to the movement toward multicore. Understanding the limitations remains important in balancing the use of ILP and thread-level parallelism.

In this section, we discuss features of both programs and processors that limit the amount of parallelism that can be exploited among instructions, as well as the critical mapping between program structure and hardware structure, which is key to understanding whether a program property will actually limit performance and under what circumstances.

The value of the CPI (cycles per instruction) for a pipelined processor is the sum of the base CPI and all contributions from stalls:

$$\text{Pipeline CPI} = \text{Ideal pipeline CPI} + \text{Structural stalls} + \text{Data hazard stalls} + \text{Control stalls}$$

Technique	Reduces	Section
Forwarding and bypassing	Potential data hazard stalls	C.2
Simple branch scheduling and prediction	Control hazard stalls	C.2
Basic compiler pipeline scheduling	Data hazard stalls	C.2, 3.2
Basic dynamic scheduling (scoreboarding)	Data hazard stalls from true dependences	C.7
Loop unrolling	Control hazard stalls	3.2
Advanced branch prediction	Control stalls	3.3
Dynamic scheduling with renaming	Stalls from data hazards, output dependences, and antidependences	3.4
Hardware speculation	Data hazard and control hazard stalls	3.6
Dynamic memory disambiguation	Data hazard stalls with memory	3.6
Issuing multiple instructions per cycle	Ideal CPI	3.7, 3.8
Compiler dependence analysis, software pipelining, trace scheduling	Ideal CPI, data hazard stalls	H.2, H.3
Hardware support for compiler speculation	Ideal CPI, data hazard stalls, branch hazard stalls	H.4, H.5

Figure 3.1 The major techniques examined in Appendix C, Chapter 3, and Appendix H are shown together with the component of the CPI equation that the technique affects.

The *ideal pipeline CPI* is a measure of the maximum performance attainable by the implementation. By reducing each of the terms of the right-hand side, we decrease the overall pipeline CPI or, alternatively, increase the IPC (instructions per clock). The preceding equation allows us to characterize various techniques by what component of the overall CPI a technique reduces. Figure 3.1 shows the techniques we examine in this chapter and in Appendix H, as well as the topics covered in the introductory material in Appendix C. In this chapter, we will see that the techniques we introduce to decrease the ideal pipeline CPI can increase the importance of dealing with hazards.

What Is Instruction-Level Parallelism?

All the techniques in this chapter exploit parallelism among instructions. The amount of parallelism available within a *basic block*—a straight-line code sequence with no branches in except to the entry and no branches out except at the exit—is quite small. For typical RISC programs, the average dynamic branch frequency is often between 15% and 25%, meaning that between three and six instructions execute between a pair of branches. Because these instructions are likely to depend upon one another, the amount of overlap we can exploit within a basic block is likely to be less than the average basic block size. To obtain substantial performance enhancements, we must exploit ILP across multiple basic blocks.

The simplest and most common way to increase the ILP is to exploit parallelism among iterations of a loop. This type of parallelism is often called *loop-level*

parallelism. Here is a simple example of a loop that adds two 1000-element arrays and is completely parallel:

```
for (i=0; i<=999; i=i+1)
        x[i] = x[i] + y[i];
```

Every iteration of the loop can overlap with any other iteration, although within each loop iteration, there is little or no opportunity for overlap.

We will examine a number of techniques for converting such loop-level parallelism into instruction-level parallelism. Basically, such techniques work by unrolling the loop either statically by the compiler (as in the next section) or dynamically by the hardware (as in Sections 3.5 and 3.6).

An important alternative method for exploiting loop-level parallelism is the use of SIMD in both vector processors and graphics processing units (GPUs), both of which are covered in Chapter 4. A SIMD instruction exploits data-level parallelism by operating on a small to moderate number of data items in parallel (typically two to eight). A vector instruction exploits data-level parallelism by operating on many data items in parallel using both parallel execution units and a deep pipeline. For example, the preceding code sequence, which in simple form requires seven instructions per iteration (two loads, an add, a store, two address updates, and a branch) for a total of 7000 instructions, might execute in one-quarter as many instructions in some SIMD architecture where four data items are processed per instruction. On some vector processors, this sequence might take only four instructions: two instructions to load the vectors `x` and `y` from memory, one instruction to add the two vectors, and an instruction to store back the result vector. Of course, these instructions would be pipelined and have relatively long latencies, but these latencies may be overlapped.

Data Dependences and Hazards

Determining how one instruction depends on another is critical to determining how much parallelism exists in a program and how that parallelism can be exploited. In particular, to exploit instruction-level parallelism, we must determine which instructions can be executed in parallel. If two instructions are *parallel*, they can execute simultaneously in a pipeline of arbitrary depth without causing any stalls, assuming the pipeline has sufficient resources (and thus no structural hazards exist). If two instructions are dependent, they are not parallel and must be executed in order, although they may often be partially overlapped. The key in both cases is to determine whether an instruction is dependent on another instruction.

Data Dependences

There are three different types of dependences: *data dependences* (also called true data dependences), *name dependences*, and *control dependences*. An instruction *j* is *data-dependent* on instruction *i* if either of the following holds:

- Instruction *i* produces a result that may be used by instruction *j*.
- Instruction *j* is data-dependent on instruction *k*, and instruction *k* is data-dependent on instruction *i*.

The second condition simply states that one instruction is dependent on another if there exists a chain of dependences of the first type between the two instructions. This dependence chain can be as long as the entire program. Note that a dependence within a single instruction (such as `add x1,x1,x1`) is not considered a dependence.

For example, consider the following RISC-V code sequence that increments a vector of values in memory (starting at `0(x1)` ending with the last element at `0(x2)`) by a scalar in register `f2`.

```
Loop:  fld     f0,0(x1)    //f0=array element
       fadd.d  f4,f0,f2    //add scalar in f2
       fsd     f4,0(x1)    //store result
       addi    x1,x1,−8    //decrement pointer 8 bytes
       bne     x1,x2,Loop  //branch x1≠x2
```

The data dependences in this code sequence involve both floating-point data:

```
Loop: fld     f0,0(x1)   //f0=array element
      fadd.d  f4,f0,f2   //add scalar in f2
      fsd     f4,0(x1)   //store result
```

and integer data:

```
addi     x1,x1,-8  //decrement pointer
                   //8 bytes (per DW)
bne      x1,x2,Loop//branch x1ax2
```

In both of the preceding dependent sequences, as shown by the arrows, each instruction depends on the previous one. The arrows here and in following examples show the order that must be preserved for correct execution. The arrow points from an instruction that must precede the instruction that the arrowhead points to.

If two instructions are data-dependent, they must execute in order and cannot execute simultaneously or be completely overlapped. The dependence implies that there would be a chain of one or more data hazards between the two instructions. (See Appendix C for a brief description of data hazards, which we will define precisely in a few pages.) Executing the instructions simultaneously will cause a processor with pipeline interlocks (and a pipeline depth longer than the distance between the instructions in cycles) to detect a hazard and stall, thereby reducing or eliminating the overlap. In a processor without interlocks that relies on compiler scheduling, the compiler cannot schedule dependent instructions in such a way that

they completely overlap because the program will not execute correctly. The presence of a data dependence in an instruction sequence reflects a data dependence in the source code from which the instruction sequence was generated. The effect of the original data dependence must be preserved.

Dependences are a property of *programs.* Whether a given dependence results in an actual hazard being detected and whether that hazard actually causes a stall are properties of the *pipeline organization.* This difference is critical to understanding how instruction-level parallelism can be exploited.

A data dependence conveys three things: (1) the possibility of a hazard, (2) the order in which results must be calculated, and (3) an upper bound on how much parallelism can possibly be exploited. Such limits are explored in a pitfall on page 262 and in Appendix H in more detail.

Because a data dependence can limit the amount of instruction-level parallelism we can exploit, a major focus of this chapter is overcoming these limitations. A dependence can be overcome in two different ways: (1) maintaining the dependence but avoiding a hazard, and (2) eliminating a dependence by transforming the code. Scheduling the code is the primary method used to avoid a hazard without altering a dependence, and such scheduling can be done both by the compiler and by the hardware.

A data value may flow between instructions either through registers or through memory locations. When the data flow occurs through a register, detecting the dependence is straightforward because the register names are fixed in the instructions, although it gets more complicated when branches intervene and correctness concerns force a compiler or hardware to be conservative.

Dependences that flow through memory locations are more difficult to detect because two addresses may refer to the same location but look different: For example, `100(x4)` and `20(x6)` may be identical memory addresses. In addition, the effective address of a load or store may change from one execution of the instruction to another (so that `20(x4)` and `20(x4)` may be different), further complicating the detection of a dependence.

In this chapter, we examine hardware for detecting data dependences that involve memory locations, but we will see that these techniques also have limitations. The compiler techniques for detecting such dependences are critical in uncovering loop-level parallelism.

Name Dependences

The second type of dependence is a *name dependence.* A name dependence occurs when two instructions use the same register or memory location, called a *name*, but there is no flow of data between the instructions associated with that name. There are two types of name dependences between an instruction *i* that *precedes* instruction *j* in program order:

1. An *antidependence* between instruction *i* and instruction *j* occurs when instruction *j* writes a register or memory location that instruction *i* reads. The original

ordering must be preserved to ensure that *i* reads the correct value. In the example on page 171, there is an antidependence between `fsd` and `addi` on register `x1`.

2. An *output dependence* occurs when instruction *i* and instruction *j* write the same register or memory location. The ordering between the instructions must be preserved to ensure that the value finally written corresponds to instruction *j*.

Both antidependences and output dependences are name dependences, as opposed to true data dependences, because there is no value being transmitted between the instructions. Because a name dependence is not a true dependence, instructions involved in a name dependence can execute simultaneously or be reordered, if the name (register number or memory location) used in the instructions is changed so the instructions do not conflict.

This renaming can be more easily done for register operands, where it is called *register renaming*. Register renaming can be done either statically by a compiler or dynamically by the hardware. Before describing dependences arising from branches, let's examine the relationship between dependences and pipeline data hazards.

Data Hazards

A hazard exists whenever there is a name or data dependence between instructions, and they are close enough that the overlap during execution would change the order of access to the operand involved in the dependence. Because of the dependence, we must preserve what is called *program order*—that is, the order that the instructions would execute in if executed sequentially one at a time as determined by the original source program. The goal of both our software and hardware techniques is to exploit parallelism by preserving program order *only where it affects the outcome of the program.* Detecting and avoiding hazards ensures that necessary program order is preserved.

Data hazards, which are informally described in Appendix C, may be classified as one of three types, depending on the order of read and write accesses in the instructions. By convention, the hazards are named by the ordering in the program that must be preserved by the pipeline. Consider two instructions *i* and *j,* with *i* preceding *j* in program order. The possible data hazards are

- RAW *(read after write)*—*j* tries to read a source before *i* writes it, so *j* incorrectly gets the *old* value. This hazard is the most common type and corresponds to a true data dependence. Program order must be preserved to ensure that *j* receives the value from *i*.
- WAW *(write after write)*—*j* tries to write an operand before it is written by *i*. The writes end up being performed in the wrong order, leaving the value written by *i* rather than the value written by *j* in the destination. This hazard corresponds to an output dependence. WAW hazards are present only in pipelines that write in more than one pipe stage or allow an instruction to proceed even when a previous instruction is stalled.

- WAR *(write after read)—j* tries to write a destination before it is read by *i,* so *i* incorrectly gets the *new* value. This hazard arises from an antidependence (or name dependence). WAR hazards cannot occur in most static issue pipelines—even deeper pipelines or floating-point pipelines—because all reads are early (in ID in the pipeline in Appendix C) and all writes are late (in WB in the pipeline in Appendix C). A WAR hazard occurs either when there are some instructions that write results early in the instruction pipeline *and* other instructions that read a source late in the pipeline, or when instructions are reordered, as we will see in this chapter.

Note that the RAR *(read after read)* case is not a hazard.

Control Dependences

The last type of dependence is a *control dependence.* A control dependence determines the ordering of an instruction, *i,* with respect to a branch instruction so that instruction *i* is executed in correct program order and only when it should be. Every instruction, except for those in the first basic block of the program, is control-dependent on some set of branches, and in general, these control dependences must be preserved to preserve program order. One of the simplest examples of a control dependence is the dependence of the statements in the "then" part of an if statement on the branch. For example, in the code segment

```
if p1 {
      S1;
};
if p2 {
      S2;
}
```

`S1` is control-dependent on `p1`, and `S2` is control-dependent on `p2` but not on `p1`.

In general, two constraints are imposed by control dependences:

1. An instruction that is control-dependent on a branch cannot be moved *before* the branch so that its execution *is no longer controlled* by the branch. For example, we cannot take an instruction from the then portion of an if statement and move it before the if statement.
2. An instruction that is not control-dependent on a branch cannot be moved *after* the branch so that its execution *is controlled* by the branch. For example, we cannot take a statement before the if statement and move it into the then portion.

When processors preserve strict program order, they ensure that control dependences are also preserved. We may be willing to execute instructions that should not have been executed, however, thereby violating the control dependences, *if* we

can do so without affecting the correctness of the program. Thus control dependence is not the critical property that must be preserved. Instead, the two properties critical to program correctness—and normally preserved by maintaining both data and control dependences—are the *exception behavior* and the *data flow*.

Preserving the exception behavior means that any changes in the ordering of instruction execution must not change how exceptions are raised in the program. Often this is relaxed to mean that the reordering of instruction execution must not cause any new exceptions in the program. A simple example shows how maintaining the control and data dependences can prevent such situations. Consider this code sequence:

```
        add  x2,x3,x4
        beq  x2,x0,L1
        ld   x1,0(x2)
L1:
```

In this case, it is easy to see that if we do not maintain the data dependence involving `x2`, we can change the result of the program. Less obvious is the fact that if we ignore the control dependence and move the load instruction before the branch, the load instruction may cause a memory protection exception. Notice that *no data dependence* prevents us from interchanging the `beqz` and the `ld`; it is only the control dependence. To allow us to reorder these instructions (and still preserve the data dependence), we want to just ignore the exception when the branch is taken. In Section 3.6, we will look at a hardware technique, *speculation*, which allows us to overcome this exception problem. Appendix H looks at software techniques for supporting speculation.

The second property preserved by maintenance of data dependences and control dependences is the data flow. The *data flow* is the actual flow of data values among instructions that produce results and those that consume them. Branches make the data flow dynamic because they allow the source of data for a given instruction to come from many points. Put another way, it is insufficient to just maintain data dependences because an instruction may be data-dependent on more than one predecessor. Program order is what determines which predecessor will actually deliver a data value to an instruction. Program order is ensured by maintaining the control dependences.

For example, consider the following code fragment:

```
        add  x1,x2,x3
        beq  x4,x0,L
        sub  x1,x5,x6
L:      ...
        or   x7,x1,x8
```

In this example, the value of `x1` used by the `or` instruction depends on whether the branch is taken or not. Data dependence alone is not sufficient to preserve correctness. The `or` instruction is data-dependent on both the `add` and `sub` instructions, but preserving that order alone is insufficient for correct execution.

Instead, when the instructions execute, the data flow must be preserved: If the branch is not taken, then the value of x1 computed by the sub should be used by the or, and if the branch is taken, the value of x1 computed by the add should be used by the or. By preserving the control dependence of the or on the branch, we prevent an illegal change to the data flow. For similar reasons, the sub instruction cannot be moved above the branch. Speculation, which helps with the exception problem, will also allow us to lessen the impact of the control dependence while still maintaining the data flow, as we will see in Section 3.6.

Sometimes we can determine that violating the control dependence cannot affect either the exception behavior or the data flow. Consider the following code sequence:

```
        add   x1,x2,x3
        beq   x12,x0,skip
        sub   x4,x5,x6
        add   x5,x4,x9
skip:   or    x7,x8,x9
```

Suppose we knew that the register destination of the sub instruction (x4) was unused after the instruction labeled skip. (The property of whether a value will be used by an upcoming instruction is called *liveness*.) If x4 were unused, then changing the value of x4 just before the branch would not affect the data flow because x4 would be *dead* (rather than live) in the code region after skip. Thus, if x4 were dead and the existing sub instruction could not generate an exception (other than those from which the processor resumes the same process), we could move the sub instruction before the branch because the data flow could not be affected by this change.

If the branch is taken, the sub instruction will execute and will be useless, but it will not affect the program results. This type of code scheduling is also a form of speculation, often called software speculation, because the compiler is betting on the branch outcome; in this case, the bet is that the branch is usually not taken. More ambitious compiler speculation mechanisms are discussed in Appendix H. Normally, it will be clear when we say speculation or speculative whether the mechanism is a hardware or software mechanism; when it is not clear, it is best to say "hardware speculation" or "software speculation."

Control dependence is preserved by implementing control hazard detection that causes control stalls. Control stalls can be eliminated or reduced by a variety of hardware and software techniques, which we examine in Section 3.3.

3.2 Basic Compiler Techniques for Exposing ILP

This section examines the use of simple compiler technology to enhance a processor's ability to exploit ILP. These techniques are crucial for processors that use static issue or static scheduling. Armed with this compiler technology, we will shortly examine the design and performance of processors using static issuing. Appendix H will investigate more sophisticated compiler and associated hardware schemes designed to enable a processor to exploit more instruction-level parallelism.

Basic Pipeline Scheduling and Loop Unrolling

To keep a pipeline full, parallelism among instructions must be exploited by finding sequences of unrelated instructions that can be overlapped in the pipeline. To avoid a pipeline stall, the execution of a dependent instruction must be separated from the source instruction by a distance in clock cycles equal to the pipeline latency of that source instruction. A compiler's ability to perform this scheduling depends both on the amount of ILP available in the program and on the latencies of the functional units in the pipeline. Figure 3.2 shows the FP unit latencies we assume in this chapter, unless different latencies are explicitly stated. We assume the standard five-stage integer pipeline so that branches have a delay of one clock cycle. We assume that the functional units are fully pipelined or replicated (as many times as the pipeline depth) so that an operation of any type can be issued on every clock cycle and there are no structural hazards.

In this section, we look at how the compiler can increase the amount of available ILP by transforming loops. This example serves both to illustrate an important technique as well as to motivate the more powerful program transformations described in Appendix H. We will rely on the following code segment, which adds a scalar to a vector:

```
for (i=999; i>=0; i=i–1)
        x[i] = x[i] + s;
```

We can see that this loop is parallel by noticing that the body of each iteration is independent. We formalize this notion in Appendix H and describe how we can test whether loop iterations are independent at compile time. First, let's look at the performance of this loop, which shows how we can use the parallelism to improve its performance for a RISC-V pipeline with the preceding latencies.

The first step is to translate the preceding segment to RISC-V assembly language. In the following code segment, `x1` is initially the address of the element in the array with the highest address, and `f2` contains the scalar value *s*. Register `x2` is precomputed so that `Regs[x2]+8` is the address of the last element to operate on.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop:  fld     f0,0(x1)      //f0=array element
       fadd.d  f4,f0,f2      //add scalar in f2
       fsd     f4,0(x1)      //store result
       addi    x1,x1,−8      //decrement pointer
                             //8 bytes (per DW)
       bne     x1,x2,Loop    //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in Figure 3.2.

Example Show how the loop would look on RISC-V, both scheduled and unscheduled, including any stalls or idle clock cycles. Schedule for delays from floating-point operations.

Answer Without any scheduling, the loop will execute as follows, taking nine cycles:

			Clock cycle issued
`Loop:`	`fld`	`f0,0(x1)`	1
	stall		2
	`fadd.d`	`f4,f0,f2`	3
	stall		4
	stall		5
	`fsd`	`f4,0(x1)`	6
	`addi`	`x1,x1,−8`	7
	`bne`	`x1,x2,Loop`	8

We can schedule the loop to obtain only two stalls and reduce the time to seven cycles:

```
Loop:  fld     f0,0(x1)
       addi    x1,x1,−8
       fadd.d  f4,f0,f2
       stall
       stall
       fsd     f4,8(x1)
       bne     x1,x2,Loop
```

The stalls after `fadd.d` are for use by the `fsd`, and repositioning the `addi` prevents the stall after the `fld`.

In the previous example, we complete one loop iteration and store back one array element every seven clock cycles, but the actual work of operating on the array element takes just three (the load, add, and store) of those seven clock cycles.

The remaining four clock cycles consist of loop overhead—the `addi` and `bne`—and two stalls. To eliminate these four clock cycles, we need to get more operations relative to the number of overhead instructions.

A simple scheme for increasing the number of instructions relative to the branch and overhead instructions is *loop unrolling*. Unrolling simply replicates the loop body multiple times, adjusting the loop termination code.

Loop unrolling can also be used to improve scheduling. Because it eliminates the branch, it allows instructions from different iterations to be scheduled together. In this case, we can eliminate the data use stalls by creating additional independent instructions within the loop body. If we simply replicated the instructions when we unrolled the loop, the resulting use of the same registers could prevent us from effectively scheduling the loop. Thus we will want to use different registers for each iteration, increasing the required number of registers.

Example Show our loop unrolled so that there are four copies of the loop body, assuming `x1 – x2` (that is, the size of the array) is initially a multiple of 32, which means that the number of loop iterations is a multiple of 4. Eliminate any obviously redundant computations and do not reuse any of the registers.

Answer Here is the result after merging the `addi` instructions and dropping the unnecessary `bne` operations that are duplicated during unrolling. Note that `x2` must now be set so that `Regs[x2]+32` is the starting address of the last four elements.

```
Loop:  fld     f0,0(x1)
       fadd.d  f4,f0,f2
       fsd     f4,0(x1)      //drop addi & bne
       fld     f6,–8(x1)
       fadd.d  f8,f6,f2
       fsd     f8,–8(x1)     //drop addi & bne
       fld     f0,–16(x1)
       fadd.d  f12,f0,f2
       fsd     f12,–16(x1)   //drop addi & bne
       fld     f14,–24(x1)
       fadd.d  f16,f14,f2
       fsd     f16,–24(x1)
       addi    x1,x1,–32
       bne     x1,x2,Loop
```

We have eliminated three branches and three decrements of `x1`. The addresses on the loads and stores have been compensated to allow the `addi` instructions on `x1` to be merged. This optimization may seem trivial, but it is not; it requires symbolic substitution and simplification. Symbolic substitution and simplification will rearrange expressions so as to allow constants to be collapsed, allowing an expression such as $((i+1)+1)$ to be rewritten as $(i+(1+1))$ and then simplified to $(i+2)$.

We will see more general forms of these optimizations that eliminate dependent computations in Appendix H.

Without scheduling, every FP load or operation in the unrolled loop is followed by a dependent operation and thus will cause a stall. This unrolled loop will run in 26 clock cycles—each `fld` has 1 stall, each `fadd.d` has 2, plus 14 instruction issue cycles—or 6.5 clock cycles for each of the four elements, but it can be scheduled to improve performance significantly. Loop unrolling is normally done early in the compilation process so that redundant computations can be exposed and eliminated by the optimizer.

In real programs, we do not usually know the upper bound on the loop. Suppose it is *n,* and we want to unroll the loop to make *k* copies of the body. Instead of a single unrolled loop, we generate a pair of consecutive loops. The first executes (*n* mod *k*) times and has a body that is the original loop. The second is the unrolled body surrounded by an outer loop that iterates (*n*/*k*) times. (As we will see in Chapter 4, this technique is similar to a technique called *strip mining*, used in compilers for vector processors.) For large values of *n*, most of the execution time will be spent in the unrolled loop body.

In the previous example, unrolling improves the performance of this loop by eliminating overhead instructions, although it increases code size substantially. How will the unrolled loop perform when it is scheduled for the pipeline described earlier?

Example Show the unrolled loop in the previous example after it has been scheduled for the pipeline with the latencies in Figure 3.2.

Answer

```
Loop:  fld     f0,0(x1)
       fld     f6,-8(x1)
       fld     f0,-16(x1)
       fld     f14,-24(x1)
       fadd.d  f4,f0,f2
       fadd.d  f8,f6,f2
       fadd.d  f12,f0,f2
       fadd.d  f16,f14,f2
       fsd     f4,0(x1)
       fsd     f8,-8(x1)
       fsd     f12,16(x1)
       fsd     f16,8(x1)
       addi    x1,x1,-32
       bne     x1,x2,Loop
```

The execution time of the unrolled loop has dropped to a total of 14 clock cycles, or 3.5 clock cycles per element, compared with 8 cycles per element before any unrolling or scheduling and 6.5 cycles when unrolled but not scheduled.

The gain from scheduling on the unrolled loop is even larger than on the original loop. This increase arises because unrolling the loop exposes more computation that can be scheduled to minimize the stalls; the preceding code has no stalls. Scheduling the loop in this fashion necessitates realizing that the loads and stores are independent and can be interchanged.

Summary of the Loop Unrolling and Scheduling

Throughout this chapter and Appendix H, we will look at a variety of hardware and software techniques that allow us to take advantage of instruction-level parallelism to fully utilize the potential of the functional units in a processor. The key to most of these techniques is to know when and how the ordering among instructions may be changed. In our example, we made many such changes, which to us, as human beings, were obviously allowable. In practice, this process must be performed in a methodical fashion either by a compiler or by hardware. To obtain the final unrolled code, we had to make the following decisions and transformations:

- Determine that unrolling the loop would be useful by finding that the loop iterations were independent, except for the loop maintenance code.
- Use different registers to avoid unnecessary constraints that would be forced by using the same registers for different computations (e.g., name dependences).
- Eliminate the extra test and branch instructions and adjust the loop termination and iteration code.
- Determine that the loads and stores in the unrolled loop can be interchanged by observing that the loads and stores from different iterations are independent. This transformation requires analyzing the memory addresses and finding that they do not refer to the same address.
- Schedule the code, preserving any dependences needed to yield the same result as the original code.

The key requirement underlying all of these transformations is an understanding of how one instruction depends on another and how the instructions can be changed or reordered given the dependences.

Three different effects limit the gains from loop unrolling: (1) a decrease in the amount of overhead amortized with each unroll, (2) code size limitations, and (3) compiler limitations. Let's consider the question of loop overhead first. When we unrolled the loop four times, it generated sufficient parallelism among the instructions that the loop could be scheduled with no stall cycles. In fact, in 14 clock cycles, only 2 cycles were loop overhead: the `addi`, which maintains the index value, and the `bne`, which terminates the loop. If the loop is unrolled eight times, the overhead is reduced from 1/2 cycle per element to 1/4.

A second limit to unrolling is the resulting growth in code size. For larger loops, the code size growth may be a concern, particularly if it causes an increase in the instruction cache miss rate.

Another factor often more important than code size is the potential shortfall in registers that is created by aggressive unrolling and scheduling. This secondary effect that results from instruction scheduling in large code segments is called *register pressure*. It arises because scheduling code to increase ILP causes the number of live values to increase. After aggressive instruction scheduling, it may not be possible to allocate all the live values to registers. The transformed code, while theoretically faster, may lose some or all of its advantage because it leads to a shortage of registers. Without unrolling, aggressive scheduling is sufficiently limited by branches so that register pressure is rarely a problem. The combination of unrolling and aggressive scheduling can, however, cause this problem. The problem becomes especially challenging in multiple-issue processors that require the exposure of more independent instruction sequences whose execution can be overlapped. In general, the use of sophisticated high-level transformations, whose potential improvements are difficult to measure before detailed code generation, has led to significant increases in the complexity of modern compilers.

Loop unrolling is a simple but useful method for increasing the size of straight-line code fragments that can be scheduled effectively. This transformation is useful in a variety of processors, from simple pipelines like those we have examined so far to the multiple-issue superscalars and VLIWs explored later in this chapter.

3.3 Reducing Branch Costs With Advanced Branch Prediction

Because of the need to enforce control dependences through branch hazards and stalls, branches will hurt pipeline performance. Loop unrolling is one way to reduce the number of branch hazards; we can also reduce the performance losses of branches by predicting how they will behave. In Appendix C, we examine simple branch predictors that rely either on compile-time information or on the observed dynamic behavior of a single branch in isolation. As the number of instructions in flight has increased with deeper pipelines and more issues per clock, the importance of more accurate branch prediction has grown. In this section, we examine techniques for improving dynamic prediction accuracy. This section makes extensive use of the simple 2-bit predictor covered in Section C.2, and it is critical that the reader understand the operation of that predictor before proceeding.

Correlating Branch Predictors

The 2-bit predictor schemes in Appendix C use only the recent behavior of a single branch to predict the future behavior of that branch. It may be possible to improve the prediction accuracy if we also look at the recent behavior of *other* branches rather than just the branch we are trying to predict. Consider a small code fragment from the eqntott benchmark, a member of early SPEC benchmark suites that displayed particularly bad branch prediction behavior:

```
if (aa==2)
        aa=0;
if (bb==2)
        bb=0;
if (aa!=bb) {
```

Here is the RISC-V code that we would typically generate for this code fragment assuming that `aa` and `bb` are assigned to registers `x1` and `x2`:

```
     addi  x3,x1,-2
     bnez  x3,L1       //branch b1   (aa!=2)
     add   x1,x0,x0    //aa=0
L1:  addi  x3,x2,-2
     bnez  x3,L2       //branch b2  (bb!=2)
     add   x2,x0,x0    //bb=0
L2:  sub   x3,x1,x2    //x3=aa-bb
     beqz  x3,L3       //branch b3   (aa==bb)
```

Let's label these branches b1, b2, and b3. The key observation is that the behavior of branch b3 is correlated with the behavior of branches b1 and b2. Clearly, if neither branches b1 nor b2 are taken (i.e., if the conditions both evaluate to true and `aa` and `bb` are both assigned 0), then b3 will be taken, because `aa` and `bb` are clearly equal. A predictor that uses the behavior of only a single branch to predict the outcome of that branch can never capture this behavior.

Branch predictors that use the behavior of other branches to make a prediction are called *correlating predictors* or *two-level predictors*. Existing correlating predictors add information about the behavior of the most recent branches to decide how to predict a given branch. For example, a (1,2) predictor uses the behavior of the last branch to choose from among a pair of 2-bit branch predictors in predicting a particular branch. In the general case, an (*m,n*) predictor uses the behavior of the last *m* branches to choose from 2^m branch predictors, each of which is an *n*-bit predictor for a single branch. The attraction of this type of correlating branch predictor is that it can yield higher prediction rates than the 2-bit scheme and requires only a trivial amount of additional hardware.

The simplicity of the hardware comes from a simple observation: the global history of the most recent *m* branches can be recorded in an *m*-bit shift register, where each bit records whether the branch was taken or not taken. The branch-prediction buffer can then be indexed using a concatenation of the low-order bits from the branch address with the *m*-bit global history. For example, in a (2,2) buffer with 64 total entries, the 4 low-order address bits of the branch (word address) and the 2 global bits representing the behavior of the two most recently executed branches form a 6-bit index that can be used to index the 64 counters. By combining the local and global information by concatenation (or a simple hash function), we can index the predictor table with the result and get a prediction as fast as we could for the standard 2-bit predictor, as we will do very shortly.

How much better do the correlating branch predictors work when compared with the standard 2-bit scheme? To compare them fairly, we must compare predictors that use the same number of state bits. The number of bits in an (m,n) predictor is

$$2^m \times n \times \text{Number of prediction entries selected by the branch address}$$

A 2-bit predictor with no global history is simply a (0,2) predictor.

Example How many bits are in the (0,2) branch predictor with 4K entries? How many entries are in a (2,2) predictor with the same number of bits?

Answer The predictor with 4K entries has

$$2^0 \times 2 \times 4\text{K} = 8\text{K bits}$$

How many branch-selected entries are in a (2,2) predictor that has a total of 8K bits in the prediction buffer? We know that

$$2^2 \times 2 \times \text{Number of prediction entries selected by the branch} = 8\text{K}$$

Therefore the number of prediction entries selected by the branch = 1K.

Figure 3.3 compares the misprediction rates of the earlier (0,2) predictor with 4K entries and a (2,2) predictor with 1K entries. As you can see, this correlating predictor not only outperforms a simple 2-bit predictor with the same total number of state bits, but it also often outperforms a 2-bit predictor with an unlimited number of entries.

Perhaps the best-known example of a correlating predictor is McFarling's gshare predictor. In gshare the index is formed by combining the address of the branch and the most recent conditional branch outcomes using an exclusive-OR, which essentially acts as a hash of the branch address and the branch history. The hashed result is used to index a prediction array of 2-bit counters, as shown in Figure 3.4. The gshare predictor works remarkably well for a simple predictor, and is often used as the baseline for comparison with more sophisticated predictors. Predictors that combine local branch information and global branch history are also called *alloyed predictors or hybrid predictors*.

Tournament Predictors: Adaptively Combining Local and Global Predictors

The primary motivation for correlating branch predictors came from the observation that the standard 2-bit predictor, using only local information, failed on some important branches. Adding global history could help remedy this situation. *Tournament predictors* take this insight to the next level, by using multiple predictors, usually a global predictor and a local predictor, and choosing between them

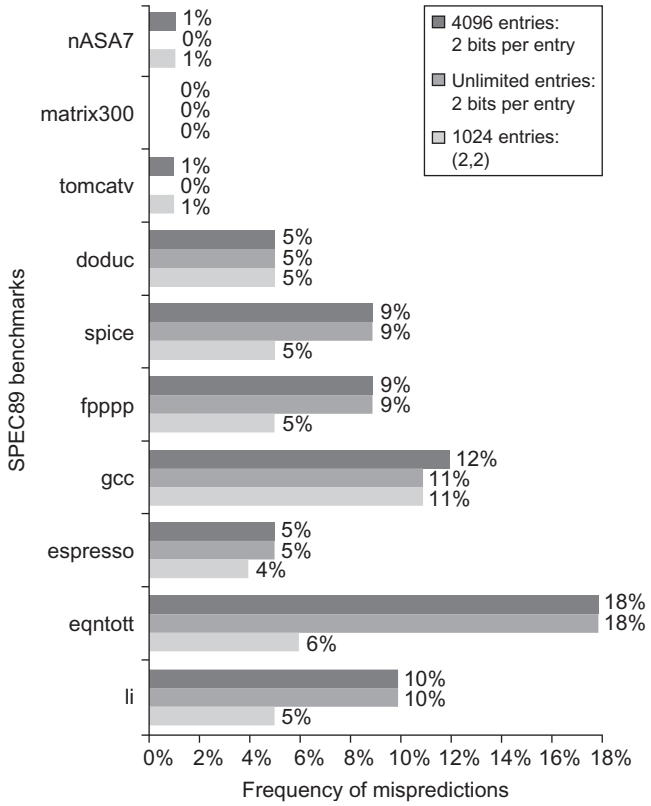


Figure 3.3 Comparison of 2-bit predictors. A noncorrelating predictor for 4096 bits is first, followed by a noncorrelating 2-bit predictor with unlimited entries and a 2-bit predictor with 2 bits of global history and a total of 1024 entries. Although these data are for an older version of SPEC, data for more recent SPEC benchmarks would show similar differences in accuracy.

with a selector, as shown in Figure 3.5. A *global predictor* uses the most recent branch history to index the predictor, while a *local predictor* uses the address of the branch as the index. Tournament predictors are another form of hybrid or alloyed predictors.

Tournament predictors can achieve better accuracy at medium sizes (8K–32K bits) and also effectively use very large numbers of prediction bits. Existing tournament predictors use a 2-bit saturating counter per branch to choose among two different predictors based on which predictor (local, global, or even some time-varying mix) was most effective in recent predictions. As in a simple 2-bit predictor, the saturating counter requires two mispredictions before changing the identity of the preferred predictor.

The advantage of a tournament predictor is its ability to select the right predictor for a particular branch, which is particularly crucial for the integer benchmarks.

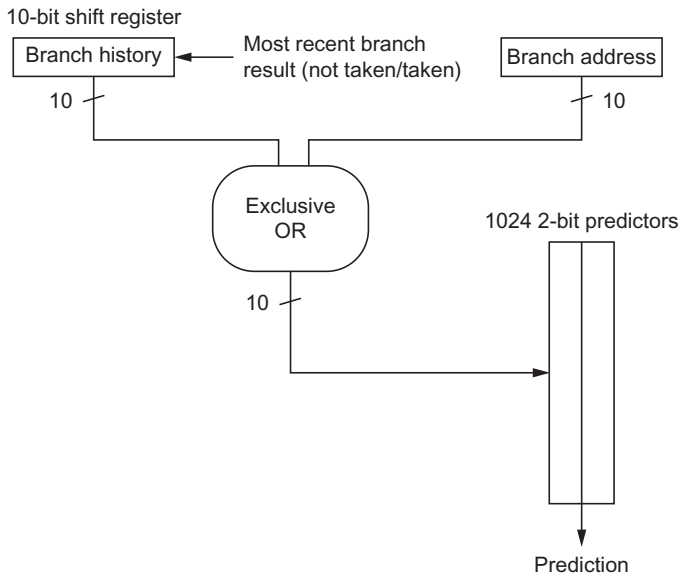


Figure 3.4 A gshare predictor with 1024 entries, each being a standard 2-bit predictor.

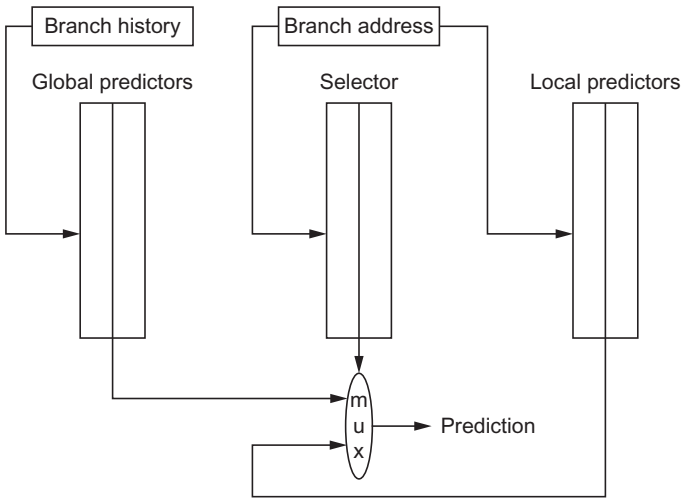


Figure 3.5 A tournament predictor using the branch address to index a set of 2-bit selection counters, which choose between a local and a global predictor. In this case, the index to the selector table is the current branch address. The two tables are also 2-bit predictors that are indexed by the global history and branch address, respectively. The selector acts like a 2-bit predictor, changing the preferred predictor for a branch address when two mispredicts occur in a row. The number of bits of the branch address used to index the selector table and the local predictor table is equal to the length of the global branch history used to index the global prediction table. Note that misprediction is a bit tricky because we need to change both the selector table and either the global or local predictor.

A typical tournament predictor will select the global predictor almost 40% of the time for the SPEC integer benchmarks and less than 15% of the time for the SPEC FP benchmarks. In addition to the Alpha processors that pioneered tournament predictors, several AMD processors have used tournament-style predictors.

Figure 3.6 looks at the performance of three different predictors (a local 2-bit predictor, a correlating predictor, and a tournament predictor) for different numbers of bits using SPEC89 as the benchmark. The local predictor reaches its limit first. The correlating predictor shows a significant improvement, and the tournament predictor generates a slightly better performance. For more recent versions of the SPEC, the results would be similar, but the asymptotic behavior would not be reached until slightly larger predictor sizes.

The local predictor consists of a two-level predictor. The top level is a local history table consisting of 1024 10-bit entries; each 10-bit entry corresponds to the most recent 10 branch outcomes for the entry. That is, if the branch is taken 10 or more times in a row, the entry in the local history table will be all 1s. If the branch is alternately taken and untaken, the history entry consists of alternating 0s and 1s. This 10-bit history allows patterns of up to 10 branches to be discovered and predicted. The selected entry from the local history table is used to index a table of 1K entries consisting of 3-bit saturating counters, which provide the local prediction. This combination, which uses a total of 29K bits, leads to high accuracy in

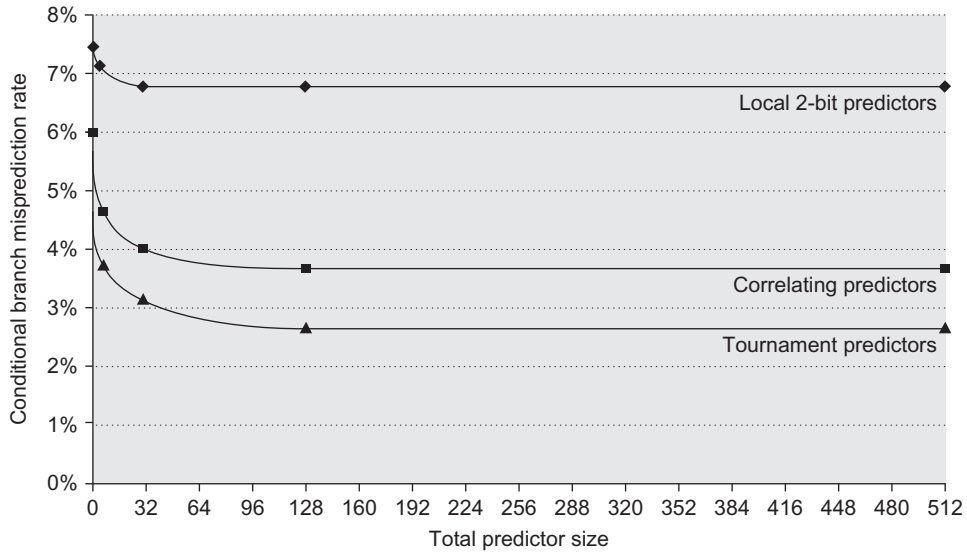


Figure 3.6 The misprediction rate for three different predictors on SPEC89 versus the size of the predictor in kilobits. The predictors are a local 2-bit predictor, a correlating predictor that is optimally structured in its use of global and local information at each point in the graph, and a tournament predictor. Although these data are for an older version of SPEC, data for more recent SPEC benchmarks show similar behavior, perhaps converging to the asymptotic limit at slightly larger predictor sizes.

branch prediction while requiring fewer bits than a single level table with the same prediction accuracy.

Tagged Hybrid Predictors

The best performing branch prediction schemes as of 2017 involve combining multiple predictors that track whether a prediction is likely to be associated with the current branch. One important class of predictors is loosely based on an algorithm for statistical compression called PPM (Prediction by Partial Matching). PPM (see Jiménez and Lin, 2001), like a branch prediction algorithm, attempts to predict future behavior based on history. This class of branch predictors, which we call *tagged hybrid predictors* (see Seznec and Michaud, 2006), employs a series of global predictors indexed with different length histories.

For example, as shown in Figure 3.7, a five-component tagged hybrid predictor has five prediction tables: P(0), P(1), . . . P(4), where P(*i*) is accessed using a hash of

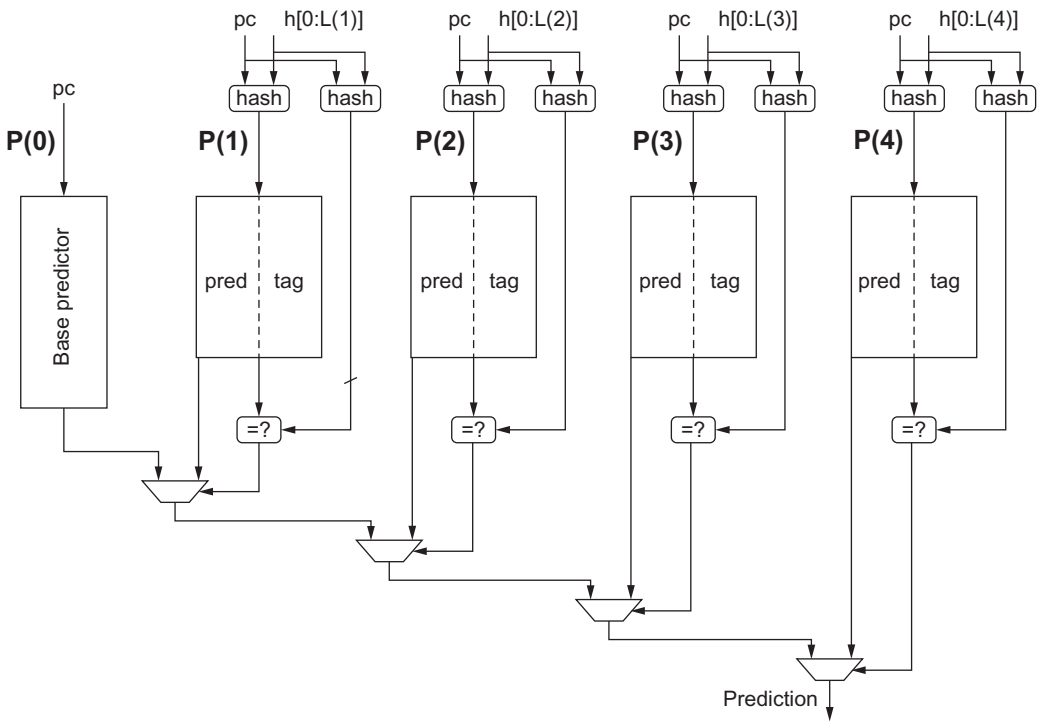


Figure 3.7 A five-component tagged hybrid predictor has five separate prediction tables, indexed by a hash of the branch address and a segment of recent branch history of length 0–4 labeled "h" in this figure. The hash can be as simple as an exclusive-OR, as in gshare. Each predictor is a 2-bit (or possibly 3-bit) predictor. The tags are typically 4–8 bits. The chosen prediction is the one with the longest history where the tags also match.

the PC and the history of the most recent *i* branches (kept in a shift register, h, just as in gshare). The use of multiple history lengths to index separate predictors is the first critical difference. The second critical difference is the use of tags in tables P(1) through P(4). The tags can be short because 100% matches are not required: a small tag of 4–8 bits appears to gain most of the advantage. A prediction from P(1), . . . P(4) is used only if the tags match the hash of the branch address and global branch history. Each of the predictors in P(0…*n*) can be a standard 2-bit predictor. In practice a 3-bit counter, which requires three mispredictions to change a prediction, gives slightly better results than a 2-bit counter.

The prediction for a given branch is the predictor with the longest branch history that also has matching tags. P(0) always matches because it uses no tags and becomes the default prediction if none of P(1) through P(*n*) match. The tagged hybrid version of this predictor also includes a 2-bit use field in each of the history-indexed predictors. The use field indicates whether a prediction was recently used and therefore likely to be more accurate; the use field can be periodically reset in all entries so that old predictions are cleared. Many more details are involved in implementing this style of predictor, especially how to handle mispredictions. The search space for the optimal predictor is also very large because the number of predictors, the exact history used for indexing, and the size of each predictor are all variable.

Tagged hybrid predictors (sometimes called TAGE—TAgged GEometic—predictors) and the earlier PPM-based predictors have been the winners in recent annual international branch-prediction competitions. Such predictors outperform gshare and the tournament predictors with modest amounts of memory (32–64 KiB), and in addition, this class of predictors seems able to effectively use larger prediction caches to deliver improved prediction accuracy.

Another issue for larger predictors is how to initialize the predictor. It could be initialized randomly, in which case, it will take a fair amount of execution time to fill the predictor with useful predictions. Some predictors (including many recent predictors) include a valid bit, indicating whether an entry in the predictor has been set or is in the "unused state." In the latter case, rather than use a random prediction, we could use some method to initialize that prediction entry. For example, some instruction sets contain a bit that indicates whether an associated branch is expected to be taken or not. In the days before dynamic branch prediction, such hint bits *were* the prediction; in recent processors, that hint bit can be used to set the initial prediction. We could also set the initial prediction on the basis of the branch direction: forward going branches are initialized as not taken, while backward going branches, which are likely to be loop branches, are initialized as taken. For programs with shorter running times and processors with larger predictors, this initial setting can have a measurable impact on prediction performance.

Figure 3.8 shows that a hybrid tagged predictor significantly outperforms gshare, especially for the less predictable programs like SPECint and server applications. In this figure, performance is measured as mispredicts per thousand instructions; assuming a branch frequency of 20%–25%, gshare has a mispredict rate (per branch) of 2.7%–3.4% for the multimedia benchmarks, while the tagged

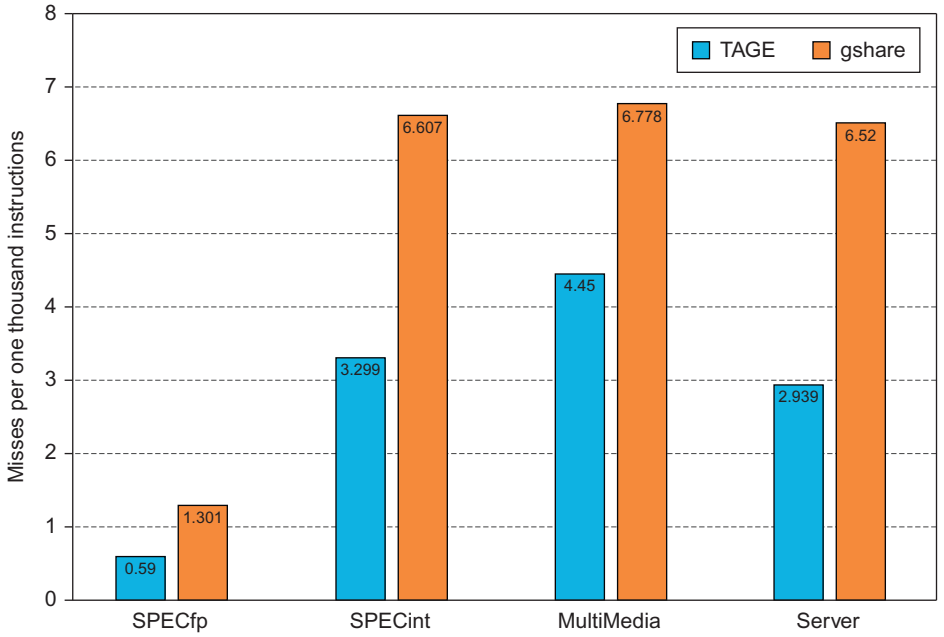


Figure 3.8 A comparison of the misprediction rate (measured as mispredicts per 1000 instructions executed) for tagged hybrid versus gshare. Both predictors use the same total number of bits, although tagged hybrid uses some of that storage for tags, while gshare contains no tags. The benchmarks consist of traces from SPECfp and SPECint, a series of multimedia and server benchmarks. The latter two behave more like SPECint.

hybrid predictor has a misprediction rate of 1.8%–2.2%, or roughly one-third fewer mispredicts. Compared to gshare, tagged hybrid predictors are more complex to implement and are probably slightly slower because of the need to check multiple tags and choose a prediction result. Nonetheless, for deeply pipelined processors with large penalties for branch misprediction, the increased accuracy outweighs those disadvantages. Thus many designers of higher-end processors have opted to include tagged hybrid predictors in their newest implementations.

The Evolution of the Intel Core i7 Branch Predictor

As mentioned in the previous chapter, there were six generations of Intel Core i7 processors between 2008 (Core i7 920 using the Nehalem microarchitecture) and 2016 (Core i7 6700 using the Skylake microarchitecture). Because of the combination of deep pipelining and multiple issues per clock, the i7 has many instructions in-flight at once (up to 256, and typically at least 30). This makes branch prediction critical, and it has been an area where Intel has been making constant improvements. Perhaps because of the performance-critical nature of the branch predictor, Intel has tended to keep the details of its branch predictors highly secret.

Even for older processors such as the Core i7 920 introduced in 2008, they have released only limited amounts of information. In this section, we briefly describe what is known and compare the performance of predictors of the Core i7 920 with those in the latest Core i7 6700.

The Core i7 920 used a two-level predictor that has a smaller first-level predictor, designed to meet the cycle constraints of predicting a branch every clock cycle, and a larger second-level predictor as a backup. Each predictor combines three different predictors: (1) the simple 2-bit predictor, which is introduced in Appendix C (and used in the preceding tournament predictor); (2) a global history predictor, like those we just saw; and (3) a loop exit predictor. The loop exit predictor uses a counter to predict the exact number of taken branches (which is the number of loop iterations) for a branch that is detected as a loop branch. For each branch, the best prediction is chosen from among the three predictors by tracking the accuracy of each prediction, like a tournament predictor. In addition to this multilevel main predictor, a separate unit predicts target addresses for indirect branches, and a stack to predict return addresses is also used.

Although even less is known about the predictors in the newest i7 processors, there is good reason to believe that Intel is employing a tagged hybrid predictor. One advantage of such a predictor is that it combines the functions of all three second-level predictors in the earlier i7. The tagged hybrid predictor with different history lengths subsumes the loop exit predictor as well as the local and global history predictor. A separate return address predictor is still employed.

As in other cases, speculation causes some challenges in evaluating the predictor because a mispredicted branch can easily lead to another branch being fetched and mispredicted. To keep things simple, we look at the number of mispredictions as a percentage of the number of successfully completed branches (those that were not the result of misspeculation). Figure 3.9 shows these data for SPECPUint2006 benchmarks. These benchmarks are considerably larger than SPEC89 or SPEC2000, with the result being that the misprediction rates are higher than those in Figure 3.6 even with a more powerful combination of predictors. Because branch misprediction leads to ineffective speculation, it contributes to the wasted work, as we will see later in this chapter.

3.4 Overcoming Data Hazards With Dynamic Scheduling

A simple statically scheduled pipeline fetches an instruction and issues it, unless there is a data dependence between an instruction already in the pipeline and the fetched instruction that cannot be hidden with bypassing or forwarding. (Forwarding logic reduces the effective pipeline latency so that the certain dependences do not result in hazards.) If there is a data dependence that cannot be hidden, then the hazard detection hardware stalls the pipeline starting with the instruction that uses the result. No new instructions are fetched or issued until the dependence is cleared.

In this section, we explore *dynamic scheduling*, a technique by which the hardware reorders the instruction execution to reduce the stalls while maintaining data

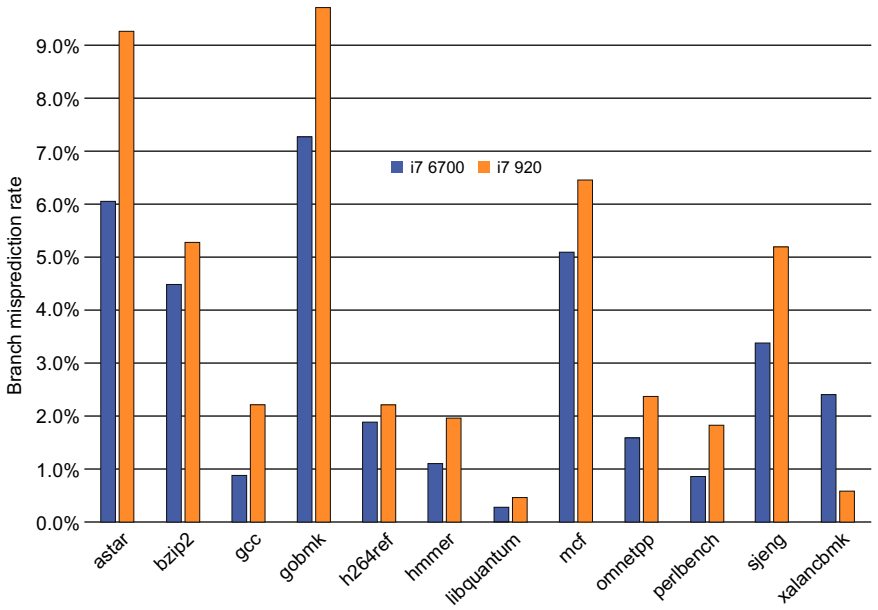


Figure 3.9 The misprediction rate for the integer SPECCPU2006 benchmarks on the Intel Core i7 920 and 6700. The misprediction rate is computed as the ratio of completed branches that are mispredicted versus all completed branches. This could understate the misprediction rate somewhat because if a branch is mispredicted and led to another mispredicted branch (which should not have been executed), it will be counted as only one misprediction. On average, the i7 920 mispredicts branches 1.3 times as often as the i7 6700.

flow and exception behavior. Dynamic scheduling offers several advantages. First, it allows code that was compiled with one pipeline in mind to run efficiently on a different pipeline, eliminating the need to have multiple binaries and recompile for a different microarchitecture. In today's computing environment, where much of the software is from third parties and distributed in binary form, this advantage is significant. Second, it enables handling some cases when dependences are unknown at compile time; for example, they may involve a memory reference or a data-dependent branch, or they may result from a modern programming environment that uses dynamic linking or dispatching. Third, and perhaps most importantly, it allows the processor to tolerate unpredictable delays, such as cache misses, by executing other code while waiting for the miss to resolve. In Section 3.6, we explore hardware speculation, a technique with additional performance advantages, which builds on dynamic scheduling. As we will see, the advantages of dynamic scheduling are gained at the cost of a significant increase in hardware complexity.

Although a dynamically scheduled processor cannot change the data flow, it tries to avoid stalling when dependences are present. In contrast, static pipeline scheduling by the compiler (covered in Section 3.2) tries to minimize stalls by separating dependent instructions so that they will not lead to hazards. Of course,

compiler pipeline scheduling can also be used in code destined to run on a processor with a dynamically scheduled pipeline.

Dynamic Scheduling: The Idea

A major limitation of simple pipelining techniques is that they use in-order instruction issue and execution: instructions are issued in program order, and if an instruction is stalled in the pipeline, no later instructions can proceed. Thus, if there is a dependence between two closely spaced instructions in the pipeline, it will lead to a hazard, and a stall will result. If there are multiple functional units, these units could lie idle. If instruction *j* depends on a long-running instruction *i,* currently in execution in the pipeline, then all instructions after *j* must be stalled until *i* is finished and *j* can execute. For example, consider this code:

```
fdiv.d  f0,f2,f4
fadd.d  f10,f0,f8
fsub.d  f12,f8,f14
```

The `fsub.d` instruction cannot execute because the dependence of `fadd.d` on `fdiv.d` causes the pipeline to stall; yet, `fsub.d` is not data-dependent on anything in the pipeline. This hazard creates a performance limitation that can be eliminated by not requiring instructions to execute in program order.

In the classic five-stage pipeline, both structural and data hazards could be checked during instruction decode (ID): when an instruction could execute without hazards, it was issued from ID, with the recognition that all data hazards had been resolved.

To allow us to begin executing the `fsub.d` in the preceding example, we must separate the issue process into two parts: checking for any structural hazards and waiting for the absence of a data hazard. Thus we still use in-order instruction issue (i.e., instructions issued in program order), but we want an instruction to begin execution as soon as its data operands are available. Such a pipeline does *out-of-order execution*, which implies *out-of-order completion*.

Out-of-order execution introduces the possibility of WAR and WAW hazards, which do not exist in the five-stage integer pipeline and its logical extension to an in-order floating-point pipeline. Consider the following RISC-V floating-point code sequence:

```
fdiv.d  f0,f2,f4
fmul.d  f6,f0,f8
fadd.d  f0,f10,f14
```

There is an antidependence between the `fmul.d` and the `fadd.d` (for the register `f0`), and if the pipeline executes the `fadd.d` before the `fmul.d` (which is waiting for the `fdiv.d`), it will violate the antidependence, yielding a WAR hazard. Likewise, to avoid violating output dependences, such as the write of `f0` by `fadd.d` before `fdiv.d` completes, WAW hazards must be handled. As we will see, both these hazards are avoided by the use of register renaming.

Out-of-order completion also creates major complications in handling exceptions. Dynamic scheduling with out-of-order completion must preserve exception behavior in the sense that *exactly* those exceptions that would arise if the program were executed in strict program order *actually* do arise. Dynamically scheduled processors preserve exception behavior by delaying the notification of an associated exception until the processor knows that the instruction should be the next one completed.

Although exception behavior must be preserved, dynamically scheduled processors could generate *imprecise* exceptions. An exception is *imprecise* if the processor state when an exception is raised does not look exactly as if the instructions were executed sequentially in strict program order. Imprecise exceptions can occur because of two possibilities:

1. The pipeline may have *already completed* instructions that are *later* in program order than the instruction causing the exception.
2. The pipeline may have *not yet completed* some instructions that are *earlier* in program order than the instruction causing the exception.

Imprecise exceptions make it difficult to restart execution after an exception. Rather than address these problems in this section, we will discuss a solution that provides precise exceptions in the context of a processor with speculation in Section 3.6. For floating-point exceptions, other solutions have been used, as discussed in Appendix J.

To allow out-of-order execution, we essentially split the ID pipe stage of our simple five-stage pipeline into two stages:

1. *Issue*—Decode instructions, check for structural hazards.
2. *Read operands*—Wait until no data hazards, then read operands.

An instruction fetch stage precedes the issue stage and may fetch either to an instruction register or into a queue of pending instructions; instructions are then issued from the register or queue. The execution stage follows the read operands stage, just as in the five-stage pipeline. Execution may take multiple cycles, depending on the operation.

We distinguish when an instruction *begins execution* and when it *completes execution*; between the two times, the instruction is *in execution.* Our pipeline allows multiple instructions to be in execution at the same time; without this capability, a major advantage of dynamic scheduling is lost. Having multiple instructions in execution at once requires multiple functional units, pipelined functional units, or both. Because these two capabilities—pipelined functional units and multiple functional units—are essentially equivalent for the purposes of pipeline control, we will assume the processor has multiple functional units.

In a dynamically scheduled pipeline, all instructions pass through the issue stage in order (in-order issue); however, they can be stalled or can bypass each

other in the second stage (read operands) and thus enter execution out of order. *Scoreboarding* is a technique for allowing instructions to execute out of order when there are sufficient resources and no data dependences; it is named after the CDC 6600 scoreboard, which developed this capability. Here we focus on a more sophisticated technique, called *Tomasulo's algorithm.* The primary difference is that Tomasulo's algorithm handles antidependences and output dependences by effectively renaming the registers dynamically. Additionally, Tomasulo's algorithm can be extended to handle *speculation*, a technique to reduce the effect of control dependences by predicting the outcome of a branch, executing instructions at the predicted destination address, and taking corrective actions when the prediction was wrong. While the use of scoreboarding is probably sufficient to support simpler processors, more sophisticated, higher performance processors make use of speculation.

Dynamic Scheduling Using Tomasulo's Approach

The IBM 360/91 floating-point unit used a sophisticated scheme to allow out-of-order execution. This scheme, invented by Robert Tomasulo, tracks when operands for instructions are available to minimize RAW hazards and introduces register renaming in hardware to minimize WAW and WAR hazards. Although there are many variations of this scheme in recent processors, they all rely on two key principles: dynamically determining when an instruction is ready to execute and renaming registers to avoid unnecessary hazards.

IBM's goal was to achieve high floating-point performance from an instruction set and from compilers designed for the entire 360 computer family, rather than from specialized compilers for the high-end processors. The 360 architecture had only four double-precision floating-point registers, which limited the effectiveness of compiler scheduling; this fact was another motivation for the Tomasulo approach. In addition, the IBM 360/91 had long memory accesses and long floating-point delays, which Tomasulo's algorithm was designed to overcome. At the end of the section, we will see that Tomasulo's algorithm can also support the overlapped execution of multiple iterations of a loop.

We explain the algorithm, which focuses on the floating-point unit and load-store unit, in the context of the RISC-V instruction set. The primary difference between RISC-V and the 360 is the presence of register-memory instructions in the latter architecture. Because Tomasulo's algorithm uses a load functional unit, no significant changes are needed to add register-memory addressing modes. The IBM 360/91 also had pipelined functional units, rather than multiple functional units, but we describe the algorithm as if there were multiple functional units. It is a simple conceptual extension to also pipeline those functional units.

RAW hazards are avoided by executing an instruction only when its operands are available, which is exactly what the simpler scoreboarding approach provides. WAR and WAW hazards, which arise from name dependences, are eliminated by register renaming. *Register renaming* eliminates these hazards by renaming all

destination registers, including those with a pending read or write for an earlier instruction, so that the out-of-order write does not affect any instructions that depend on an earlier value of an operand. The compiler could typically implement such renaming, if there were enough registers available in the ISA. The original 360/91 had only four floating-point registers, and Tomasulo's algorithm was created to overcome this shortage. Whereas modern processors have 32–64 floating-point and integer registers, the number of renaming registers available in recent implementations is in the hundreds.

To better understand how register renaming eliminates WAR and WAW hazards, consider the following example code sequence that includes potential WAR and WAW hazards:

```
fdiv.d  f0,f2,f4
fadd.d  f6,f0,f8
fsd     f6,0(x1)
fsub.d  f8,f10,f14
fmul.d  f6,f10,f8
```

There are two antidependences: between the `fadd.d` and the `fsub.d` and between the `fsd` and the `fmul.d`. There is also an output dependence between the `fadd.d` and the `fmul.d`, leading to three possible hazards: WAR hazards on the use of `f8` by `fadd.d` and its use by the `fsub.d`, as well as a WAW hazard because the `fadd.d` may finish later than the `fmul.d`. There are also three true data dependences: between the `fdiv.d` and the `fadd.d`, between the `fsub.d` and the `fmul.d`, and between the `fadd.d` and the `fsd`.

These three name dependences can all be eliminated by register renaming. For simplicity, assume the existence of two temporary registers, `S` and `T`. Using `S` and `T`, the sequence can be rewritten without any dependences as

```
fdiv.d  f0,f2,f4
fadd.d  S,f0,f8
fsd     S,0(x1)
fsub.d  T,f10,f14
fmul.d  f6,f10,T
```

In addition, any subsequent uses of `f8` must be replaced by the register `T`. In this example, the renaming process can be done statically by the compiler. Finding any uses of `f8` that are later in the code requires either sophisticated compiler analysis or hardware support because there may be intervening branches between the preceding code segment and a later use of `f8`. As we will see, Tomasulo's algorithm can handle renaming across branches.

In Tomasulo's scheme, register renaming is provided by *reservation stations*, which buffer the operands of instructions waiting to issue and are associated with the functional units. The basic idea is that a reservation station fetches and buffers an operand as soon as it is available, eliminating the need to get the operand from a register. In addition, pending instructions designate the reservation station that will provide their input. Finally, when successive writes to a register overlap in

execution, only the last one is actually used to update the register. As instructions are issued, the register specifiers for pending operands are renamed to the names of the reservation station, which provides register renaming.

Because there can be more reservation stations than real registers, the technique can even eliminate hazards arising from name dependences that could not be eliminated by a compiler. As we explore the components of Tomasulo's scheme, we will return to the topic of register renaming and see exactly how the renaming occurs and how it eliminates WAR and WAW hazards.

The use of reservation stations, rather than a centralized register file, leads to two other important properties. First, hazard detection and execution control are distributed: the information held in the reservation stations at each functional unit determines when an instruction can begin execution at that unit. Second, results are passed directly to functional units from the reservation stations where they are buffered, rather than going through the registers. This bypassing is done with a common result bus that allows all units waiting for an operand to be loaded simultaneously (on the 360/91, this is called the *common data bus*, or CDB). In pipelines that issue multiple instructions per clock and also have multiple execution units, more than one result bus will be needed.

Figure 3.10 shows the basic structure of a Tomasulo-based processor, including both the floating-point unit and the load/store unit; none of the execution control tables is shown. Each reservation station holds an instruction that has been issued and is awaiting execution at a functional unit. If the operand values for that instruction have been computed, they are also stored in that entry; otherwise, the reservation station entry keeps the names of the reservation stations that will provide the operand values.

The load buffers and store buffers hold data or addresses coming from and going to memory and behave almost exactly like reservation stations, so we distinguish them only when necessary. The floating-point registers are connected by a pair of buses to the functional units and by a single bus to the store buffers. All results from the functional units and from memory are sent on the common data bus, which goes everywhere except to the load buffer. All reservation stations have tag fields, employed by the pipeline control.

Before we describe the details of the reservation stations and the algorithm, let's look at the steps an instruction goes through. There are only three steps, although each one can now take an arbitrary number of clock cycles:

1. *Issue*—Get the next instruction from the head of the instruction queue, which is maintained in FIFO order to ensure the maintenance of correct data flow. If there is a matching reservation station that is empty, issue the instruction to the station with the operand values, if they are currently in the registers. If there is not an empty reservation station, then there is a structural hazard, and the instruction issue stalls until a station or buffer is freed. If the operands are not in the registers, keep track of the functional units that will produce the operands. This step renames registers, eliminating WAR and WAW hazards. (This stage is sometimes called *dispatch* in a dynamically scheduled processor.)

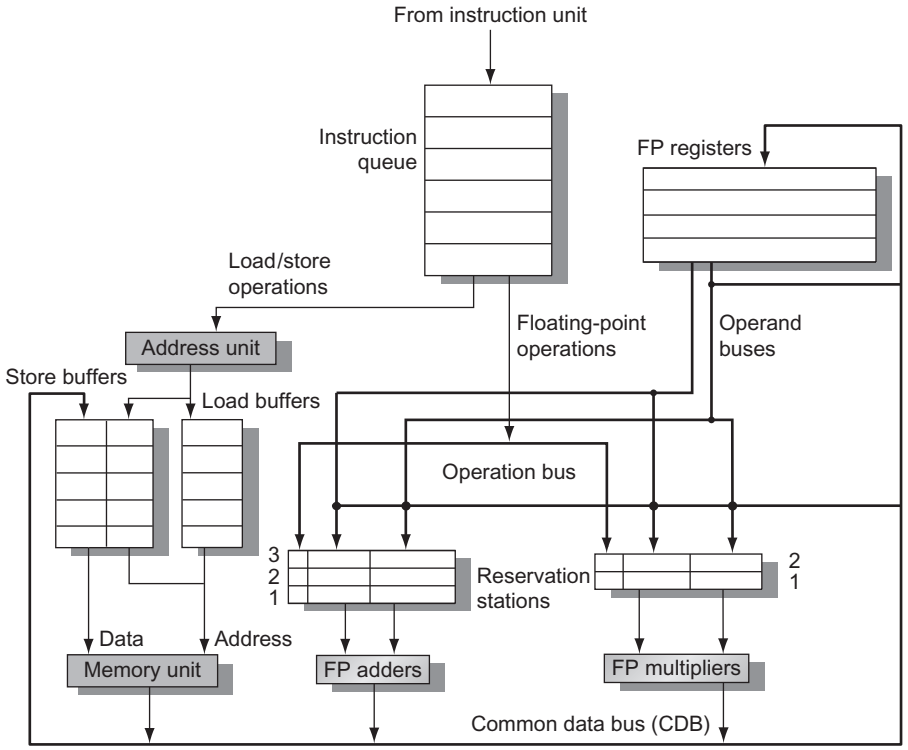


Figure 3.10 The basic structure of a RISC-V floating-point unit using Tomasulo's algorithm. Instructions are sent from the instruction unit into the instruction queue from which they are issued in first-in, first-out (FIFO) order. The reservation stations include the operation and the actual operands, as well as information used for detecting and resolving hazards. Load buffers have three functions: (1) hold the components of the effective address until it is computed, (2) track outstanding loads that are waiting on the memory, and (3) hold the results of completed loads that are waiting for the CDB. Similarly, store buffers have three functions: (1) hold the components of the effective address until it is computed, (2) hold the destination memory addresses of outstanding stores that are waiting for the data value to store, and (3) hold the address and value to store until the memory unit is available. All results from either the FP units or the load unit are put on the CDB, which goes to the FP register file as well as to the reservation stations and store buffers. The FP adders implement addition and subtraction, and the FP multipliers do multiplication and division.

2. *Execute*—If one or more of the operands is not yet available, monitor the common data bus while waiting for it to be computed. When an operand becomes available, it is placed into any reservation station awaiting it. When all the operands are available, the operation can be executed at the corresponding functional unit. By delaying instruction execution until the operands are available, RAW hazards are avoided. (Some dynamically scheduled processors call this step "issue," but we use the name "execute," which was used in the first dynamically scheduled processor, the CDC 6600.)

Notice that several instructions could become ready in the same clock cycle for the same functional unit. Although independent functional units could begin execution in the same clock cycle for different instructions, if more than one instruction is ready for a single functional unit, the unit will have to choose among them. For the floating-point reservation stations, this choice may be made arbitrarily; loads and stores, however, present an additional complication.

Loads and stores require a two-step execution process. The first step computes the effective address when the base register is available, and the effective address is then placed in the load or store buffer. Loads in the load buffer execute as soon as the memory unit is available. Stores in the store buffer wait for the value to be stored before being sent to the memory unit. Loads and stores are maintained in program order through the effective address calculation, which will help to prevent hazards through memory.

To preserve exception behavior, no instruction is allowed to initiate execution until a branch that precedes the instruction in program order has completed. This restriction guarantees that an instruction that causes an exception during execution really would have been executed. In a processor using branch prediction (as all dynamically scheduled processors do), this means that the processor must know that the branch prediction was correct before allowing an instruction after the branch to begin execution. If the processor records the occurrence of the exception, but does not actually raise it, an instruction can start execution but not stall until it enters Write Result.

Speculation provides a more flexible and more complete method to handle exceptions, so we will delay making this enhancement and show how speculation handles this problem later.

3. *Write result*—When the result is available, write it on the CDB and from there into the registers and into any reservation stations (including store buffers) waiting for this result. Stores are buffered in the store buffer until both the value to be stored and the store address are available; then the result is written as soon as the memory unit is free.

The data structures that detect and eliminate hazards are attached to the reservation stations, to the register file, and to the load and store buffers with slightly different information attached to different objects. These tags are essentially names for an extended set of virtual registers used for renaming. In our example, the tag field is a 4-bit quantity that denotes one of the five reservation stations or one of the five load buffers. This combination produces the equivalent of 10 registers (5 reservation stations + 5 load buffers) that can be designated as result registers (as opposed to the four double-precision registers that the 360 architecture contains). In a processor with more real registers, we want renaming to provide an even larger set of virtual registers, often numbering in the hundreds. The tag field describes which reservation station contains the instruction that will produce a result needed as a source operand.

Once an instruction has issued and is waiting for a source operand, it refers to the operand by the reservation station number where the instruction that will write

the register has been assigned. Unused values, such as zero, indicate that the operand is already available in the registers. Because there are more reservation stations than actual register numbers, WAW and WAR hazards are eliminated by renaming results using reservation station numbers. Although in Tomasulo's scheme the reservation stations are used as the extended virtual registers, other approaches could use a register set with additional registers or a structure like the reorder buffer, which we will see in Section 3.6.

In Tomasulo's scheme, as well as the subsequent methods we look at for supporting speculation, results are broadcast on a bus (the CDB), which is monitored by the reservation stations. The combination of the common result bus and the retrieval of results from the bus by the reservation stations implements the forwarding and bypassing mechanisms used in a statically scheduled pipeline. In doing so, however, a dynamically scheduled scheme, such as Tomasulo's algorithm, introduces one cycle of latency between source and result because the matching of a result and its use cannot be done until the end of the Write Result stage, as opposed to the end of the Execute stage for a simpler pipeline. Thus, in a dynamically scheduled pipeline, the effective latency between a producing instruction and a consuming instruction is at least one cycle longer than the latency of the functional unit producing the result.

It is important to remember that the tags in the Tomasulo scheme refer to the buffer or unit that will produce a result; the register names are discarded when an instruction issues to a reservation station. (This is a key difference between Tomasulo's scheme and scoreboarding: in scoreboarding, operands stay in the registers and are read only after the producing instruction completes and the consuming instruction is ready to execute.)

Each reservation station has seven fields:

- Op—The operation to perform on source operands S1 and S2.
- Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
- Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.
- A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
- Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

The register file has a field, Qi:

- Qi—The number of the reservation station that contains the operation whose result should be stored into this register. If the value of Qi is blank (or 0), no

currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

The load and store buffers each have a field, A, which holds the result of the effective address once the first step of execution has been completed.

In the next section, we will first consider some examples that show how these mechanisms work and then examine the detailed algorithm.

3.5 Dynamic Scheduling: Examples and the Algorithm

Before we examine Tomasulo's algorithm in detail, let's consider a few examples that will help illustrate how the algorithm works.

Example Show what the information tables look like for the following code sequence when only the first load has completed and written its result:

```
1.  fld     f6,32(x2)
2.  fld     f2,44(x3)
3.  fmul.d  f0,f2,f4
4.  fsub.d  f8,f2,f6
5.  fdiv.d  f0,f0,f6
6.  fadd.d  f6,f8,f2
```

Answer Figure 3.11 shows the result in three tables. The numbers appended to the names Add, Mult, and Load stand for the tag for that reservation station—Add1 is the tag for the result from the first add unit. In addition, we have included an instruction status table. This table is included only to help you understand the algorithm; it is *not* actually a part of the hardware. Instead, the reservation station keeps the state of each operation that has issued.

Tomasulo's scheme offers two major advantages over earlier and simpler schemes: (1) the distribution of the hazard detection logic, and (2) the elimination of stalls for WAW and WAR hazards.

The first advantage arises from the distributed reservation stations and the use of the CDB. If multiple instructions are waiting on a single result, and each instruction already has its other operand, then the instructions can be released simultaneously by the broadcast of the result on the CDB. If a centralized register file were used, the units would have to read their results from the registers when register buses were available.

The second advantage, the elimination of WAW and WAR hazards, is accomplished by renaming registers using the reservation stations and by the process of storing operands into the reservation station as soon as they are available.

For example, the code sequence in Figure 3.11 issues both the `fdiv.d` and the `fadd.d`, even though there is a WAR hazard involving `f6`. The hazard is

eliminated in one of two ways. First, if the instruction providing the value for the `fdiv.d` has completed, then Vk will store the result, allowing `fdiv.d` to execute independent of the `fadd.d` (this is the case shown). On the other hand, if the `fld` hasn't completed, then Qk will point to the Load1 reservation station, and the `fdiv.d` instruction will be independent of the `fadd.d`. Thus, in either case, the `fadd.d` can issue and begin executing. Any uses of the result of the `fdiv.d` will point to the reservation station, allowing the `fadd.d` to complete and store its value into the registers without affecting the `fdiv.d`.

	Instruction status		
Instruction	**Issue**	**Execute**	**Write result**
`fld f6,32(x2)`	√	√	√
`fld f2,44(x3)`	√	√	
`fmul.d f0,f2,f4`	√		
`fsub.d f8,f2,f6`	√		
`fdiv.d f0,f0,f6`	√		
`fadd.d f6,f8,f2`	√		

	Reservation stations						
Name	**Busy**	**Op**	**Vj**	**Vk**	**Qj**	**Qk**	**A**
Load1	No						
Load2	Yes	Load					`44 + Regs[x3]`
Add1	Yes	SUB		`Mem[32 + Regs[x2]]`	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		`Regs[f4]`	Load2		
Mult2	Yes	DIV		`Mem[32 + Regs[x2]]`	Mult1		

	Register status								
Field	**f0**	**f2**	**f4**	**f6**	**f8**	**f10**	**f12**	…	**f30**
Qi	Mult1	Load2		Add2	Add1	Mult2			

Figure 3.11 Reservation stations and register tags shown when all of the instructions have issued but only the first load instruction has completed and written its result to the CDB. The second load has completed effective address calculation but is waiting on the memory unit. We use the array Regs[ ] to refer to the register file and the array Mem[ ] to refer to the memory. Remember that an operand is specified by either a Q field or a V field at any time. Notice that the `fadd.d` instruction, which has a WAR hazard at the WB stage, has issued and could complete before the `fdiv.d` initiates.

We'll see an example of the elimination of a WAW hazard shortly. But let's first look at how our earlier example continues execution. In this example, and the ones that follow in this chapter, assume the following latencies: load is 1 clock cycle, add is 2 clock cycles, multiply is 6 clock cycles, and divide is 12 clock cycles.

Example Using the same code segment as in the previous example (page 201), show what the status tables look like when the `fmul.d` is ready to write its result.

Answer The result is shown in the three tables in Figure 3.12. Notice that `fadd.d` has completed because the operands of `fdiv.d` were copied, thereby overcoming the WAR hazard. Notice that even if the load of `f6` was `fdiv.d`, the add into `f6` could be executed without triggering a WAW hazard.

	Instruction status		
Instruction	**Issue**	**Execute**	**Write result**
`fld f6,32(x2)`	√	√	√
`fld f2,44(x3)`	√	√	√
`fmul.d f0,f2,f4`	√	√	
`fsub.d f8,f2,f6`	√	√	√
`fdiv.d f0,f0,f6`	√		
`fadd.d f6,f8,f2`	√	√	√

	Reservation stations						
Name	**Busy**	**Op**	**Vj**	**Vk**	**Qj**	**Qk**	**A**
Load1	No						
Load2	No						
Add1	No						
Add2	No						
Add3	No						
Mult1	Yes	MUL	`Mem[44 + Regs[x3]]`	`Regs[f4]`			
Mult2	Yes	DIV		`Mem[32 + Regs[x2]]`	Mult1		

	Register status								
Field	**f0**	**f2**	**f4**	**f6**	**f8**	**f10**	**f12**	…	**f30**
Qi	Mult1					Mult2			

Figure 3.12 Multiply and divide are the only instructions not finished.

Tomasulo's Algorithm: The Details

Figure 3.13 specifies the checks and steps that each instruction must go through. As mentioned earlier, loads and stores go through a functional unit for effective address computation before proceeding to independent load or store buffers. Loads take a second execution step to access memory and then go to Write Result to send the value from memory to the register file and/or any waiting reservation stations. Stores complete their execution in the Write Result stage, which writes the result to memory. Notice that all writes occur in Write Result, whether the destination is a register or memory. This restriction simplifies Tomasulo's algorithm and is critical to its extension with speculation in Section 3.6.

Tomasulo's Algorithm: A Loop-Based Example

To understand the full power of eliminating WAW and WAR hazards through dynamic renaming of registers, we must look at a loop. Consider the following simple sequence for multiplying the elements of an array by a scalar in `f2`:

```
Loop:  fld     f0,0(x1)
       fmul.d  f4,f0,f2
       fsd     f4,0(x1)
       addi    x1,x1,−8
       bne     x1,x2,Loop  // branches if x1≠x2
```

If we predict that branches are taken, using reservation stations will allow multiple executions of this loop to proceed at once. This advantage is gained without changing the code—in effect, the loop is unrolled dynamically by the hardware using the reservation stations obtained by renaming to act as additional registers.

Let's assume we have issued all the instructions in two successive iterations of the loop, but none of the floating-point load/stores or operations have completed. Figure 3.14 shows reservation stations, register status tables, and load and store buffers at this point. (The integer ALU operation is ignored, and it is assumed the branch was predicted as taken.) Once the system reaches this state, two copies of the loop could be sustained with a CPI close to 1.0, provided the multiplies could complete in four clock cycles. With a latency of six cycles, additional iterations will need to be processed before the steady state can be reached. This requires more reservation stations to hold instructions that are in execution. As we will see later in this chapter, when extended with multiple issue instructions, Tomasulo's approach can sustain more than one instruction per clock.

A load and a store can be done safely out of order, provided they access different addresses. If a load and a store access the same address, one of two things happens:

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Q ← r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes;
Load only		RegisterStat[rt].Qi ← r;
Store only		if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0};
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/storestep 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result;RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result;RS[x].Qk ← 0}); RS[r].Busy ← no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;

Figure 3.13 Steps in the algorithm and what is required for each step. For the issuing instruction, rd is the destination, rs and rt are the source register numbers, imm is the sign-extended immediate field, and r is the reservation station or buffer that the instruction is assigned to. RS is the reservation station data structure. The value returned by an FP unit or by the load unit is called result. RegisterStat is the register status data structure (not the register file, which is Regs[]). When an instruction is issued, the destination register has its Qi field set to the number of the buffer or reservation station to which the instruction is issued. If the operands are available in the registers, they are stored in the V fields. Otherwise, the Q fields are set to indicate the reservation station that will produce the values needed as source operands. The instruction waits at the reservation station until both its operands are available, indicated by zero in the Q fields. The Q fields are set to zero either when this instruction is issued or when an instruction on which this instruction depends completes and does its write back. When an instruction has finished execution and the CDB is available, it can do its write back. All the buffers, registers, and reservation stations whose values of Qj or Qk are the same as the completing reservation station update their values from the CDB and mark the Q fields to indicate that values have been received. Thus the CDB can broadcast its result to many destinations in a single clock cycle, and if the waiting instructions have their operands, they can all begin execution on the next clock cycle. Loads go through two steps in execute, and stores perform slightly differently during Write Result, where they may have to wait for the value to store. Remember that, to preserve exception behavior, instructions should not be allowed to execute if a branch that is earlier in program order has not yet completed. Because no concept of program order is maintained after the issue stage, this restriction is usually implemented by preventing any instruction from leaving the issue step if there is a pending branch already in the pipeline. In Section 3.6, we will see how speculation support removes this restriction.

		Instruction status			
Instruction		From iteration	Issue	Execute	Write result
`fld`	`f0,0(x1)`	1	√	√	
`fmul.d`	`f4,f0,f2`	1	√		
`fsd`	`f4,0(x1)`	1	√		
`fld`	`f0,0(x1)`	2	√	√	
`fmul.d`	`f4,f0,f2`	2	√		
`fsd`	`f4,0(x1)`	2	√		

	Reservation stations						
Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	Yes	Load					`Regs[x1] + 0`
Load2	Yes	Load					`Regs[x1] – 8`
Add1	No						
Add2	No						
Add3	No						
Mult1	Yes	MUL		`Regs[f2]`	Load1		
Mult2	Yes	MUL		`Regs[f2]`	Load2		
Store1	Yes	Store	`Regs[x1]`			Mult1	
Store2	Yes	Store	`Regs[x1] – 8`			Mult2	

	Register status								
Field	f0	f2	f4	f6	f8	f10	f12	…	f30
Qi	Load2		Mult2						

Figure 3.14 Two active iterations of the loop with no instruction yet completed. Entries in the multiplier reservation stations indicate that the outstanding loads are the sources. The store reservation stations indicate that the multiply destination is the source of the value to store.

- The load is before the store in program order and interchanging them results in a WAR hazard.
- The store is before the load in program order and interchanging them results in a RAW hazard.

Similarly, interchanging two stores to the same address results in a WAW hazard.

Therefore, to determine if a load can be executed at a given time, the processor can check whether any uncompleted store that precedes the load in program order

shares the same data memory address as the load. Similarly, a store must wait until there are no unexecuted loads or stores that are earlier in program order and share the same data memory address. We consider a method to eliminate this restriction in Section 3.9.

To detect such hazards, the processor must have computed the data memory address associated with any earlier memory operation. A simple, but not necessarily optimal, way to guarantee that the processor has all such addresses is to perform the effective address calculations in program order. (We really only need to keep the relative order between stores and other memory references; that is, loads can be reordered freely.)

Let's consider the situation of a load first. If we perform effective address calculation in program order, then when a load has completed effective address calculation, we can check whether there is an address conflict by examining the A field of all active store buffers. If the load address matches the address of any active entries in the store buffer, that load instruction is not sent to the load buffer until the conflicting store completes. (Some implementations bypass the value directly to the load from a pending store, reducing the delay for this RAW hazard.)

Stores operate similarly, except that the processor must check for conflicts in both the load buffers and the store buffers because conflicting stores cannot be reordered with respect to either a load or a store.

A dynamically scheduled pipeline can yield very high performance, provided branches are predicted accurately—an issue we addressed in the previous section. The major drawback of this approach is the complexity of the Tomasulo scheme, which requires a large amount of hardware. In particular, each reservation station must contain an associative buffer, which must run at high speed, as well as complex control logic. The performance can also be limited by the single CDB. Although additional CDBs can be added, each CDB must interact with each reservation station, and the associative tag-matching hardware would have to be duplicated at each station for each CDB. In the 1990s, only high-end processors could take advantage of dynamic scheduling (and its extension to speculation); however, recently even processors designed for PMDs are using these techniques, and processors for high-end desktops and small servers have hundreds of buffers to support dynamic scheduling.

In Tomasulo's scheme, two different techniques are combined: the renaming of the architectural registers to a larger set of registers and the buffering of source operands from the register file. Source operand buffering resolves WAR hazards that arise when the operand is available in the registers. As we will see later, it is also possible to eliminate WAR hazards by the renaming of a register together with the buffering of a result until no outstanding references to the earlier version of the register remain. This approach will be used when we discuss hardware speculation.

Tomasulo's scheme was unused for many years after the 360/91, but was widely adopted in multiple-issue processors starting in the 1990s for several reasons:

1. Although Tomasulo's algorithm was designed before caches, the presence of caches, with the inherently unpredictable delays, has become one of the major motivations for dynamic scheduling. Out-of-order execution allows the processors to continue executing instructions while awaiting the completion of a cache miss, thus hiding all or part of the cache miss penalty.
2. As processors became more aggressive in their issue capability and designers were concerned with the performance of difficult-to-schedule code (such as most nonnumeric code), techniques such as register renaming, dynamic scheduling, and speculation became more important.
3. It can achieve high performance without requiring the compiler to target code to a specific pipeline structure, a valuable property in the era of shrink-wrapped mass market software.

3.6 Hardware-Based Speculation

As we try to exploit more instruction-level parallelism, maintaining control dependences becomes an increasing burden. Branch prediction reduces the direct stalls attributable to branches, but for a processor executing multiple instructions per clock, just predicting branches accurately may not be sufficient to generate the desired amount of instruction-level parallelism. A wide-issue processor may need to execute a branch every clock cycle to maintain maximum performance. Thus exploiting more parallelism requires that we overcome the limitation of control dependence.

Overcoming control dependence is done by speculating on the outcome of branches and executing the program as if our guesses are correct. This mechanism represents a subtle, but important, extension over branch prediction with dynamic scheduling. In particular, with speculation, we fetch, issue, and *execute* instructions, as if our branch predictions are always correct; dynamic scheduling only fetches and issues such instructions. Of course, we need mechanisms to handle the situation where the speculation is incorrect. Appendix H discusses a variety of mechanisms for supporting speculation by the compiler. In this section, we explore *hardware speculation,* which extends the ideas of dynamic scheduling.

Hardware-based speculation combines three key ideas: (1) dynamic branch prediction to choose which instructions to execute, (2) speculation to allow the execution of instructions before the control dependences are resolved (with the ability to undo the effects of an incorrectly speculated sequence), and (3) dynamic scheduling to deal with the scheduling of different combinations of basic blocks. (In comparison, dynamic scheduling without speculation only partially overlaps basic blocks because it requires that a branch be resolved before actually executing any instructions in the successor basic block.)

Hardware-based speculation follows the predicted flow of data values to choose when to execute instructions. This method of executing programs is essentially a *data flow execution*: Operations execute as soon as their operands are available.

To extend Tomasulo's algorithm to support speculation, we must separate the bypassing of results among instructions, which is needed to execute an instruction speculatively, from the actual completion of an instruction. By making this separation, we can allow an instruction to execute and to bypass its results to other instructions, without allowing the instruction to perform any updates that cannot be undone, until we know that the instruction is no longer speculative.

Using the bypassed value is like performing a speculative register read because we do not know whether the instruction providing the source register value is providing the correct result until the instruction is no longer speculative. When an instruction is no longer speculative, we allow it to update the register file or memory; we call this additional step in the instruction execution sequence *instruction commit*.

The key idea behind implementing speculation is to allow instructions to execute out of order but to force them to commit *in order* and to prevent any irrevocable action (such as updating state or taking an exception) until an instruction commits. Therefore, when we add speculation, we need to separate the process of completing execution from instruction commit, because instructions may finish execution considerably before they are ready to commit. Adding this commit phase to the instruction execution sequence requires an additional set of hardware buffers that hold the results of instructions that have finished execution but have not committed. This hardware buffer, which we call the *reorder buffer*, is also used to pass results among instructions that may be speculated.

The reorder buffer (ROB) provides additional registers in the same way as the reservation stations in Tomasulo's algorithm extend the register set. The ROB holds the result of an instruction between the time the operation associated with the instruction completes and the time the instruction commits. The ROB therefore is a source of operands for instructions, just as the reservation stations provide operands in Tomasulo's algorithm. The key difference is that in Tomasulo's algorithm, once an instruction writes its result, all subsequently issued instructions will find the result in the register file. With speculation, the register file is not updated until the instruction commits (and we know definitively that the instruction should execute); thus, the ROB supplies operands in the interval between completion of instruction execution and instruction commit. The ROB is similar to the store buffer in Tomasulo's algorithm, and we integrate the function of the store buffer into the ROB for simplicity.

Figure 3.15 shows the hardware structure of the processor including the ROB. Each entry in the ROB contains four fields: the instruction type, the destination field, the value field, and the ready field. The instruction type field indicates whether the instruction is a branch (and has no destination result), a store (which has a memory address destination), or a register operation (ALU operation or load, which has register destinations). The destination field supplies the register number (for loads

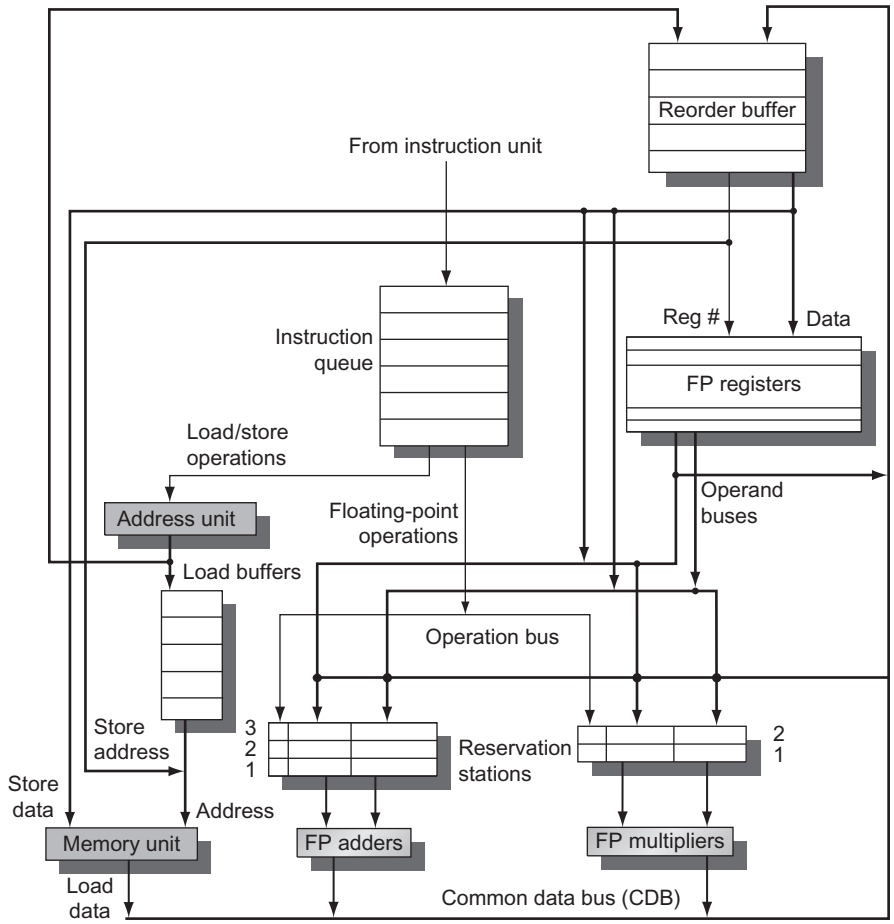


Figure 3.15 The basic structure of a FP unit using Tomasulo's algorithm and extended to handle speculation. Comparing this to Figure 3.10 on page 198, which implemented Tomasulo's algorithm, we can see that the major change is the addition of the ROB and the elimination of the store buffer, whose function is integrated into the ROB. This mechanism can be extended to allow multiple issues per clock by making the CDB wider to allow for multiple completions per clock.

and ALU operations) or the memory address (for stores) where the instruction result should be written. The value field is used to hold the value of the instruction result until the instruction commits. We will see an example of ROB entries shortly. Finally, the ready field indicates that the instruction has completed execution, and the value is ready.

The ROB subsumes the store buffers. Stores still execute in two steps, but the second step is performed by instruction commit. Although the renaming function of the reservation stations is replaced by the ROB, we still need a place to buffer operations (and operands) between the time they issue and the time they begin execution.

This function is still provided by the reservation stations. Because every instruction has a position in the ROB until it commits, we tag a result using the ROB entry number rather than using the reservation station number. This tagging requires that the ROB assigned for an instruction must be tracked in the reservation station. Later in this section, we will explore an alternative implementation that uses extra registers for renaming and a queue that replaces the ROB to decide when instructions can commit.

Here are the four steps involved in instruction execution:

1. *Issue*—Get an instruction from the instruction queue. Issue the instruction if there is an empty reservation station and an empty slot in the ROB; send the operands to the reservation station if they are available in either the registers or the ROB. Update the control entries to indicate the buffers are in use. The number of the ROB entry allocated for the result is also sent to the reservation station so that the number can be used to tag the result when it is placed on the CDB. If either all reservations are full or the ROB is full, then the instruction issue is stalled until both have available entries.
2. *Execute*—If one or more of the operands is not yet available, monitor the CDB while waiting for the register to be computed. This step checks for RAW hazards. When both operands are available at a reservation station, execute the operation. Instructions may take multiple clock cycles in this stage, and loads still require two steps in this stage. Stores only need the base register at this step, because execution for a store at this point is only effective address calculation.
3. *Write result*—When the result is available, write it on the CDB (with the ROB tag sent when the instruction issued) and from the CDB into the ROB, as well as to any reservation stations waiting for this result. Mark the reservation station as available. Special actions are required for store instructions. If the value to be stored is available, it is written into the Value field of the ROB entry for the store. If the value to be stored is not available yet, the CDB must be monitored until that value is broadcast, at which time the Value field of the ROB entry of the store is updated. For simplicity we assume that this occurs during the Write Result stage of a store; we discuss relaxing this requirement later.
4. *Commit*—This is the final stage of completing an instruction, after which only its result remains. (Some processors call this commit phase "completion" or "graduation.") There are three different sequences of actions at commit depending on whether the committing instruction is a branch with an incorrect prediction, a store, or any other instruction (normal commit). The normal commit case occurs when an instruction reaches the head of the ROB and its result is present in the buffer; at this point, the processor updates the register with the result and removes the instruction from the ROB. Committing a store is similar except that memory is updated rather than a result register. When a branch with incorrect prediction reaches the head of the ROB, it indicates that the speculation was wrong. The ROB is flushed and execution is restarted at the correct successor of the branch. If the branch was correctly predicted, the branch is finished.

Once an instruction commits, its entry in the ROB is reclaimed, and the register or memory destination is updated, eliminating the need for the ROB entry. If the ROB fills, we simply stop issuing instructions until an entry is made free. Now let's examine how this scheme would work with the same example we used for Tomasulo's algorithm.

Example Assume the same latencies for the floating-point functional units as in earlier examples: add is 2 clock cycles, multiply is 6 clock cycles, and divide is 12 clock cycles. Using the following code segment, the same one we used to generate Figure 3.12, show what the status tables look like when the `fmul.d` is ready to go to commit.

```
fld     f6,32(x2)
fld     f2,44(x3)
fmul.d  f0,f2,f4
fsub.d  f8,f2,f6
fdiv.d  f0,f0,f6
fadd.d  f6,f8,f2
```

Answer Figure 3.16 shows the result in the three tables. Notice that although the `fsub.d` instruction has completed execution, it does not commit until the `fmul.d` commits. The reservation stations and register status field contain the same basic information that they did for Tomasulo's algorithm (see page 200 for a description of those fields). The differences are that reservation station numbers are replaced with ROB entry numbers in the Qj and Qk fields, as well as in the register status fields, and we added the Dest field to the reservation stations. The Dest field designates the ROB entry that is the destination for the result produced by this reservation station entry.

The preceding example illustrates the key important difference between a processor with speculation and a processor with dynamic scheduling. Compare the content of Figure 3.16 with that of Figure 3.12 on page 184, which shows the same code sequence in operation on a processor with Tomasulo's algorithm. The key difference is that, in the preceding example, no instruction after the earliest uncompleted instruction (`fmul.d` in preceding example) is allowed to complete. In contrast, in Figure 3.12 the `fsub.d` and `fadd.d` instructions have also completed.

One implication of this difference is that the processor with the ROB can dynamically execute code while maintaining a precise interrupt model. For example, if the `fmul.d` instruction caused an interrupt, we could simply wait until it reached the head of the ROB and take the interrupt, flushing any other pending instructions from the ROB. Because instruction commit happens in order, this yields a precise exception.

By contrast, in the example using Tomasulo's algorithm, the `fsub.d` and `fadd.d` instructions could both complete before the `fmul.d` raised the exception. The result is that the registers `f8` and `f6` (destinations of the `fsub.d` and

	Reorder buffer				
Entry	**Busy**	**Instruction**	**State**	**Destination**	**Value**
1	No	`fld f6,32(x2)`	Commit	`f6`	`Mem[32 + Regs[x2]]`
2	No	`fld f2,44(x3)`	Commit	`f2`	`Mem[44 + Regs[x3]]`
3	Yes	`fmul.d f0,f2,f4`	Write result	`f0`	#2 × `Regs[f4]`
4	Yes	`fsub.d f8,f2,f6`	Write result	`f8`	#2 − #1
5	Yes	`fdiv.d f0,f0,f6`	Execute	`f0`	
6	Yes	`fadd.d f6,f8,f2`	Write result	`f6`	#4 + #2

	Reservation stations							
Name	**Busy**	**Op**	**Vj**	**Vk**	**Qj**	**Qk**	**Dest**	**A**
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	`fmul.d`	`Mem[44 + Regs[x3]]`	`Regs[f4]`			#3	
Mult2	Yes	`fdiv.d`		`Mem[32 + Regs[x2]]`	#3		#5	

	FP register status									
Field	**f0**	**f1**	**f2**	**f3**	**f4**	**f5**	**f6**	**f7**	**f8**	**f10**
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	…	Yes	Yes

Figure 3.16 At the time the `fmul.d` is ready to commit, only the two `fld` instructions have committed, although several others have completed execution. The `fmul.d` is at the head of the ROB, and the two `fld` instructions are there only to ease understanding. The `fsub.d` and `fadd.d` instructions will not commit until the `fmul.d` instruction commits, although the results of the instructions are available and can be used as sources for other instructions. The `fdiv.d` is in execution, but has not completed solely because of its longer latency than that of `fmul.d`. The Value column indicates the value being held; the format #X is used to refer to a value field of ROB entry X. Reorder buffers 1 and 2 are actually completed but are shown for informational purposes. We do not show the entries for the load/store queue, but these entries are kept in order.

`fadd.d` instructions) could be overwritten, in which case the interrupt would be imprecise.

Some users and architects have decided that imprecise floating-point exceptions are acceptable in high-performance processors because the program will likely terminate; see Appendix J for further discussion of this topic. Other types

of exceptions, such as page faults, are much more difficult to accommodate if they are imprecise because the program must transparently resume execution after handling such an exception.

The use of a ROB with in-order instruction commit provides precise exceptions, in addition to supporting speculative execution, as the next example shows.

Example Consider the code example used earlier for Tomasulo's algorithm and shown in Figure 3.14 in execution:

```
Loop:  fld     f0,0(x1)
       fmul.d  f4,f0,f2
       fsd     f4,0(x1)
       addi    x1,x1,−8
       bne     x1,x2,Loop    //branches if x1≠x2
```

Assume that we have issued all the instructions in the loop twice. Let's also assume that the `fld` and `fmul.d` from the first iteration have committed and all other instructions have completed execution. Normally, the store would wait in the ROB for both the effective address operand (`x1` in this example) and the value (`f4` in this example). Because we are only considering the floating-point pipeline, assume the effective address for the store is computed by the time the instruction is issued.

Answer Figure 3.17 shows the result in two tables.

Because neither the register values nor any memory values are actually written until an instruction commits, the processor can easily undo its speculative actions when a branch is found to be mispredicted. Suppose that the branch `bne` is not taken the first time in Figure 3.17. The instructions prior to the branch will simply commit when each reaches the head of the ROB; when the branch reaches the head of that buffer, the buffer is simply cleared and the processor begins fetching instructions from the other path.

In practice, processors that speculate try to recover as early as possible after a branch is mispredicted. This recovery can be done by clearing the ROB for all entries that appear after the mispredicted branch, allowing those that are before the branch in the ROB to continue, and restarting the fetch at the correct branch successor. In speculative processors, performance is more sensitive to the branch prediction because the impact of a misprediction will be higher. Thus all the aspects of handling branches—prediction accuracy, latency of misprediction detection, and misprediction recovery time—increase in importance.

Exceptions are handled by not recognizing the exception until it is ready to commit. If a speculated instruction raises an exception, the exception is recorded in the ROB. If a branch misprediction arises and the instruction should not have been executed, the exception is flushed along with the instruction when the

	Reorder buffer					
Entry	**Busy**	**Instruction**		**State**	**Destination**	**Value**
1	No	`fld`	`f0,0(x1)`	Commit	`f0`	`Mem[0 + Regs[x1]]`
2	No	`fmul.d`	`f4,f0,f2`	Commit	`f4`	#1 × `Regs[f2]`
3	Yes	`fsd`	`f4,0(x1)`	Write result	`0 + Regs[x1]`	#2
4	Yes	`addi`	`x1,x1,−8`	Write result	`x1`	`Regs[x1] − 8`
5	Yes	`bne`	`x1,x2,Loop`	Write result		
6	Yes	`fld`	`f0,0(x1)`	Write result	`f0`	`Mem[#4]`
7	Yes	`fmul.d`	`f4,f0,f2`	Write result	`f4`	#6 × `Regs[f2]`
8	Yes	`fsd`	`f4,0(x1)`	Write result	0 + #4	#7
9	Yes	`addi`	`x1,x1,−8`	Write result	`x1`	#4 − 8
10	Yes	`bne`	`x1,x2,Loop`	Write result		

	FP register status								
Field	**f0**	**f1**	**f2**	**f3**	**f4**	**F5**	**f6**	**F7**	**f8**
Reorder #	6								
Busy	Yes	No	No	No	Yes	No	No	…	No

Figure 3.17 Only the `fld` and `fmul.d` instructions have committed, although all the others have completed execution. Thus no reservation stations are busy and none are shown. The remaining instructions will be committed as quickly as possible. The first two reorder buffers are empty, but are shown for completeness.

ROB is cleared. If the instruction reaches the head of the ROB, then we know it is no longer speculative and the exception should really be taken. We can also try to handle exceptions as soon as they arise and all earlier branches are resolved, but this is more challenging in the case of exceptions than for branch mispredict and, because it occurs less frequently, not as critical.

Figure 3.18 shows the steps of execution for an instruction, as well as the conditions that must be satisfied to proceed to the step and the actions taken. We show the case where mispredicted branches are not resolved until commit. Although speculation seems like a simple addition to dynamic scheduling, a comparison of Figure 3.18 with the comparable figure for Tomasulo's algorithm in Figure 3.13 shows that speculation adds significant complications to the control. In addition, remember that branch mispredictions are somewhat more complex.

There is an important difference in how stores are handled in a speculative processor versus in Tomasulo's algorithm. In Tomasulo's algorithm, a store can update memory when it reaches Write Result (which ensures that the effective address has been calculated) and the data value to store is available. In a speculative processor, a store updates memory only when it reaches the head of

Status	Wait until	Action or bookkeeping
Issue all instructions	Reservation station (r) and ROB (b) both available	`if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no;`
FP operations and stores		`if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;};`
FP operations		`RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd;`
Loads		`RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt;`
Stores		`RS[r].A ← imm;`
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	`RS[r].A ← RS[r].Vj + RS[r].A;`
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from `Mem[RS[r].A]`
Store	(RS[r].Qj == 0) and store at queue head	`ROB[h].Address ← RS[r].Vj + RS[r].A;`
Write result all but store	Execution done at *r* and CDB available	`b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;`
Store	Execution done at *r* and (RS[r].Qk == 0)	`ROB[h].Value ← RS[r].Vk;`
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	`d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};`

Figure 3.18 Steps in the algorithm and what is required for each step. For the issuing instruction, `rd` is the destination, `rs` and `rt` are the sources, `r` is the reservation station allocated, `b` is the assigned ROB entry, and `h` is the head entry of the ROB. `RS` is the reservation station data structure. The value returned by a reservation station is called the `result`. `Register-Stat` is the register data structure, `Regs` represents the actual registers, and `ROB` is the reorder buffer data structure.

the ROB. This difference ensures that memory is not updated until an instruction is no longer speculative.

Figure 3.18 has one significant simplification for stores, which is unneeded in practice. Figure 3.18 requires stores to wait in the Write Result stage for the register source operand whose value is to be stored; the value is then moved from the Vk field of the store's reservation station to the Value field of the store's ROB entry. In reality, however, the value to be stored need not arrive until *just before* the store commits and can be placed directly into the store's ROB entry by the sourcing instruction. This is accomplished by having the hardware track when the source value to be stored is available in the store's ROB entry and searching the ROB on every instruction completion to look for dependent stores.

This addition is not complicated, but adding it has two effects: we would need to add a field to the ROB, and Figure 3.18, which is already in a small font, would be even longer! Although Figure 3.18 makes this simplification, in our examples, we will allow the store to pass through the Write Result stage and simply wait for the value to be ready when it commits.

Like Tomasulo's algorithm, we must avoid hazards through memory. WAW and WAR hazards through memory are eliminated with speculation because the actual updating of memory occurs in order, when a store is at the head of the ROB, so no earlier loads or stores can still be pending. RAW hazards through memory are maintained by two restrictions:

1. Not allowing a load to initiate the second step of its execution if any active ROB entry occupied by a store has a Destination field that matches the value of the A field of the load
2. Maintaining the program order for the computation of an effective address of a load with respect to all earlier stores

Together, these two restrictions ensure that any load that accesses a memory location written to by an earlier store cannot perform the memory access until the store has written the data. Some speculative processors will actually bypass the value from the store to the load directly when such a RAW hazard occurs. Another approach is to predict potential collisions using a form of value prediction; we consider this in Section 3.9.

Although this explanation of speculative execution has focused on floating point, the techniques easily extend to the integer registers and functional units. Indeed, because such programs tend to have code where the branch behavior is less predictable, speculation may be more useful in integer programs. Additionally, these techniques can be extended to work in a multiple-issue processor by allowing multiple instructions to issue and commit every clock. In fact, speculation is probably most interesting in such processors because less ambitious techniques can probably exploit sufficient ILP within basic blocks when assisted by a compiler.

3.7 Exploiting ILP Using Multiple Issue and Static Scheduling

The techniques of the preceding sections can be used to eliminate data, control stalls, and achieve an ideal CPI of one. To improve performance further, we want to decrease the CPI to less than one, but the CPI cannot be reduced below one if we issue only one instruction every clock cycle.

The goal of the *multiple-issue processors*, discussed in the next few sections, is to allow multiple instructions to issue in a clock cycle. Multiple-issue processors come in three major flavors:

1. Statically scheduled superscalar processors
2. VLIW (very long instruction word) processors
3. Dynamically scheduled superscalar processors

The two types of superscalar processors issue varying numbers of instructions per clock and use in-order execution if they are statically scheduled or out-of-order execution if they are dynamically scheduled.

VLIW processors, in contrast, issue a fixed number of instructions formatted either as one large instruction or as a fixed instruction packet with the parallelism among instructions explicitly indicated by the instruction. VLIW processors are inherently statically scheduled by the compiler. When Intel and HP created the IA-64 architecture, described in Appendix H, they also introduced the name EPIC (explicitly parallel instruction computer) for this architectural style.

Although statically scheduled superscalars issue a varying rather than a fixed number of instructions per clock, they are actually closer in concept to VLIWs because both approaches rely on the compiler to schedule code for the processor. Because of the diminishing advantages of a statically scheduled superscalar as the issue width grows, statically scheduled superscalars are used primarily for narrow issue widths, normally just two instructions. Beyond that width, most designers choose to implement either a VLIW or a dynamically scheduled superscalar. Because of the similarities in hardware and required compiler technology, we focus on VLIWs in this section, and we will see them again in Chapter 7. The insights of this section are easily extrapolated to a statically scheduled superscalar.

Figure 3.19 summarizes the basic approaches to multiple issue and their distinguishing characteristics and shows processors that use each approach.

The Basic VLIW Approach

VLIWs use multiple, independent functional units. Rather than attempting to issue multiple, independent instructions to the units, a VLIW packages the multiple operations into one very long instruction or requires that the instructions in the

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the Cortex-A53
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

Figure 3.19 The five primary approaches in use for multiple-issue processors and the primary characteristics that distinguish them. This chapter has focused on the hardware-intensive techniques, which are all some form of superscalar. Appendix H focuses on compiler-based approaches. The EPIC approach, as embodied in the IA-64 architecture, extends many of the concepts of the early VLIW approaches, providing a blend of static and dynamic approaches.

issue packet satisfy the same constraints. Because there is no fundamental difference in the two approaches, we will just assume that multiple operations are placed in one instruction, as in the original VLIW approach.

Because the advantage of a VLIW increases as the maximum issue rate grows, we focus on a wider issue processor. Indeed, for simple two-issue processors, the overhead of a superscalar is probably minimal. Many designers would probably argue that a four-issue processor has manageable overhead, but as we will see later in this chapter, the growth in overhead is a major factor limiting wider issue processors.

Let's consider a VLIW processor with instructions that contain five operations, including one integer operation (which could also be a branch), two floating-point operations, and two memory references. The instruction would have a set of fields for each functional unit—perhaps 16–24 bits per unit, yielding an instruction length of between 80 and 120 bits. By comparison, the Intel Itanium 1 and 2 contain six operations per instruction packet (i.e., they allow concurrent issue of two three-instruction bundles, as Appendix H describes).

To keep the functional units busy, there must be enough parallelism in a code sequence to fill the available operation slots. This parallelism is uncovered by unrolling loops and scheduling the code within the single larger loop body. If the unrolling generates straight-line code, then *local scheduling* techniques, which operate on a single basic block, can be used. If finding and exploiting the parallelism require scheduling code across branches, a substantially more complex *global*

scheduling algorithm must be used. Global scheduling algorithms are not only more complex in structure, but they also must deal with significantly more complicated trade-offs in optimization, because moving code across branches is expensive.

In Appendix H, we discuss *trace scheduling*, one of these global scheduling techniques developed specifically for VLIWs; we will also explore special hardware support that allows some conditional branches to be eliminated, extending the usefulness of local scheduling and enhancing the performance of global scheduling.

For now, we will rely on loop unrolling to generate long, straight-line code sequences so that we can use local scheduling to build up VLIW instructions and focus on how well these processors operate.

Example Suppose we have a VLIW that could issue two memory references, two FP operations, and one integer operation or branch in every clock cycle. Show an unrolled version of the loop `x[i] = x[i] + s` (see page 158 for the RISC-V code) for such a processor. Unroll as many times as necessary to eliminate any stalls.

Answer Figure 3.20 shows the code. The loop has been unrolled to make seven copies of the body, which eliminates all stalls (i.e., completely empty issue cycles), and runs in 9 cycles for the unrolled and scheduled loop. This code yields a running rate of seven results in 9 cycles, or 1.29 cycles per result, nearly twice as fast as the two-issue superscalar of Section 3.2 that used unrolled and scheduled code.

Memory reference 1	Memory reference 2	FP operation 1	FP operation 2	Integer operation/branch
`fld f0,0(x1)`	`fld f6,-8(x1)`			
`fld f10,-16(x1)`	`fld f14,-24(x1)`			
`fld f18,-32(x1)`	`fld f22,-40(x1)`	`fadd.d f4,f0,f2`	`fadd.d f8,f6,f2`	
`fld f26,-48(x1)`		`fadd.d f12,f0,f2`	`fadd.d f16,f14,f2`	
		`fadd.d f20,f18,f2`	`fadd.d f24,f22,f2`	
`fsd f4,0(x1)`	`fsd f8,-8(x1)`	`fadd.d f28,f26,f24`		
`fsd f12,-16(x1)`	`fsd f16,-24(x1)`			`addi x1,x1,-56`
`fsd f20,24(x1)`	`fsd f24,16(x1)`			
`fsd f28,8(x1)`				`bne x1,x2,Loop`

Figure 3.20 VLIW instructions that occupy the inner loop and replace the unrolled sequence. This code takes 9 cycles assuming correct branch prediction. The issue rate is 23 operations in 9 clock cycles, or 2.5 operations per cycle. The efficiency, the percentage of available slots that contained an operation, is about 60%. To achieve this issue rate requires a larger number of registers than RISC-V would normally use in this loop. The preceding VLIW code sequence requires at least eight FP registers, whereas the same code sequence for the base RISC-V processor can use as few as two FP registers or as many as five when unrolled and scheduled.

For the original VLIW model, there were both technical and logistical problems that made the approach less efficient. The technical problems were the increase in code size and the limitations of the lockstep operation. Two different elements combine to increase code size substantially for a VLIW. First, generating enough operations in a straight-line code fragment requires ambitiously unrolling loops (as in earlier examples), thereby increasing code size. Second, whenever instructions are not full, the unused functional units translate to wasted bits in the instruction encoding. In Appendix H, we examine software scheduling approaches, such as software pipelining, that can achieve the benefits of unrolling without as much code expansion.

To combat this code size increase, clever encodings are sometimes used. For example, there may be only one large immediate field for use by any functional unit. Another technique is to compress the instructions in main memory and expand them when they are read into the cache or are decoded. In Appendix H, we show other techniques, as well as document the significant code expansion seen in IA-64.

Early VLIWs operated in lockstep; there was no hazard-detection hardware at all. This structure dictated that a stall in any functional unit pipeline must cause the entire processor to stall because all the functional units had to be kept synchronized. Although a compiler might have been able to schedule the deterministic functional units to prevent stalls, predicting which data accesses would encounter a cache stall and scheduling them were very difficult to do. Thus caches needed to be blocking and causing *all* the functional units to stall. As the issue rate and number of memory references became large, this synchronization restriction became unacceptable. In more recent processors, the functional units operate more independently, and the compiler is used to avoid hazards at issue time, while hardware checks allow for unsynchronized execution once instructions are issued.

Binary code compatibility has also been a major logistical problem for general-purpose VLIWs or those that run third-party software. In a strict VLIW approach, the code sequence makes use of both the instruction set definition and the detailed pipeline structure, including both functional units and their latencies. Thus different numbers of functional units and unit latencies require different versions of the code. This requirement makes migrating between successive implementations, or between implementations with different issue widths, more difficult than it is for a superscalar design. Of course, obtaining improved performance from a new superscalar design may require recompilation. Nonetheless, the ability to run old binary files is a practical advantage for the superscalar approach. In the domain-specific architectures, which we examine in Chapter 7, this problem is not serious because applications are written specifically for an architectural configuration.

The EPIC approach, of which the IA-64 architecture is the primary example, provides solutions to many of the problems encountered in early general-purpose VLIW designs, including extensions for more aggressive software speculation and methods to overcome the limitation of hardware dependence while preserving binary compatibility.

The major challenge for all multiple-issue processors is to try to exploit large amounts of ILP. When the parallelism comes from unrolling simple loops in FP

programs, the original loop probably could have been run efficiently on a vector processor (described in the next chapter). It is not clear that a multiple-issue processor is preferred over a vector processor for such applications; the costs are similar, and the vector processor is typically the same speed or faster. The potential advantages of a multiple-issue processor versus a vector processor are the former's ability to extract some parallelism from less structured code and to easily cache all forms of data. For these reasons, multiple-issue approaches have become the primary method for taking advantage of instruction-level parallelism, and vectors have become primarily an extension to these processors.

3.8 Exploiting ILP Using Dynamic Scheduling, Multiple Issue, and Speculation

So far we have seen how the individual mechanisms of dynamic scheduling, multiple issue, and speculation work. In this section, we put all three together, which yields a microarchitecture quite similar to those in modern microprocessors. For simplicity we consider only an issue rate of two instructions per clock, but the concepts are no different from modern processors that issue three or more instructions per clock.

Let's assume we want to extend Tomasulo's algorithm to support multiple-issue superscalar pipeline with separate integer, load/store, and floating-point units (both FP multiply and FP add), each of which can initiate an operation on every clock. We do not want to issue instructions to the reservation stations out of order because this could lead to a violation of the program semantics. To gain the full advantage of dynamic scheduling, we will allow the pipeline to issue any combination of two instructions in a clock, using the scheduling hardware to actually assign operations to the integer and floating-point unit. Because the interaction of the integer and floating-point instructions is crucial, we also extend Tomasulo's scheme to deal with both the integer and floating-point functional units and registers, as well as incorporating speculative execution. As Figure 3.21 shows, the basic organization is similar to that of a processor with speculation with one issue per clock, except that the issue and completion logic must be enhanced to allow multiple instructions to be processed per clock.

Issuing multiple instructions per clock in a dynamically scheduled processor (with or without speculation) is very complex for the simple reason that the multiple instructions may depend on one another. Because of this, the tables must be updated for the instructions in parallel; otherwise, the tables will be incorrect or the dependence may be lost.

Two different approaches have been used to issue multiple instructions per clock in a dynamically scheduled processor, and both rely on the observation that the key is assigning a reservation station and updating the pipeline control tables. One approach is to run this step in half a clock cycle so that two instructions can be processed in one clock cycle; this approach cannot be easily extended to handle four instructions per clock, unfortunately.

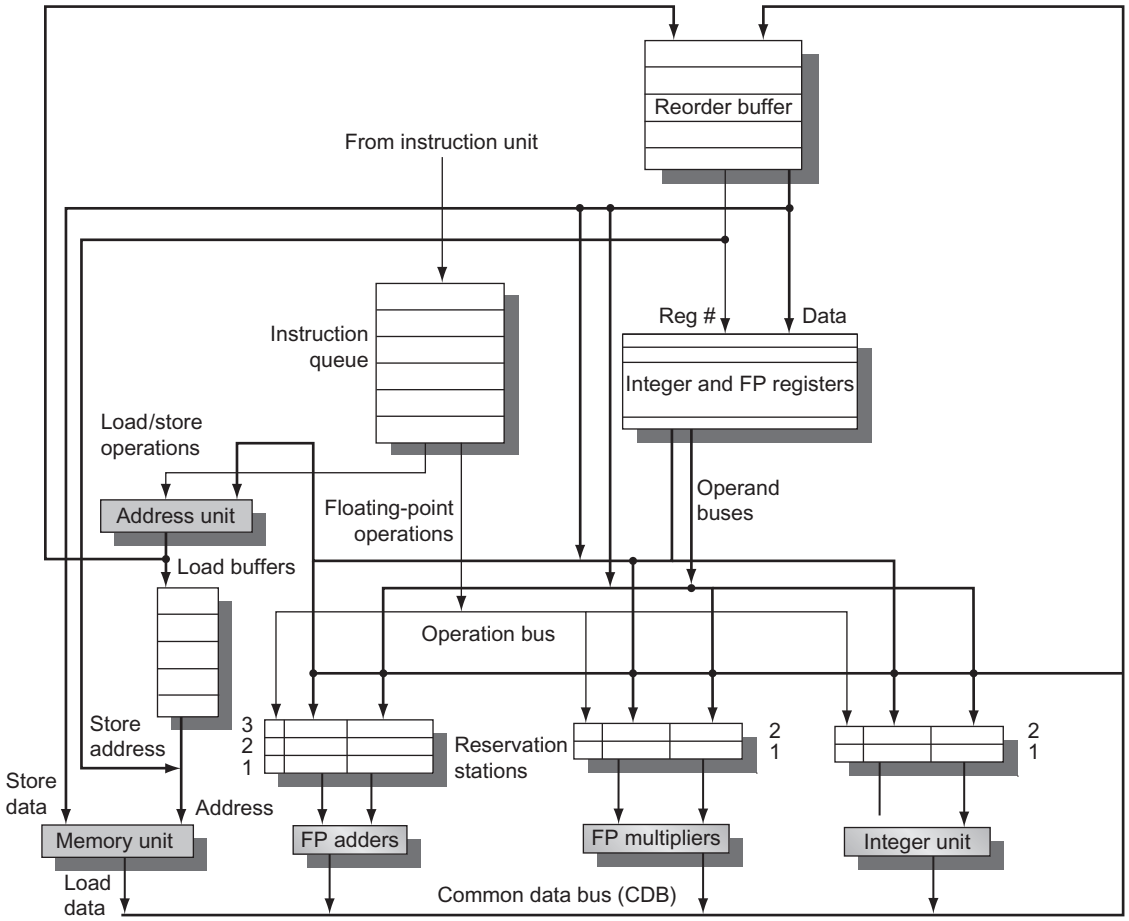


Figure 3.21 The basic organization of a multiple issue processor with speculation. In this case, the organization could allow a FP multiply, FP add, integer, and load/store to all issues simultaneously (assuming one issue per clock per functional unit). Note that several datapaths must be widened to support multiple issues: the CDB, the operand buses, and, critically, the instruction issue logic, which is not shown in this figure. The last is a difficult problem, as we discuss in the text.

A second alternative is to build the logic necessary to handle two or more instructions at once, including any possible dependences between the instructions. Modern superscalar processors that issue four or more instructions per clock may include both approaches: They both pipeline and widen the issue logic. A key observation is that we cannot simply pipeline away the problem. By making instruction issues take multiple clocks because new instructions are issuing every clock cycle, we must be able to assign the reservation station and to update the pipeline tables so that a dependent instruction issuing on the next clock can use the updated information.

This issue step is one of the most fundamental bottlenecks in dynamically scheduled superscalars. To illustrate the complexity of this process, Figure 3.22 shows the issue logic for one case: issuing a load followed by a dependent FP operation. The logic is based on that in Figure 3.18 on page 197, but represents only one case. In a modern superscalar, every possible combination of dependent instructions that is allowed to issue in the same clock cycle must be considered. Because the number of possibilities climbs as the square of the number of instructions that can be issued in a clock, the issue step is a likely bottleneck for attempts to go beyond four instructions per clock.

We can generalize the detail of Figure 3.22 to describe the basic strategy for updating the issue logic and the reservation tables in a dynamically scheduled superscalar with up to n issues per clock as follows:

1. Assign a reservation station and a reorder buffer for *every* instruction that *might* be issued in the next issue bundle. This assignment can be done before the instruction types are known simply by preallocating the reorder buffer entries sequentially to the instructions in the packet using n available reorder buffer entries and by ensuring that enough reservation stations are available to issue the whole bundle, independent of what it contains. By limiting the number of instructions of a given class (say, one FP, one integer, one load, one store), the necessary reservation stations can be preallocated. Should sufficient reservation stations not be available (such as when the next few instructions in the program are all of one instruction type), the bundle is broken, and only a subset of the instructions, in the original program order, is issued. The remainder of the instructions in the bundle can be placed in the next bundle for potential issue.
2. Analyze all the dependences among the instructions in the issue bundle.
3. If an instruction in the bundle depends on an earlier instruction in the bundle, use the assigned reorder buffer number to update the reservation table for the dependent instruction. Otherwise, use the existing reservation table and reorder buffer information to update the reservation table entries for the issuing instruction.

Of course, what makes the preceding very complicated is that it is all done in parallel in a single clock cycle!

At the back-end of the pipeline, we must be able to complete and commit multiple instructions per clock. These steps are somewhat easier than the issue problems because multiple instructions that can actually commit in the same clock cycle must have already dealt with and resolved any dependences. As we will see, designers have figured out how to handle this complexity: The Intel i7, which we examine in Section 3.12, uses essentially the scheme we have described for speculative multiple issue, including a large number of reservation stations, a reorder buffer, and a load and store buffer that is also used to handle nonblocking cache misses.

From a performance viewpoint, we can show how the concepts fit together with an example.

Action or bookkeeping	Comments

`if (RegisterStat[rs1].Busy)/*in-flight instr. writes rs*/`
`  {h ← RegisterStat[rs1].Reorder;`
`  if (ROB[h].Ready)/* Instr completed already */`
`  {RS[x1].Vj ← ROB[h].Value; RS[x1].Qj ← 0;}`
`  else {RS[x1].Qj ← h;} /* wait for instruction */`
`} else {RS[x1].Vj ← Regs[rs]; RS[x1].Qj ← 0;};`
`RS[x1].Busy ← yes; RS[x1].Dest ← b1;`
`ROB[b1].Instruction ← Load; ROB[b1].Dest ← rd1;`
`ROB[b1].Ready ← no;`
`RS[r].A ← imm1; RegisterStat[rt1].Reorder ← b1;`
`RegisterStat[rt1].Busy ← yes; ROB[b1].Dest ← rt1;`
	Updating the reservation tables for the load instruction, which has a single source operand. Because this is the first instruction in this issue bundle, it looks no different than what would normally happen for a load.

`RS[x2].Qj ← b1;} /* wait for load instruction */`	Because we know that the first operand of the FP operation is from the load, this step simply updates the reservation station to point to the load. Notice that the dependence must be analyzed on the fly and the ROB entries must be allocated during this issue step so that the reservation tables can be correctly updated.

`if (RegisterStat[rt2].Busy) /*in-flight instr writes rt*/`
`  {h ← RegisterStat[rt2].Reorder;`
`  if (ROB[h].Ready)/* Instr completed already */`
`  {RS[x2].Vk ← ROB[h].Value; RS[x2].Qk ← 0;}`
`  else {RS[x2].Qk ← h;} /* wait for instruction */`
`} else {RS[x2].Vk ← Regs[rt2]; RS[x2].Qk ← 0;};`
`RegisterStat[rd2].Reorder ← b2;`
`RegisterStat[rd2].Busy ← yes;`
`ROB[b2].Dest ← rd2;`
	Because we assumed that the second operand of the FP instruction was from a prior issue bundle, this step looks like it would in the single-issue case. Of course, if this instruction were dependent on something in the same issue bundle, the tables would need to be updated using the assigned reservation buffer.

`RS[x2].Busy ← yes; RS[x2].Dest ← b2;`
`ROB[b2].Instruction ← FP operation; ROB[b2].Dest ← rd2;`
`ROB[b2].Ready ← no;`
	This section simply updates the tables for the FP operation and is independent of the load. Of course, if further instructions in this issue bundle depended on the FP operation (as could happen with a four-issue superscalar), the updates to the reservation tables for those instructions would be effected by this instruction.

Figure 3.22 The issue steps for a pair of dependent instructions (called 1 and 2), where instruction 1 is FP load and instruction 2 is an FP operation whose first operand is the result of the load instruction; `x1` **and** `x2` **are the assigned reservation stations for the instructions; and** `b1` **and** `b2` **are the assigned reorder buffer entries.** For the issuing instructions, `rd1` and `rd2` are the destinations; `rs1`, `rs2`, and `rt2` are the sources (the load has only one source); `x1` and `x2` are the reservation stations allocated; and `b1` and `b2` are the assigned ROB entries. `RS` is the reservation station data structure. `RegisterStat` is the register data structure, `Regs` represents the actual registers, and `ROB` is the reorder buffer data structure. Notice that we need to have assigned reorder buffer entries for this logic to operate properly, and recall that all these updates happen in a single clock cycle in parallel, not sequentially.

Example Consider the execution of the following loop, which increments each element of an integer array, on a two-issue processor, once without speculation and once with speculation:

```
Loop:  ld    x2,0(x1)      //x2=array element
       addi  x2,x2,1       //increment x2
       sd    x2,0(x1)      //store result
       addi  x1,x1,8       //increment pointer
       bne   x2,x3,Loop    //branch if not last
```

Assume that there are separate integer functional units for effective address calculation, for ALU operations, and for branch condition evaluation. Create a table for the first three iterations of this loop for both processors. Assume that up to two instructions of any type can commit per clock.

Answer Figures 3.23 and 3.24 show the performance for a two-issue, dynamically scheduled processor, without and with speculation. In this case, where a branch

Iteration number	Instructions	Issues at clock cycle number	Executes at clock cycle number	Memory access at clock cycle number	Write CDB at clock cycle number	Comment
1	`ld x2,0(x1)`	1	2	3	4	First issue
1	`addi x2,x2,1`	1	5		6	Wait for `ld`
1	`sd x2,0(x1)`	2	3	7		Wait for `addi`
1	`addi x1,x1,8`	2	3		4	Execute directly
1	`bne x2,x3,Loop`	3	7			Wait for `addi`
2	`ld x2,0(x1)`	4	8	9	10	Wait for `bne`
2	`addi x2,x2,1`	4	11		12	Wait for `ld`
2	`sd x2,0(x1)`	5	9	13		Wait for `addi`
2	`addi x1,x1,8`	5	8		9	Wait for `bne`
2	`bne x2,x3,Loop`	6	13			Wait for `addi`
3	`ld x2,0(x1)`	7	14	15	16	Wait for `bne`
3	`addi x2,x2,1`	7	17		18	Wait for `ld`
3	`sd x2,0(x1)`	8	15	19		Wait for `addi`
3	`addi x1,x1,8`	8	14		15	Wait for `bne`
3	`bne x2,x3,Loop`	9	19			Wait for `addi`

Figure 3.23 The time of issue, execution, and writing result for a dual-issue version of our pipeline *without* speculation. Note that the `ld` following the `bne` cannot start execution earlier because it must wait until the branch outcome is determined. This type of program, with data-dependent branches that cannot be resolved earlier, shows the strength of speculation. Separate functional units for address calculation, ALU operations, and branch-condition evaluation allow multiple instructions to execute in the same cycle. Figure 3.24 shows this example with speculation.

Iteration number	Instructions	Issues at clock number	Executes at clock number	Read access at clock number	Write CDB at clock number	Commits at clock number	Comment
1	`ld x2,0(x1)`	1	2	3	4	5	First issue
1	`addi x2,x2,1`	1	5		6	7	Wait for `ld`
1	`sd x2,0(x1)`	2	3			7	Wait for `addi`
1	`addi x1,x1,8`	2	3		4	8	Commit in order
1	`bne x2,x3,Loop`	3	7			8	Wait for `addi`
2	`ld x2,0(x1)`	4	5	6	7	9	No execute delay
2	`addi x2,x2,1`	4	8		9	10	Wait for `ld`
2	`sd x2,0(x1)`	5	6			10	Wait for `addi`
2	`addi x1,x1,8`	5	6		7	11	Commit in order
2	`bne x2,x3,Loop`	6	10			11	Wait for `addi`
3	`ld x2,0(x1)`	7	8	9	10	12	Earliest possible
3	`addi x2,x2,1`	7	11		12	13	Wait for `ld`
3	`sd x2,0(x1)`	8	9			13	Wait for `addi`
3	`addi x1,x1,8`	8	9		10	14	Executes earlier
3	`bne x2,x3,Loop`	9	13			14	Wait for `addi`

Figure 3.24 The time of issue, execution, and writing result for a dual-issue version of our pipeline *with* speculation. Note that the `ld` following the `bne` can start execution early because it is speculative.

can be a critical performance limiter, speculation helps significantly. The third branch in the speculative processor executes in clock cycle 13, whereas it executes in clock cycle 19 on the nonspeculative pipeline. Because the completion rate on the nonspeculative pipeline is falling behind the issue rate rapidly, the nonspeculative pipeline will stall when a few more iterations are issued. The performance of the nonspeculative processor could be improved by allowing load instructions to complete effective address calculation before a branch is decided, but unless speculative memory accesses are allowed, this improvement will gain only 1 clock per iteration.

This example clearly shows how speculation can be advantageous when there are data-dependent branches, which otherwise would limit performance. This advantage depends, however, on accurate branch prediction. Incorrect speculation does not improve performance; in fact, it typically harms performance and, as we shall see, dramatically lowers energy efficiency.

3.9 Advanced Techniques for Instruction Delivery and Speculation

In a high-performance pipeline, especially one with multiple issues, predicting branches well is not enough; we actually have to be able to deliver a high-bandwidth instruction stream. In recent multiple-issue processors, this has meant delivering 4–8 instructions every clock cycle. We look at methods for increasing instruction delivery bandwidth first. We then turn to a set of key issues in implementing advanced speculation techniques, including the use of register renaming versus reorder buffers, the aggressiveness of speculation, and a technique called *value prediction*, which attempts to predict the result of a computation and which could further enhance ILP.

Increasing Instruction Fetch Bandwidth

A multiple-issue processor will require that the average number of instructions fetched every clock cycle be at least as large as the average throughput. Of course, fetching these instructions requires wide enough paths to the instruction cache, but the most difficult aspect is handling branches. In this section, we look at two methods for dealing with branches and then discuss how modern processors integrate the instruction prediction and prefetch functions.

Branch-Target Buffers

To reduce the branch penalty for our simple five-stage pipeline, as well as for deeper pipelines, we must know whether the as-yet-undecoded instruction is a branch and, if so, what the next program counter (PC) should be. If the instruction is a branch and we know what the next PC should be, we can have a branch penalty of zero. A branch-prediction cache that stores the predicted address for the next instruction after a branch is called a *branch-target buffer* or *branch-target cache.* Figure 3.25 shows a branch-target buffer.

Because a branch-target buffer predicts the next instruction address and will send it out *before* decoding the instruction, we *must* know whether the fetched instruction is predicted as a taken branch. If the PC of the fetched instruction matches an address in the prediction buffer, then the corresponding predicted PC is used as the next PC. The hardware for this branch-target buffer is essentially identical to the hardware for a cache.

If a matching entry is found in the branch-target buffer, fetching begins immediately at the predicted PC. Note that unlike a branch-prediction buffer, the predictive entry must be matched to this instruction because the predicted PC will be sent

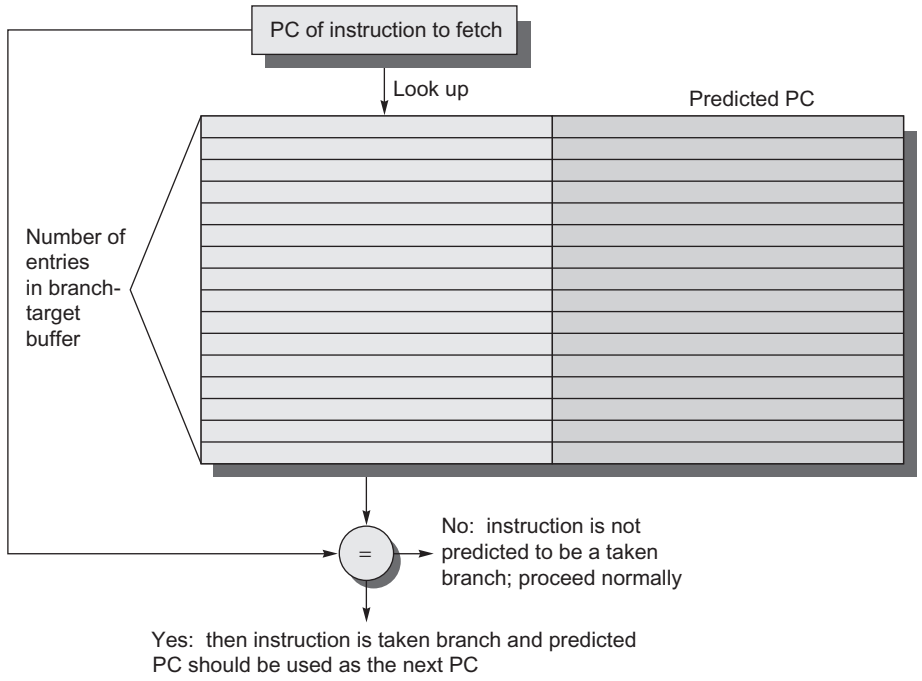


Figure 3.25 A branch-target buffer. The PC of the instruction being fetched is matched against a set of instruction addresses stored in the first column; these represent the addresses of known branches. If the PC matches one of these entries, then the instruction being fetched is a taken branch, and the second field, predicted PC, contains the prediction for the next PC after the branch. Fetching begins immediately at that address. The third field, which is optional, may be used for extra prediction state bits.

out before it is known whether this instruction is even a branch. If the processor did not check whether the entry matched this PC, then the wrong PC would be sent out for instructions that were not branches, resulting in worse performance. We need to store only the predicted-taken branches in the branch-target buffer because an untaken branch should simply fetch the next sequential instruction, as if it were not a branch.

Figure 3.26 shows the steps when using a branch-target buffer for a simple five-stage pipeline. As we can see in this figure, there will be no branch delay if a branch-prediction entry is found in the buffer and the prediction is correct. Otherwise, there will be a penalty of at least two clock cycles. Dealing with the mispredictions and misses is a significant challenge because we typically will have to halt instruction fetch while we rewrite the buffer entry. Thus we want to make this process fast to minimize the penalty.

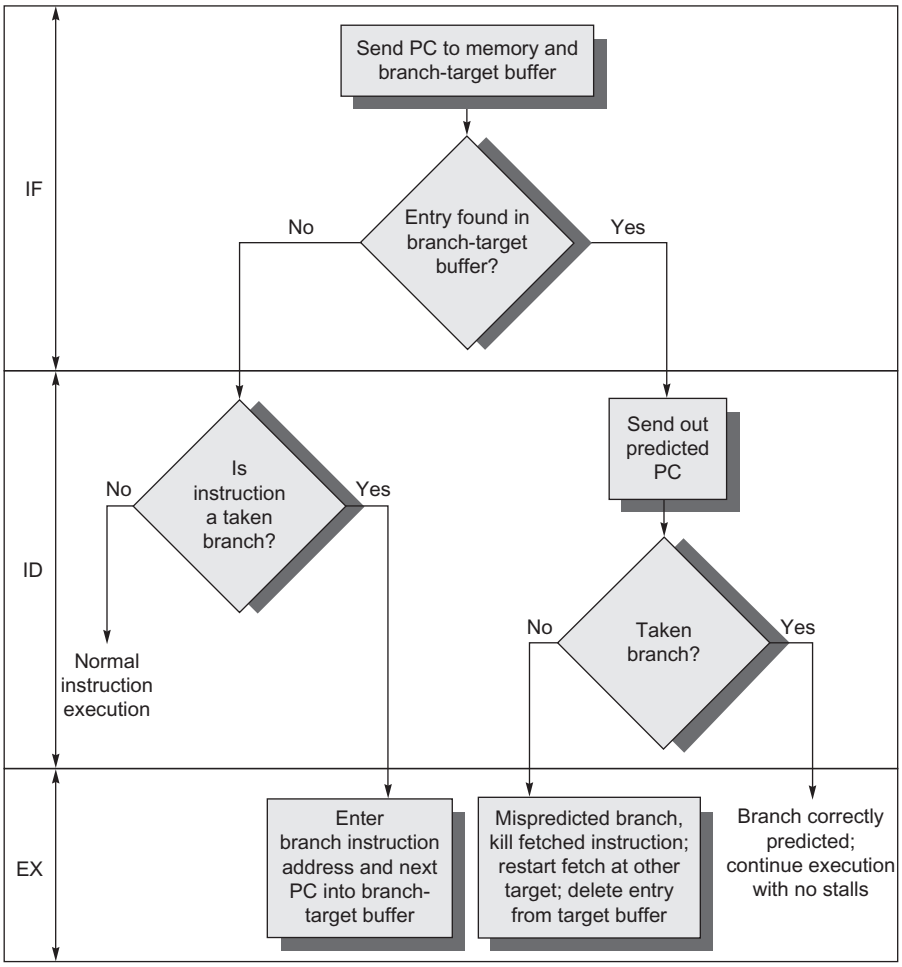


Figure 3.26 The steps involved in handling an instruction with a branch-target buffer.

To evaluate how well a branch-target buffer works, we first must determine the penalties in all possible cases. Figure 3.27 contains this information for a simple five-stage pipeline.

Example Determine the total branch penalty for a branch-target buffer assuming the penalty cycles for individual mispredictions in Figure 3.27. Make the following assumptions about the prediction accuracy and hit rate:

- Prediction accuracy is 90% (for instructions in the buffer).
- Hit rate in the buffer is 90% (for branches predicted taken).

Answer We compute the penalty by looking at the probability of two events: the branch is predicted taken but ends up being not taken, and the branch is taken but is not found in the buffer. Both carry a penalty of two cycles.

$$\begin{aligned}\text{Probability (branch in buffer, but actually not taken)} &= \text{Percent buffer hit rate} \\ &\quad \times \text{Percent incorrect predictions} \\ &= 90\% \times 10\% = 0.09 \\ \text{Probability (branch not in buffer, but actually taken)} &= 10\% \\ \text{Branch penalty} &= (0.09 + 0.10) \times 2 \\ \text{Branch penalty} &= 0.38\end{aligned}$$

The improvement from dynamic branch prediction will grow as the pipeline length, and thus the branch delay grows; in addition, better predictors will yield a greater performance advantage. Modern high-performance processors have branch misprediction delays on the order of 15 clock cycles; clearly, accurate prediction is critical!

One variation on the branch-target buffer is to store one or more *target instructions* instead of, or in addition to, the predicted *target address*. This variation has two potential advantages. First, it allows the branch-target buffer access to take longer than the time between successive instruction fetches, possibly allowing a larger branch-target buffer. Second, buffering the actual target instructions allows us to perform an optimization called *branch folding*. Branch folding can be used to obtain 0-cycle unconditional branches and sometimes 0-cycle conditional branches. As we will see, the Cortex A-53 uses a single-entry branch target cache that stores the predicted target instructions.

Consider a branch-target buffer that buffers instructions from the predicted path and is being accessed with the address of an unconditional branch. The only function of the unconditional branch is to change the PC. Thus, when the branch-target buffer signals a hit and indicates that the branch is

Instruction in buffer	Prediction	Actual branch	Penalty cycles
Yes	Taken	Taken	0
Yes	Taken	Not taken	2
No		Taken	2
No		Not taken	0

Figure 3.27 Penalties for all possible combinations of whether the branch is in the buffer and what it actually does, assuming we store only taken branches in the buffer. There is no branch penalty if everything is correctly predicted and the branch is found in the target buffer. If the branch is not correctly predicted, the penalty is equal to 1 clock cycle to update the buffer with the correct information (during which an instruction cannot be fetched) and 1 clock cycle, if needed, to restart fetching the next correct instruction for the branch. If the branch is not found and taken, a 2-cycle penalty is encountered, during which time the buffer is updated.

unconditional, the pipeline can simply substitute the instruction from the branch-target buffer in place of the instruction that is returned from the cache (which is the unconditional branch). If the processor is issuing multiple instructions per cycle, then the buffer will need to supply multiple instructions to obtain the maximum benefit. In some cases, it may be possible to eliminate the cost of a conditional branch.

Specialized Branch Predictors: Predicting Procedure Returns, Indirect Jumps, and Loop Branches

As we try to increase the opportunity and accuracy of speculation, we face the challenge of predicting indirect jumps, that is, jumps whose destination address varies at runtime. High-level language programs will generate such jumps for indirect procedure calls, select or case statements, and FORTRAN-computed gotos, although many indirect jumps simply come from procedure returns. For example, for the SPEC95 benchmarks, procedure returns account for more than 15% of the branches and the vast majority of the indirect jumps on average. For object-oriented languages such as C++ and Java, procedure returns are even more frequent. Thus focusing on procedure returns seems appropriate.

Though procedure returns can be predicted with a branch-target buffer, the accuracy of such a prediction technique can be low if the procedure is called from multiple sites and the calls from one site are not clustered in time. For example, in SPEC CPU95, an aggressive branch predictor achieves an accuracy of less than 60% for such return branches. To overcome this problem, some designs use a small buffer of return addresses operating as a stack. This structure caches the most recent return addresses, pushing a return address on the stack at a call and popping one off at a return. If the cache is sufficiently large (i.e., as large as the maximum call depth), it will predict the returns perfectly. Figure 3.28 shows the performance of such a return buffer with 0–16 elements for a number of the SPEC CPU95 benchmarks. We will use a similar return predictor when we examine the studies of ILP in Section 3.10. Both the Intel Core processors and the AMD Phenom processors have return address predictors.

In large server applications, indirect jumps also occur for various function calls and control transfers. Predicting the targets of such branches is not as simple as in a procedure return. Some processors have opted to add specialized predictors for all indirect jumps, whereas others rely on a branch target buffer.

Although a simple predictor like gshare does a good job of predicting many conditional branches, it is not tailored to predicting loop branches, especially for long running loops. As we observed earlier, the Intel Core i7 920 used a specialized loop branch predictor. With the emergence of tagged hybrid predictors, which are as good at predicting loop branches, some recent designers have opted to put the resources into larger tagged hybrid predictors rather than a separate loop branch predictor.

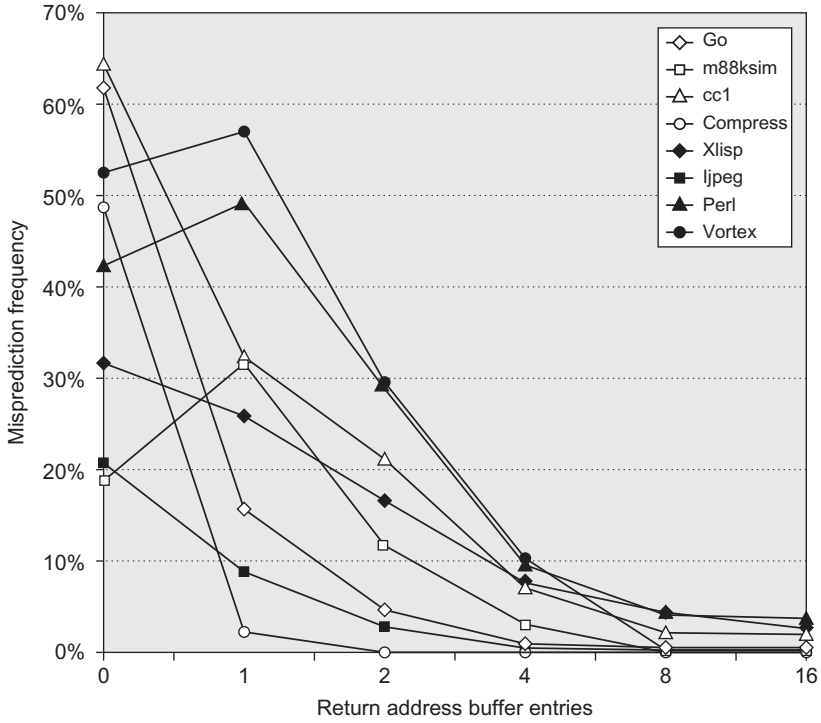


Figure 3.28 Prediction accuracy for a return address buffer operated as a stack on a number of SPEC CPU95 benchmarks. The accuracy is the fraction of return addresses predicted correctly. A buffer of 0 entries implies that the standard branch prediction is used. Because call depths are typically not large, with some exceptions, a modest buffer works well. These data come from Skadron et al. (1999) and use a fix-up mechanism to prevent corruption of the cached return addresses.

Integrated Instruction Fetch Units

To meet the demands of multiple-issue processors, many recent designers have chosen to implement an integrated instruction fetch unit as a separate autonomous unit that feeds instructions to the rest of the pipeline. Essentially, this amounts to recognizing that characterizing instruction fetch as a simple single pipe stage given the complexities of multiple issue is no longer valid.

Instead, recent designs have used an integrated instruction fetch unit that integrates several functions:

1. *Integrated branch prediction*—The branch predictor becomes part of the instruction fetch unit and is constantly predicting branches, so as to drive the fetch pipeline.

2. *Instruction prefetch*—To deliver multiple instructions per clock, the instruction fetch unit will likely need to fetch ahead. The unit autonomously manages the prefetching of instructions (see Chapter 2 for a discussion of techniques for doing this), integrating it with branch prediction.
3. *Instruction memory access and buffering*—When fetching multiple instructions per cycle, a variety of complexities are encountered, including the difficulty that fetching multiple instructions may require accessing multiple cache lines. The instruction fetch unit encapsulates this complexity, using prefetch to try to hide the cost of crossing cache blocks. The instruction fetch unit also provides buffering, essentially acting as an on-demand unit to provide instructions to the issue stage as needed and in the quantity needed.

Virtually all high-end processors now use a separate instruction fetch unit connected to the rest of the pipeline by a buffer containing pending instructions.

Speculation: Implementation Issues and Extensions

In this section, we explore five issues that involve the design trade-offs and challenges in multiple-issue and speculation, starting with the use of register renaming, the approach that is sometimes used instead of a reorder buffer. We then discuss one important possible extension to speculation on control flow: an idea called value prediction.

Speculation Support: Register Renaming Versus Reorder Buffers

One alternative to the use of a reorder buffer (ROB) is the explicit use of a larger physical set of registers combined with register renaming. This approach builds on the concept of renaming used in Tomasulo's algorithm and extends it. In Tomasulo's algorithm, the values of the *architecturally visible registers* (x0, . . . r31 and f0, . . . f31) are contained, at any point in execution, in some combination of the register set and the reservation stations. With the addition of speculation, register values may also temporarily reside in the ROB. In either case, if the processor does not issue new instructions for a period of time, all existing instructions will commit, and the register values will appear in the register file, which directly corresponds to the architecturally visible registers.

In the register-renaming approach, an extended set of physical registers is used to hold both the architecturally visible registers as well as temporary values. Thus the extended registers replace most of the function of the ROB and the reservation stations; only a queue to ensure that instructions complete in order is needed. During instruction issue, a renaming process maps the names of architectural registers to physical register numbers in the extended register set, allocating a new unused register for the destination. WAW and WAR hazards are avoided by renaming of the destination register, and speculation recovery is handled

because a physical register holding an instruction destination does not become the architectural register until the instruction commits.

The *renaming map* is a simple data structure that supplies the physical register number of the register that currently corresponds to the specified architectural register, a function performed by the register status table in Tomasulo's algorithm. When an instruction commits, the renaming table is permanently updated to indicate that a physical register corresponds to the actual architectural register, thus effectively finalizing the update to the processor state. Although an ROB is not necessary with register renaming, the hardware must still track instructions in a queue-like structure and update the renaming table in strict order.

An advantage of the renaming approach versus the ROB approach is that instruction commit is slightly simplified because it requires only two simple actions: (1) record that the mapping between an architectural register number and physical register number is no longer speculative, and (2) free up any physical registers being used to hold the "older" value of the architectural register. In a design with reservation stations, a station is freed up when the instruction using it completes execution, and a ROB entry is freed up when the corresponding instruction commits.

With register renaming, deallocating registers is more complex because before we free up a physical register, we must know that it no longer corresponds to an architectural register and that no further uses of the physical register are outstanding. A physical register corresponds to an architectural register until the architectural register is rewritten, causing the renaming table to point elsewhere. That is, if no renaming entry points to a particular physical register, then it no longer corresponds to an architectural register. There may, however, still be outstanding uses of the physical register. The processor can determine whether this is the case by examining the source register specifiers of all instructions in the functional unit queues. If a given physical register does not appear as a source and it is not designated as an architectural register, it may be reclaimed and reallocated.

Alternatively, the processor can simply wait until another instruction that writes the same architectural register commits. At that point, there can be no further uses of the older value outstanding. Although this method may tie up a physical register slightly longer than necessary, it is easy to implement and is used in most recent superscalars.

One question you may be asking is how do we ever know which registers are the architectural registers if they are constantly changing? Most of the time when the program is executing, it does not matter. There are clearly cases, however, where another process, such as the operating system, must be able to know exactly where the contents of a certain architectural register reside. To understand how this capability is provided, assume the processor does not issue instructions for some period of time. Eventually all instructions in the pipeline will commit, and the mapping between the architecturally visible registers and physical registers will become stable. At that point, a subset of the physical registers contains the architecturally visible registers, and the value of any physical register not associated with an architectural register is unneeded. It is then easy to move the architectural

registers to a fixed subset of physical registers so that the values can be communicated to another process.

Both register renaming and reorder buffers continue to be used in high-end processors, which now feature the ability to have as many as 100 or more instructions (including loads and stores waiting on the cache) in flight. Whether renaming or a reorder buffer is used, the key complexity bottleneck for a dynamically scheduled superscalar remains issuing bundles of instructions with dependences within the bundle. In particular, dependent instructions in an issue bundle must be issued with the assigned virtual registers of the instructions on which they depend. A strategy for instruction issue with register renaming similar to that used for multiple issue with reorder buffers (see page 205) can be deployed, as follows:

1. The issue logic reserves enough physical registers for the entire issue bundle (say, four registers for a four-instruction bundle with at most one register result per instruction).
2. The issue logic determines what dependences exist within the bundle. If a dependence does not exist within the bundle, the register renaming structure is used to determine the physical register that holds, or will hold, the result on which instruction depends. When no dependence exists within the bundle, the result is from an earlier issue bundle, and the register renaming table will have the correct register number.
3. If an instruction depends on an instruction that is earlier in the bundle, then the pre-reserved physical register in which the result will be placed is used to update the information for the issuing instruction.

Note that just as in the reorder buffer case, the issue logic must both determine dependences within the bundle and update the renaming tables in a single clock, and as before, the complexity of doing this for a larger number of instructions per clock becomes a chief limitation in the issue width.

The Challenge of More Issues per Clock

Without speculation, there is little motivation to try to increase the issue rate beyond two, three, or possibly four issues per clock because resolving branches would limit the average issue rate to a smaller number. Once a processor includes accurate branch prediction and speculation, we might conclude that increasing the issue rate would be attractive. Duplicating the functional units is straightforward assuming silicon capacity and power; the real complications arise in the issue step and correspondingly in the commit step. The Commit step is the dual of the issue step, and the requirements are similar, so let's take a look at what has to happen for a six-issue processor using register renaming.

Figure 3.29 shows a six-instruction code sequence and what the issue step must do. Remember that this must all occur in a single clock cycle, if the processor is to maintain a peak rate of six issues per clock! All the dependences must be detected,

Instr. #	Instruction	Physical register assigned or destination	Instruction with physical register numbers	Rename map changes
1	`add x1,x2,x3`	p32	`add p32,p2,p3`	**x1-> p32**
2	`sub x1,x1,x2`	p33	`sub p33,p32,p2`	**x1->p33**
3	`add x2,x1,x2`	p34	`add p34,p33,x2`	**x2->p34**
4	`sub x1,x3,x2`	p35	`sub p35,p3,p34`	**x1->p35**
5	`add x1,x1,x2`	p36	`add p36,p35,p34`	**x1->p36**
6	`sub x1,x3,x1`	p37	`sub p37,p3,p36`	**x1->p37**

Figure 3.29 An example of six instructions to be issued in the same clock cycle and what has to happen. The instructions are shown in program order: 1–6; they are, however, issued in 1 clock cycle! The notation `pi` is used to refer to a physical register; the contents of that register at any point is determined by the renaming map. For simplicity, we assume that the physical registers holding the architectural registers `x1`, `x2`, and `x3` are initially `p1`, `p2`, and `p3` (they could be any physical register). The instructions are issued with physical register numbers, as shown in column four. The rename map, which appears in the last column, shows how the map would change if the instructions were issued sequentially. The difficulty is that all this renaming and replacement of architectural registers by physical renaming registers happens effectively in 1 cycle, not sequentially. The issue logic must find all the dependences and "rewrite" the instruction in parallel.

the physical registers must be assigned, and the instructions must be rewritten using the physical register numbers: in one clock. This example makes it clear why issue rates have grown from 3–4 to only 4–8 in the past 20 years. The complexity of the analysis required during the issue cycle grows as the square of the issue width, and a new processor is typically targeted to have a higher clock rate than in the last generation! Because register renaming and the reorder buffer approaches are duals, the same complexities arise independent of the implementation scheme.

How Much to Speculate

One of the significant advantages of speculation is its ability to uncover events that would otherwise stall the pipeline early, such as cache misses. This potential advantage, however, comes with a significant potential disadvantage. Speculation is not free. It takes time and energy, and the recovery of incorrect speculation further reduces performance. In addition, to support the higher instruction execution rate needed to benefit from speculation, the processor must have additional resources, which take silicon area and power. Finally, if speculation causes an exceptional event to occur, such as a cache or translation lookaside buffer (TLB) miss, the potential for significant performance loss increases, if that event would not have occurred without speculation.

To maintain most of the advantage while minimizing the disadvantages, most pipelines with speculation will allow only low-cost exceptional events (such as a first-level cache miss) to be handled in speculative mode. If an expensive exceptional event occurs, such as a second-level cache miss or a TLB miss, the processor will wait

until the instruction causing the event is no longer speculative before handling the event. Although this may slightly degrade the performance of some programs, it avoids significant performance losses in others, especially those that suffer from a high frequency of such events coupled with less-than-excellent branch prediction.

In the 1990s the potential downsides of speculation were less obvious. As processors have evolved, the real costs of speculation have become more apparent, and the limitations of wider issue and speculation have been obvious. We return to this issue shortly.

Speculating Through Multiple Branches

In the examples we have considered in this chapter, it has been possible to resolve a branch before having to speculate on another. Three different situations can benefit from speculating on multiple branches simultaneously: (1) a very high branch frequency, (2) significant clustering of branches, and (3) long delays in functional units. In the first two cases, achieving high performance may mean that multiple branches are speculated, and it may even mean handling more than one branch per clock. Database programs and other less structured integer computations, often exhibit these properties, making speculation on multiple branches important. Likewise, long delays in functional units can raise the importance of speculating on multiple branches as a way to avoid stalls from the longer pipeline delays.

Speculating on multiple branches slightly complicates the process of speculation recovery but is straightforward otherwise. As of 2017, no processor has yet combined full speculation with resolving multiple branches per cycle, and it is unlikely that the costs of doing so would be justified in terms of performance versus complexity and power.

Speculation and the Challenge of Energy Efficiency

What is the impact of speculation on energy efficiency? At first glance, one might argue that using speculation always decreases energy efficiency because whenever speculation is wrong, it consumes excess energy in two ways:

1. Instructions that are speculated and whose results are not needed generate excess work for the processor, wasting energy.
2. Undoing the speculation and restoring the state of the processor to continue execution at the appropriate address consumes additional energy that would not be needed without speculation.

Certainly, speculation will raise the power consumption, and if we could control speculation, it would be possible to measure the cost (or at least the dynamic power cost). But, if speculation lowers the execution time by more than it increases the average power consumption, then the total energy consumed may be less.

Thus, to understand the impact of speculation on energy efficiency, we need to look at how often speculation is leading to unnecessary work. If a significant number of unneeded instructions is executed, it is unlikely that speculation will

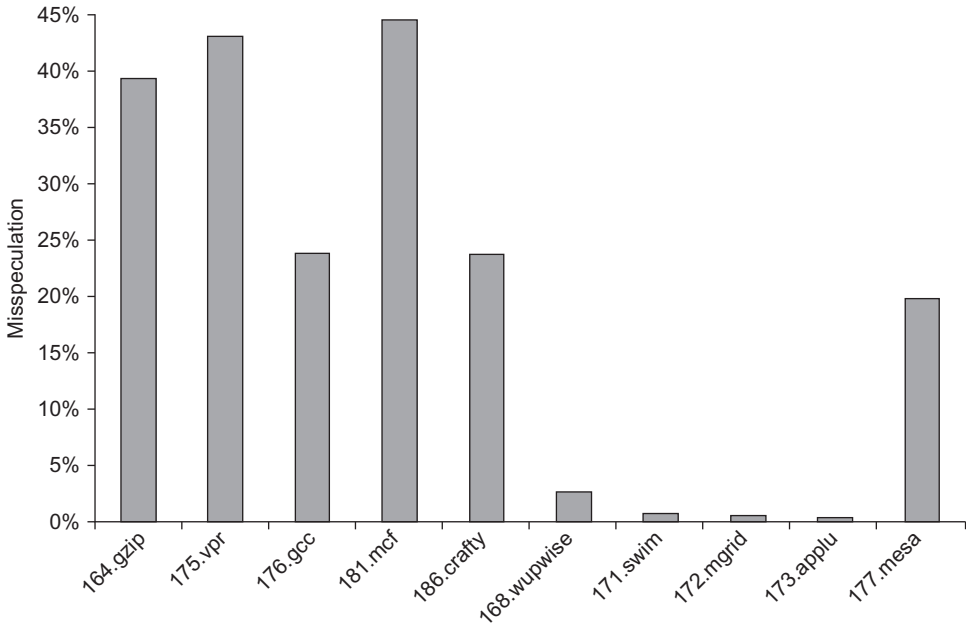


Figure 3.30 The fraction of instructions that are executed as a result of misspeculation is typically much higher for integer programs (the first five) versus FP programs (the last five).

improve running time by a comparable amount. Figure 3.30 shows the fraction of instructions that are executed from misspeculation for a subset of the SPEC2000 benchmarks using a sophisticated branch predictor. As we can see, this fraction of executed misspeculated instructions is small in scientific code and significant (about 30% on average) in integer code. Thus it is unlikely that speculation is energy-efficient for integer applications, and the end of Dennard scaling makes imperfect speculation more problematic. Designers could avoid speculation, try to reduce the misspeculation, or think about new approaches, such as only speculating on branches that are known to be highly predictable.

Address Aliasing Prediction

Address aliasing prediction is a technique that predicts whether two stores or a load and a store refer to the same memory address. If two such references do not refer to the same address, then they may be safely interchanged. Otherwise, we must wait until the memory addresses accessed by the instructions are known. Because we need not actually predict the address values, only whether such values conflict, the prediction can be reasonably accurate with small predictors. Address prediction relies on the ability of a speculative processor to recover after a misprediction; that is, if the actual addresses that were predicted to be different (and thus not alias) turn out to be the same (and thus are aliases), the processor simply restarts the sequence,

just as though it had mispredicted a branch. Address value speculation has been used in several processors already and may become universal in the future.

Address prediction is a simple and restricted form of *value prediction,* which attempts to predict the value that will be produced by an instruction. Value prediction could, if it were highly accurate, eliminate data flow restrictions and achieve higher rates of ILP. Despite many researchers focusing on value prediction in the past 15 years in dozens of papers, the results have never been sufficiently attractive to justify general value prediction in real processors.

3.10 Cross-Cutting Issues

Hardware Versus Software Speculation

The hardware-intensive approaches to speculation in this chapter and the software approaches of Appendix H provide alternative approaches to exploiting ILP. Some of the trade-offs, and the limitations, for these approaches are listed here:

- To speculate extensively, we must be able to disambiguate memory references. This capability is difficult to do at compile time for integer programs that contain pointers. In a hardware-based scheme, dynamic runtime disambiguation of memory addresses is done using the techniques we saw earlier for Tomasulo's algorithm. This disambiguation allows us to move loads past stores at runtime. Support for speculative memory references can help overcome the conservatism of the compiler, but unless such approaches are used carefully, the overhead of the recovery mechanisms may swamp the advantages.
- Hardware-based speculation works better when control flow is unpredictable and when hardware-based branch prediction is superior to software-based branch prediction done at compile time. These properties hold for many integer programs, where the misprediction rates for dynamic predictors are usually less than one-half of those for static predictors. Because speculated instructions may slow down the computation when the prediction is incorrect, this difference is significant. One result of this difference is that even statically scheduled processors normally include dynamic branch predictors.
- Hardware-based speculation maintains a completely precise exception model even for speculated instructions. Recent software-based approaches have added special support to allow this as well.
- Hardware-based speculation does not require compensation or bookkeeping code, which is needed by ambitious software speculation mechanisms.
- Compiler-based approaches may benefit from the ability to see further into the code sequence, resulting in better code scheduling than a purely hardware-driven approach.
- Hardware-based speculation with dynamic scheduling does not require different code sequences to achieve good performance for different implementations

of an architecture. Although this advantage is the hardest to quantify, it may be the most important one in the long run. Interestingly, this was one of the motivations for the IBM 360/91. On the other hand, more recent explicitly parallel architectures, such as IA-64, have added flexibility that reduces the hardware dependence inherent in a code sequence.

The major disadvantage of supporting speculation in hardware is the complexity and additional hardware resources required. This hardware cost must be evaluated against both the complexity of a compiler for a software-based approach and the amount and usefulness of the simplifications in a processor that relies on such a compiler.

Some designers have tried to combine the dynamic and compiler-based approaches to achieve the best of each. Such a combination can generate interesting and obscure interactions. For example, if conditional moves are combined with register renaming, a subtle side effect appears. A conditional move that is annulled must still copy a value to the destination register because it was renamed earlier in the instruction pipeline. These subtle interactions complicate the design and verification process and can also reduce performance.

The Intel Itanium processor was the most ambitious computer ever designed based on the software support for ILP and speculation. It did not deliver on the hopes of the designers, especially for general-purpose, nonscientific code. As designers' ambitions for exploiting ILP were reduced in light of the difficulties described on page 244, most architectures settled on hardware-based mechanisms with issue rates of three to four instructions per clock.

Speculative Execution and the Memory System

Inherent in processors that support speculative execution or conditional instructions is the possibility of generating invalid addresses that would not occur without speculative execution. Not only would this be incorrect behavior if protection exceptions were taken, but also the benefits of speculative execution would be swamped by false exception overhead. Therefore the memory system must identify speculatively executed instructions and conditionally executed instructions and suppress the corresponding exception.

By similar reasoning, we cannot allow such instructions to cause the cache to stall on a miss because, again, unnecessary stalls could overwhelm the benefits of speculation. Thus these processors must be matched with nonblocking caches.

In reality, the penalty of a miss that goes to DRAM is so large that speculated misses are handled only when the next level is on-chip cache (L2 or L3). Figure 2.5 on page 84 shows that for some well-behaved scientific programs, the compiler can sustain multiple outstanding L2 misses to cut the L2 miss penalty effectively. Once again, for this to work, the memory system behind the cache must match the goals of the compiler in number of simultaneous memory accesses.

3.11 Multithreading: Exploiting Thread-Level Parallelism to Improve Uniprocessor Throughput

The topic we cover in this section, multithreading, is truly a cross-cutting topic, because it has relevance to pipelining and superscalars, to graphics processing units (Chapter 4), and to multiprocessors (Chapter 5). A *thread* is like a process in that it has state and a current program counter, but threads typically share the address space of a single process, allowing a thread to easily access data of other threads within the same process. Multithreading is a technique whereby multiple threads share a processor without requiring an intervening process switch. The ability to switch between threads rapidly is what enables multithreading to be used to hide pipeline and memory latencies.

In the next chapter, we will see how multithreading provides the same advantages in GPUs. Finally, Chapter 5 will explore the combination of multithreading and multiprocessing. These topics are closely interwoven because multithreading is a primary technique for exposing more parallelism to the hardware. In a strict sense, multithreading uses thread-level parallelism, and thus is properly the subject of Chapter 5, but its role in both improving pipeline utilization and in GPUs motivates us to introduce the concept here.

Although increasing performance by using ILP has the great advantage that it is reasonably transparent to the programmer, as we have seen, ILP can be quite limited or difficult to exploit in some applications. In particular, with reasonable instruction issue rates, cache misses that go to memory or off-chip caches are unlikely to be hidden by available ILP. Of course, when the processor is stalled waiting on a cache miss, the utilization of the functional units drops dramatically.

Because attempts to cover long memory stalls with more ILP have limited effectiveness, it is natural to ask whether other forms of parallelism in an application could be used to hide memory delays. For example, an online transaction processing system has natural parallelism among the multiple queries and updates that are presented by requests. Of course, many scientific applications contain natural parallelism because they often model the three-dimensional, parallel structure of nature, and that structure can be exploited by using separate threads. Even desktop applications that use modern Windows-based operating systems often have multiple active applications running, providing a source of parallelism.

Multithreading allows multiple threads to share the functional units of a single processor in an overlapping fashion. In contrast, a more general method to exploit *thread-level parallelism* (TLP) is with a multiprocessor that has multiple independent threads operating at once and in parallel. Multithreading, however, does not duplicate the entire processor as a multiprocessor does. Instead, multithreading shares most of the processor core among a set of threads, duplicating only private state, such as the registers and program counter. As we will see in Chapter 5, many recent processors incorporate both multiple processor cores on a single chip and provide multithreading within each core.

Duplicating the per-thread state of a processor core means creating a separate register file and a separate PC for each thread. The memory itself can be shared through the virtual memory mechanisms, which already support multiprogramming. In addition, the hardware must support the ability to change to a different thread relatively quickly; in particular, a thread switch should be much more efficient than a process switch, which typically requires hundreds to thousands of processor cycles. Of course, for multithreading hardware to achieve performance improvements, a program must contain multiple threads (we sometimes say that the application is multithreaded) that could execute in concurrent fashion. These threads are identified either by a compiler (typically from a language with parallelism constructs) or by the programmer.

There are three main hardware approaches to multithreading: fine-grained, coarse-grained, and simultaneous. *Fine-grained multithreading* switches between threads on each clock cycle, causing the execution of instructions from multiple threads to be interleaved. This interleaving is often done in a round-robin fashion, skipping any threads that are stalled at that time. One key advantage of fine-grained multithreading is that it can hide the throughput losses that arise from both short and long stalls because instructions from other threads can be executed when one thread stalls, even if the stall is only for a few cycles. The primary disadvantage of fine-grained multithreading is that it slows down the execution of an individual thread because a thread that is ready to execute without stalls will be delayed by instructions from other threads. It trades an increase in multithreaded throughput for a loss in the performance (as measured by latency) of a single thread.

The SPARC T1 through T5 processors (originally made by Sun, now made by Oracle and Fujitsu) use fine-grained multithreading. These processors were targeted at multithreaded workloads such as transaction processing and web services. The T1 supported 8 cores per processor and 4 threads per core, while the T5 supports 16 cores and 128 threads per core. Later versions (T2–T5) also supported 4–8 processors. The NVIDIA GPUs, which we look at in the next chapter, also make use of fine-grained multithreading.

Coarse-grained multithreading was invented as an alternative to fine-grained multithreading. Coarse-grained multithreading switches threads only on costly stalls, such as level two or three cache misses. Because instructions from other threads will be issued only when a thread encounters a costly stall, coarse-grained multithreading relieves the need to have thread-switching be essentially free and is much less likely to slow down the execution of any one thread.

Coarse-grained multithreading suffers, however, from a major drawback: it is limited in its ability to overcome throughput losses, especially from shorter stalls. This limitation arises from the pipeline start-up costs of coarse-grained multithreading. Because a processor with coarse-grained multithreading issues instructions from a single thread, when a stall occurs, the pipeline will see a bubble before the new thread begins executing. Because of this start-up overhead, coarse-grained multithreading is much more useful for reducing the penalty of very high-cost stalls, where pipeline refill is negligible compared to the stall time. Several research

projects have explored coarse-grained multithreading, but no major current processors use this technique.

The most common implementation of multithreading is called *simultaneous multithreading* (SMT). Simultaneous multithreading is a variation on fine-grained multithreading that arises naturally when fine-grained multithreading is implemented on top of a multiple-issue, dynamically scheduled processor. As with other forms of multithreading, SMT uses thread-level parallelism to hide long-latency events in a processor, thereby increasing the usage of the functional units. The key insight in SMT is that register renaming and dynamic scheduling allow multiple instructions from independent threads to be executed without regard to the dependences among them; the resolution of the dependences can be handled by the dynamic scheduling capability.

Figure 3.31 conceptually illustrates the differences in a processor's ability to exploit the resources of a superscalar for the following processor configurations:

- A superscalar with no multithreading support
- A superscalar with coarse-grained multithreading

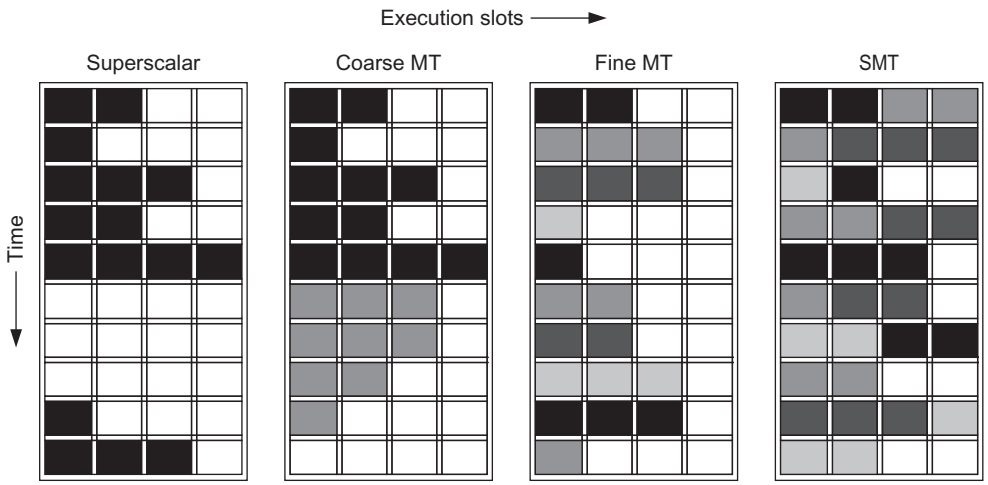


Figure 3.31 How four different approaches use the functional unit execution slots of a superscalar processor. The horizontal dimension represents the instruction execution capability in each clock cycle. The vertical dimension represents a sequence of clock cycles. An empty (white) box indicates that the corresponding execution slot is unused in that clock cycle. The shades of gray and black correspond to four different threads in the multithreading processors. Black is also used to indicate the occupied issue slots in the case of the superscalar without multithreading support. The Sun T1 and T2 (aka Niagara) processors are fine-grained, multithreaded processors, while the Intel Core i7 and IBM Power7 processors use SMT. The T2 has 8 threads, the Power7 has 4, and the Intel i7 has 2. In all existing SMTs, instructions issue from only one thread at a time. The difference in SMT is that the subsequent decision to execute an instruction is decoupled and could execute the operations coming from several different instructions in the same clock cycle.

- A superscalar with fine-grained multithreading
- A superscalar with simultaneous multithreading

In the superscalar without multithreading support, the use of issue slots is limited by a lack of ILP, including ILP to hide memory latency. Because of the length of L2 and L3 cache misses, much of the processor can be left idle.

In the coarse-grained multithreaded superscalar, the long stalls are partially hidden by switching to another thread that uses the resources of the processor. This switching reduces the number of completely idle clock cycles. In a coarse-grained multithreaded processor, however, thread switching occurs only when there is a stall. Because the new thread has a start-up period, there are likely to be some fully idle cycles remaining.

In the fine-grained case, the interleaving of threads can eliminate fully empty slots. In addition, because the issuing thread is changed on every clock cycle, longer latency operations can be hidden. Because instruction issue and execution are connected, a thread can issue only as many instructions as are ready. With a narrow issue width, this is not a problem (a cycle is either occupied or not), which is why fine-grained multithreading works perfectly for a single issue processor, and SMT would make no sense. Indeed, in the Sun T2, there are two issues per clock, but they are from different threads. This eliminates the need to implement the complex dynamic scheduling approach and relies instead on hiding latency with more threads.

If one implements fine-grained threading on top of a multiple-issue, dynamically schedule processor, the result is SMT. In all existing SMT implementations, all issues come from one thread, although instructions from different threads can initiate execution in the same cycle, using the dynamic scheduling hardware to determine what instructions are ready. Although Figure 3.31 greatly simplifies the real operation of these processors, it does illustrate the potential performance advantages of multithreading in general and SMT in wider issue, dynamically scheduled processors.

Simultaneous multithreading uses the insight that a dynamically scheduled processor already has many of the hardware mechanisms needed to support the mechanism, including a large virtual register set. Multithreading can be built on top of an out-of-order processor by adding a per-thread renaming table, keeping separate PCs, and providing the capability for instructions from multiple threads to commit.

Effectiveness of Simultaneous Multithreading on Superscalar Processors

A key question is, how much performance can be gained by implementing SMT? When this question was explored in 2000–2001, researchers assumed that dynamic superscalars would get much wider in the next five years, supporting six to eight issues per clock with speculative dynamic scheduling, many simultaneous loads and stores, large primary caches, and four to eight contexts with simultaneous issue

and retirement from multiple contexts. No processor has gotten close to this combination.

As a result, simulation research results that showed gains for multiprogrammed workloads of two or more times are unrealistic. In practice, the existing implementations of SMT offer only two to four contexts with fetching and issue from only one, and up to four issues per clock. The result is that the gain from SMT is also more modest.

Esmaeilzadeh et al. (2011) did an extensive and insightful set of measurements that examined both the performance and energy benefits of using SMT in a single i7 920 core running a set of multithreaded applications. The Intel i7 920 supported SMT with two threads per core, as does the recent i7 6700. The changes between the i7 920 and the 6700 are relatively small and are unlikely to significantly change the results as shown in this section.

The benchmarks used consist of a collection of parallel scientific applications and a set of multithreaded Java programs from the DaCapo and SPEC Java suite, as summarized in Figure 3.32. Figure 3.31 shows the ratios of performance and energy efficiency for these benchmarks when run on one core of a i7 920 with SMT turned off and on. (We plot the energy efficiency ratio, which is the inverse of energy consumption, so that, like speedup, a higher ratio is better.)

The harmonic mean of the speedup for the Java benchmarks is 1.28, despite the two benchmarks that see small gains. These two benchmarks, pjbb2005 and tradebeans, while multithreaded, have limited parallelism. They are included because they are typical of a multithreaded benchmark that might be run on an SMT processor with the hope of extracting some performance, which they find in limited amounts. The PARSEC benchmarks obtain somewhat better speedups than the full set of Java benchmarks (harmonic mean of 1.31). If tradebeans and pjbb2005 were omitted, the Java workload would actually have significantly better speedup (1.39) than the PARSEC benchmarks. (See the discussion of the implication of using harmonic mean to summarize the results in the caption of Figure 3.33.)

Energy consumption is determined by the combination of speedup and increase in power consumption. For the Java benchmarks, on average, SMT delivers the same energy efficiency as non-SMT (average of 1.0), but it is brought down by the two poor performing benchmarks; without pjbb2005 and tradebeans, the average energy efficiency for the Java benchmarks is 1.06, which is almost as good as the PARSEC benchmarks. In the PARSEC benchmarks, SMT reduces energy by $1-(1/1.08)=7\%$. Such energy-reducing performance enhancements are *very difficult* to find. Of course, the static power associated with SMT is paid in both cases, thus the results probably slightly overstate the energy gains.

These results clearly show that SMT with extensive support in an aggressive speculative processor can improve performance in an energy-efficient fashion. In 2011, the balance between offering multiple simpler cores and fewer more sophisticated cores has shifted in favor of more cores, with each core typically being a three- to four-issue superscalar with SMT supporting two to four threads. Indeed, Esmaeilzadeh et al. (2011) show that the energy improvements from SMT are even larger on the Intel i5 (a processor similar to the i7, but with smaller caches and a

blackscholes	Prices a portfolio of options with the Black-Scholes PDE
bodytrack	Tracks a markerless human body
canneal	Minimizes routing cost of a chip with cache-aware simulated annealing
facesim	Simulates motions of a human face for visualization purposes
ferret	Search engine that finds a set of images similar to a query image
fluidanimate	Simulates physics of fluid motion for animation with SPH algorithm
raytrace	Uses physical simulation for visualization
streamcluster	Computes an approximation for the optimal clustering of data points
swaptions	Prices a portfolio of swap options with the Heath–Jarrow–Morton framework
vips	Applies a series of transformations to an image
x264	MPG-4 AVC/H.264 video encoder
eclipse	Integrated development environment
lusearch	Text search tool
sunflow	Photo-realistic rendering system
tomcat	Tomcat servlet container
tradebeans	Tradebeans Daytrader benchmark
xalan	An XSLT processor for transforming XML documents
pjbb2005	Version of SPEC JBB2005 (but fixed in problem size rather than time)

Figure 3.32 The parallel benchmarks used here to examine multithreading, as well as in Chapter 5 to examine multiprocessing with an i7. The top half of the chart consists of PARSEC benchmarks collected by Bienia et al. (2008). The PARSEC benchmarks are meant to be indicative of compute-intensive, parallel applications that would be appropriate for multicore processors. The lower half consists of multithreaded Java benchmarks from the DaCapo collection (see Blackburn et al., 2006) and pjbb2005 from SPEC. All of these benchmarks contain some parallelism; other Java benchmarks in the DaCapo and SPEC Java workloads use multiple threads but have little or no true parallelism and, hence, are not used here. See Esmaeilzadeh et al. (2011) for additional information on the characteristics of these benchmarks, relative to the measurements here and in Chapter 5.

lower clock rate) and the Intel Atom (an 80 x 86 processor originally designed for the netbook and PMD market, now focused on low-end PCs, and described in Section 3.13).

3.12 Putting It All Together: The Intel Core i7 6700 and ARM Cortex-A53

In this section, we explore the design of two multiple issue processors: the ARM Cortex-A53 core, which is used as the basis for several tablets and cell phones, and the Intel Core i7 6700, a high-end, dynamically scheduled, speculative processor intended for high-end desktops and server applications. We begin with the simpler processor.

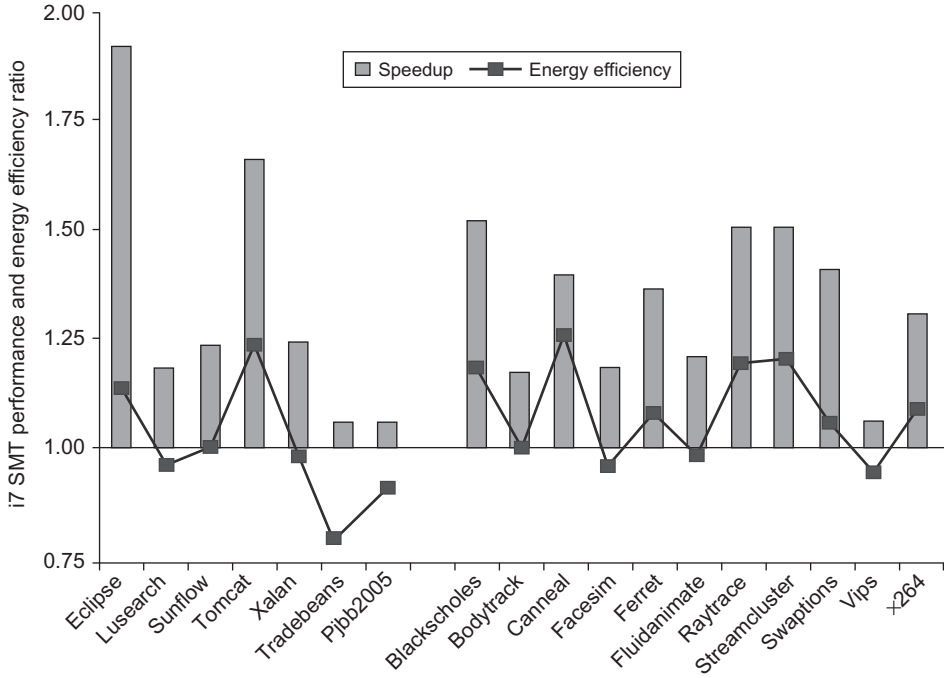


Figure 3.33 The speedup from using multithreading on one core on an i7 processor averages 1.28 for the Java benchmarks and 1.31 for the PARSEC benchmarks (using an unweighted harmonic mean, which implies a workload where the total time spent executing each benchmark in the single-threaded base set was the same). The energy efficiency averages 0.99 and 1.07, respectively (using the harmonic mean). Recall that anything above 1.0 for energy efficiency indicates that the feature reduces execution time by more than it increases average power. Two of the Java benchmarks experience little speedup and have significant negative energy efficiency because of this issue. Turbo Boost is off in all cases. These data were collected and analyzed by Esmaeilzadeh et al. (2011) using the Oracle (Sun) HotSpot build 16.3-b01 Java 1.6.0 Virtual Machine and the gcc v4.4.1 native compiler.

The ARM Cortex-A53

The A53 is a dual-issue, statically scheduled superscalar with dynamic issue detection, which allows the processor to issue two instructions per clock. Figure 3.34 shows the basic pipeline structure of the pipeline. For nonbranch, integer instructions, there are eight stages: F1, F2, D1, D2, D3/ISS, EX1, EX2, and WB, as described in the caption. The pipeline is in order, so an instruction can initiate execution only when its results are available and when proceeding instructions have initiated. Thus, if the next two instructions are dependent, both can proceed to the appropriate execution pipeline, but they will be serialized when they get to the beginning of that pipeline. When the scoreboard-based issue logic indicates that the result from the first instruction is available, the second instruction can issue.

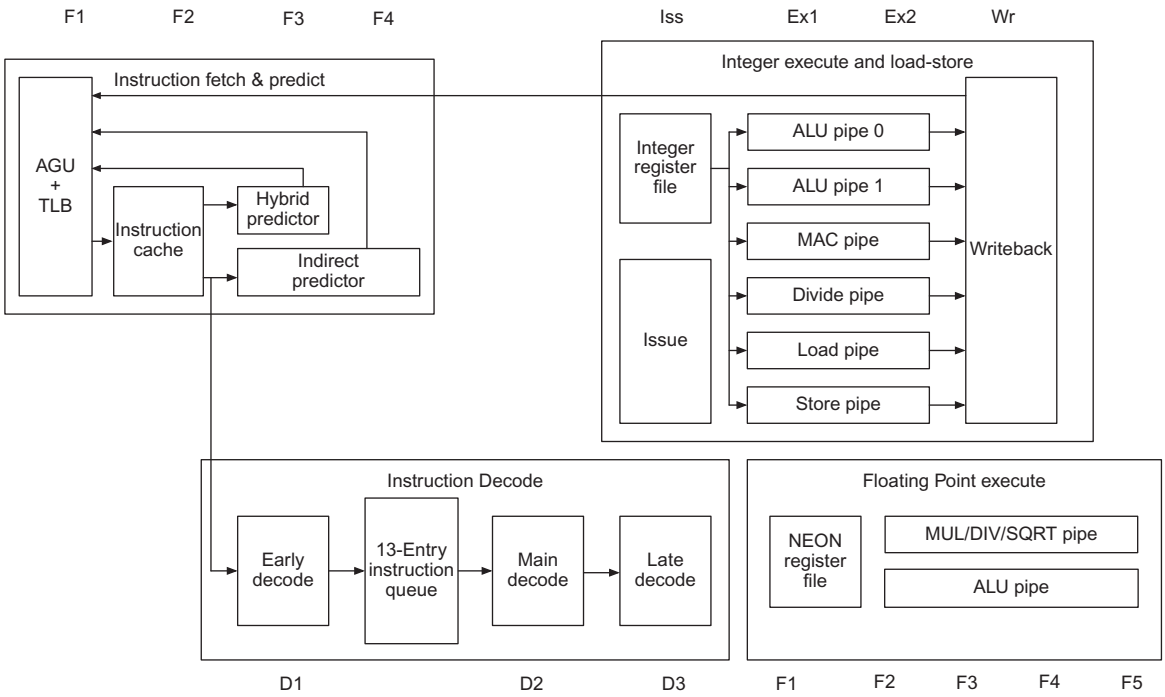


Figure 3.34 The basic structure of the A53 integer pipeline is 8 stages: F1 and F2 fetch the instruction, D1 and D2 do the basic decoding, and D3 decodes some more complex instructions and is overlapped with the first stage of the execution pipeline (ISS). After ISS, the Ex1, EX2, and WB stages complete the integer pipeline. Branches use four different predictors, depending on the type. The floating-point execution pipeline is 5 cycles deep, in addition to the 5 cycles needed for fetch and decode, yielding 10 stages in total.

The four cycles of instruction fetch include an address generation unit that produces the next PC either by incrementing the last PC or from one of four predictors:

1. A single-entry branch target cache containing two instruction cache fetches (the next two instructions following the branch, assuming the prediction is correct). This target cache is checked during the first fetch cycle, if it hits; then the next two instructions are supplied from the target cache. In case of a hit and a correct prediction, the branch is executed with no delay cycles.
2. A 3072-entry hybrid predictor, used for all instructions that do not hit in the branch target cache, and operating during F3. Branches handled by this predictor incur a 2-cycle delay.
3. A 256-entry indirect branch predictor that operates during F4; branches predicted by this predictor incur a three-cycle delay when predicted correctly.
4. An 8-deep return stack, operating during F4 and incurring a three-cycle delay.

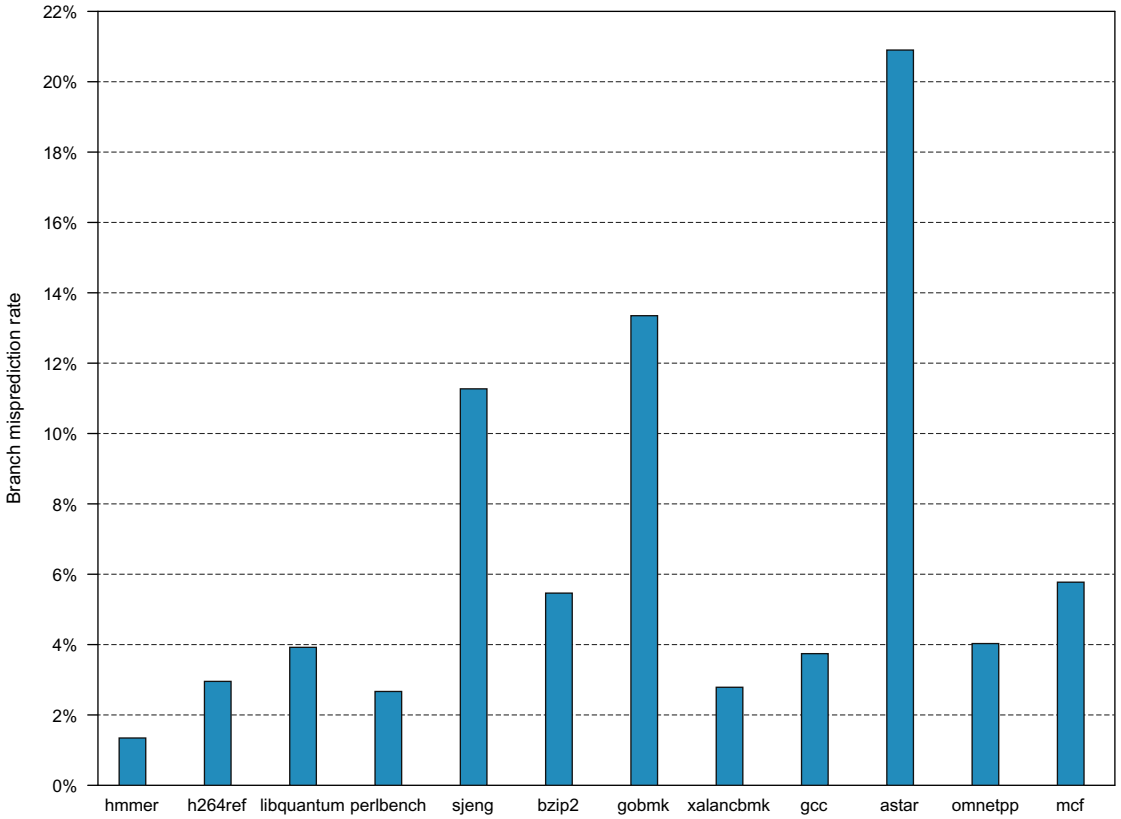


Figure 3.35 Misprediction rate of the A53 branch predictor for SPECint2006.

Branch decisions are made in ALU pipe 0, resulting in a branch misprediction penalty of 8 cycles. Figure 3.35 shows the misprediction rate for SPECint2006. The amount of work that is wasted depends on both the misprediction rate and the issue rate sustained during the time that the mispredicted branch was followed. As Figure 3.36 shows, wasted work generally follows the misprediction rate, though it may be larger or occasionally shorter.

Performance of the A53 Pipeline

The A53 has an ideal CPI of 0.5 because of its dual-issue structure. Pipeline stalls can arise from three sources:

1. Functional hazards, which occur because two adjacent instructions selected for issue simultaneously use the same functional pipeline. Because the A53 is statically scheduled, the compiler should try to avoid such conflicts. When such

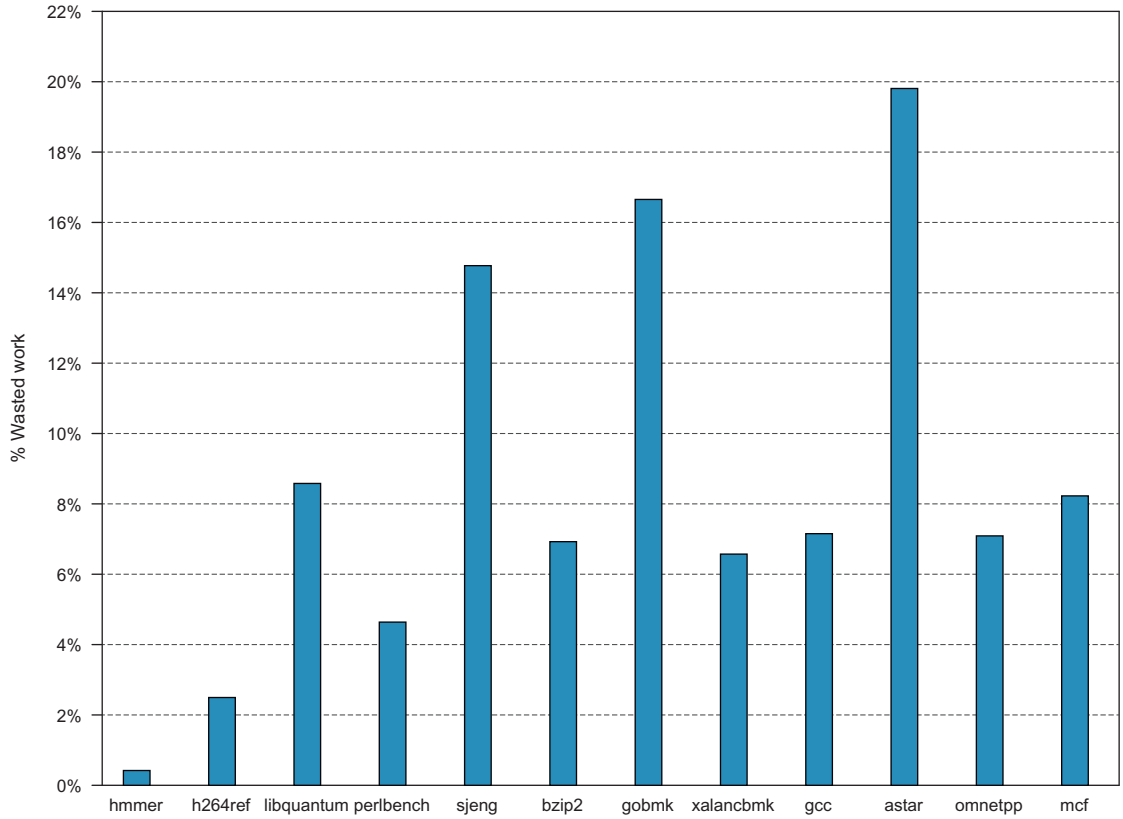


Figure 3.36 Wasted work due to branch misprediction on the A53. Because the A53 is an in-order machine, the amount of wasted work depends on a variety of factors, including data dependences and cache misses, both of which will cause a stall.

instructions appear sequentially, they will be serialized at the beginning of the execution pipeline, when only the first instruction will begin execution.

2. Data hazards, which are detected early in the pipeline and may stall either both instructions (if the first cannot issue, the second is always stalled) or the second of a pair. Again, the compiler should try to prevent such stalls when possible.
3. Control hazards, which arise only when branches are mispredicted.

Both TLB misses and cache misses also cause stalls. On the instruction side, a TLB or cache miss causes a delay in filling the instruction queue, likely leading to a downstream stall of the pipeline. Of course, this depends on whether it is an L1 miss, which might be largely hidden if the instruction queue was full at the time of the miss, or an L2 miss, which takes considerably longer. On the data side, a cache or TLB miss will cause the pipeline to stall because the load or store that

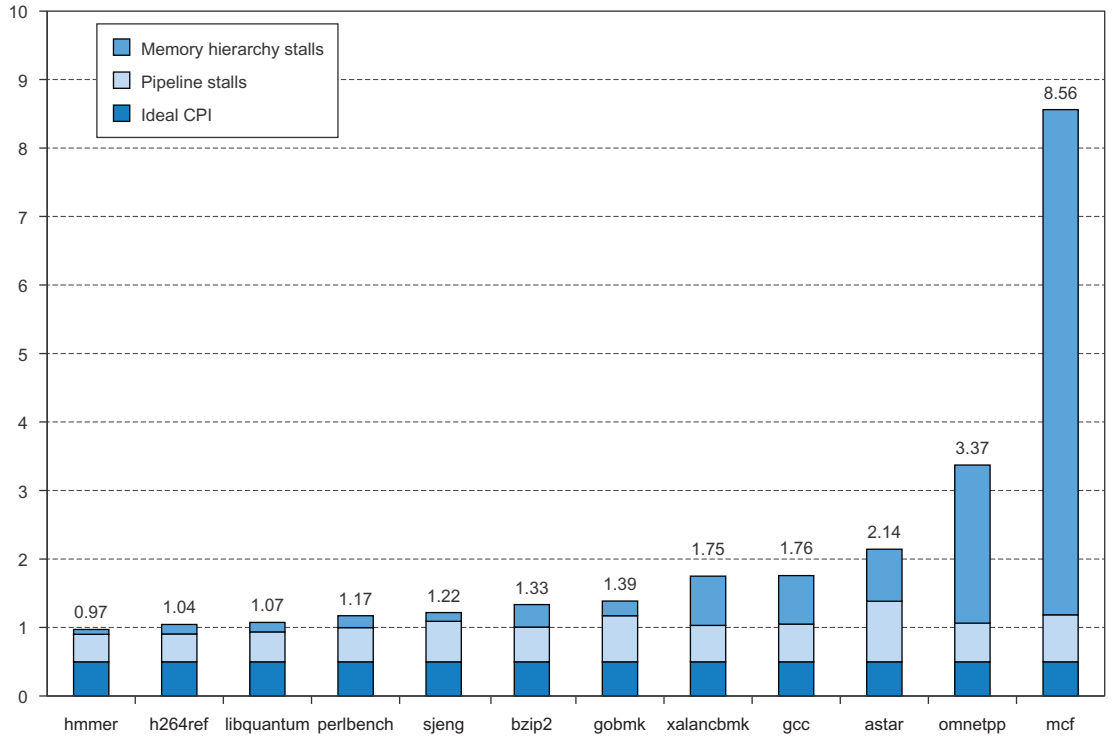


Figure 3.37 The estimated composition of the CPI on the ARM A53 shows that pipeline stalls are significant but are outweighed by cache misses in the poorest performing programs. This estimate is obtained by using the L1 and L2 miss rates and penalties to compute the L1 and L2 generated stalls per instruction. These are subtracted from the CPI measured by a detailed simulator to obtain the pipeline stalls. Pipeline stalls include all three hazards.

caused the miss cannot proceed down the pipeline. All other subsequent instructions will thus be stalled. Figure 3.37 shows the CPI and the estimated contributions from various sources.

The A53 uses a shallow pipeline and a reasonably aggressive branch predictor, leading to modest pipeline losses, while allowing the processor to achieve high clock rates at modest power consumption. In comparison with the i7, the A53 consumes approximately 1/200 the power for a quad core processor!

The Intel Core i7

The i7 uses an aggressive out-of-order speculative microarchitecture with deep pipelines with the goal of achieving high instruction throughput by combining multiple issue and high clock rates. The first i7 processor was introduced in 2008; the i7 6700 is the sixth generation. The basic structure of the i7 is similar, but successive

generations have enhanced performance by changing cache strategies (e.g., the aggressiveness of prefetching), increasing memory bandwidth, expanding the number of instructions in flight, enhancing branch prediction, and improving graphics support. The early i7 microarchitectures used reservations stations and reorder buffers for their out-of-order, speculative pipeline. Later microarchitectures, including the i7 6700, use register renaming, with the reservations stations acting as functional unit queues and the reorder buffer simply tracking control information.

Figure 3.38 shows the overall structure of the i7 pipeline. We will examine the pipeline by starting with instruction fetch and continuing on to instruction commit, following steps labeled in the figure.

1. Instruction fetch—The processor uses a sophisticated multilevel branch predictor to achieve a balance between speed and prediction accuracy. There is also a return address stack to speed up function return. Mispredictions cause a penalty of about 17 cycles. Using the predicted address, the instruction fetch unit fetches 16 bytes from the instruction cache.
2. The 16 bytes are placed in the predecode instruction buffer—In this step, a process called macro-op fusion is executed. *Macro-op fusion* takes instruction combinations such as compare followed by a branch and fuses them into a single operation, which can issue and dispatch as one instruction. Only certain special cases can be fused, since we must know that the only use of the first result is by the second instruction (i.e., compare and branch). In a study of the Intel Core architecture (which has many fewer buffers), Bird et al. (2007) discovered that macrofusion had a significant impact on the performance of integer programs resulting in an 8%–10% average increase in performance with a few programs showing negative results. There was little impact on FP programs; in fact, about half of the SPECFP benchmarks showed negative results from macro-op fusion. The predecode stage also breaks the 16 bytes into individual x86 instructions. This predecode is nontrivial because the length of an x86 instruction can be from 1 to 17 bytes and the predecoder must look through a number of bytes before it knows the instruction length. Individual x86 instructions (including some fused instructions) are placed into the instruction queue.
3. Micro-op decode—Individual x86 instructions are translated into micro-ops. Micro-ops are simple RISC-V-like instructions that can be executed directly by the pipeline; this approach of translating the x86 instruction set into simple operations that are more easily pipelined was introduced in the Pentium Pro in 1997 and has been used since. Three of the decoders handle x86 instructions that translate directly into one micro-op. For x86 instructions that have more complex semantics, there is a microcode engine that is used to produce the micro-op sequence; it can produce up to four micro-ops every cycle and continues until the necessary micro-op sequence has been generated. The micro-ops are placed according to the order of the x86 instructions in the 64-entry micro-op buffer.

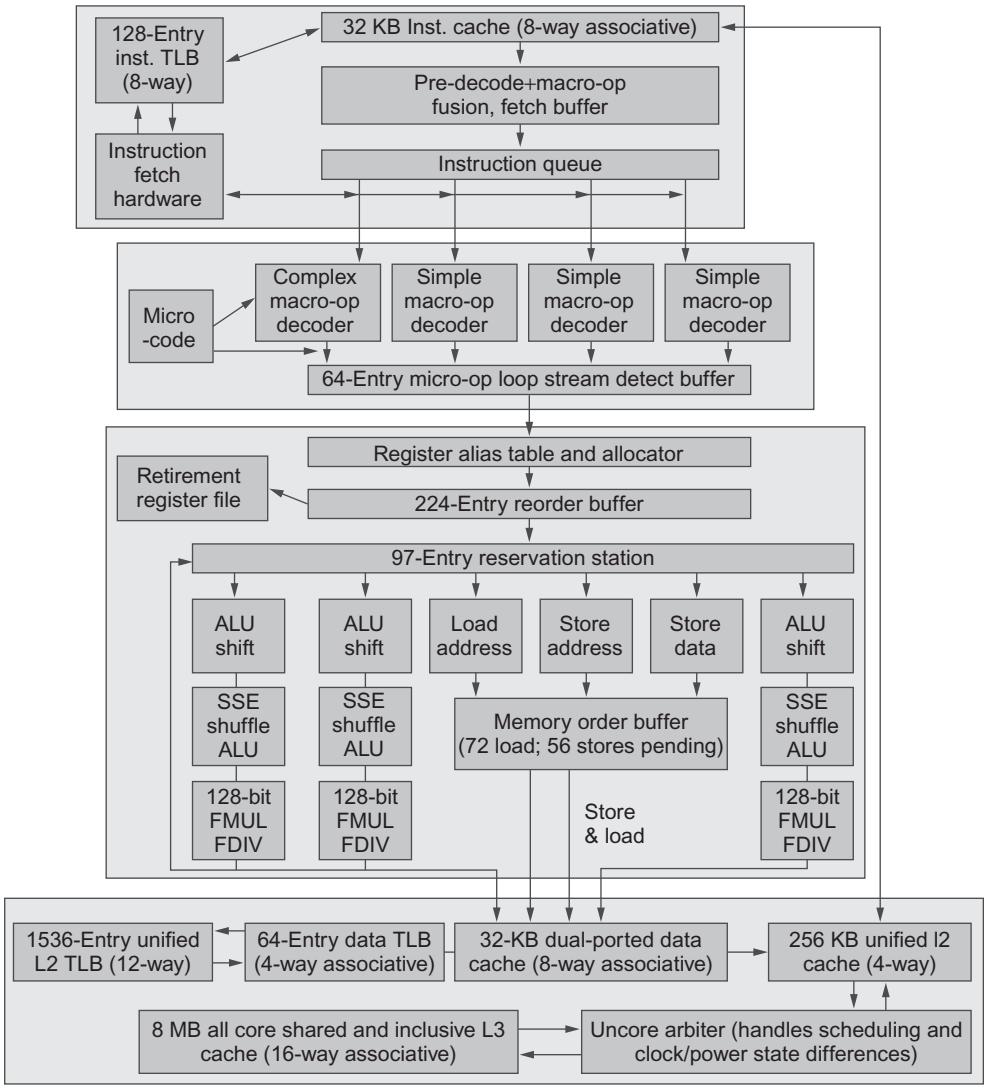


Figure 3.38 The Intel Core i7 pipeline structure shown with the memory system components. The total pipeline depth is 14 stages, with branch mispredictions typically costing 17 cycles, with the extra few cycles likely due to the time to reset the branch predictor. The six independent functional units can each begin execution of a ready micro-op in the same cycle. Up to four micro-ops can be processed in the register renaming table.

4. The micro-op buffer preforms *loop stream detection* and *microfusion*—If there is a small sequence of instructions (less than 64 instructions) that comprises a loop, the loop stream detector will find the loop and directly issue the micro-ops from the buffer, eliminating the need for the instruction fetch and instruction decode stages to be activated. Microfusion combines

instruction pairs such as ALU operation and a dependent store and issues them to a single reservation station (where they can still issue independently), thus increasing the usage of the buffer. Micro-op fusion produces smaller gains for integer programs and larger ones for FP, but the results vary widely. The different results for integer and FP programs with macro and micro fusion, probably arise from the patterns recognized and fused and the frequency of occurrence in integer versus FP programs. In the i7, which has a much larger number of reorder buffer entries, the benefits from both techniques are likely to be smaller.

5. Perform the basic instruction issue—Looking up the register location in the register tables, renaming the registers, allocating a reorder buffer entry, and fetching any results from the registers or reorder buffer before sending the micro-ops to the reservation stations. Up to four micro-ops can be processed every clock cycle; they are assigned the next available reorder buffer entries.
6. The i7 uses a centralized reservation station shared by six functional units. Up to six micro-ops may be dispatched to the functional units every clock cycle.
7. Micro-ops are executed by the individual function units, and then results are sent back to any waiting reservation station as well as to the register retirement unit, where they will update the register state once it is known that the instruction is no longer speculative. The entry corresponding to the instruction in the reorder buffer is marked as complete.
8. When one or more instructions at the head of the reorder buffer have been marked as complete, the pending writes in the register retirement unit are executed, and the instructions are removed from the reorder buffer.

In addition to the changes in the branch predictor, the major changes between the first generation i7 (the 920, Nehalem microarchitecture) and the sixth generation (i7 6700, Skylake microarchitecture) are in the sizes of the various buffers, renaming registers, and resources so as to allow many more outstanding instructions. Figure 3.39 summarizes these differences.

Performance of the i7

In earlier sections, we examined the performance of the i7's branch predictor and also the performance of SMT. In this section, we look at single-thread pipeline performance. Because of the presence of aggressive speculation as well as nonblocking caches, it is difficult to accurately attribute the gap between idealized performance and actual performance. The extensive queues and buffers on the 6700 reduce the probability of stalls because of a lack of reservation stations, renaming registers, or reorder buffers significantly. Indeed, even on the earlier i7 920 with notably fewer buffers, only about 3% of the loads were delayed because no reservation station was available.

Resource	i7 920 (Nehalem)	i7 6700 (Skylake)
Micro-op queue (per thread)	28	64
Reservation stations	36	97
Integer registers	NA	180
FP registers	NA	168
Outstanding load buffer	48	72
Outstanding store buffer	32	56
Reorder buffer	128	256

Figure 3.39 The buffers and queues in the first generation i7 and the latest generation i7. Nehalem used a reservation station plus reorder buffer organization. In later microarchitectures, the reservation stations serve as scheduling resources, and register renaming is used rather than the reorder buffer; the reorder buffer in the Skylake microarchitecture serves only to buffer control information. The choices of the size of various buffers and renaming registers, while appearing sometimes arbitrary, are likely based on extensive simulation.

Thus most losses come either from branch mispredicts or cache misses. The cost of a branch mispredict is 17 cycles, whereas the cost of an L1 miss is about 10 cycles. An L2 miss is slightly more than three times as costly as an L1 miss, and an L3 miss costs about 13 times what an L1 miss costs (130–135 cycles). Although the processor will attempt to find alternative instructions to execute during L2 and L3 misses, it is likely that some of the buffers will fill before a miss completes, causing the processor to stop issuing instructions.

Figure 3.40 shows the overall CPI for the 19 SPECCPUint2006 benchmarks compared to the CPI for the earlier i7 920. The average CPI on the i7 6700 is 0.71, whereas it is almost 1.5 times better on the i7 920, at 1.06. This difference derives from improved branch prediction and a reduction in the demand miss rates (see Figure 2.26 on page 135).

To understand how the 6700 achieves the significant improvement in CPI, let's look at the benchmarks that achieve the largest improvement. Figure 3.41 shows the five benchmarks that have a CPI ratio on the 920 that is at least 1.5 times higher than that of the 6700. Interestingly, three other benchmarks show a significant improvement in branch prediction accuracy (1.5 or more); however, those three benchmarks (HMMER, LIBQUANTUM, and SJENG) show equal or slightly higher L1 demand miss rates on the i7 6700. These misses likely arise because the aggressive prefetching is replacing cache blocks that are actually used. This type of behavior reminds designers of the challenges of maximizing performance in complex speculative multiple issue processors: rarely can significant performance be achieved by tuning only one part of the microarchitecture!

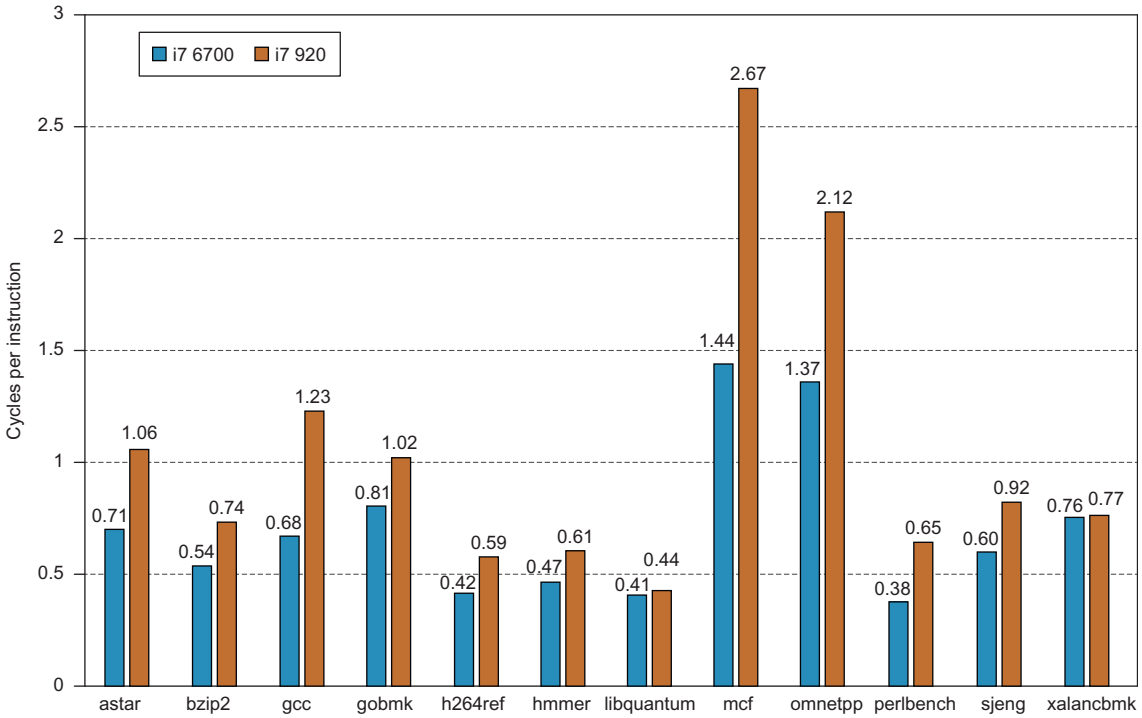


Figure 3.40 The CPI for the SPECCPUint2006 benchmarks on the i7 6700 and the i7 920. The data in this section were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University.

Benchmark	CPI ratio (920/6700)	Branch mispredict ratio (920/6700)	L1 demand miss ratio (920/6700)
ASTAR	1.51	1.53	2.14
GCC	1.82	2.54	1.82
MCF	1.85	1.27	1.71
OMNETPP	1.55	1.48	1.96
PERLBENCH	1.70	2.11	1.78

Figure 3.41 An analysis of the five integer benchmarks with the largest performance gap between the i7 6700 and 920. These five benchmarks show an improvement in the branch prediction rate and a reduction in the L1 demand miss rate.

3.13 Fallacies and Pitfalls

Our few fallacies focus on the difficulty of predicting performance and energy efficiency and extrapolating from single measures such as clock rate or CPI. We also show that different architectural approaches can have radically different behaviors for different benchmarks.

Fallacy *It is easy to predict the performance and energy efficiency of two different versions of the same instruction set architecture, if we hold the technology constant.*

Intel offers a processor for the low-end Netbook and PMD space called the Atom 230, which implements both the 64-bit and 32-bit versions of the x86 architecture. The Atom is a statically scheduled, 2-issue superscalar, quite similar in its microarchitecture to the ARM A8, a single-core predecessor of the A53. Interestingly, both the Atom 230 and the Core i7 920 have been fabricated in the same 45 nm Intel technology. Figure 3.42 summarizes the Intel Core i7 920, the ARM Cortex-A8, and the Intel Atom 230. These similarities provide a rare opportunity to directly compare two radically different microarchitectures for the same instruction set while holding constant the underlying fabrication technology. Before we do the comparison, we need to say a little more about the Atom 230.

The Atom processors implement the x86 architecture using the standard technique of translating x86 instructions into RISC-like instructions (as every x86 implementation since the mid-1990s has done). Atom uses a slightly more powerful microoperation, which allows an arithmetic operation to be paired with a load or a store; this capability was added to later i7s by the use of macrofusion. This means that on average for a typical instruction mix, only 4% of the instructions require more than one microoperation. The microoperations are then executed in a 16-deep pipeline capable of issuing two instructions per clock, in order, as in the ARM A8. There are dual-integer ALUs, separate pipelines for FP add and other FP operations, and two memory operation pipelines, supporting more general dual execution than the ARM A8 but still limited by the in-order issue capability. The Atom 230 has a 32 KiB instruction cache and a 24 KiB data cache, both backed by a shared 512 KiB L2 on the same die. (The Atom 230 also supports multithreading with two threads, but we will consider only single-threaded comparisons.)

We might expect that these two processors, implemented in the same technology and with the same instruction set, would exhibit predictable behavior, in terms of relative performance and energy consumption, meaning that power and performance would scale close to linearly. We examine this hypothesis using three sets of benchmarks. The first set is a group of Java single-threaded benchmarks that come from the DaCapo benchmarks and the SPEC JVM98 benchmarks (see Esmaeilzadeh et al. (2011) for a discussion of the benchmarks and measurements). The second and third sets of benchmarks are from SPEC CPU2006 and consist of the integer and FP benchmarks, respectively.

Area	Specific characteristic	Intel i7 920 Four cores, each with FP	ARM A8 One core, no FP	Intel Atom 230 One core, with FP
Physical chip properties	Clock rate	2.66 GHz	1 GHz	1.66 GHz
	Thermal design power	130 W	2 W	4 W
	Package	1366-pin BGA	522-pin BGA	437-pin BGA
	TLB	Two-level All four-way set associative 128 I/64 D 512 L2	One-level fully associative 32 I/32 D	Two-level All four-way set associative 16 I/16 D 64 L2
	Caches	Three-level 32 KiB/32 KiB 256 KiB 2–8 MiB	Two-level 16/16 or 32/32 KiB 128 KiB–1 MiB	Two-level 32/24 KiB 512 KiB
Memory system	Peak memory BW	17 GB/s	12 GB/sec	8 GB/s
Pipeline structure	Peak issue rate	4 ops/clock with fusion	2 ops/clock	2 ops/clock
	Pipe line scheduling	Speculating out of order	In-order dynamic issue	In-order dynamic issue
	Branch prediction	Two-level	Two-level 512-entry BTB 4 K global history 8-entry return stack	Two-level

Figure 3.42 An overview of the four-core Intel i7 920, an example of a typical ARM A8 processor chip (with a 256 MiB L2, 32 KiB L1s, and no floating point), and the Intel ARM 230, clearly showing the difference in design philosophy between a processor intended for the PMD (in the case of ARM) or netbook space (in the case of Atom) and a processor for use in servers and high-end desktops. Remember, the i7 includes four cores, each of which is higher in performance than the one-core A8 or Atom. All these processors are implemented in a comparable 45 nm technology.

As we can see in Figure 3.43, the i7 significantly outperforms the Atom. All benchmarks are at least four times faster on the i7, two SPECFP benchmarks are over 10 times faster, and one SPECINT benchmark runs over eight times faster! Because the ratio of clock rates of these two processors is 1.6, most of the advantage comes from a much lower CPI for the i7 920: a factor of 2.8 for the Java benchmarks, a factor of 3.1 for the SPECINT benchmarks, and a factor of 4.3 for the SPECFP benchmarks.

But the average power consumption for the i7 920 is just under 43 W, while the average power consumption of the Atom is 4.2 W, or about one-tenth of the power! Combining the performance and power leads to an energy efficiency advantage for the Atom that is typically more than 1.5 times better and often 2 times better! This comparison of two processors using the same underlying technology makes it clear that the performance advantages of an aggressive

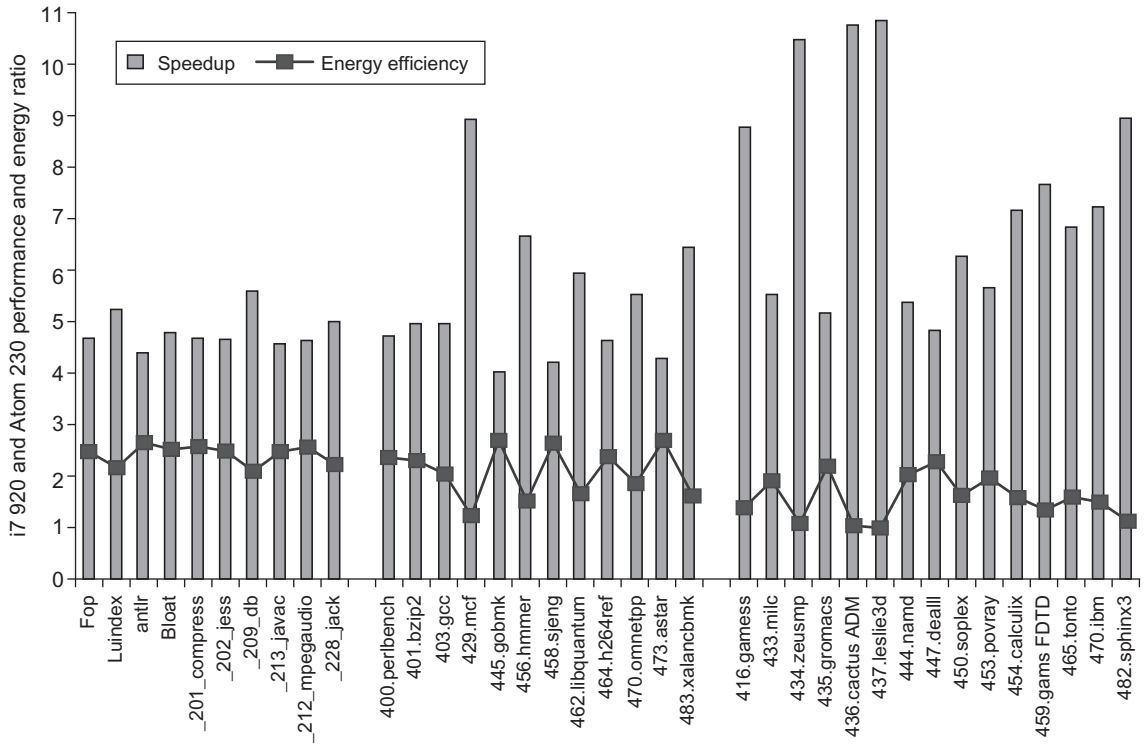


Figure 3.43 The relative performance and energy efficiency for a set of single-threaded benchmarks shows the i7 920 is 4 to over 10 times faster than the Atom 230 but that it is about 2 times *less* power-efficient on average! Performance is shown in the columns as i7 relative to Atom, which is execution time (i7)/execution time (Atom). Energy is shown with the line as Energy (Atom)/Energy (i7). The i7 never beats the Atom in energy efficiency, although it is essentially as good on four benchmarks, three of which are floating point. The data shown here were collected by Esmaeilzadeh et al. (2011). The SPEC benchmarks were compiled with optimization using the standard Intel compiler, while the Java benchmarks use the Sun (Oracle) Hotspot Java VM. Only one core is active on the i7, and the rest are in deep power saving mode. Turbo Boost is used on the i7, which increases its performance advantage but slightly decreases its relative energy efficiency.

superscalar with dynamic scheduling and speculation come with a *significant disadvantage* in energy efficiency.

Fallacy *Processors with lower CPIs will always be faster.*

Fallacy *Processors with faster clock rates will always be faster.*

The key is that it is the product of CPI and clock rate that determines performance. A high clock rate obtained by deeply pipelining the processor must maintain a low CPI to get the full benefit of the faster clock. Similarly, a simple processor with a high clock rate but a low CPI may be slower.

As we saw in the previous fallacy, performance and energy efficiency can diverge significantly among processors designed for different environments even when they have the same ISA. In fact, large differences in performance can show up even within a family of processors from the same company all designed for high-end applications. Figure 3.44 shows the integer and FP performance of two different implementations of the x86 architecture from Intel, as well as a version of the Itanium architecture, also by Intel.

The Pentium 4 was the most aggressively pipelined processor ever built by Intel. It used a pipeline with over 20 stages, had seven functional units, and cached micro-ops rather than x86 instructions. Its relatively inferior performance, given the aggressive implementation, was a clear indication that the attempt to exploit more ILP (there could easily be 50 instructions in flight) had failed. The Pentium's power consumption was similar to the i7, although its transistor count was lower, as its primary caches were half as large as the i7, and it included only a 2 MiB secondary cache with no tertiary cache.

The Intel Itanium is a VLIW-style architecture, which despite the potential decrease in complexity compared to dynamically scheduled superscalars, never attained competitive clock rates with the mainline x86 processors (although it appears to achieve an overall CPI similar to that of the i7). In examining these results, the reader should be aware that they use different implementation technologies, giving the i7 an advantage in terms of transistor speed and hence clock rate for an equivalently pipelined processor. Nonetheless, the wide variation in

Processor	Implementation technology	Clock rate	Power	SPECCInt2006 base	SPECCFP2006 baseline
Intel Pentium 4 670	90 nm	3.8 GHz	115 W	11.5	12.2
Intel Itanium 2	90 nm	1.66 GHz	104 W approx. 70 W one core	14.5	17.3
Intel i7 920	45 nm	3.3 GHz	130 W total approx. 80 W one core	35.5	38.4

Figure 3.44 Three different Intel processors vary widely. Although the Itanium processor has two cores and the i7 four, only one core is used in the benchmarks; the Power column is the thermal design power with estimates for only one core active in the multicore cases.

performance—more than three times between the Pentium and i7—is astonishing. The next pitfall explains where a significant amount of this advantage comes from.

Pitfall *Sometimes bigger and dumber is better.*

Much of the attention in the early 2000s went to building aggressive processors to exploit ILP, including the Pentium 4 architecture, which used the deepest pipeline ever seen in a microprocessor, and the Intel Itanium, which had the highest peak issue rate per clock ever seen. What quickly became clear was that the main limitation in exploiting ILP often turned out to be the memory system. Although speculative out-of-order pipelines were fairly good at hiding a significant fraction of the 10- to 15-cycle miss penalties for a first-level miss, they could do very little to hide the penalties for a second-level miss that, when going to main memory, were likely to be 50–100 clock cycles.

The result was that these designs never came close to achieving the peak instruction throughput despite the large transistor counts and extremely sophisticated and clever techniques. Section 3.15 discusses this dilemma and the turning away from more aggressive ILP schemes to multicore, but there was another change that exemplified this pitfall. Instead of trying to hide even more memory latency with ILP, designers simply used the transistors to build much larger caches. Both the Itanium 2 and the i7 use three-level caches compared to the two-level cache of the Pentium 4, and the third-level caches are 9 and 8 MiB compared to the 2 MiB second-level cache of the Pentium 4. Needless to say, building larger caches is a lot easier than designing the 20+-stage Pentium 4 pipeline, and based on the data in Figure 3.44, doing so seems to be more effective.

Pitfall *And sometimes smarter is better than bigger and dumber.*

One of the more surprising results of the past decade has been in branch prediction. The emergence of hybrid tagged predictors has shown that a more sophisticated predictor can outperform the simple gshare predictor with the same number of bits (see Figure 3.8 on page 171). One reason this result is so surprising is that the tagged predictor actually stores fewer predictions, because it also consumes bits to store tags, whereas gshare has only a large array of predictions. Nonetheless, it appears that the advantage gained by not misusing a prediction for one branch on another branch more than justifies the allocation of bits to tags versus predictions.

Pitfall *Believing that there are large amounts of ILP available, if only we had the right techniques.*

The attempts to exploit large amounts of ILP failed for several reasons, but one of the most important ones, which some designers did not initially accept, is that it is hard to find large amounts of ILP in conventionally structured programs, even with speculation. A famous study by David Wall in 1993 (see Wall, 1993) analyzed the amount of ILP available under a variety of idealistic conditions. We summarize his results for a processor configuration with roughly five to ten times the capability of the most advanced processors in 2017. Wall's study extensively documented a variety of different approaches,

and the reader interested in the challenge of exploiting ILP should read the complete study.

The aggressive processor we consider has the following characteristics:

1. Up to 64 instruction issues and dispatches per clock with *no* issue restrictions, or 8 times the total issue width of the widest processor in 2016 (the IBM Power8) and with up to 32 times as many loads and stores allowed per clock! As we have discussed, there are serious complexity and power problems with large issue rates.
2. A tournament predictor with 1K entries and a 16-entry function return predictor. This predictor is comparable to the best predictors in 2016; the predictor is not a primary bottleneck. Mispredictions are handled in one cycle, but they limit the ability to speculate.
3. Perfect disambiguation of memory references done dynamically—this is ambitious but perhaps attainable for small window sizes.
4. Register renaming with 64 additional integer and 64 additional FP registers, which is somewhat less than the most aggressive processor in 2011. Because the study assumes a latency of only one cycle for all instructions (versus 15 or more on processors like the i7 or Power8), the effective number of rename registers is about five times larger than either of those processors.

Figure 3.45 shows the result for this configuration as we vary the window size. This configuration is more complex and expensive than existing implementations, especially in terms of the number of instruction issues. Nonetheless, it gives a useful upper limit on what future implementations might yield. The data in these figures are likely to be very optimistic for another reason. There are no issue restrictions among the 64 instructions: for example, they may all be memory references. No one would even contemplate this capability in a processor for the near future. In addition, remember that in interpreting these results, cache misses and non-unit latencies were not taken into account, and both these effects have significant impacts.

The most startling observation in Figure 3.45 is that with the preceding realistic processor constraints, the effect of the window size for the integer programs is not as severe as for FP programs. This result points to the key difference between these two types of programs. The availability of loop-level parallelism in two of the FP programs means that the amount of ILP that can be exploited is higher, but for integer programs other factors—such as branch prediction, register renaming, and less parallelism, to start with—are all important limitations. This observation is critical because most of the market growth in the past decade—transaction processing, web servers, and the like—depended on integer performance, rather than floating point.

Wall's study was not believed by some, but 10 years later, the reality had sunk in, and the combination of modest performance increases with significant hardware resources and major energy issues coming from incorrect speculation forced a change in direction. We will return to this discussion in our concluding remarks.

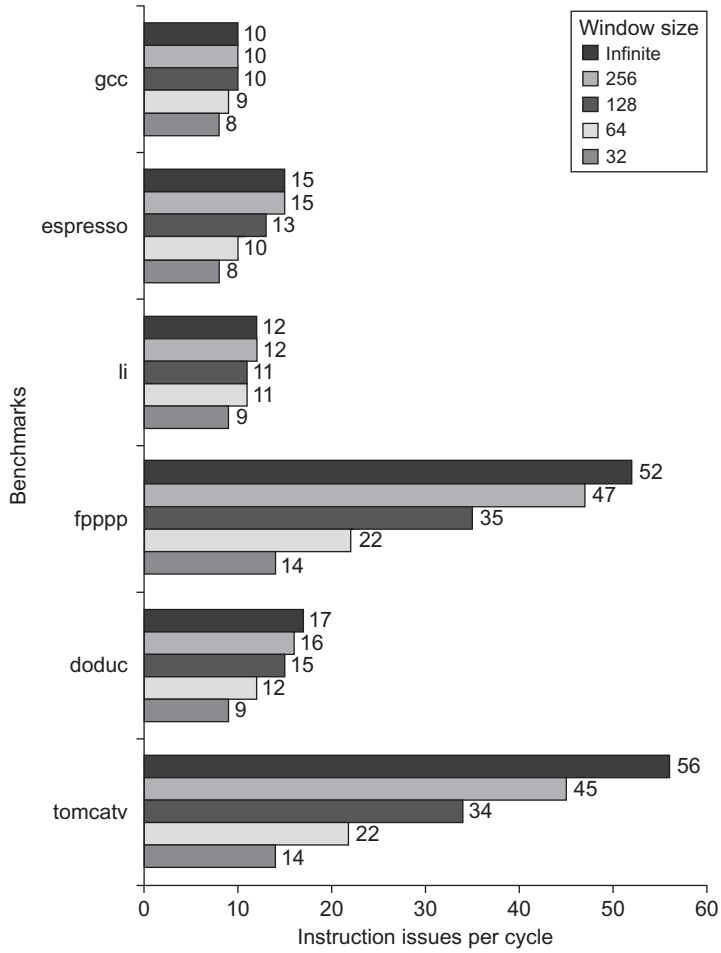


Figure 3.45 The amount of parallelism available versus the window size for a variety of integer and floating-point programs with up to 64 arbitrary instruction issues per clock. Although there are fewer renaming registers than the window size, the fact that all operations have 1-cycle latency and that the number of renaming registers equals the issue width allows the processor to exploit parallelism within the entire window.

3.14 Concluding Remarks: What's Ahead?

As 2000 began the focus on exploiting instruction-level parallelism was at its peak. In the first five years of the new century, it became clear that the ILP approach had likely peaked and that new approaches would be needed. By 2005 Intel and all the other major processor manufacturers had revamped their approach to focus on multicore. Higher performance would be achieved through thread-level parallelism rather than instruction-level parallelism, and the responsibility for using the

processor efficiently would largely shift from the hardware to the software and the programmer. This change was the most significant change in processor architecture since the early days of pipelining and instruction-level parallelism some 25+ years earlier.

During the same period, designers began to explore the use of more data-level parallelism as another approach to obtaining performance. SIMD extensions enabled desktop and server microprocessors to achieve moderate performance increases for graphics and similar functions. More importantly, graphics processing units (GPUs) pursued aggressive use of SIMD, achieving significant performance advantages for applications with extensive data-level parallelism. For scientific applications, such approaches represent a viable alternative to the more general, but less efficient, thread-level parallelism exploited in multicores. The next chapter explores these developments in the use of data-level parallelism.

Many researchers predicted a major retrenchment in the use of ILP, predicting that two issue superscalar processors and larger numbers of cores would be the future. The advantages, however, of slightly higher issue rates and the ability of speculative dynamic scheduling to deal with unpredictable events, such as level-one cache misses, led to moderate ILP (typically about 4 issues/clock) being the primary building block in multicore designs. The addition of SMT and its effectiveness (both for performance and energy efficiency) further cemented the position of the moderate issue, out-of-order, speculative approaches. Indeed, even in the embedded market, the newest processors (e.g., the ARM Cortex-A9 and Cortex-A73) have introduced dynamic scheduling, speculation, and wider issues rates.

It is highly unlikely that future processors will try to increase the width of issue significantly. It is simply too inefficient from the viewpoint of silicon utilization and power efficiency. Consider the data in Figure 3.46 that show the five processors in the IBM Power series. Over more than a decade, there has been a modest improvement in the ILP support in the Power processors, but the dominant portion

	Power4	Power5	Power6	Power7	Power8
Introduced	2001	2004	2007	2010	2014
Initial clock rate (GHz)	1.3	1.9	4.7	3.6	3.3 GHz
Transistor count (M)	174	276	790	1200	4200
Issues per clock	5	5	7	6	8
Functional units per core	8	8	9	12	16
SMT threads per core	0	2	2	4	8
Cores/chip	2	2	2	8	12
SMT threads per core	0	2	2	4	8
Total on-chip cache (MiB)	1.5	2	4.1	32.3	103.0

Figure 3.46 Characteristics of five generations of IBM Power processors. All except the Power6, which is static and in-order, were dynamically scheduled; all the processors support two load/store pipelines. The Power6 has the same functional units as the Power5 except for a decimal unit. Power7 and Power8 use embedded DRAM for the L3 cache. Power9 has been described briefly; it further expands the caches and supports off-chip HBM.

of the increase in transistor count (a factor of more than 10 from the Power4 to the Power8) went to increasing the caches and the number of cores per die. Even the expansion in SMT support seems to be more of a focus than is an increase in the ILP throughput: The ILP structure from Power4 to Power8 went from 5 issues to 8, from 8 functional units to 16 (but not increasing from the original 2 load/store units), whereas the SMT support went from nonexistent to 8 threads/processor. A similar trend can be observed across the six generations of i7 processors, where almost all the additional silicon has gone to supporting more cores. The next two chapters focus on approaches that exploit data-level and thread-level parallelism.

3.15 Historical Perspective and References

Section M.5 (available online) features a discussion on the development of pipelining and instruction-level parallelism. We provide numerous references for further reading and exploration of these topics. Section M.5 covers both Chapter 3 and Appendix H.

Case Studies and Exercises by Jason D. Bakos and Robert P. Colwell

Case Study: Exploring the Impact of Microarchitectural Techniques

Concepts illustrated by this case study

- Basic Instruction Scheduling, Reordering, Dispatch
- Multiple Issue and Hazards
- Register Renaming
- Out-of-Order and Speculative Execution
- Where to Spend Out-of-Order Resources

You are tasked with designing a new processor microarchitecture and you are trying to determine how best to allocate your hardware resources. Which of the hardware and software techniques you learned in Chapter 3 should you apply? You have a list of latencies for the functional units and for memory, as well as some representative code. Your boss has been somewhat vague about the performance requirements of your new design, but you know from experience that, all else being equal, faster is usually better. Start with the basics. Figure 3.47 provides a sequence of instructions and list of latencies.

3.1 [10] <3.1, 3.2> What is the baseline performance (in cycles, per loop iteration) of the code sequence in Figure 3.47 if no new instruction's execution could be

Latencies beyond single cycle	
Memory LD	+3
Memory SD	+1
Integer ADD, SUB	+0
Branches	+1
fadd.d	+2
fmul.d	+4
fdiv.d	+10

```
Loop:   fld     f2,0(Rx)
I0:     fmul.d  f2,f0,f2
I1:     fdiv.d  f8,f2,f0
I2:     fld     f4,0(Ry)
I3:     fadd.d  f4,f0,f4
I4:     fadd.d  f10,f8,f2
I5:     fsd     f4,0(Ry)
I6:     addi    Rx,Rx,8
I7:     addi    Ry,Ry,8
I8:     sub     x20,x4,Rx
I9:     bnz     x20,Loop
```

Figure 3.47 Code and latencies for Exercises 3.1 through 3.6.

initiated until the previous instruction's execution had completed? Ignore front-end fetch and decode. Assume for now that execution does not stall for lack of the next instruction, but only one instruction/cycle can be issued. Assume the branch is taken, and that there is a one-cycle branch delay slot.

3.2 [10] <3.1, 3.2> Think about what latency numbers really mean—they indicate the number of cycles a given function requires to produce its output. If the overall pipeline stalls for the latency cycles of each functional unit, then you are at least guaranteed that any pair of back-to-back instructions (a "producer" followed by a "consumer") will execute correctly. But not all instruction pairs have a producer/consumer relationship. Sometimes two adjacent instructions have nothing to do with each other. How many cycles would the loop body in the code sequence in Figure 3.47 require if the pipeline detected true data dependences and only stalled on those, rather than blindly stalling everything just because one functional unit is busy? Show the code with <stall> inserted where necessary to accommodate stated latencies. (*Hint:* an instruction with latency +2 requires two <stall> cycles to be inserted into the code sequence.) Think of it this

way: a one-cycle instruction has latency 1 + 0, meaning zero extra wait states. So, latency 1 + 1 implies one stall cycle; latency 1 + *N* has *N* extra stall cycles.

3.3 [15] <3.1, 3.2> Consider a multiple-issue design. Suppose you have two execution pipelines, each capable of beginning execution of one instruction per cycle, and enough fetch/decode bandwidth in the front end so that it will not stall your execution. Assume results can be immediately forwarded from one execution unit to another, or to itself. Further assume that the only reason an execution pipeline would stall is to observe a true data dependency. Now how many cycles does the loop require?

3.4 [10] <3.1, 3.2> In the multiple-issue design of Exercise 3.3, you may have recognized some subtle issues. Even though the two pipelines have the exact same instruction repertoire, they are neither identical nor interchangeable, because there is an implicit ordering between them that must reflect the ordering of the instructions in the original program. If instruction *N* + 1 begins execution in Execution Pipe 1 at the same time that instruction *N* begins in Pipe 0, and *N* + 1 happens to require a shorter execution latency than *N*, then *N* + 1 will complete before *N* (even though program ordering would have implied otherwise). Recite at least two reasons why that could be hazardous and will require special considerations in the microarchitecture. Give an example of two instructions from the code in Figure 3.47 that demonstrate this hazard.

3.5 [20] <3.1, 3.2> Reorder the instructions to improve performance of the code in Figure 3.47. Assume the two-pipe machine in Exercise 3.3 and that the out-of-order completion issues of Exercise 3.4 have been dealt with successfully. Just worry about observing true data dependences and functional unit latencies for now. How many cycles does your reordered code take?

3.6 [10/10/10] <3.1, 3.2> Every cycle that does not initiate a new operation in a pipe is a lost opportunity, in the sense that your hardware is not living up to its potential.

a. [10] <3.1, 3.2> In your reordered code from Exercise 3.5, what fraction of all cycles, counting both pipes, were wasted (did not initiate a new op)?

b. [10] <3.1, 3.2> Loop unrolling is one standard compiler technique for finding more parallelism in code, in order to minimize the lost opportunities for performance. Hand-unroll two iterations of the loop in your reordered code from Exercise 3.5.

c. [10] <3.1, 3.2> What speedup did you obtain? (For this exercise, just color the *N* + 1 iteration's instructions green to distinguish them from the *N*th iteration's instructions; if you were actually unrolling the loop, you would have to reassign registers to prevent collisions between the iterations.)

3.7 [15] <3.4> Computers spend most of their time in loops, so multiple loop iterations are great places to speculatively find more work to keep CPU resources busy. Nothing is ever easy, though; the compiler emitted only one copy of that loop's code, so even though multiple iterations are handling distinct data, they will

appear to use the same registers. To keep multiple iterations' register usages from colliding, we rename their registers. Figure 3.48 shows example code that we would like our hardware to rename. A compiler could have simply unrolled the loop and used different registers to avoid conflicts, but if we expect our hardware to unroll the loop, it must also do the register renaming. How? Assume your hardware has a pool of temporary registers (call them T registers, and assume that there are 64 of them, T0 through T63) that it can substitute for those registers designated by the compiler. This rename hardware is indexed by the src (source) register designation, and the value in the table is the T register of the last destination that targeted that register. (Think of these table values as producers, and the src registers are the consumers; it doesn't much matter where the producer puts its result as long as its consumers can find it.) Consider the code sequence in Figure 3.48. Every time you see a destination register in the code, substitute the next available T, beginning with T9. Then update all the src registers accordingly, so that true data dependences are maintained. Show the resulting code. (*Hint:* see Figure 3.49.)

3.8 [20] <3.4> Exercise 3.7 explored simple register renaming: when the hardware register renamer sees a source register, it substitutes the destination T register of the last instruction to have targeted that source register. When the rename table sees a destination register, it substitutes the next available T for it, but superscalar designs need to handle multiple instructions per clock cycle at every stage in the machine, including the register renaming. A SimpleScalar processor would therefore look up both src register mappings for each instruction and allocate a new dest mapping per clock cycle. Superscalar processors must be able to do that as well, but they must also ensure that any dest-to-src relationships between the two concurrent instructions are handled correctly. Consider the sample code sequence in Figure 3.50. Assume that we would like to simultaneously

```
Loop:          fld          f2,0(Rx)
I0:            fmul.d       f5,f0,f2
I1:            fdiv.d       f8,f0,f2
I2:            fld          f4,0(Ry)
I3:            fadd.d       f6,f0,f4
I4:            fadd.d       f10,f8,f2
I5:            sd           f4,0(Ry)
```

Figure 3.48 Sample code for register renaming practice.

```
I0:          fld          T9,0(Rx)
I1:          fmul.d       T10,F0,T9
...
```

Figure 3.49 Expected output of register renaming.

```
I0:     fmul.d    f5,f0,f2
I1:     fadd.d    f9,f5,f4
I2:     fadd.d    f5,f5,f2
I3:     fdiv.d    f2,f9,f0
```

Figure 3.50 Sample code for superscalar register renaming.

rename the first two instructions. Further assume that the next two available T registers to be used are known at the beginning of the clock cycle in which these two instructions are being renamed. Conceptually, what we want is for the first instruction to do its rename table lookups and then update the table per its destination's T register. Then the second instruction would do exactly the same thing, and any inter-instruction dependency would thereby be handled correctly. But there's not enough time to write that T register designation into the renaming table and then look it up again for the second instruction, all in the same clock cycle. That register substitution must instead be done live (in parallel with the register rename table update). Figure 3.51 shows a circuit diagram, using multiplexers and comparators, that will accomplish the necessary on-the-fly register renaming. Your task is to show the cycle-by-cycle state of the rename table for every instruction of the code shown in Figure 3.50. Assume the table starts out with every entry equal to its index (T0 = 0; T1 = 1, …) (Figure 3.51).

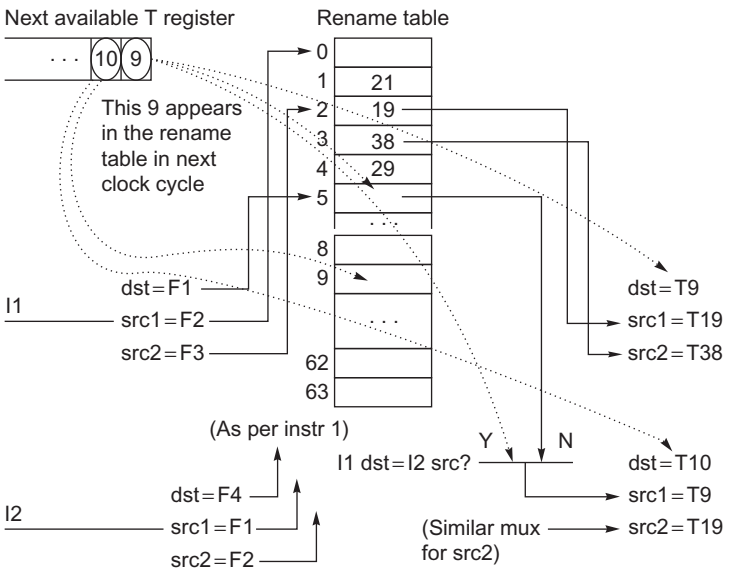


Figure 3.51 Initial state of the register renaming table.

3.9 [5] <3.4> If you ever get confused about what a register renamer has to do, go back to the assembly code you're executing, and ask yourself what has to happen for the right result to be obtained. For example, consider a three-way superscalar machine renaming these three instructions concurrently:

```
addi x1, x1, x1
addi x1, x1, x1
addi x1, x1, x1
```

If the value of `x1` starts out as 5, what should its value be when this sequence has executed?

3.10 [20] <3.4, 3.7> Very long instruction word (VLIW) designers have a few basic choices to make regarding architectural rules for register use. Suppose a VLIW is designed with self-draining execution pipelines: once an operation is initiated, its results will appear in the destination register at most L cycles later (where L is the latency of the operation). There are never enough registers, so there is a temptation to wring maximum use out of the registers that exist. Consider Figure 3.52. If loads have a 1 + 2 cycle latency, unroll this loop once, and show how a VLIW capable of two loads and two adds per cycle can use the minimum number of registers, in the absence of any pipeline interruptions or stalls. Give an example of an event that, in the presence of self-draining pipelines, could disrupt this pipelining and yield wrong results.

3.11 [10/10/10] <3.3> Assume a five-stage single-pipeline microarchitecture (fetch, decode, execute, memory, write-back) and the code in Figure 3.53. All ops are one cycle except `LW` and `SW`, which are 1 + 2 cycles, and branches, which are 1 + 1 cycles. There is no forwarding. Show the phases of each instruction per clock cycle for one iteration of the loop.

a. [10] <3.3> How many clock cycles per loop iteration are lost to branch overhead?

b. [10] <3.3> Assume a static branch predictor, capable of recognizing a backward branch in the Decode stage. Now how many clock cycles are wasted on branch overhead?

```
Loop:   lw      x1,0(x2);     lw      x3,8(x2)
        <stall>
        <stall>
        addi    x10,x1,1;     addi    x11,x3,1
        sw      x1,0(x2);     sw      x3,8(x2)
        addi    x2,x2,8
        sub     x4,x3,x2
        bnz     x4,Loop
```

Figure 3.52 Sample VLIW code with two adds, two loads, and two stalls.

```
Loop:   lw x1,0(x2)
        addi    x1,x1, 1
        sw      x1,0(x2)
        addi    x2,x2,4
        sub     x4,x3,x2
        bnz     x4,Loop
```

Figure 3.53 Code loop for Exercise 3.11.

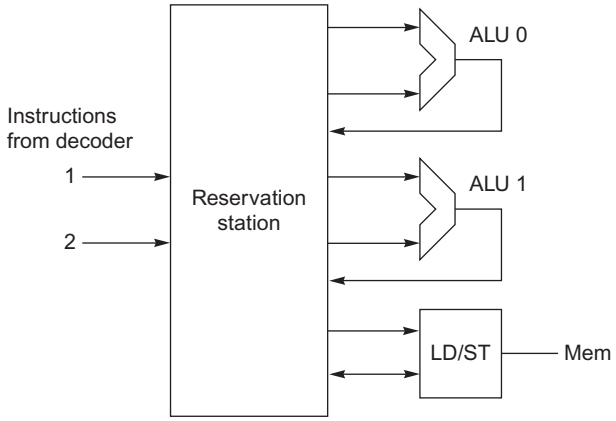


Figure 3.54 Microarchitecture for Exercise 3.12.

c. [10] <3.3> Assume a dynamic branch predictor. How many cycles are lost on a correct prediction?

3.12 [15/20/20/10/20] <3.4, 3.6> Let's consider what dynamic scheduling might achieve here. Assume a microarchitecture as shown in Figure 3.54. Assume that the arithmetic-logical units (ALUs) can do all arithmetic ops (`fmul.d`, `fdiv.d`, `fadd.d`, `addi`, `sub`) and branches, and that the Reservation Station (RS) can dispatch, at most, one operation to each functional unit per cycle (one op to each ALU plus one memory op to the `fld`/`fsd`).

a. [15] <3.4> Suppose all of the instructions from the sequence in Figure 3.47 are present in the RS, with no renaming having been done. Highlight any instructions in the code where register renaming would improve performance. (*Hint:* look for read-after-write and write-after-write hazards. Assume the same functional unit latencies as in Figure 3.47.)

b. [20] <3.4> Suppose the register-renamed version of the code from part (a) is resident in the RS in clock cycle *N*, with latencies as given in Figure 3.47. Show how the RS should dispatch these instructions out of order, clock by clock, to

obtain optimal performance on this code. (Assume the same RS restrictions as in part (a). Also assume that results must be written into the RS before they're available for use—no bypassing.) How many clock cycles does the code sequence take?

c. [20] <3.4> Part (b) lets the RS try to optimally schedule these instructions. But in reality, the whole instruction sequence of interest is not usually present in the RS. Instead, various events clear the RS, and as a new code sequence streams in from the decoder, the RS must choose to dispatch what it has. Suppose that the RS is empty. In cycle 0, the first two register-renamed instructions of this sequence appear in the RS. Assume it takes one clock cycle to dispatch any op, and assume functional unit latencies are as they were for Exercise 3.2. Further assume that the front end (decoder/register-renamer) will continue to supply two new instructions per clock cycle. Show the cycle-by-cycle order of dispatch of the RS. How many clock cycles does this code sequence require now?

d. [10] <3.4> If you wanted to improve the results of part (c), which would have helped most: (1) Another ALU? (2) Another LD/ST unit? (3) Full bypassing of ALU results to subsequent operations? or (4) Cutting the longest latency in half? What's the speedup?

e. [20] <3.6> Now let's consider speculation, the act of fetching, decoding, and executing beyond one or more conditional branches. Our motivation to do this is twofold: the dispatch schedule we came up with in part (c) had lots of nops, and we know computers spend most of their time executing loops (which implies the branch back to the top of the loop is pretty predictable). Loops tell us where to find more work to do; our sparse dispatch schedule suggests we have opportunities to do some of that work earlier than before. In part (d) you found the critical path through the loop. Imagine folding a second copy of that path onto the schedule you got in part (b). How many more clock cycles would be required to do two loops' worth of work (assuming all instructions are resident in the RS)? (Assume all functional units are fully pipelined.)

Exercises

3.13 [25] <3.7, 3.8> In this exercise, you will explore performance trade-offs between three processors that each employ different types of multithreading (MT). Each of these processors is superscalar, uses in-order pipelines, requires a fixed three-cycle stall following all loads and branches, and has identical L1 caches. Instructions from the same thread issued in the same cycle are read in program order and must not contain any data or control dependences.

- Processor A is a superscalar simultaneous MT architecture, capable of issuing up to two instructions per cycle from two threads.
- Processor B is a fine-grained MT architecture, capable of issuing up to four instructions per cycle from a single thread and switches threads on any pipeline stall.

- Processor C is a coarse-grained MT architecture, capable of issuing up to eight instructions per cycle from a single thread and switches threads on an L1 cache miss.

Our application is a list searcher, which scans a region of memory for a specific value stored in R9 between the address range specified in R16 and R17. It is parallelized by evenly dividing the search space into four equal-sized contiguous blocks and assigning one search thread to each block (yielding four threads). Most of each thread's runtime is spent in the following unrolled loop body:

```
loop: lw x1,0(x16)
      lw x2,8(x16)
      lw x3,16(x16)
      lw x4,24(x16)
      lw x5,32(x16)
      lw x6,40(x16)
      lw x7,48(x16)
      lw x8,56(x16)
      beq x9,x1,match0
      beq x9,x2,match1
      beq x9,x3,match2
      beq x9,x4,match3
      beq x9,x5,match4
      beq x9,x6,match5
      beq x9,x7,match6
      beq x9,x8,match7
      DADDIU x16,x16,#64
      blt x16,x17,loop
```

Assume the following:

- A barrier is used to ensure that all threads begin simultaneously.
- The first L1 cache miss occurs after two iterations of the loop.
- None of the BEQAL branches is taken.
- The BLT is always taken.
- All three processors schedule threads in a round-robin fashion.

Determine how many cycles are required for each processor to complete the first two iterations of the loop.

3.14 [25/25/25] <3.2, 3.7> In this exercise, we look at how software techniques can extract instruction-level parallelism (ILP) in a common vector loop. The following loop is the so-called DAXPY loop (double-precision aX plus Y) and is the central operation in Gaussian elimination. The following code implements the DAXPY operation, $Y = aX + Y$, for a vector length 100. Initially, R1 is set to the base address of array X and R2 is set to the base address of Y:

```
addi   x4,x1,#800 ; x1 = upper bound for X
```

```
foo: fld      F2,0(x1)   ;(F2) = X(i)
     fmul.d   F4,F2,F0   ;(F4) = a*X(i)
     fld      F6,0(x2)   ;(F6) = Y(i)
     fadd.d   F6,F4,F6   ;(F6) = a*X(i) + Y(i)
     fsd      F6,0(x2)   ;Y(i) = a*X(i) + Y(i)
     addi     x1,x1,#8   ;increment X index
     addi     x2,x2,#8   ;increment Y index
     sltu     x3,x1,x4   ;test: continue loop?
     bnez     x3,foo     ;loop if needed
```

Assume the functional unit latencies as shown in the following table. Assume a one-cycle delayed branch that resolves in the ID stage. Assume that results are fully bypassed.

Instruction producing result	Instruction using result	Latency in clock cycles
FP multiply	FP ALU op	6
FP add	FP ALU op	4
FP multiply	FP store	5
FP add	FP store	4
Integer operations and all loads	Any	2

a. [25] <3.2> Assume a single-issue pipeline. Show how the loop would look both unscheduled by the compiler and after compiler scheduling for both floating-point operation and branch delays, including any stalls or idle clock cycles. What is the execution time (in cycles) per element of the result vector, Y, unscheduled and scheduled? How much faster must the clock be for processor hardware alone to match the performance improvement achieved by the scheduling compiler? (Neglect any possible effects of increased clock speed on memory system performance.)

b. [25] <3.2> Assume a single-issue pipeline. Unroll the loop as many times as necessary to schedule it without any stalls, collapsing the loop overhead instructions. How many times must the loop be unrolled? Show the instruction schedule. What is the execution time per element of the result?

c. [25] <3.7> Assume a VLIW processor with instructions that contain five operations, as shown in Figure 3.20. We will compare two degrees of loop unrolling. First, unroll the loop 6 times to extract ILP and schedule it without any stalls (i.e., completely empty issue cycles), collapsing the loop overhead instructions, and then repeat the process but unroll the loop 10 times. Ignore the branch delay slot. Show the two schedules. What is the execution time per element of the result vector for each schedule? What percent of the operation slots are used in each schedule? How much does the size of the code differ between the two schedules? What is the total register demand for the two schedules?

3.15 [20/20] <3.4, 3.5, 3.7, 3.8> In this exercise, we will look at how variations on Tomasulo's algorithm perform when running the loop from Exercise 3.14. The functional units (FUs) are described in the following table.

FU type	Cycles in EX	Number of FUs	Number of reservation stations
Integer	1	1	5
FP adder	10	1	3
FP multiplier	15	1	2

Assume the following:

- Functional units are not pipelined.
- There is no forwarding between functional units; results are communicated by the common data bus (CDB).
- The execution stage (EX) does both the effective address calculation and the memory access for loads and stores. Thus, the pipeline is IF/ID/IS/EX/WB.
- Loads require one clock cycle.
- The issue (IS) and write-back (WB) result stages each require one clock cycle.
- There are five load buffer slots and five store buffer slots.
- Assume that the Branch on Not Equal to Zero (BNEZ) instruction requires one clock cycle.

a. [20] <3.4–3.5> For this problem use the single-issue Tomasulo MIPS pipeline of Figure 3.10 with the pipeline latencies from the preceding table. Show the number of stall cycles for each instruction and what clock cycle each instruction begins execution (i.e., enters its first EX cycle) for three iterations of the loop. How many cycles does each loop iteration take? Report your answer in the form of a table with the following column headers:
   - Iteration (loop iteration number)
   - Instruction
   - Issues (cycle when instruction issues)
   - Executes (cycle when instruction executes)
   - Memory access (cycle when memory is accessed)
   - Write CDB (cycle when result is written to the CDB)
   - Comment (description of any event on which the instruction is waiting)

   Show three iterations of the loop in your table. You may ignore the first instruction.

b. [20] <3.7, 3.8> Repeat part (a) but this time assume a two-issue Tomasulo algorithm and a fully pipelined floating-point unit (FPU).

3.16 [10] <3.4> Tomasulo's algorithm has a disadvantage: only one result can compute per clock per CDB. Use the hardware configuration and latencies from the previous question and find a code sequence of no more than 10 instructions where Tomasulo's algorithm must stall due to CDB contention. Indicate where this occurs in your sequence.

3.17 [20] <3.3> An (m,n) correlating branch predictor uses the behavior of the most recent m executed branches to choose from 2^m predictors, each of which is an n-bit predictor. A two-level local predictor works in a similar fashion, but only keeps track of the past behavior of each individual branch to predict future behavior.

There is a design trade-off involved with such predictors: correlating predictors require little memory for history, which allows them to maintain 2-bit predictors for a large number of individual branches (reducing the probability of branch instructions reusing the same predictor), while local predictors require substantially more memory to keep history and are thus limited to tracking a relatively small number of branch instructions. For this exercise, consider a (1,2) correlating predictor that can track four branches (requiring 16 bits) versus a (1,2) local predictor that can track two branches using the same amount of memory. For the following branch outcomes, provide each prediction, the table entry used to make the prediction, any updates to the table as a result of the prediction, and the final misprediction rate of each predictor. Assume that all branches up to this point have been taken. Initialize each predictor to the following:

Correlating predictor

Entry	Branch	Last outcome	Prediction
0	0	T	T with one misprediction
1	0	NT	NT
2	1	T	NT
3	1	NT	T
4	2	T	T
5	2	NT	T
6	3	T	NT with one misprediction
7	3	NT	NT

Local predictor

Entry	Branch	Last 2 outcomes (right is most recent)	Prediction
0	0	T,T	T with one misprediction
1	0	T,NT	NT
2	0	NT,T	NT
3	0	NT	T
4	1	T,T	T
5	1	T,NT	T with one misprediction
6	1	NT,T	NT
7	1	NT,NT	NT

Branch PC (word address)	Outcome
454	T
543	NT
777	NT
543	NT
777	NT
454	T
777	NT
454	T
543	T

3.18 [10] <3.9> Suppose we have a deeply pipelined processor, for which we implement a branch-target buffer for the conditional branches only. Assume that the misprediction penalty is always four cycles and the buffer miss penalty is always three cycles. Assume a 90% hit rate, 90% accuracy, and 15% branch frequency. How much faster is the processor with the branch-target buffer versus a processor that has a fixed two-cycle branch penalty? Assume a base clock cycle per instruction (CPI) without branch stalls of one.

3.19 [10/5] <3.9> Consider a branch-target buffer that has penalties of zero, two, and two clock cycles for correct conditional branch prediction, incorrect prediction, and a buffer miss, respectively. Consider a branch-target buffer design that distinguishes conditional and unconditional branches, storing the target address for a conditional branch and the target instruction for an unconditional branch.

a. [10] <3.9> What is the penalty in clock cycles when an unconditional branch is found in the buffer?

b. [10] <3.9> Determine the improvement from branch folding for unconditional branches. Assume a 90% hit rate, an unconditional branch frequency of 5%, and a two-cycle penalty for a buffer miss. How much improvement is gained by this enhancement? How high must the hit rate be for this enhancement to provide a performance gain?

This page intentionally left blank

4.1	Introduction	282
4.2	Vector Architecture	283
4.3	SIMD Instruction Set Extensions for Multimedia	304
4.4	Graphics Processing Units	310
4.5	Detecting and Enhancing Loop-Level Parallelism	336
4.6	Cross-Cutting Issues	345
4.7	Putting It All Together: Embedded Versus Server GPUs and Tesla Versus Core i7	346
4.8	Fallacies and Pitfalls	353
4.9	Concluding Remarks	357
4.10	Historical Perspective and References	357
	Case Study and Exercises by Jason D. Bakos	357

4

Data-Level Parallelism in Vector, SIMD, and GPU Architectures

We call these algorithms *data parallel* algorithms because their parallelism comes from simultaneous operations across large sets of data rather than from multiple threads of control.

W. Daniel Hillis and Guy L. Steele,
"Data parallel algorithms," *Commun. ACM* (1986)

If you were plowing a field, which would you rather use: two strong oxen or 1024 chickens?

Seymour Cray, Father of the Supercomputer
(arguing for two powerful vector processors versus many simple processors)

Computer Architecture. https://doi.org/10.1016/B978-0-12-811905-1.00004-3
© 2019 Elsevier Inc. All rights reserved.

4.1 Introduction

A question for the single instruction multiple data (SIMD) architecture, which Chapter 1 introduced, has always been just how wide a set of applications has significant data-level parallelism (DLP). Five years after the SIMD classification was proposed (Flynn, 1966), the answer is not only the matrix-oriented computations of scientific computing but also the media-oriented image and sound processing and machine learning algorithms, as we will see in Chapter 7. Since a multiple instruction multiple data (MIMD) architecture needs to fetch one instruction per data operation, single instruction multiple data (SIMD) is potentially more energy-efficient since a single instruction can launch many data operations. These two answers make SIMD attractive for personal mobile devices as well as for servers. Finally, perhaps the biggest advantage of SIMD versus MIMD is that the programmer continues to think sequentially yet achieves parallel speedup by having parallel data operations.

This chapter covers three variations of SIMD: vector architectures, multimedia SIMD instruction set extensions, and graphics processing units (GPUs).[1]

The first variation, which predates the other two by more than 30 years, extends pipelined execution of many data operations. These *vector architectures* are easier to understand and to compile to than other SIMD variations, but they were considered too expensive for microprocessors until recently. Part of that expense was in transistors, and part was in the cost of sufficient dynamic random access memory (DRAM) bandwidth, given the widespread reliance on caches to meet memory performance demands on conventional microprocessors.

The second SIMD variation borrows from the SIMD name to mean basically simultaneous parallel data operations and is now found in most instruction set architectures that support multimedia applications. For x86 architectures, the SIMD instruction extensions started with the MMX (multimedia extensions) in 1996, which were followed by several SSE (streaming SIMD extensions) versions in the next decade, and they continue until this day with AVX (advanced vector extensions). To get the highest computation rate from an x86 computer, you often need to use these SIMD instructions, especially for floating-point programs.

The third variation on SIMD comes from the graphics accelerator community, offering higher potential performance than is found in traditional multicore computers today. Although GPUs share features with vector architectures, they have their own distinguishing characteristics, in part because of the ecosystem in which they evolved. This environment has a system processor and system memory in addition to the GPU and its graphics memory. In fact, to recognize those distinctions, the GPU community refers to this type of architecture as *heterogeneous*.

[1] This chapter is based on material in Appendix F, "Vector Processors," by Krste Asanovic, and Appendix G, "Hardware and Software for VLIW and EPIC" from the 5th edition of this book; on material in Appendix A, "Graphics and Computing GPUs," by John Nickolls and David Kirk, from the 5th edition of *Computer Organization and Design*; and to a lesser extent on material in "Embracing and Extending 20th-Century Instruction Set Architectures," by Joe Gebis and David Patterson, *IEEE Computer*, April 2007.

For problems with lots of data parallelism, all three SIMD variations share the advantage of being easier on the programmer than classic parallel MIMD programming.

The goal of this chapter is for architects to understand why vector is more general than multimedia SIMD, as well as the similarities and differences between vector and GPU architectures. Because vector architectures are supersets of the multimedia SIMD instructions, including a better model for compilation, and because GPUs share several similarities with vector architectures, we start with vector architectures to set the foundation for the following two sections. The next section introduces vector architectures, and Appendix G goes much deeper into the subject.

4.2 Vector Architecture

The most efficient way to execute a vectorizable application is a vector processor.

Jim Smith,
International Symposium on Computer Architecture (1994)

Vector architectures grab sets of data elements scattered about memory, place them into large sequential register files, operate on data in those register files, and then disperse the results back into memory. A single instruction works on vectors of data, which results in dozens of register-register operations on independent data elements.

These large register files act as compiler-controlled buffers, both to hide memory latency and to leverage memory bandwidth. Because vector loads and stores are deeply pipelined, the program pays the long memory latency only once per vector load or store versus once per element, thus amortizing the latency over, say, 32 elements. Indeed, vector programs strive to keep the memory busy.

The power wall leads architects to value architectures that can deliver good performance without the energy and design complexity costs of highly out-of-order superscalar processors. Vector instructions are a natural match to this trend because architects can use them to increase performance of simple in-order scalar processors without greatly raising energy demands and design complexity. In practice, developers can express many of the programs that ran well on complex out-of-order designs more efficiently as data-level parallelism in the form of vector instructions, as Kozyrakis and Patterson (2002) showed.

RV64V Extension

We begin with a vector processor consisting of the primary components that Figure 4.1 shows. It is loosely based on the 40-year-old Cray-1, which was one of the first supercomputers. At the time of the writing of this edition, the RISC-V vector instruction set extension RVV was still under development. (The vector extension by itself is called RVV, so RV64V refers to the RISC-V base instructions

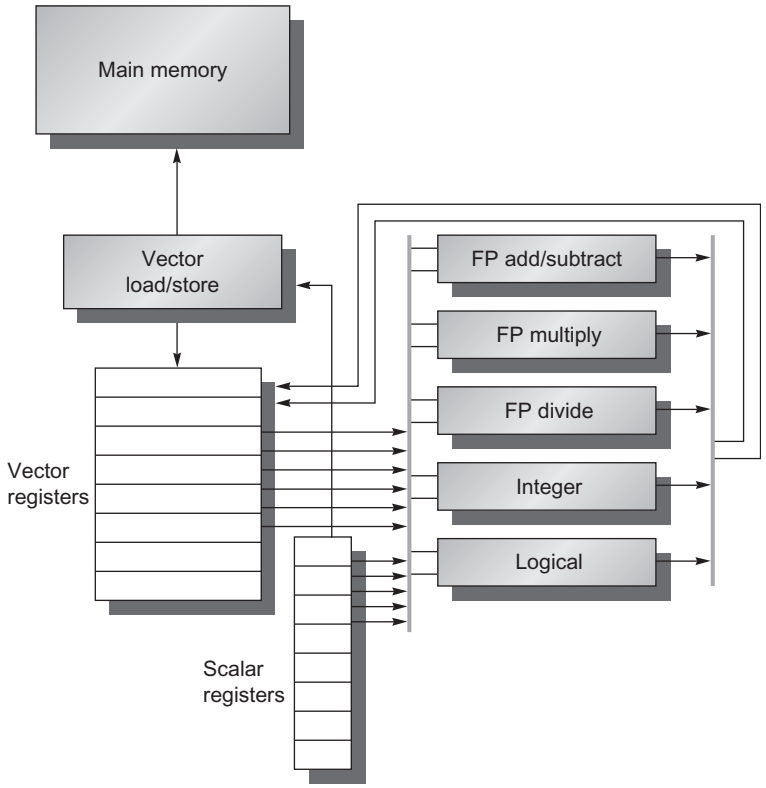


Figure 4.1 The basic structure of a vector architecture, RV64V, which includes a RISC-V scalar architecture. There are also 32 vector registers, and all the functional units are vector functional units. The vector and scalar registers have a significant number of read and write ports to allow multiple simultaneous vector operations. A set of crossbar switches *(thick gray lines)* connects these ports to the inputs and outputs of the vector functional units.

plus the vector extension.) We show a subset of RV64V, trying to capture its essence in a few pages.

The primary components of the instruction set architecture of RV64V are the following:

- *Vector registers*—Each vector register holds a single vector, and RV64V has 32 of them, each 64 bits wide. The vector register file needs to provide enough ports to feed all the vector functional units. These ports will allow a high degree of overlap among vector operations to different vector registers. The read and write ports, which total at least 16 read ports and 8 write ports, are connected to the functional unit inputs or outputs by a pair of crossbar switches. One way to

increase the register file bandwidth is to compose it from multiple banks, which work well with relatively long vectors.

- *Vector functional units*—Each unit is fully pipelined in our implementation, and it can start a new operation on every clock cycle. A control unit is needed to detect hazards, both structural hazards for functional units and data hazards on register accesses. Figure 4.1 shows that we assume an implementation of RV64V has five functional units. For simplicity, we focus on the floating-point functional units in this section.
- *Vector load/store unit*—The vector memory unit loads or stores a vector to or from memory. The vector loads and stores are fully pipelined in our hypothetical RV64V implementation so that words can be moved between the vector registers and memory with a bandwidth of one word per clock cycle, after an initial latency. This unit would also normally handle scalar loads and stores.
- *A set of scalar registers*—Scalar registers can likewise provide data as input to the vector functional units, as well as compute addresses to pass to the vector load/store unit. These are the normal 31 general-purpose registers and 32 floating-point registers of RV64G. One input of the vector functional units latches scalar values as they are read out of the scalar register file.

Figure 4.2 lists the RV64V vector instructions we use in this section. The description in Figure 4.2 assumes that the input operands are all vector registers, but there are also versions of these instructions where an operand can be a scalar register (`xi` or `fi`). RV64V uses the suffix `.vv` when both are vectors, `.vs` when the second operand is a scalar, and `.sv` when the first is a scalar register. Thus these three are all valid RV64V instructions: `vsub.vv`, `vsub.vs`, and `vsub.sv`. (Add and other commutative operations have only the first two versions, as `vadd.sv` and `vadd.sv` would be redundant.) Because the operands determine the version of the instruction, we usually let the assembler supply the appropriate suffix. The vector functional unit gets a copy of the scalar value at instruction issue time.

Although the traditional vector architectures didn't support narrow data types efficiently, vectors naturally accommodate varying data sizes (Kozyrakis and Patterson, 2002). Thus, if a vector register has 32 64-bit elements, then 128×16-bit elements, and even 256×8-bit elements are equally valid views. Such hardware multiplicity is why a vector architecture can be useful for multimedia applications as well as for scientific applications.

Note that the RV64V instructions in Figure 4.2 omit the data type and size! An innovation of RV64V is to associate a data type and data size *with each vector register*, rather than the normal approach of the instruction supplying that information. Thus, before executing the vector instructions, a program configures the vector registers being used to specify their data type and widths. Figure 4.3 lists the options for RV64V.

Mnemonic	Name	Description
vadd	ADD	Add elements of V[rs1] and V[rs2], then put each result in V[rd]
vsub	SUBtract	Subtract elements of V[rs2] frpm V[rs1], then put each result in V[rd]
vmul	MULtiply	Multiply elements of V[rs1] and V[rs2], then put each result in V[rd]
vdiv	DIVide	Divide elements of V[rs1] by V[rs2], then put each result in V[rd]
vrem	REMainder	Take remainder of elements of V[rs1] by V[rs2], then put each result in V[rd]
vsqrt	SQuare RooT	Take square root of elements of V[rs1], then put each result in V[rd]
vsll	Shift Left	Shift elements of V[rs1] left by V[rs2], then put each result in V[rd]
vsrl	Shift Right	Shift elements of V[rs1] right by V[rs2], then put each result in V[rd]
vsra	Shift Right Arithmetic	Shift elements of V[rs1] right by V[rs2] while extending sign bit, then put each result in V[rd]
vxor	XOR	Exclusive OR elements of V[rs1] and V[rs2], then put each result in V[rd]
vor	OR	Inclusive OR elements of V[rs1] and V[rs2], then put each result in V[rd]
vand	AND	Logical AND elements of V[rs1] and V[rs2], then put each result in V[rd]
vsgnj	SiGN source	Replace sign bits of V[rs1] with sign bits of V[rs2], then put each result in V[rd]
vsgnjn	Negative SiGN source	Replace sign bits of V[rs1] with complemented sign bits of V[rs2], then put each result in V[rd]
vsgnjx	Xor SiGN source	Replace sign bits of V[rs1] with xor of sign bits of V[rs1] and V[rs2], then put each result in V[rd]
vld	Load	Load vector register V[rd] from memory starting at address R[rs1]
vlds	Strided Load	Load V[rd] from address at R[rs1] with stride in R[rs2] (i.e., R[rs1] + i × R[rs2])
vldx	Indexed Load (Gather)	Load V[rs1] with vector whose elements are at R[rs2] + V[rs2] (i.e., V[rs2] is an index)
vst	Store	Store vector register V[rd] into memory starting at address R[rs1]
vsts	Strided Store	Store V[rd] into memory at address R[rs1] with stride in R[rs2] (i.e., R[rs1] + i × R[rs2])
vstx	Indexed Store (Scatter)	Store V[rs1] into memory vector whose elements are at R[rs2] + V[rs2] (i.e., V[rs2] is an index)
vpeq	Compare =	Compare elements of V[rs1] and V[rs2]. When equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vpne	Compare !=	Compare elements of V[rs1] and V[rs2]. When not equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vplt	Compare <	Compare elements of V[rs1] and V[rs2]. When less than, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vpxor	Predicate XOR	Exclusive OR 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
vpor	Predicate OR	Inclusive OR 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
vpand	Predicate AND	Logical AND 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
setvl	Set Vector Length	Set vl and the destination register to the smaller of mvl and the source regsiter

Figure 4.2 The RV64V vector instructions. All use the R instruction format. Each vector operation with two operands is shown with both operands being vector (.vv), but there are also versions where the second operand is a scalar register (.vs) and, when it makes a difference, where the first operand is a scalar register and the second is a vector register (.sv). The type and width of the operands are determined by configuring each vector register rather than being supplied by the instruction. In addition to the vector registers and predicate registers, there are two vector control and status registers (CSRs), vl and vctype, discussed below. The strided and indexed data transfers are also explained later. Once completed, RV64 will surely have more instructions, but the ones in this figure will be included.

Integer	8, 16, 32, and 64 bits	Floating point	16, 32, and 64 bits

Figure 4.3 Data sizes supported for RV64V assuming it also has the single- and double-precision floating-point extensions RVS and RVD. Adding RVV to such a RISC-V design means the scalar unit must also add RVH, which is a scalar instruction extension to support half-precision (16-bit) IEEE 754 floating point. Because RV32V would not have doubleword scalar operations, it could drop 64-bit integers from the vector unit. If a RISC-V implementation didn't include RVS or RVD, it could omit the vector floating-point instructions.

One reason for *dynamic register typing* is that many instructions are required for a conventional vector architecture that supports such variety. Given the combinations of data types and sizes in Figure 4.3, if not for dynamic register typing, Figure 4.2 would be several pages long!

Dynamic typing also lets programs disable unused vector registers. As a consequence, enabled vector registers are allocated all the vector memory as long vectors. For example, assume we have 1024 bytes of vector memory, if 4 vector registers are enabled and they are type 64–bit floats, the processor would give each vector register 256 bytes or 256/8=32 elements. This valiue is called the maximum vector length (mvl), which is set by the processor and cannot be changed by sofware.

One complaint about vector architectures is that their larger state means slower context switch time. Our implementation of RV64V increases state a factor of 3: from $2 \times 32 \times 8 = 512$ bytes to $2 \times 32 \times 1024 = 1536$ bytes. A pleasant side effect of dynamic register typing is that the program can configure vector registers as *disabled* when they are not being used, so there is no need to save and restore them on a context switch.

A third benefit of dynamic register typing is that conversions between different size operands can be implicit depending on the configuration of the registers rather than as additional explicit conversion instructions. We'll see an example of this benefit in the next section.

The names `vld` and `vst` denote vector load and vector store, and they load or store an entire vectors of data. One operand is the vector register to be loaded or stored; the other operand, which is a RV64G general-purpose register, is the starting address of the vector in memory. Vector needs more registers beyond the vector registers themselves. The vector-length register `vl` is used when the natural vector length is not equal to mvl, the vector-type register `vctype` records register types, and the predicate registers $\mathtt{p}_i$ are used when loops involve IF statements. We'll see them in action in the following example.

With a vector instruction, the system can perform the operations on the vector data elements in many ways, including operating on many elements simultaneously. This flexibility lets vector designs use slow but wide execution units to achieve high performance at low power. Furthermore, the independence of elements within a vector instruction set allows scaling of functional units without performing additional costly dependency checks, as superscalar processors require.

How Vector Processors Work: An Example

We can best understand a vector processor by looking at a vector loop for RV64V. Let's take a typical vector problem, which we use throughout this section:

```
Y = a × X + Y
```

`X` and `Y` are vectors, initially resident in memory, and `a` is a scalar. This problem is the *SAXPY* or *DAXPY* loop that forms the inner loop of the Linpack benchmark (Dongarra et al., 2003). (SAXPY stands for single-precision a × X plus Y, and DAXPY for double precision a × X plus Y.) Linpack is a collection of linear algebra routines, and the Linpack benchmark consists of routines for performing Gaussian elimination.

For now, let us assume that the number of elements, or *length*, of a vector register (32) matches the length of the vector operation we are interested in. (This restriction will be lifted shortly.)

Example Show the code for RV64G and RV64V for the DAXPY loop. For this example, assume that X and Y have 32 elements and the starting addresses of `X` and `Y` are in `x5` and `x6`, respectively. (A subsequent example covers when they do not have 32 elements.)

Answer Here is the RISC-V code:

```
        fld    f0,a           # Load scalar a
        addi   x28,x5,#256    # Last address to load
Loop:   fld    f1,0(x5)       # Load X[i]
        fmul.d f1,f1,f0       # a × X[i]
        fld    f2,0(x6)       # Load Y[i]
        fadd.d f2,f2,f1       # a × X[i] + Y[i]
        fsd    f2,0(x6)       # Store into Y[i]
        addi   x5,x5,#8       # Increment index to X
        addi   x6,x6,#8       # Increment index to Y
        bne    x28,x5,Loop    # Check if done
```

Here is the RV64V code for DAXPY:

```
        vsetdcfg  4*FP64      # Enable 4 DP FP vregs
        fld       f0,a        # Load scalar a
        vld       v0,x5       # Load vector X
        vmul      v1,v0,f0    # Vector-scalar mult
        vld       v2,x6       # Load vector Y
        vadd      v3,v1,v2    # Vector-vector add
        vst       v3,x6       # Store the sum
        vdisable              # Disable vector regs
```

Note that the assembler determines which version of the vector operations to generate. Because the multiply has a scalar operand, it generates `vmul.vs`, whereas the add doesn't, so it generates `vadd.vv`.

The initial instruction configures the first four vector registers to hold 64-bit floating-point data. The last instruction disables all vector registers. If a context switch happened after the last instruction, there is no additional state to save.

The most dramatic difference between the preceding scalar and vector code is that the vector processor greatly reduces the dynamic instruction bandwidth, executing only 8 instructions versus 258 for RV64G. This reduction occurs because the vector operations work on 32 elements and the overhead instructions that constitute nearly half the loop on RV64G are not present in the RV64V code. When the compiler produces vector instructions for such a sequence, and the resulting code spends much of its time running in vector mode, the code is said to be *vectorized* or *vectorizable*. Loops can be vectorized when they do not have dependences between iterations of a loop, which are called *loop-carried dependences* (see Section 4.5).

Another important difference between RV64G and RV64V is the frequency of pipeline interlocks for a simple implementation of RV64G. In the straightforward RV64G code, every `fadd.d` must wait for a `fmul.d`, and every `fsd` must wait for the `fadd.d`. On the vector processor, each vector instruction will stall only for the first element in each vector, and then subsequent elements will flow smoothly down the pipeline. Thus pipeline stalls are required only once per vector *instruction*, rather than once per vector *element*. Vector architects call forwarding of element-dependent operations *chaining*, in that the dependent operations are "chained" together. In this example, the pipeline stall frequency on RV64G will be about 32 × higher than it is on RV64V. Software pipelining, loop unrolling (Appendix H), or out-of-order execution can reduce the pipeline stalls on RV64G; however, the large difference in instruction bandwidth cannot be reduced substantially.

Let's show off the dynamic register typing before discussing performance of the code.

Example A common use of multiply-accumulate operations is to multiply using narrow data and to accumulate at a wider size to increase the accuracy of a sum of products. Show how the preceding code would change if `X` and `a` were single-precision instead of a double-precision floating point. Next, show the changes to this code if we switch `X`, `Y`, and `a` from floating-point type to integers.

Answer The changes are underlined in the following code. Amazingly, the same code works with two small changes: the configuration instruction includes one single-precision vector, and the scalar load is now single-precision:

```
vsetdcfg 1*FP32,3*FP64  # 1 32b, 3 64b vregs
flw      f0,a           # Load scalar a
vld      v0,x5          # Load vector X
vmul     v1,v0,f0       # Vector-scalar mult
vld      v2,x6          # Load vector Y
vadd     v3,v1,v2       # Vector-vector add
vst      v3,x6          # Store the sum
vdisable                # Disable vector regs
```

Note that RV64V hardware will implicitly perform a conversion from the narrower single-precision to the wider double-precision in this setup.

The switch to integers is almost as easy, but we must now use an integer load instruction and integer register to hold the scalar value:

```
vsetdcfg 1*X32,3*X64  # 1 32b, 3 64b int reg
lw       x7,a         # Load scalar a
vld      v0,x5        # Load vector X
vmul     v1,v0,x7     # Vector-scalar mult
vld      v2,x6        # Load vector Y
vadd     v3,v1,v2     # Vector-vector add
vst      v3,x6        # Store the sum
vdisable              # Disable vector regs
```

Vector Execution Time

The execution time of a sequence of vector operations primarily depends on three factors: (1) the length of the operand vectors, (2) structural hazards among the operations, and (3) the data dependences. Given the vector length and the *initiation rate*, which is the rate at which a vector unit consumes new operands and produces new results, we can compute the time for a single vector instruction.

All modern vector computers have vector functional units with multiple parallel pipelines (or *lanes*) that can produce two or more results per clock cycle, but they may also have some functional units that are not fully pipelined. For simplicity, our RV64V implementation has one lane with an initiation rate of one element per clock cycle for individual operations. Thus the execution time in clock cycles for a single vector instruction is approximately the vector length.

To simplify the discussion of vector execution and vector performance, we use the notion of a *convoy*, which is the set of vector instructions that could potentially execute together. The instructions in a convoy *must not* contain any structural hazards; if such hazards were present, the instructions would need to be serialized and initiated in different convoys. Thus the `vld` and the following `vmul` in the preceding example can be in the same convoy. As we will soon see, you can estimate performance of a section of code by counting the number of convoys. To keep this analysis simple, we assume that a convoy of instructions must complete execution before any other instructions (scalar or vector) can begin execution.

It might seem that in addition to vector instruction sequences with structural hazards, sequences with read-after-write dependency hazards should also be in separate convoys. However, chaining allows them to be in the same convoy since it allows a vector operation to start as soon as the individual elements of its vector source operand become available: the results from the first functional unit in the chain are "forwarded" to the second functional unit. In practice, we often implement chaining by allowing the processor to read and write a particular vector register at the same time, albeit to different elements. Early implementations of

chaining worked just like forwarding in scalar pipelining, but this restricted the timing of the source and destination instructions in the chain. Recent implementations use *flexible chaining*, which allows a vector instruction to chain to essentially any other active vector instruction, assuming that we don't generate a structural hazard. All modern vector architectures support flexible chaining, which we assume throughout this chapter.

To turn convoys into execution time, we need a metric to estimate the length of a convoy. It is called a *chime*, which is simply the unit of time taken to execute one convoy. Thus a vector sequence that consists of m convoys executes in m chimes; for a vector length of n, for our simple RV64V implementation, this is approximately $m \times n$ clock cycles.

The chime approximation ignores some processor-specific overheads, many of which are dependent on vector length. Therefore measuring time in chimes is a better approximation for long vectors than for short ones. We will use the chime measurement, rather than clock cycles per result, to indicate explicitly that we are ignoring certain overheads.

If we know the number of convoys in a vector sequence, we know the execution time in chimes. One source of overhead ignored in measuring chimes is any limitation on initiating multiple vector instructions in a single clock cycle. If only one vector instruction can be initiated in a clock cycle (the reality in most vector processors), the chime count will underestimate the actual execution time of a convoy. Because the length of vectors is typically much greater than the number of instructions in the convoy, we will simply assume that the convoy executes in one chime.

Example Show how the following code sequence lays out in convoys, assuming a single copy of each vector functional unit:

```
vld   v0,x5          # Load vector X
vmul  v1,v0,f0       # Vector-scalar multiply
vld   v2,x6          # Load vector Y
vadd  v3,v1,v2       # Vector-vector add
vst   v3,x6          # Store the sum
```

How many chimes will this vector sequence take? How many cycles per FLOP (floating-point operation) are needed, ignoring vector instruction issue overhead?

Answer The first convoy starts with the first `vld` instruction. The `vmul` is dependent on the first `vld`, but chaining allows it to be in the same convoy.

The second `vld` instruction must be in a separate convoy because there is a structural hazard on the load/store unit for the prior `vld` instruction. The `vadd` is dependent on the second `vld`, but it can again be in the same convoy via chaining. Finally, the `vst` has a structural hazard on the `vld` in the second convoy, so it must go in the third convoy. This analysis leads to the following layout of vector instructions into convoys:

1. `vld  vmul`
2. `vld  vadd`
3. `vst`

The sequence requires three convoys. Because the sequence takes three chimes and there are two floating-point operations per result, the number of cycles per FLOP is 1.5 (ignoring any vector instruction issue overhead). Note that, although we allow the `vld` and `vmul` both to execute in the first convoy, most vector machines will take 2 clock cycles to initiate the instructions.

This example shows that the chime approximation is reasonably accurate for long vectors. For example, for 32-element vectors, the time in chimes is 3, so the sequence would take about 32×3 or 96 clock cycles. The overhead of issuing convoys in two separate clock cycles would be small.

Another source of overhead is far more significant than the issue limitation. The most important source of overhead ignored by the chime model is vector *start-up time*, which is the latency in clock cycles until the pipeline is full. The start-up time is principally determined by the pipelining latency of the vector functional unit. For RV64V, we will use the same pipeline depths as the Cray-1, although latencies in more modern processors have tended to increase, especially for vector loads. All functional units are fully pipelined. The pipeline depths are 6 clock cycles for floating-point add, 7 for floating-point multiply, 20 for floating-point divide, and 12 for vector load.

Given these vector basics, the next several sections will give optimizations that either improve the performance or increase the types of programs that can run well on vector architectures. In particular, they will answer these questions:

- How can a vector processor execute a single vector faster than one element per clock cycle? Multiple elements per clock cycle improve performance.
- How does a vector processor handle programs where the vector lengths are not the same as the maximum vector length (mvl)? Because most application vectors don't match the architecture vector length, we need an efficient solution to this common case.
- What happens when there is an IF statement inside the code to be vectorized? More code can vectorize if we can efficiently handle conditional statements.
- What does a vector processor need from the memory system? Without sufficient memory bandwidth, vector execution can be futile.
- How does a vector processor handle multiple dimensional matrices? This popular data structure must vectorize for vector architectures to do well.
- How does a vector processor handle sparse matrices? This popular data structure must vectorize also.

- How do you program a vector computer? Architectural innovations that are a mismatch to programming languages and their compilers may not get widespread use.

The rest of this section introduces each of these optimizations of the vector architecture, and Appendix G goes into greater depth.

Multiple Lanes: Beyond One Element per Clock Cycle

A critical advantage of a vector instruction set is that it allows software to pass a large amount of parallel work to hardware using only a single short instruction. One vector instruction can include scores of independent operations yet be encoded in the same number of bits as a conventional scalar instruction. The parallel semantics of a vector instruction allow an implementation to execute these elemental operations using a deeply pipelined functional unit, as in the RV64V implementation we've studied so far; an array of parallel functional units; or a combination of parallel and pipelined functional units. Figure 4.4 illustrates how to improve vector performance by using parallel pipelines to execute a vector add instruction.

The RV64V instruction set has the property that all vector arithmetic instructions only allow element *N* of one vector register to take part in operations with element *N* from other vector registers. This dramatically simplifies the design of a highly parallel vector unit, which can be structured as multiple parallel *lanes*. As with a traffic highway, we can increase the peak throughput of a vector unit by adding more lanes. Figure 4.5 shows the structure of a four-lane vector unit. Thus going to four lanes from one lane reduces the number of clocks for a chime from 32 to 8. For multiple lanes to be advantageous, both the applications and the architecture must support long vectors; otherwise, they will execute so quickly that you'll run out of instruction bandwidth, requiring ILP techniques (see Chapter 3) to supply enough vector instructions.

Each lane contains one portion of the vector register file and one execution pipeline from each vector functional unit. Each vector functional unit executes vector instructions at the rate of one element group per cycle using multiple pipelines, one per lane. The first lane holds the first element (element 0) for all vector registers, and so the first element in any vector instruction will have its source and destination operands located in the first lane. This allocation allows the arithmetic pipeline local to the lane to complete the operation without communicating with other lanes. Avoiding interlane communication reduces the wiring cost and register file ports required to build a highly parallel execution unit and helps explain why vector computers can complete up to 64 operations per clock cycle (2 arithmetic units and 2 load/store units across 16 lanes).

Adding multiple lanes is a popular technique to improve vector performance as it requires little increase in control complexity and does not require changes to existing machine code. It also allows designers to trade off die area, clock rate, voltage, and energy without sacrificing peak performance. If the clock rate of a

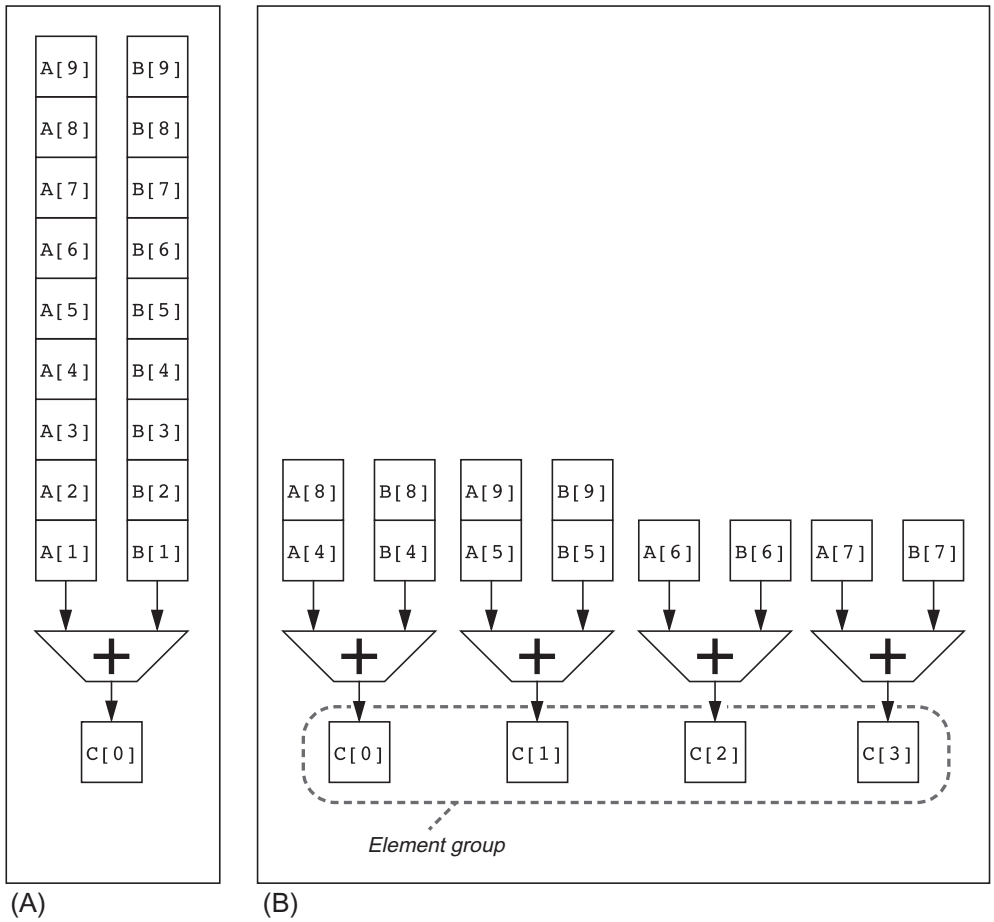


Figure 4.4 Using multiple functional units to improve the performance of a single vector add instruction, C = A + B. The vector processor (A) on the left has a single add pipeline and can complete one addition per clock cycle. The vector processor (B) on the right has four add pipelines and can complete four additions per clock cycle. The elements within a single vector add instruction are interleaved across the four pipelines. The set of elements that move through the pipelines together is termed an *element group*. Reproduced with permission from Asanovic, K., 1998. Vector Microprocessors (Ph.D. thesis). Computer Science Division, University of California, Berkeley.

vector processor is halved, doubling the number of lanes will retain the same peak performance.

Vector-Length Registers: Handling Loops Not Equal to 32

A vector register processor has a natural vector length determined by the maximum vector length (mvl). This length, which was 32 in our example above, is unlikely

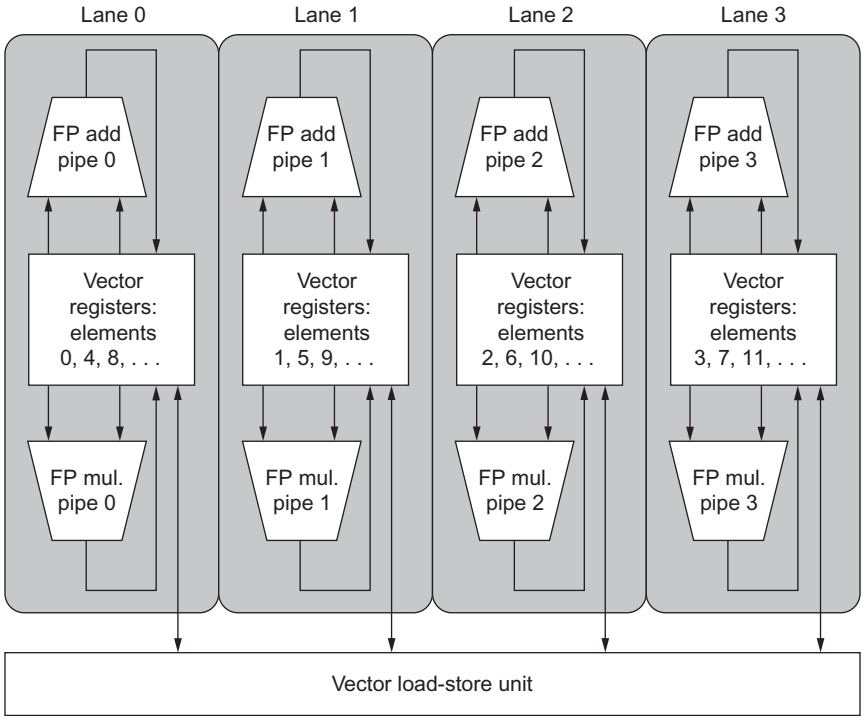


Figure 4.5 Structure of a vector unit containing four lanes. The vector register memory is divided across the lanes, with each lane holding every fourth element of each vector register. The figure shows three vector functional units: an FP add, an FP multiply, and a load-store unit. Each of the vector arithmetic units contains four execution pipelines, one per lane, which act in concert to complete a single vector instruction. Note how each section of the vector register file needs to provide only enough ports for pipelines local to its lane. This figure does not show the path to provide the scalar operand for vector-scalar instructions, but the scalar processor (or Control Processor) broadcasts a scalar value to all lanes.

to match the real vector length in a program. Moreover, in a real program, the length of a particular vector operation is often *unknown* at compile time. In fact, a single piece of code may require different vector lengths. For example, consider this code:

```
for (i=0; i <n; i=i+1)
    Y[i] = a * X[i] + Y[i];
```

The size of all the vector operations depends on n, which may not even be known until run time. The value of n might also be a parameter to a procedure containing the preceding loop and therefore subject to change during execution.

The solution to these problems is to add a *vector-length register* (`vl`). The `vl` controls the length of any vector operation, including a vector load or store. The value in the `vl`, however, cannot be greater than the maximum vector length (mvl). This solves our problem as long as the real length is less than or equal to the maximum vector length (`mvl`). This parameter means the length of vector registers can grow in later computer generations without changing the instruction set. As we will see in the next section, multimedia SIMD extensions have no equivalent of `mvl`, so they expand the instruction set every time they increase their vector length.

What if the value of `n` is not known at compile time and thus may be greater than the `mvl`? To tackle the second problem where the vector is longer than the maximum length, a technique called *strip mining* is traditionally used. Strip mining is the generation of code such that each vector operation is done for a size less than or equal to the `mvl`. One loop handles any number of iterations that is a multiple of the `mvl` and another loop that handles any remaining iterations and must be less than the `mvl`. RISC-V has a better solution than a separate loop for strip mining. The instruction `setvl` writes the smaller of the `mvl` and the loop variable `n` into `vl` (and to another register). If the number of iterations of the loop is larger than `n`, then the fastest the loop can compute is `mvl` values at time, so `setvl` sets `vl` to `mvl`. If n is smaller than `mvl`, it should compute only on the last `n` elements in this final iteration of the loop, so `setvl` sets `vl` to `n`. `setvl` also writes another scalar register to help with later loop bookkeeping. Below is the RV64V code for vector DAXPY for any value of `n`.

```
      vsetdcfg  2 DP FP    # Enable 2 64b Fl.Pt. registers
      fld       f0,a       # Load scalar a
loop: setvl     t0,a0      # vl = t0 = min(mvl,n)
      vld       v0,x5      # Load vector X
      slli      t1,t0,3    # t1 = vl * 8 (in bytes)
      add       x5,x5,t1   # Increment pointer to X by vl*8
      vmul      v0,v0,f0   # Vector-scalar mult
      vld       v1,x6      # Load vector Y
      vadd      v1,v0,v1   # Vector-vector add
      sub       a0,a0,t0   # n -= vl (t0)
      vst       v1,x6      # Store the sum into Y
      add       x6,x6,t1   # Increment pointer to Y by vl*8
      bnez      a0,loop    # Repeat if n != 0
      vdisable             # Disable vector regs
```

Predicate Registers: Handling IF Statements in Vector Loops

From Amdahl's law, we know that the speedup on programs with low to moderate levels of vectorization will be very limited. The presence of conditionals (IF statements) inside loops and the use of sparse matrices are two main reasons for lower levels of vectorization. Programs that contain IF statements in loops cannot be run

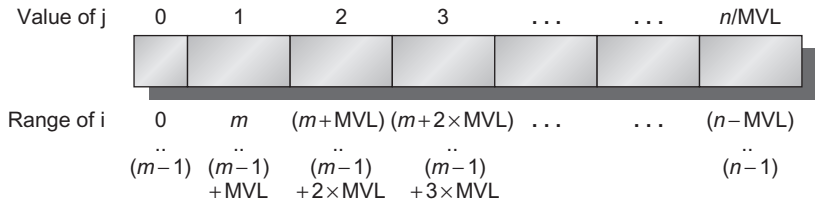


Figure 4.6 A vector of arbitrary length processed with strip mining. All blocks but the first are of length MVL, utilizing the full power of the vector processor. In this figure, we use the variable *m* for the expression (`n % MVL`). (The C operator `%` is modulo.)

in vector mode using the techniques we have discussed up to now because the IF statements introduce control dependences into a loop. Likewise, we cannot implement sparse matrices efficiently using any of the capabilities we have seen so far. We examine strategies for dealing with conditional execution here, leaving the discussion of sparse matrices for later.

Consider the following loop written in C:

```
for (i = 0; i < 64;   i=i+1)
  if (X[i] != 0)
    X[i] = X[i] - Y[i];
```

This loop cannot normally be vectorized because of the conditional execution of the body; however, if the inner loop could be run for the iterations for which X[i] ≠ 0, then the subtraction could be vectorized.

The common extension for this capability is *vector-mask control*. In RV64V, predicate registers hold the mask and essentially provide conditional execution of each element operation in a vector instruction. These registers use a Boolean vector to control the execution of a vector instruction, just as conditionally executed instructions use a Boolean condition to determine whether to execute a scalar instruction (see Chapter 3). When the predicate register `p0` is set, all following vector instructions operate only on the vector elements whose corresponding entries in the predicate register are 1. The entries in the destination vector register that correspond to a 0 in the mask register are unaffected by the vector operation. Like vector registers, predicate registers are configured and can be disabled. Enabling a predicate register initializes it to all 1 s, meaning that subsequent vector instructions operate on all vector elements. We can now use the following code for the previous loop, assuming that the starting addresses of `X` and `Y` are in `x5` and `x6`, respectively:

```
vsetdcfg    2*FP64      # Enable 2 64b FP vector regs
vsetpcfgi   1           # Enable 1 predicate register
vld         v0,x5       # Load vector X into v0
vld         v1,x6       # Load vector Y into v1
fmv.d.x     f0,x0       # Put (FP) zero into f0
```

```
vpne        p0,v0,f0    # Set p0(i) to 1 if v0(i)!=f0
vsub        v0,v0,v1    # Subtract under vector mask
vst         v0,x5       # Store the result in X
vdisable                # Disable vector registers
vpdisable               # Disable predicate registers
```

Compiler writers use the term *IF-conversion* to transform an IF statement into a straight-line code sequence using conditional execution.

Using a vector-mask register does have overhead, however. With scalar architectures, conditionally executed instructions still require execution time when the condition is not satisfied. Nonetheless, the elimination of a branch and the associated control dependences can make a conditional instruction faster even if it sometimes does useless work. Similarly, vector instructions executed with a vector mask still take the same execution time, even for the elements where the mask is zero. Likewise, despite a significant number of zeros in the mask, using vector-mask control may still be significantly faster than using scalar mode.

As we will see in Section 4.4, one difference between vector processors and GPUs is the way they handle conditional statements. Vector processors make the predicate registers part of the architectural state and rely on compilers to manipulate mask registers explicitly. In contrast, GPUs get the same effect using hardware to manipulate internal mask registers that are invisible to GPU software. In both cases, the hardware spends the time to execute a vector element whether the corresponding mask bit is 0 or 1, so the GFLOPS rate drops when masks are used.

Memory Banks: Supplying Bandwidth for Vector Load/Store Units

The behavior of the load/store vector unit is significantly more complicated than that of the arithmetic functional units. The start-up time for a load is the time to get the first word from memory into a register. If the rest of the vector can be supplied without stalling, then the vector initiation rate is equal to the rate at which new words are fetched or stored. Unlike simpler functional units, the initiation rate may not necessarily be 1 clock cycle because memory bank stalls can reduce effective throughput.

Typically, penalties for start-ups on load/store units are higher than those for arithmetic units—over 100 clock cycles on many processors. For RV64V, we assume a start-up time of 12 clock cycles, the same as the Cray-1. (Recent vector computers use caches to bring down latency of vector loads and stores.)

To maintain an initiation rate of one word fetched or stored per clock cycle, the memory system must be capable of producing or accepting this much data. Spreading accesses across multiple independent memory banks usually delivers the desired rate. As we will soon see, having significant numbers of banks is useful for dealing with vector loads or stores that access rows or columns of data.

Most vector processors use memory banks, which allow several independent accesses rather than simple memory interleaving for three reasons:

1. Many vector computers support many loads or stores per clock cycle, and the memory bank cycle time is usually several times larger than the processor cycle time. To support simultaneous accesses from multiple loads or stores, the memory system needs multiple banks and needs to be able to control the addresses to the banks independently.
2. Most vector processors support the ability to load or store data words that are not sequential. In such cases, independent bank addressing, rather than interleaving, is required.
3. Most vector computers support multiple processors sharing the same memory system, so each processor will be generating its own separate stream of addresses.

In combination, these features lead to the desire for a large number of independent memory banks, as the following example shows.

Example The largest configuration of a Cray T90 (Cray T932) has 32 processors, each capable of generating 4 loads and 2 stores per clock cycle. The processor clock cycle is 2.167 ns, while the cycle time of the SRAMs used for the memory system is 15 ns. Calculate the minimum number of memory banks required to allow all processors to run at the full memory bandwidth.

Answer The maximum number of memory references each cycle is 192: 32 processors times 6 references per processor. Each SRAM bank is busy for $15/2.167 = 6.92$ clock cycles, which we round up to 7 processor clock cycles. Therefore we require a minimum of $192 \times 7 = 1344$ memory banks!

The Cray T932 actually has 1024 memory banks, so the early models could not sustain the full bandwidth to all processors simultaneously. A subsequent memory upgrade replaced the 15 ns asynchronous SRAMs with pipelined synchronous SRAMs that more than halved the memory cycle time, thereby providing sufficient bandwidth.

Taking a higher-level perspective, vector load/store units play a similar role to prefetch units in scalar processors in that both try to deliver data bandwidth by supplying processors with streams of data.

Stride: Handling Multidimensional Arrays in Vector Architectures

The position in memory of adjacent elements in a vector may not be sequential. Consider this straightforward code for matrix multiply in C:

```
for (i = 0; i < 100;  i=i+1)
  for (j = 0; j < 100;  j=j+1) {
    A[i][j] = 0.0;
    for (k = 0; k < 100; k=k+1)
      A[i][j] = A[i][j] + B[i][k] * D[k][j];
    }
```

We could vectorize the multiplication of each row of B with each column of D and strip-mine the inner loop with k as the index variable.

To do so, we must consider how to address adjacent elements in B and adjacent elements in D. When an array is allocated memory, it is linearized and must be laid out in either row-major order (as in C) or column-major order (as in Fortran). This linearization means that either the elements in the row or the elements in the column are not adjacent in memory. For example, the preceding C code allocates in row-major order, so the elements of D that are accessed by iterations in the inner loop are separated by the row size times 8 (the number of bytes per entry) for a total of 800 bytes. In Chapter 2, we saw that blocking could improve locality in cache-based systems. For vector processors without caches, we need another technique to fetch elements of a vector that are not adjacent in memory.

This distance separating elements to be gathered into a single vector register is called the *stride*. In this example, matrix D has a stride of 100 double words (800 bytes), and matrix B would have a stride of 1 double word (8 bytes). For column-major order, which is used by Fortran, the strides would be reversed. Matrix D would have a stride of 1, or 1 double word (8 bytes), separating successive elements, while matrix B would have a stride of 100, or 100 double words (800 bytes). Thus, without reordering the loops, the compiler can't hide the long distances between successive elements for both B and D.

Once a vector is loaded into a vector register, it acts as if it had logically adjacent elements. Thus a vector processor can handle strides greater than one, called *nonunit strides*, using only vector load and vector store operations with stride capability. This ability to access nonsequential memory locations and to reshape them into a dense structure is one of the major advantages of a vector architecture.

Caches inherently deal with unit-stride data; increasing block size can help reduce miss rates for large scientific datasets with unit stride, but increasing block size can even have a negative effect for data that are accessed with nonunit strides. While blocking techniques can solve some of these problems (see Chapter 2), the ability to access noncontiguous data efficiently remains an advantage for vector processors on certain problems, as we will see in Section 4.7.

On RV64V, where the addressable unit is a byte, the stride for our example would be 800. The value must be computed dynamically because the size of the matrix may not be known at compile time or—just like vector length—may change for different executions of the same statement. The vector stride, like the vector starting address, can be put in a general-purpose register. Then the RV64V instruction VLDS (load vector with stride) fetches the vector into a vector

register. Likewise, when storing a nonunit stride vector, use the instruction VSTS (store vector with stride).

Supporting strides greater than one complicates the memory system. Once we introduce nonunit strides, it becomes possible to request accesses from the same bank frequently. When multiple accesses contend for a bank, a memory bank conflict occurs, thereby stalling one access. A bank conflict and thus a stall will occur if

$$\frac{\text{Number of banks}}{\text{Least common multiple (Stride, Number of banks)}} < \text{Bank busy time}$$

Example Suppose we have 8 memory banks with a bank busy time of 6 clocks and a total memory latency of 12 cycles. How long will it take to complete a 64-element vector load with a stride of 1? With a stride of 32?

Answer Because the number of banks is larger than the bank busy time, for a stride of 1, the load will take $12 + 64 = 76$ clock cycles, or 1.2 clock cycles per element. The worst possible stride is a value that is a multiple of the number of memory banks, as in this case with a stride of 32 and 8 memory banks. Every access to memory (after the first one) will collide with the previous access and will have to wait for the 6-clock-cycle bank busy time. The total time will be $12 + 1 + 6 * 63 = 391$ clock cycles, or 6.1 clock cycles per element, slowing it down by a factor of 5!

Gather-Scatter: Handling Sparse Matrices in Vector Architectures

As previously mentioned, sparse matrices are commonplace, so it is important to have techniques to allow programs with sparse matrices to execute in vector mode. In a sparse matrix, the elements of a vector are usually stored in some compacted form and then accessed indirectly. Assuming a simplified sparse structure, we might see code that looks like this:

```
for (i = 0; i < n;  i=i+1)
  A[K[i]] = A[K[i]] + C[M[i]];
```

This code implements a sparse vector sum on the arrays A and C, using index vectors K and M to designate the nonzero elements of A and C. (A and C must have the same number of nonzero elements—`n` of them—so `K` and `M` are the same size.)

The primary mechanism for supporting sparse matrices is *gather-scatter operations* using index vectors. The goal of such operations is to support moving between a compressed representation (i.e., zeros are not included) and normal representation (i.e., the zeros are included) of a sparse matrix. A *gather* operation takes an *index vector* and fetches the vector whose elements are at the addresses given by adding a base address to the offsets given in the index vector. The result is a dense vector in a vector register. After these elements are operated on in a dense

form, the sparse vector can be stored in an expanded form by a *scatter* store, using the same index vector. Hardware support for such operations is called *gather-scatter*, and it appears on nearly all modern vector processors. The RV64V instructions are `vldi` (load vector indexed or gather) and `vsti` (store vector indexed or scatter). For example, if `x5`, `x6`, `x7`, and `x28` contain the starting addresses of the vectors in the previous sequence, we can code the inner loop with vector instructions such as:

```
vsetdcfg  4*FP64       # 4  64b  FP vector registers
vld       v0, x7       # Load K[]
vldx      v1, x5, v0)  # Load A[K[]]
vld       v2, x28      # Load M[]
vldi      v3, x6, v2)  # Load C[M[]]
vadd      v1, v1, v3   # Add them
vstx      v1, x5, v0)  # Store A[K[]]
vdisable               # Disable vector registers
```

This technique allows code with sparse matrices to run in vector mode. A simple vectorizing compiler could not automatically vectorize the preceding source code because the compiler would not know that the elements of K are distinct values, and thus that no dependences exist. Instead, a programmer directive would tell the compiler that it was safe to run the loop in vector mode.

Although indexed loads and stores (gather and scatter) can be pipelined, they typically run much more slowly than nonindexed loads or stores, because the memory banks are not known from the start of the instruction. The register file must also provide communication between the lanes of a vector unit to support gather and scatter.

Each element of a gather or scatter has an individual address, so they can't be handled in groups, and there can be conflicts at many places throughout the memory system. Thus each individual access incurs significant latency even on cache-based systems. However, as Section 4.7 shows, a memory system can deliver better performance by designing for this case and by using more hardware resources versus when architects have a laissez-faire attitude toward such unpredictable accesses.

As we will see in Section 4.4, all loads are gathers and all stores are scatters in GPUs in that no separate instructions restrict addresses to be sequential. To turn the potentially slow gathers and scatters into the more efficient unit-stride accesses to memory, the GPU hardware must recognize the sequential addresses during execution and the GPU programmer to ensure that all the addresses in a gather or scatter are to adjacent locations.

Programming Vector Architectures

An advantage of vector architectures is that compilers can tell programmers at compile time whether a section of code will vectorize or not, often giving hints

Benchmark name	Operations executed in vector mode, compiler-optimized	Operations executed in vector mode, with programmer aid	Speedup from hint optimization
BDNA	96.1%	97.2%	1.52
MG3D	95.1%	94.5%	1.00
FLO52	91.5%	88.7%	N/A
ARC3D	91.1%	92.0%	1.01
SPEC77	90.3%	90.4%	1.07
MDG	87.7%	94.2%	1.49
TRFD	69.8%	73.7%	1.67
DYFESM	68.8%	65.6%	N/A
ADM	42.9%	59.6%	3.60
OCEAN	42.8%	91.2%	3.92
TRACK	14.4%	54.6%	2.52
SPICE	11.5%	79.9%	4.06
QCD	4.2%	75.1%	2.15

Figure 4.7 Level of vectorization among the Perfect Club benchmarks when executed on the Cray Y-MP (Vajapeyam, 1991). The first column shows the vectorization level obtained with the compiler without hints, and the second column shows the results after the codes have been improved with hints from a team of Cray Research programmers.

as to why it did not vectorize the code. This straightforward execution model allows experts in other domains to learn how to improve performance by revising their code or by giving hints to the compiler when it's okay to assume independence between operations, such as for gather-scatter data transfers. It is this dialogue between the compiler and the programmer, with each side giving hints to the other on how to improve performance, that simplifies programming of vector computers.

Today, the main factor that affects the success with which a program runs in vector mode is the structure of the program itself: Do the loops have true data dependences (see Section 4.5), or can they be restructured so as not to have such dependences? This factor is influenced by the algorithms chosen and, to some extent, by how they are coded.

As an indication of the level of vectorization achievable in scientific programs, let's look at the vectorization levels observed for the Perfect Club benchmarks. Figure 4.7 shows the percentage of operations executed in vector mode for two versions of the code running on the Cray Y-MP. The first version is that obtained with just compiler optimization on the original code, while the second version uses extensive hints from a team of Cray Research programmers. Several studies of the performance of applications on vector processors show a wide variation in the level of compiler vectorization.

The hint-rich versions show significant gains in vectorization level for codes that the compiler could not vectorize well by itself, with all codes now above 50% vectorization. The median vectorization improved from about 70% to about 90%.

4.3 SIMD Instruction Set Extensions for Multimedia

SIMD Multimedia Extensions started with the simple observation that many media applications operate on narrower data types than the 32-bit processors were optimized for. Graphics systems would use 8 bits to represent each of the three primary colors plus 8 bits for transparency. Depending on the application, audio samples are usually represented with 8 or 16 bits. By partitioning the carry chains within, say, a 256-bit adder, a processor could perform simultaneous operations on short vectors of thirty-two 8-bit operands, sixteen 16-bit operands, eight 32-bit operands, or four 64-bit operands. The additional cost of such partitioned adders was small. Figure 4.8 summarizes typical multimedia SIMD instructions. Like vector instructions, a SIMD instruction specifies the same operation on vectors of data. Unlike vector machines with large register files such as the RISC-V RV64V vector registers, which can hold, say, thirty-two 64-bit elements in each of 32 vector registers, SIMD instructions tend to specify fewer operands and thus use much smaller register files.

In contrast to vector architectures, which offer an elegant instruction set that is intended to be the target of a vectorizing compiler, SIMD extensions have three major omissions: no vector length register, no strided or gather/scatter data transfer instructions, and no mask registers.

1. Multimedia SIMD extensions fix the number of data operands in the opcode, which has led to the addition of hundreds of instructions in the MMX, SSE, and AVX extensions of the x86 architecture. Vector architectures have a vector-length register that specifies the number of operands for the current operation. These variable-length vector registers easily accommodate programs that naturally have shorter vectors than the maximum size the architecture supports. Moreover, vector architectures have an implicit maximum vector length in the

Instruction category	Operands
Unsigned add/subtract	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Maximum/minimum	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Average	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Shift right/left	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Floating point	Sixteen 16-bit, eight 32-bit, four 64-bit, or two 128-bit

Figure 4.8 Summary of typical SIMD multimedia support for 256-bit-wide operations. Note that the IEEE 754-2008 floating-point standard added half-precision (16-bit) and quad-precision (128-bit) floating-point operations.

architecture, which combined with the vector length register avoids the use of many opcodes.

2. Up until recently, multimedia SIMD did not offer the more sophisticated addressing modes of vector architectures, namely strided accesses and gather-scatter accesses. These features increase the number of programs that a vector compiler can successfully vectorize (see Section 4.7).
3. Although this is changing, multimedia SIMD usually did not offer the mask registers to support conditional execution of elements as in vector architectures.

Such omissions make it harder for the compiler to generate SIMD code and increase the difficulty of programming in SIMD assembly language.

For the x86 architecture, the MMX instructions added in 1996 repurposed the 64-bit floating-point registers, so the basic instructions could perform eight 8-bit operations or four 16-bit operations simultaneously. These were joined by parallel MAX and MIN operations, a wide variety of masking and conditional instructions, operations typically found in digital signal processors, and ad hoc instructions that were believed to be useful in important media libraries. Note that MMX reused the floating-point data-transfer instructions to access memory.

The Streaming SIMD Extensions (SSE) successor in 1999 added 16 separate registers (XMM registers) that were 128 bits wide, so now instructions could simultaneously perform sixteen 8-bit operations, eight 16-bit operations, or four 32-bit operations. It also performed parallel single-precision floating-point arithmetic. Because SSE had separate registers, it needed separate data transfer instructions. Intel soon added double-precision SIMD floating-point data types via SSE2 in 2001, SSE3 in 2004, and SSE4 in 2007. Instructions with four single-precision floating-point operations or two parallel double-precision operations increased the peak floating-point performance of the x86 computers, as long as programmers placed the operands side by side. With each generation, they also added ad hoc instructions whose aim was to accelerate specific multimedia functions perceived to be important.

The Advanced Vector Extensions (AVX), added in 2010, doubled the width of the registers again to 256 bits (YMM registers) and thereby offered instructions that double the number of operations on all narrower data types. Figure 4.9 shows AVX instructions useful for double-precision floating-point computations. AVX2 in 2013 added 30 new instructions such as gather (VGATHER) and vector shifts (VPSLL, VPSRL, VPSRA). AVX-512 in 2017 doubled the width again to 512 bits (ZMM registers), doubled the number of the registers again to 32, and added about 250 new instructions including scatter (VPSCATTER) and mask registers (OPMASK). AVX includes preparations to extend registers to 1024 bits in future editions of the architecture.

In general, the goal of these extensions has been to accelerate carefully written libraries rather than for the compiler to generate them (see Appendix H), but recent x86 compilers are trying to generate such code, particularly for floating-point-intensive applications. Since the opcode determines the width of the SIMD register, every time the width doubles, so must the number of SIMD instructions.

AVX instruction	Description
VADDPD	Add four packed double-precision operands
VSUBPD	Subtract four packed double-precision operands
VMULPD	Multiply four packed double-precision operands
VDIVPD	Divide four packed double-precision operands
VFMADDPD	Multiply and add four packed double-precision operands
VFMSUBPD	Multiply and subtract four packed double-precision operands
VCMPxx	Compare four packed double-precision operands for EQ, NEQ, LT, LE, GT, GE, ...
VMOVAPD	Move aligned four packed double-precision operands
VBROADCASTSD	Broadcast one double-precision operand to four locations in a 256-bit register

Figure 4.9 AVX instructions for x86 architecture useful in double-precision floating-point programs. Packed-double for 256-bit AVX means four 64-bit operands executed in SIMD mode. As the width increases with AVX, it is increasingly important to add data permutation instructions that allow combinations of narrow operands from different parts of the wide registers. AVX includes instructions that shuffle 32-bit, 64-bit, or 128-bit operands within a 256-bit register. For example, BROADCAST replicates a 64-bit operand four times in an AVX register. AVX also includes a large variety of fused multiply-add/subtract instructions; we show just two here.

Given these weaknesses, why are multimedia SIMD extensions so popular? First, they initially cost little to add to the standard arithmetic unit and they were easy to implement. Second, they require scant extra processor state compared to vector architectures, which is always a concern for context switch times. Third, you need a lot of memory bandwidth to support a vector architecture, which many computers don't have. Fourth, SIMD does not have to deal with problems in virtual memory when a single instruction can generate 32 memory accesses and any of which can cause a page fault. The original SIMD extensions used separate data transfers per SIMD group of operands that are aligned in memory, and so they cannot cross page boundaries. Another advantage of short, fixed-length "vectors" of SIMD is that it is easy to introduce instructions that can help with new media standards, such as instructions that perform permutations or instructions that consume either fewer or more operands than vectors can produce. Finally, there was concern about how well vector architectures can work with caches. More recent vector architectures have addressed all of these problems. The overarching issue, however, is that due the overiding importance of backwards binary compatability, once an architecture gets started on the SIMD path it's very hard to get off it.

Example To get an idea about what multimedia instructions look like, assume we added a 256-bit SIMD multimedia instruction extension to RISC-V, tentatively called RVP for "packed." We concentrate on floating-point in this example. We add the suffix "4D" on instructions that operate on four double-precision operands at once. Like vector architectures, you can think of a SIMD Processor as having lanes, four in this case. RV64P expands the F registers to be the full width, in this case 256 bits. This example shows the RISC-V SIMD code for the DAXPY loop,

with the changes to the RISC-V code for SIMD underlined. We assume that the starting addresses of X and Y are in x5 and x6, respectively.

Answer Here is the RISC-V SIMD code:

```
        fld       f0,a          #Load scalar a
        splat.4D  f0,f0         #Make 4 copies of a
        addi      x28,x5,#256   #Last address to load
Loop:   fld.4D    f1,0(x5)      #Load X[i] ... X[i+3]
        fmul.4D   f1,f1,f0      #a×X[i] ... a×X[i+3]
        fld.4D    f2,0(x6)      #Load Y[i] ... Y[i+3]
        fadd.4D   f2,f2,f1      # a×X[i]+Y[i]...
                                # a×X[i+3]+Y[i+3]
        fsd.4D    f2,0(x6)      #Store Y[i]... Y[i+3]
        addi      x5,x5,#32     #Increment index to X
        addi      x6,x6,#32     #Increment index to Y
        bne       x28,x5,Loop   #Check if done
```

The changes were replacing every RISC-V double-precision instruction with its 4D equivalent, increasing the increment from 8 to 32, and adding the splat instruction that makes 4 copies of a in the 256 bits of f0. While not as dramatic as the 32× reduction of dynamic instruction bandwidth of RV64V, RISC-V SIMD does get almost a 4× reduction: 67 versus 258 instructions executed for RV64G. This code knows the number of elements. That number is often determined at run time, which would require an extra strip-mine loop to handle the case when the number is not a modulo of 4.

Programming Multimedia SIMD Architectures

Given the ad hoc nature of the SIMD multimedia extensions, the easiest way to use these instructions has been through libraries or by writing in assembly language.

Recent extensions have become more regular, giving compilers a more reasonable target. By borrowing techniques from vectorizing compilers, compilers are starting to produce SIMD instructions automatically. For example, advanced compilers today can generate SIMD floating-point instructions to deliver much higher performance for scientific codes. However, programmers must be sure to align all the data in memory to the width of the SIMD unit on which the code is run to prevent the compiler from generating scalar instructions for otherwise vectorizable code.

The Roofline Visual Performance Model

One visual, intuitive way to compare potential floating-point performance of variations of SIMD architectures is the Roofline model (Williams et al., 2009). The horizontal and diagonal lines of the graphs it produces give this simple model its name and indicate its value (see Figure 4.11). It ties together floating-point performance, memory performance, and arithmetic intensity in a two-dimensional graph.

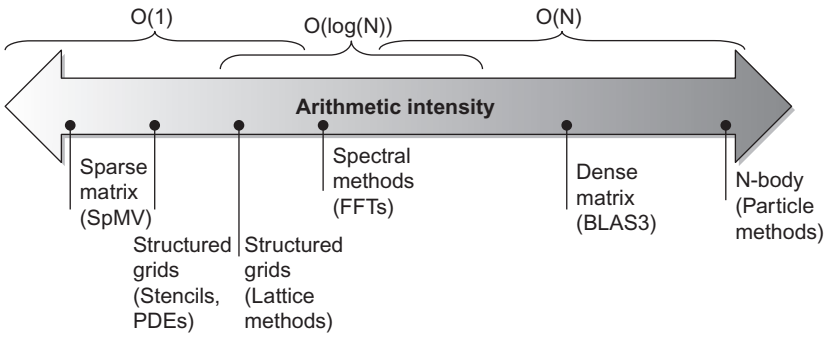


Figure 4.10 Arithmetic intensity, specified as the number of floating-point operations to run the program divided by the number of bytes accessed in main memory (Williams et al., 2009). Some kernels have an arithmetic intensity that scales with problem size, such as a dense matrix, but there are many kernels with arithmetic intensities independent of problem size.

Arithmetic intensity is the ratio of floating-point operations per byte of memory accessed. It can be calculated by taking the total number of floating-point operations for a program divided by the total number of data bytes transferred to main memory during program execution. Figure 4.10 shows the relative arithmetic intensity of several example kernels.

Peak floating-point performance can be found using the hardware specifications. Many of the kernels in this case study do not fit in on-chip caches, so peak memory performance is defined by the memory system behind the caches. Note that we need the peak memory bandwidth that is available to the processors, not just at the DRAM pins as in Figure 4.27 on page 328. One way to find the (delivered) peak memory performance is to run the Stream benchmark.

Figure 4.11 shows the Roofline model for the NEC SX-9 vector processor on the left and the Intel Core i7 920 multicore computer on the right. The vertical Y-axis is achievable floating-point performance from 2 to 256 GFLOPS/s. The horizontal X-axis is arithmetic intensity, varying from 1/8 FLOP/DRAM byte accessed to 16 FLOP/DRAM byte accessed in both graphs. Note that the graph is a log-log scale, and that Rooflines are done just once for a computer.

For a given kernel, we can find a point on the X-axis based on its arithmetic intensity. If we drew a vertical line through that point, the performance of the kernel on that computer must lie somewhere along that line. We can plot a horizontal line showing peak floating-point performance of the computer. Obviously, the actual floating-point performance can be no higher than the horizontal line because that is a hardware limit.

How could we plot the peak memory performance? Because the X-axis is FLOP/byte and the Y-axis is FLOP/s, bytes/s is just a diagonal line at a 45-degree angle in this figure. Thus we can plot a third line that gives the maximum floating-point performance that the memory system of that computer can support for a given

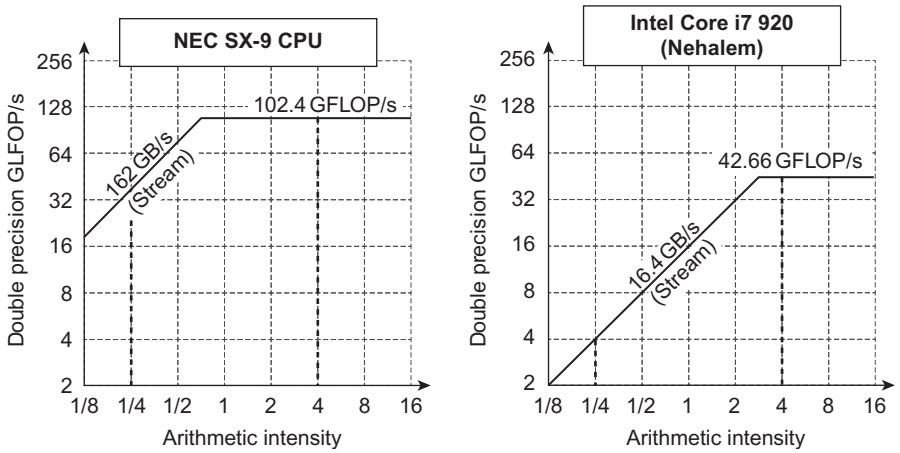


Figure 4.11 Roofline model for one NEC SX-9 vector processor on the left and the Intel Core i7 920 multicore computer with SIMD extensions on the right (Williams et al., 2009). This Roofline is for unit-stride memory accesses and double-precision floating-point performance. NEC SX-9 is a vector supercomputer announced in 2008 that cost millions of dollars. It has a peak DP FP performance of 102.4 GFLOP/s and a peak memory bandwidth of 162 GB/s from the Stream benchmark. The Core i7 920 has a peak DP FP performance of 42.66 GFLOP/s and a peak memory bandwidth of 16.4 GB/s. The dashed vertical lines at an arithmetic intensity of 4 FLOP/byte show that both processors operate at peak performance. In this case, the SX-9 at 102.4 FLOP/s is 2.4× faster than the Core i7 at 42.66 GFLOP/s. At an arithmetic intensity of 0.25 FLOP/byte, the SX-9 is 10× faster at 40.5 GFLOP/s versus 4.1 GFLOP/s for the Core i7.

arithmetic intensity. We can express the limits as a formula to plot these lines in the graphs in Figure 4.11:

$$\text{Attainable GFLOPs/s} = \text{Min}(\text{Peak Memory BW} \times \text{Arithmetic Intensity},\ \text{Peak Floating} - \text{Point Perf.})$$

The "Roofline" sets an upper bound on performance of a kernel depending on its arithmetic intensity. If we think of arithmetic intensity as a pole that hits the roof, either it hits the flat part of the roof, which means performance is computationally limited, or it hits the slanted part of the roof, which means performance is ultimately limited by memory bandwidth. In Figure 4.11, the vertical dashed line on the right (arithmetic intensity of 4) is an example of the former and the vertical dashed line on the left (arithmetic intensity of 1/4) is an example of the latter. Given a Roofline model of a computer, you can apply it repeatedly, because it doesn't vary by kernel.

Note that the "ridge point," where the diagonal and horizontal roofs meet, offers an interesting insight into a computer. If it is far to the right, then only kernels with very high arithmetic intensity can achieve the maximum performance of that computer. If it is far to the left, then almost any kernel can potentially hit the maximum performance. As we will see, this vector processor has both much higher

memory bandwidth and a ridge point far to the left as compared to other SIMD Processors.

Figure 4.11 shows that the peak computational performance of the SX-9 is 2.4 × faster than Core i7, but the memory performance is 10 × faster. For programs with an arithmetic intensity of 0.25, the SX-9 is 10 × faster (40.5 versus 4.1 GFLOP/s). The higher memory bandwidth moves the ridge point from 2.6 in the Core i7 to 0.6 on the SX-9, which means many more programs can reach the peak computational performance on the vector processor.

4.4 Graphics Processing Units

People can buy a GPU chip with thousands of parallel floating-point units for a few hundred dollars and plug it into their desk side PC. Such affordability and convenience makes high performance computing available to many. The interest in GPU computing blossomed when this potential was combined with a programming language that made GPUs easier to program. Therefore many programmers of scientific and multimedia applications today are pondering whether to use GPUs or CPUs. For programmers interested in machine learning, which is the subject of Chapter 7, GPUs are currently the preferred platform.

GPUs and CPUs do not go back in computer architecture genealogy to a common ancestor; there is no "missing link" that explains both. As Section 4.10 describes, the primary ancestors of GPUs are graphics accelerators, as doing graphics well is the reason why GPUs exist. While GPUs are moving toward mainstream computing, they can't abandon their responsibility to continue to excel at graphics. Thus the design of GPUs may make more sense when architects ask, given the hardware invested to do graphics well, how can we supplement it to improve the performance of a wider range of applications?

Note that this section concentrates on using GPUs for computing. To see how GPU computing combines with the traditional role of graphics acceleration, see "Graphics and Computing GPUs," by John Nickolls and David Kirk (Appendix A in the 5th edition of *Computer Organization and Design* by the same authors as this book).

Because the terminology and some hardware features are quite different from vector and SIMD architectures, we believe it will be easier if we start with the simplified programming model for GPUs before we describe the architecture.

Programming the GPU

CUDA is an elegant solution to the problem of representing parallelism in algorithms, not all algorithms, but enough to matter. It seems to resonate in some way with the way we think and code, allowing an easier, more natural expression of parallelism beyond the task level.

Vincent Natol,
"Kudos for CUDA," *HPC Wire* (2010)

The challenge for the GPU programmer is not simply getting good performance on the GPU, but also in coordinating the scheduling of computation on the system processor and the GPU and the transfer of data between system memory and GPU memory. Moreover, as we see will see later in this section, GPUs have virtually every type of parallelism that can be captured by the programming environment: multithreading, MIMD, SIMD, and even instruction-level.

NVIDIA decided to develop a C-like language and programming environment that would improve the productivity of GPU programmers by attacking both the challenges of heterogeneous computing and of multifaceted parallelism. The name of their system is *CUDA*, for Compute Unified Device Architecture. CUDA produces C/C++ for the system processor (*host*) and a C and C++ dialect for the GPU (*device*, thus the D in CUDA). A similar programming language is *OpenCL*, which several companies are developing to offer a vendor-independent language for multiple platforms.

NVIDIA decided that the unifying theme of all these forms of parallelism is the *CUDA Thread*. Using this lowest level of parallelism as the programming primitive, the compiler and the hardware can gang thousands of CUDA Threads together to utilize the various styles of parallelism within a GPU: multithreading, MIMD, SIMD, and instruction-level parallelism. Therefore NVIDIA classifies the CUDA programming model as single instruction, multiple thread (*SIMT*). For reasons we will soon see, these threads are blocked together and executed in groups of threads, called a *Thread Block*. We call the hardware that executes a whole block of threads a *multithreaded SIMD Processor*.

We need just a few details before we can give an example of a CUDA program:

- To distinguish between functions for the GPU (device) and functions for the system processor (host), CUDA uses `__device__` or `__global__` for the former and `__host__` for the latter.
- CUDA variables declared with `__device__` are allocated to the GPU Memory (see below), which is accessible by all multithreaded SIMD Processors.
- The extended function call syntax for the function *name* that runs on the GPU is

  name < <<`dimGrid`, `dimBlock`>> > (… *parameter list*…)

  where `dimGrid` and `dimBlock` specify the dimensions of the code (in Thread Blocks) and the dimensions of a block (in threads).
- In addition to the identifier for blocks (`blockIdx`) and the identifier for each thread in a block (`threadIdx`), CUDA provides a keyword for the number of threads per block (`blockDim`), which comes from the `dimBlock` parameter in the preceding bullet.

Before seeing the CUDA code, let's start with conventional C code for the DAXPY loop from Section 4.2:

```
// Invoke DAXPY
daxpy(n, 2.0, x, y);
// DAXPY in C
void daxpy(int n, double a, double *x, double *y)
{
          for (int i = 0; i < n; ++i)
                y[i] = a*x[i] + y[i];
}
```

Following is the CUDA version. We launch n threads, one per vector element, with 256 CUDA Threads per Thread Block in a multithreaded SIMD Processor. The GPU function starts by calculating the corresponding element index i based on the block ID, the number of threads per block, and the thread ID. As long as this index is within the array (i < n), it performs the multiply and add.

```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n+ 255) / 256;
   daxpy<<<nblocks, 256>>>(n, 2.0, x, y);
// DAXPY in CUDA
__global__
void daxpy(int n, double a, double *x, double *y)
{
   int i = blockIdx.x*blockDim.x + threadIdx.x;
   if (i < n) y[i] = a*x[i] + y[i];
}
```

Comparing the C and CUDA codes, we see a common pattern to parallelizing data-parallel CUDA code. The C version has a loop where each iteration is independent from the others, allowing the loop to be transformed straightforwardly into a parallel code where each loop iteration becomes a separate thread. (As previously mentioned and described in detail in Section 4.5, vectorizing compilers also rely on a lack of dependences between iterations of a loop, which are called *loop-carried dependences*.) The programmer determines the parallelism in CUDA explicitly by specifying the grid dimensions and the number of threads per SIMD Processor. By assigning a single thread to each element, there is no need to synchronize between threads when writing results to memory.

The GPU hardware handles parallel execution and thread management; it is not done by applications or by the operating system. To simplify scheduling by the hardware, CUDA requires that Thread Blocks be able to execute independently and in any order. Different Thread Blocks cannot communicate directly, although they can *coordinate* using atomic memory operations in global memory.

As we will soon see, many GPU hardware concepts are not obvious in CUDA. Writing efficient GPU code requires that programmers think in terms of SIMD

operations, even though the CUDA programming model looks like MIMD. Performance programmers must keep the GPU hardware in mind when writing in CUDA. That could hurt programmer productivity, but then most programmers are using GPUs instead of CPUs to get performance. For reasons explained shortly, they know that they need to keep groups of 32 threads together in control flow to get the best performance from multithreaded SIMD Processors and to create many more threads per multithreaded SIMD Processor to hide latency to DRAM. They also need to keep the data addresses localized in one or a few blocks of memory to get the expected memory performance.

Like many parallel systems, a compromise between productivity and performance is for CUDA to include intrinsics to give programmers explicit control over the hardware. The struggle between productivity on the one hand versus allowing the programmer to be able to express anything that the hardware can do on the other hand happens often in parallel computing. It will be interesting to see how the language evolves in this classic productivity-performance battle as well as to see whether CUDA becomes popular for other GPUs or even other architectural styles.

NVIDIA GPU Computational Structures

The uncommon heritage mentioned above helps explain why GPUs have their own architectural style and their own terminology independent from CPUs. One obstacle to understanding GPUs has been the jargon, with some terms even having misleading names. This obstacle has been surprisingly difficult to overcome, as the many rewrites of this chapter can attest.

To try to bridge the twin goals of making the architecture of GPUs understandable *and* learning the many GPU terms with nontraditional definitions, our approach is to use the CUDA terminology for software but initially use more descriptive terms for the hardware, sometimes borrowing terms from OpenCL. Once we explain the GPU architecture in our terms, we'll map them into the official jargon of NVIDIA GPUs.

From left to right, Figure 4.12 lists the descriptive term used in this section, the closest term from mainstream computing, the official NVIDIA GPU jargon in case you are interested, and then a short explanation of the term. The rest of this section explains the microarchitectural features of GPUs using the descriptive terms on the left in the figure.

We use NVIDIA systems as our example as they are representative of GPU architectures. Specifically, we follow the terminology of the preceding CUDA parallel programming language and use the NVIDIA Pascal GPU as the example (see Section 4.7).

Like vector architectures, GPUs work well only with data-level parallel problems. Both styles have gather-scatter data transfers and mask registers, and GPU processors have even more registers than do vector processors. Sometimes, GPUs implement certain features in hardware that vector processors would implement in software. This difference is because vector processors have a scalar processor that can execute a software function. Unlike most vector architectures,

Type	Descriptive name	Closest old term outside of GPUs	Official CUDA/NVIDIA GPU term	Short explanation
Program abstractions	Vectorizable Loop	Vectorizable Loop	Grid	A vectorizable loop, executed on the GPU, made up of one or more Thread Blocks (bodies of vectorized loop) that can execute in parallel
	Body of Vectorized Loop	Body of a (Strip-Mined) Vectorized Loop	Thread Block	A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. They can communicate via local memory
	Sequence of SIMD Lane Operations	One iteration of a Scalar Loop	CUDA Thread	A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask and predicate register
Machine object	A Thread of SIMD Instructions	Thread of Vector Instructions	Warp	A traditional thread, but it only contains SIMD instructions that are executed on a multithreaded SIMD Processor. Results stored depending on a per-element mask
	SIMD Instruction	Vector Instruction	PTX Instruction	A single SIMD instruction executed across SIMD Lanes
Processing hardware	Multithreaded SIMD Processor	(Multithreaded) Vector Processor	Streaming Multiprocessor	A multithreaded SIMD Processor executes threads of SIMD instructions, independent of other SIMD Processors
	Thread Block Scheduler	Scalar Processor	Giga Thread Engine	Assigns multiple Thread Blocks (bodies of vectorized loop) to multithreaded SIMD Processors
	SIMD Thread Scheduler	Thread Scheduler in a Multithreaded CPU	Warp Scheduler	Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution
	SIMD Lane	Vector Lane	Thread Processor	A SIMD Lane executes the operations in a thread of SIMD instructions on a single element. Results stored depending on mask
Memory hardware	GPU Memory	Main Memory	Global Memory	DRAM memory accessible by all multithreaded SIMD Processors in a GPU
	Private Memory	Stack or Thread Local Storage (OS)	Local Memory	Portion of DRAM memory private to each SIMD Lane
	Local Memory	Local Memory	Shared Memory	Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors
	SIMD Lane Registers	Vector Lane Registers	Thread Processor Registers	Registers in a single SIMD Lane allocated across a full Thread Block (body of vectorized loop)

Figure 4.12 Quick guide to GPU terms used in this chapter. We use the first column for hardware terms. Four groups cluster these 11 terms. From top to bottom: program abstractions, machine objects, processing hardware, and memory hardware. Figure 4.21 on page 312 associates vector terms with the closest terms here, and Figure 4.24 on page 317 and Figure 4.25 on page 318 reveal the official CUDA/NVIDIA and AMD terms and definitions along with the terms used by OpenCL.

GPUs also rely on multithreading within a single multithreaded SIMD Processor to hide memory latency (see Chapters 2 and 3). However, efficient code for both vector architectures and GPUs requires programmers to think in groups of SIMD operations.

A *Grid* is the code that runs on a GPU that consists of a set of *Thread Blocks*. Figure 4.12 draws the analogy between a grid and a vectorized loop and between a Thread Block and the body of that loop (after it has been strip-mined, so that it is a full computation loop). To give a concrete example, let's suppose we want to multiply two vectors together, each 8192 elements long: `A = B * C`. We'll return to this example throughout this section. Figure 4.13 shows the relationship between this example and these first two GPU terms. The GPU code that works on the whole 8192 element multiply is called a *Grid* (or vectorized loop). To break it down into more manageable sizes, a Grid is composed of *Thread Blocks* (or body of a vectorized loop), each with up to 512 elements. Note that a SIMD instruction executes 32 elements at a time. With 8192 elements in the vectors, this example thus has 16 Thread Blocks because $16 = 8192 \div 512$. The Grid and Thread Block are programming abstractions implemented in GPU hardware that help programmers organize their CUDA code. (The Thread Block is analogous to a strip-mined vector loop with a vector length of 32.)

A Thread Block is assigned to a processor that executes that code, which we call a *multithreaded SIMD Processor*, by the *Thread Block Scheduler*. The programmer tells the Thread Block Scheduler, which is implemented in hardware, how many Thread Blocks to run. In this example, it would send 16 Thread Blocks to multithreaded SIMD Processors to compute all 8192 elements of this loop

Figure 4.14 shows a simplified block diagram of a multithreaded SIMD Processor. It is similar to a vector processor, but it has many parallel functional units instead of a few that are deeply pipelined, as in a vector processor. In the programming example in Figure 4.13, each multithreaded SIMD Processor is assigned 512 elements of the vectors to work on. SIMD Processors are full processors with separate PCs and are programmed using threads (see Chapter 3).

The GPU hardware then contains a collection of multithreaded SIMD Processors that execute a Grid of Thread Blocks (bodies of vectorized loop); that is, a GPU is a multiprocessor composed of multithreaded SIMD Processors.

A GPU can have from one to several dozen multithreaded SIMD Processors. For example, the Pascal P100 system has 56, while the smaller chips may have as few as one or two. To provide transparent scalability across models of GPUs with a differing number of multithreaded SIMD Processors, the Thread Block Scheduler assigns Thread Blocks (bodies of a vectorized loop) to multithreaded SIMD Processors. Figure 4.15 shows the floor plan of the P100 implementation of the Pascal architecture.

Dropping down one more level of detail, the machine object that the hardware creates, manages, schedules, and executes is a *thread of SIMD instructions*. It is a traditional thread that contains exclusively SIMD instructions. These threads of SIMD instructions have their own PCs, and they run on a multithreaded SIMD Processor. The *SIMD Thread Scheduler* knows which threads of SIMD instructions are ready to run and then sends them off to a dispatch unit to be run on

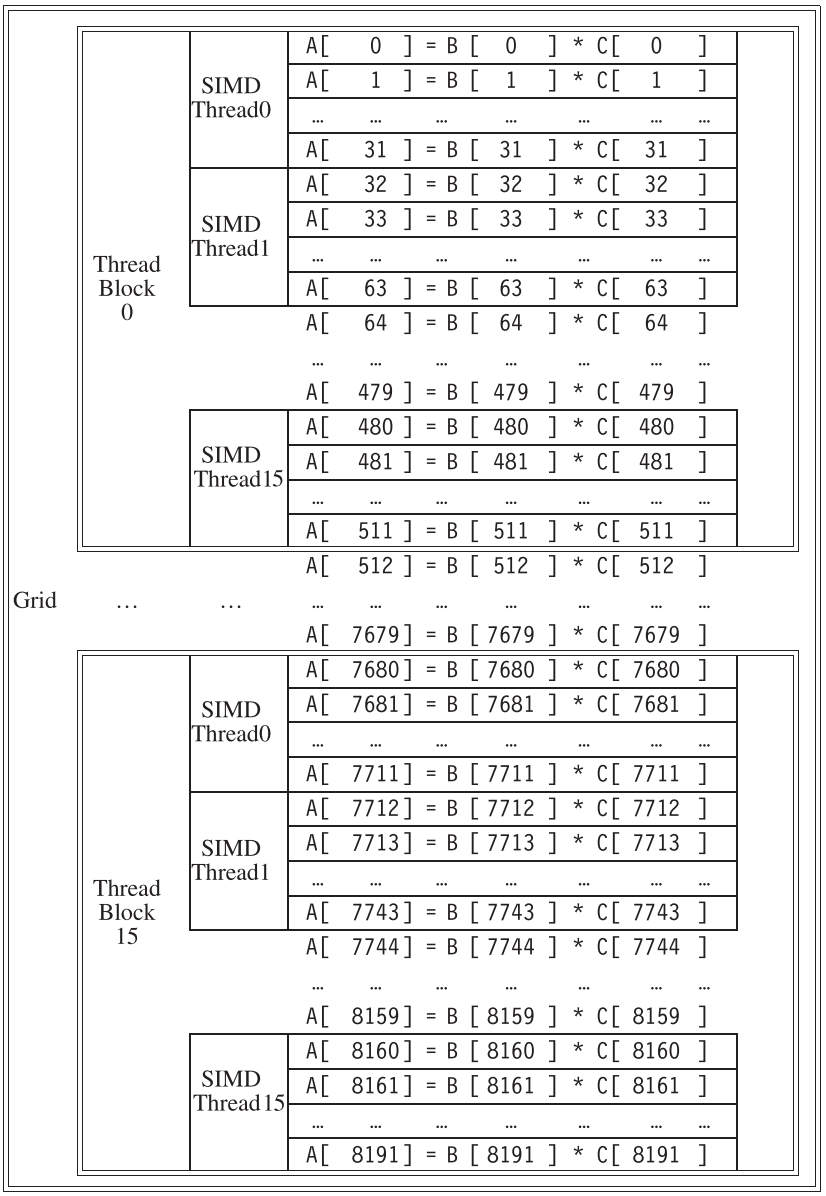


Figure 4.13 The mapping of a Grid (vectorizable loop), Thread Blocks (SIMD basic blocks), and threads of SIMD instructions to a vector-vector multiply, with each vector being 8192 elements long. Each thread of SIMD instructions calculates 32 elements per instruction, and in this example, each Thread Block contains 16 threads of SIMD instructions and the Grid contains 16 Thread Blocks. The hardware Thread Block Scheduler assigns Thread Blocks to multithreaded SIMD Processors, and the hardware Thread Scheduler picks which thread of SIMD instructions to run each clock cycle within a SIMD Processor. Only SIMD Threads in the same Thread Block can communicate via local memory. (The maximum number of SIMD Threads that can execute simultaneously per Thread Block is 32 for Pascal GPUs.)

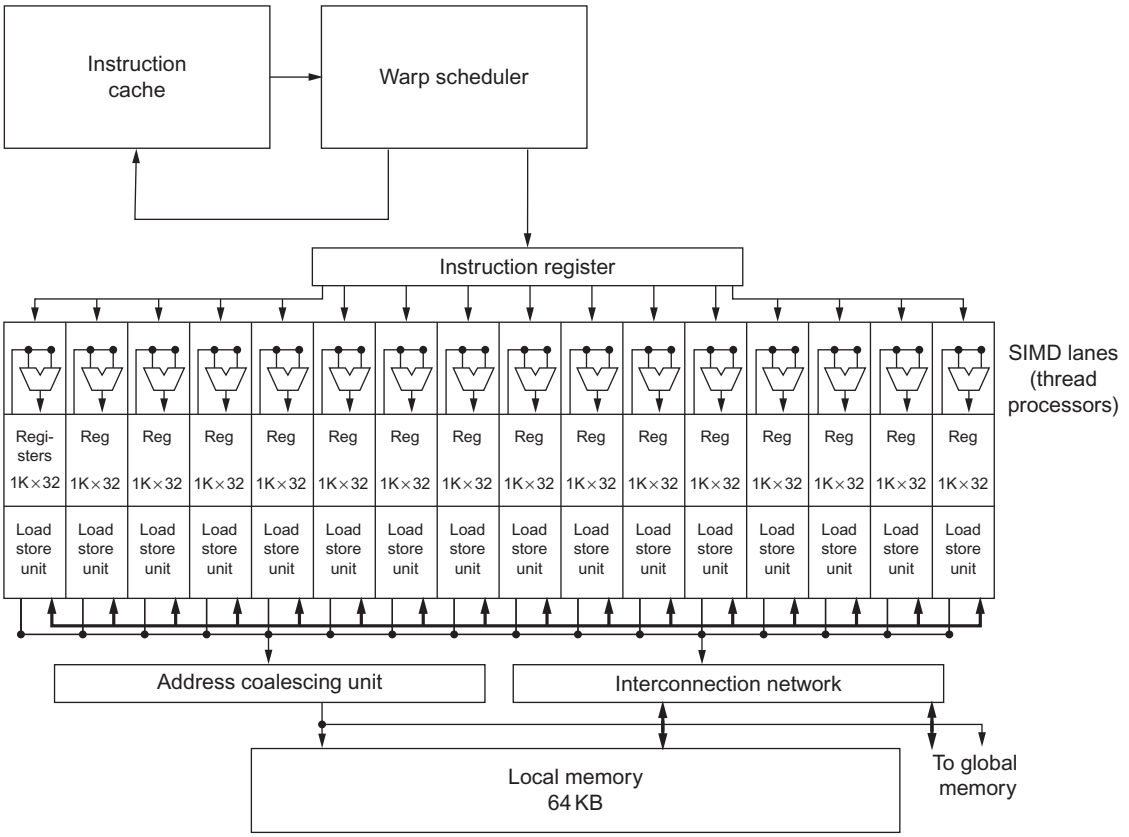


Figure 4.14 Simplified block diagram of a multithreaded SIMD Processor. It has 16 SIMD Lanes. The SIMD Thread Scheduler has, say, 64 independent threads of SIMD instructions that it schedules with a table of 64 program counters (PCs). Note that each lane has 1024 32-bit registers.

the multithreaded SIMD Processor. Thus GPU hardware has two levels of hardware schedulers: (1) the *Thread Block Scheduler* that assigns Thread Blocks (bodies of vectorized loops) to multithreaded SIMD Processors and (2) the SIMD Thread Scheduler *within* a SIMD Processor, which schedules when threads of SIMD instructions should run.

The SIMD instructions of these threads are 32 wide, so each thread of SIMD instructions in this example would compute 32 of the elements of the computation. In this example, Thread Blocks would contain 512/32 = 16 SIMD Threads (see Figure 4.13).

Because the thread consists of SIMD instructions, the SIMD Processor must have parallel functional units to perform the operation. We call them *SIMD Lanes*, and they are quite similar to the Vector Lanes in Section 4.2.

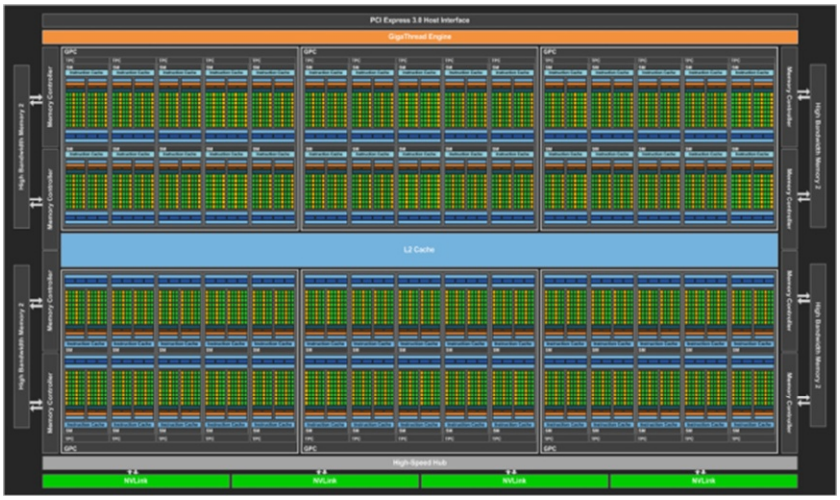


Figure 4.15 Full-chip block diagram of the Pascal P100 GPU. It has 56 multithreaded SIMD Processors, each with an L1 cache and local memory, 32 L2 units, and a memory-bus width of 4096 data wires. (It has 60 blocks, with four spares to improve yield.) The P100 has 4 HBM2 ports supporting up to 16 GB of capacity. It contains 15.4 billion transistors.

With the Pascal GPU, each 32-wide thread of SIMD instructions is mapped to 16 physical SIMD Lanes, so each SIMD instruction in a thread of SIMD instructions takes 2 clock cycles to complete. Each thread of SIMD instructions is executed in lock step and scheduled only at the beginning. Staying with the analogy of a SIMD Processor as a vector processor, you could say that it has 16 lanes, the vector length is 32, and the chime is 2 clock cycles. (This wide but shallow nature is why we use the more accurate term SIMD Processor rather than vector.)

Note that the number of lanes in a GPU SIMD Processor can be anything up to the number of threads in a Thread Block, just as the number of lanes in a vector processor can vary between 1 and the maximum vector length. For example, across GPU generations, the number of lanes per SIMD Processor has fluctuated between 8 and 32.

Because by definition the threads of SIMD instructions are independent, the SIMD Thread Scheduler can pick whatever thread of SIMD instructions is ready, and need not stick with the next SIMD instruction in the sequence within a thread. The SIMD Thread Scheduler includes a scoreboard (see Chapter 3) to keep track of up to 64 threads of SIMD instructions to see which SIMD instruction is ready to go. The latency of memory instructions is variable because of hits and misses in the caches and the TLB, thus the requirement of a scoreboard to determine when these instructions are complete. Figure 4.16 shows the SIMD Thread Scheduler picking threads of SIMD instructions in a different order over time. The assumption of GPU architects is that GPU applications have so many threads of SIMD instructions that multithreading can both hide the latency to DRAM and increase utilization of multithreaded SIMD Processors.

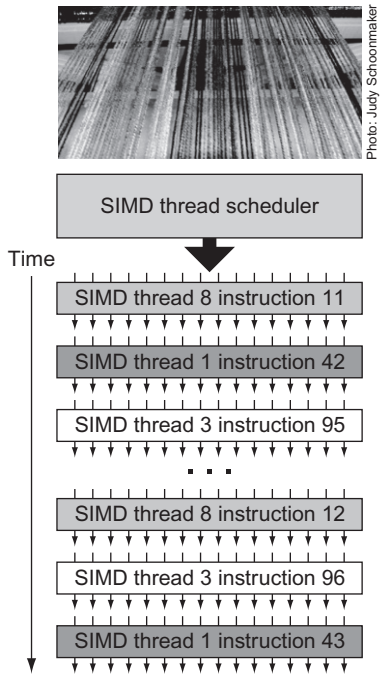


Figure 4.16 Scheduling of threads of SIMD instructions. The scheduler selects a ready thread of SIMD instructions and issues an instruction synchronously to all the SIMD Lanes executing the SIMD Thread. Because threads of SIMD instructions are independent, the scheduler may select a different SIMD Thread each time.

Continuing our vector multiply example, each multithreaded SIMD Processor must load 32 elements of two vectors from memory into registers, perform the multiply by reading and writing registers, and store the product back from registers into memory. To hold these memory elements, a SIMD Processor has between an impressive 32,768–65,536 32-bit registers (1024 per lane in Figure 4.14), depending on the model of the Pascal GPU. Just like a vector processor, these registers are divided logically across the Vector Lanes or, in this case, SIMD Lanes.

Each SIMD Thread is limited to no more than 256 registers, so you might think of a SIMD Thread as having up to 256 vector registers, with each vector register having 32 elements and each element being 32 bits wide. (Because double-precision floating-point operands use two adjacent 32-bit registers, an alternative view is that each SIMD Thread has 128 vector registers of 32 elements, each of which is 64 bits wide.)

There is a trade-off between register use and maximum number of threads; fewer registers per thread means more threads are possible, and more registers

mean fewer threads. That is, not all SIMD Threads need to have the maximum number of registers. Pascal architects believe much of this precious silicon area would be idle if all threads had the maximum number of registers.

To be able to execute many threads of SIMD instructions, each is dynamically allocated a set of the physical registers on each SIMD Processor when threads of SIMD instructions are created and freed when the SIMD Thread exits. For example, a programmer can have a Thread Block that uses 36 registers per thread with, say, 16 SIMD Threads alongside another Thread Block that has 20 registers per thread with 32 SIMD Threads. Subsequent Thread Blocks may show up in any order, and the registers have to be allocated on demand. While this variability can lead to fragmentation and make some registers unavailable, in practice most Thread Blocks use the same number of registers for a given vectorizable loop ("grid"). The hardware must know where the registers for each Thread Block are in the large register file, and this is recorded on a per Thread-Block basis. This flexibility requires routing, arbitration, and banking in the hardware because a specific register for a given Thread Block could end up in any location in the register file.

Note that a CUDA Thread is just a vertical cut of a thread of SIMD instructions, corresponding to one element executed by one SIMD Lane. Beware that CUDA Threads are very different from POSIX Threads; you can't make arbitrary system calls from a CUDA Thread.

We're now ready to see what GPU instructions look like.

NVIDA GPU Instruction Set Architecture

Unlike most system processors, the instruction set target of the NVIDIA compilers is an abstraction of the hardware instruction set. *PTX* (*Parallel Thread Execution*) provides a stable instruction set for compilers as well as compatibility across generations of GPUs. The hardware instruction set is hidden from the programmer. PTX instructions describe the operations on a single CUDA Thread and usually map one-to-one with hardware instructions, but one PTX instruction can expand to many machine instructions, and vice versa. PTX uses an unlimited number of write-once registers and the compiler must run a register allocation procedure to map the PTX registers to a fixed number of read-write hardware registers available on the actual device. The optimizer runs subsequently and can reduce register use even further. This optimizer also eliminates dead code, folds instructions together, and calculates places where branches might diverge and places where diverged paths could converge.

Although there is some similarity between the x86 microarchitecture and PTX, in that both translate to an internal form (microinstructions for x86), the difference is that this translation happens in hardware at runtime during execution on the x86 versus in software and load time on a GPU.

The format of a PTX instruction is

```
opcode.type d, a, b, c;
```

where `d` is the destination operand; `a`, `b`, and `c` are source operands; and the operation type is one of the following:

Type	.type specifier
Untyped bits 8, 16, 32, and 64 bits	.b8, .b16, .b32, .b64
Unsigned integer 8, 16, 32, and 64 bits	.u8, .u16, .u32, .u64
Signed integer 8, 16, 32, and 64 bits	.s8, .s16, .s32, .s64
Floating Point 16, 32, and 64 bits	.f16, .f32, .f64

Source operands are 32-bit or 64-bit registers or a constant value. Destinations are registers, except for store instructions.

Figure 4.17 shows the basic PTX instruction set. All instructions can be predicated by 1-bit predicate registers, which can be set by a set predicate instruction (setp). The control flow instructions are functions call and return, thread exit, branch, and barrier synchronization for threads within a Thread Block (bar.sync). Placing a predicate in front of a branch instruction gives us conditional branches. The compiler or PTX programmer declares virtual registers as 32-bit or 64-bit typed or untyped values. For example, R0, R1, . . . are for 32-bit values and RD0, RD1, . . . are for 64-bit registers. Recall that the assignment of virtual registers to physical registers occurs at load time with PTX.

The following sequence of PTX instructions is for one iteration of our DAXPY loop on page 292:

```
shl.u32 R8, blockIdx, 8   ; Thread Block ID * Block size
                          ;(256 or 2^8)
add.u32 R8, R8, threadIdx ; R8 = i = my CUDA Thread ID
shl.u32 R8, R8, 3         ; byte offset
ld.global.f64 RD0, [X+R8]; RD0 = X[i]
ld.global.f64 RD2, [Y+R8]; RD2 = Y[i]
mul.f64 RD0, RD0, RD4     ; Product in RD0 = RD0 * RD4
                          ; (scalar a)
add.f64 RD0, RD0, RD2     ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0; Y[i] = sum (X[i]*a + Y[i])
```

As demonstrated above, the CUDA programming model assigns one CUDA Thread to each loop iteration and offers a unique identifier number to each Thread Block (blockIdx) and one to each CUDA Thread within a block (threadIdx). Thus it creates 8192 CUDA Threads and uses the unique number to address each element within the array, so there is no incrementing or branching code. The first three PTX instructions calculate that unique element byte offset in R8, which is added to the base of the arrays. The following PTX instructions load two double-precision floating-point operands, multiply and add them, and store the sum. (We'll describe the PTX code corresponding to the CUDA code "if (i < n)" below.)

Note that unlike vector architectures, GPUs don't have separate instructions for sequential data transfers, strided data transfers, and gather-scatter data transfers.

Group	Instruction	Example	Meaning	Comments
Arithmetic	arithmetic .type = .s32, .u32, .f32, .s64, .u64, .f64			
	add.type	add.f32 d, a, b	d = a + b;	
	sub.type	sub.f32 d, a, b	d = a - b;	
	mul.type	mul.f32 d, a, b	d = a * b;	
	mad.type	mad.f32 d, a, b, c	d = a * b + c;	multiply-add
	div.type	div.f32 d, a, b	d = a / b;	multiple microinstructions
	rem.type	rem.u32 d, a, b	d = a % b;	integer remainder
	abs.type	abs.f32 d, a	d = |a|;	
	neg.type	neg.f32 d, a	d = 0 - a;	
	min.type	min.f32 d, a, b	d = (a < b)? a:b;	floating selects non-NaN
	max.type	max.f32 d, a, b	d = (a > b)? a:b;	floating selects non-NaN
	setp.cmp.type	setp.lt.f32 p, a, b	p = (a < b);	compare and set predicate
	numeric .cmp = eq, ne, lt, le, gt, ge; unordered cmp = equ, neu, ltu, leu, gtu, geu, num, nan			
	mov.type	mov.b32 d, a	d = a;	move
	selp.type	selp.f32 d, a, b, p	d = p? a: b;	select with predicate
	cvt.dtype.atype	cvt.f32.s32 d, a	d = convert(a);	convert atype to dtype
Special function	special .type = .f32 (some .f64)			
	rcp.type	rcp.f32 d, a	d = 1/a;	reciprocal
	sqrt.type	sqrt.f32 d, a	d = sqrt(a);	square root
	rsqrt.type	rsqrt.f32 d, a	d = 1/sqrt(a);	reciprocal square root
	sin.type	sin.f32 d, a	d = sin(a);	sine
	cos.type	cos.f32 d, a	d = cos(a);	cosine
	lg2.type	lg2.f32 d, a	d = log(a)/log(2)	binary logarithm
	ex2.type	ex2.f32 d, a	d = 2 ** a;	binary exponential
Logical	logic.type = .pred,.b32, .b64			
	and.type	and.b32 d, a, b	d = a & b;	
	or.type	or.b32 d, a, b	d = a | b;	
	xor.type	xor.b32 d, a, b	d = a ^b;	
	not.type	not.b32 d, a, b	d = ~a;	one's complement
	cnot.type	cnot.b32 d, a, b	d = (a==0)? 1:0;	C logical not
	shl.type	shl.b32 d, a, b	d = a << b;	shift left
	shr.type	shr.s32 d, a, b	d = a >> b;	shift right
Memory access	memory.space = .global, .shared, .local, .const; .type = .b8, .u8, .s8, .b16, .b32, .b64			
	ld.space.type	ld.global.b32 d, [a+off]	d = *(a+off);	load from memory space
	st.space.type	st.shared.b32 [d+off], a	*(d+off) = a;	store to memory space
	tex.nd.dtyp.btype	tex.2d.v4.f32.f32 d, a, b	d = tex2d(a, b);	texture lookup
	atom.spc.op.type	atom.global.add.u32 d,[a], b atom.global.cas.b32 d,[a], b, c	atomic { d = *a; *a = op(*a, b); }	atomic read-modify-write operation
	atom.op = and, or, xor, add, min, max, exch, cas; .spc = .global; .type = .b32			
Control flow	branch	@p bra target	if (p) goto target;	conditional branch
	call	call (ret), func, (params)	ret = func(params);	call function
	ret	ret	return;	return from function call
	bar.sync	bar.sync d	wait for threads	barrier synchronization
	exit	exit	exit;	terminate thread execution

Figure 4.17 Basic PTX GPU thread instructions.

All data transfers are gather-scatter! To regain the efficiency of sequential (unit-stride) data transfers, GPUs include special Address Coalescing hardware to recognize when the SIMD Lanes within a thread of SIMD instructions are collectively issuing sequential addresses. That runtime hardware then notifies the Memory Interface Unit to request a block transfer of 32 sequential words. To get this important performance improvement, the GPU programmer must ensure that adjacent CUDA Threads access nearby addresses at the same time so that they can be coalesced into one or a few memory or cache blocks, which our example does.

Conditional Branching in GPUs

Just like the case with unit-stride data transfers, there are strong similarities between how vector architectures and GPUs handle IF statements, with the former implementing the mechanism largely in software with limited hardware support and the latter making use of even more hardware. As we will see, in addition to explicit predicate registers, GPU branch hardware uses internal masks, a branch synchronization stack, and instruction markers to manage when a branch diverges into multiple execution paths and when the paths converge.

At the PTX assembler level, control flow of one CUDA Thread is described by the PTX instructions branch, call, return, and exit, plus individual per-thread-lane predication of each instruction, specified by the programmer with per-thread-lane 1-bit predicate registers. The PTX assembler analyzes the PTX branch graph and optimizes it to the fastest GPU hardware instruction sequence. Each can make its own decision on a branch and does not need to be in lock step.

At the GPU hardware instruction level, control flow includes branch, jump, jump indexed, call, call indexed, return, exit, and special instructions that manage the branch synchronization stack. GPU hardware provides each SIMD Thread with its own stack; a stack entry contains an identifier token, a target instruction address, and a target thread-active mask. There are GPU special instructions that push stack entries for a SIMD Thread and special instructions and instruction markers that pop a stack entry or unwind the stack to a specified entry and branch to the target instruction address with the target thread-active mask. GPU hardware instructions also have an individual per-lane predication (enable/disable), specified with a 1-bit predicate register for each lane.

The PTX assembler typically optimizes a simple outer-level IF-THEN-ELSE statement coded with PTX branch instructions to solely predicated GPU instructions, without any GPU branch instructions. A more complex control flow often results in a mixture of predication and GPU branch instructions with special instructions and markers that use the branch synchronization stack to push a stack entry when some lanes branch to the target address, while others fall through. NVIDIA says a branch *diverges* when this happens. This mixture is also used when a SIMD Lane executes a synchronization marker or *converges*, which pops a stack entry and branches to the stack-entry address with the stack-entry thread-active mask.

The PTX assembler identifies loop branches and generates GPU branch instructions that branch to the top of the loop, along with special stack instructions to handle individual lanes breaking out of the loop and converging the SIMD Lanes when all lanes have completed the loop. GPU indexed jump and indexed call instructions push entries on the stack so that when all lanes complete the switch statement or function call, the SIMD Thread converges.

A GPU set predicate instruction (`setp` in Figure 4.17) evaluates the conditional part of the IF statement. The PTX branch instruction then depends on that predicate. If the PTX assembler generates predicated instructions with no GPU branch instructions, it uses a per-lane predicate register to enable or disable each SIMD Lane for each instruction. The SIMD instructions in the threads inside the THEN part of the IF statement broadcast operations to all the SIMD Lanes. Those lanes with the predicate set to 1 perform the operation and store the result, and the other SIMD Lanes don't perform an operation or store a result. For the ELSE statement, the instructions use the complement of the predicate (relative to the THEN statement), so the SIMD Lanes that were idle now perform the operation and store the result while their formerly active siblings don't. At the end of the ELSE statement, the instructions are unpredicated so the original computation can proceed. Thus, for equal length paths, an IF-THEN-ELSE operates at 50% efficiency or less.

IF statements can be nested, thus the use of a stack, and the PTX assembler typically generates a mix of predicated instructions and GPU branch and special synchronization instructions for complex control flow. Note that deep nesting can mean that most SIMD Lanes are idle during execution of nested conditional statements. Thus, doubly nested IF statements with equal-length paths run at 25% efficiency, triply nested at 12.5% efficiency, and so on. The analogous case would be a vector processor operating where only a few of the mask bits are ones.

Dropping down a level of detail, the PTX assembler sets a "branch synchronization" marker on appropriate conditional branch instructions that pushes the current active mask on a stack inside each SIMD Thread. If the conditional branch diverges (some lanes take the branch but some fall through), it pushes a stack entry and sets the current internal active mask based on the condition. A branch synchronization marker pops the diverged branch entry and flips the mask bits before the ELSE portion. At the end of the IF statement, the PTX assembler adds another branch synchronization marker that pops the prior active mask off the stack into the current active mask.

If all the mask bits are set to 1, then the branch instruction at the end of the THEN skips over the instructions in the ELSE part. There is a similar optimization for the THEN part in case all the mask bits are 0 because the conditional branch jumps over the THEN instructions. Parallel IF statements and PTX branches often use branch conditions that are unanimous (all lanes agree to follow the same path) such that the SIMD Thread does not diverge into a different individual lane control flow. The PTX assembler optimizes such branches to skip over blocks of instructions that are not executed by any lane of a SIMD Thread. This optimization is

useful in conditional error checking, for example, where the test must be made but is rarely taken.

The code for a conditional statement similar to the one in Section 4.2 is

```
if (X[i] != 0)
  X[i] = X[i] - Y[i];
else X[i] = Z[i];
```

This IF statement could compile to the following PTX instructions (assuming that R8 already has the scaled thread ID), with **Push*, **Comp*, **Pop* indicating the branch synchronization markers inserted by the PTX assembler that push the old mask, complement the current mask, and pop to restore the old mask:

```
   ld.global.f64 RD0, [X+R8]      ; RD0 = X[i]
   setp.neq.s32 P1, RD0, #0       ;P1 is predicate reg 1
   @!P1, bra ELSE1, *Push         ; Push old mask, set new
                     ; mask bits if P1 false, go to ELSE1
   ld.global.f64 RD2, [Y+R8]      ; RD2 = Y[i]
   sub.f64 RD0, RD0, RD2          ; Difference in RD0
   st.global.f64 [X+R8], RD0      ; X[i] = RD0
   @P1, bra ENDIF1, *Comp         ; complement mask bits
                                  ; if P1 true, go to ENDIF1
ELSE1: ld.global.f64 RD0, [Z+R8] ; RD0 = Z[i]
       st.global.f64 [X+R8], RD0 ; X[i] = RD0
ENDIF1:<next instruction>, *Pop  ; pop to restore old mask
```

Once again, normally all instructions in the IF-THEN-ELSE statement are executed by a SIMD Processor. It's just that only some of the SIMD Lanes are enabled for the THEN instructions and some lanes for the ELSE instructions. As previously mentioned, in the surprisingly common case that the individual lanes agree on the predicated branch—such as branching on a parameter value that is the same for all lanes so that all active mask bits are 0s or all are 1s—the branch skips the THEN instructions or the ELSE instructions.

This flexibility makes it appear that an element has its own program counter; however, in the slowest case, only one SIMD Lane could store its result every 2 clock cycles, with the rest idle. The analogous slowest case for vector architectures is operating with only one mask bit set to 1. This flexibility can lead naive GPU programmers to poor performance, but it can be helpful in the early stages of program development. Keep in mind, however, that the only choice for a SIMD Lane in a clock cycle is to perform the operation specified in the PTX instruction or be idle; two SIMD Lanes cannot simultaneously execute different instructions.

This flexibility also helps explain the name *CUDA Thread* given to each element in a thread of SIMD instructions, because it gives the illusion of acting independently. A naive programmer may think that this thread abstraction means GPUs handle conditional branches more gracefully. Some threads go one way, the rest go

another, which seems true as long as you're not in a hurry. Each CUDA Thread is either executing the same instruction as every other thread in the Thread Block or it is idle. This synchronization makes it easier to handle loops with conditional branches because the mask capability can turn off SIMD Lanes and it detects the end of the loop automatically.

The resulting performance sometimes belies that simple abstraction. Writing programs that operate SIMD Lanes in this highly independent MIMD mode is like writing programs that use lots of virtual address space on a computer with a smaller physical memory. Both are correct, but they may run so slowly that the programmer will not be pleased with the result.

Conditional execution is a case where GPUs do in runtime hardware what vector architectures do at compile time. Vector compilers do a double IF-conversion, generating four different masks. The execution is basically the same as GPUs, but there are some more overhead instructions executed for vectors. Vector architectures have the advantage of being integrated with a scalar processor, allowing them to avoid the time for the 0 cases when they dominate a calculation. Although it will depend on the speed of the scalar processor versus the vector processor, the crossover point when it's better to use scalar might be when less than 20% of the mask bits are 1s. One optimization available at runtime for GPUs, but not at compile time for vector architectures, is to skip the THEN or ELSE parts when mask bits are all 0s or all 1s.

Thus the efficiency with which GPUs execute conditional statements comes down to how frequently the branches will diverge. For example, one calculation of eigenvalues has deep conditional nesting, but measurements of the code show that around 82% of clock cycle issues have between 29 and 32 out of the 32 mask bits set to 1, so GPUs execute this code more efficiently than one might expect.

Note that the same mechanism handles the strip-mining of vector loops—when the number of elements doesn't perfectly match the hardware. The example at the beginning of this section shows that an IF statement checks to see if this SIMD Lane element number (stored in `R8` in the preceding example) is less than the limit `(i < n)`, and it sets masks appropriately.

NVIDIA GPU Memory Structures

Figure 4.18 shows the memory structures of an NVIDIA GPU. Each SIMD Lane in a multithreaded SIMD Processor is given a private section of off-chip DRAM, which we call the *private memory*. It is used for the stack frame, for spilling registers, and for private variables that don't fit in the registers. SIMD Lanes do *not* share private memories. GPUs cache this private memory in the L1 and L2 caches to aid register spilling and to speed up function calls.

We call the on-chip memory that is local to each multithreaded SIMD Processor *local memory*. It is a small scratchpad memory with low latency (a few dozen clocks) and high bandwidth (128 bytes/clock) where the programmer can store data that needs to be reused, either by the same thread or another thread in the same

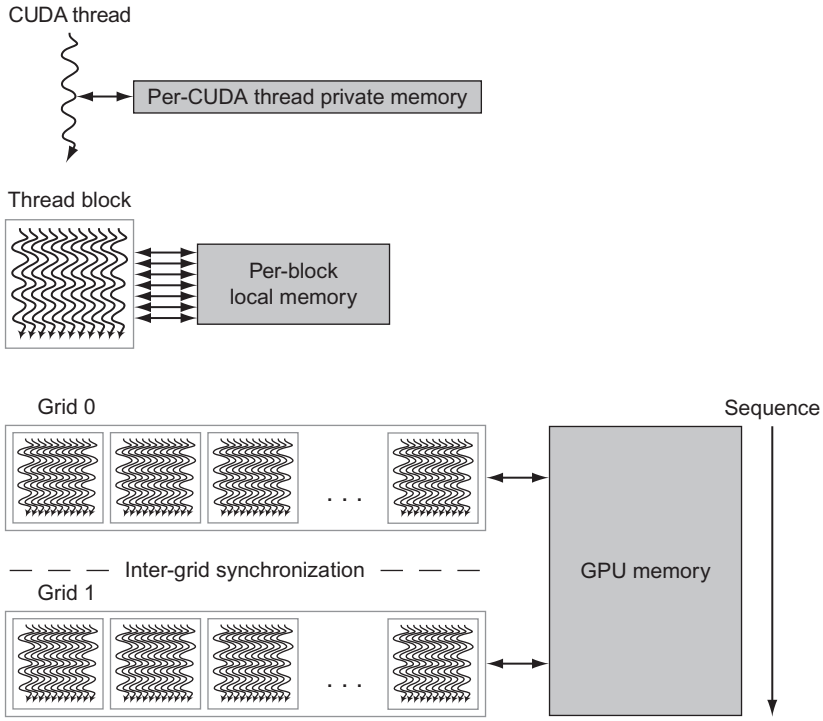


Figure 4.18 GPU memory structures. GPU memory is shared by all Grids (vectorized loops), local memory is shared by all threads of SIMD instructions within a Thread Block (body of a vectorized loop), and private memory is private to a single CUDA Thread. Pascal allows preemption of a Grid, which requires that all local and private memory be able to be saved in and restored from global memory. For completeness sake, the GPU can also access CPU memory via the PCIe bus. This path is commonly used for a final result when its address is in host memory. This option eliminates a final copy from the GPU memory to the host memory.

Thread Block. Local memory is limited in size, typically to 48 KiB. It carries no state between Thread Blocks executed on the same processor. It is shared by the SIMD Lanes within a multithreaded SIMD Processor, but this memory is not shared between multithreaded SIMD Processors. The multithreaded SIMD Processor dynamically allocates portions of the local memory to a Thread Block when it creates the Thread Block, and frees the memory when all the threads of the Thread Block exit. That portion of local memory is private to that Thread Block.

Finally, we call the off-chip DRAM shared by the whole GPU and all Thread Blocks *GPU Memory*. Our vector multiply example used only GPU Memory.

The system processor, called the *host*, can read or write GPU Memory. Local memory is unavailable to the host, as it is private to each multithreaded SIMD Processor. Private memories are unavailable to the host as well.

Rather than rely on large caches to contain the whole working sets of an application, GPUs traditionally use smaller streaming caches and, because their working sets can be hundreds of megabytes, rely on extensive multithreading of threads of SIMD instructions to hide the long latency to DRAM. Given the use of multithreading to hide DRAM latency, the chip area used for large L2 and L3 caches in system processors is spent instead on computing resources and on the large number of registers to hold the state of many threads of SIMD instructions. In contrast, as mentioned, vector loads and stores amortize the latency across many elements because they pay the latency only once and then pipeline the rest of the accesses.

Although hiding memory latency behind many threads was the original philosophy of GPUs and vector processors, all recent GPUs and vector processors have caches to reduce latency. The argument follows Little's Law from queuing theory: the longer the latency, the more threads need to run during a memory access, which in turn requires more registers. Thus GPU caches are added to lower average latency and thereby mask potential shortages of the number of registers.

To improve memory bandwidth and reduce overhead, as mentioned, PTX data transfer instructions in cooperation with the memory controller coalesce individual parallel thread requests from the same SIMD Thread together into a single memory block request when the addresses fall in the same block. These restrictions are placed on the GPU program, somewhat analogous to the guidelines for system processor programs to engage hardware prefetching (see Chapter 2). The GPU memory controller will also hold requests and send ones together to the same open page to improve memory bandwidth (see Section 4.6). Chapter 2 describes DRAM in sufficient detail for readers to understand the potential benefits of grouping related addresses.

Innovations in the Pascal GPU Architecture

The multithreaded SIMD Processor of Pascal is more complicated than the simplified version in Figure 4.20. To increase hardware utilization, each SIMD Processor has two SIMD Thread Schedulers, each with multiple instruction dispatch units (some GPUs have four thread schedulers). The dual SIMD Thread Scheduler selects two threads of SIMD instructions and issues one instruction from each to two sets of 16 SIMD Lanes, 16 load/store units, or 8 special function units. With multiple execution units available, two threads of SIMD instructions are scheduled each clock cycle, allowing 64 lanes to be active. Because the threads are independent, there is no need to check for data dependences in the instruction stream. This innovation would be analogous to a multithreaded vector processor that can issue vector instructions from two independent threads. Figure 4.19 shows the Dual Scheduler issuing instructions, and Figure 4.20 shows the block diagram of the multithreaded SIMD Processor of a Pascal GP100 GPU.

Each new generation of GPU typically adds some new features that increase performance or make it easier for programmers. Here are the four main innovations of Pascal:

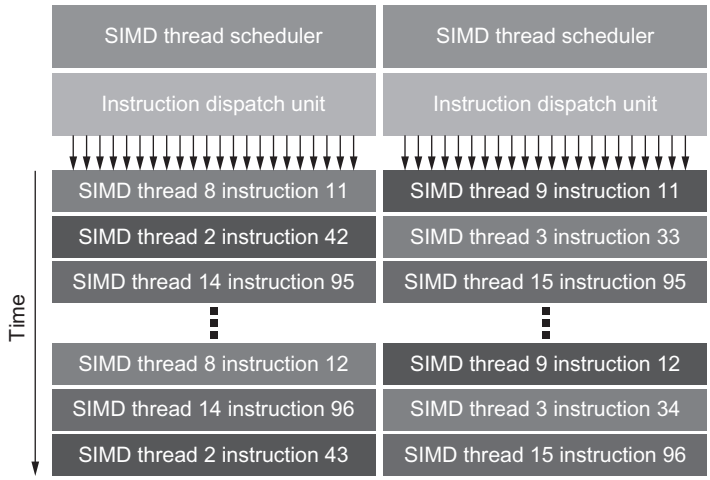


Figure 4.19 Block diagram of Pascal's dual SIMD Thread scheduler. Compare this design to the single SIMD Thread design in Figure 4.16.

- *Fast single-precision, double-precision, and half-precision floating-point arithmetic*—Pascal GP100 chip has significant floating-point performance in three sizes, all part of the IEEE standard for floating-point. The single-precision floating-point of the GPU runs at a peak of 10 TeraFLOP/s. Double-precision is roughly half-speed at 5 TeraFLOP/s, and half-precision is about double-speed at 20 TeraFLOP/s when expressed as 2-element vectors. The atomic memory operations include floating-point add for all three sizes. Pascal GP100 is the first GPU with such high performance for half-precision.
- *High-bandwidth memory*—The next innovation of the Pascal GP100 GPU is the use of stacked, high-bandwidth memory (*HBM2*). This memory has a wide bus with 4096 data wires running at 0.7 GHz offering a peak bandwidth of 732 GB/s, which is more than twice as fast as previous GPUs.
- *High-speed chip-to-chip interconnect*—Given the coprocessor nature of GPUs, the PCI bus can be a communications bottleneck when trying to use multiple GPUs with one CPU. Pascal GP100 introduces the *NVLink* communications channel that supports data transfers of up to 20 GB/s in each direction. Each GP100 has 4 NVLink channels, providing a peak aggregate chip-to-chip bandwidth of 160 GB/s per chip. Systems with 2, 4, and 8 GPUs are available for multi-GPU applications, where each GPU can perform load, store, and atomic operations to any GPU connected by NVLink. Additionally, an NVLink channel can communicate with the CPU in some cases. For example, the IBM Power9 CPU supports CPU-GPU communication. In this chip, NVLink provides a coherent view of memory between all GPUs and CPUs connected together. It also provides cache-to-cache communication instead of memory-to-memory communication.



Figure 4.20 Block diagram of the multithreaded SIMD Processor of a Pascal GPU. Each of the 64 SIMD Lanes (cores) has a pipelined floating-point unit, a pipelined integer unit, some logic for dispatching instructions and operands to these units, and a queue for holding results. The 64 SIMD Lanes interact with 32 double-precision ALUs (DP units) that perform 64-bit floating-point arithmetic, 16 load-store units (LD/STs), and 16 special function units (SFUs) that calculate functions such as square roots, reciprocals, sines, and cosines.

- *Unified virtual memory and paging support*—The Pascal GP100 GPU adds page-fault capabilities within a unified virtual address space. This feature allows a single virtual address for every data structure that is identical across all the GPUs and CPUs in a single system. When a thread accesses an address that is remote, a page of memory is transferred to the local GPU for subsequent use. Unified memory simplifies the programming model by providing demand paging instead of explicit memory copying between the CPU and GPU or

between GPUs. It also allows allocating far more memory than exists on the GPU to solve problems with large memory requirements. As with any virtual memory system, care must be taken to avoid excessive page movement.

Similarities and Differences Between Vector Architectures and GPUs

As we have seen, there really are many similarities between vector architectures and GPUs. Along with the quirky jargon of GPUs, these similarities have contributed to the confusion in architecture circles about how novel GPUs really are. Now that you've seen what is under the covers of vector computers and GPUs, you can appreciate both the similarities and the differences. Because both architectures are designed to execute data-level parallel programs, but take different paths, this comparison is in depth in order to provide a better understanding of what is needed for DLP hardware. Figure 4.21 shows the vector term first and then the closest equivalent in a GPU.

A SIMD Processor is like a vector processor. The multiple SIMD Processors in GPUs act as independent MIMD cores, just as many vector computers have multiple vector processors. This view will consider the NVIDIA Tesla P100 as a 56-core machine with hardware support for multithreading, where each core has 64 lanes. The biggest difference is multithreading, which is fundamental to GPUs and missing from most vector processors.

Looking at the registers in the two architectures, the RV64V register file in our implementation holds entire vectors—that is, a contiguous block of elements. In contrast, a single vector in a GPU will be distributed across the registers of all SIMD Lanes. A RV64V processor has 32 vector registers with perhaps 32 elements, or 1024 elements total. A GPU thread of SIMD instructions has up to 256 registers with 32 elements each, or 8192 elements. These extra GPU registers support multithreading.

Figure 4.22 is a block diagram of the execution units of a vector processor on the left and a multithreaded SIMD Processor of a GPU on the right. For pedagogic purposes, we assume the vector processor has four lanes and the multithreaded SIMD Processor also has four SIMD Lanes. This figure shows that the four SIMD Lanes act in concert much like a four-lane vector unit, and that a SIMD Processor acts much like a vector processor.

In reality, there are many more lanes in GPUs, so GPU "chimes" are shorter. While a vector processor might have 2 to 8 lanes and a vector length of, say, 32—making a chime 4 to 16 clock cycles—a multithreaded SIMD Processor might have 8 or 16 lanes. A SIMD Thread is 32 elements wide, so a GPU chime would just be 2 or 4 clock cycles. This difference is why we use "SIMD Processor" as the more descriptive term because it is closer to a SIMD design than it is to a traditional vector processor design.

Type	Vector term	Closest CUDA/NVIDIA GPU term	Comment
Program abstractions	Vectorized Loop	Grid	Concepts are similar, with the GPU using the less descriptive term
	Chime	—	Because a vector instruction (PTX instruction) takes just 2 cycles on Pascal to complete, a chime is short in GPUs. Pascal has two execution units that support the most common floating-point instructions that are used alternately, so the effective issue rate is 1 instruction every clock cycle
Machine objects	Vector Instruction	PTX Instruction	A PTX instruction of a SIMD Thread is broadcast to all SIMD Lanes, so it is similar to a vector instruction
	Gather/ Scatter	Global load/store (ld. global/st.global)	All GPU loads and stores are gather and scatter, in that each SIMD Lane sends a unique address. It's up to the GPU Coalescing Unit to get unit-stride performance when addresses from the SIMD Lanes allow it
	Mask Registers	Predicate Registers and Internal Mask Registers	Vector mask registers are explicitly part of the architectural state, while GPU mask registers are internal to the hardware. The GPU conditional hardware adds a new feature beyond predicate registers to manage masks dynamically
Processing and memory hardware	Vector Processor	Multithreaded SIMD Processor	These are similar, but SIMD Processors tend to have many lanes, taking a few clock cycles per lane to complete a vector, while vector architectures have few lanes and take many cycles to complete a vector. They are also multithreaded where vectors usually are not
	Control Processor	Thread Block Scheduler	The closest is the Thread Block Scheduler that assigns Thread Blocks to a multithreaded SIMD Processor. But GPUs have no scalar-vector operations and no unit-stride or strided data transfer instructions, which Control Processors often provide in vector architectures
	Scalar Processor	System Processor	Because of the lack of shared memory and the high latency to communicate over a PCI bus (1000s of clock cycles), the system processor in a GPU rarely takes on the same tasks that a scalar processor does in a vector architecture
	Vector Lane	SIMD Lane	Very similar; both are essentially functional units with registers
	Vector Registers	SIMD Lane Registers	The equivalent of a vector register is the same register in all 16 SIMD Lanes of a multithreaded SIMD Processor running a thread of SIMD instructions. The number of registers per SIMD Thread is flexible, but the maximum is 256 in Pascal, so the maximum number of vector registers is 256
	Main Memory	GPU Memory	Memory for GPU versus system memory in vector case

Figure 4.21 GPU equivalent to vector terms.

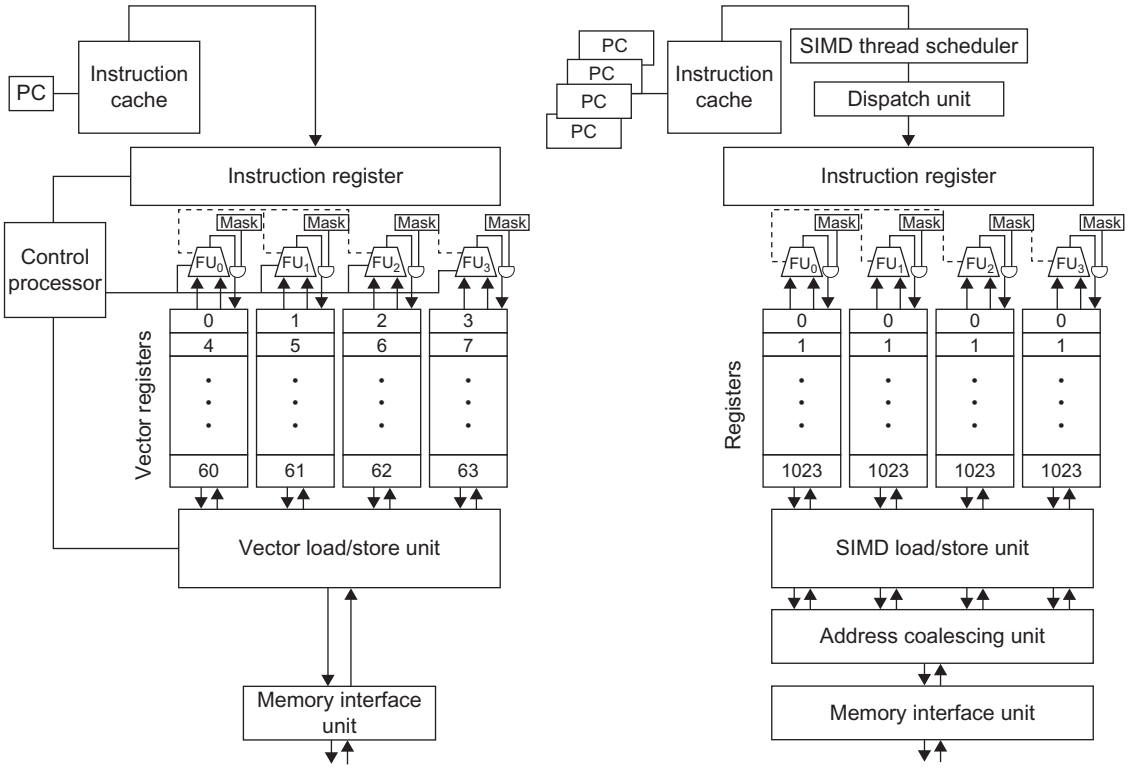


Figure 4.22 A vector processor with four lanes on the left and a multithreaded SIMD Processor of a GPU with four SIMD Lanes on the right. (GPUs typically have 16 or 32 SIMD Lanes.) The Control Processor supplies scalar operands for scalar-vector operations, increments addressing for unit and nonunit stride accesses to memory, and performs other accounting-type operations. Peak memory performance occurs only in a GPU when the Address Coalescing Unit can discover localized addressing. Similarly, peak computational performance occurs when all internal mask bits are set identically. Note that the SIMD Processor has one PC per SIMD Thread to help with multithreading.

The closest GPU term to a vectorized loop is Grid, and a PTX instruction is the closest to a vector instruction because a SIMD Thread broadcasts a PTX instruction to all SIMD Lanes.

With respect to memory access instructions in the two architectures, all GPU loads are gather instructions and all GPU stores are scatter instructions. If data addresses of CUDA Threads refer to nearby addresses that fall into the same cache/memory block at the same time, the Address Coalescing Unit of the GPU will ensure high memory bandwidth. The *explicit* unit-stride load and store instructions of vector architectures versus the *implicit* unit stride of GPU programming is why writing efficient GPU code requires that programmers think in terms of SIMD operations, even though the CUDA programming model looks like MIMD. Because CUDA Threads can generate their own addresses, strided as well as gather-scatter, addressing vectors are found in both vector architectures and GPUs.

As we mentioned several times, the two architectures take very different approaches to hiding memory latency. Vector architectures amortize it across all the elements of the vector by having a deeply pipelined access, so you pay the latency only once per vector load or store. Therefore vector loads and stores are like a block transfer between memory and the vector registers. In contrast, GPUs hide memory latency using multithreading. (Some researchers are investigating adding multithreading to vector architectures to try to capture the best of both worlds.)

With respect to conditional branch instructions, both architectures implement them using mask registers. Both conditional branch paths occupy time and/or space even when they do not store a result. The difference is that the vector compiler manages mask registers explicitly in software while the GPU hardware and assembler manages them implicitly using branch synchronization markers and an internal stack to save, complement, and restore masks.

The Control Processor of a vector computer plays an important role in the execution of vector instructions. It broadcasts operations to all the Vector Lanes and broadcasts a scalar register value for vector-scalar operations. It also does implicit calculations that are explicit in GPUs, such as automatically incrementing memory addresses for unit-stride and nonunit-stride loads and stores. The Control Processor is missing in the GPU. The closest analogy is the Thread Block Scheduler, which assigns Thread Blocks (bodies of vector loop) to multithreaded SIMD Processors. The runtime hardware mechanisms in a GPU that both generate addresses and then discover if they are adjacent, which is commonplace in many DLP applications, are likely less power-efficient than using a Control Processor.

The scalar processor in a vector computer executes the scalar instructions of a vector program; that is, it performs operations that would be too slow to do in the vector unit. Although the system processor that is associated with a GPU is the closest analogy to a scalar processor in a vector architecture, the separate address spaces plus transferring over a PCIe bus means thousands of clock cycles of overhead to use them together. The scalar processor can be slower than a vector processor for floating-point computations in a vector computer, but not by the same ratio as the system processor versus a multithreaded SIMD Processor (given the overhead).

Therefore each "vector unit" in a GPU must do computations that you would expect to do using a scalar processor in a vector computer. That is, rather than calculate on the system processor and communicate the results, it can be faster to disable all but one SIMD Lane using the predicate registers and built-in masks and do the scalar work with one SIMD Lane. The relatively simple scalar processor in a vector computer is likely to be faster and more power-efficient than the GPU solution. If system processors and GPUs become more closely tied together in the future, it will be interesting to see if system processors can play the same role as scalar processors do for vector and multimedia SIMD architectures.

Similarities and Differences Between Multimedia SIMD Computers and GPUs

At a high level, multicore computers with multimedia SIMD instruction extensions do share similarities with GPUs. Figure 4.23 summarizes the similarities and differences.

Both are multiprocessors whose processors use multiple SIMD Lanes, although GPUs have more processors and many more lanes. Both use hardware multithreading to improve processor utilization, although GPUs have hardware support for many more threads. Both have roughly 2:1 performance ratios between peak performance of single-precision and double-precision floating-point arithmetic. Both use caches, although GPUs use smaller streaming caches, and multicore computers use large multilevel caches that try to contain whole working sets completely. Both use a 64-bit address space, although the physical main memory is much smaller in GPUs. Both support memory protection at the page level as well as demand paging, which allows them to address far more memory than they have on board.

In addition to the large numerical differences in processors, SIMD Lanes, hardware thread support, and cache sizes, there are many architectural differences. The scalar processor and multimedia SIMD instructions are tightly integrated in traditional computers; they are separated by an I/O bus in GPUs, and they even have separate main memories. The multiple SIMD Processors in a GPU use a single address space and can support a coherent view of all memory on some systems given support from CPU vendors (such as the IBM Power9). Unlike GPUs, multimedia SIMD instructions historically did not support gather-scatter memory accesses, which Section 4.7 shows is a significant omission.

Feature	Multicore with SIMD	GPU
SIMD Processors	4–8	8–32
SIMD Lanes/Processor	2–4	up to 64
Multithreading hardware support for SIMD Threads	2–4	up to 64
Typical ratio of single-precision to double-precision performance	2:1	2:1
Largest cache size	40 MB	4 MB
Size of memory address	64-bit	64-bit
Size of main memory	up to 1024 GB	up to 24 GB
Memory protection at level of page	Yes	Yes
Demand paging	Yes	Yes
Integrated scalar processor/SIMD Processor	Yes	No
Cache coherent	Yes	Yes on some systems

Figure 4.23 Similarities and differences between multicore with multimedia SIMD extensions and recent GPUs.

Summary

Now that the veil has been lifted, we can see that GPUs are really just multithreaded SIMD Processors, although they have more processors, more lanes per processor, and more multithreading hardware than do traditional multicore computers. For example, the Pascal P100 GPU has 56 SIMD Processors with 64 lanes per processor and hardware support for 64 SIMD Threads. Pascal embraces instruction-level parallelism by issuing instructions from two SIMD Threads to two sets of SIMD Lanes. GPUs also have less cache memory—Pascal's L2 cache is 4 MiB—and it can be coherent with a cooperative distant scalar processor or distant GPUs.

The CUDA programming model wraps up all these forms of parallelism around a single abstraction, the CUDA Thread. Thus the CUDA programmer can think of programming thousands of threads, although they are really executing each block of 32 threads on the many lanes of the many SIMD Processors. The CUDA programmer who wants good performance keeps in mind that these threads are organized in blocks and executed 32 at a time and that addresses need to be to adjacent addresses to get good performance from the memory system.

Although we've used CUDA and the NVIDIA GPU in this section, rest assured that the same ideas are found in the OpenCL programming language and in GPUs from other companies.

Now that you understand better how GPUs work, we reveal the real jargon. Figures 4.24 and 4.25 match the descriptive terms and definitions of this section with the official CUDA/NVIDIA and AMD terms and definitions. We also include the OpenCL terms. We believe the GPU learning curve is steep in part because of using terms such as "streaming multiprocessor" for the SIMD Processor, "thread processor" for the SIMD Lane, and "shared memory" for local memory—especially because local memory is *not* shared between SIMD Processors! We hope that this two-step approach gets you up that curve quicker, even if it's a bit indirect.

4.5 Detecting and Enhancing Loop-Level Parallelism

Loops in programs are the fountainhead of many of the types of parallelism we previously discussed here and in Chapter 5. In this section, we discuss compiler technology used for discovering the amount of parallelism that we can exploit in a program as well as hardware support for these compiler techniques. We define precisely when a loop is parallel (or vectorizable), how a dependence can prevent a loop from being parallel, and techniques for eliminating some types of dependences. Finding and manipulating loop-level parallelism is critical to exploiting both DLP and TLP, as well as the more aggressive static ILP approaches (e.g., VLIW) that we examine in Appendix H.

Type	More descriptive name used in this book	Official CUDA/ NVIDIA term	Short explanation and AMD and OpenCL terms	Official CUDA/NVIDIA definition
Program abstractions	Vectorizable loop	Grid	A vectorizable loop, executed on the GPU, made up of one or more "Thread Blocks" (or bodies of vectorized loop) that can execute in parallel. OpenCL name is "index range." AMD name is "NDRange"	A Grid is an array of Thread Blocks that can execute concurrently, sequentially, or a mixture
	Body of Vectorized loop	Thread Block	A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. These SIMD Threads can communicate via local memory. AMD and OpenCL name is "work group"	A Thread Block is an array of CUDA Threads that execute concurrently and can cooperate and communicate via shared memory and barrier synchronization. A Thread Block has a Thread Block ID within its Grid
	Sequence of SIMD Lane operations	CUDA Thread	A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask. AMD and OpenCL call a CUDA Thread a "work item"	A CUDA Thread is a lightweight thread that executes a sequential program and that can cooperate with other CUDA Threads executing in the same Thread Block. A CUDA Thread has a thread ID within its Thread Block
Machine object	A thread of SIMD instructions	Warp	A traditional thread, but it contains just SIMD instructions that are executed on a multithreaded SIMD Processor. Results are stored depending on a per-element mask. AMD name is "wavefront"	A warp is a set of parallel CUDA Threads (e.g., 32) that execute the same instruction together in a multithreaded SIMT/SIMD Processor
	SIMD instruction	PTX instruction	A single SIMD instruction executed across the SIMD Lanes. AMD name is "AMDIL" or "FSAIL" instruction	A PTX instruction specifies an instruction executed by a CUDA Thread

Figure 4.24 Conversion from terms used in this chapter to official NVIDIA/CUDA and AMD jargon. OpenCL names are given in the book's definitions.

Loop-level parallelism is normally investigated at the source level or close to it, while most analysis of ILP is done once instructions have been generated by the compiler. Loop-level analysis involves determining what dependences exist among the operands in a loop across the iterations of that loop. For now, we will consider only data dependences, which arise when an operand is written at some point and read at a later point. Name dependences also exist and may be removed by the renaming techniques discussed in Chapter 3.

The analysis of loop-level parallelism focuses on determining whether data accesses in later iterations are dependent on data values produced in earlier iterations; such dependence is called a *loop-carried dependence*. Most of the examples

Type	More descriptive name used in this book	Official CUDA/ NVIDIA term	Short explanation and AMD and OpenCL terms	Official CUDA/NVIDIA definition
Processing hardware	Multithreaded SIMD processor	Streaming multiprocessor	Multithreaded SIMD Processor that executes thread of SIMD instructions, independent of other SIMD Processors. Both AMD and OpenCL call it a "compute unit." However, the CUDA programmer writes program for one lane rather than for a "vector" of multiple SIMD Lanes	A streaming multiprocessor (SM) is a multithreaded SIMT/SIMD Processor that executes warps of CUDA Threads. A SIMT program specifies the execution of one CUDA Thread, rather than a vector of multiple SIMD Lanes
	Thread Block Scheduler	Giga Thread Engine	Assigns multiple bodies of vectorized loop to multithreaded SIMD Processors. AMD name is "Ultra-Threaded Dispatch Engine"	Distributes and schedules Thread Blocks of a grid to streaming multiprocessors as resources become available
	SIMD Thread scheduler	Warp scheduler	Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution. AMD name is "Work Group Scheduler"	A warp scheduler in a streaming multiprocessor schedules warps for execution when their next instruction is ready to execute
	SIMD Lane	Thread processor	Hardware SIMD Lane that executes the operations in a thread of SIMD instructions on a single element. Results are stored depending on mask. OpenCL calls it a "processing element." AMD name is also "SIMD Lane"	A thread processor is a datapath and register file portion of a streaming multiprocessor that executes operations for one or more lanes of a warp
Memory hardware	GPU Memory	Global memory	DRAM memory accessible by all multithreaded SIMD Processors in a GPU. OpenCL calls it "global memory"	Global memory is accessible by all CUDA Threads in any Thread Block in any grid; implemented as a region of DRAM, and may be cached
	Private memory	Local memory	Portion of DRAM memory private to each SIMD Lane. Both AMD and OpenCL call it "private memory"	Private "thread-local" memory for a CUDA Thread; implemented as a cached region of DRAM
	Local memory	Shared memory	Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors. OpenCL calls it "local memory." AMD calls it "group memory"	Fast SRAM memory shared by the CUDA Threads composing a Thread Block, and private to that Thread Block. Used for communication among CUDA Threads in a Thread Block at barrier synchronization points
	SIMD Lane registers	Registers	Registers in a single SIMD Lane allocated across body of vectorized loop. AMD also calls them "registers"	Private registers for a CUDA Thread; implemented as multithreaded register file for certain lanes of several warps for each thread processor

Figure 4.25 Conversion from terms used in this chapter to official NVIDIA/CUDA and AMD jargon. Note that our descriptive terms "local memory" and "private memory" use the OpenCL terminology. NVIDIA uses SIMT (single-instruction multiple-thread) rather than SIMD to describe a streaming multiprocessor. SIMT is preferred over SIMD because the per-thread branching and control flow are unlike any SIMD machine.

we considered in Chapters 2 and 3 had no loop-carried dependences and thus are loop-level parallel. To see that a loop is parallel, let us first look at the source representation:

```
for (i=999; i>=0; i=i-1)
      x[i] = x[i] + s;
```

In this loop, the two uses of x[i] are dependent, but this dependence is within a single iteration and is not loop-carried. There is a loop-carried dependence between successive uses of i in different iterations, but this dependence involves an induction variable that can be easily recognized and eliminated. We saw examples of how to eliminate dependences involving induction variables during loop unrolling in Section 2.2 of Chapter 2, and we will look at additional examples later in this section.

Because finding loop-level parallelism involves recognizing structures such as loops, array references, and induction variable computations, a compiler can do this analysis more easily at or near the source level, in contrast to the machine-code level. Let's look at a more complex example.

Example Consider a loop like this one:

```
for (i=0; i<100; i=i+1) {
      A[i+1] = A[i] + C[i];   /* S1 */
      B[i+1] = B[i] + A[i+1]; /* S2 */
}
```

Assume that A, B, and C are distinct, nonoverlapping arrays. (In practice, the arrays may sometimes be the same or may overlap. Because the arrays may be passed as parameters to a procedure that includes this loop, determining whether arrays overlap or are identical often requires sophisticated, interprocedural analysis of the program.) What are the data dependences among the statements S1 and S2 in the loop?

Answer There are two different dependences:

1. S1 uses a value computed by S1 in an earlier iteration, because iteration i computes A[i + 1], which is read in iteration i + 1. The same is true of S2 for B[i] and B[i + 1].
2. S2 uses the value A[i + 1] computed by S1 in the same iteration.

These two dependences are distinct and have different effects. To see how they differ, let's assume that only one of these dependences exists at a time. Because the dependence of statement S1 is on an earlier iteration of S1, this dependence is loop-carried. This dependence forces successive iterations of this loop to execute in series.

The second dependence (S2 depending on S1) is within an iteration and is not loop-carried. Thus, if this were the only dependence, multiple iterations of the loop would execute in parallel, as long as each pair of statements in an iteration were kept in order. We saw this type of dependence in an example in Section 2.2, where unrolling could expose the parallelism. These intra-loop dependences are common; for example, a sequence of vector instructions that uses chaining exhibits exactly this sort of dependence.

It is also possible to have a loop-carried dependence that does not prevent parallelism, as the next example shows.

Example Consider a loop like this one:

```
for (i=0; i<100; i=i+1) {
      A[i] = A[i] + B[i];   /* S1 */
      B[i+1] = C[i] + D[i]; /* S2 */
}
```

What are the dependences between S1 and S2? Is this loop parallel? If not, show how to make it parallel.

Answer Statement S1 uses the value assigned in the previous iteration by statement S2, so there is a loop-carried dependence between S2 and S1. Despite this loop-carried dependence, this loop can be made parallel. Unlike the earlier loop, this dependence is not circular; neither statement depends on itself, and although S1 depends on S2, S2 does not depend on S1. A loop is parallel if it can be written without a cycle in the dependences because the absence of a cycle means that the dependences give a partial ordering on the statements.

Although there are no circular dependences in the preceding loop, it must be transformed to conform to the partial ordering and expose the parallelism. Two observations are critical to this transformation:

1. There is no dependence from S1 to S2. If there were, then there would be a cycle in the dependences and the loop would not be parallel. Because this other dependence is absent, interchanging the two statements will not affect the execution of S2.
2. On the first iteration of the loop, statement S2 depends on the value of B[0] computed *prior* to initiating the loop.

These two observations allow us to replace the preceding loop with the following code sequence:

```
A[0] = A[0] + B[0];
for (i=0; i<99; i=i+1) {
```

```
        B[i+1] = C[i] + D[i];
        A[i+1] = A[i+1] + B[i+1];
}
B[100] = C[99] + D[99];
```

The dependence between the two statements is no longer loop-carried so that iterations of the loop may be overlapped, provided the statements in each iteration are kept in order.

Our analysis needs to begin by finding all loop-carried dependences. This dependence information is *inexact*, in the sense that it tells us that such dependence *may* exist. Consider the following example:

```
for (i=0;i<100;i=i+1) {
        A[i] = B[i] + C[i]
        D[i] = A[i] * E[i]
}
```

The second reference to A in this example need not be translated to a load instruction because we know that the value is computed and stored by the previous statement. Thus the second reference to A can simply be a reference to the register into which A was computed. Performing this optimization requires knowing that the two references are *always* to the same memory address and that there is no intervening access to the same location. Normally, data dependence analysis tells that only one reference *may* depend on another; a more complex analysis is required to determine that two references *must be* to the exact same address. In the preceding example, a simple version of this analysis suffices because the two references are in the same basic block.

Often loop-carried dependences are in the form of a *recurrence*. A recurrence occurs when a variable is defined based on the value of that variable in an earlier iteration, usually the one immediately preceding, as in the following code fragment:

```
for (i=1;i<100;i=i+1) {
        Y[i] = Y[i-1] + Y[i];
}
```

Detecting a recurrence can be important for two reasons: some architectures (especially vector computers) have special support for executing recurrences, and in an ILP context, it may still be possible to exploit a fair amount of parallelism.

Finding Dependences

Clearly, finding the dependences in a program is important both to determine which loops might contain parallelism and to eliminate name dependences. The complexity of dependence analysis arises also because of the presence of arrays

and pointers in languages such as C or C++, or pass-by-reference parameter passing in Fortran. Because scalar variable references explicitly refer to a name, they can usually be analyzed quite easily with aliasing because of pointers and reference parameters causing some complications and uncertainty in the analysis.

How does the compiler detect dependences in general? Nearly all dependence analysis algorithms work on the assumption that array indices are *affine*. In simplest terms, a one-dimensional array index is affine if it can be written in the form $a \times i + b$, where a and b are constants and i is the loop index variable. The index of a multidimensional array is affine if the index in each dimension is affine. Sparse array accesses, which typically have the form x[y[i]], are one of the major examples of nonaffine accesses.

Determining whether there is a dependence between two references to the same array in a loop is thus equivalent to determining whether two affine functions can have the identical value for different indices between the bounds of the loop. For example, suppose we have stored to an array element with index value $a \times i + b$ and loaded from the same array with index value $c \times i + d$, where i is the for-loop index variable that runs from m to n. A dependence exists if two conditions hold:

1. There are two iteration indices, j and k, that are both within the limits of the for-loop. That is, $m \leq j \leq n$, $m \leq k \leq n$.
2. The loop stores into an array element indexed by $a \times j + b$ and later fetches from that *same* array element when it is indexed by $c \times k + d$, that is, $a \times j + b = c \times k + d$.

In general, we cannot determine whether dependence exists at compile time. For example, the values of a, b, c, and d may not be known (they could be values in other arrays), making it impossible to tell if a dependence exists. In other cases, the dependence testing may be very expensive but decidable at compile time; for example, the accesses may depend on the iteration indices of multiple nested loops. Many programs, however, contain primarily simple indices where a, b, c, and d are all constants. For these cases, it is possible to devise reasonable compile time tests for dependence.

As an example, a simple and sufficient test for the absence of a dependence is the *greatest common divisor* (GCD) test. It is based on the observation that if a loop-carried dependence exists, then GCD (c, a) must divide ($d - b$). (Recall that an integer, x, *divides* another integer, y, if we get an integer quotient when we do the division y/x and there is no remainder.)

Example Use the GCD test to determine whether dependences exist in the following loop:

```
for (i=0; i<100; i=i+1) {
     X[2*i+3] = X[2*i] * 5.0;
}
```

Answer Given the values $a = 2$, $b = 3$, $c = 2$, and $d = 0$, then GCD(a, c) = 2, and $d - b = -3$. Because 2 does not divide -3, no dependence is possible.

The GCD test is sufficient to guarantee that no dependence exists; however, there are cases where the GCD test succeeds but no dependence exists. This can arise, for example, because the GCD test does not consider the loop bounds.

In general, determining whether a dependence actually exists is NP-complete. In practice, however, many common cases can be analyzed precisely at low cost. Recently, approaches using a hierarchy of exact tests increasing in generality and cost have been shown to be both accurate and efficient. (A test is *exact* if it precisely determines whether a dependence exists. Although the general case is NP-complete, there exist exact tests for restricted situations that are much cheaper.)

In addition to detecting the presence of a dependence, a compiler wants to classify the type of dependence. This classification allows a compiler to recognize name dependences and eliminate them at compile time by renaming and copying.

Example The following loop has multiple types of dependences. Find all the true dependences, output dependences, and antidependences, and eliminate the output dependences and antidependences by renaming.

```
for (i=0; i<100; i=i+1) {
    Y[i] = X[i] / c;  /* S1 */
    X[i] = X[i] + c;  /* S2 */
    Z[i] = Y[i] + c;  /* S3 */
    Y[i] = c - Y[i];  /* S4 */
}
```

Answer The following dependences exist among the four statements:

1. There are true dependences from S1 to S3 and from S1 to S4 because of Y[i]. These are not loop-carried, so they do not prevent the loop from being considered parallel. These dependences will force S3 and S4 to wait for S1 to complete.
2. There is an antidependence from S1 to S2, based on X[i].
3. There is an antidependence from S3 to S4 for Y[i].
4. There is an output dependence from S1 to S4, based on Y[i].

The following version of the loop eliminates these false (or pseudo) dependences.

```
for (i=0; i<100; i=i+1 {
    T[i] = X[i] / c; /* Y renamed to T to remove output
dependence */
    X1[i] = X[i] + c;/* X renamed to X1 to remove
antidependence */
    Z[i] = T[i] + c;/* Y renamed to T to remove
antidependence */
    Y[i] = c - T[i];
}
```

After the loop, the variable X has been renamed X1. In code that follows the loop, the compiler can simply replace the name X by X1. In this case, renaming does not require an actual copy operation, as it can be done by substituting names or by register allocation. In other cases, however, renaming will require copying.

Dependence analysis is a critical technology for exploiting parallelism, as well as for the transformation-like blocking that Chapter 2 covers. For detecting loop-level parallelism, dependence analysis is the basic tool. Effectively compiling programs for vector computers, SIMD computers, or multiprocessors depends critically on this analysis. The major drawback of dependence analysis is that it applies only under a limited set of circumstances, namely, among references within a single loop nest and using affine index functions. Thus there are many situations where array-oriented dependence analysis *cannot* tell us what we want to know; for example, analyzing accesses done with pointers, rather than with array indices can be much harder. (This is one reason why Fortran is still preferred over C and C++ for many scientific applications designed for parallel computers.) Similarly, analyzing references across procedure calls is extremely difficult. Thus, while analysis of code written in sequential languages remains important, we also need approaches such as OpenMP and CUDA that write explicitly parallel loops.

Eliminating Dependent Computations

As previously mentioned, one of the most important forms of dependent computations is a recurrence. A dot product is a perfect example of a recurrence:

```
for (i=9999; i>=0; i=i-1)
      sum = sum + x[i] * y[i];
```

This loop is not parallel because it has a loop-carried dependence on the variable `sum`. We can, however, transform it to a set of loops, one of which is completely parallel and the other partly parallel. The first loop will execute the completely parallel portion of this loop. It looks like this:

```
for (i=9999; i>=0; i=i-1)
      sum[i] = x[i] * y[i];
```

Notice that sum has been expanded from a scalar into a vector quantity (a transformation called *scalar expansion*) and that this transformation makes this new loop completely parallel. When we are done, however, we need to do the reduce step, which sums up the elements of the vector. It looks like this:

```
for (i=9999; i>=0; i=i-1)
      finalsum = finalsum + sum[i];
```

Although this loop is not parallel, it has a very specific structure called a *reduction*. Reductions are common in linear algebra, and as we will see in Chapter 6,

they are also a key part of the primary parallelism primitive MapReduce used in warehouse-scale computers. In general, any function can be used as a reduction operator, and common cases include operators such as max and min.

Reductions are sometimes handled by special hardware in a vector and SIMD architecture that allows the reduce step to be done much faster than it could be done in scalar mode. These work by implementing a technique similar to what can be done in a multiprocessor environment. While the general transformation works with any number of processors, suppose for simplicity we have 10 processors. In the first step of reducing the sum, each processor executes the following (with `p` as the processor number ranging from 0 to 9):

```
for (i=999; i>=0; i=i-1)
      finalsum[p] = finalsum[p] + sum[i+1000*p];
```

This loop, which sums up 1000 elements on each of the 10 processors, is completely parallel. A simple scalar loop can then complete the summation of the last 10 sums. Similar approaches are used in vector processors and SIMD Processors.

It is important to observe that the preceding transformation relies on associativity of addition. Although arithmetic with unlimited range and precision is associative, computer arithmetic is not associative, for either integer arithmetic, because of limited range, or floating-point arithmetic, because of both range and precision. Thus using these restructuring techniques can sometimes lead to erroneous behavior, although such occurrences are rare. For this reason, most compilers require that optimizations that rely on associativity be explicitly enabled.

4.6 Cross-Cutting Issues

Energy and DLP: Slow and Wide Versus Fast and Narrow

A fundamental power advantage of data-level parallel architectures comes from the energy equation in Chapter 1. Assuming ample data-level parallelism, the performance is the same if we halve the clock rate and double the execution resources: twice the number of lanes for a vector computer, wider registers and ALUs for multimedia SIMD, and more SIMD Lanes for GPUs. If we can lower the voltage while dropping the clock rate, we can actually reduce energy as well as the power for the computation while maintaining the same peak performance. Thus GPUs tend to have lower clock rates than system processors, which rely on high clock rates for performance (see Section 4.7).

Compared to out-of-order processors, DLP processors can have simpler control logic to launch a large number of operations per clock cycle; for example, the control is identical for all lanes in vector processors, and there is no logic to decide on multiple instruction issues or speculative execution logic. They also fetch and decode far fewer instructions. Vector architectures can also make it easier to turn off unused portions of the chip. Each vector instruction explicitly describes all the resources it needs for a number of cycles when the instruction issues.

Banked Memory and Graphics Memory

Section 4.2 noted the importance of substantial memory bandwidth for vector architectures to support unit stride, nonunit stride, and gather-scatter accesses.

To achieve the highest memory performance, stacked DRAMs are used in the top-end GPUs from AMD and NVIDIA. Intel also uses stacked DRAM in its Xeon Phi product. Also known as *high bandwidth memory* (*HBM*, *HBM2*), the memory chips are stacked and placed in the same package as the processing chip. The extensive width (typically 1024–4096 data wires) provides high bandwidth, while placing the memory chips in the same package as the processor chip reduces latency and power consumption. The capacity of stacked DRAM is typically 8–32 GB.

Given all the potential demands on the memory from both the computation tasks and the graphics acceleration tasks, the memory system could see a large number of uncorrelated requests. Unfortunately, this diversity hurts memory performance. To cope, the GPU's memory controller maintains separate queues of traffic bound for different banks, waiting until there is enough traffic to justify opening a row and transferring all requested data at once. This delay improves bandwidth but stretches latency, and the controller must ensure that no processing units starve while waiting for data, for otherwise neighboring processors could become idle. Section 4.7 shows that gather-scatter techniques and memory-bank-aware access techniques can deliver substantial increases in performance versus conventional cache-based architectures.

Strided Accesses and TLB Misses

One problem with strided accesses is how they interact with the translation lookaside buffer (TLB) for virtual memory in vector architectures or GPUs. (GPUs also use TLBs for memory mapping.) Depending on how the TLB is organized and the size of the array being accessed in memory, it is even possible to get one TLB miss for every access to an element in the array! The same type of collision can happen with caches, but the performance impact is probably less.

4.7 Putting It All Together: Embedded Versus Server GPUs and Tesla Versus Core i7

Given the popularity of graphics applications, GPUs are now found in both mobile clients and traditional servers and heavy-duty desktop computers. Figure 4.26 lists the key characteristics of the NVIDIA Tegra Parker system on a chip for embedded clients, which is popular in automobiles, and the Pascal GPU for servers. GPU server engineers hope to be able to do live animation within five years after a movie is released. GPU-embedded engineers in turn want to do what a server or game console does today on their hardware within five more years.

The NVIDIA Tegra P1 has six ARMv8 cores and a smaller Pascal GPU (capable of 750 GFLOPS) and 50 GB/s of memory bandwidth. It is the key component

	NVIDIA Tegra 2	NVIDIA Tesla P100
Market	Automotive, Embedded, Console, Tablet	Desktop, server
System processor	Six-Core ARM (2 Denver2 +4 A57)	Not applicable
System interface	Not applicable	PCI Express × 16 Gen 3
System interface bandwidth	Not applicable	16 GB/s (each direction), 32 GB/s (total)
Clock rate	1.5 GHz	1.4 GHz
SIMD multiprocessors	2	56
SIMD Lanes/SIMD multiprocessor	128	64
Memory interface	128-bit LP-DDR4	4096-bit HBM2
Memory bandwidth	50 GB/s	732 GB/s
Memory capacity	up to 16 GB	up to 16 GB
Transistors	7 billion	15.3 billion
Process	TSMC 16 nm FinFET	TSMC 16 nm FinFET
Die area	147 mm^2	645 mm^2
Power	20 W	300 W

Figure 4.26 Key features of the GPUs for embedded clients and servers.

of the NVIDIA DRIVE PX2 computing platform that is used in cars for autonomous driving. The NVIDIA Tegra X1 is the previous generation and is used in several high-end tablets, such as the Google Pixel C and the NVIDIA Shield TV. It has a Maxwell-class GPU capable of 512 GFLOPS.

The NVIDIA Tesla P100 is the Pascal GPU discussed extensively in this chapter. (Tesla is Nvidia's name for products targeting general-purpose computing.) The clock rate is 1.4 GHz, and it includes 56 SIMD Processors. The path to HBM2 memory is 4096-bits wide, and it transfers data on both the rising and falling edge of a 0.715 GHz clock, which means a peak memory bandwidth of 732 GB/s. It connects to the host system processor and memory via a PCI Express × 16 Gen 3 link, which has a peak bidirectional rate of 32 GB/s.

All physical characteristics of the P100 die are impressively large: it contains 15.3 billion transistors, the die size is 645 mm^2 in a 16-nm TSMC process, and the typical power is 300 W.

Comparison of a GPU and a MIMD With Multimedia SIMD

A group of Intel researchers published a paper (Lee et al., 2010) comparing a quad-core Intel i7 with multimedia SIMD extensions to the Tesla GTX 280. Although the study did not compare the latest versions of CPUs and GPUs, it was the most

	Core i7-960	GTX 280	Ratio 280/i7
Number of processing elements (cores or SMs)	4	30	7.5
Clock frequency (GHz)	3.2	1.3	0.41
Die size	263	576	2.2
Technology	Intel 45 nm	TSMC 65 nm	1.6
Power (chip, not module)	130	130	1.0
Transistors	700 M	1400 M	2.0
Memory bandwidth (GB/s)	32	141	4.4
Single-precision SIMD width	4	8	2.0
Double-precision SIMD width	2	1	0.5
Peak single-precision scalar FLOPS (GFLOP/S)	26	117	4.6
Peak single-precision SIMD FLOPS (GFLOP/S)	102	311–933	3.0–9.1
(SP 1 add or multiply)	N.A.	(311)	(3.0)
(SP 1 instruction fused multiply-adds)	N.A.	(622)	(6.1)
(Rare SP dual issue fused multiply-add and multiply)	N.A.	(933)	(9.1)
Peak double-precision SIMD FLOPS (GFLOP/S)	51	78	1.5

Figure 4.27 Intel Core i7-960 and NVIDIA GTX 280. The rightmost column shows the ratios of GTX 280 to Core i7. For single-precision SIMD FLOPS on the GTX 280, the higher speed (933) comes from a very rare case of dual issuing of fused multiply-add and multiply. More reasonable is 622 for single fused multiply-adds. Note that these memory bandwidths are higher than in Figure 4.28 because these are DRAM pin bandwidths and those in Figure 4.28 are at the processors as measured by a benchmark program. From Table 2 in Lee, W.V., et al., 2010. Debunking the 100× GPU vs. CPU myth: an evaluation of throughput computing on CPU and GPU. In: Proc. 37th Annual Int'l. Symposium on Computer Architecture (ISCA), June 19–23, 2010, Saint-Malo, France.

in-depth comparison of the two styles in that it explained the reasons behind the differences in performance. Moreover, the current versions of these architectures share many similarities to the ones in the study.

Figure 4.27 lists the characteristics of the two systems. Both products were purchased in the fall of 2009. The Core i7 is in Intel's 45-nanometer semiconductor technology, while the GPU is in TSMC's 65-nanometer technology. Although it might have been fairer to have a comparison done by a neutral party or by both interested parties, the purpose of this section is *not* to determine how much faster one product is than the other, but to try to understand the relative value of features of these two contrasting architecture styles.

The rooflines of the Core i7 920 and GTX 280 in Figure 4.28 illustrate the differences in the computers. The 920 has a slower clock rate than the 960 (2.66 GHz vs. 3.2 GHz), but the rest of the system is the same. Not only does the GTX 280 have much higher memory bandwidth and double-precision floating-point performance, but also its double-precision ridge point is considerably to the left. As previously mentioned, it is much easier to hit peak computational performance the further the ridge point of the roofline is to the left. The double-precision ridge point

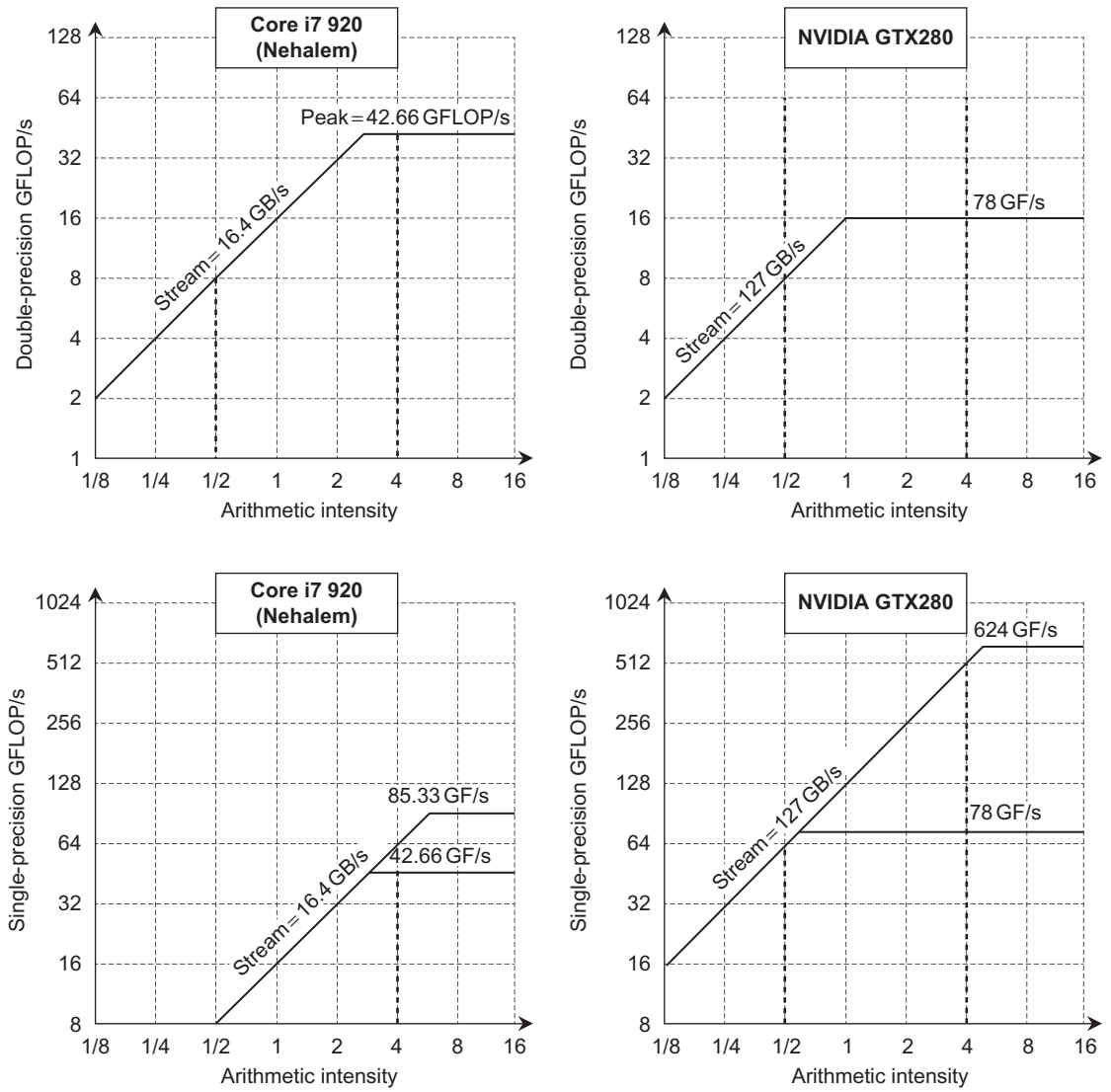


Figure 4.28 Roofline model (Williams et al. 2009). These rooflines show double-precision floating-point performance in the top row and single-precision performance in the bottom row. (The DP FP performance ceiling is also in the bottom row to give perspective.) The Core i7 920 on the left has a peak DP FP performance of 42.66 GFLOP/s, a SP FP peak of 85.33 GFLOP/s, and a peak memory bandwidth of 16.4 GB/s. The NVIDIA GTX 280 has a DP FP peak of 78 GFLOP/s, SP FP peak of 624 GFLOP/s, and 127 GB/s of memory bandwidth. The dashed vertical line on the left represents an arithmetic intensity of 0.5 FLOP/byte. It is limited by memory bandwidth to no more than 8 DP GFLOP/s or 8 SP GFLOP/s on the Core i7. The dashed vertical line to the right has an arithmetic intensity of 4 FLOP/byte. It is limited only computationally to 42.66 DP GFLOP/s and 64 SP GFLOP/s on the Core i7 and to 78 DP GFLOP/s and 512 DP GFLOP/s on the GTX 280. To hit the highest computation rate on the Core i7, you need to use all 4 cores and SSE instructions with an equal number of multiplies and adds. For the GTX 280, you need to use fused multiply-add instructions on all multithreaded SIMD Processors.

is 0.6 for the GTX 280 versus 2.6 for the Core i7. For single-precision performance, the ridge point moves far to the right, as it's considerably harder to hit the roof of single-precision performance because it is so much higher. Note that the arithmetic intensity of the kernel is based on the bytes that go to main memory, not the bytes that go to cache memory. Thus caching can change the arithmetic intensity of a kernel on a particular computer, presuming that most references really go to the cache. The Rooflines help explain the relative performance in this case study. Note also that this bandwidth is for unit-stride accesses in both architectures. Real gather-scatter addresses that are not coalesced are slower on the GTX 280 and on the Core i7, as we will see.

The researchers said that they selected the benchmark programs by analyzing the computational and memory characteristics of four recently proposed benchmark suites and then "formulated the set of *throughput computing kernels* that capture these characteristics." Figure 4.29 describes these 14 kernels, and Figure 4.30 shows the performance results, with larger numbers meaning faster.

Given that the raw performance specifications of the GTX 280 vary from 2.5 × slower (clock rate) to 7.5 × faster (cores per chip) while the performance varies from 2.0 × slower (Solv) to 15.2 × faster (GJK), the Intel researchers explored the reasons for the differences:

- *Memory bandwidth*. The GPU has 4.4 × the memory bandwidth, which helps explain why LBM and SAXPY run 5.0 and 5.3 × faster; their working sets are hundreds of megabytes and thus don't fit into the Core i7 cache. (To access memory intensively, they did not use cache blocking on SAXPY.) Thus the slope of the rooflines explains their performance. SpMV also has a large working set, but it only runs 1.9 × because the double-precision floating point of the GTX 280 is just 1.5 × faster than the Core i7.
- *Compute bandwidth*. Five of the remaining kernels are compute bound: SGEMM, Conv, FFT, MC, and Bilat. The GTX is faster by 3.9, 2.8, 3.0, 1.8, and 5.7, respectively. The first three of these use single-precision floating-point arithmetic, and GTX 280 single-precision is 3–6 × faster. (The 9 × faster than the Core i7 as shown in Figure 4.27 occurs only in the very special case when the GTX 280 can issue a fused multiply-add and a multiply per clock cycle.) MC uses double-precision, which explains why it's just 1.8 × faster since DP performance is only 1.5 × faster. Bilat uses transcendental functions, which the GTX 280 supports directly (see Figure 4.17). The Core i7 spends two-thirds of its time calculating transcendental functions, so the GTX 280 is 5.7 × faster. This observation helps point out the value of hardware support for operations that occur in your workload: double-precision floating-point and perhaps even transcendentals.
- *Cache benefits*. Ray casting (RC) is only 1.6 × faster on the GTX because cache blocking with the Core i7 caches prevents it from becoming memory bandwidth bound, as it is on GPUs. Cache blocking can help Search, too. If the index trees are small so that they fit into the cache, the Core i7 is twice

Kernel	Application	SIMD	TLP	Characteristics
SGEMM (**SGEMM**)	Linear algebra	Regular	Across 2D tiles	Compute bound after tiling
Monte Carlo (**MC**)	Computational finance	Regular	Across paths	Compute bound
Convolution (**Conv**)	Image analysis	Regular	Across pixels	Compute bound; BW bound for small filters
FFT (**FFT**)	Signal processing	Regular	Across smaller FFTs	Compute bound or BW bound depending on size
SAXPY (**SAXPY**)	Dot product	Regular	Across vector	BW bound for large vectors
LBM (**LBM**)	Time migration	Regular	Across cells	BW bound
Constraint solver (**Solv**)	Rigid body physics	Gather/ Scatter	Across constraints	Synchronization bound
SpMV (**SpMV**)	Sparse solver	Gather	Across nonzero	BW bound for typical large matrices
GJK (**GJK**)	Collision detection	Gather/ Scatter	Across objects	Compute bound
Sort (**Sort**)	Database	Gather/ Scatter	Across elements	Compute bound
Ray casting (**RC**)	Volume rendering	Gather	Across rays	4–8 MB first level working set; over 500 MB last level working set
Search (**Search**)	Database	Gather/ Scatter	Across queries	Compute bound for small tree, BW bound at bottom of tree for large tree
Histogram (**Hist**)	Image analysis	Requires conflict detection	Across pixels	Reduction/synchronization bound
Bilateral (**Bilat**)	Image analysis	Regular	Across pixels	Compute bound

Figure 4.29 Throughput computing kernel characteristics. The name in parentheses identifies the benchmark name in this section. The authors suggest that code for both machines had equal optimization effort. From Table 1 in Lee, W.V., et al., 2010. Debunking the 100× GPU vs. CPU myth: an evaluation of throughput computing on CPU and GPU. In: Proc. 37th Annual Int'l. Symposium on Computer Architecture (ISCA), June 19–23, 2010, Saint-Malo, France.

as fast. Larger index trees make them memory bandwidth bound. Overall, the GTX 280 runs Search 1.8× faster. Cache blocking also helps Sort. While most programmers wouldn't run Sort on a SIMD Processor, it can be written with a 1-bit Sort primitive called *split*. However, the split algorithm executes many more instructions than a scalar sort does. As a result, the GTX 280 runs only 0.8× as fast as the Core i7. Note that caches also help other kernels on the Core i7 because cache blocking allows SGEMM, FFT, and SpMV to become

Kernel	Units	Core i7-960	GTX 280	GTX 280/ i7-960
SGEMM	GFLOP/s	94	364	3.9
MC	Billion paths/s	0.8	1.4	1.8
Conv	Million pixels/s	1250	3500	2.8
FFT	GFLOP/s	71.4	213	3.0
SAXPY	GB/s	16.8	88.8	5.3
LBM	Million lookups/s	85	426	5.0
Solv	Frames/s	103	52	0.5
SpMV	GFLOP/s	4.9	9.1	1.9
GJK	Frames/s	67	1020	15.2
Sort	Million elements/s	250	198	0.8
RC	Frames/s	5	8.1	1.6
Search	Million queries/s	50	90	1.8
Hist	Million pixels/s	1517	2583	1.7
Bilat	Million pixels/s	83	475	5.7

Figure 4.30 Raw and relative performance measured for the two platforms. In this study, SAXPY is used only as a measure of memory bandwidth, so the right unit is GB/s and not GFLOP/s. Based on Table 3 in Lee, W.V., et al., 2010. Debunking the 100× GPU vs. CPU myth: an evaluation of throughput computing on CPU and GPU. In: Proc. 37th Annual Int'l. Symposium on Computer Architecture (ISCA), June 19–23, 2010, Saint-Malo, France.

compute bound. This observation reemphasizes the importance of cache blocking optimizations in Chapter 2.

- *Gather-Scatter*. The multimedia SIMD extensions are of little help if the data are scattered throughout main memory; optimal performance comes only when data are aligned on 16-byte boundaries. Thus GJK gets little benefit from SIMD on the Core i7. As previously mentioned, GPUs offer gather-scatter addressing that is found in a vector architecture but omitted from SIMD extensions. The Address Coalescing Unit helps as well by combining accesses to the same DRAM line, thereby reducing the number of gathers and scatters. The memory controller also batches together accesses to the identical DRAM page. This combination means the GTX 280 runs GJK a startling 15.2× faster than the Core i7, which is larger than any single physical parameter in Figure 4.27. This observation reinforces the importance of gather-scatter to vector and GPU architectures that is missing from SIMD extensions.
- *Synchronization*. The performance synchronization of Hist is limited by atomic updates, which are responsible for 28% of the total runtime on the Core i7 despite its having a hardware fetch-and-increment instruction. Thus Hist is

only 1.7× faster on the GTX 280. Solv solves a batch of independent constraints in a small amount of computation followed by barrier synchronization. The Core i7 benefits from the atomic instructions and a memory consistency model that ensures the right results even if not all previous accesses to memory hierarchy have completed. Without the memory consistency model, the GTX 280 version launches some batches from the system processor, which leads to the GTX 280 running 0.5× as fast as the Core i7. This observation points out how synchronization performance can be important for some data parallel problems.

It was interesting that the gather-scatter support of vector architectures, which predate the SIMD instructions by decades, was so important to the effective usefulness of these SIMD extensions, which some had predicted before the comparison (Gebis and Patterson, 2007). The Intel researchers noted that 6 of the 14 kernels would exploit SIMD better with more efficient gather-scatter support on the Core i7.

Note that an important feature missing from this comparison was describing the level of effort to get the results for the two systems. Ideally, future comparisons would release the code used on both systems so that others could re-create the same experiments on different hardware platforms and possibly improve on the results.

Comparison Update

In the intervening years, the weaknesses of the Core i7 and Tesla GTX 280 have been addressed by their successors. Intel's ACV2 added gather instructions, and AVX/512 added scatter instructions, both of which are found in the Intel Skylake series. Nvidia Pascal has double-precision floating-point performance that is one-half instead of one-eighth the speed of single precision, fast atomic operations, and caches.

Figure 4.31 lists the characteristics of these two successors, Figure 4.32 compares performance using 3 of the 14 benchmarks in the original paper (those were the ones for which we could find source code), and Figure 4.33 shows the two new roofline models. The new GPU chip is 15 to 50 times faster and the new CPU chips is 50 times faster than their predecessors, and the new GPU is 2–5 times faster than the new CPU.

4.8 Fallacies and Pitfalls

While data-level parallelism is the easiest form of parallelism after ILP from the programmer's perspective, and plausibly the simplest from the architect's perspective, it still has many fallacies and pitfalls.

Fallacy *GPUs suffer from being coprocessors.*

Although the split between main memory and GPU memory has disadvantages, there are advantages to being at a distance from the CPU.

	Xeon Platinum 8180	P100	Ratio P100/Xeon
Number of processing elements (cores or SMs)	28	56	2.0
Clock frequency (GHz)	2.5	1.3	0.52
Die size	N.A.	610 mm^2	–
Technology	Intel 14 nm	TSMC 16 nm	1.1
Power (chip, not module)	80 W	300 W	3.8
Transistors	N.A.	15.3 B	–
Memory bandwidth (GB/s)	199	732	3.7
Single-precision SIMD width	16	8	0.5
Double-precision SIMD width	8	4	0.5
Peak single-precision SIMD FLOPS (GFLOP/s)	4480	10,608	2.4
Peak double-precision SIMD FLOPS (GFLOP/s)	2240	5304	2.4

Figure 4.31 Intel Xeon ?? and NVIDIA P100. The rightmost column shows the ratios of P100 to the Xeon. Note that these memory bandwidths are higher than in Figure 4.28 because these are DRAM pin bandwidths and those in Figure 4.28 are at the processors as measured by a benchmark program.

Kernel	Units	Xeon Platinum 8180	P100	P100/Xeon	GTX 280/i7-960
SGEMM	GFLOP/s	3494	6827	2.0	3.9
DGEMM	GFLOP/s	1693	3490	2.1	—
FFT-S	GFLOP/s	410	1820	4.4	3.0
FFT-D	GFLOP/s	190	811	4.2	—
SAXPY	GB/s	207	544	2.6	5.3
DAXPY	GB/s	212	556	2.6	—

Figure 4.32 Raw and relative performance measured for modern versions of the two platforms as compared to the relative performance of the original platforms. Like Figure 4.30, SAXPY and DAXPY are used only as a measure of memory bandwidth, so the proper unit is GB/s and not GFLOP/s.

For example, PTX exists in part because of the I/O device nature of GPUs. This level of indirection between the compiler and the hardware gives GPU architects much more flexibility than system processor architects. It's often hard to know in advance whether an architecture innovation will be well supported by compilers and libraries and be important to applications. Sometimes a new mechanism will even prove useful for one or two generations and then fade in importance as the IT world changes. PTX allows GPU architects to try innovations speculatively and drop them in subsequent generations if they disappoint or fade in importance, which encourages experimentation. The justification for inclusion is understandably considerably higher for system processors—and thus much less

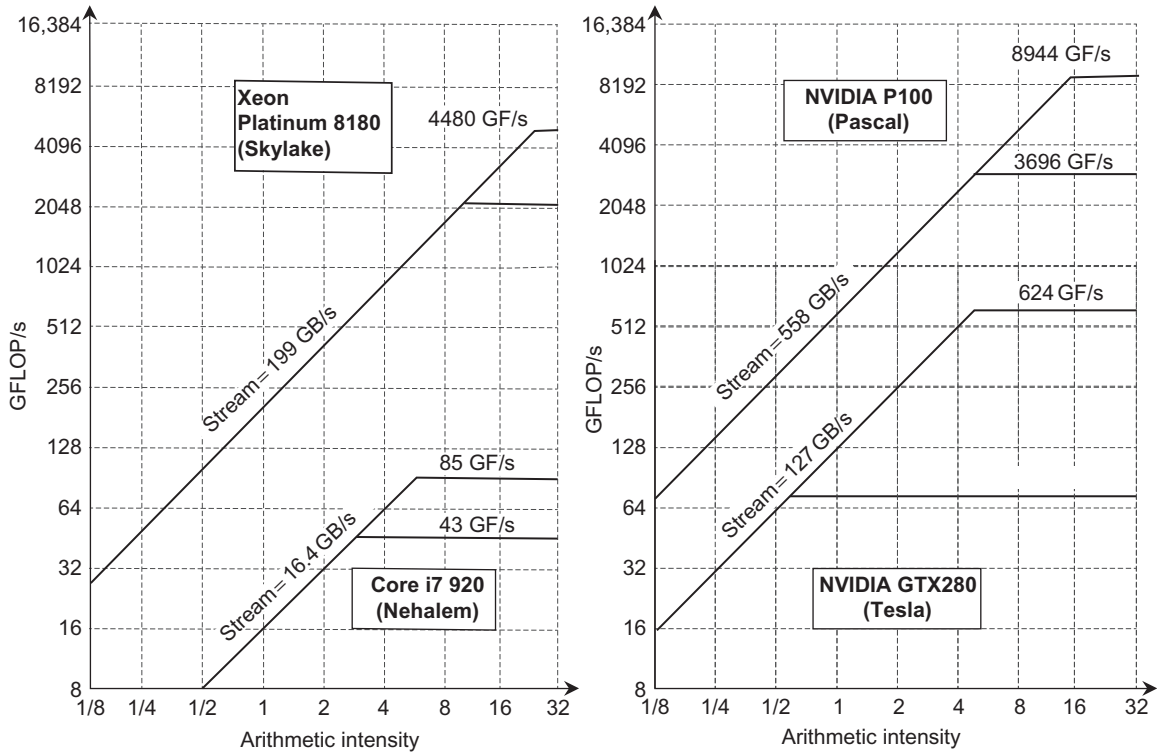


Figure 4.33 Roofline models of older and newer CPUs versus older and newer GPUs. The higher roofline for each computer is single-precision floating-point performance, and the lower one is double-precision performance.

experimentation can occur—as distributing binary machine code normally implies that new features must be supported by all future generations of that architecture.

A demonstration of the value of PTX is that the different generation architecture radically changed the hardware instruction set—from being memory-oriented like x86 to being register-oriented like RISC-V *as well as* doubling the address size to 64 bits—without disrupting the NVIDIA software stack.

Pitfall *Concentrating on peak performance in vector architectures and ignoring start-up overhead.*

Early memory-memory vector processors such as the TI ASC and the CDC STAR-100 had long start-up times. For some vector problems, vectors had to be longer than 100 for the vector code to be faster than the scalar code! On the CYBER 205—derived from the STAR-100—the start-up overhead for DAXPY is 158 clock cycles, which substantially increases the break-even point. If the clock rates of the Cray-1 and the CYBER 205 were identical, the Cray-1 would be faster until the vector length was greater than 64. Because the Cray-1 clock rate was also higher (even though the 205 was newer), the crossover point was a vector length over 100.

Pitfall *Increasing vector performance, without comparable increases in scalar performance.*

This imbalance was a problem on many early vector processors, and a place where Seymour Cray (the architect of the Cray computers) rewrote the rules. Many of the early vector processors had comparatively slow scalar units (as well as large start-up overheads). Even today, a processor with lower vector performance but better scalar performance can outperform a processor with higher peak vector performance. Good scalar performance keeps down overhead costs (strip mining, for example) and reduces the impact of Amdahl's law.

An excellent example of this comes from comparing a fast scalar processor and a vector processor with lower scalar performance. The Livermore Fortran kernels are a collection of 24 scientific kernels with varying degrees of vectorization. Figure 4.34 shows the performance of two different processors on this benchmark. Despite the vector processor's higher peak performance, its low scalar performance makes it slower than a fast scalar processor as measured by the harmonic mean.

The flip of this danger today is increasing vector performance—say, by increasing the number of lanes—without increasing scalar performance. Such myopia is another path to an unbalanced computer.

The next fallacy is closely related.

Fallacy *You can get good vector performance without providing memory bandwidth.*

As we saw with the DAXPY loop and the Roofline model, memory bandwidth is quite important to all SIMD architectures. DAXPY requires 1.5 memory references per floating-point operation, and this ratio is typical of many scientific codes. Even if the floating-point operations took no time, a Cray-1 could not increase the performance of the vector sequence used, because it is memory-limited. The Cray-1 performance on Linpack jumped when the compiler used blocking to change the computation so that values could be kept in the vector registers. This approach lowered the number of memory references per FLOP and improved the performance by nearly a factor of two! Thus the memory bandwidth on the Cray-1 became sufficient for a loop that formerly required more bandwidth.

Processor	Minimum rate for any loop (MFLOPS)	Maximum rate for any loop (MFLOPS)	Harmonic mean of all 24 loops (MFLOPS)
MIPS M/120-5	0.80	3.89	1.85
Stardent-1500	0.41	10.08	1.72

Figure 4.34 Performance measurements for the Livermore Fortran kernels on two different processors. Both the MIPS M/120-5 and the Stardent-1500 (formerly the Ardent Titan-1) use a 16.7 MHz MIPS R2000 chip for the main CPU. The Stardent-1500 uses its vector unit for scalar FP and has about half the scalar performance (as measured by the minimum rate) of the MIPS M/120-5, which uses the MIPS R2010 FP chip. The vector processor is more than a factor of 2.5 × faster for a highly vectorizable loop (maximum rate). However, the lower scalar performance of the Stardent-1500 negates the higher vector performance when total performance is measured by the harmonic mean on all 24 loops.

Fallacy *On GPUs, just add more threads if you don't have enough memory performance.*

GPUs use many CUDA Threads to hide the latency to main memory. If memory accesses are scattered or not correlated among CUDA Threads, the memory system will get progressively slower in responding to each individual request. Eventually, even many threads will not cover the latency. For the "more CUDA Threads" strategy to work, not only do you need lots of CUDA Threads, but the CUDA Threads themselves also must be well behaved in terms of locality of memory accesses.

4.9 Concluding Remarks

Data-level parallelism is increasing in importance for personal mobile devices, given the popularity of applications showing the importance of audio, video, and games on these devices. When combined with a model that is easier to program than task-level parallelism and with potentially better energy efficiency, it's easy to see why there has been a renaissance for data-level parallelism in this decade.

We are seeing system processors take on more of the characteristics of GPUs, and vice versa. One of the biggest differences in performance between conventional processors and GPUs has been for gather-scatter addressing. Traditional vector architectures show how to add such addressing to SIMD instructions, and we expect to see more ideas added from the well-proven vector architectures to SIMD extensions over time.

As we said in the opening of Section 4.4, the GPU question is not simply which architecture is best, but given the hardware investment to do graphics well, how can it be enhanced to support computation that is more general? Although vector architectures have many advantages on paper, it remains to be proven whether vector architectures can be as good a foundation for graphics as GPUs. RISC-V has embraced vector over SIMD. Thus, like architecture debates of the past, the marketplace will help determine the importance of the strengths and weaknesses of two styles of data parallel architectures.

4.10 Historical Perspective and References

Section M.6 (available online) features a discussion on the Illiac IV (a representative of the early SIMD architectures) and the Cray-1 (a representative of vector architectures). We also look at multimedia SIMD extensions and the history of GPUs.

Case Study and Exercises by Jason D. Bakos

Case Study: Implementing a Vector Kernel on a Vector Processor and GPU

Concepts illustrated by this case study

- Programming Vector Processors
- Programming GPUs
- Performance Estimation

MrBayes is a popular computational biology application for inferring the evolutionary histories among a set of input species based on their prealigned DNA sequence data of length n. MrBayes works by performing an heuristic search over the space of all binary tree topologies for which the inputs are the leaves. In order to evaluate a particular tree, the application must compute an $n \times 4$ conditional likelihood table (named clP) for each interior node. The table is a function of the conditional likelihood tables of the node's two descendent nodes (`clL` and `clR`, single precision floating point) and the 4×4 transition probability table (`tiPL` and `tiPR`, single precision floating point). One of this application's kernels is the computation of this conditional likelihood table and is shown as follows:

```
for (k=0; k<seq_length; k++) {
   clP[h++] = (tiPL[AA]*clL[A] + tiPL[AC]*clL[C] +
               tiPL[AG]*clL[G] + tiPL[AT]*clL[T])*
              (tiPR[AA]*clR[A] + tiPR[AC]*clR[C] +
               tiPR[AG]*clR[G] + tiPR[AT]*clR[T]);
   clP[h++] = (tiPL[CA]*clL[A] + tiPL[CC]*clL[C] +
               tiPL[CG]*clL[G] + tiPL[CT]*clL[T])*
              (tiPR[CA]*clR[A] + tiPR[CC]*clR[C] +
               tiPR[CG]*clR[G] + tiPR[CT]*clR[T]);
   clP[h++] = (tiPL[GA]*clL[A] + tiPL[GC]*clL[C] +
               tiPL[GG]*clL[G] + tiPL[GT]*clL[T])*
              (tiPR[GA]*clR[A] + tiPR[GC]*clR[C] +
               tiPR[GG]*clR[G] + tiPR[GT]*clR[T]);
   clP[h++] = (tiPL[TA]*clL[A] + tiPL[TC]*clL[C] +
               tiPL[TG]*clL[G] + tiPL[TT]*clL[T])*
              (tiPR[TA]*clR[A] + tiPR[TC]*clR[C] +
               tiPR[TG]*clR[G] + tiPR[TT]*clR[T]);
   clL += 4;
   clR += 4;
}
```

4.1 [25] <4.1, 4.2> Assume the constants shown as follows.

Constants	Values
AA,AC,AG,AT	0,1,2,3
CA,CC,CG,CT	4,5,6,7
GA,GC,GG,GT	8,9,10,11
TA,TC,TG,TT	12,13,14,15
A,C,G,T	0,1,2,3

Write code for RISC-V and RV64V. Assume the starting addresses of `tiPL`, `tiPR`, `clL`, `clR`, and `clP` are in `RtiPL`, `RtiPR`, `RclL`, `RclR`, and `RclP`, respectively. Do not unroll the loop. To facilitate vector addition reductions, assume that we add the following instructions to RV64V:
Vector Summation Reduction Single Precision:

```
vsum Fd, Vs
```

This instruction performs a summation reduction on a vector register `Vs`, writing to the sum into scalar register `Fd`.

4.2 [5] <4.1, 4.2> Assuming `seq_length == 500`, what is the dynamic instruction count for both implementations?

4.3 [25] <4.1, 4.2> Assume that the vector reduction instruction is executed on the vector functional unit, similar to a vector add instruction. Show how the code sequence lays out in convoys assuming a single instance of each vector functional unit. How many chimes will the code require? How many cycles per FLOP are needed, ignoring vector instruction issue overhead?

4.4 [15] <4.1, 4.2> Consider the possibility of unrolling the loop and mapping multiple iterations to vector operations. Assume that you can use scatter-gather loads and stores (`vldi` and `vsti`). How does this affect the way you can write the RV64Vcode for this kernel?

4.5 [25] <4.4> Now assume we want to implement the MrBayes kernel on a GPU using a single thread block. Rewrite the C code of the kernel using CUDA. Assume that pointers to the conditional likelihood and transition probability tables are specified as parameters to the kernel. Invoke one thread for each iteration of the loop. Load any reused values into shared memory before performing operations on it.

4.6 [15] <4.4> With CUDA we can use coarse-grain parallelism at the block level to compute the conditional likelihood of multiple nodes in parallel. Assume that we want to compute the conditional likelihood from the bottom of the tree up. Assume seq_length == 500 for all notes and that the group of tables for each of the 12 leaf nodes is stored in consecutive memory locations in the order of node number (e.g., the mth element of clP on node n is at clP [n*4*seq_length + m*4]). Assume that we want to compute the conditional likelihood for nodes 12–17, as shown in Figure 4.35. Change the method by which you compute the array indices in your answer from Exercise 4.5 to include the block number.

4.7 [15] <4.4> Convert your code from Exercise 4.6 into PTX code. How many instructions are needed for the kernel?

4.8 [10] <4.4> How well do you expect this code to perform on a GPU? Explain your answer.

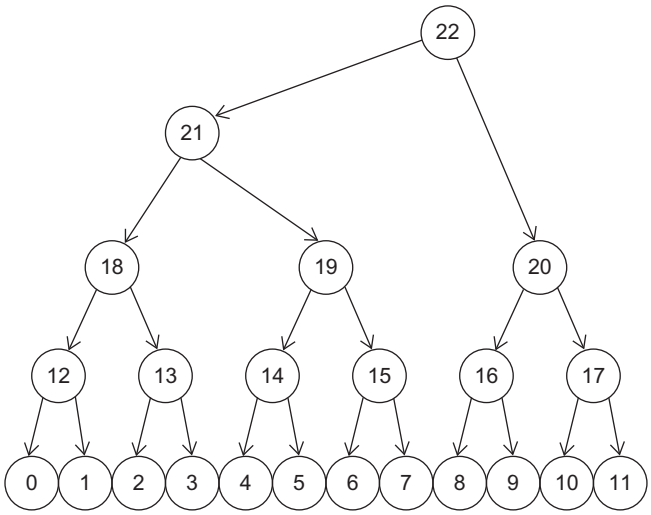


Figure 4.35 Sample tree.

Exercises

4.9 [10/20/20/15/15] <4.2> Consider the following code, which multiplies two vectors that contain single-precision complex values:

```
for (i=0;i<300;i++) {
     c_re[i] = a_re[i] * b_re[i] – a_im[i] * b_im[i];
     c_im[i] = a_re[i] * b_im[i] + a_im[i] * b_re[i];
}
```

Assume that the processor runs at 700 MHz and has a maximum vector length of 64. The load/store unit has a start-up overhead of 15 cycles; the multiply unit, 8 cycles; and the add/subtract unit, 5 cycles.

a. [10] <4.3> What is the arithmetic intensity of this kernel? Justify your answer.

b. [20] <4.2> Convert this loop into RV64V assembly code using strip mining.

c. [20] <4.2> Assuming chaining and a single memory pipeline, how many chimes are required? How many clock cycles are required per complex result value, including start-up overhead?

d. [15] <4.2> If the vector sequence is chained, how many clock cycles are required per complex result value, including overhead?

e. [15] <4.2> Now assume that the processor has three memory pipelines and chaining. If there are no bank conflicts in the loop's accesses, how many clock cycles are required per result?

4.10 [30] <4.2,4.3,4.4> In this problem, we will compare the performance of a vector processor with a hybrid system that contains a scalar processor and a GPU-based coprocessor. In the hybrid system, the host processor has superior scalar performance to the GPU, so in this case all scalar code is executed on the host processor while all vector code is executed on the GPU. We will refer to the first system as the vector computer and the second system as the hybrid computer. Assume that your target application contains a vector kernel with an arithmetic intensity of 0.5 FLOPs per DRAM byte accessed; however, the application also has a scalar component that must be performed before and after the kernel in order to prepare the input vectors and output vectors, respectively. For a sample dataset, the scalar portion of the code requires 400 ms of execution time on both the vector processor and the host processor in the hybrid system. The kernel reads input vectors consisting of 200 MB of data and has output data consisting of 100 MB of data. The vector processor has a peak memory bandwidth of 30 GB/s and the GPU has a peak memory bandwidth of 150 GB/s. The hybrid system has an additional overhead that requires all input vectors to be transferred between the host memory and GPU local memory before and after the kernel is invoked. The hybrid system has a direct memory access (DMA) bandwidth of 10 GB/s and an average latency of 10 ms. Assume that both the vector processor and GPU are performance bound by memory bandwidth. Compute the execution time required by both computers for this application.

4.11 [15/25/25] <4.4, 4.5> Section 4.5 discussed the reduction operation that reduces a vector down to a scalar by repeated application of an operation. A reduction is a special type of a loop recurrence. An example is shown as follows:

```
dot=0.0;
for (i=0;i<64;i++) dot = dot + a[i] * b[i];
```

A vectorizing compiler might apply a transformation called *scalar expansion*, which expands dot into a vector and splits the loop such that the multiply can be performed with a vector operation, leaving the reduction as a separate scalar operation:

```
for (i=0;i<64;i++) dot[i] = a[i] * b[i];
for (i=1;i<64;i++) dot[0] = dot[0] + dot[i];
```

As mentioned in Section 4.5, if we allow the floating-point addition to be associative, there are several techniques available for parallelizing the reduction.

a. [15] <4.4, 4.5> One technique is called recurrence doubling, which adds sequences of progressively shorter vectors (ie, two 32-element vectors, then two 16-element vectors, and so on). Show how the C code would look for executing the second loop in this way.

b. [25] <4.4, 4.5> In some vector processors, the individual elements within the vector registers are addressable. In this case, the operands to a vector operation may be two different parts of the same vector register. This allows another solution for the reduction called *partial sums*. The idea is to reduce the vector to *m* sums where *m* is

the total latency through the vector functional unit, including the operand read and write times. Assume that the VMIPS vector registers are addressable (e.g., you can initiate a vector operation with the operand `V1(16)`, indicating that the input operand begins with element 16). Also, assume that the total latency for adds, including the operand read and result write, is eight cycles. Write a VMIPS code sequence that reduces the contents of `V1` to eight partial sums.

c. [25] <4.4, 4.5> When performing a reduction on a GPU, one thread is associated with each element in the input vector. The first step is for each thread to write its corresponding value into shared memory. Next, each thread enters a loop that adds each pair of input values. This reduces the number of elements by half after each iteration, meaning that the number of active threads also reduces by half after each iteration. In order to maximize the performance of the reduction, the number of fully populated warps should be maximized throughout the course of the loop. In other words, the active threads should be contiguous. Also, each thread should index the shared array in such a way as to avoid bank conflicts in the shared memory. The following loop violates only the first of these guidelines and also uses the modulo operator, which is very expensive for GPUs:

```
unsigned int tid = threadIdx.x;
 for(unsigned int s=1; s <blockDim.x; s * = 2) {
 if ((tid % (2*s)) == 0) {
 sdata[tid] += sdata[tid + s];
 }
 __syncthreads();
 }
```

Rewrite the loop to meet these guidelines and eliminate the use of the modulo operator. Assume that there are 32 threads per warp and a bank conflict occurs whenever two or more threads from the same warp reference an index whose modulo by 32 are equal.

4.12 [10/10/10/10] <4.3> The following kernel performs a portion of the finite-difference time-domain (FDTD) method for computing Maxwell's equations in a three-dimensional space, part of one of the SPEC06fp benchmarks:

```
for (int x=0; x <NX −1; x++) {
 for (int y=0; y <NY −1; y++) {
 for (int z=0; z <NZ −1; z++) {
 int index = x*NY*NZ + y*NZ + z;
 if (y >0 && x >0) {
 material = IDx[index];
 dH1 = (Hz[index] −Hz[index-incrementY])/dy[y];
 dH2 = (Hy[index] −Hy[index-incrementZ])/dz[z];
 Ex[index] = Ca[material]*Ex[index]+Cb[material]*
 (dH2 −dH1);
}}}}
```

Assume that `dH1`, `dH2`, `Hy`, `Hz`, `dy`, `dz`, `Ca`, `Cb`, and `Ex` are all single-precision floating-point arrays. Assume `IDx` is an array of unsigned int.

a. [10] <4.3> What is the arithmetic intensity of this kernel?

b. [10] <4.3> Is this kernel amenable to vector or SIMD execution? Why or why not?

c. [10] <4.3> Assume this kernel is to be executed on a processor that has 30 GB/s of memory bandwidth. Will this kernel be memory bound or compute bound?

d. [10] <4.3> Develop a roofline model for this processor, assuming it has a peak computational throughput of 85 GFLOP/s.

4.13 [10/15] <4.4> Assume a GPU architecture that contains 10 SIMD processors. Each SIMD instruction has a width of 32 and each SIMD processor contains 8 lanes for single-precision arithmetic and load/store instructions, meaning that each nondiverged SIMD instruction can produce 32 results every 4 cycles. Assume a kernel that has divergent branches that causes, on average, 80% of threads to be active. Assume that 70% of all SIMD instructions executed are single-precision arithmetic and 20% are load/store. Because not all memory latencies are covered, assume an average SIMD instruction issue rate of 0.85. Assume that the GPU has a clock speed of 1.5 GHz.

a. [10] <4.4> Compute the throughput, in GFLOP/s, for this kernel on this GPU.

b. [15] <4.4> Assume that you have the following choices:
   (1) Increasing the number of single-precision lanes to 16
   (2) Increasing the number of SIMD processors to 15 (assume this change doesn't affect any other performance metrics and that the code scales to the additional processors)
   (3) Adding a cache that will effectively reduce memory latency by 40%, which will increase instruction issue rate to 0.95

What is speedup in throughput for each of these improvements?

4.14 [10/15/15] <4.5> In this exercise, we will examine several loops and analyze their potential for parallelization.

a. [10] <4.5> Does the following loop have a loop-carried dependency?

```
for (i=0;i <100;i++) {
 A[i] = B[2*i+4];
 B[4*i+5] = A[i];
}
```

b. [15] <4.5> In the following loop, find all the true dependences, output dependences, and antidependences. Eliminate the output dependences and antidependences by renaming.

```
for (i=0;i <100;i++) {
A[i] = A[i] * B[i]; /* S1 */
```

```
B[i] = A[i] + c; /* S2 */
A[i] = C[i] * c; /* S3 */
C[i] = D[i] * A[i]; /* S4 */
```

c. [15] <4.5> Consider the following loop:

```
for (i=0;i <100;i++) {
 A[i] = A[i] + B[i]; /* S1 */
 B[i+1] = C[i] + D[i]; /* S2 */
}
```

Are there dependences between `S1` and `S2`? Is this loop parallel? If not, show how to make it parallel.

4.15 [10] <4.4> List and describe at least four factors that influence the performance of GPU kernels. In other words, which runtime behaviors that are caused by the kernel code cause a reduction in resource utilization during kernel execution?

4.16 [10] <4.4> Assume a hypothetical GPU with the following characteristics:

- Clock rate 1.5 GHz
- Contains 16 SIMD processors, each containing 16 single-precision floating-point units
- Has 100 GB/s off-chip memory bandwidth

Without considering memory bandwidth, what is the peak single-precision floating-point throughput for this GPU in GLFOP/s, assuming that all memory latencies can be hidden? Is this throughput sustainable given the memory bandwidth limitation?

4.17 [60] <4.4> For this programming exercise, you will write and characterize the behavior of a CUDA kernel that contains a high amount of data-level parallelism but also contains conditional execution behavior. Use the NVIDIA CUDA Toolkit along with GPU-SIM from the University of British Columbia (http://www.gpgpu-sim.org/) or the CUDA Profiler to write and compile a CUDA kernel that performs 100 iterations of Conway's Game of Life for a 256×256 game board and returns the final state of the game board to the host. Assume that the board is initialized by the host. Associate one thread with each cell. Make sure you add a barrier after each game iteration. Use the following game rules:

- Any live cell with fewer than two live neighbors dies.
- Any live cell with two or three live neighbors lives on to the next generation.
- Any live cell with more than three live neighbors dies.
- Any dead cell with exactly three live neighbors becomes a live cell.

After finishing the kernel answer the following questions:

a. [60] <4.4> Compile your code using the –ptx option and inspect the PTX representation of your kernel. How many PTX instructions make up the PTX

implementation of your kernel? Did the conditional sections of your kernel include branch instructions or only predicated nonbranch instructions?

b. [60] <4.4> After executing your code in the simulator, what is the dynamic instruction count? What is the achieved instructions per cycle (IPC) or instruction issue rate? What is the dynamic instruction breakdown in terms of control instructions, arithmetic-logical unit (ALU) instructions, and memory instructions? Are there any shared memory bank conflicts? What is the effective off-chip memory bandwidth?

c. [60] <4.4> Implement an improved version of your kernel where off-chip memory references are coalesced and observe the differences in runtime performance.

5.1	Introduction	368
5.2	Centralized Shared-Memory Architectures	377
5.3	Performance of Symmetric Shared-Memory Multiprocessors	393
5.4	Distributed Shared-Memory and Directory-Based Coherence	404
5.5	Synchronization: The Basics	412
5.6	Models of Memory Consistency: An Introduction	417
5.7	Cross-Cutting Issues	422
5.8	Putting It All Together: Multicore Processors and Their Performance	426
5.9	Fallacies and Pitfalls	438
5.10	The Future of Multicore Scaling	442
5.11	Concluding Remarks	444
5.12	Historical Perspectives and References	445
	Case Studies and Exercises by Amr Zaky and David A. Wood	446

5

Thread-Level Parallelism

The turning away from the conventional organization came in the middle 1960s, when the law of diminishing returns began to take effect in the effort to increase the operational speed of a computer. . . . Electronic circuits are ultimately limited in their speed of operation by the speed of light . . . and many of the circuits were already operating in the nanosecond range.

W. Jack Bouknight et al.,
The Illiac IV System (1972)

We are dedicating all of our future product development to multicore designs. We believe this is a key inflection point for the industry.

Intel President Paul Otellini,
describing Intel's future direction at the Intel Developer Forum in 2005

Since 2004 processor designers have increased core counts to exploit Moore's Law scaling, rather than focusing on single-core performance. The failure of Dennard scaling, to which the shift to multicore parts is partially a response, may soon limit multicore scaling just as single-core scaling has been curtailed.

Hadi Esmaeilzadeh, et al.,
Power Limitations and Dark Silicon Challenge the Future of Multicore (2012)

Computer Architecture. https://doi.org/10.1016/B978-0-12-811905-1.00005-5
© 2019 Elsevier Inc. All rights reserved.

5.1 Introduction

As the quotations that open this chapter show, the view that advances in uniprocessor architecture were nearing an end has been held by some researchers for many years. Clearly, these views were premature; in fact, during the period of 1986–2003, uniprocessor performance growth, driven by the microprocessor, was at its highest rate since the first transistorized computers in the late 1950s and early 1960s.

Nonetheless, the importance of multiprocessors was growing throughout the 1990s as designers sought a way to build servers and supercomputers that achieved higher performance than a single microprocessor, while exploiting the tremendous cost-performance advantages of commodity microprocessors. As we discussed in Chapters 1 and 3, the slowdown in uniprocessor performance arising from diminishing returns in exploiting instruction-level parallelism (ILP) combined with growing concern over power has led to a new era in computer architecture—an era where multiprocessors play a major role from the low end to the high end. The second quotation captures this clear inflection point.

This increased importance of multiprocessing reflects several major factors:

- The dramatically lower efficiencies in silicon and energy use that were encountered between 2000 and 2005 as designers attempted to find and exploit more ILP, which turned out to be inefficient, since power and silicon costs grew faster than performance. Other than ILP, the only scalable and general-purpose way we know to increase performance faster than the basic technology allows (from a switching perspective) is through multiprocessing.
- A growing interest in high-end servers as cloud computing and software-as-a-service become more important.
- A growth in data-intensive applications driven by the availability of massive amounts of data on the Internet.
- The insight that increasing performance on the desktop is less important (outside of graphics, at least), either because current performance is acceptable or because highly compute- and data-intensive applications are being done on the cloud.
- An improved understanding of how to use multiprocessors effectively, especially in server environments where there is significant inherent parallelism, arising from large datasets (usually in the form of data parallelism), "natural-world" parallelism (which occurs in scientific and engineering codes), or parallelism among large numbers of independent requests (request-level parallelism).
- The advantages of leveraging a design investment by replication rather than unique design; all multiprocessor designs provide such leverage.

The third quotation reminds us that multicore may provide only limited possibilities for scaling performance. The combination of Amdahl's Law effects and the

end of Dennard scaling mean that the future of multicore may be limited, at least as a method of scaling up the performance of single applications. We return to this topic late in the chapter.

In this chapter, we focus on exploiting thread-level parallelism (TLP). TLP implies the existence of multiple program counters and thus is exploited primarily through MIMDs. Although MIMDs have been around for decades, the movement of thread-level parallelism to the forefront across the range of computing from embedded applications to high-end severs is relatively recent. Likewise, the extensive use of thread-level parallelism for a wide-range of general-purpose applications, versus either transaction processing or scientific applications, is relatively new.

Our focus in this chapter is on *multiprocessors*, which we define as computers consisting of tightly coupled processors whose coordination and usage are typically controlled by a single operating system and that share memory through a shared address space. Such systems exploit thread-level parallelism through two different software models. The first is the execution of a tightly coupled set of threads collaborating on a single task, which is typically called *parallel processing*. The second is the execution of multiple, relatively independent processes that may originate from one or more users, which is a form of *request-level parallelism*, although at a much smaller scale than what we explore in the next chapter. Request-level parallelism may be exploited by a single application running on multiple processors, such as a database responding to queries, or multiple applications running independently, often called *multiprogramming*.

The multiprocessors we examine in this chapter typically range in size from a dual processor to dozens and sometimes hundreds of processors and communicate and coordinate through the sharing of memory. Although sharing through memory implies a shared address space, it does not necessarily mean there is a single physical memory. Such multiprocessors include both single-chip systems with multiple cores, known as *multicore*, and computers consisting of multiple chips, each of which is typically a multicore. Many companies make such multiprocessors, including HP, Dell, Cisco, IBM, SGI, Lenovo, Oracle, Fujitsu, and many others.

In addition to true multiprocessors, we will return to the topic of multithreading, a technique that supports multiple threads executing in an interleaved fashion on a single multiple-issue processor. Many multicore processors also include support for multithreading.

In the next chapter, we consider ultrascale computers built from very large numbers of processors, connected with networking technology (not necessarily the same networking technology used to connect computers to the Internet) and often called *clusters*; these large-scale systems are used for cloud computing primarily with massive numbers of independent tasks being executed in parallel. More recently, computationally intensive tasks that can be easily made parallel, such as Search and certain machine learning algorithms have also made use of clusters. When these clusters grow to tens of thousands of servers and beyond, we call them *warehouse-scale computers*. Amazon, Google, Microsoft, and Facebook all make warehouse-scale computers.

In addition to the multiprocessors we study here and the warehouse-scaled systems of the next chapter, there are a range of special large-scale multiprocessor systems, sometimes called *multicomputers*, which are less tightly coupled than the multiprocessors examined in this chapter but usually more tightly coupled than the warehouse-scale systems of the next chapter. The primary use for such multicomputers is in high-end scientific computation, although they are sometimes used for commercial applications filling the niche between multiprocessors and warehouse-scale computers. The Cray X series and IBM BlueGene are typical examples of these multicomputers.

Many other books, such as Culler et al. (1999), cover such systems in detail. Because of the large and changing nature of the field of multiprocessing (the just-mentioned Culler et al. reference is over 1000 pages and discusses only multiprocessing!), we have chosen to focus our attention on what we believe is the most important and general-purpose portions of the computing space. Appendix I discusses some of the issues that arise in building such computers in the context of large-scale scientific applications.

Our focus will be on multiprocessors with roughly 4–256 processor cores, which might occupy anywhere from 4 to 16 separate chips. Such designs vastly dominate in terms of both units and dollars. In large-scale multiprocessors, the interconnection networks are a critical part of the design; Appendix F focuses on that topic.

Multiprocessor Architecture: Issues and Approach

To take advantage of an MIMD multiprocessor with n processors, we must usually have at least n threads or processes to execute; with multithreading, which is present in most multicore chips today, that number is 2–4 times higher. The independent threads within a single process are typically identified by the programmer or created by the operating system (from multiple independent requests). At the other extreme, a thread may consist of a few tens of iterations of a loop, generated by a parallel compiler exploiting data parallelism in the loop. Although the amount of computation assigned to a thread, called the *grain size*, is important in considering how to exploit thread-level parallelism efficiently, the important qualitative distinction from instruction-level parallelism is that thread-level parallelism is identified at a high level by the software system or programmer and that the threads consist of hundreds to millions of instructions that may be executed in parallel.

Threads can also be used to exploit data-level parallelism, although the overhead is usually higher than would be seen with an SIMD processor or with a GPU (see Chapter 4). This overhead means that grain size must be sufficiently large to exploit the parallelism efficiently. For example, although a vector processor or GPU may be able to efficiently parallelize operations on short vectors, the resulting grain size when the parallelism is split among many threads may be so small that the overhead makes the exploitation of the parallelism prohibitively expensive in an MIMD.

Existing shared-memory multiprocessors fall into two classes, depending on the number of processors involved, which in turn dictates a memory organization and interconnect strategy. We refer to the multiprocessors by their memory organization because what constitutes a small or large number of processors continues to change over time.

The first group, which we call *symmetric (shared-memory) multiprocessors* (SMPs), or *centralized shared-memory multiprocessors*, features small to moderate numbers of cores, typically 32 or fewer. For multiprocessors with such small processor counts, it is possible for the processors to share a single centralized memory that all processors have equal access to, thus the term *symmetric*. In multicore chips, the memory is often shared in a centralized fashion among the cores; most existing multicores are SMPs, but not all. (Note that some literature mistakenly appears to use SMP to stand for Shared Memory Processor, but this usage is erroneous.)

Some multicores have nonuniform access to the outermost cache, a structure called *NUCA* for *Nonuniform Cache Access*, and are thus are not truly SMPs, even if they have a single main memory. The IBM Power8 has distributed L3 caches with nonuniform access time to different addresses in L3.

In multiprocessors consisting of multiple multicore chips, there are often separate memories for each multicore chip. Thus the memory is distributed rather than centralized. As we will see later in the chapter, many designs with distributed memory have fast access to a local memory and much slower access to remote memory; often the differences in access time to various remote memories are small in comparison to the difference between the access times to the local memory and to a remote memory. In such designs, the programmer and software system need to be aware of whether accesses are to local or remote memory, but may be able to ignore the distribution of accesses among remote memories. Because an SMP approach becomes less attractive with a growing number of processors, most of the very largest multiprocessors use some form of distributed memory.

SMP architectures are also sometimes called *uniform memory access* (UMA) multiprocessors, arising from the fact that all processors have a uniform latency from memory, even if the memory is organized into multiple banks. Figure 5.1 shows what these multiprocessors look like. The architecture of SMPs is the topic of Section 5.2, and we explain the approach in the context of a multicore.

The alternative design approach consists of multiprocessors with physically distributed memory, called *distributed shared memory* (DSM). Figure 5.2 shows what these multiprocessors look like. To support larger processor counts, memory must be distributed among the processors rather than centralized; otherwise, the memory system would not be able to support the bandwidth demands of a larger number of processors without incurring excessively long access latency.

With the rapid increase in processor performance and the associated increase in a processor's memory bandwidth requirements, the size of a multiprocessor for

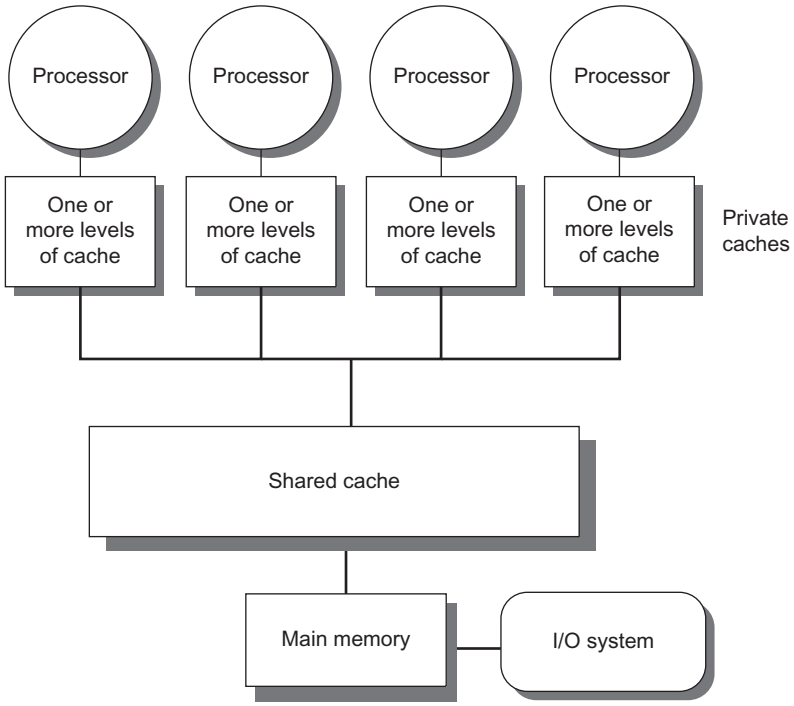


Figure 5.1 Basic structure of a centralized shared-memory multiprocessor based on a multicore chip. Multiple processor-cache subsystems share the same physical memory, typically with one level of shared cache on the multicore, and one or more levels of private per-core cache. The key architectural property is the uniform access time to all of the memory from all of the processors. In a multichip design, an interconnection network links the processors and the memory, which may be one or more banks. In a single-chip multicore, the interconnection network is simply the memory bus.

which distributed memory is preferred continues to shrink. The introduction of multicore processors has meant that even some 2-chip multiprocessors, which might have 16–64 processor cores, use distributed memory. The larger number of processors also raises the need for a high-bandwidth interconnect, of which we will see examples in Appendix F. Both directed networks (i.e., switches) and indirect networks (typically multidimensional meshes) are used.

Distributing the memory among the nodes both increases the bandwidth and reduces the latency to local memory. A DSM multiprocessor is also called a *NUMA* (nonuniform memory access) because the access time depends on the location of a data word in memory. The key disadvantages for a DSM are that communicating data among processors becomes somewhat more complex and a DSM requires more effort in the software to take advantage of the increased memory bandwidth

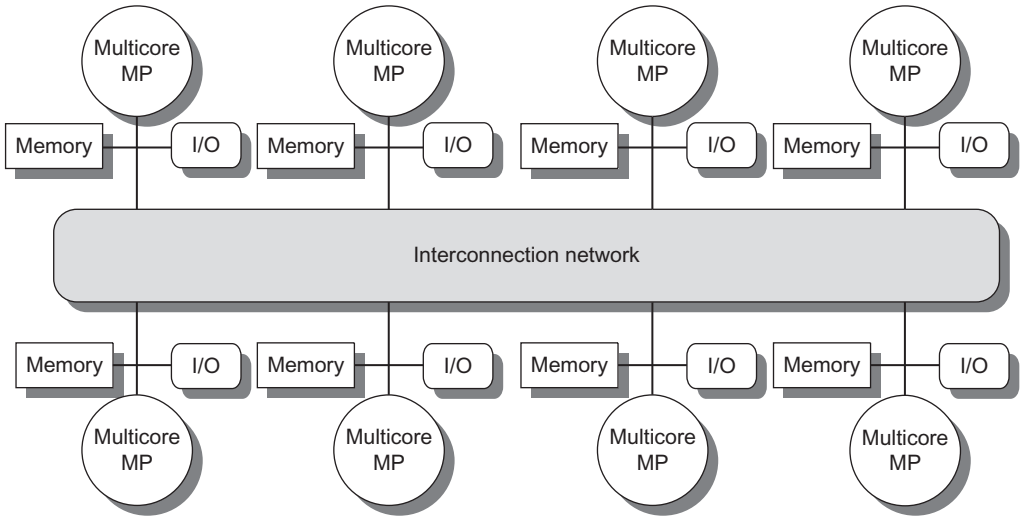


Figure 5.2 The basic architecture of a distributed-memory multiprocessor in 2017 typically consists of a multicore multiprocessor chip with memory and possibly I/O attached and an interface to an interconnection network that connects all the nodes. Each processor core shares the entire memory, although the access time to the local memory attached to the core's chip will be much faster than the access time to remote memories.

afforded by distributed memories. Because most multicore-based multiprocessors with more than a few processor chips use distributed memory, we will explain the operation of distributed memory multiprocessors from this viewpoint.

In both SMP and DSM architectures, communication among threads occurs through a shared address space, meaning that a memory reference can be made by any processor to any memory location, assuming it has the correct access rights. The term *shared memory* associated with both SMP and DSM refers to the fact that the *address space* is shared.

In contrast, the clusters and warehouse-scale computers in the next chapter look like individual computers connected by a network, and the memory of one processor cannot be accessed by another processor without the assistance of software protocols running on both processors. In such designs, message-passing protocols are used to communicate data among processors.

Challenges of Parallel Processing

The application of multiprocessors ranges from running independent tasks with essentially no communication to running parallel programs where threads must communicate to complete the task. Two important hurdles, both explainable with Amdahl's Law, make parallel processing challenging. To overcome these hurdles typically requires a comprehensive approach that addresses the choice of

algorithm and its implementation, the underlying programming language and system, the operating system and its support functions, and the architecture and hardware implementation. Although in many instances, one of these is a key bottleneck, when scaling to a larger processor counts (approaching 100 or more), often *all* aspects of the software and hardware need attention.

The first hurdle has to do with the limited parallelism available in programs, and the second arises from the relatively high cost of communications. Limitations in available parallelism make it difficult to achieve good speedups in any parallel processor, as our first example shows.

Example Suppose you want to achieve a speedup of 80 with 100 processors. What fraction of the original computation can be sequential?

Answer Recall from Chapter 1 that Amdahl's Law is

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} + (1 - \text{Fraction}_{\text{enhanced}})}$$

For simplicity in this example, assume that the program operates in only two modes: parallel with all processors fully used, which is the enhanced mode, or serial with only one processor in use. With this simplification, the speedup in enhanced mode is simply the number of processors, whereas the fraction of enhanced mode is the time spent in parallel mode. Substituting into the previous equation:

$$80 = \frac{1}{\frac{\text{Fraction}_{\text{parallel}}}{100} + \left(1 - \text{Fraction}_{\text{parallel}}\right)}$$

Simplifying this equation yields:

$$\begin{aligned} 0.8 \times \text{Fraction}_{\text{parallel}} + 80 \times \left(1 - \text{Fraction}_{\text{parallel}}\right) &= 1 \\ 80 - 79.2 \times \text{Fraction}_{\text{parallel}} &= 1 \\ \text{Fraction}_{\text{parallel}} &= \frac{80 - 1}{79.2} \\ \text{Fraction}_{\text{parallel}} &= 0.9975 \end{aligned}$$

Thus, to achieve a speedup of 80 with 100 processors, only 0.25% of the original computation can be sequential! Of course, to achieve linear speedup (speedup of n with n processors), the entire program must usually be parallel with no serial portions. In practice, programs do not just operate in fully parallel or sequential mode, but often use less than the full complement of the processors when running in parallel mode. Amdahl's Law can be used to analyze applications with varying amounts of speedup, as the next example shows.

Example Suppose we have an application running on a 100-processor multiprocessor, and assume that application can use 1, 50, or 100 processors. If we assume that 95% of the time we can use all 100 processors, how much of the remaining 5% of the execution time must employ 50 processors if we want a speedup of 80?

Answer We use Amdahl's Law with more terms:

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{100}}{\text{Speedup}_{100}} + \frac{\text{Fraction}_{50}}{\text{Speedup}_{50}} + (1 - \text{Fraction}_{100} - \text{Fraction}_{50})}$$

Substituting in:

$$80 = \frac{1}{\frac{0.95}{100} + \frac{\text{Fraction}_{50}}{50} + (1 - 0.95 - \text{Fraction}_{80})}$$

Simplifying:

$$0.76 + 1.6 \times \text{Fraction}_{50} + 4.0 - 80 \times \text{Fraction}_{50} = 1$$
$$4.76 - 78.4 \times \text{Fraction}_{50} = 1$$
$$\text{Fraction}_{50} = 0.048$$

If 95% of an application can use 100 processors perfectly, to get a speedup of 80, 4.8% of the remaining time must be spent using 50 processors and only 0.2% can be serial!

The second major challenge in parallel processing involves the large latency of remote access in a parallel processor. In existing shared-memory multiprocessors, communication of data between separate cores may cost 35–50 clock cycles and among cores on separate chips anywhere from 100 clock cycles to as much as 300 or more clock cycles (for large-scale multiprocessors), depending on the communication mechanism, the type of interconnection network, and the scale of the multiprocessor. The effect of long communication delays is clearly substantial. Let's consider a simple example.

Example Suppose we have an application running on a 32-processor multiprocessor that has a 100 ns delay to handle a reference to a remote memory. For this application, assume that all the references except those involving communication hit in the local memory hierarchy, which is obviously optimistic. Processors are stalled on a remote request, and the processor clock rate is 4 GHz. If the base CPI (assuming that all references hit in the cache) is 0.5, how much faster is the multiprocessor if there is no communication versus if 0.2% of the instructions involve a remote communication reference?

Answer It is simpler to first calculate the clock cycles per instruction. The effective CPI for the multiprocessor with 0.2% remote references is

$$\text{CPI} = \text{Base CPI} + \text{Remote request rate} \times \text{Remote request cost}$$

$$= 0.5 + 0.2\% \times \text{Remote request cost}$$

The remote request cost is

$$\frac{\text{Remote access cost}}{\text{Cycle time}} = \frac{100\,\text{ns}}{0.25\,\text{ns}} = 400\,\text{cycles}$$

Therefore we can compute the CPI:

$$\text{CPI} = 0.5 + 0.20\% \times 400$$

$$= 1.3$$

The multiprocessor with all local references is 1.3/0.5 = 2.6 times faster. In practice, the performance analysis is much more complex because some fraction of the noncommunication references will miss in the local hierarchy and the remote access time does not have a single constant value. For example, the cost of a remote reference could be worse because contention caused by many references trying to use the global interconnect can lead to increased delays, or the access time might be better if memory were distributed and the access was to the local memory.

This problem could have also been analyzed using Amdahl's Law, an exercise we leave to the reader.

These problems—insufficient parallelism and long-latency remote communication—are the two biggest performance challenges in using multiprocessors. The problem of inadequate application parallelism must be attacked primarily in software with new algorithms that offer better parallel performance, as well as by software systems that maximize the amount of time spent executing with the full complement of processors. Reducing the impact of long remote latency can be attacked both by the architecture and by the programmer. For example, we can reduce the frequency of remote accesses with either hardware mechanisms, such as caching shared data, or software mechanisms, such as restructuring the data to make more accesses local. We can try to tolerate the latency by using multithreading (discussed later in this chapter) or by using prefetching (a topic we cover extensively in Chapter 2).

Much of this chapter focuses on techniques for reducing the impact of long remote communication latency. For example, Sections 5.2 through 5.4 discuss how caching can be used to reduce remote access frequency, while maintaining a coherent view of memory. Section 5.5 discusses synchronization, which, because it inherently involves interprocessor communication and also can limit parallelism, is a major potential bottleneck. Section 5.6 covers latency-hiding techniques and memory consistency models for shared memory. In Appendix I, we focus primarily on larger-scale multiprocessors that are used predominantly for scientific work.

In that appendix, we examine the nature of such applications and the challenges of achieving speedup with dozens to hundreds of processors.

5.2 Centralized Shared-Memory Architectures

The observation that the use of large, multilevel caches can substantially reduce the memory bandwidth demands of a processor is the key insight that motivates centralized memory multiprocessors. Originally, these processors were all single-core and often took an entire board, and memory was located on a shared bus. With more recent, higher-performance processors, the memory demands have outstripped the capability of reasonable buses, and recent microprocessors directly connect memory to a single chip, which is sometimes called a *backside* or *memory bus* to distinguish it from the bus used to connect to I/O. Accessing a chip's local memory whether for an I/O operation or for an access from another chip requires going through the chip that "owns" that memory. Thus access to memory is asymmetric: faster to the local memory and slower to the remote memory. In a multicore that memory is shared among all the cores on a single chip, but the asymmetric access to the memory of one multicore from the memory of another usually remains.

Symmetric shared-memory machines usually support the caching of both shared and private data. *Private data* are used by a single processor, while *shared data* are used by multiple processors, essentially providing communication among the processors through reads and writes of the shared data. When a private item is cached, its location is migrated to the cache, reducing the average access time as well as the memory bandwidth required. Because no other processor uses the data, the program behavior is identical to that in a uniprocessor. When shared data are cached, the shared value may be replicated in multiple caches. In addition to the reduction in access latency and required memory bandwidth, this replication also provides a reduction in contention that may exist for shared data items that are being read by multiple processors simultaneously. Caching of shared data, however, introduces a new problem: cache coherence.

What Is Multiprocessor Cache Coherence?

Unfortunately, caching shared data introduces a new problem. Because the view of memory held by two different processors is through their individual caches, the processors could end up seeing different values for the same memory location, as Figure 5.3 illustrates. This difficulty is generally referred to as the *cache coherence problem*. Notice that the coherence problem exists because we have both a global state, defined primarily by the main memory, and a local state, defined by the individual caches, which are private to each processor core. Thus, in a multicore where some level of caching may be shared (e.g., an L3), although some levels are private (e.g., L1 and L2), the coherence problem still exists and must be solved.

Informally, we could say that a memory system is coherent if any read of a data item returns the most recently written value of that data item. This definition,

Time	Event	Cache contents for processor A	Cache contents for processor B	Memory contents for location X
0				1
1	Processor A reads X	1		1
2	Processor B reads X	1	1	1
3	Processor A stores 0 into X	0	1	0

Figure 5.3 The cache coherence problem for a single memory location (X), read and written by two processors (A and B). We initially assume that neither cache contains the variable and that X has the value 1. We also assume a write-through cache; a write-back cache adds some additional but similar complications. After the value of X has been written by A, A's cache and the memory both contain the new value, but B's cache does not, and if B reads the value of X it will receive 1!

although intuitively appealing, is vague and simplistic; the reality is much more complex. This simple definition contains two different aspects of memory system behavior, both of which are critical to writing correct shared-memory programs. The first aspect, called *coherence*, defines what values can be returned by a read. The second aspect, called *consistency*, determines when a written value will be returned by a read. Let's look at coherence first.

A memory system is coherent if

1. A read by processor P to location X that follows a write by P to X, with no writes of X by another processor occurring between the write and the read by P, always returns the value written by P.
2. A read by a processor to location X that follows a write by another processor to X returns the written value if the read and write are sufficiently separated in time and no other writes to X occur between the two accesses.
3. Writes to the same location are *serialized*; that is, two writes to the same location by any two processors are seen in the same order by all processors. For example, if the values 1 and then 2 are written to a location, processors can never read the value of the location as 2 and then later read it as 1.

The first property simply preserves program order—we expect this property to be true even in uniprocessors. The second property defines the notion of what it means to have a coherent view of memory: if a processor could continuously read an old data value, we would clearly say that memory was incoherent.

The need for write serialization is more subtle, but equally important. Suppose we did not serialize writes, and processor P1 writes location X followed by P2 writing location X. Serializing the writes ensures that every processor will see the write done by P2 at some point. If we did not serialize the writes, it might be the case that some processors could see the write of P2 first and then see the write of P1, maintaining the value written by P1 indefinitely. The simplest way

to avoid such difficulties is to ensure that all writes to the same location are seen in the same order; this property is called *write serialization*.

Although the three properties just described are sufficient to ensure coherence, the question of when a written value will be seen is also important. To see why, observe that we cannot require that a read of X instantaneously see the value written for X by some other processor. If, for example, a write of X on one processor precedes a read of X on another processor by a very small time, it may be impossible to ensure that the read returns the value of the data written, since the written data may not even have left the processor at that point. The issue of exactly *when* a written value must be seen by a reader is defined by a *memory consistency model*—a topic discussed in Section 5.6.

Coherence and consistency are complementary: *Coherence defines the behavior of reads and writes to the same memory location, while consistency defines the behavior of reads and writes with respect to accesses to other memory locations*. For now, make the following two assumptions. First, a write does not complete (and allow the next write to occur) until all processors have seen the effect of that write. Second, the processor does not change the order of any write with respect to any other memory access. These two conditions mean that, if a processor writes location A followed by location B, any processor that sees the new value of B must also see the new value of A. These restrictions allow the processor to reorder reads, but forces the processor to finish a write in program order. We will rely on this assumption until we reach Section 5.6, where we will see exactly the implications of this definition, as well as the alternatives.

Basic Schemes for Enforcing Coherence

The coherence problem for multiprocessors and I/O, although similar in origin, has different characteristics that affect the appropriate solution. Unlike I/O, where multiple data copies are a rare event—one to be avoided whenever possible—a program running on multiple processors will normally have copies of the same data in several caches. In a coherent multiprocessor, the caches provide both *migration* and *replication* of shared data items.

Coherent caches provide migration because a data item can be moved to a local cache and used there in a transparent fashion. This migration reduces both the latency to access a shared data item that is allocated remotely and the bandwidth demand on the shared memory.

Because the caches make a copy of the data item in the local cache, coherent caches also provide replication for shared data that are being read simultaneously. Replication reduces both latency of access and contention for a read shared data item. Supporting this migration and replication is critical to performance in accessing shared data. Thus, rather than trying to solve the problem by avoiding it in software, multiprocessors adopt a hardware solution by introducing a protocol to maintain coherent caches.

The protocols to maintain coherence for multiple processors are called *cache coherence protocols*. Key to implementing a cache coherence protocol is tracking

the state of any sharing of a data block. The state of any cache block is kept using status bits associated with the block, similar to the valid and dirty bits kept in a uniprocessor cache. There are two classes of protocols in use, each of which uses different techniques to track the sharing status:

- *Directory based*—The sharing status of a particular block of physical memory is kept in one location, called the *directory*. There are two very different types of directory-based cache coherence. In an SMP, we can use one centralized directory, associated with the memory or some other single serialization point, such as the outermost cache in a multicore. In a DSM, it makes no sense to have a single directory because that would create a single point of contention and make it difficult to scale to many multicore chips given the memory demands of multicores with eight or more cores. Distributed directories are more complex than a single directory, and such designs are the subject of Section 5.4.
- *Snooping*—Rather than keeping the state of sharing in a single directory, every cache that has a copy of the data from a block of physical memory could track the sharing status of the block. In an SMP, the caches are typically all accessible via some broadcast medium (e.g., a bus connects the per-core caches to the shared cache or memory), and all cache controllers monitor or *snoop* on the medium to determine whether they have a copy of a block that is requested on a bus or switch access. Snooping can also be used as the coherence protocol for a multichip multiprocessor, and some designs support a snooping protocol on top of a directory protocol within each multicore.

Snooping protocols became popular with multiprocessors using microprocessors (single-core) and caches attached to a single shared memory by a bus. The bus provided a convenient broadcast medium to implement the snooping protocols. Multicore architectures changed the picture significantly because all multicores share some level of cache on the chip. Thus some designs switched to using directory protocols, since the overhead was small. To allow the reader to become familiar with both types of protocols, we focus on a snooping protocol here and discuss a directory protocol when we come to DSM architectures.

Snooping Coherence Protocols

There are two ways to maintain the coherence requirement described in the prior section. One method is to ensure that a processor has exclusive access to a data item before writing that item. This style of protocol is called a *write invalidate protocol* because it invalidates other copies on a write. It is by far the most common protocol. Exclusive access ensures that no other readable or writable copies of an item exist when the write occurs: all other cached copies of the item are invalidated.

Figure 5.4 shows an example of an invalidation protocol with write-back caches in action. To see how this protocol ensures coherence, consider a write followed by a read by another processor: because the write requires exclusive

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

Figure 5.4 An example of an invalidation protocol working on a snooping bus for a single cache block (X) with write-back caches. We assume that neither cache initially holds X and that the value of X in memory is 0. The processor and memory contents show the value after the processor and bus activity have both completed. A blank indicates no activity or no copy cached. When the second miss by B occurs, processor A responds with the value canceling the response from memory. In addition, both the contents of B's cache and the memory contents of X are updated. This update of memory, which occurs when a block becomes shared, simplifies the protocol, but it is possible to track the ownership and force the write-back only if the block is replaced. This requires the introduction of an additional status bit indicating ownership of a block. The ownership bit indicates that a block may be shared for reads, but only the owning processor can write the block, and that processor is responsible for updating any other processors and memory when it changes the block or replaces it. If a multicore uses a shared cache (e.g., L3), then all memory is seen through the shared cache; L3 acts like the memory in this example, and coherency must be handled for the private L1 and L2 caches for each core. It is this observation that led some designers to opt for a directory protocol within the multicore. To make this work, the L3 cache must be inclusive; recall from Chapter 2, that a cache is inclusive if any location in a higher level cache (L1 and L2 in this case) is also in L3. We return to the topic of inclusion on page 423.

access, any copy held by the reading processor must be invalidated (thus the protocol name). Therefore when the read occurs, it misses in the cache and is forced to fetch a new copy of the data. For a write, we require that the writing processor has exclusive access, preventing any other processor from being able to write simultaneously. If two processors do attempt to write the same data simultaneously, one of them wins the race (we'll see how we decide who wins shortly), causing the other processor's copy to be invalidated. For the other processor to complete its write, it must obtain a new copy of the data, which must now contain the updated value. Therefore this protocol enforces write serialization.

The alternative to an invalidate protocol is to update all the cached copies of a data item when that item is written. This type of protocol is called a *write update* or *write broadcast* protocol. Because a write update protocol must broadcast all writes to shared cache lines, it consumes considerably more bandwidth. For this reason, virtually all recent multiprocessors have opted to implement a write invalidate protocol, and we will focus only on invalidate protocols for the rest of the chapter.

Basic Implementation Techniques

The key to implementing an invalidate protocol in a multicore is the use of the bus, or another broadcast medium, to perform invalidates. In older multiple-chip multiprocessors, the bus used for coherence is the shared-memory access bus. In a single-chip multicore, the bus can be the connection between the private caches (L1 and L2 in the Intel i7) and the shared outer cache (L3 in the i7). To perform an invalidate, the processor simply acquires bus access and broadcasts the address to be invalidated on the bus. All processors continuously snoop on the bus, watching the addresses. The processors check whether the address on the bus is in their cache. If so, the corresponding data in the cache are invalidated.

When a write to a block that is shared occurs, the writing processor must acquire bus access to broadcast its invalidation. If two processors attempt to write shared blocks at the same time, their attempts to broadcast an invalidate operation will be serialized when they arbitrate for the bus. The first processor to obtain bus access will cause any other copies of the block it is writing to be invalidated. If the processors were attempting to write the same block, the serialization enforced by the bus would also serialize their writes. One implication of this scheme is that a write to a shared data item cannot actually complete until it obtains bus access. All coherence schemes require some method of serializing accesses to the same cache block, either by serializing access to the communication medium or to another shared structure.

In addition to invalidating outstanding copies of a cache block that is being written into, we also need to locate a data item when a cache miss occurs. In a write-through cache, it is easy to find the recent value of a data item because all written data are always sent to the memory, from which the most recent value of a data item can always be fetched. (Write buffers can lead to some additional complexities and must effectively be treated as additional cache entries.)

For a write-back cache, the problem of finding the most recent data value is harder because the most recent value of a data item can be in a private cache rather than in the shared cache or memory. Fortunately, write-back caches can use the same snooping scheme both for cache misses and for writes: each processor snoops every address placed on the shared bus. If a processor finds that it has a dirty copy of the requested cache block, it provides that cache block in response to the read request and causes the memory (or L3) access to be aborted. The additional complexity comes from having to retrieve the cache block from another processor's private cache (L1 or L2), which can often take longer than retrieving it from L3. Because write-back caches generate lower requirements for memory bandwidth, they can support larger numbers of faster processors. As a result, all multicore processors use write-back at the outermost levels of the cache, and we will examine the implementation of coherence with write-back caches.

The normal cache tags can be used to implement the process of snooping, and the valid bit for each block makes invalidation easy to implement. Read misses, whether generated by an invalidation or by some other event, are also straightforward because they simply rely on the snooping capability. For writes, we want to know whether any other copies of the block are cached because, if there are no other cached copies, then the write does not need to be placed on the bus in a

write-back cache. Not sending the write reduces both the time to write and the required bandwidth.

To track whether or not a cache block is shared, we can add an extra state bit associated with each cache block, just as we have a valid bit and a dirty bit. By adding a bit indicating whether the block is shared, we can decide whether a write must generate an invalidate. When a write to a block in the shared state occurs, the cache generates an invalidation on the bus and marks the block as *exclusive*. No further invalidations will be sent by that core for that block. The core with the sole copy of a cache block is normally called the *owner* of the cache block.

When an invalidation is sent, the state of the owner's cache block is changed from shared to unshared (or exclusive). If another processor later requests this cache block, the state must be made shared again. Because our snooping cache also sees any misses, it knows when the exclusive cache block has been requested by another processor and the state should be made shared.

Every bus transaction must check the cache-address tags, which could potentially interfere with processor cache accesses. One way to reduce this interference is to duplicate the tags and have snoop accesses directed to the duplicate tags. Another approach is to use a directory at the shared L3 cache; the directory indicates whether a given block is shared and possibly which cores have copies. With the directory information, invalidates can be directed only to those caches with copies of the cache block. This requires that L3 must always have a copy of any data item in L1 or L2, a property called *inclusion*, which we will return to in Section 5.7.

An Example Protocol

A snooping coherence protocol is usually implemented by incorporating a finite-state controller in each core. This controller responds to requests from the processor in the core and from the bus (or other broadcast medium), changing the state of the selected cache block, as well as using the bus to access data or to invalidate it. Logically, you can think of a separate controller as being associated with each block; that is, snooping operations or cache requests for different blocks can proceed independently. In actual implementations, a single controller allows multiple operations to distinct blocks to proceed in interleaved fashion (i.e., one operation may be initiated before another is completed, even though only one cache access or one bus access is allowed at a time). Also, remember that, although we refer to a bus in the following description, any interconnection network that supports a broadcast to all the coherence controllers and their associated private caches can be used to implement snooping.

The simple protocol we consider has three states: invalid, shared, and modified. The shared state indicates that the block in the private cache is potentially shared, whereas the modified state indicates that the block has been updated in the private cache; note that the modified state *implies* that the block is exclusive. Figure 5.5 shows the requests generated by a core (in the top half of the table) as well as those coming from the bus (in the bottom half of the table). This protocol is for a write-back cache but is easily changed to work for a write-through cache by reinterpreting the modified state as an exclusive state and updating the cache on writes

in the normal fashion for a write-through cache. The most common extension of this basic protocol is the addition of an exclusive state, which describes a block that is unmodified but held in only one private cache. We describe this and other extensions on page 388.

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block; then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or *ownership* misses, because they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block; then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to read shared data: place cache block on bus, write-back block, and change state to shared.
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

Figure 5.5 The cache coherence mechanism receives requests from both the core's processor and the shared bus and responds to these based on the type of request, whether it hits or misses in the local cache, and the state of the local cache block specified in the request. The fourth column describes the type of cache action as normal hit or miss (the same as a uniprocessor cache would see), replacement (a uniprocessor cache replacement miss), or coherence (required to maintain cache coherence); a normal or replacement action may cause a coherence action depending on the state of the block in other caches. For read, misses, write misses, or invalidates snooped from the bus, an action is required *only* if the read or write addresses match a block in the local cache and the block is valid.

When an invalidate or a write miss is placed on the bus, any cores whose private caches have copies of the cache block invalidate it. For a write miss in a write-back cache, if the block is exclusive in just one private cache, that cache also writes back the block; otherwise, the data can be read from the shared cache or memory.

Figure 5.6 shows a finite-state transition diagram for a single private cache block using a write invalidation protocol and a write-back cache. For simplicity, the three states of the protocol are duplicated to represent transitions based on

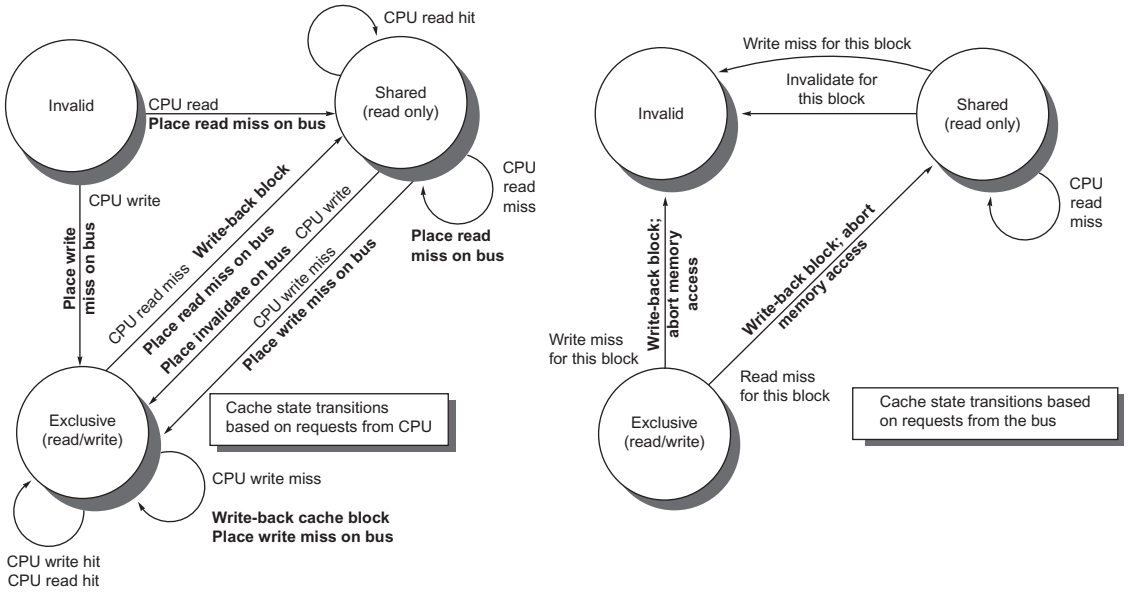


Figure 5.6 A write invalidate, cache coherence protocol for a private write-back cache showing the states and state transitions for each block in the cache. The cache states are shown in circles, with any access permitted by the local processor without a state transition shown in parentheses under the name of the state. The stimulus causing a state change is shown on the transition arcs in regular type, and any bus actions generated as part of the state transition are shown on the transition arc in *bold*. The stimulus actions apply to a block in the private cache, not to a specific address in the cache. Thus a read miss to a block in the shared state is a miss for that cache block but for a different address. The left side of the diagram shows state transitions based on actions of the processor associated with this cache; the right side shows transitions based on operations on the bus. A read miss in the exclusive or shared state and a write miss in the exclusive state occur when the address requested by the processor does not match the address in the local cache block. Such a miss is a standard cache replacement miss. An attempt to write a block in the shared state generates an invalidate. Whenever a bus transaction occurs, all private caches that contain the cache block specified in the bus transaction take the action dictated by the right half of the diagram. The protocol assumes that memory (or a shared cache) provides data on a read miss for a block that is clean in all local caches. In actual implementations, these two sets of state diagrams are combined. In practice, there are many subtle variations on invalidate protocols, including the introduction of the exclusive unmodified state, as to whether a processor or memory provides data on a miss. In a multicore chip, the shared cache (usually L3, but sometimes L2) acts as the equivalent of memory, and the bus is the bus between the private caches of each core and the shared cache, which in turn interfaces to the memory.

processor requests (on the left, which corresponds to the top half of the table in Figure 5.5), as opposed to transitions based on bus requests (on the right, which corresponds to the bottom half of the table in Figure 5.5). Boldface type is used to distinguish the bus actions, as opposed to the conditions on which a state transition depends. The state in each node represents the state of the selected private cache block specified by the processor or bus request.

All of the states in this cache protocol would be needed in a uniprocessor cache, where they would correspond to the invalid, valid (and clean), and dirty states. Most of the state changes indicated by arcs in the left half of Figure 5.6 would be needed in a write-back uniprocessor cache, with the exception being the invalidate on a write hit to a shared block. The state changes represented by the arcs in the right half of Figure 5.6 are needed only for coherence and would not appear at all in a uniprocessor cache controller.

As mentioned earlier, there is only one finite-state machine per cache, with stimuli coming either from the attached processor or from the bus. Figure 5.7 shows how the state transitions in the right half of Figure 5.6 are combined with those in the left half of the figure to form a single state diagram for each cache block.

To understand why this protocol works, observe that any valid cache block is either in the shared state in one or more private caches or in the exclusive state in exactly one cache. Any transition to the exclusive state (which is required for a processor to write to the block) requires an invalidate or write miss to be placed on the bus, causing all local caches to make the block invalid. In addition, if some other local cache had the block in exclusive state, that local cache generates a write-back, which supplies the block containing the desired address. Finally, if a read miss occurs on the bus to a block in the exclusive state, the local cache with the exclusive copy changes its state to shared.

The actions in gray in Figure 5.7, which handle read and write misses on the bus, are essentially the snooping component of the protocol. One other property that is preserved in this protocol, and in most other protocols, is that any memory block in the shared state is always up to date in the outer shared cache (L2 or L3, or memory if there is no shared cache), which simplifies the implementation. In fact, it does not matter whether the level out from the private caches is a shared cache or memory; the key is that all accesses from the cores go through that level.

Although our simple cache protocol is correct, it omits a number of complications that make the implementation much trickier. The most important of these is that the protocol assumes that operations are *atomic*—that is, an operation can be done in such a way that no intervening operation can occur. For example, the protocol described assumes that write misses can be detected, acquire the bus, and receive a response as a single atomic action. In reality this is not true. In fact, even a read miss might not be atomic; after detecting a miss in the L2 of a multicore, the core must arbitrate for access to the bus connecting to the shared L3. Nonatomic actions introduce the possibility that the protocol can *deadlock*, meaning that it reaches a state where it cannot continue. We will explore these complications later in this section and when we examine DSM designs.

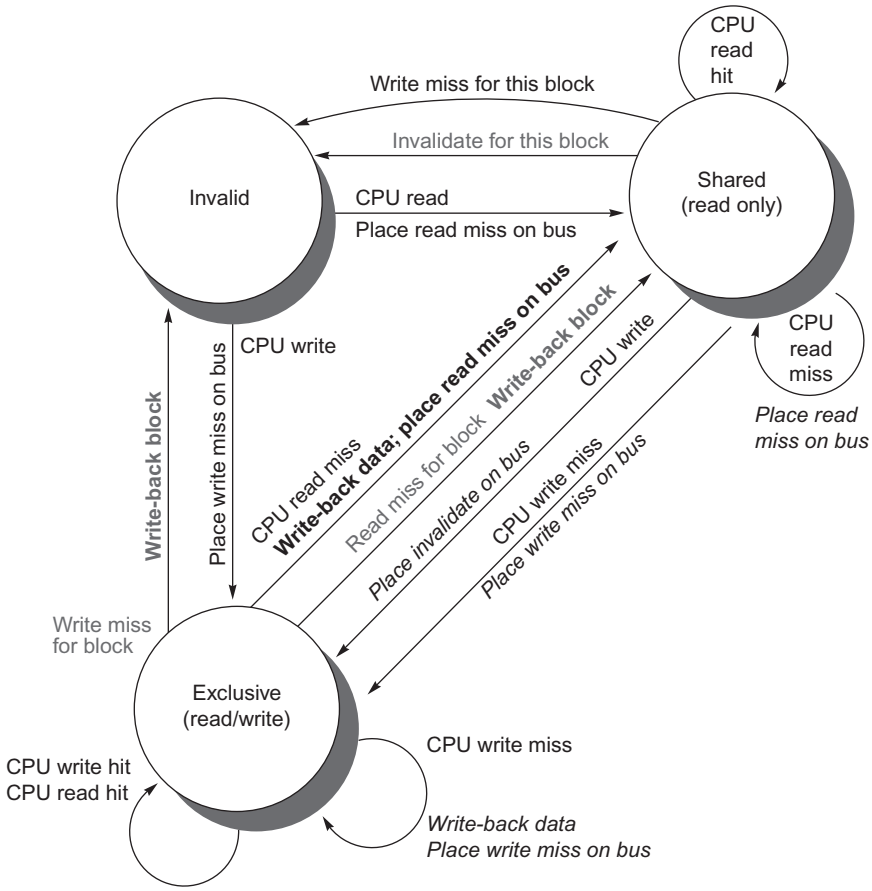


Figure 5.7 Cache coherence state diagram with the state transitions induced by the local processor shown in *black* and by the bus activities shown in *gray*. As in Figure 5.6, the activities on a transition are shown in *bold*.

With multicore processors, the coherence among the processor cores is all implemented on chip, using either a snooping or simple central directory protocol. Many multiprocessor chips, including the Intel Xeon and AMD Opteron, support multichip multiprocessors that could be built by connecting a high-speed interface already incorporated in the chip. These next-level interconnects are not just extensions of the shared bus, but use a different approach for interconnecting multicores.

A multiprocessor built with multiple multicore chips will usually have a distributed memory architecture and will need an interchip coherency mechanism above and beyond the one within the chip. In most cases, some form of directory scheme is used.

Extensions to the Basic Coherence Protocol

The coherence protocol we have just described is a simple three-state protocol and is often referred to by the first letter of the states, making it a MSI (Modified, Shared, Invalid) protocol. There are many extensions of this basic protocol, which we mention in the captions of figures in this section. These extensions are created by adding additional states and transactions that optimize certain behaviors, possibly resulting in improved performance. Two of the most common extensions are

1. *MESI* adds the state Exclusive to the basic MSI protocol, yielding four states (Modified, Exclusive, Shared, and Invalid). The exclusive state indicates that a cache block is resident in only a single cache but is clean. If a block is in the E state, it can be written without generating any invalidates, which optimizes the case where a block is read by a single cache before being written by that same cache. Of course, when a read miss to a block in the E state occurs, the block must be changed to the S state to maintain coherence. Because all subsequent accesses are snooped, it is possible to maintain the accuracy of this state. In particular, if another processor issues a read miss, the state is changed from exclusive to shared. The advantage of adding this state is that a subsequent write to a block in the exclusive state by the same core need not acquire bus access or generate an invalidate, since the block is known to be exclusively in this local cache; the processor merely changes the state to modified. This state is easily added by using the bit that encodes the coherent state as an exclusive state and using the dirty bit to indicate that a bock is modified. The Intel i7 uses a variant of a MESI protocol, called MESIF, which adds a state (Forward) to designate which sharing processor should respond to a request. It is designed to enhance performance in distributed memory organizations.
2. *MOESI* adds the state Owned to the MESI protocol to indicate that the associated block is owned by that cache and out-of-date in memory. In MSI and MESI protocols, when there is an attempt to share a block in the Modified state, the state is changed to Shared (in both the original and newly sharing cache), and the block must be written back to memory. In a MOESI protocol, the block can be changed from the Modified to Owned state in the original cache without writing it to memory. Other caches, which are newly sharing the block, keep the block in the Shared state; the O state, which only the original cache holds, indicates that the main memory copy is out of date and that the designated cache is the owner. The owner of the block must supply it on a miss, since memory is not up to date and must write the block back to memory if it is replaced. The AMD Opteron processor family uses the MOESI protocol.

The next section examines the performance of these protocols for our parallel and multiprogrammed workloads; the value of these extensions to a basic protocol will be clear when we examine the performance. But, before we do that, let's take a brief look at the limitations on the use of a symmetric memory structure and a snooping coherence scheme.

Limitations in Symmetric Shared-Memory Multiprocessors and Snooping Protocols

As the number of processors in a multiprocessor grows, or as the memory demands of each processor grow, any centralized resource in the system can become a bottleneck. For multicores, a single shared bus became a bottleneck with only a few cores. As a result, multicore designs have gone to higher bandwidth interconnection schemes, as well as multiple, independent memories to allow larger numbers of cores. The multicore chips we examine in Section 5.8 use three different approaches:

1. The IBM Power8, which has up to 12 processors in a single multicore, uses 8 parallel buses that connect the distributed L3 caches and up to 8 separate memory channels.
2. The Xeon E7 uses three rings to connect up to 32 processors, a distributed L3 cache, and two or four memory channels (depending on the configuration).
3. The Fujitsu SPARC64 X+ uses a crossbar to connect a shared L2 to up to 16 cores and multiple memory channels.

The SPARC64 X+ is a symmetric organization with uniform access time. The Power8 has nonuniform access time for both L3 and memory. Although the uncontended access time differences among memory addresses within a single Power8 multicore are not large, with contention for memory, the access time differences can become significant even within one chip. The Xeon E7 can operate as if access times were uniform; in practice, software systems usually organize memory so that the memory channels are associated with a subset of the cores.

Snooping bandwidth at the caches can also become a problem because every cache must examine every miss, and having additional interconnection bandwidth only pushes the problem to the cache. To understand this problem, consider the following example.

Example Consider an 8-processor multicore where each processor has its own L1 and L2 caches, and snooping is performed on a shared bus among the L2 caches. Assume the average L2 request, whether for a coherence miss or other miss, is 15 cycles. Assume a clock rate of 3.0 GHz, a CPI of 0.7, and a load/store frequency of 40%. If our goal is that no more than 50% of the L2 bandwidth is consumed by coherence traffic, what is the maximum coherence miss rate per processor?

Answer Start with an equation for the number of cache cycles that can be used (where CMR is the coherence miss rate):

$$\text{Cache cycles available} = \frac{\text{Clock rate}}{\text{Cycles per request} \times 2} = \frac{3.0\text{Ghz}}{30} = 0.1 \times 10^9$$

$$\begin{aligned}\text{Cache cycles available} &= \text{Memory references/clock/processor} \times \text{Clock rate} \\ &\quad \times \text{processor count} \times \text{CMR} \\ &= \frac{0.4}{0.7} \times 3.0\text{GHz} \times 8 \times \text{CMR} = 13.7 \times 10^9 \times \text{CMR}\end{aligned}$$

$$\text{CMR} = \frac{0.1}{13.7} = 0.0073 = 0.73\%$$

This means that the coherence miss rate must be 0.73% or less. In the next section, we will see several applications with coherence miss rates in excess of 1%. Alternatively, if we assume that CMR can be 1%, then we could support just under 6 processors. Clearly, even small multicores will require a method for scaling snoop bandwidth.

There are several techniques for increasing the snoop bandwidth:

1. As mentioned earlier, the tags can be duplicated. This doubles the effective cache-level snoop bandwidth. If we assume that half the coherence requests do not hit on a snoop request and the cost of the snoop request is only 10 cycles (versus 15), then we can cut the average cost of a CMR to 12.5 cycles. This reduction allows the coherence miss rate to be 0.88, or alternatively to support one additional processor (7 versus 6).
2. If the outermost cache on a multicore (typically L3) is shared, we can distribute that cache so that each processor has a portion of the memory and handles snoops for that portion of the address space. This approach, used by the IBM 12-core Power8, leads to a NUCA design, but effectively scales the snoop bandwidth at L3 by the number of processors. If there is a snoop hit in L3, then we must still broadcast to all L2 caches, which must in turn snoop their contents. Since L3 is acting as a filter on the snoop requests, L3 must be inclusive.
3. We can place a directory at the level of the outermost shared cache (say, L3). L3 acts as a filter on snoop requests and must be inclusive. The use of a directory at L3 means that we need not snoop or broadcast to all the L2 caches, but only those that the directory indicates may have a copy of the block. Just as L3 may be distributed, the associated directory entries may also be distributed. This approach is used in the Intel Xeon E7 series, which supports from 8 to 32 cores.

Figure 5.8 shows how a multicore with a distributed cache system, such as that used in schemes 2 or 3, might look. If additional multicore chips were added to form a larger multiprocessor, an off-chip network would needed, as well as a method to extend the coherence mechanisms (as we will see in Section 5.8).

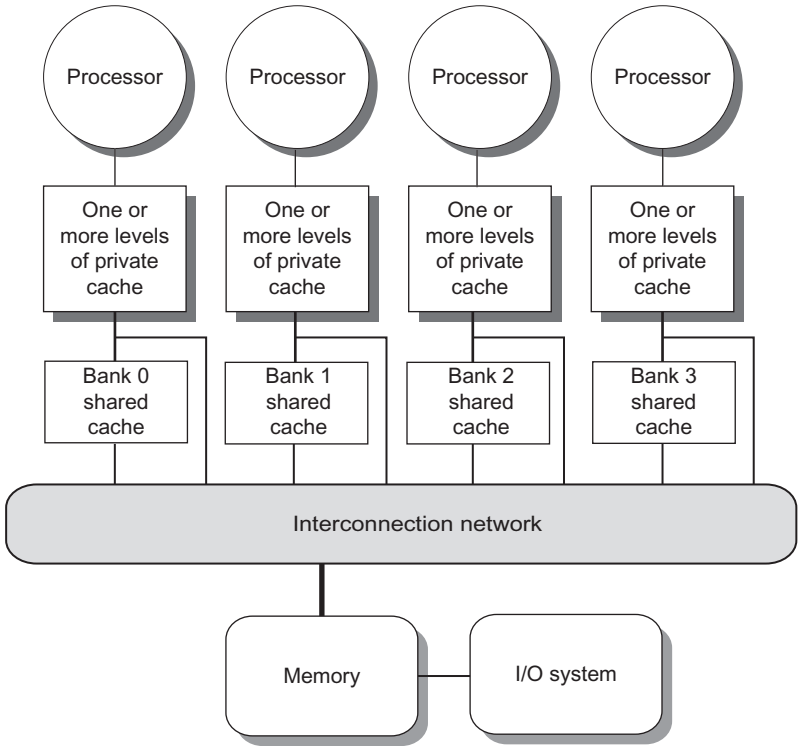


Figure 5.8 A single-chip multicore with a distributed cache. In current designs, the distributed shared cache is usually L3, and levels L1 and L2 are private. There are typically multiple memory channels (2–8 in today's designs). This design is NUCA, since the access time to L3 portions varies with faster access time for the directly attached core. Because it is NUCA, it is also NUMA.

The AMD Opteron represents another intermediate point in the spectrum between a snooping and a directory protocol. Memory is directly connected to each multicore chip, and up to four multicore chips can be connected. The system is a NUMA because local memory is somewhat faster. The Opteron implements its coherence protocol using the point-to-point links to broadcast up to three other chips. Because the interprocessor links are not shared, the only way a processor can know when an invalid operation has completed is by an explicit acknowledgment. Thus the coherence protocol uses a broadcast to find potentially shared copies, like a snooping protocol, but uses the acknowledgments to order operations, like a directory protocol. Because local memory is only somewhat faster than remote memory in the Opteron implementation, some software treats the Opteron multiprocessor as having uniform memory access.

In Section 5.4, we examine directory-based protocols, which eliminate the need for broadcast to all caches on a miss. Some multicore designs use directories

within the multicore (Intel Xeon E7), while others add directories when scaling beyond a multicore. Distributed directories eliminate the need for a single point to serialize all accesses (typically a single shared bus in a snooping scheme), and any scheme that removes the single point of serialization must deal with many of the same challenges as a distributed directory scheme.

Implementing Snooping Cache Coherence

The devil is in the details.

Classic proverb

When we wrote the first edition of this book in 1990, our final "Putting It All Together" was a 30-processor, single-bus multiprocessor using snoop-based coherence; the bus had a capacity of just over 50 MiB/s, which would not be enough bus bandwidth to support even one core of an Intel i7 in 2017! When we wrote the second edition of this book in 1995, the first cache coherence multiprocessors with more than a single bus had recently appeared, and we added an appendix describing the implementation of snooping in a system with multiple buses. In 2017, *every* multicore multiprocessor system that supports 8 or more cores uses an interconnect other than a single bus, and designers must face the challenge of implementing snooping (or a directory scheme) without the simplification of a bus to serialize events.

As we observed on page 386, the major complication in actually implementing the snooping coherence protocol we have described is that write and upgrade misses are not atomic in any recent multiprocessor. The steps of detecting a write or upgrade miss, communicating with the other processors and memory, getting the most recent value for a write miss and ensuring that any invalidates are processed, and updating the cache cannot be done as though they took a single cycle.

In a multicore with a single bus, these steps can be made effectively atomic by arbitrating for the bus to the shared cache or memory first (before changing the cache state) and not releasing the bus until all actions are complete. How can the processor know when all the invalidates are complete? In early designs, a single line was used to signal when all necessary invalidates had been received and were being processed. Following that signal, the processor that generated the miss could release the bus, knowing that any required actions would be completed before any activity related to the next miss. By holding the bus exclusively during these steps, the processor effectively made the individual steps atomic.

In a system without a single, central bus, we must find some other method of making the steps in a miss atomic. In particular, we must ensure that two processors that attempt to write the same block at the same time, a situation which is called a *race*, are strictly ordered: one write is processed and precedes before the next is begun. It does not matter which of two writes in a race wins the race, just that there be only a single winner whose coherence actions are completed first. In a multicore

using multiple buses, races can be eliminated if each block of memory is associated with only a single bus, ensuring that two attempts to access the same block must be serialized by that common bus. This property, together with the ability to restart the miss handling of the loser in a race, are the keys to implementing snooping cache coherence without a bus. We explain the details in Appendix I.

It is possible to combine snooping and directories, and several designs use snooping within a multicore and directories among multiple chips or a combination of directories at one cache level and snooping at another level.

5.3 Performance of Symmetric Shared-Memory Multiprocessors

In a multicore using a snooping coherence protocol, several different phenomena combine to determine performance. In particular, the overall cache performance is a combination of the behavior of uniprocessor cache miss traffic and the traffic caused by communication, which results in invalidations and subsequent cache misses. Changing t he processor count, cache size, and block size can affect these two components of the miss rate in different ways, leading to overall system behavior that is a combination of the two effects.

Appendix B breaks the uniprocessor miss rate into the three C's classification (capacity, compulsory, and conflict) and provides insight into both application behavior and potential improvements to the cache design. Similarly, the misses that arise from interprocessor communication, which are often called *coherence misses*, can be broken into two separate sources.

The first source is the *true sharing misses* that arise from the communication of data through the cache coherence mechanism. In an invalidation-based protocol, the first write by a processor to a shared cache block causes an invalidation to establish ownership of that block. Additionally, when another processor attempts to read a modified word in that cache block, a miss occurs and the resultant block is transferred. Both these misses are classified as true sharing misses because they directly arise from the sharing of data among processors.

The second effect, called *false sharing*, arises from the use of an invalidation-based coherence algorithm with a single valid bit per cache block. False sharing occurs when a block is invalidated (and a subsequent reference causes a miss) because some word in the block, other than the one being read, is written into. If the word written into is actually used by the processor that received the invalidate, then the reference was a true sharing reference and would have caused a miss independent of the block size. If, however, the word being written and the word read are different and the invalidation does not cause a new value to be communicated, but only causes an extra cache miss, then it is a false sharing miss. In a false sharing miss, the block is shared, but no word in the cache is actually shared, and the miss would not occur if the block size were a single word. The following example makes the sharing patterns clear.

Example Assume that words z1 and z2 are in the same cache block, which is in the shared state in the caches of both P1 and P2. Assuming the following sequence of events, identify each miss as a true sharing miss, a false sharing miss, or a hit. Any miss that would occur if the block size were one word is designated a true sharing miss.

Time	P1	P2
1	Write z1	
2		Read z2
3	Write z1	
4		Write z2
5	Read z2	

Answer Here are the classifications by time step:

1. This event is a true sharing miss, since z1 is in the shared state in P2 and needs to be invalidated from P2.
2. This event is a false sharing miss, since z2 was invalidated by the write of z1 in P1, but that value of z1 is not used in P2.
3. This event is a false sharing miss, since the block containing z1 is marked shared due to the read in P2, but P2 did not read z1. The cache block containing z1 will be in the shared state after the read by P2; a write miss is required to obtain exclusive access to the block. In some protocols, this will be handled as an *upgrade request*, which generates a bus invalidate, but does not transfer the cache block.
4. This event is a false sharing miss for the same reason as step 3.
5. This event is a true sharing miss, since the value being read was written by P2.

Although we will see the effects of true and false sharing misses in commercial workloads, the role of coherence misses is more significant for tightly coupled applications that share significant amounts of user data. We examine their effects in detail in Appendix I when we consider the performance of a parallel scientific workload.

A Commercial Workload

In this section, we examine the memory system behavior of a 4-processor shared-memory multiprocessor when running an online transaction processing workload. The study we examine was done with a 4-processor Alpha system in 1998, but it remains the most comprehensive and insightful study of the performance of a multiprocessor for such workloads. We will focus on understanding the multiprocessor cache activity, and particularly the behavior in L3, where much of the traffic is coherence-related.

Cache level	Characteristic	Alpha 21164	Intel i7
L1	Size	8 KB I/8 KB D	32 KB I/32 KB D
	Associativity	Direct-mapped	8-way I/8-way D
	Block size	32 B	64 B
	Miss penalty	7	10
L2	Size	96 KB	256 KB
	Associativity	3-way	8-way
	Block size	32 B	64 B
	Miss penalty	21	35
L3	Size	2 MiB (total 8 MiB unshared)	2 MiB per core (8 MiB total shared)
	Associativity	Direct-mapped	16-way
	Block size	64 B	64 B
	Miss penalty	80	∼100

Figure 5.9 The characteristics of the cache hierarchy of the Alpha 21164 used in this study and the Intel i7. Although the sizes are larger and the associativity is higher on the i7, the miss penalties are also higher, so the behavior may differ only slightly. Both systems have a high penalty (125 cycles or more) for a transfer required from a private cache. A key difference is that L3 is shared in the i7 versus four separate, unshared caches in the Alpha server.

The results were collected either on an AlphaServer 4100 or using a configurable simulator modeled after the AlphaServer 4100. Each processor in the AlphaServer 4100 is an Alpha 21164, which issues up to four instructions per clock and runs at 300 MHz. Although the clock rate of the Alpha processor in this system is considerably slower than processors in systems designed in 2017, the basic structure of the system, consisting of a four-issue processor and a three-level cache hierarchy, is very similar to the multicore Intel i7 and other processors, as shown in Figure 5.9. Rather than focus on the performance details, we consider data that looks at the simulated L3 behavior for L3 caches varying from 2 to 8 MiB per processor.

Although the original study considered three different workloads, we focus our attention on the online transaction-processing (OLTP) workload modeled after TPC-B (which has memory behavior similar to its newer cousin TPC-C, described in Chapter 1) and using Oracle 7.3.2 as the underlying database. The workload consists of a set of client processes that generate requests and a set of servers that handle them. The server processes consume 85% of the user time, with the remaining going to the clients. Although the I/O latency is hidden by careful tuning and enough requests to keep the processor busy, the server processes typically block for I/O after about 25,000 instructions. Overall, 71% of the execution time is spent in user mode, 18% in the operating system, and 11% idle, primarily waiting for I/O. Of the commercial applications studied, the OLTP application stresses the memory system the hardest and shows significant challenges even when evaluated with much larger L3 caches. For example, on the AlphaServer, the processors are stalled

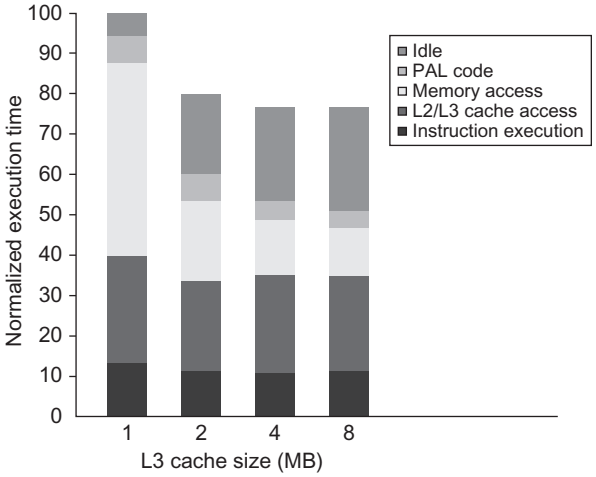


Figure 5.10 The relative performance of the OLTP workload as the size of the L3 cache, which is set as two-way set associative, grows from 1 to 8 MiB. The idle time also grows as cache size is increased, reducing some of the performance gains. This growth occurs because, with fewer memory system stalls, more server processes are needed to cover the I/O latency. The workload could be retuned to increase the computation/communication balance, holding the idle time in check. The PAL code is a set of sequences of specialized OS-level instructions executed in privileged mode; an example is the TLB miss handler.

for approximately 90% of the cycles with memory accesses occupying almost half the stall time and L2 misses 25% of the stall time.

We start by examining the effect of varying the size of the L3 cache. In these studies, the L3 cache is varied from 1 to 8 MiB per processor; at 2 MiB per processor, the total size of L3 is equal to that of the Intel i7 6700. In the case of the i7, however, the cache is shared, which provides both some advantages and disadvantages. It is unlikely that the shared 8 MiB cache will outperform separate L3 caches with a total size of 16 MiB. Figure 5.10 shows the effect of increasing the cache size, using two-way set associative caches, which reduces the large number of conflict misses. The execution time is improved as the L3 cache grows because of the reduction in L3 misses. Surprisingly, almost all of the gain occurs in going from 1 to 2 MiB (or 4 to 8 MiB of total cache for the four processors). There is little additional gain beyond that, despite the fact that cache misses are still a cause of significant performance loss with 2 MiB and 4 MiB caches. The question is, Why?

To better understand the answer to this question, we need to determine what factors contribute to the L3 miss rate and how they change as the L3 cache grows. Figure 5.11 shows these data, displaying the number of memory access cycles contributed per instruction from five sources. The two largest sources of L3 memory access cycles with a 1 MiB L3 are instruction and capacity/conflict misses. With a larger L3, these two sources shrink to be minor contributors. Unfortunately, the

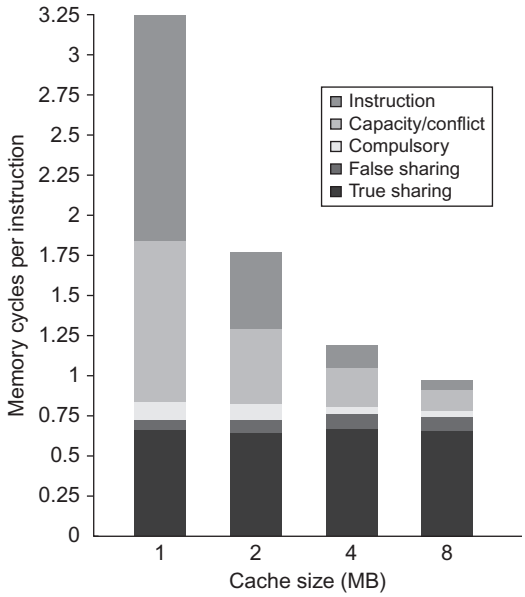


Figure 5.11 The contributing causes of memory access cycle shift as the cache size is increased. The L3 cache is simulated as two-way set associative.

compulsory, false sharing, and true sharing misses are unaffected by a larger L3. Thus, at 4 and 8 MiB, the true sharing misses generate the dominant fraction of the misses; the lack of change in true sharing misses leads to the limited reductions in the overall miss rate when increasing the L3 cache size beyond 2 MiB.

Increasing the cache size eliminates most of the uniprocessor misses while leaving the multiprocessor misses untouched. How does increasing the processor count affect different types of misses? Figure 5.12 shows these data assuming a base configuration with a 2 MiB, two-way set associative L3 cache (the same effective per processor cache size as the i7 but with less associativity). As we might expect, the increase in the true sharing miss rate, which is not compensated for by any decrease in the uniprocessor misses, leads to an overall increase in the memory access cycles per instruction.

The final question we examine is whether increasing the block size—which should decrease the instruction and cold miss rate and, within limits, also reduce the capacity/conflict miss rate and possibly the true sharing miss rate—is helpful for this workload. Figure 5.13 shows the number of misses per 1000 instructions as the block size is increased from 32 to 256 bytes. Increasing the block size from 32 to 256 bytes affects four of the miss rate components:

- The true sharing miss rate decreases by more than a factor of 2, indicating some locality in the true sharing patterns.

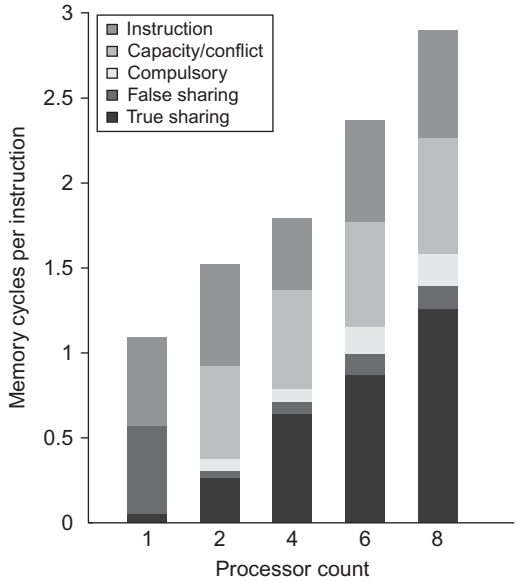


Figure 5.12 The contribution to memory access cycles increases as processor count increases primarily because of increased true sharing. The compulsory misses slightly increase because each processor must now handle more compulsory misses.

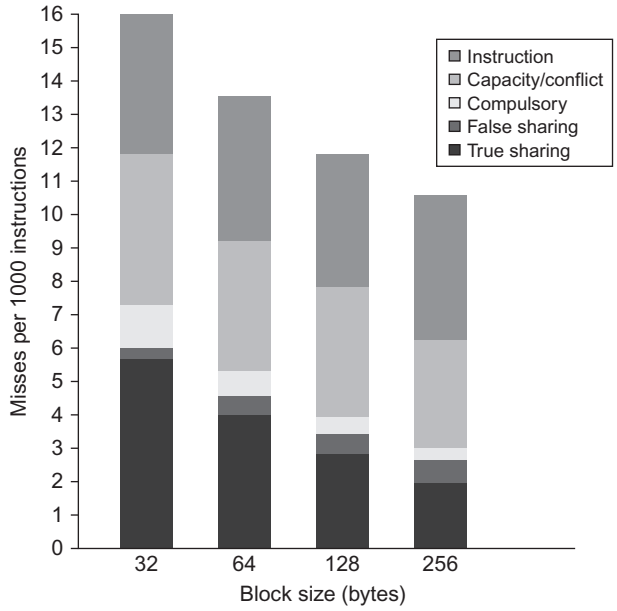


Figure 5.13 The number of misses per 1000 instructions drops steadily as the block size of the L3 cache is increased, making a good case for an L3 block size of at least 128 bytes. The L3 cache is 2 MiB, two-way set associative.

- The compulsory miss rate significantly decreases, as we would expect.
- The conflict/capacity misses show a small decrease (a factor of 1.26 compared to a factor of 8 increase in block size), indicating that the spatial locality is not high in the uniprocessor misses that occur with L3 caches larger than 2 MiB.
- The false sharing miss rate, although small in absolute terms, nearly doubles.

The lack of a significant effect on the instruction miss rate is startling. If there were an instruction-only cache with this behavior, we would conclude that the spatial locality is very poor. In the case of mixed L2 and L3 caches, other effects such as instruction-data conflicts may also contribute to the high instruction cache miss rate for larger blocks. Other studies have documented the low spatial locality in the instruction stream of large database and OLTP workloads, which have lots of short basic blocks and special-purpose code sequences. Based on these data, the miss penalty for a larger block size L3 to perform as well as the 32-byte block size L3 can be expressed as a multiplier on the 32-byte block size penalty.

Block size	Miss penalty relative to 32-byte block miss penalty
64 bytes	1.19
128 bytes	1.36
256 bytes	1.52

With modern DDR SDRAMs that make block access fast, these numbers are attainable, especially at the 64 byte (the i7 block size) and the 128 byte block size. Of course, we must also worry about the effects of the increased traffic to memory and possible contention for the memory with other cores. This latter effect may easily negate the gains obtained from improving the performance of a single processor.

A Multiprogramming and OS Workload

Our next study is a multiprogrammed workload consisting of both user activity and OS activity. The workload used is two independent copies of the compile phases of the Andrew benchmark, a benchmark that emulates a software development environment. The compile phase consists of a parallel version of the UNIX "make" command executed using eight processors. The workload runs for 5.24 seconds on eight processors, creating 203 processes and performing 787 disk requests on three different file systems. The workload is run with 128 MiB of memory, and no paging activity takes place.

The workload has three distinct phases: compiling the benchmarks, which involves substantial compute activity; installing the object files in a library; and removing the object files. The last phase is completely dominated by I/O, and only two processes are active (one for each of the runs). In the middle phase, I/O also

plays a major role, and the processor is largely idle. The overall workload is much more system- and I/O-intensive than the OLTP workload.

For the workload measurements, we assume the following memory and I/O systems:

- *Level 1 instruction cache*—32 KB, two-way set associative with a 64-byte block, 1 clock cycle hit time.
- *Level 1 data cache*—32 KB, two-way set associative with a 32-byte block, 1 clock cycle hit time. Our focus is on examining the behavior in the Level 1 data cache, in contrast to the OLTP study, which focused on the L3 cache.
- *Level 2 cache*—1 MiB unified, two-way set associative with a 128-byte block, 10 clock cycle hit time.
- *Main memory*—Single memory on a bus with an access time of 100 clock cycles.
- *Disk system*—Fixed-access latency of 3 ms (less than normal to reduce idle time).

Figure 5.14 shows how the execution time breaks down for the eight processors using the parameters just listed. Execution time is broken down into four components:

1. *Idle*—Execution in the kernel mode idle loop
2. *User*—Execution in user code
3. *Synchronization*—Execution or waiting for synchronization variables
4. *Kernel*—Execution in the OS that is neither idle nor in synchronization access

This multiprogramming workload has a significant instruction cache performance loss, at least for the OS. The instruction cache miss rate in the OS for a 64-byte block size, two-way set associative cache varies from 1.7% for a 32 KB

	User execution	Kernel execution	Synchronization wait	Processor idle (waiting for I/O)
Instructions executed	27%	3%	1%	69%
Execution time	27%	7%	2%	64%

Figure 5.14 The distribution of execution time in the multiprogrammed parallel "make" workload. The high fraction of idle time is due to disk latency when only one of the eight processors is active. These data and the subsequent measurements for this workload were collected with the SimOS system (Rosenblum et al., 1995). The actual runs and data collection were done by M. Rosenblum, S. Herrod, and E. Bugnion of Stanford University.

cache to 0.2% for a 256 KB cache. User-level instruction cache misses are roughly one-sixth of the OS rate, across the variety of cache sizes. This partially accounts for the fact that, although the user code executes nine times as many instructions as the kernel, those instructions take only about four times as long as the smaller number of instructions executed by the kernel.

Performance of the Multiprogramming and OS Workload

In this section, we examine the cache performance of the multiprogrammed workload as the cache size and block size are changed. Because of differences between the behavior of the kernel and that of the user processes, we keep these two components separate. Remember, though, that the user processes execute more than eight times as many instructions, so the overall miss rate is determined primarily by the miss rate in user code, which as we will see, is often one-fifth of the kernel miss rate.

Although the user code executes more instructions, the behavior of the operating system can cause more cache misses than the user processes for two reasons beyond larger code size and lack of locality. First, the kernel initializes all pages before allocating them to a user, which significantly increases the compulsory component of the kernel's miss rate. Second, the kernel actually shares data and thus has a nontrivial coherence miss rate. In contrast, user processes cause coherence misses only when the process is scheduled on a different processor, and this component of the miss rate is small. This is a major difference between a multiprogrammed workload and one like the OLTP workload.

Figure 5.15 shows the data miss rate versus data cache size and versus block size for the kernel and user components. Increasing the data cache size affects the user miss rate more than it affects the kernel miss rate. Increasing the block size has beneficial effects for both miss rates because a larger fraction of the misses arise from compulsory and capacity, both of which can be potentially improved with larger block sizes. Because coherence misses are relatively rarer, the negative effects of increasing block size are small. To understand why the kernel and user processes behave differently, we can look at how the kernel misses behave.

Figure 5.16 shows the variation in the kernel misses versus increases in cache size and in block size. The misses are broken into three classes: compulsory misses, coherence misses (from both true and false sharing), and capacity/conflict misses (which include misses caused by interference between the OS and the user process and between multiple user processes). Figure 5.16 confirms that, for the kernel references, increasing the cache size reduces only the uniprocessor capacity/conflict miss rate. In contrast, increasing the block size causes a reduction in the compulsory miss rate. The absence of large increases in the coherence miss rate as block size is increased means that false sharing effects are probably insignificant, although such misses may be offsetting some of the gains from reducing the true sharing misses.

If we examine the number of bytes needed per data reference, as in Figure 5.17, we see that the kernel has a higher traffic ratio that grows with block size. It is easy to see why this occurs: when going from a 16-byte block to a 128-byte block, the

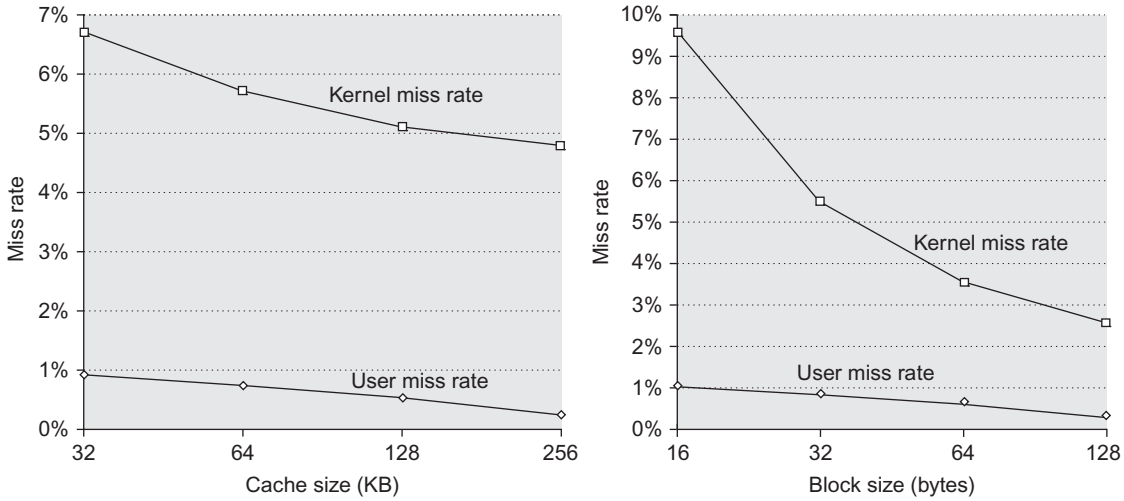


Figure 5.15 The data miss rates for the user and kernel components behave differently for increases in the L1 data cache size (on the left) versus increases in the L1 data cache block size (on the right). Increasing the L1 data cache from 32 to 256 KB (with a 32-byte block) causes the user miss rate to decrease proportionately more than the kernel miss rate: the user-level miss rate drops by almost a factor of 3, whereas the kernel-level miss rate drops by a factor of only 1.3. At the largest size, the L1 is closer to the size of L2 in a modern multicore processors. Thus the data indicates that the kernel miss rate will still be significant in an L2 cache. The miss rate for both user and kernel components drops steadily as the L1 block size is increased (while keeping the L1 cache at 32 KB). In contrast to the effects of increasing the cache size, increasing the block size improves the kernel miss rate more significantly (just under a factor of 4 for the kernel references when going from 16-byte to 128-byte blocks versus just under a factor of 3 for the user references).

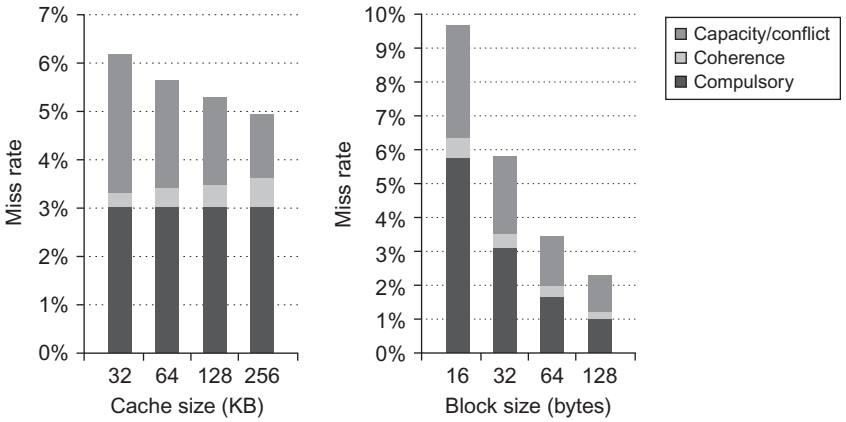


Figure 5.16 The components of the kernel data miss rate change as the L1 data cache size is increased from 32 to 256 KB, when the multiprogramming workload is run on eight processors. The compulsory miss rate component stays constant because it is unaffected by cache size. The capacity component drops by more than a factor of 2, whereas the coherence component nearly doubles. The increase in coherence misses occurs because the probability of a miss being caused by an invalidation increases with cache size, since fewer entries are bumped due to capacity. As we would expect, the increasing block size of the L1 data cache substantially reduces the compulsory miss rate in the kernel references. It also has a significant impact on the capacity miss rate, decreasing it by a factor of 2.4 over the range of block sizes. The increased block size has a small reduction in coherence traffic, which appears to stabilize at 64 bytes, with no change in the coherence miss rate in going to 128-byte lines. Because there are no significant reductions in the coherence miss rate as the block size increases, the fraction of the miss rate caused by coherence grows from about 7% to about 15%.

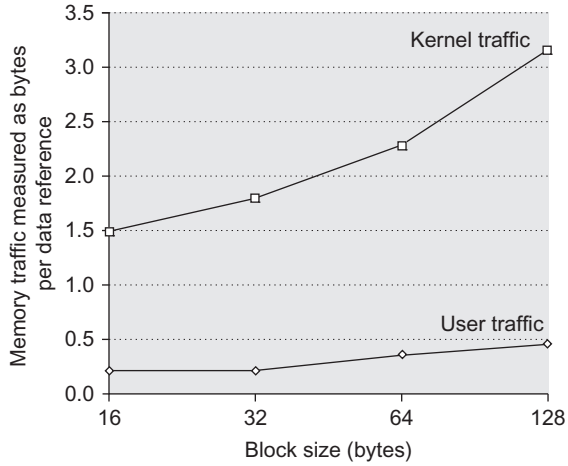


Figure 5.17 The number of bytes needed per data reference grows as block size is increased for both the kernel and user components. It is interesting to compare this chart with the data on scientific programs shown in Appendix I.

miss rate drops by about 3.7, but the number of bytes transferred per miss increases by 8, so the total miss traffic increases by just over a factor of 2. The user program also more than doubles as the block size goes from 16 to 128 bytes, but it starts out at a much lower level.

For the multiprogrammed workload, the OS is a much more demanding user of the memory system. If more OS or OS-like activity is included in the workload, and the behavior is similar to what was measured for this workload, it will become very difficult to build a sufficiently capable memory system. One possible route to improving performance is to make the OS more cache-aware through either better programming environments or through programmer assistance. For example, the OS reuses memory for requests that arise from different system calls. Despite the fact that the reused memory will be completely overwritten, the hardware, not recognizing this, will attempt to preserve coherency and the possibility that some portion of a cache block may be read, even if it is not. This behavior is analogous to the reuse of stack locations on procedure invocations. The IBM Power series has support to allow the compiler to indicate this type of behavior on procedure invocations, and the newest AMD processors have similar support. It is harder to detect such behavior by the OS, and doing so may require programmer assistance, but the payoff is potentially even greater.

OS and commercial workloads pose tough challenges for multiprocessor memory systems, and unlike scientific applications, which we examine in Appendix I, they are less amenable to algorithmic or compiler restructuring. As the number of cores increases, predicting the behavior of such applications is likely to get more difficult. Emulation or simulation methodologies that allow the simulation of tens

to hundreds of cores with large applications (including operating systems) will be crucial to maintaining an analytical and quantitative approach to design.

5.4 Distributed Shared-Memory and Directory-Based Coherence

As we saw in Section 5.2, a snooping protocol requires communication with all caches on every cache miss, including writes of potentially shared data. The absence of any centralized data structure that tracks the state of the caches is both the fundamental advantage of a snooping-based scheme, since it allows it to be inexpensive, as well as its Achilles' heel when it comes to scalability.

For example, consider a multiprocessor consisting of four 4-core multicores capable of sustaining one data reference per clock and a 4 GHz clock. From the data in Section I.5 in Appendix I, we can see that the applications may require 4–170 GiB/s of memory bus bandwidth. The maximum memory bandwidth supported by the i7 with two DDR4 memory channels is 34 GiB/s. If several i7 multicore processors shared the same memory system, they would easily swamp it. In the last few years, the development of multicore processors forced all designers to shift to some form of distributed memory to support the bandwidth demands of the individual processors.

We can increase the memory bandwidth and interconnection bandwidth by distributing the memory, as shown in Figure 5.2 on page 373; this immediately separates local memory traffic from remote memory traffic, reducing the bandwidth demands on the memory system and on the interconnection network. Unless we eliminate the need for the coherence protocol to broadcast on every cache miss, distributing the memory will gain us little.

As we mentioned earlier, the alternative to a snooping-based coherence protocol is a *directory protocol*. A directory keeps the state of every block that may be cached. Information in the directory includes which caches (or collections of caches) have copies of the block, whether it is dirty, and so on. Within a multicore with a shared outermost cache (say, L3), it is easy to implement a directory scheme: simply keep a bit vector of the size equal to the number of cores for each L3 block. The bit vector indicates which private L2 caches may have copies of a block in L3, and invalidations are sent only to those caches. This works perfectly for a single multicore if L3 is inclusive, and this scheme is the one used in the Intel i7.

The solution of a single directory used in a multicore is not scalable, even though it avoids broadcast. The directory must be distributed, but the distribution must be done in a way that the coherence protocol knows where to find the directory information for any cached block of memory. The obvious solution is to distribute the directory along with the memory so that different coherence requests can go to different directories, just as different memory requests go to different memories. If the information is maintained at an outer cache, like L3, which is multibanked, the directory information can be distributed with the different cache banks, effectively increasing the bandwidth.

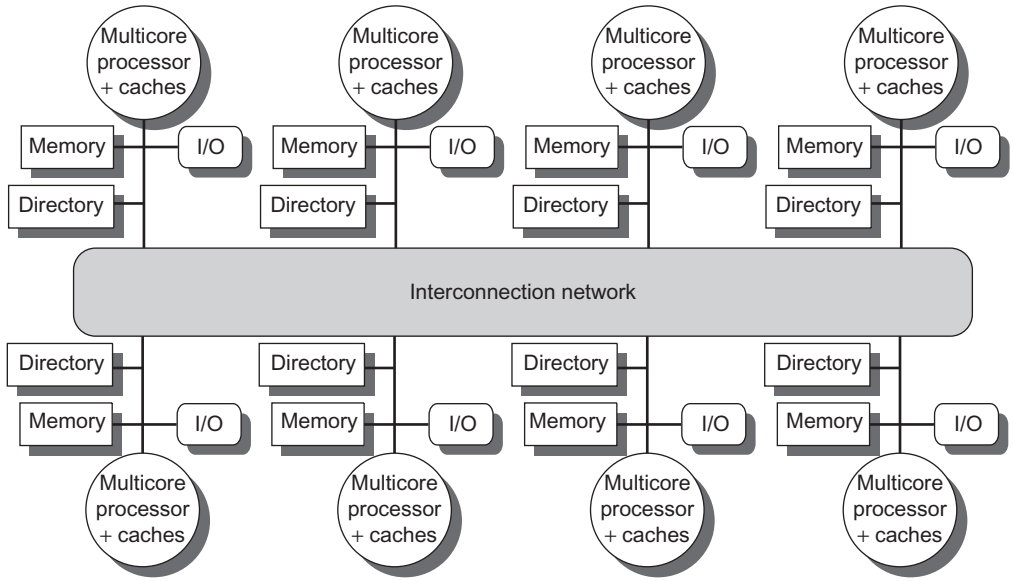


Figure 5.18 A directory is added to each node to implement cache coherence in a distributed-memory multiprocessor. In this case, a node is shown as a single multicore chip, and the directory information for the associated memory may reside either on or off the multicore. Each directory is responsible for tracking the caches that share the memory addresses of the portion of memory in the node. The coherence mechanism will handle both the maintenance of the directory information and any coherence actions needed within the multicore node.

A distributed directory retains the characteristic that the sharing status of a block is always in a single known location. This property, together with the maintenance of information that says what other nodes may be caching the block, is what allows the coherence protocol to avoid broadcast. Figure 5.18 shows how our distributed-memory multiprocessor looks with the directories added to each node.

The simplest directory implementations associate an entry in the directory with each memory block. In such implementations, the amount of information is proportional to the product of the number of memory blocks (where each block is the same size as the L2 or L3 cache block) times the number of nodes, where a node is a single multicore processor or a small collection of processors that implements coherence internally. This overhead is not a problem for multiprocessors with less than a few hundred processors (each of which might be a multicore) because the directory overhead with a reasonable block size will be tolerable. For larger multiprocessors, we need methods to allow the directory structure to be efficiently scaled, but only supercomputer-sized systems need to worry about this.

Directory-Based Cache Coherence Protocols: The Basics

Just as with a snooping protocol, there are two primary operations that a directory protocol must implement: handling a read miss and handling a write to a shared, clean cache block. (Handling a write miss to a block that is currently shared is a simple combination of these two.) To implement these operations, a directory must track the state of each cache block. In a simple protocol, these states could be the following:

- *Shared*—One or more nodes have the block cached, and the value in memory is up to date (as well as in all the caches).
- *Uncached*—No node has a copy of the cache block.
- *Modified*—Exactly one node has a copy of the cache block, and it has written the block, so the memory copy is out of date. The processor is called the *owner* of the block.

In addition to tracking the state of each potentially shared memory block, we must track which nodes have copies of that block because those copies will need to be invalidated on a write. The simplest way to do this is to keep a bit vector for each memory block. When the block is shared, each bit of the vector indicates whether the corresponding processor chip (which is likely a multicore) has a copy of that block. We can also use the bit vector to keep track of the owner of the block when the block is in the exclusive state. For efficiency reasons, we also track the state of each cache block at the individual caches.

The states and transitions for the state machine at each cache are identical to what we used for the snooping cache, although the actions on a transition are slightly different. The processes of invalidating and locating an exclusive copy of a data item are different because they both involve communication between the requesting node and the directory and between the directory and one or more remote nodes. In a snooping protocol, these two steps are combined through the use of a broadcast to all the nodes.

Before we see the protocol state diagrams, it is useful to examine a catalog of the message types that may be sent between the processors and the directories for the purpose of handling misses and maintaining coherence. Figure 5.19 shows the types of messages sent among nodes. The *local node* is the node where a request originates. The *home node* is the node where the memory location and the directory entry of an address reside. The physical address space is statically distributed, so the node that contains the memory and directory for a given physical address is known. For example, the high-order bits may provide the node number, whereas the low-order bits provide the offset within the memory on that node. The local node may also be the home node. The directory must be accessed when the home node is the local node because copies may exist in yet a third node, called a *remote node*.

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/ invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write back a data value for address A.

Figure 5.19 The possible messages sent among nodes to maintain coherence, along with the source and destination node, the contents (where P = requesting node number, A = requested address, and D = data contents), and the function of the message. The first three messages are requests sent by the local node to the home. The fourth through sixth messages are messages sent to a remote node by the home when the home needs the data to satisfy a read or write miss request. Data value replies are used to send a value from the home node back to the requesting node. Data value write-backs occur for two reasons: when a block is replaced in a cache and must be written back to its home memory, and also in reply to fetch or fetch/invalidate messages from the home. Writing back the data value whenever the block becomes shared simplifies the number of states in the protocol because any dirty block must be exclusive and any shared block is always available in the home memory.

A remote node is the node that has a copy of a cache block, whether exclusive (in which case it is the only copy) or shared. A remote node may be the same as either the local node or the home node. In such cases, the basic protocol does not change, but interprocessor messages may be replaced with intraprocessor messages.

In this section, we assume a simple model of memory consistency. To minimize the type of messages and the complexity of the protocol, we make an assumption that messages will be received and acted upon in the same order they are sent. This assumption may not be true in practice and can result in additional complications, some of which we address in Section 5.6 when we discuss memory consistency models. In this section, we use this assumption to ensure that invalidates sent by a node are honored before new messages are transmitted, just as we assumed in the discussion of implementing snooping protocols. As we did in the snooping case, we omit some details necessary to implement the coherence

protocol. In particular, the serialization of writes and knowing that the invalidates for a write have completed are not as simple as in the broadcast-based snooping mechanism. Instead, explicit acknowledgments are required in response to write misses and invalidate requests. We discuss these issues in more detail in Appendix I.

An Example Directory Protocol

The basic states of a cache block in a directory-based protocol are exactly like those in a snooping protocol, and the states in the directory are also analogous to those we showed earlier. Thus we can start with simple state diagrams that show the state transitions for an individual cache block and then examine the state diagram for the directory entry corresponding to each block in memory. As in the snooping case, these state transition diagrams do not represent all the details of a coherence protocol; however, the actual controller is highly dependent on a number of details of the multiprocessor (message delivery properties, buffering structures, and so on). In this section, we present the basic protocol state diagrams. The knotty issues involved in implementing these state transition diagrams are examined in Appendix I.

Figure 5.20 shows the protocol actions to which an individual cache responds. We use the same notation as in the last section, with requests coming from outside the node in gray and actions in bold. The state transitions for an individual cache are caused by read misses, write misses, invalidates, and data fetch requests; Figure 5.20 shows these operations. An individual cache also generates read miss, write miss, and invalidate messages that are sent to the home directory. Read and write misses require data value replies, and these events wait for replies before changing state. Knowing when invalidates complete is a separate problem and is handled separately.

The operation of the state transition diagram for a cache block in Figure 5.20 is essentially the same as it is for the snooping case: the states are identical, and the stimulus is almost identical. The write miss operation, which was broadcast on the bus (or other network) in the snooping scheme, is replaced by the data fetch and invalidate operations that are selectively sent by the directory controller. Like the snooping protocol, any cache block must be in the exclusive state when it is written, and any shared block must be up to date in memory. In many multicore processors, the outermost level in the processor cache is shared among the cores (as is the L3 in the Intel i7, the AMD Opteron, and the IBM Power7), and hardware at that level maintains coherence among the private caches of each core on the same chip, using either an internal directory or snooping. Thus the on-chip multicore coherence mechanism can be used to extend coherence among a larger set of processors simply by interfacing to the outermost shared cache. Because this interface is at L3, contention between the processor and coherence requests is less of an issue, and duplicating the tags could be avoided.

In a directory-based protocol, the directory implements the other half of the coherence protocol. A message sent to a directory causes two different types of

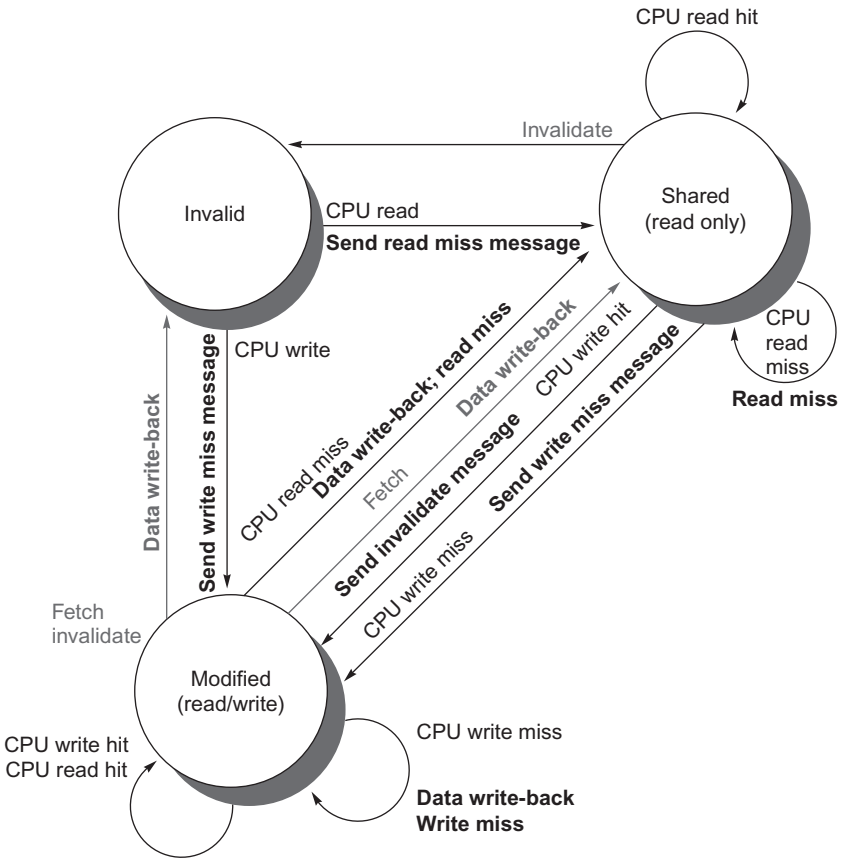


Figure 5.20 State transition diagram for an individual cache block in a directory-based system. Requests by the local processor are shown in *black*, and those from the home directory are shown in *gray*. The states are identical to those in the snooping case, and the transactions are very similar, with explicit invalidate and write-back requests replacing the write misses that were formerly broadcast on the bus. As we did for the snooping controller, we assume that an attempt to write a shared cache block is treated as a miss; in practice, such a transaction can be treated as an ownership request or upgrade request and can deliver ownership without requiring that the cache block be fetched.

actions: updating the directory state and sending additional messages to satisfy the request. The states in the directory represent the three standard states for a block; unlike in a snooping scheme, however, the directory state indicates the state of all the cached copies of a memory block, rather than for a single cache block.

The memory block may be uncached by any node, cached in multiple nodes and readable (shared), or cached exclusively and writable in exactly one node. In addition to the state of each block, the directory must track the set of nodes that have a copy of a block; we use a set called *Sharers* to perform this function. In multiprocessors with fewer than 64 nodes (each of which may represent four to eight times as many processors), this set is typically kept as a bit vector. Directory

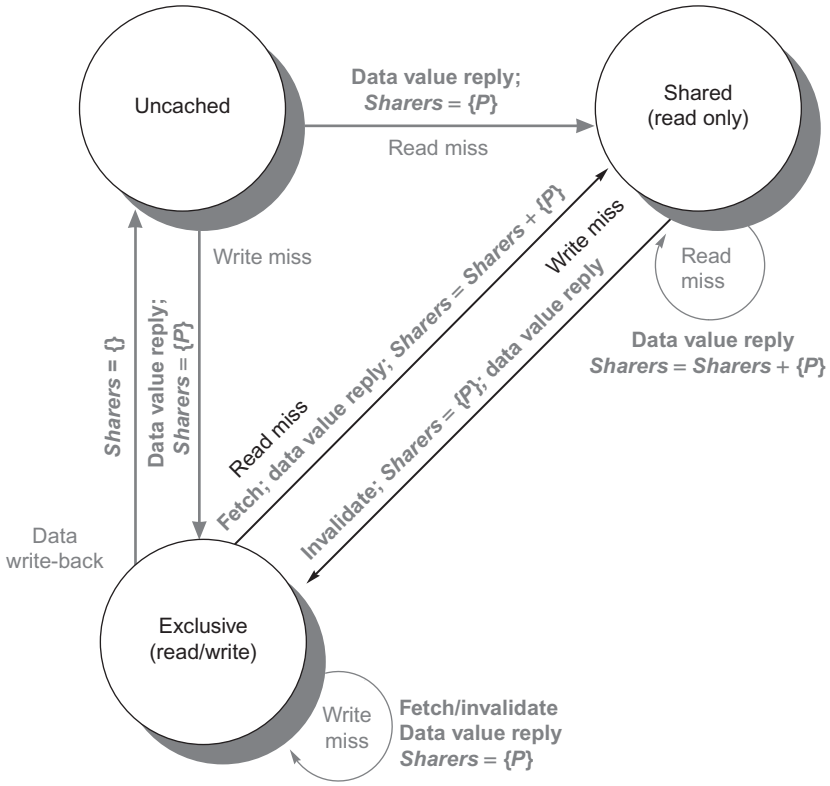


Figure 5.21 The state transition diagram for the directory has the same states and structure as the transition diagram for an individual cache. All actions are in gray because they are all externally caused. *Bold* indicates the action taken by the directory in response to the request.

requests need to update the set Sharers and also read the set to perform invalidations.

Figure 5.21 shows the actions taken at the directory in response to messages received. The directory receives three different requests: read miss, write miss, and data write-back. The messages sent in response by the directory are shown in bold, and the updating of the set Sharers is shown in bold italics. Because all the stimulus messages are external, all actions are shown in gray. Our simplified protocol assumes that some actions are atomic, such as requesting a value and sending it to another node; a realistic implementation cannot use this assumption.

To understand these directory operations, let's examine the requests received and actions taken state by state. When a block is in the uncached state, the copy in memory is the current value, so the only possible requests for that block are

- *Read miss*—The requesting node is sent the requested data from memory, and the requester is made the only sharing node. The state of the block is made shared.

- *Write miss*—The requesting node is sent the value and becomes the sharing node. The block is made exclusive to indicate that the only valid copy is cached. Sharers indicates the identity of the owner.

When the block is in the shared state, the memory value is up to date, so the same two requests can occur:

- *Read miss*—The requesting node is sent the requested data from memory, and the requesting node is added to the sharing set.
- *Write miss*—The requesting node is sent the value. All nodes in the set Sharers are sent invalidate messages, and the Sharers set is to contain the identity of the requesting node. The state of the block is made exclusive.

When the block is in the exclusive state, the current value of the block is held in a cache on the node identified by the set Sharers (the owner), so there are three possible directory requests:

- *Read miss*—The owner is sent a data fetch message, which causes the state of the block in the owner's cache to transition to shared and causes the owner to send the data to the directory, where it is written to memory and sent back to the requesting processor. The identity of the requesting node is added to the set Sharers, which still contains the identity of the processor that was the owner (since it still has a readable copy).
- *Data write-back*—The owner is replacing the block and therefore must write it back. This write-back makes the memory copy up to date (the home directory essentially becomes the owner), the block is now uncached, and the Sharers set is empty.
- *Write miss*—The block has a new owner. A message is sent to the old owner, causing the cache to invalidate the block and send the value to the directory, from which it is sent to the requesting node, which becomes the new owner. Sharers is set to the identity of the new owner, and the state of the block remains exclusive.

This state transition diagram in Figure 5.21 is a simplification, just as it was in the snooping cache case. In the case of a directory, as well as a snooping scheme implemented with a network other than a bus, our protocols will need to deal with nonatomic memory transactions. Appendix I explores these issues in depth.

The directory protocols used in real multiprocessors contain additional optimizations. In particular, in this protocol when a read or write miss occurs for a block that is exclusive, the block is first sent to the directory at the home node. From there it is stored into the home memory and also sent to the original requesting node. Many of the protocols in use in commercial multiprocessors forward the data from the owner node to the requesting node directly (as well as performing the write-back to the home). Such optimizations often add complexity by increasing the possibility of deadlock and by increasing the types of messages that must be handled.

Implementing a directory scheme requires solving most of the same challenges we discussed for snooping protocols. There are, however, new and additional problems, which we describe in Appendix I. In Section 5.8, we briefly describe how modern multicores extend coherence beyond a single chip. The combinations of multichip coherence and multicore coherence include all four possibilities of snooping/snooping (AMD Opteron), snooping/directory, directory/snooping, and directory/directory! Many multiprocessors have chosen some form of snooping within a single chip, which is attractive if the outermost cache is shared and inclusive, and directories across multiple chips. Such an approach simplifies implementation because only the processor chip, rather than an individual core, need be tracked.

5.5 Synchronization: The Basics

Synchronization mechanisms are typically built with user-level software routines that rely on hardware-supplied synchronization instructions. For smaller multiprocessors or low-contention situations, the key hardware capability is an uninterruptible instruction or instruction sequence capable of atomically retrieving and changing a value. Software synchronization mechanisms are then constructed using this capability. In this section, we focus on the implementation of lock and unlock synchronization operations. Lock and unlock can be used straightforwardly to create mutual exclusion, as well as to implement more complex synchronization mechanisms.

In high-contention situations, synchronization can become a performance bottleneck because contention introduces additional delays and because latency is potentially greater in such a multiprocessor. We discuss how the basic synchronization mechanisms of this section can be extended for large processor counts in Appendix I.

Basic Hardware Primitives

The key ability we require to implement synchronization in a multiprocessor is a set of hardware primitives with the ability to atomically read and modify a memory location. Without such a capability, the cost of building basic synchronization primitives will be too high and will increase as the processor count increases. There are a number of alternative formulations of the basic hardware primitives, all of which provide the ability to atomically read and modify a location, together with some way to tell whether the read and write were performed atomically. These hardware primitives are the basic building blocks that are used to build a wide variety of user-level synchronization operations, including things such as locks and barriers. In general, architects do not expect users to employ the basic hardware primitives, but instead expect that the primitives will be used by system programmers to build a synchronization library, a process that is often complex and tricky. Let's start with one such hardware primitive and show how it can be used to build some basic synchronization operations.

One typical operation for building synchronization operations is the *atomic exchange*, which interchanges a value in a register for a value in memory. To see how to use this to build a basic synchronization operation, assume that we want to build a simple lock where the value 0 is used to indicate that the lock is free and 1 is used to indicate that the lock is unavailable. A processor tries to set the lock by doing an exchange of 1, which is in a register, with the memory address corresponding to the lock. The value returned from the exchange instruction is 1 if some other processor had otherwise already claimed access and 0. In the latter case, the value is also changed to 1, preventing any competing exchange from also retrieving a 0.

For example, consider two processors that each try to do the exchange simultaneously: This race is broken because exactly one of the processors will perform the exchange first, returning 0, and the second processor will return 1 when it does the exchange. The key to using the exchange (or swap) primitive to implement synchronization is that the operation is atomic: the exchange is indivisible, and two simultaneous exchanges will be ordered by the write serialization mechanisms. It is impossible for two processors trying to set the synchronization variable in this manner to both determine they have simultaneously set the variable.

There are a number of other atomic primitives that can be used to implement synchronization. They all have the key property that they read and update a memory value in such a manner that we can tell whether the two operations executed atomically. One operation, present in many older multiprocessors, is *test-and-set*, which tests a value and sets it if the value passes the test. For example, we could define an operation that tested for 0 and set the value to 1, which can be used in a fashion similar to how we used atomic exchange. Another atomic synchronization primitive is *fetch-and-increment*: it returns the value of a memory location and atomically increments it. By using the value 0 to indicate that the synchronization variable is unclaimed, we can use fetch-and-increment, just as we used exchange. There are other uses of operations like fetch-and-increment, which we will see shortly.

Implementing a single atomic memory operation introduces some challenges because it requires both a memory read and a write in a single, uninterruptible instruction. This requirement complicates the implementation of coherence because the hardware cannot allow any other operations between the read and the write, and yet must not deadlock.

An alternative is to have a pair of instructions where the second instruction returns a value from which it can be deduced whether the pair of instructions was executed as though the instructions were atomic. The pair of instructions is effectively atomic if it appears as though all other operations executed by any processor occurred before or after the pair. Thus, when an instruction pair is effectively atomic, no other processor can change the value between the instruction pair. This is the approach used in the MIPS processors and in RISC V.

In RISC V, the pair of instructions includes a special load called a *load reserved* (also called load linked or *load locked)* and a special store called a *store conditional*. Load reserved loads the contents of memory given by rs1 into rd and creates a reservation on that memory address. Store conditional stores the value in rs2 into the memory address given by rs1. If the reservation of the load is broken by

a write to the same memory location, the store conditional fails and writes a nonzero to rd; if it succeeds, the store conditional writes 0. If the processor does a context switch between the two instructions, then the store conditional always fails.

These instructions are used in sequence, and because the load reserved returns the initial value and the store conditional returns 0 only if it succeeds, the following sequence implements an atomic exchange on the memory location specified by the contents of x1 with the value in x4:

```
try:    mov   x3,x4     ;mov exchange value
        lr    x2,x1     ;load reserved from
        sc    x3,0(x1)  ;store conditional
        bnez  x3,try    ;branch store fails
        mov   x4,x2     ;put load value in x4
```

At the end of this sequence, the contents of `x4` and the memory location specified by `x1` have been atomically exchanged. Anytime a processor intervenes and modifies the value in memory between the `lr` and `sc` instructions, the `sc` returns 0 in `x3`, causing the code sequence to try again.

An advantage of the load reserved/store conditional mechanism is that it can be used to build other synchronization primitives. For example, here is an atomic fetch-and-increment:

```
try:    lr    x2,x1     ;load reserved 0(x1)
        addi  x3,x2,1   ;increment
        sc    x3,0(x1)  ;store conditional
        bnez  x3,try    ;branch store fails
```

These instructions are typically implemented by keeping track of the address specified in the `lr` instruction in a register, often called the *reserved register*. If an interrupt occurs, or if the cache block matching the address in the link register is invalidated (e.g., by another `sc`), the link register is cleared. The `sc` instruction simply checks that its address matches that in the reserved register. If so, the `sc` succeeds; otherwise, it fails. Because the store conditional will fail after either another attempted store to the load reserved address or any exception, care must be taken in choosing what instructions are inserted between the two instructions. In particular, only register-register instructions can safely be permitted; otherwise, it is possible to create deadlock situations where the processor can never complete the `sc`. In addition, the number of instructions between the load reserved and the store conditional should be small to minimize the probability that either an unrelated event or a competing processor causes the store conditional to fail frequently.

Implementing Locks Using Coherence

Once we have an atomic operation, we can use the coherence mechanisms of a multiprocessor to implement *spin locks*—locks that a processor continuously tries to acquire, spinning around a loop until it succeeds. Spin locks are used when programmers expect the lock to be held for a very short amount of time and when they want the process of locking to be low latency when the lock is available. Because

spin locks tie up the processor waiting in a loop for the lock to become free, they are inappropriate in some circumstances.

The simplest implementation, which we would use if there were no cache coherence, would be to keep the lock variables in memory. A processor could continually try to acquire the lock using an atomic operation, say, atomic exchange, and test whether the exchange returned the lock as free. To release the lock, the processor simply stores the value 0 to the lock. Here is the code sequence to lock a spin lock whose address is in `x1`. It uses `EXCH` as a macro for the atomic exchange sequence from page 414:

```
          addi  x2,R0,#1
lockit:   EXCH  x2,0(x1)      ;atomic exchange
          bnez  x2,lockit     ;already locked?
```

If our multiprocessor supports cache coherence, we can cache the locks using the coherence mechanism to maintain the lock value coherently. Caching locks has two advantages. First, it allows an implementation where the process of "spinning" (trying to test and acquire the lock in a tight loop) could be done on a local cached copy rather than requiring a global memory access on each attempt to acquire the lock. The second advantage comes from the observation that there is often locality in lock accesses; that is, the processor that used the lock last will use it again in the near future. In such cases, the lock value may reside in the cache of that processor, greatly reducing the time to acquire the lock.

Obtaining the first advantage—being able to spin on a local cached copy rather than generating a memory request for each attempt to acquire the lock—requires a change in our simple spin procedure. Each attempt to exchange in the preceding loop requires a write operation. If multiple processors are attempting to get the lock, each will generate the write. Most of these writes will lead to write misses because each processor is trying to obtain the lock variable in an exclusive state.

Thus we should modify our spin lock procedure so that it spins by doing reads on a local copy of the lock until it successfully sees that the lock is available. Then it attempts to acquire the lock by doing a swap operation. A processor first reads the lock variable to test its state. A processor keeps reading and testing until the value of the read indicates that the lock is unlocked. The processor then races against all other processes that were similarly "spin waiting" to see which can lock the variable first. All processes use a swap instruction that reads the old value and stores a 1 into the lock variable. The single winner will see the 0, and the losers will see a 1 that was placed there by the winner. (The losers will continue to set the variable to the locked value, but that doesn't matter.) The winning processor executes the code after the lock and, when finished, stores a 0 into the lock variable to release the lock, which starts the race all over again. Here is the code to perform this spin lock (remember that 0 is unlocked and 1 is locked):

```
lockit:   ld    x2,0(x1)    ;load of lock
          bnez  x2,lockit   ;not available-spin
          addi  x2,R0,#1    ;load locked value
          EXCH  x2,0(x1)    ;swap
          bnez  x2,lockit   ;branch if lock wasn't 0
```

Step	P0	P1	P2	Coherence state of lock at end of step	Bus/directory activity
1	Has lock	Begins spin, testing if lock = 0	Begins spin, testing if lock = 0	Shared	Cache misses for P1 and P2 satisfied in either order. Lock state becomes shared.
2	Set lock to 0	(Invalidate received)	(Invalidate received)	Exclusive (P0)	Write invalidate of lock variable from P0.
3		Cache miss	Cache miss	Shared	Bus/directory services P2 cache miss; write-back from P0; state shared.
4		(Waits while bus/directory busy)	Lock = 0 test succeeds	Shared	Cache miss for P2 satisfied.
5		Lock = 0	Executes swap, gets cache miss	Shared	Cache miss for P1 satisfied.
6		Executes swap, gets cache miss	Completes swap: returns 0 and sets lock = 1	Exclusive (P2)	Bus/directory services P2 cache miss; generates invalidate; lock is exclusive.
7		Swap completes and returns 1, and sets lock = 1	Enter critical section	Exclusive (P1)	Bus/directory services P1 cache miss; sends invalidate and generates write-back from P2.
8		Spins, testing if lock = 0			None

Figure 5.22 Cache coherence steps and bus traffic for three processors, P0, P1, and P2. This figure assumes write invalidate coherence. P0 starts with the lock (step 1), and the value of the lock is 1 (i.e., locked); it is initially exclusive and owned by P0 before step 1 begins. P0 exits and unlocks the lock (step 2). P1 and P2 race to see which reads the unlocked value during the swap (steps 3–5). P2 wins and enters the critical section (steps 6 and 7), while P1's attempt fails, so it starts spin waiting (steps 7 and 8). In a real system, these events will take many more than 8 clock ticks because acquiring the bus and replying to misses take much longer. Once step 8 is reached, the process can repeat with P2, eventually getting exclusive access and setting the lock to 0.

Let's examine how this "spin lock" scheme uses the cache coherence mechanisms. Figure 5.22 shows the processor and bus or directory operations for multiple processes trying to lock a variable using an atomic swap. Once the processor with the lock stores a 0 into the lock, all other caches are invalidated and must fetch the new value to update their copy of the lock. One such cache gets the copy of the unlocked value (0) first and performs the swap. When the cache miss of other processors is satisfied, they find that the variable is already locked, so they must return to testing and spinning.

This example shows another advantage of the load reserved/store conditional primitives: the read and write operations are explicitly separated. The load reserved need not cause any bus traffic. This fact allows the following simple code sequence, which has the same characteristics as the optimized version using exchange (`x1` has the address of the lock, the `lr` has replaced the `LD`, and the `sc` has replaced the `EXCH`):

```
lockit:  lr    x2,0(x1)    ;load reserved
         bnez  x2,lockit   ;not available-spin
         addi  x2,R0,#1    ;locked value
         sc    x2,0(x1)    ;store
         bnez  x2,lockit   ;branch if store fails
```

The first branch forms the spinning loop; the second branch resolves races when two processors see the lock available simultaneously.

5.6 Models of Memory Consistency: An Introduction

Cache coherence ensures that multiple processors see a consistent view of memory. It does not answer the question of *how* consistent the view of memory must be. By "how consistent," we are really asking when a processor must see a value that has been updated by another processor. Because processors communicate through shared variables (used both for data values and for synchronization), the question boils down to this: In what order must a processor observe the data writes of another processor? Because the only way to "observe the writes of another processor" is through reads, the question becomes what properties must be enforced among reads and writes to different locations by different processors?

Although the question of how consistent memory must be seems simple, it is remarkably complicated, as we can see with a simple example. Here are two code segments from processes P1 and P2, shown side by side:

```
P1:       A = 0;            P2:    B = 0;
          .....                    .....
          A = 1;                   B = 1;
L1:       if (B == 0)...    L2:    if (A == 0)...
```

Assume that the processes are running on different processors, and that locations A and B are originally cached by both processors with the initial value of 0. If writes always take immediate effect and are immediately seen by other processors, it will be impossible for *both* IF statements (labeled L1 and L2) to evaluate their conditions as true, since reaching the IF statement means that either A or B must have been assigned the value 1. But suppose the write invalidate is delayed, and the processor is allowed to continue during this delay. Then it is possible that both P1 and P2 have not seen the invalidations for B and A (respectively) *before* they attempt to read the values. The question now is should this behavior be allowed, and, if so, under what conditions?

The most straightforward model for memory consistency is called *sequential consistency*. Sequential consistency requires that the result of any execution be the same as though the memory accesses executed by each processor were kept in order and the accesses among different processors were arbitrarily interleaved. Sequential consistency eliminates the possibility of some nonobvious execution in the previous example because the assignments must be completed before the IF statements are initiated.

The simplest way to implement sequential consistency is to require a processor to delay the completion of any memory access until all the invalidations caused by that access are completed. Of course, it is equally effective to delay the next memory access until the previous one is completed. Remember that memory consistency involves operations among different variables: the two accesses that must be ordered are actually to different memory locations. In our example, we must delay the read of A or B (A == 0 or B == 0) until the previous write has completed (B = 1 or A = 1). Under sequential consistency, we cannot, for example, simply place the write in a write buffer and continue with the read.

Although sequential consistency presents a simple programming paradigm, it reduces potential performance, especially in a multiprocessor with a large number of processors or long interconnect delays, as we can see in the following example.

Example Suppose we have a processor where a write miss takes 50 cycles to establish ownership, 10 cycles to issue each invalidate after ownership is established, and 80 cycles for an invalidate to complete and be acknowledged once it is issued. Assuming that four other processors share a cache block, how long does a write miss stall the writing processor if the processor is sequentially consistent? Assume that the invalidates must be explicitly acknowledged before the coherence controller knows they are completed. Suppose we could continue executing after obtaining ownership for the write miss without waiting for the invalidates; how long would the write take?

Answer When we wait for invalidates, each write takes the sum of the ownership time plus the time to complete the invalidates. Because the invalidates can overlap, we need only worry about the last one, which starts $10 + 10 + 10 + 10 = 40$ cycles after ownership is established. Therefore the total time for the write is $50 + 40 + 80 = 170$ cycles. In comparison, the ownership time is only 50 cycles. With appropriate write buffer implementations, it is even possible to continue before ownership is established.

To provide better performance, researchers and architects have explored two different routes. First, they developed ambitious implementations that preserve sequential consistency but use latency-hiding techniques to reduce the penalty; we discuss these in Section 5.7. Second, they developed less restrictive memory consistency models that allow for faster hardware. Such models can affect how the programmer sees the multiprocessor, so before we discuss these less restrictive models, let's look at what the programmer expects.

The Programmer's View

Although the sequential consistency model has a performance disadvantage, from the viewpoint of the programmer, it has the advantage of simplicity. The challenge

is to develop a programming model that is simple to explain and yet allows a high-performance implementation.

One such programming model that allows us to have a more efficient implementation is to assume that programs are *synchronized*. A program is synchronized if all accesses to shared data are ordered by synchronization operations. A data reference is ordered by a synchronization operation if, in every possible execution, a write of a variable by one processor and an access (either a read or a write) of that variable by another processor are separated by a pair of synchronization operations, one executed after the write by the writing processor and one executed before the access by the second processor. Cases where variables may be updated without ordering by synchronization are called *data races* because the execution outcome depends on the relative speed of the processors, and like races in hardware design, the outcome is unpredictable, which leads to another name for synchronized programs: *data-race-free*.

As a simple example, consider a variable being read and updated by two different processors. Each processor surrounds the read and update with a lock and an unlock, both to ensure mutual exclusion for the update and to ensure that the read is consistent. Clearly, every write is now separated from a read by the other processor by a pair of synchronization operations: one unlock (after the write) and one lock (before the read). Of course, if two processors are writing a variable with no intervening reads, then the writes must also be separated by synchronization operations.

It is a broadly accepted observation that most programs are synchronized. This observation is true primarily because, if the accesses were unsynchronized, the behavior of the program would likely be unpredictable because the speed of execution would determine which processor won a data race and thus affect the results of the program. Even with sequential consistency, reasoning about such programs is very difficult.

Programmers could attempt to guarantee ordering by constructing their own synchronization mechanisms, but this is extremely tricky, can lead to buggy programs, and may not be supported architecturally, meaning that they may not work in future generations of the multiprocessor. Instead, almost all programmers will choose to use synchronization libraries that are correct and optimized for the multiprocessor and the type of synchronization.

Finally, the use of standard synchronization primitives ensures that even if the architecture implements a more relaxed consistency model than sequential consistency, a synchronized program will behave as though the hardware implemented sequential consistency.

Relaxed Consistency Models: The Basics and Release Consistency

The key idea in relaxed consistency models is to allow reads and writes to complete out of order, but to use synchronization operations to enforce ordering so that a synchronized program behaves as though the processor were sequentially

consistent. There are a variety of relaxed models that are classified according to what read and write orderings they relax. We specify the orderings by a set of rules of the form $X \rightarrow Y$, meaning that operation X must complete before operation Y is done. Sequential consistency requires maintaining all four possible orderings: $R \rightarrow W$, $R \rightarrow R$, $W \rightarrow R$, and $W \rightarrow W$. The relaxed models are defined by the subset of four orderings they relax:

1. Relaxing only the $W \rightarrow R$ ordering yields a model known as *total store ordering* or *processor consistency*. Because this model retains ordering among writes, many programs that operate under sequential consistency operate under this model, without additional synchronization.
2. Relaxing both the $W \rightarrow R$ ordering and the $W \rightarrow W$ ordering yields a model known as *partial store order*.
3. Relaxing all four orderings yields a variety of models including *weak ordering*, the PowerPC consistency model, and *release consistency*, the RISC V consistency model.

By relaxing these orderings, the processor may obtain significant performance advantages, which is the reason that RISC V, ARMv8, as well as the C++ and C language standards chose release consistency as the model.

Release consistency distinguishes between synchronization operations that are used to *acquire* access to a shared variable (denoted S_A) and those that *release* an object to allow another processor to acquire access (denoted S_R). Release consistency is based on the observation that in synchronized programs, an acquire operation must precede a use of shared data, and a release operation must follow any updates to shared data and also precede the time of the next acquire. This property allows us to slightly relax the ordering by observing that a read or write that precedes an acquire need not complete before the acquire, and also that a read or write that follows a release need not wait for the release. Thus the orderings that are preserved involve only S_A and S_R, as shown in Figure 5.23; as the example in Figure 5.24 shows, this model imposes the fewest orders of the five models.

Release consistency provides one of the least restrictive models that is easily checkable and ensures that synchronized programs will see a sequentially consistent execution. Although most synchronization operations are either an acquire or a release (an acquire normally reads a synchronization variable and atomically updates it, and a release usually just writes it), some operations, such as a barrier, act as both an acquire and a release and cause the ordering to be equivalent to weak ordering. Although synchronization operations always ensure that previous writes have completed, we may want to guarantee that writes are completed without an identified synchronization operation. In such cases, an explicit instruction, called FENCE in RISC V, is used to ensure that all previous instructions in that thread have completed, including completion of all memory writes and associated invalidates. For more information about the complexities, implementation issues, and

Model	Used in	Ordinary orderings	Synchronization orderings
Sequential consistency	Most machines as an optional mode	$R \rightarrow R$, $R \rightarrow W$, $W \rightarrow R$, $W \rightarrow W$	$S \rightarrow W$, $S \rightarrow R$, $R \rightarrow S$, $W \rightarrow S$, $S \rightarrow S$
Total store order or processor consistency	IBMS/370, DEC VAX, SPARC	$R \rightarrow R$, $R \rightarrow W$, $W \rightarrow W$	$S \rightarrow W$, $S \rightarrow R$, $R \rightarrow S$, $W \rightarrow S$, $S \rightarrow S$
Partial store order	SPARC	$R \rightarrow R$, $R \rightarrow W$	$S \rightarrow W$, $S \rightarrow R$, $R \rightarrow S$, $W \rightarrow S$, $S \rightarrow S$
Weak ordering	PowerPC		$S \rightarrow W$, $S \rightarrow R$, $R \rightarrow S$, $W \rightarrow S$, $S \rightarrow S$
Release consistency	MIPS, RISC V, Armv8, C, and C++ specifications		$S_A \rightarrow W$, $S_A \rightarrow R$, $R \rightarrow S_R$, $W \rightarrow S_R$, $S_A \rightarrow S_A$, $S_A \rightarrow S_R$, $S_R \rightarrow S_A$, $S_R \rightarrow S_R$

Figure 5.23 The orderings imposed by various consistency models are shown for both ordinary accesses and synchronization accesses. The models grow from most restrictive (sequential consistency) to least restrictive (release consistency), allowing increased flexibility in the implementation. The weaker models rely on fences created by synchronization operations, as opposed to an implicit fence at every memory operation. S_A and S_R stand for acquire and release operations, respectively, and are needed to define release consistency. If we were to use the notation S_A and S_R for each S consistently, each ordering with one S would become two orderings (e.g., $S \rightarrow W$ becomes $S_A \rightarrow W$, $S_R \rightarrow W$), and each $S \rightarrow S$ would become the four orderings shown in the last line of the bottom-right table entry.

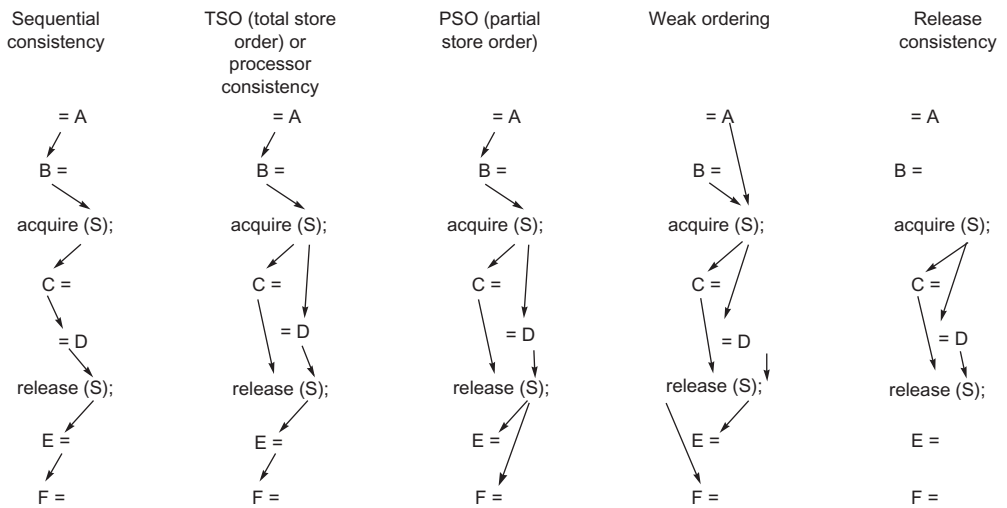


Figure 5.24 These examples of the five consistency models discussed in this section show the reduction in the number of orders imposed as the models become more relaxed. Only the minimum orders are shown with arrows. Orders implied by transitivity, such as the write of C before the release of S in the sequential consistency model or the acquire before the release in weak ordering or release consistency, are not shown.

performance potential from relaxed models, we highly recommend the excellent tutorial by Adve and Gharachorloo (1996).

5.7 Cross-Cutting Issues

Because multiprocessors redefine many system characteristics (e.g., performance assessment, memory latency, and the importance of scalability), they introduce interesting design problems that cut across the spectrum, affecting both hardware and software. In this section, we give several examples related to the issue of memory consistency. We then examine the performance gained when multithreading is added to multiprocessing.

Compiler Optimization and the Consistency Model

Another reason for defining a model for memory consistency is to specify the range of legal compiler optimizations that can be performed on shared data. In explicitly parallel programs, unless the synchronization points are clearly defined and the programs are synchronized, the compiler cannot interchange a read and a write of two different shared data items because such transformations might affect the semantics of the program. This restriction prevents even relatively simple optimizations, such as register allocation of shared data, because such a process usually interchanges reads and writes. In implicitly parallelized programs—for example, those written in High Performance Fortran (HPF)—programs must be synchronized and the synchronization points are known, so this issue does not arise. Whether compilers can get significant advantage from more relaxed consistency models remains an open question, both from a research viewpoint and from a practical viewpoint, where the lack of uniform models is likely to retard progress on deploying compilers.

Using Speculation to Hide Latency in Strict Consistency Models

As we saw in Chapter 3, speculation can be used to hide memory latency. It can also be used to hide latency arising from a strict consistency model, giving much of the benefit of a relaxed memory model. The key idea is for the processor to use dynamic scheduling to reorder memory references, letting them possibly execute out of order. Executing the memory references out of order may generate violations of sequential consistency, which might affect the execution of the program. This possibility is avoided by using the delayed commit feature of a speculative processor. Assume the coherency protocol is based on invalidation. If the processor receives an invalidation for a memory reference before the memory reference is committed, the processor uses speculation recovery to back out of the computation and restart with the memory reference whose address was invalidated.

If the reordering of memory requests by the processor yields an execution order that could result in an outcome that differs from what would have been seen under

sequential consistency, the processor will redo the execution. The key to using this approach is that the processor need only guarantee that the result would be the same as if all accesses were completed in order, and it can achieve this by detecting when the results might differ. The approach is attractive because the speculative restart will rarely be triggered. It will be triggered only when there are unsynchronized accesses that actually cause a race (Gharachorloo et al., 1992).

Hill (1998) advocated the combination of sequential or processor consistency together with speculative execution as the consistency model of choice. His argument has three parts. First, an aggressive implementation of either sequential consistency or processor consistency will gain most of the advantage of a more relaxed model. Second, such an implementation adds very little to the implementation cost of a speculative processor. Third, such an approach allows the programmer to reason using the simpler programming models of either sequential or processor consistency. The MIPS R10000 design team had this insight in the mid-1990s and used the R10000's out-of-order capability to support this type of aggressive implementation of sequential consistency.

One open question is how successful compiler technology will be in optimizing memory references to shared variables. The state of optimization technology and the fact that shared data are often accessed via pointers or array indexing have limited the use of such optimizations. If this technology were to become available and lead to significant performance advantages, compiler writers would want to be able to take advantage of a more relaxed programming model. This possibility and the desire to keep the future as flexible as possible led the RISC V designers to opt for release consistency, after a long series of debates.

Inclusion and Its Implementation

All multiprocessors use multilevel cache hierarchies to reduce both the demand on the global interconnect and the latency of cache misses. If the cache also provides *multilevel inclusion*—every level of cache hierarchy is a subset of the level farther away from the processor—then we can use the multilevel structure to reduce the contention between coherence traffic and processor traffic that occurs when snoops and processor cache accesses must contend for the cache. Many multiprocessors with multilevel caches enforce the inclusion property, although recent multiprocessors with smaller L1 caches and different block sizes have sometimes chosen not to enforce inclusion. This restriction is also called the *subset property* because each cache is a subset of the cache below it in the hierarchy.

At first glance, preserving the multilevel inclusion property seems trivial. Consider a two-level example: Any miss in L1 either hits in L2 or generates a miss in L2, causing it to be brought into both L1 and L2. Likewise, any invalidate that hits in L2 must be sent to L1, where it will cause the block to be invalidated if it exists.

The catch is what happens when the block sizes of L1 and L2 are different. Choosing different block sizes is quite reasonable, since L2 will be much larger and have a much longer latency component in its miss penalty, and thus will want

to use a larger block size. What happens to our "automatic" enforcement of inclusion when the block sizes differ? A block in L2 represents multiple blocks in L1, and a miss in L2 causes the replacement of data that is equivalent to multiple L1 blocks. For example, if the block size of L2 is four times that of L1, then a miss in L2 will replace the equivalent of four L1 blocks. Let's consider a detailed example.

Example Assume that L2 has a block size four times that of L1. Show how a miss for an address that causes a replacement in L1 and L2 can lead to violation of the inclusion property.

Answer Assume that L1 and L2 are direct-mapped and that the block size of L1 is b bytes and the block size of L2 is $4b$ bytes. Suppose L1 contains two blocks with starting addresses x and $x+b$ and that $x \bmod 4b = 0$, meaning that x also is the starting address of a block in L2; then that single block in L2 contains the L1 blocks x, $x+b$, $x+2b$, and $x+3b$. Suppose the processor generates a reference to block y that maps to the block containing x in both caches and thus misses. Because L2 missed, it fetches $4b$ bytes and replaces the block containing x, $x+b$, $x+2b$, and $x+3b$, while L1 takes b bytes and replaces the block containing x. Because L1 still contains $x+b$, but L2 does not, the inclusion property no longer holds.

To maintain inclusion with multiple block sizes, we must probe the higher levels of the hierarchy when a replacement is done at the lower level to ensure that any words replaced in the lower level are invalidated in the higher-level caches; different levels of associativity create the same sort of problems. Baer and Wang (1988) described the advantages and challenges of inclusion in detail, and in 2017 most designers have opted to implement inclusion, often by settling on one block size for all levels in the cache. For example, the Intel i7 uses inclusion for L3, meaning that L3 always includes the contents of all of L2 and L1. This decision allows the i7 to implement a straightforward directory scheme at L3 and to minimize the interference from snooping on L1 and L2 to those circumstances where the directory indicates that L1 or L2 have a cached copy. The AMD Opteron, in contrast, makes L2 inclusive of L1 but has no such restriction for L3. It uses a snooping protocol, but only needs to snoop at L2 unless there is a hit, in which case a snoop is sent to L1.

Performance Gains From Multiprocessing and Multithreading

In this section, we briefly look at a study of the effectiveness of using multithreading on a multicore processor, the IBM Power5; we will return to this topic in the next section, when we examine the performance of the Intel i7. The IBM Power5 is a dual-core that supports simultaneous multithreading (SMT); its basic architecture is very similar to the more recent Power8 (which we examine in the next section), but it has only two cores per processor.

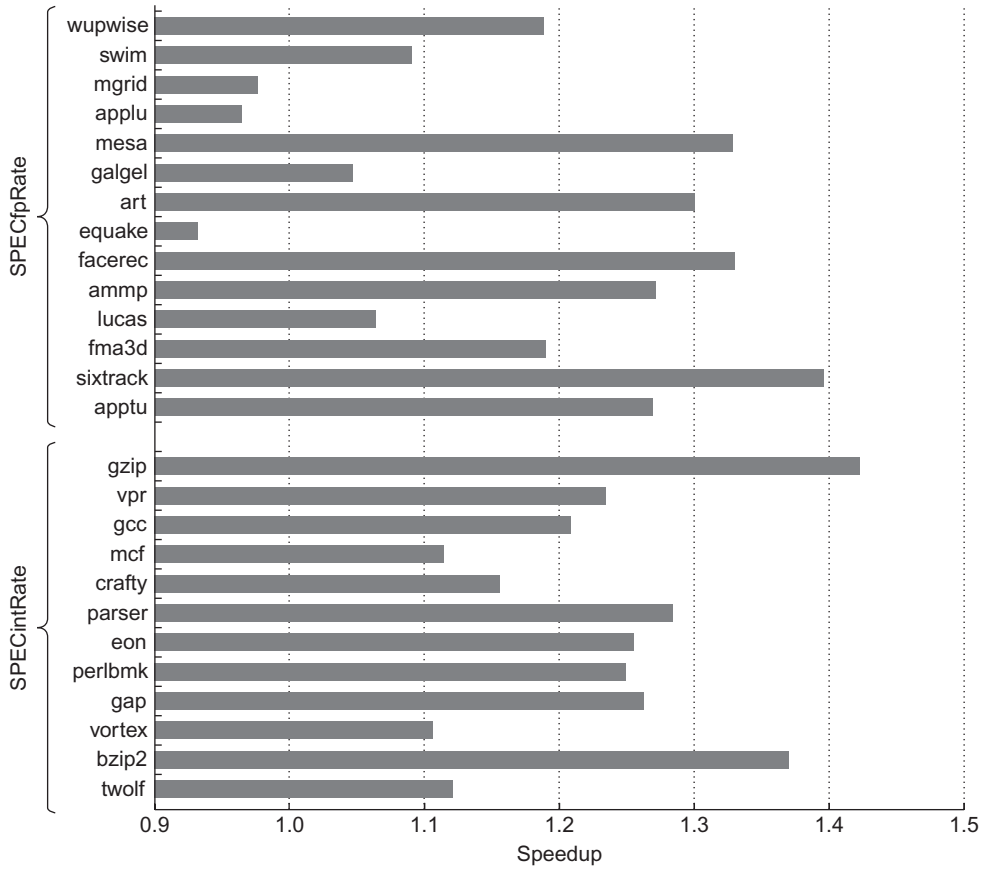


Figure 5.25 A comparison of SMT and single-thread (ST) performance on the 8-processor IBM eServer p5 575 using SPECfpRate (top half) and SPECintRate (bottom half) as benchmarks. Note that the *x*-axis starts at a speedup of 0.9, a performance loss. Only one processor in each Power5 core is active, which should slightly improve the results from SMT by decreasing destructive interference in the memory system. The SMT results are obtained by creating 16 user threads, whereas the ST results use only eight threads; with only one thread per processor, the Power5 is switched to single-threaded mode by the OS. These results were collected by John McCalpin at IBM. As we can see from the data, the standard deviation of the results for the SPECfpRate is higher than for SPECintRate (0.13 versus 0.07), indicating that the SMT improvement for FP programs is likely to vary widely.

To examine the performance of multithreading in a multiprocessor, measurements were made on an IBM system with eight Power5 processors, using only one core on each processor. Figure 5.25 shows the speedup for an 8-processor Power5 multiprocessor, with and without SMT, for the SPECRate2000 benchmarks, as described in the caption. On average, the SPECintRate is 1.23 times faster, and the SPECfpRate is 1.16 times faster. Note that a few floating-point benchmarks experience a slight decrease in performance in SMT mode, with the maximum

reduction in speedup being 0.93. Although one might expect that SMT would do a better job of hiding the higher miss rates of the SPECFP benchmarks, it appears that limits in the memory system are encountered when running in SMT mode on such benchmarks.

5.8 Putting It All Together: Multicore Processors and Their Performance

For roughly 10 years, multicore has been the primary focus for scaling performance, although the implementations vary widely, as does their support for larger multichip multiprocessors. In this section, we examine the design of three different multicores, the support they provide for larger multiprocessors, and some performance characteristics, before doing a broader evaluation of small to large multiprocessor Xeon systems, and concluding with a detailed evaluation of the multicore i7 920, a predecessor of the i7 6700.

Performance of Multicore-Based Multiprocessors on a Multiprogrammed Workload

Figure 5.26 shows the key characteristics of three multicore processors designed for server applications and available in 2015 through 2017. The Intel Xeon E7 is based on the same basic design as the i7, but it has more cores, a slightly slower clock rate (power is the limitation), and a larger L3 cache. The Power8 is the newest in the IBM Power series and features more cores and bigger caches. The Fujitsu SPARC64 X+ is the newest SPARC server chip; unlike the T-series mentioned in Chapter 3, it uses SMT. Because these processors are configured for multicore and multiprocessor servers, they are available as a family, varying processor count, cache size, and so on, as the figure shows.

These three systems show a range of techniques both for connecting the on-chip cores and for connecting multiple processor chips. First, let's look at how the cores are connected within a chip. The SPARC64 X+ is the simplest: it shares a single L2 cache, which is 24-way set associative, among the 16 cores. There are four separate DIMM channels to attach memory accessible with a 16×4 switch between the cores and the channels.

Figure 5.27 shows how the Power8 and Xeon E7 chips are organized. Each core in the Power8 has an 8 MiB bank of L3 directly connected; other banks are accessed via the interconnection network, which has 8 separate buses. Thus the Power8 is a true NUCA (*Nonuniform Cache Architecture*), because the access time to the attached bank of L3 will be much faster than accessing another L3. Each Power8 chip has a set of links that can be used to build a large multiprocessor using an organization we will see shortly. The memory links are connected to a special memory controller that includes an L4 and interfaces directly with DIMMs.

Part B of Figure 5.27, shows how the Xeon E7 processor chip is organized when there are 18 or more cores (20 cores are shown in this figure). Three rings

Feature	IBM Power8	Intel Xeon E7	Fujitsu SPARC64 X+
Cores/chip	4, 6, 8, 10, 12	4, 8, 10, 12, 22, 24	16
Multithreading	SMT	SMT	SMT
Threads/core	8	2	2
Clock rate	3.1–3.8 GHz	2.1–3.2 GHz	3.5 GHz
L1 I cache	32 KB per core	32 KB per core	64 KB per core
L1 D cache	64 KB per core	32 KB per core	64 KB per core
L2 cache	512 KB per core	256 KB per core	24 MiB shared
L3 cache	L3: 32–96 MiB: 8 MiB per core (using eDRAM); shared with nonuniform access time	10–60 MiB @ 2.5 MiB per core; shared, with larger core counts	None
Inclusion	Yes, L3 superset	Yes, L3 superset	Yes
Multicore coherence protocol	Extended MESI with behavioral and locality hints (13-states)	MESIF: an extended form of MESI allowing direct transfers of clean blocks	MOESI
Multichip coherence implementation	Hybrid strategy with snooping and directory	Hybrid strategy with snooping and directory	Hybrid strategy with snooping and directory
Multiprocessor interconnect support	Can connect up to 16 processor chips with 1 or 2 hops to reach any processor	Up to 8 processor chips directly via Quickpath; larger system and directory support with additional logic	Crossbar interconnect chip, supports up to 64 processors; includes directory support
Processor chip range	1–16	2–32	1–64
Core count range	4–192	12–576	8–1024

Figure 5.26 Summary of the characteristics of three recent high-end multicore processors (2015–2017 releases) designed for servers. The table shows the range of processor counts, clock rates, and cache sizes within each processor family. The Power8 L3 is a NUCA (Nonuniform Cache Access) design, and it also supports off-chip L4 of up to 128 MiB using EDRAM. A 32-core Xeon has recently been announced, but no system shipments have occurred. The Fujitsu SPARC64 is also available as an 8-core design, which is normally configured as a single processor system. The last row shows the range of configured systems with published performance data (such as SPECintRate) with both processor chip counts and total core counts. The Xeon systems include multiprocessors that extend the basic interconnect with additional logic; for example, using the standard Quickpath interconnect limits the processor count to 8 and the largest system to $8 \times 24 = 192$ cores, but SGI extends the interconnect (and coherence mechanisms) with extra logic to offer a 32 processor system using 18-core processor chips for a total size of 576 cores. Newer releases of these processors increased clock rates (significantly in the Power8 case, less so in others) and core counts (significantly in the case of Xeon).

connect the cores and the L3 cache banks, and each core and each bank of L3 is connected to two rings. Thus any cache bank or any core is accessible from any other core by choosing the right ring. Therefore, within the chip, the E7 has uniform access time. In practice, however, the E7 is normally operated as a NUMA architecture by logically associating half the cores with each memory channel; this

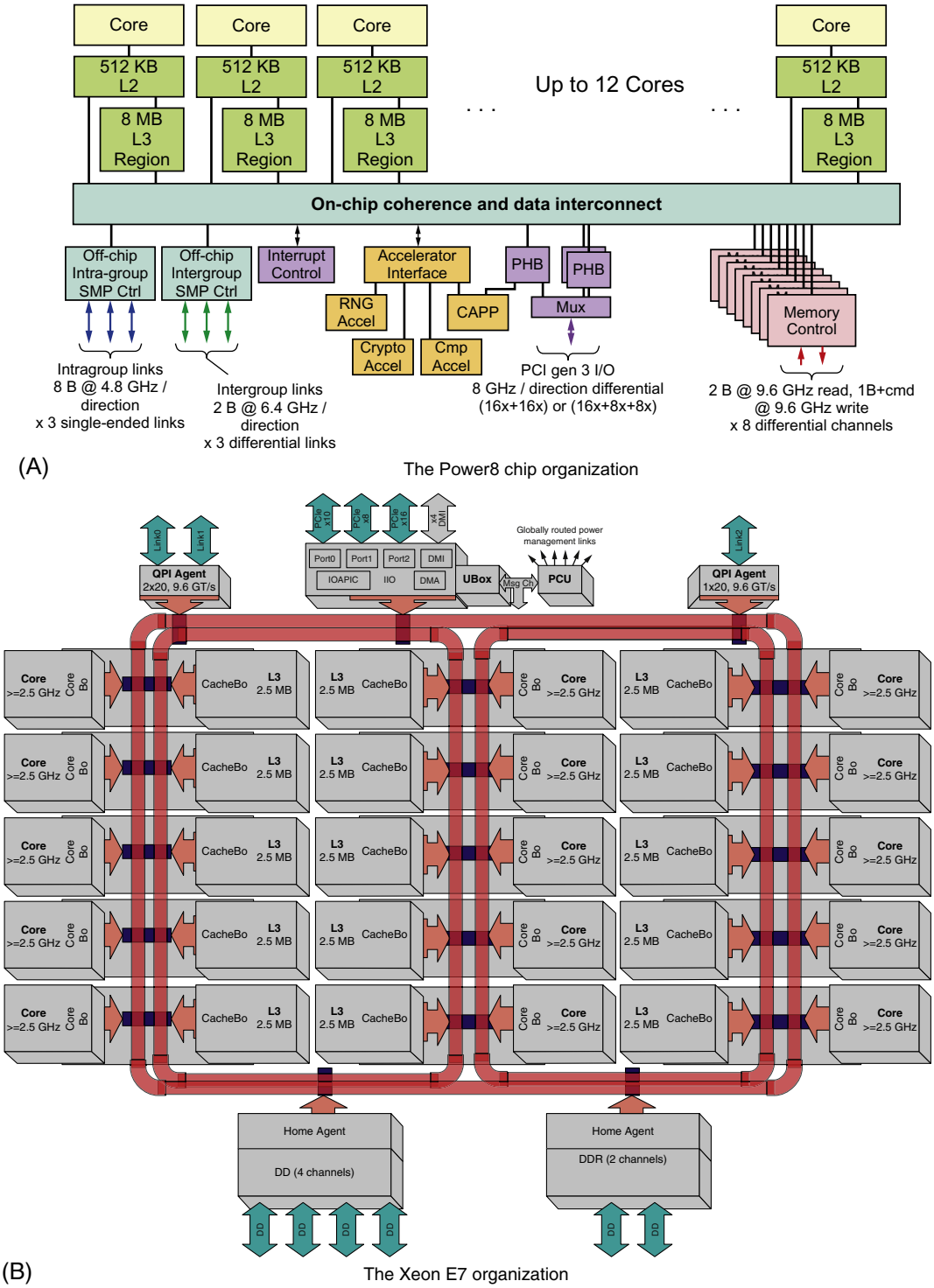


Figure 5.27 The on-chip organizations of the Power8 and Xeon E7 are shown. The Power8 uses 8 separate buses between L3 and the CPU cores. Each Power8 also has two sets of links for connecting larger multiprocessors. The Xeon uses three rings to connect processors and L3 cache banks, as well QPI for interchip links. Software is used to logically associate half the cores with each memory channel.

increases the probability that a desired memory page is open on a given access. The E7 provides 3 QuickPath Interconnect (QPI) links for connecting multiple E7s.

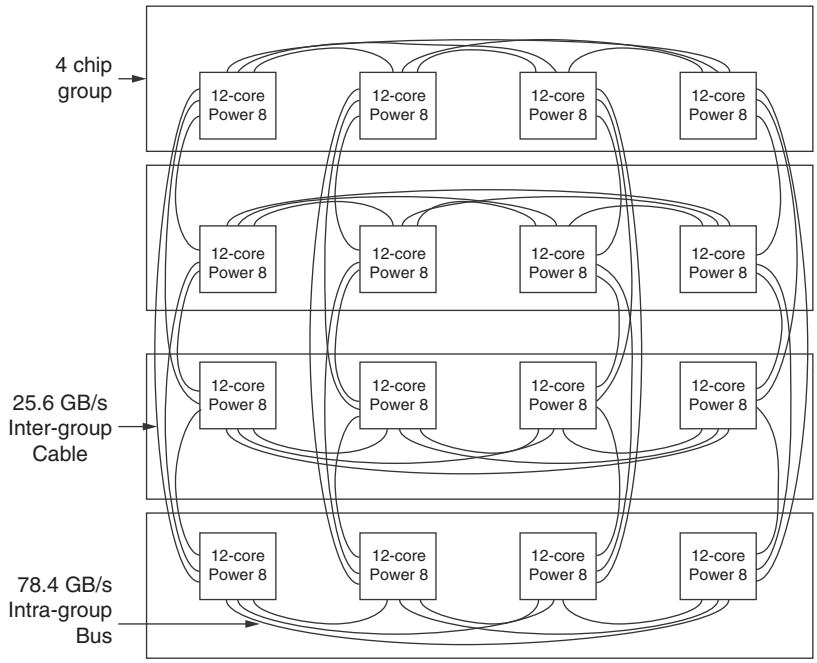
Multiprocessors consisting of these multicores use a variety of different interconnection strategies, as Figure 5.28 shows. The Power8 design provides support for connecting 16 Power8 chips for a total of 192 cores. The intragroup links provide higher bandwidth interconnect among a completely connected module of 4 processor chips. The intergroup links are used to connect each processor chip to the 3 other modules. Thus each processor is two hops from any other, and the memory access time is determined by whether an address resides in local memory, cluster memory, or intercluster memory (actually the latter can have two different values, but the difference is swamped by the intercluster time).

The Xeon E7 uses QPI to interconnect multiple multicore chips. In a 4-chip, multiprocessor, which with the latest announced Xeon could have 128 cores, the three QPI links on each processor are connected to three neighbors, yielding a 4-chip fully connected multiprocessor. Because memory is directly connected to each E7 multicore, even this 4-chip arrangement has nonuniform memory access time (local versus remote). Figure 5.28 shows how 8 E7 processors can be connected; like the Power8, this leads to a situation where every processor is one or two hops from every other processor. There are a number of Xeon-based multiprocessor servers that have more than 8 processor chips. In such designs, the typical organization is to connect 4 processor chips together in a square, as a module, with each processor connecting to two neighbors. The third QPI in each chip is connected to a crossbar switch. Very large systems can be created in this fashion. Memory accesses can then occur at four locations with different timings: local to the processor, an immediate neighbor, the neighbor in the cluster that is two hops away, and through the crossbar. Other organizations are possible and require less than a full crossbar in return for more hops to get to remote memory.

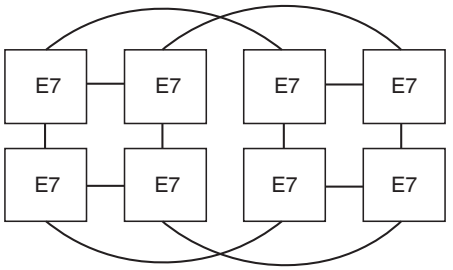
The SPARC64 X+ also uses a 4-processor module, but each processor has three connections to its immediate neighbors plus two (or three in the largest configuration) connections to a crossbar. In the largest configuration, 64 processor chips can be connected to two crossbar switches, for a total of 1024 cores. Memory access is NUMA (local, within a module, and through the crossbar), and coherency is directory-based.

Performance of Multicore-Based Multiprocessors on a Multiprogrammed Workload

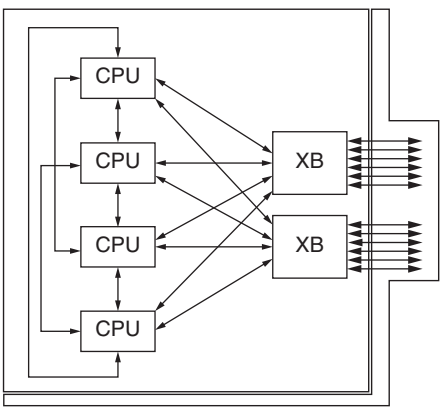
First, we compare the performance scalability of these three multicore processors using SPECintRate, considering configurations up to 64 cores. Figure 5.29 shows how the performance scales relative to the performance of the smallest configuration, which varies between 4 and 16 cores. In the plot, the smallest configuration is assumed to have perfect speedup (i.e., 8 for 8 cores, 12 for 12 cores, etc.). This figure does *not* show performance among these different processors. Indeed such performance varies significantly: in the 4-core configuration, the IBM Power8 is



(A) Power8 system with up to 16 chips.



(B) Xeon E7 system showing with up to 8 chips.



(C) SPARC64 X+ with the 4-chip building block.

Figure 5.28 The system architecture for three multiprocessors built from multicore chips.

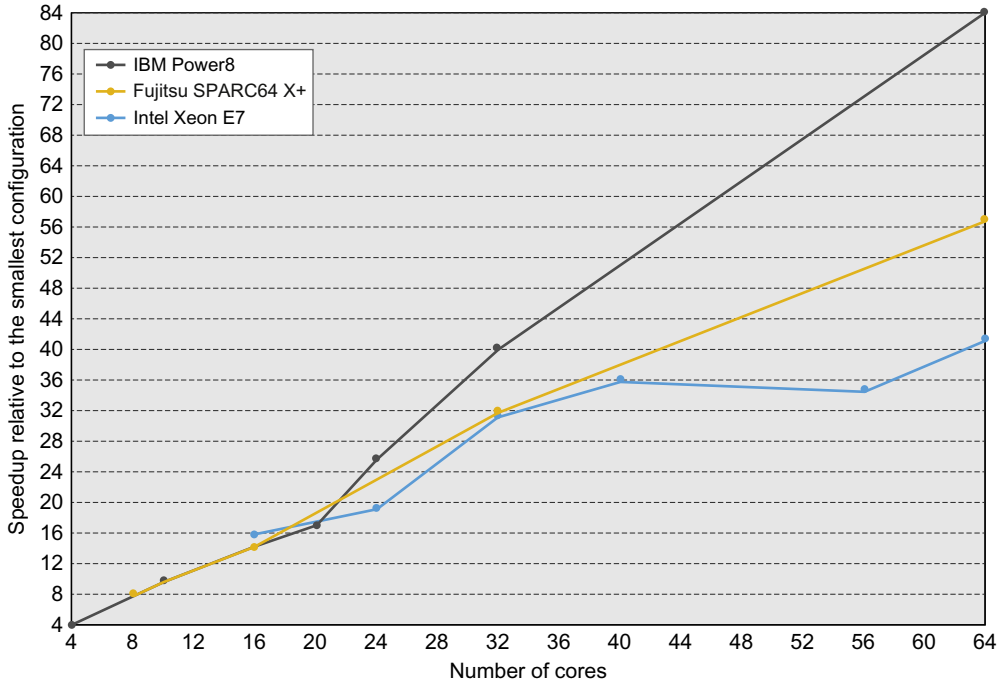


Figure 5.29 The performance scaling on the SPECintRate benchmarks for four multicore processors as the number of cores is increased to 64. Performance for each processor is plotted relative to the smallest configuration and assuming that configuration had perfect speedup. Although this chart shows how a given multiprocessor scales with additional cores, it does not supply any data about performance among processors. There are differences in the clock rates, even within a given processor family. These are generally swamped by the core scaling effects, except for the Power8 that shows a clock range spread of 1.5× from the smallest configuration to the 64 core configuration.

1.5 times as fast as the SPARC64 X+ on a per core basis! Instead Figure 5.29 shows how the performance scales for each processor family as additional cores are added.

Two of the three processors show diminishing returns as they scale to 64 cores. The Xeon systems appear to show the most degradation at 56 and 64 cores. This may be largely due to having more cores share a smaller L3. For example, the 40-core system uses 4 chips, each with 60 MiB of L3, yielding 6 MiB of L3 per core. The 56-core and 64-core systems also use 4 chips but have 35 or 45 MiB of L3 per chip, or 2.5–2.8 MiB per core. It is likely that the resulting larger L3 miss rates lead to the reduction in speedup for the 56-core and 64-core systems.

The IBM Power8 results are also unusual, appearing to show significant super-linear speedup. This effect, however, is due largely to differences in the clock rates, which are much larger across the Power8 processors than for the other processors

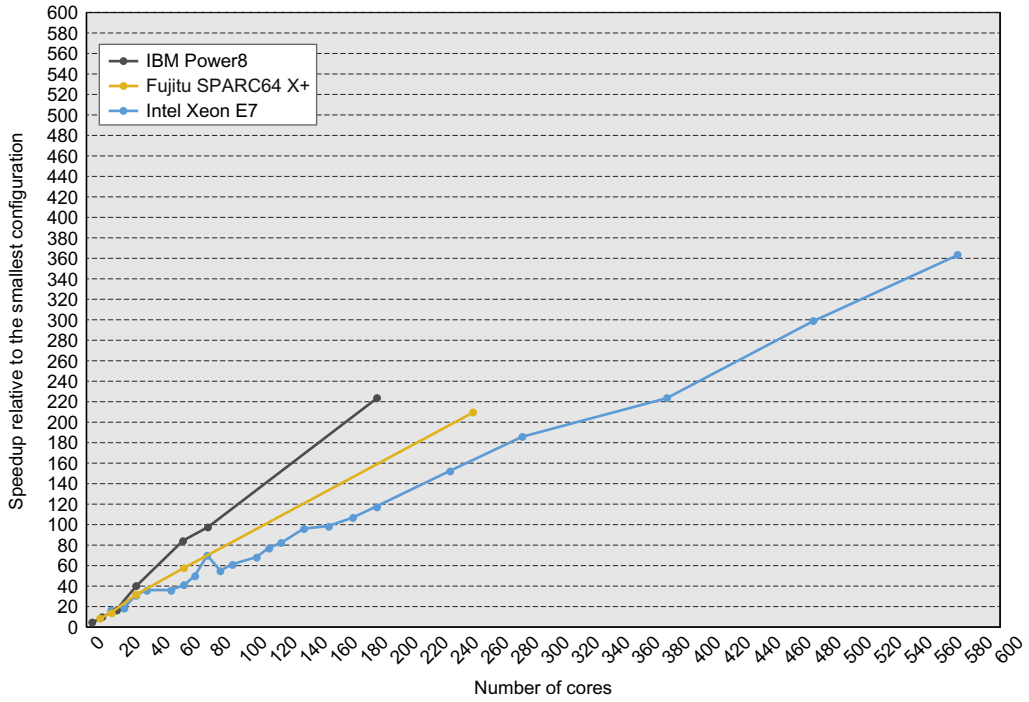


Figure 5.30 The scaling of relative performance for multiprocessor multicore. As before, performance is shown relative to the smallest available system. The Xeon result at 80 cores is the same L3 effect that showed up for smaller configurations. All systems larger than 80 cores have between 2.5 and 3.8 MiB of L3 per core, and the 80-core, or smaller, systems have 6 MiB per core.

in this figure. In particular, the 64-core configuration has the highest clock rate (4.4 GHz), whereas the 4-core configuration has a 3.0 GHz clock. If we normalize the relative speedup for the 64-core system based on the clock rate differential with the 4-core system, the effective speedup is 57 rather than 84. Therefore, while the Power8 system scales well, and perhaps the best among these processors, it is not miraculous.

Figure 5.30 shows scaling for these three systems at configurations above 64 processors. Once again, the clock rate differential explains the Power8 results; the clock-rate equivalent scaled speedup with 192 processors is 167, versus 223, when not accounting for clock rate differences. Even at 167, the Power8 scaling is somewhat better than that on the SPARC64 X+ or Xeon systems. Surprisingly, although there are some effects on speedup in going from the smallest system to 64 cores, they do not seem to get dramatically worse at these larger configurations. The nature of the workload, which is highly parallel and user-CPU-intensive, and the overheads paid in going to 64 cores probably lead to this result.

Scalability in an Xeon MP With Different Workloads

In this section, we focus on the scalability of the Xeon E7 multiprocessors on three different workloads: a Java-based commercially oriented workload, a virtual machine workload, and a scientific parallel processing workload, all from the SPEC benchmarking organization, as described next.

- SPECjbb2015: Models a supermarket IT system that handles a mix of point-of-sale requests, online purchases, and data-mining operations. The performance metric is throughput-oriented, and we use the maximum performance measurement on the server side running multiple Java virtual machines.
- SPECVirt2013: Models a collection of virtual machines running independent mixes of other SPEC benchmarks, including CPU benchmarks, web servers, and mail servers. The system must meet a quality of service guarantee for each virtual machine.
- SPECOMP2012: A collection of 14 scientific and engineering programs written with the OpenMP standard for shared-memory parallel processing. The codes are written in Fortran, C, and C++ and range from fluid dynamics to molecular modeling to image manipulation.

As with the previous results, Figure 5.31 shows performance assuming linear speedup on the smallest configuration, which for these benchmarks varies from 48 cores to 72 cores, and plotting performance relative to the that smallest configuration. The SPECjbb2015 and SPECVirt2013 include significant systems software, including the Java VM software and the VM hypervisor. Other than the system software, the interaction among the processes is very small. In contrast, SPECOMP2012 is a true parallel code with multiple user processes sharing data and collaborating in the computation.

Let's begin by examining SPECjbb2015. It obtains speedup efficiency (speedup/processor ratio) of between 78% and 95%, showing good speedup, even in the largest configuration. SPECVirt2013 does even better (for the range of system measured), obtaining almost linear speedup at 192 cores. Both SPECjbb2015 and SPECVirt2013 are benchmarks that scale up the application size (as in the TPC benchmarks discussed in Chapter 1) with larger systems so that the effects of Amdahl's Law and interprocess communication are minor.

Finally, let's turn to SPECOMP2012, the most compute-intensive of these benchmarks and the one that truly involves parallel processing. The major trend visible here is a steady loss of efficiency as we scale from 30 to 576 cores so that by 576 cores, the system exhibits only half the efficiency it showed at 30 cores. This reduction leads to a relative speedup of 284, assuming that the 30-core speedup is 30. These are probably Amdahl's Law effects resulting from limited parallelism as well as synchronization and communication overheads. Unlike

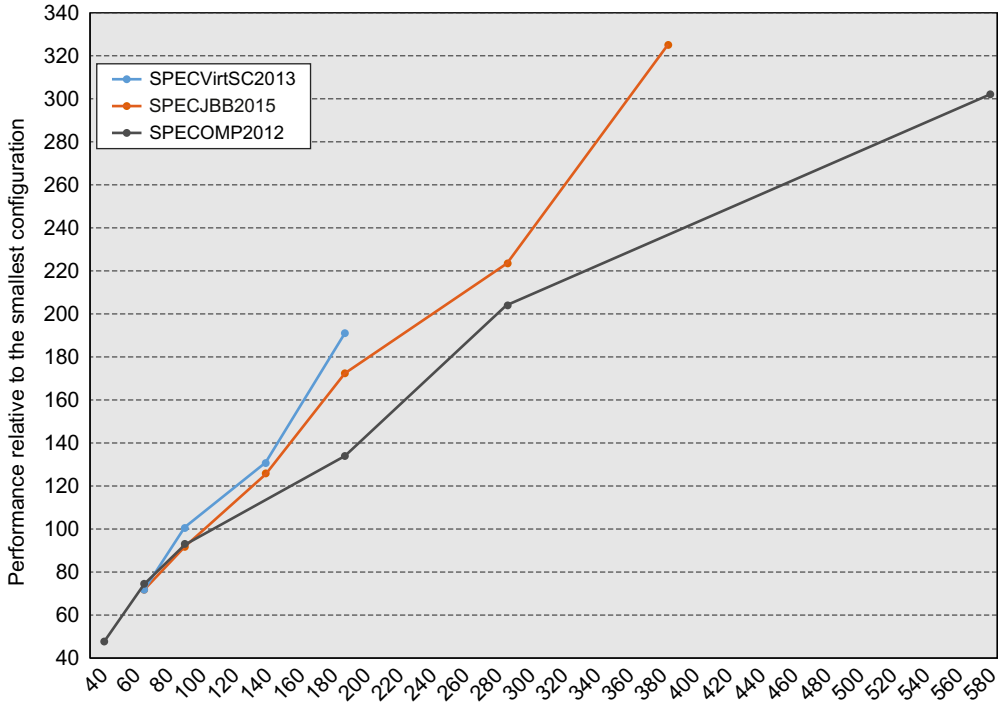


Figure 5.31 Scaling of performance on a range of Xeon E7 systems showing performance relative to the smallest benchmark configuration, and assuming that configuration gets perfect speedup (e.g., the smallest SPEWCOMP configuration is 30 cores and we assume a performance of 30 for that system). Only relative performance can be assessed from this data, and comparisons across the benchmarks have no relevance. Note the difference in the scale of the vertical and horizontal axes.

the SPECjbb2015 and SPECVirt2013, these benchmarks are not scaled for larger systems.

Performance and Energy Efficiency of the Intel i7 920 Multicore

In this section, we closely examine the performance of the i7 920, a predecessor of the 6700, on the same two groups of benchmarks we considered in Chapter 3: the parallel Java benchmarks and the parallel PARSEC benchmarks (described in detail in Figure 3.32 on page 247). Although this study uses the older i7 920, it remains, by far, the most comprehensive study of energy efficiency in multicore processors and the effects of multicore combined with SMT. The fact that the i7 920 and 6700 are similar indicates that the basic insights should also apply to the 6700.

First, we look at the multicore performance and scaling versus a single-core without the use of SMT. Then we combine both the multicore and SMT capability. All the data in this section, like that in the earlier i7 SMT evaluation (Chapter 3) come from Esmaeilzadeh et al. (2011). The dataset is the same as that used earlier (see Figure 3.32 on page 247), except that the Java benchmarks tradebeans and pjbb2005 are removed (leaving only the five scalable Java benchmarks); tradebeans and pjbb2005 never achieve speedup above 1.55 even with four cores and a total of eight threads, and thus are not appropriate for evaluating more cores.

Figure 5.32 plots both the speedup and energy efficiency of the Java and PARSEC benchmarks without the use of SMT. Energy efficiency is computed by the ratio: energy consumed by the single-core run divided by the energy consumed by the two- or four-core run (i.e., efficiency is the inverse of energy consumed). Higher energy efficiency means that the processor consumes less energy for the same computation, with a value of 1.0 being the break-even

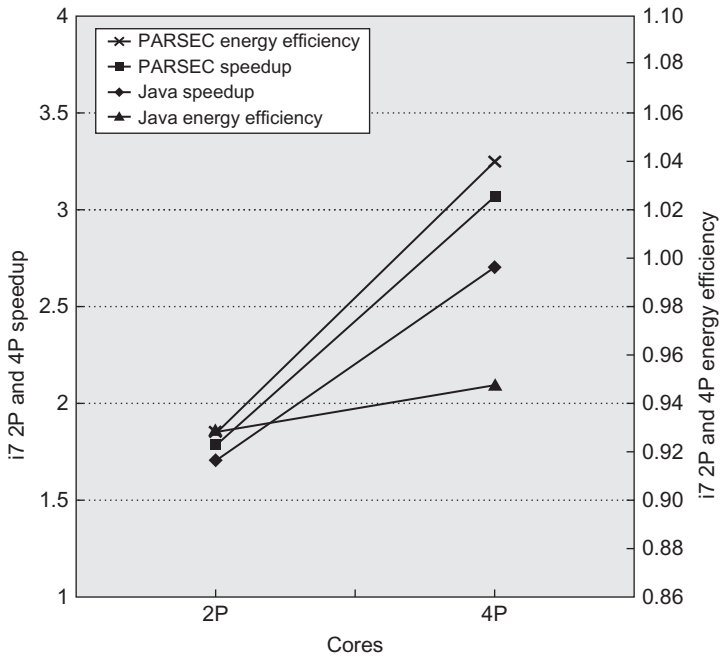


Figure 5.32 This chart shows the speedup and energy efficiency for two- and four-core executions of the parallel Java and PARSEC workloads without SMT. These data were collected by Esmaeilzadeh et al. (2011) using the same setup as described in Chapter 3. Turbo Boost is turned off. The speedup and energy efficiency are summarized using harmonic mean, implying a workload where the total time spent running each benchmark on 2 cores is equivalent.

point. The unused cores in all cases were in deep sleep mode, which minimized their power consumption by essentially turning them off. In comparing the data for the single-core and multicore benchmarks, it is important to remember that the full energy cost of the L3 cache and memory interface is paid in the single-core (as well as the multicore) case. This fact increases the likelihood that energy consumption will improve for applications that scale reasonably well. Harmonic mean is used to summarize results with the implication described in the caption.

As the figure shows, the PARSEC benchmarks get better speedup than the Java benchmarks, achieving 76% speedup efficiency (i.e., actual speedup divided by processor count) on four cores, whereas the Java benchmarks achieve 67% speedup efficiency on four cores. Although this observation is clear from the data, analyzing why this difference exists is difficult. It is quite possible that Amdahl's Law effects have reduced the speedup for the Java workload, which includes some typically serial parts, such as the garbage collector. In addition, interaction between the processor architecture and the application, which affects issues such as the cost of synchronization or communication, may also play a role. In particular, well-parallelized applications, such as those in PARSEC, sometimes benefit from an advantageous ratio between computation and communication, which reduces the dependence on communications costs (see Appendix I).

These differences in speedup translate to differences in energy efficiency. For example, the PARSEC benchmarks actually slightly improve energy efficiency over the single-core version; this result may be significantly affected by the fact that the L3 cache is more effectively used in the multicore runs than in the single-core case and the energy cost is identical in both cases. Thus, for the PARSEC benchmarks, the multicore approach achieves what designers hoped for when they switched from an ILP-focused design to a multicore design; namely, it scales performance as fast or faster than scaling power, resulting in constant or even improved energy efficiency. In the Java case, we see that neither the two- nor four-core runs break even in energy efficiency because of the lower speedup levels of the Java workload (although Java energy efficiency for the 2p run is the same as for PARSEC). The energy efficiency in the four-core Java case is reasonably high (0.94). It is likely that an ILP-centric processor would need *even more* power to achieve a comparable speedup on either the PARSEC or Java workload. Thus the TLP-centric approach is also certainly better than the ILP-centric approach for improving performance for these applications. As we will see in Section 5.10, there are reasons to be pessimistic about simple, efficient, long-term scaling of multicore.

Putting Multicore and SMT Together

Finally, we consider the combination of multicore and multithreading by measuring the two benchmark sets for two to four processors and one to two threads (a total of four data points and up to eight threads). Figure 5.33 shows the speedup

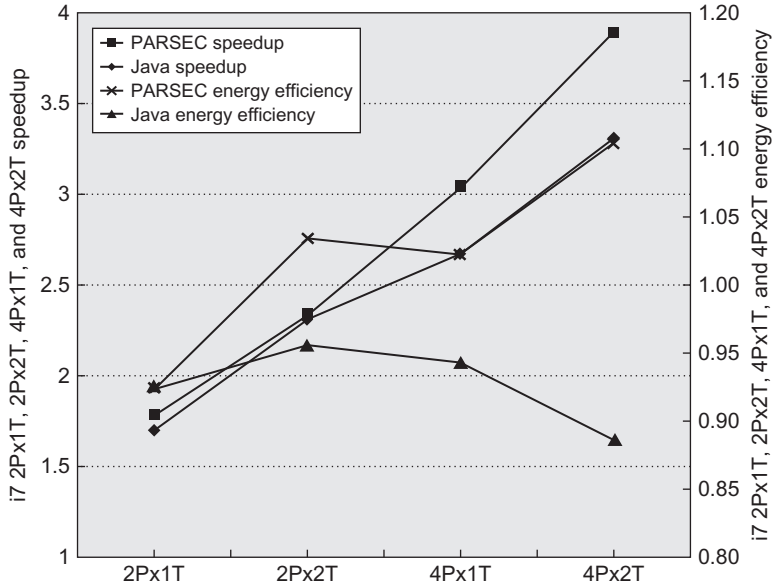


Figure 5.33 This chart shows the speedup for two- and four-core executions of the parallel Java and PARSEC workloads both with and without SMT. Remember that the preceding results vary in the number of threads from two to eight and reflect both architectural effects and application characteristics. Harmonic mean is used to summarize results, as discussed in the Figure 5.32 caption.

and energy efficiency obtained on the Intel i7 when the processor count is two or four and SMT is or is not employed, using harmonic mean to summarize the two benchmarks sets. Clearly, SMT can add to performance when there is sufficient thread-level parallelism available even in the multicore situation. For example, in the four-core, no-SMT case, the speedup efficiencies were 67% and 76% for Java and PARSEC, respectively. With SMT on four cores, those ratios are an astonishing 83% and 97%.

Energy efficiency presents a slightly different picture. In the case of PARSEC, speedup is essentially linear for the four-core SMT case (eight threads), and power scales more slowly, resulting in an energy efficiency of 1.1 for that case. The Java situation is more complex; energy efficiency peaks for the two-core SMT (four-thread) run at 0.97 and drops to 0.89 in the four-core SMT (eight-thread) run. It seems highly likely that the Java benchmarks are encountering Amdahl's Law effects when more than four threads are deployed. As some architects have observed, multicore does shift more responsibility for performance (and thus energy efficiency) to the programmer, and the results for the Java workload certainly bear this out.

5.9 Fallacies and Pitfalls

Given the lack of maturity in our understanding of parallel computing, there are many hidden pitfalls that will be uncovered either by careful designers or by unfortunate ones. Given the large amount of hype that has surrounded multiprocessors over the years, common fallacies abound. We have included a selection of them.

Pitfall *Measuring performance of multiprocessors by linear speedup versus execution time.*

Graphs like those in Figures 5.32 and 5.33, which plot performance versus number of processors, showing linear speedup, a plateau, and then a falling off, have long been used to judge the success of parallel processors. Although speedup is one facet of a parallel program, it is not a direct measure of performance. The first issue is the power of the processors being scaled: a program that linearly improves performance to equal 100 Intel Atom processors (the low-end processor used for netbooks) may be slower than the version run on an 8-core Xeon. Be especially careful of floating-point-intensive programs; processing elements without hardware assist may scale wonderfully but have poor collective performance.

Comparing execution times is fair only if you are comparing the best algorithms on each computer. Comparing the identical code on two computers may seem fair, but it is not; the parallel program may be slower on a uniprocessor than on a sequential version. Developing a parallel program will sometimes lead to algorithmic improvements, so comparing the previously best-known sequential program with the parallel code—which seems fair—will not compare equivalent algorithms. To reflect this issue, the terms *relative speedup* (same program) and *true speedup* (best program) are sometimes used.

Results that suggest *superlinear* performance, when a program on n processors is more than n times faster than the equivalent uniprocessor, may indicate that the comparison is unfair, although there are instances where "real" superlinear speedups have been encountered. For example, some scientific applications regularly achieve superlinear speedup for small increases in processor count (2 or 4 to 8 or 16). These results usually arise because critical data structures that do not fit into the aggregate caches of a multiprocessor with 2 or 4 processors fit into the aggregate cache of a multiprocessor with 8 or 16 processors. As we saw in the previous section, other differences (such as high clock rate) may appear to yield superlinear speedups when comparing slightly different systems.

In summary, comparing performance by comparing speedups is at best tricky and at worst misleading. Comparing the speedups for two different multiprocessors does not necessarily tell us anything about the relative performance of the multiprocessors, as we also saw in the previous section. Even comparing two different algorithms on the same multiprocessor is tricky because we must use true speedup, rather than relative speedup, to obtain a valid comparison.

Fallacy *Amdahl's Law doesn't apply to parallel computers.*

In 1987 the head of a research organization claimed that Amdahl's Law (see Section 1.9) had been broken by an MIMD multiprocessor. This statement hardly

meant, however, that the law has been overturned for parallel computers; the neglected portion of the program will still limit performance. To understand the basis of the media reports, let's see what Amdahl (1967) originally said:

> A fairly obvious conclusion which can be drawn at this point is that the effort expended on achieving high parallel processing rates is wasted unless it is accompanied by achievements in sequential processing rates of very nearly the same magnitude. [p. 483]

One interpretation of the law was that, because portions of every program must be sequential, there is a limit to the useful economic number of processors—say, 100. By showing linear speedup with 1000 processors, this interpretation of Amdahl's Law was disproved.

The basis for the statement that Amdahl's Law had been "overcome" was the use of *scaled speedup*, also called *weak scaling*. The researchers scaled the benchmark to have a dataset size that was 1000 times larger and compared the uniprocessor and parallel execution times of the scaled benchmark. For this particular algorithm, the sequential portion of the program was constant independent of the size of the input, and the rest was fully parallel—thus, linear speedup with 1000 processors. Because the running time grew faster than linear, the program actually ran longer after scaling, even with 1000 processors.

Speedup that assumes scaling of the input is not the same as true speedup, and reporting it as if it were is misleading. Because parallel benchmarks are often run on different-sized multiprocessors, it is important to specify what type of application scaling is permissible and how that scaling should be done. Although simply scaling the data size with processor count is rarely appropriate, assuming a fixed problem size for a much larger processor count (called *strong scaling*) is often inappropriate, as well, because it is likely that users given a much larger multiprocessor would opt to run a larger or more detailed version of an application. See Appendix I for more discussion on this important topic.

Fallacy *Linear speedups are needed to make multiprocessors cost-effective.*

It is widely recognized that one of the major benefits of parallel computing is to offer a "shorter time to solution" than the fastest uniprocessor. Many people, however, also hold the view that parallel processors cannot be as cost-effective as uniprocessors unless they can achieve perfect linear speedup. This argument says that, because the cost of the multiprocessor is a linear function of the number of processors, anything less than linear speedup means that the performance/cost ratio decreases, making a parallel processor less cost-effective than using a uniprocessor.

The problem with this argument is that cost is not only a function of processor count but also depends on memory, I/O, and the overhead of the system (box, power supply, interconnect, etc.). It also makes less sense in the multicore era, when there are multiple processors per chip.

The effect of including memory in the system cost was pointed out by Wood and Hill (1995). We use an example based on more recent data using TPC-C and SPECRate benchmarks, but the argument could also be made with a parallel scientific application workload, which would likely make the case even stronger.

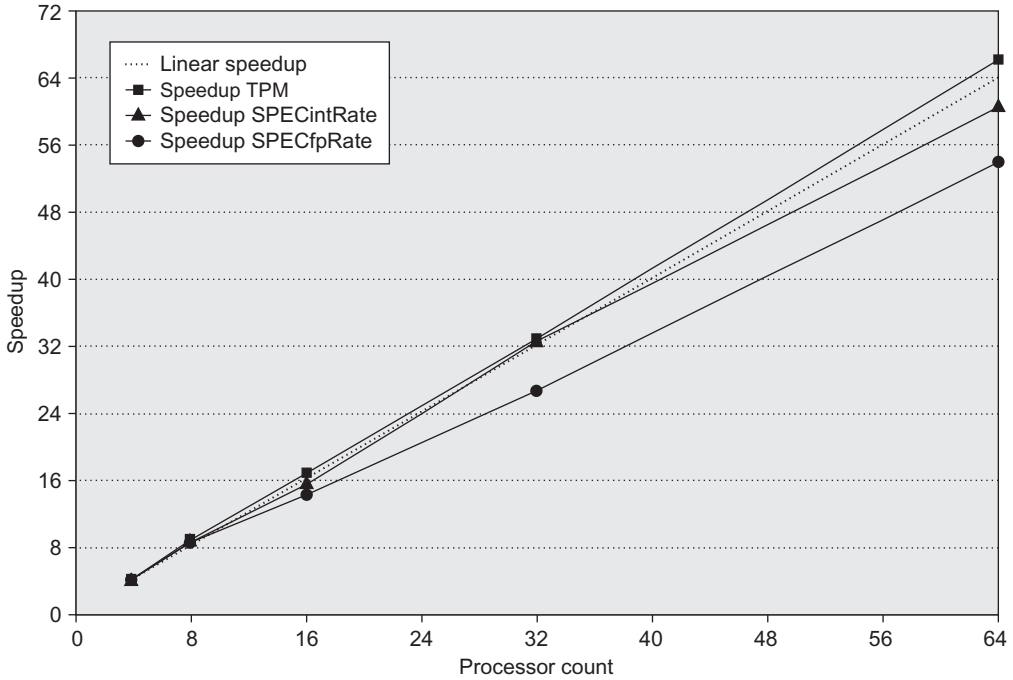


Figure 5.34 Speedup for three benchmarks on an IBM eServer p5 multiprocessor when configured with 4, 8, 16, 32, and 64 processors. The *dashed line* shows linear speedup.

Figure 5.34 shows the speedup for TPC-C, SPECintRate, and SPECfpRate on an IBM eServer p5 multiprocessor configured with 4–64 processors. The figure shows that only TPC-C achieves better than linear speedup. For SPECintRate and SPECfpRate, speedup is less than linear, but so is the cost, because unlike TPC-C, the amount of main memory and disk required both scale less than linearly.

As Figure 5.35 shows, larger processor counts can actually be more cost-effective than the 4-processor configuration. In comparing the cost-performance of two computers, we must be sure to include accurate assessments of both total system cost and what performance is achievable. For many applications with larger memory demands, such a comparison can dramatically increase the attractiveness of using a multiprocessor.

Pitfall *Not developing the software to take advantage of, or optimize for, a multiprocessor architecture.*

There is a long history of software lagging behind on multiprocessors, probably because the software problems are much harder. We give one example to show the subtlety of the issues, but there are many examples we could choose from.

One frequently encountered problem occurs when software designed for a uniprocessor is adapted to a multiprocessor environment. For example, the SGI

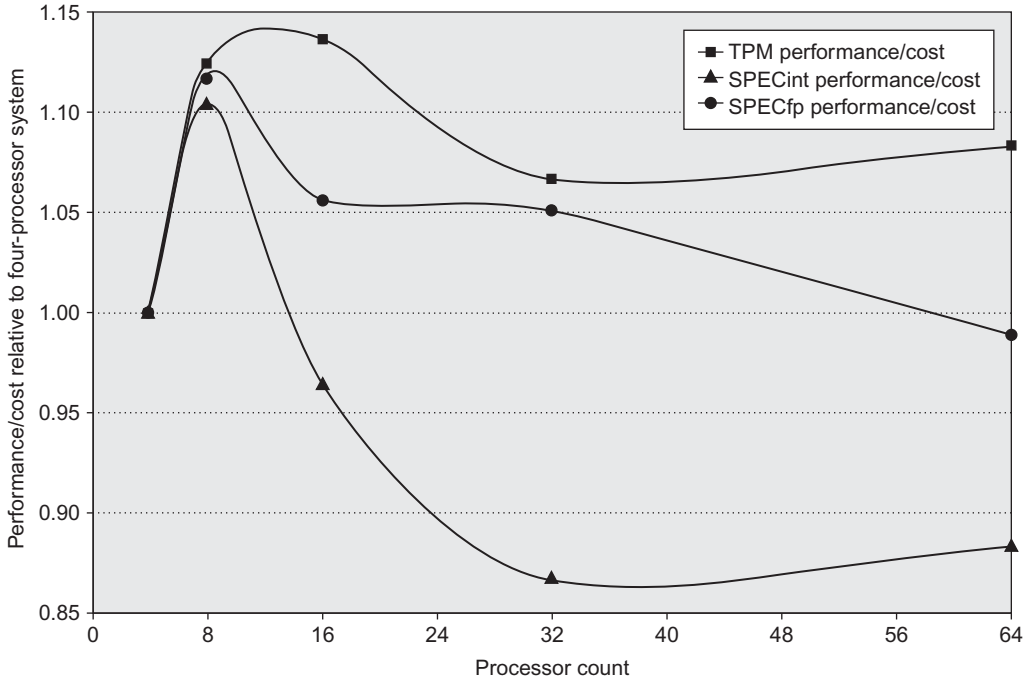


Figure 5.35 The performance/cost for IBM eServer p5 multiprocessors with 4–64 processors is shown relative to the 4-processor configuration. Any measurement above 1.0 indicates that the configuration is more cost-effective than the 4-processor system. The 8-processor configurations show an advantage for all three benchmarks, whereas two of the three benchmarks show a cost-performance advantage in the 16- and 32-processor configurations. For TPC-C, the configurations are those used in the official runs, which means that disk and memory scale nearly linearly with processor count, and a 64-processor machine is approximately twice as expensive as a 32-processor version. In contrast, the disk and memory are scaled more slowly (although still faster than necessary to achieve the best SPECRate at 64 processors). In particular, the disk configurations go from one drive for the 4-processor version to four drives (140 GB) for the 64-processor version. Memory is scaled from 8 GiB for the 4-processor system to 20 GiB for the 64-processor system.

operating system in 2000 originally protected the page table data structure with a single lock, assuming that page allocation was infrequent. In a uniprocessor, this does not represent a performance problem. In a multiprocessor, it can become a major performance bottleneck for some programs.

Consider a program that uses a large number of pages that are initialized at startup, which UNIX does for statically allocated pages. Suppose the program is parallelized so that multiple processes allocate the pages. Because page allocation requires the use of the page table data structure, which is locked whenever it is in use, even an OS kernel that allows multiple threads in the OS will be serialized if the processes all try to allocate their pages at once (which is exactly what we might expect at initialization time).

This page table serialization eliminates parallelism in initialization and has significant impact on overall parallel performance. This performance bottleneck persists even under multiprogramming. For example, suppose we split the parallel program apart into separate processes and run them, one process per processor, so that there is no sharing between the processes. (This is exactly what one user did, because he reasonably believed that the performance problem was due to unintended sharing or interference in his application.) Unfortunately, the lock still serializes all the processes, so even the multiprogramming performance is poor. This pitfall indicates the kind of subtle but significant performance bugs that can arise when software runs on multiprocessors. Like many other key software components, the OS algorithms and data structures must be rethought in a multiprocessor context. Placing locks on smaller portions of the page table effectively eliminates the problem. Similar problems exist in memory structures, which increases the coherence traffic in cases where no sharing is actually occurring.

As multicore became the dominant theme in everything from desktops to servers, the lack of an adequate investment in parallel software became apparent. Given the lack of focus, it will likely be many years before the software systems we use adequately exploit the growing numbers of cores.

5.10 The Future of Multicore Scaling

For more than 30 years, researchers and designers have predicted the end of uniprocessors and their dominance by multiprocessors. Until the early years of this century, this prediction was constantly proven wrong. As we saw in Chapter 3, the costs of trying to find and exploit more ILP became prohibitive in efficiency (both in silicon area and in power). Of course, multicore does not magically solve the power problem because it clearly increases both the transistor count and the active number of transistors switching, which are the two dominant contributions to power. As we will see in this section, energy issues are likely to limit multicore scaling more severely than previously thought.

ILP scaling failed because of both limitations in the ILP available and the efficiency of exploiting that ILP. Similarly, a combination of two factors means that simply scaling performance by adding cores is unlikely to be broadly successful. This combination arises from the challenges posed by Amdahl's Law, which assesses the efficiency of exploiting parallelism, and the end of Dennard's Scaling, which dictates the energy required for a multicore processor.

To understand these factors, we take a simple model of both technology scaling (based on an extensive and highly detailed analysis in Esmaeilzadeh et al. (2012)). Let's start by reviewing energy consumption and power in CMOS. Recall from Chapter 1 that the energy to switch a transistor is given as

$$\text{Energy} \propto \text{Capacitive load} \times \text{Voltage}^2$$

CMOS scaling is limited primarily by thermal power, which is a combination of static leakage power and dynamic power, which tends to dominate. Power is given by

Device count scaling (since a transistor is 1/4 the size)	4
Frequency scaling (based on projections of device speed)	1.75
Voltage scaling projected	0.81
Capacitance scaling projected	0.39
Energy per switched transistor scaling (CV^2)	0.26
Power scaling assuming fraction of transistors switching is the same and chip exhibits full frequency scaling	1.79

Figure 5.36 A comparison of the 22 nm technology of 2016 with a future 11 nm technology, likely to be available sometime between 2022 and 2024. The characteristics of the 11 nm technology are based on the International Technology Roadmap for Semiconductors, which has been recently discontinued because of uncertainty about the continuation of Moore's Law and what scaling characteristics will be seen.

$$\text{Power} = \text{Energy per Transistor} \times \text{Frequency} \times \text{Transistors switched}$$

$$= \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency} \times \text{Transistors switched}$$

To understand the implications of how energy and power scale, let's compare today's 22 nm technology with a technology projected to be available in 2021–24 (depending on the rate at which Moore's Law continues to slow down). Figure 5.36 shows this comparison based on technology projections and resulting effects on energy and power scaling. Notice that power scaling > 1.0 means that the future device consumes more power; in this case, $1.79\times$ as much.

Consider the implications of this for one of the latest Intel Xeon processors, the E7-8890, which has 24 cores, 7.2 billion transistors (including almost 70 MiB of cache), operates at 2.2 GHz, has a thermal power rating of 165 watts, and a die size of 456 mm^2. The clock frequency is already limited by power dissipation: a 4-core version has a clock of 3.2 GHz, and a 10-core version has a 2.8 GHz clock. With the 11 nm technology, the same size die would accommodate 96 cores with almost 280 MiB of cache and operate at a clock rate (assuming perfect frequency scaling) of 4.9 GHz. Unfortunately, with all cores operating and no efficiency improvements, it would consume $165 \times 1.79 = 295$ watts. If we assume the 165-W heat dissipation limit remains, then only 54 cores can be active. This limit yields a maximum performance speedup of $54/24 = 2.25$ over a 5–6 year period, less than one-half the performance scaling seen in the late 1990s. Furthermore, we may have Amdahl's Law effects, as the next example shows.

Example Suppose we have a 96-core future generation processor, but on average only 54 cores can be busy. Suppose that 90% of the time, we can use all available cores; 9% of the time, we can use 50 cores; and 1% of the time is strictly serial. How much speedup might we expect? Assume that cores can be turned off when not in use and draw no power and assume that the use of a different number of cores is distributed so that we need to worry only about average power consumption. How would the

multicore speedup compare to the 24-processor count version that can use all its processor 99% of the time?

Answer We can find how many cores can be used for the 90% of the time when more than 54 are usable, as follows:

$$\begin{aligned}\text{Average Processor Usage} &= 0.09 \times 50 + 0.01 \times 1 + 0.90 \times \text{Max processor}\\ 54 &= 4.51 + 0.90 \times \text{Max processor}\\ \text{Max processor} &= 55\end{aligned}$$

Now, we can find the speedup:

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{55}}{55} + \frac{\text{Fraction}_{50}}{50} + (1 - \text{Fraction}_{55} - \text{Fraction}_{50})}$$

$$\text{Speedup} = \frac{1}{\frac{0.90}{55} + \frac{0.09}{50} + 0.01} = 35.5$$

Now compute the speedup on 24 processors:

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{24}}{24} + (1 - \text{Fraction}_{24})}$$

$$\text{Speedup} = \frac{1}{\frac{0.99}{24} + 0.01} = 19.5$$

When considering both power constraints and Amdahl's Law effects, the 96-processor version achieves less than a factor of 2 speedup over the 24-processor version. In fact, the speedup from clock rate increase nearly matches the speedup from the 4 × processor count increase. We comment on these issues further in the concluding remarks.

5.11 Concluding Remarks

As we saw in the previous section, multicore does not magically solve the power problem because it clearly increases both the transistor count and the active number of transistors switching, which are the two dominant contributions to power. The failure of Dennard scaling merely makes it more extreme.

But multicore does alter the game. By allowing idle cores to be placed in power-saving mode, some improvement in power efficiency can be achieved, as the results in this chapter have shown. For example, shutting down cores in the Intel i7 allows other cores to operate in Turbo mode. This capability allows a trade-off between higher clock rates with fewer processors and more processors with lower clock rates.

More importantly, multicore shifts the burden for keeping the processor busy by relying more on TLP, which the application and programmer are responsible for

identifying, rather than on ILP, for which the hardware is responsible. Multiprogrammed and highly parallel workloads that avoid Amdahl's Law effects will benefit more easily.

Although multicore provides some help with the energy efficiency challenge and shifts much of the burden to the software system, there remain difficult challenges and unresolved questions. For example, attempts to exploit thread-level versions of aggressive speculation have so far met the same fate as their ILP counterparts. That is, the performance gains have been modest and are likely less than the increase in energy consumption, so ideas such as speculative threads or hardware run-ahead have not been successfully incorporated in processors. As in speculation for ILP, unless the speculation is almost always right, the costs exceed the benefits.

Thus, at the present, it seems unlikely that some form of simple multicore scaling will provide a cost-effective path to growing performance. A fundamental problem must be overcome: finding and exploiting significant amounts of parallelism in an energy- and silicon-efficient manner. In the previous chapter, we examined the exploitation of data parallelism via a SIMD approach. In many applications, data parallelism occurs in large amounts, and SIMD is a more energy-efficient method for exploiting data parallelism. In the next chapter, we explore large-scale cloud computing. In such environments, massive amounts of parallelism are available from millions of independent tasks generated by individual users. Amdahl's Law plays little role in limiting the scale of such systems because the tasks (e.g., millions of Google search requests) are independent. Finally, in Chapter 7, we explore the rise of domain-specific architectures (DSAs). Most domain-specific architectures exploit the parallelism of the targeted domain, which is often data parallelism, and as with GPUs, DSAs can achieve much higher efficiency as measured by energy consumption or silicon utilization.

In the last edition, published in 2012, we raised the question of whether it would be worthwhile to consider heterogeneous processors. At that time, no such multicore was delivered or announced, and heterogeneous multiprocessors had seen only limited success in special-purpose computers or embedded systems. While the programming models and software systems remain challenging, it appears inevitable that multiprocessors with heterogeneous processors will play an important role. Combining domain-specific processors, like those discussed in Chapters 4 and 7, with general-purpose processors is perhaps the best road forward to achieve increased performance and energy efficiency while maintaining some of the flexibility that general-purpose processors offer.

5.12 Historical Perspectives and References

Section M.7 (available online) looks at the history of multiprocessors and parallel processing. Divided by both time period and architecture, the section features discussions on early experimental multiprocessors and some of the great debates in parallel processing. Recent advances are also covered. References for further reading are included.

Case Studies and Exercises by Amr Zaky and David A. Wood

Case Study 1: Single Chip Multicore Multiprocessor

Concepts illustrated by this case study

- Snooping Coherence Protocol Transitions
- Coherence Protocol Performance
- Coherence Protocol Optimizations
- Synchronization

A multicore SMT multiprocessor is illustrated in Figure 5.37. Only the cache contents are shown. Each core has a single, private cache with coherence maintained using the snooping coherence protocol of Figure 5.7. Each cache is direct-mapped, with four lines, each holding 2 bytes (to simplify diagram). For further simplification, the whole line addresses in memory are shown in the address fields in the caches, where the tag would normally exist. The coherence states are denoted M, S, and I for Modified, Shared, and Invalid.

5.1. [10/10/10/10/10/10/10] <5.2> For each part of this exercise, the initial cache and memory state are assumed to initially have the contents shown in Figure 5.37. Each part of this exercise specifies a sequence of one or more CPU operations of the form

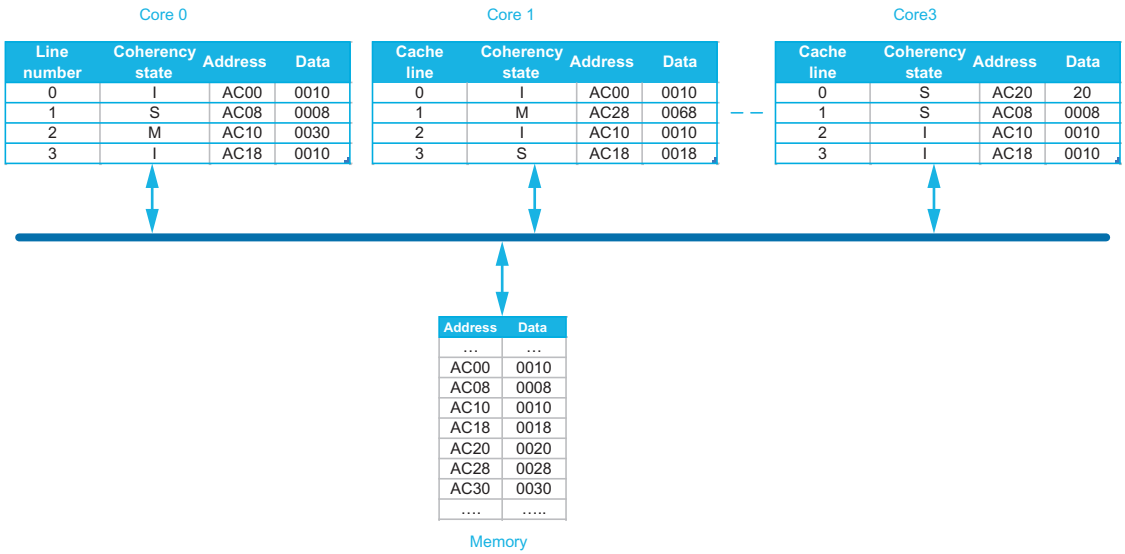


Figure 5.37 Multicore (point-to-point) multiprocessor.

`Ccore#: R, <address>` for reads

and

`Ccore#: W, <address> <-- <value written>` for writes.

For example,

`C3: R, AC10 & C0: W, AC18 <-- 0018`

Read and write operations are for 1 byte at a time. Show the resulting state (i.e., coherence state, tags, and data) of the caches and memory after the actions given below. Show only the cache lines that experience some state change; for example:

`C0.L0: (I, AC20, 0001)` indicates that line 0 in core 0 assumes an "invalid" coherence state (`I`), stores `AC20` from the memory, and has data contents `0001`. Furthermore, represent any changes to the memory state as `M: <address> <- value`.

Different parts (a) through (g) do not depend on one another: assume the actions in all parts are applied to the initial cache and memory states.

a. [10] <5.2> `C0: R, AC20`
b. [10] <5.2> `C0: W, AC20 <-- 80`
c. [10] <5.2> `C3: W, AC20 <-- 80`
d. [10] <5.2> `C1: R, AC10`
e. [10] <5.2> `C0: W, AC08 <-- 48`
f. [10] <5.2> `C0: W, AC30 <-- 78`
g. [10] <5.2> `C3: W, AC30 <-- 78`

5.2. [20/20/20/20] <5.3> The performance of a snooping cache-coherent multiprocessor depends on many detailed implementation issues that determine how quickly a cache responds with data in an exclusive or M state block. In some implementations, a processor read miss to a cache block that is exclusive in another processor's cache is faster than a miss to a block in memory. This is because caches are smaller, and thus faster, than main memory. Conversely, in some implementations, misses satisfied by memory are faster than those satisfied by caches. This is because caches are generally optimized for "front side" or CPU references, rather than "back side" or snooping accesses. For the multiprocessor illustrated in Figure 5.37, consider the execution of a sequence of operations on a single processor core where

- read and write hits generate no stall cycles;
- read and write misses generate N_{memory} and N_{cache} stall cycles if satisfied by memory and cache, respectively;
- write hits that generate an invalidate incur $N_{invalidate}$ stall cycles; and
- a write-back of a block, either due to a conflict or another processor's request to an exclusive block, incurs an additional $N_{writeback}$ stall cycles.

Consider two implementations with different performance characteristics summarized in Figure 5.38.

Parameter	Implementation 1 Cycles	Implementation 2 Cycles
N_{memory}	100	100
N_{cache}	40	130
$N_{invalidate}$	15	15
$N_{writeback}$	10	10

Figure 5.38 Snooping coherence latencies.

To observe how these cycle values are used, we illustrate how the following sequence of operations, assuming the initial caches' states in Figure 5.37, behave under implementation 1.

```
C1: R, AC10
C3: R, AC10
```

For simplicity, assume that the second operation begins after the first completes, even though they are on different processor cores.

For Implementation 1,

- the first read generates 50 stall cycles because the read is satisfied by C0's cache: C1 stalls for 40 cycles while it waits for the block, and C0 stalls for 10 cycles while it writes the block back to memory in response to C1's request; and
- the second read by C3 generates 100 stall cycles because its miss is satisfied by memory.

Therefore this sequence generates a total of 150 stall cycles.

For the following sequences of operations, how many stall cycles are generated by each implementation?

a. [20] <5.3>
```
C0: R, AC20
C0: R, AC28
C0: R, AC30
```

b. [20] <5.3>
```
C0: R, AC00
C0: W, AC08 <-- 48
C0: W, AC30 <-- 78
```

c. [20] <5.3>
```
C1: R, AC20
C1: R, AC28
C1: R, AC30
```

d. [20] <5.3>
```
C1: R, AC00
C1: W, AC08 <-- 48
C1: W, AC30 <-- 78
```

5.3. [20] <5.2> Some applications read a large dataset first and then modify most or all of it. The base MSI coherence protocol will first fetch all of the cache blocks in the Shared state and then be forced to perform an invalidate operation to upgrade them to the

Modified state. The additional delay has a significant impact on some workloads. The MESI addition to the standard protocol (see Section 5.2) provides some relief in these cases. Draw new protocol diagrams for a MESI protocol that adds the Exclusive state and transitions to the base MSI protocol's Modified, Shared, and Invalidate states.

5.4. [20/20/20/20/20] <5.2> Assume the cache contents of Figure 5.37 and the timing of Implementation 1 in Figure 5.38. What are the total stall cycles for the following code sequences with both the base protocol and the new MESI protocol in Exercise 5.3? Assume state transitions that require zero interconnect transactions incur no additional stall cycles.

a. [20] <5.2>
```
C0: R, AC00
C0: W, AC00 <-- 40
```

b. [20] <5.2>
```
C0: R, AC20
C0: W, AC20 <-- 60
```

c. [20] <5.2>
```
C0: R, AC00
C0: R, AC20
```

d. [20] <5.2>
```
C0: R, AC00
C1: W, AC00 <-- 60
```

e. [20] <5.2>
```
C0: R, AC00
C0: W, AC00 <-- 60
C1: W, AC00 <-- 40
```

5.5. Code running on a single core and not sharing any variables with other cores can suffer some performance degradation because of the snooping coherence protocol. Consider the two following iterative loops are NOT functionally equivalent but they seem similar in complexity. One could be led to conclude that they would spend a comparably close number of cycles when executed on the same processor core.

Loop 1	Loop 2
`Repeat i: 1 .. n`	`Repeat i:1 .. n`
`A[i] <-- A[i-1] +B[i];`	`A[i] <-- A[i] +B[i];`

Assume that

- every cache line can hold exactly one element of A or B;
- arrays A and B do not interfere in the cache; and
- all the elements of A or B are in the cache before either loop is executed.

Compare their performance when run on a core whose cache uses the **MESI coherence** protocol. Use the stall cycles data for Implementation 1 in Figure 5.38.

Assume that a cache line can hold multiple elements of A and B (A and B go to separate cache lines). How will this affect the relative performances of Loop1 and Loop2?

Suggest hardware and/or software mechanisms that would improve the performance of Loop1 on a single core.

5.6. [20] <5.2> Many snooping coherence protocols have additional states, state transitions, or bus transactions to reduce the overhead of maintaining cache coherency. In Implementation 1 of Exercise 5.2, misses are incurring fewer stall cycles when they are supplied by cache than when they are supplied by memory. The MOESI protocol extension (see Section 5.2) addresses this need.

Draw new protocol diagrams with the additional state and transitions.

5.7. [20/20/20/20] <5.2> For the following code sequences and the timing parameters for the two implementations in Figure 5.36, compute the total stall cycles for the base MSI protocol and the optimized MESI protocol in Exercise 5.3. Assume state transitions that do not require bus transactions incur no additional stall cycles.

a. [20] <5.2>
```
C1: R, AC10
C3: R, AC10
C0: R, AC10
```

b. [20] <5.2>
```
C1: R, AC20
C3: R, AC20
C0: R, AC20
```

c. [20] <5.2>
```
C0: W, AC20 <-- 80
C3: R, AC20
C0: R, AC20
```

d. [20] <5.2>
```
C0: W, AC08 <--88
C3: R, AC08
C0: W, AC08 <-- 98
```

5.8. [20/20/20/20] <5.5> The spin lock is the simplest synchronization mechanism possible on most commercial shared-memory machines. This spin lock relies on the exchange primitive to atomically load the old value and store a new value. The lock routine performs the exchange operation repeatedly until it finds the lock unlocked (i.e., the returned value is 0).

```
          addi x2, x0, #1
lockit:   EXCH x2, 0(x1)
          bnez x2, lockit
```

The lock is released simply by storing a 0 into x2.

As discussed in Section 5.5, the more optimized spin lock employs cache coherence and uses a load to check the lock, allowing it to spin with a shared variable in the cache.

```
lockit:   ld     x2, 0(x1)
          bnez   x2, lockit
          addi   x2, x0, #1
          EXCH   x2,0(x1)
          bnez   x2, lockit
```

Assume that processor cores C0, C1, and C3 are all trying to acquire a lock at address 0xAC00 (i.e., register R1 holds the value 0xAC00). Assume the cache contents from Figure 5.37 and the timing parameters from Implementation 1 in Figure 5.38. For simplicity, assume the critical sections are 1000 cycles long.

a. [20] <5.5> Using the simple spin lock, determine *approximately* how many memory stall cycles each processor incurs before acquiring the lock.

b. [20] <5.5> Using the optimized spin lock, determine *approximately* how many memory stall cycles each processor incurs before acquiring the lock.

c. [20] <5.5> Using the simple spin lock, *approximately* how many memory accesses occur?

d. [20] <5.5> Using the optimized spin lock, *approximately* how many memory accesses occur?

Case Study 2: Simple Directory-Based Coherence

Concepts illustrated by this case study

- Directory Coherence Protocol Transitions
- Coherence Protocol Performance
- Coherence Protocol Optimizations

Consider the distributed shared-memory system illustrated in Figure 5.39. It consists of 8 nodes of processor cores organized as three-dimensional hypercube with point-to-point interconnections, as shown in the figure. For simplification, we assume the following scaled-down configuration:

- Every node has a *single processor core* with a direct-mapped L1 data cache with its dedicated cache controller.
- The L1 data cache has a capacity of two cache lines with a line size of B bytes.
- The L1 cache states are denoted `M`, `S`, and `I` for Modified, Shared, and Invalid. An example cache entry in some would like

```
1: S, M3, 0xabcd -->
```

Cache line 1 is in the "Shared" state; it contains memory block `M3` and the data value of the block is `0xabcd`.

- The system memory comprises 8 memory blocks (i.e., one memory block per node) and is distributed among the eight nodes, with every node owning a memory block. Node `Ci` owns memory block `Mi`.
- Each memory block is B-bytes wide and is tracked by a coherency directory entry stored with the memory block.

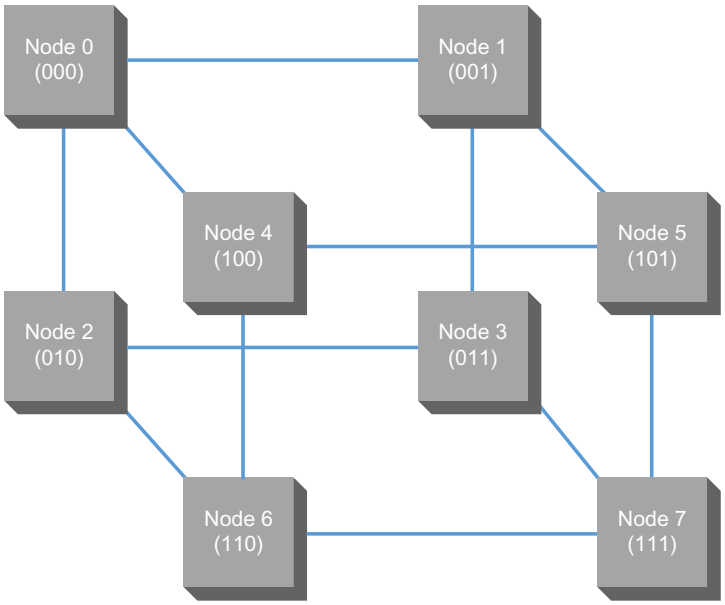


Figure 5.39 Multicore multiprocessor with DSM.

- The state of each memory directory entry is denoted `DM`, `DS`, and `DI` for Directory Modified, Directory Shared, and Directory Invalid. Additionally, the directory entry lists the block sharers using a bit vector with 1 bit for every node. Here is an example memory block and associated directory entry:

```
M3: 0XABCD, DS, 00000011 -->
```

Memory block `M3` (in node C3) contains the value `0xABCD` and is shared by nodes `0` and `1` (corresponding to `1`s in the bit vector).

Read/Write Notation

To describe read/write transactions, we will use the notation

`Ci#: R, <Mi>` for reads

and

`Ci#: W, <Mi> <-- <value written>` for writes.

For example,

`C3: R, M2` describes the core in node 3 issuing a read transaction from an address at memory block `M2` (the address may possibly be cached in `C3` already).

`C0: W, M3 <-- 0018` describes the core in node 0 issuing a write transaction (data is `0X0018`) to an address at memory block `M3` (the address may possibly be cached in `C0` already).

Messages

The directory coherency schemes depend on exchange of command and/or data messages as described by the directory protocol described in Figure 5.20. An example of a command message is a read request. An example of a data message is a read response (with data included).

- Messages originating an ending in the same node do not cross any inter-node links.
- Message with distinct source/destination nodes travel through inter-node links. These messages may be destined from one cache controller to another, from a cache controller to a directory controller, or from a directory controller to a cache controller.
- Messages traveling from a source node to a distinct destination node are statically routed.
  - The static routing algorithm selects a shorted path between the source and destination nodes.
  - The short path is determined by considering the binary representations of the source and destination indices (e.g., `001` for node `C1` and `100` for node `C4`), then by moving from one node to a neighboring node that was not already crossed by the message.
    - For example, to go from node `6` to node `0` (`110 --> 000`), the path is `110--> 100--> 000.`
    - Because more than one shorted path may exist (`110--> 010--> 000` is another path for the preceding example), we assume that the path is selected by inverting first the least significant bit that is different from the corresponding bit in destination index. For example, to travel from node 1 to node 6 (`001--> 110`), the path is `001--> 000--> 010--> 110`.
  - The longest possible path traveled by any message has 3 links (equal to the number of bits in the binary representation of a node index).
- A node can simultaneously process up to three messages from/to distinct neighboring nodes' links if no two of them are competing for the same link resource as clarified by the following examples of **messages sent/received to/from/through** node `000`.

Messages: from 001 --> 010; 010 --> 000 (to cache/directory controller); 100 --> 001. **OK** (distinct destinations).

Message: from 001 --> 010; 000 --> 001 (from cache/directory controller); 100 --> 001.

Not OK as two messages are destined to node 001

In case of destination contention, ties are broken assigning priority to

a. message destined to the node (000 in example) cache or directory controller; then

b. messages forwarded from one to another (through 000 in example); then

c. messages originating from the node (000 in example) cache or directory controller.

- Assume the transmission and service delays in the following table.

Message type	Cache controller	Directory controller	Link
No data	2 cycles	5 cycles	10 cycles
With data	$(3 + \lceil B/4 \rceil)$ cycles	$(6 + 10 * B)$ cycles	$(4 + B)$

- If a message is forwarded through a node, it is first completely received by the node before being sent to the next node on the path.
- Assume any cache controller; directory controller has unlimited capacity to enqueue messages and service them in FCFS order.

5.9. [10/10/10] <5.4> For each part of this exercise, assume that initially all caches lines are invalid, and the data in memory Mi is the byte i (0X00 <= i <= 0x07) repeated as many times as the block size. Assume that successive requests are completely serialized. That is, no core will issue a coherency request until the previous request (by same or different core) is completed.

For each of the following parts,

- show the final state (i.e., coherence state, sharers/owners, tags, and data) of the caches and directory controller (including data values) after the given transaction sequence has completed; and
- show the messages transferred (choose a suitable format for message types).

a. [10] <5.4>
```
C3: R, M4
C3: R, M2
C7: W, M4 <--0xaaaa
C1: W, M4 <--0xbbbb
```

b. [10] <5.4>
```
C3: R, M0
C3: R, M2
C6: W, M4 <--0xaaaa
C3: W, M4 <--0xbbbb
```

c. [10] <5.4>
```
C0: R, M7
C3: R, M4
C6: W, M2 <--0xaaaa
C2: W, M2 <--0xbbbb
```

5.10. [10/10/10] <5.4> The directory protocol used in 5.9 (based on Figure 5.20) assumes that the directory controller receives requests, sends invalidates, receives modified data, sends modified data to requester if block was dirty, and so on. Assume now that the directory controller will delegate some work to the cores. For example, it will notify the exclusive owner of a modified block when some other core needs the block and will have the owner send the block to the new sharer. Specifically, consider the following optimizations and indicate what their benefits (if any) are. Also, specify how the messages will be modified (in comparison with Figure 5.20 protocol) to support the new change.

Hint: Benefits might be reduction in number of messages, faster response time, and so on.

a. [10] <5.4> On a write miss to a shared memory block, the directory controller sends the data to the requester and instructs the sharers to send their invalidate acknowledgements directly to the requester.

b. [10] <5.4> On a read miss to a block modified in some other core, the directory controller instructs the owner of the modified copy to directly forward the data to the requester.

c. [10] <5.4> On a read miss to a block in shared (S) state in some other cores, the directory controller instructs one of the sharers (say, the one closest to the requester) to directly forward the data to the requester.

5.11. [15/15/15] <5.4> In problem 5.9, it was assumed that all transactions on the system were serially executed, which is both unrealistic and inefficient in a DSM multicore. We now relax this condition. We will require only that all transactions originating in one core are serialized. However, different cores can independently issue their read/write transactions and even compete for the same memory block. The transactions of problem 5.9 are represented next to reflect the new, relaxed constraints. Redo problem 5.9 with the new, relaxed constraints.

a. [15] <5.4>
```
C1: W, M4 <--0xbbbb     C3: R, M4     C7: R, M2
                        C3: W, M4 <--0xaaaa
```

b. [15] <5.4>
```
C3: R, M0       C6: W, M4 <--0xaaaa
C3: R, M2
C3: W, M4 <--0xbbbb
```

c. [15] <5.4>
```
C0: R, M7  C2: W, M2 <--0xbbbb  C3: R, M4  C6: W, M2 <--0xaaaa
```

5.12. [10/10] <5.4> Use the routing and delay information described earlier and trace how the following groups of transactions will progress in the system (assume that all accesses are misses).

a. C0: R, M7 C2: W, M2 <--0xbbbb C3: R, M4 C6: W, M2 <--0xaaaa

b. C0: R, M7 C3: R, M7
C2: W, M7 <--0xbbbb

5.13. [20] <5.4> What extra complexities may arise if the messages can be adaptively rerouted on the links? For example, a coherency message from core M1 directory controller to C2 (expressed in binary as M_{001} --> C_{010}) will be routed either through the inter-node path C_{001} --> C_{000} --> C_{010} or the inter-node path C_{001} --> C_{011} --> C_{010}, depending on link availability.

5.14. [20] <5.4> In a read miss, a cache might overwrite a line in the shared (S) state without notifying the directory that owns the corresponding memory block. Alternatively, it will notify the directory so that it deletes this cache from the list of sharers.

Show how the following transaction groups (performed one at a time in series) will proceed under both approaches.

```
C3:  R, M4
C3:  R, M2
C2:   W, M4 <--0xabcd
```

Case Study 3: Memory Consistency

Concepts Illustrated by This Case Study

- Legitimate Program Behavior Under Sequential Consistency (SC) Models
- Hardware Optimization Allowed for SC Models
- Using Synchronization Primitives to Make a Consistency Model Emulate a More Restrictive Model

5.15. [10/10] <5.6> Consider the following code segments running on two processors P1 and P2. Assume A and B are initially 0.

P1:	**P2:**
While (B == 0);	While (A==0);
A=1;	B = 1;

a. If the processors adhere to sequential consistency (SC) consistency model. What are the possible values of A and B at the end of the segments? Show the statement interleaving supporting your answer(s).

b. Repeat (a) if the processors adhere to the total store order (TSO) consistency model.

5.16. [5] <5.6> Consider the following code segments running on two processors P1 and P2. Assume A, and B, are initially 0. Explain how an optimizing compiler might make it impossible for B to be ever set to 2 in a sequentially consistent execution model.

P1:	P2:
A=1;	B=1;
A=2;	While (A <> 1);
While (B == 0);	B= 2;

5.17. [10] <5.4>. In a processor implementing a SC consistency model, the data cache is augmented with a data prefetch unit. Will that alter the SC implementation execution results? Why or why not?

5.18. [10/10] <5.6> Assume that the following code segment is executed on a processor that implements partial store order (PSO),

```
A=1;
B=2;
If (C== 3)
D=B;
```

a. Augment the code with synchronization primitives to make it emulate the behavior of a total store order (TSO) implementation.

b. Augment the code with synchronization primitives to make it emulate the behavior of a sequential consistency (SC) implementation.

5.19. [20/20/20] <5.6> Sequential consistency (SC) requires that all reads and writes appear to have executed in some total order. This may require the processor to stall in certain cases before committing a read or write instruction. Consider the code sequence

```
write A
read B
```

where the write A results in a cache miss and the read B results in a cache hit.

Under SC, the processor must stall read B until after it can order (and thus perform) write A. Simple implementations of SC will stall the processor until the cache receives the data and can perform the write.

Release consistency (RC) consistency mode (see Section 5.6) relaxes these constraints: ordering—when desired—is enforced by judicious use of synchronization operations. This allows, among other optimizations, processors to implement write buffers, which hold committed writes that have not yet been ordered with respect to other processors' writes. Reads can pass (and potentially bypass) the write buffer in RC (which they could not do in SC).

Assume that one memory operation can be performed per cycle and that operations that hit in the cache or that can be satisfied by the write buffer introduce no stall cycles. Operations that miss incur the latencies listed in Figure 5.38.

How many stall cycles occur *prior* to each operation for both the SC and RC consistency models? (Write buffer can hold at most one write.)

a. [20] <5.6>

```
P0: write 110 <-- 80   //assume miss (no other
                       cache has the line)
P0: read 108           //assume miss (no other
                       cache has the line)
```

b. [20] <5.6>
```
P0: read 110                  //assume miss (no other
                              cache has the line)
P0: write 100 <-- 90          //assume hit
```

c. [20] <5.6>
```
P0: write 100 <-- 80          //assume miss
P0: write 110 <-- 90          //assume hit
```

5.20. [20] <5.6> Repeat part (a) of problem 5.19 under an SC model on a processor that has a read prefetch unit. Assume a read prefetch was triggered 20 cycles in advance of the write operation.

Exercises

5.21. [15] <5.1> Assume that we have a function for an application of the form F(i, p), which gives the fraction of time that exactly i processors are usable given that a total of p processors are available. This means that

$$\sum_{i=1}^{p} F(i,p) = 1$$

Assume that when i processors are in use, the applications run i times faster.

a. Rewrite Amdahl's Law so that it gives the speedup as a function of p for some application.

b. An application A runs on single processor for a time T seconds. Different portions of its running time can improve if a larger number of processors is used. Figure 5.40 provides the details.

How much speedup will A achieve when on 8 processors?

c. Repeat for 32 processors and an infinite number of processors.

5.22. [15/20/10] <5.1> In this exercise, we examine the effect of the interconnection network topology on the CPI of programs running on a 64-processor distributed-memory multiprocessor. The processor clock rate is 2.0 GHz, and the base CPI of an application with all references hitting in the cache is 0.75. Assume that 0.2% of the instructions involve a remote communication reference. The cost of a remote communication reference is (100 + 10 h) ns, h being the number of communication network hops that a remote reference has to make to the remote processor memory and back. Assume all communication links are bidirectional.

a. [15] <5.1> Calculate the worst-case remote communication cost when the 64 processors are arranged as a ring, as an 8×8 processor grid, or as a hypercube (hint: longest communication path on a 2^n hypercube has n links).

Fraction of T	20%	20%	10%	5%	15%	20%	10%
Processors (P)	1	2	4	6	8	16	128

Figure 5.40 Percentage of application's A time that can use up to P processors.

b. [20] <5.1> Compare the base CPI of the application with no remote communication to the CPI achieved with each of the three topologies in part (a).

5.23. [15] <5.2> Show how the basic snooping protocol of Figure 5.6 can be changed for a write-through cache. What is the major hardware functionality that is not needed with a write-through cache compared with a write-back cache?

5.24. [20/20] <5.2> Please answer the following problems:

a. [20] <5.2> Add a clean exclusive state to the basic snooping cache coherence protocol (Figure 5.6). Show the protocol in the finite state machine format used in the figure.

b. [20] <5.2> Add an "owned" state to the protocol of part (a) and describe using the same finite state machine format used in Figure 5.6.

5.25. [15] <5.2> One proposed solution for the problem of false sharing is to add a valid bit per word. This would allow the protocol to invalidate a word without removing the entire block, letting a processor keep a portion of a block in its cache while another processor writes a different portion of the block. What extra complications are introduced into the basic snooping cache coherence protocol (Figure 5.6) by this addition? Consider all possible protocol actions.

5.26. [15/20] <5.3> This exercise studies the impact of aggressive techniques to exploit instruction-level parallelism in the processor when used in the design of shared-memory multiprocessor systems. Consider two systems identical except for the processor. System A uses a processor with a simple single-issue, in-order pipeline, and system B uses a processor with four-way issue, out-of-order execution and a reorder buffer with 64 entries.

a. [15] <5.3> Following the convention of Figure 5.11, let us divide the execution time into instruction execution, cache access, memory access, and other stalls. How would you expect each of these components to differ between system A and system B?

b. [10] <5.3> Based on the discussion of the behavior of OLTP workload in Section 5.3, what is the important difference between the OLTP workload and other benchmarks that limit benefit from a more aggressive processor design?

5.27. [15] <5.3> How would you change the code of an application to avoid false sharing? What might be done by a compiler and what might require programmer directives?

5.28. [15] <5.3> An application is calculating the number of occurrences of a certain word in a very large number of documents. A very large number of processors divided the work, searching the different documents. They created a huge array—word_count—of 32-bit integers, every element of which is the number of times the word occurred in some document. In a second phase, the computation is moved to a small SMP server with four processors. Each processor sums up approximately ¼ of the array elements. Later, one processor calculates the total sum.

```
for (int p=0; p<=3; p++) // Each iteration of is executed on a
                            separate processor.
{
     sum [p] = 0;
     for (int i=0; i < n/4; i++) // n is size of word_count and
                                    is divisible by 4
          sum[p] = sum[p] + word_count[p+4*i];
}
total_sum = sum[0] +sum[1]+sum[2]+sum[3] //executed only
                                           on processor.
```

a. Assuming each processor has a 32-byte L1 data cache. Identify the cache line sharing (true or false) that the code exhibits.

b. Rewrite the code to reduce the number of misses to elements of the array word_count.

c. Identify a manual fix you can make to the code to rid it of any false sharing.

5.29. [15] <5.4> Assume a directory-based cache coherence protocol. The directory currently has information that indicates that processor P1 has the data in "exclusive" mode. If the directory now gets a request for the same cache block from processor P1, what could this mean? What should the directory controller do? (Such cases are called "race conditions" and are the reason why coherence protocols are so hard to design and verify.)

5.30. [20] <5.4> A directory controller can send invalidates for lines that have been replaced by the local cache controller. To avoid such messages, and to keep the directory consistent, replacement hints are used. Such messages tell the controller that a block has been replaced. Modify the directory coherence protocol of Section 5.4 to use such replacement hints.

5.31. [20/15/20/15] <5.4> One downside of a straightforward implementation of directories using fully populated bit vectors is that the total size of the directory information scales as the product: processor count × memory blocks. If memory grows linearly with processor count, the total size of the directory grows quadratically in the processor count. In practice, because the directory needs only 1 bit per memory block (which is typically 32–128 bytes), this problem is not serious for small-to-moderate processor counts. For example, assuming a 128-byte block, and P processors, the amount of directory storage compared to main memory is P/(128*8) = P/1024, which is 12.5% overhead for P = 128 processors. We can avoid this problem by observing that we need to keep only an amount of information that is proportional to the cache size of each processor. We explore some solutions in these exercises.

a. [20] <5.4> One method to obtain a scalable directory protocol is to organize the multiprocessor as a logical hierarchy with the processors as leaves of the hierarchy and directories positioned at the root of each subtree. The directory at each subtree records which descendants cache which memory blocks. It also

records the memory blocks—with a home in that subtree—that are cached outside the subtree. Compute the amount of storage needed to record the processor information for the directories, assuming that each directory is fully associative. Your answer should incorporate both the number of nodes at each level of the hierarchy as well as the total number of nodes.

b. [15] <5.4> Another approach to reducing the directory size is to allow only a limited number of the directory's memory blocks to be shared at any given time. Implement the directory as a four-way set-associative cache storing full bit vectors. If a directory cache miss occurs, choose a directory entry and invalidate the entry. Describe how this organization will work elaborating what will happen as a is block read, written replaced and written back to memory. Modify the protocol in Figure 5.20 to reflect the new transitions required by this directory organization.

c. [20] <5.4> Rather than reducing the number of directory entries, we can implement bit vectors that are not dense. For example, we can set every directory entry to 9 bits. If a block is cached in only one node outside its home, this field contains the node number. If the block is cached in more than one node outside its home, this field is a bit vector with each bit indicating a group of eight processors, at least one of which caches the block. Illustrate how this scheme would work for a 64-processor DSM machine that consists of eight 8-processors groups.

d. [15] An extreme approach to reducing the directory size is to implement an "empty" directory; that is, the directory in every processor does not store any memory states. It receives requests and forwards them as *appropriate*. What is the benefit of having such a directory over having no directory at all for a DSM system?

5.32. [10] <5.5> Implement the classical compare-and-swap instruction using the *load linked/store conditional* instruction pair.

5.33. [15] <5.5> One performance optimization commonly used is to pad synchronization variables so as not to have any other useful data in the same cache line. Construct an example demonstrating that this optimization can be extremely useful in some situations. Assume a snoopy write invalidate protocol.

5.34. [30] <5.5> One possible implementation of the *load linked/store conditional pair* for multicore processors is to constrain these instructions to using uncached memory operations. A monitor unit intercepts all reads and writes from any core to the memory. It keeps track of the source of the *load linked* instructions and whether any intervening stores occur between the *load linked* and its corresponding *store conditional* instruction. The monitor can prevent any failing store conditional from writing any data and can use the interconnect signals to inform the processor that this store failed.

Design such a monitor for a memory system supporting a four-core SMP. Take into account that, generally, read and write requests can have different data sizes (4/8/16/32 bytes). Any memory location can be the target of a *load linked/store*

conditional pair, and the memory monitor should assume that *load linked/store conditional* references to any location can, possibly, be interleaved with regular accesses to the same location. The monitor complexity should be independent of the memory size.

5.35. [25] <5.5> Prove that, in a two-level cache hierarchy where L1 is closer to the processor, inclusion is maintained with no extra action if L2 has at least as much associativity as L1, both caches use LRU replacement, and both caches have the same block sizes.

5.36. [Discussion] <5> When trying to perform detailed performance evaluation of a multiprocessor system, system designers use one of three tools: analytical models, trace-driven simulation, and execution-driven simulation. Analytical models use mathematical expressions to model the behavior of programs. Trace-driven simulations run the applications on a real machine and generate a trace, typically of memory operations. These traces can be replayed through a cache simulator or a simulator with a simple processor model to predict the performance of the system when various parameters are changed. Execution-driven simulators simulate the entire execution maintaining an equivalent structure for the processor state and so on.

a. What are the accuracy/speed trade-offs between these approaches?

b. CPU traces, if not carefully collected, can exhibit artifacts of the system they are collected on. Discuss this issue while using branch-prediction and spin-wait synchronization as examples. (Hint: The program itself is not available to a pure CPU trace; just the trace is available.)

5.37. [40] <5.7, 5.9> Multiprocessors and clusters usually show performance increases as you increase the number of the processors, with the ideal being n times speedup for n processors. The goal of this biased benchmark is to make a program that gets worse performance as you add processors. For example, this means that one processor on the multiprocessor or cluster runs the program fastest, two are slower, four are slower than two, and so on. What are the key performance characteristics for each organization that give inverse linear speedup?

This page intentionally left blank

6.1	Introduction	466
6.2	Programming Models and Workloads for Warehouse-Scale Computers	471
6.3	Computer Architecture of Warehouse-Scale Computers	477
6.4	The Efficiency and Cost of Warehouse-Scale Computers	482
6.5	Cloud Computing: The Return of Utility Computing	490
6.6	Cross-Cutting Issues	501
6.7	Putting It All Together: A Google Warehouse-Scale Computer	503
6.8	Fallacies and Pitfalls	514
6.9	Concluding Remarks	518
6.10	Historical Perspectives and References	519
	Case Studies and Exercises by Parthasarathy Ranganathan	519

6

Warehouse-Scale Computers to Exploit Request-Level and Data-Level Parallelism

The datacenter is the computer.

Luiz André Barroso,
Google (2007)

A hundred years ago, companies stopped generating their own power with steam engines and dynamos and plugged into the newly built electric grid. The cheap power pumped out by electric utilities didn't just change how businesses operate. It set off a chain reaction of economic and social transformations that brought the modern world into existence. Today, a similar revolution is under way. Hooked up to the Internet's global computing grid, massive information-processing plants have begun pumping data and software code into our homes and businesses. This time, it's computing that's turning into a utility.

Nicholas Carr,
The Big Switch: Rewiring the World, from Edison to Google (2008)

Computer Architecture. https://doi.org/10.1016/B978-0-12-811905-1.00006-7
© 2019 Elsevier Inc. All rights reserved.

6.1 Introduction

Anyone can build a fast CPU. The trick is to build a fast system.

Seymour Cray,
Considered the father of the supercomputer

The warehouse-scale computer (WSC)[1] is the foundation of Internet services billions of people use every day: search, social networking, online maps, video sharing, online shopping, email services, and so on. The tremendous popularity of such Internet services necessitated the creation of WSCs that could keep up with the rapid demands of the public. Although WSCs may appear to be just large data centers, their architecture and operation are quite different, as we will see. Today's WSCs act as one giant machine that costs hundreds of million dollars for the building, the electrical and cooling infrastructure, the servers, and the networking equipment that connects and houses 50,000–100,000 servers. Moreover, the rapid growth of commercial cloud computing (see Section 6.5) makes WSCs accessible to anyone with a credit card.

Computer architecture extends naturally to designing WSCs. For example, Luiz Barroso of Google (quoted earlier) did his dissertation research in computer architecture. He believes that an architect's skills of designing for scale, designing for dependability, and a knack for debugging hardware are very helpful in the creation and operation of WSCs.

At this leading-edge scale, which requires innovation in power distribution, cooling, monitoring, and operations, the WSC is the modern descendant of the supercomputer—making Seymour Cray the godfather of today's WSC architects. His extreme computers handled computations that could be done nowhere else, but were so expensive that only a few companies could afford them. This time the target is providing information technology for the world instead of high-performance computing (HPC) for scientists and engineers; thus WSCs arguably play a more important role for society today than Cray's supercomputers did in the past.

Unquestionably, WSCs have many orders of magnitude more users than high-performance computing, and they represent a much greater share of the IT market. Whether measured by the number of users or revenue, Google is 1000 times larger than Cray Research ever was.

[1]This chapter is based on material from the book *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines, Second Edition*, by Luiz André Barroso, Jimmy Clidaras, and Urs Hölzle of Google (2013); the blog Perspectives at mvdirona.com and the talks "Cloud-Computing Economies of Scale" and "Data Center Networks Are in My Way," by James Hamilton of Amazon Web Services (2009, 2010); and the paper *Above the Clouds: A View of Cloud Computi*ng, by Michael Armbrust et al. (2010).

WSC architects share many goals and requirements with server architects:

- *Cost-performance*—Work done per dollar is critical in part because of the scale. Reducing the costs of a collection of WSCs by few percent could save millions of dollars.
- *Energy efficiency*—Except for the photons that leave WSCs, they are essentially closed systems, with almost all the energy consumed turned into heat that must be removed. Thus, peak power and consumed power drive both the cost of power distribution and the cost of cooling systems. The majority of the infrastructure costs of building a WSC goes toward power and cooling. Moreover, energy efficiency is an important part of environmental stewardship. Therefore, work done per joule is critical for both WSCs and its servers because of the high cost of building the power and mechanical infrastructure for a warehouse of computers and for the resulting monthly utility bills.
- *Dependability via redundancy*—The long-running nature of Internet services means that the hardware and software in a WSC must collectively provide at least 99.99% (called "four nines") of availability; that is, services must be down less than 1 h per year. Redundancy is the key to dependability for both WSCs and servers. Although server architects often utilize more hardware at higher costs to reach high availability, WSC architects rely instead on numerous cost-effective servers connected by a network and redundancy managed by software. In addition to local redundancy inside a WSC, an organization needs redundant WSCs to mask events that can take out whole WSCs. Indeed, although every cloud service needs to be available at least 99.99% of the time, the dependability of a full Internet company like Amazon, Google, or Microsoft needs to be even higher. If one of these companies was completely offline for 1 h per year—that is, 99.99% availability—that would be front page news. Multiple WSCs have the added benefit of reducing latency for services that are widely deployed (Figures 6.18–6.20).
- *Network I/O*—Server architects must provide a good network interface to the external world, and WSC architects must also. Networking is needed to keep data consistent between multiple WSCs as well as to interface with the public.
- *Both interactive and batch processing workloads*—Although one expects highly interactive workloads for services like search and social networking with billions of users, WSCs, like servers, also run massively parallel batch programs to calculate metadata useful to such services. For example, MapReduce jobs are run to convert the pages returned from crawling the web into search indices (see Section 6.2).

Not surprisingly, there are also characteristics *not* shared with server architecture:

- *Ample parallelism*—A concern for a server architect is whether the applications in the targeted marketplace have enough concurrency to justify the amount of parallel hardware and whether the cost is too high for sufficient communication

hardware to exploit this parallelism. A WSC architect has no such concern. First, batch applications benefit from the large number of distinct datasets that require independent processing, such as billions of web pages from a web crawl. This processing is *data-level parallelism*, which we saw in Chapter 4, this time applied to data in storage instead of data in memory. Second, interactive Internet service applications, also known as *software as a service* (*SaaS*), can benefit from millions of independent users of interactive Internet services. Reads and writes are seldom dependent in SaaS, so SaaS rarely needs to synchronize. For example, search uses a read-only index and email normally reads and writes independent information. We call this type of easy parallelism *request-level parallelism*, as many independent efforts can proceed in parallel naturally with little need for communication or synchronization; an example is that journal-based updating can reduce throughput demands. Even read-/write-dependent features are sometimes dropped to offer storage that can scale to the size of modern WSCs. In any case, WSC applications have no choice but to find algorithms that can scale across hundreds to thousands of servers, as that is what customers expect and that is what the WSC technology provides.

- *Operational costs count*—Server architects typically ignore operational costs of a server, assuming that they pale in comparison to purchase costs. WSCs have longer lifetimes—the building and electrical and cooling infrastructure are often amortized 10–15 years—so the operational costs add up: energy, power distribution, and cooling represent more than 30% of the costs of a WSC over 10 years.
- *Location counts*—To build a WSC, the first step is building a warehouse. One question is where? Real estate agents emphasize location, but the location for a WSC means access to water, inexpensive electricity, proximity to Internet backbone optical fibers, people nearby to work in the WSC, and low risk from environmental disasters, such as earthquakes, floods, and hurricanes. A more obvious concern is just the cost of the land, including enough space to grow the WSC. For companies with many WSCs, another concern is finding a place geographically near a current or future population of Internet users, to reduce latency over the Internet. Other factors include taxes, property costs, social issues (people sometimes want a facility in their country), political issues (some jurisdictions require local hosting), cost of networking, reliability of networking, cost of power, source of power (e.g., hydroelectric versus coal), weather (cooler is cheaper, as Section 6.4 shows), and overall Internet connectivity (Australia is close to Singapore geographically, but the network link bandwidth between them is not great).
- *Computing efficiently at low utilization*—Server architects usually design their systems for peak performance within a cost budget and worry about power only to make sure they don't exceed the cooling capacity of their enclosure. As we will see (Figure 6.3), WSC servers are rarely fully utilized, in part to ensure low response time and in part to offer the redundancy needed to deliver dependable computing. Given that operational costs count, such servers need to compute efficiently at all utilization levels.
- *Scale and the opportunities/problems associated with scale*—Often extreme computers are extremely expensive because they require custom hardware,